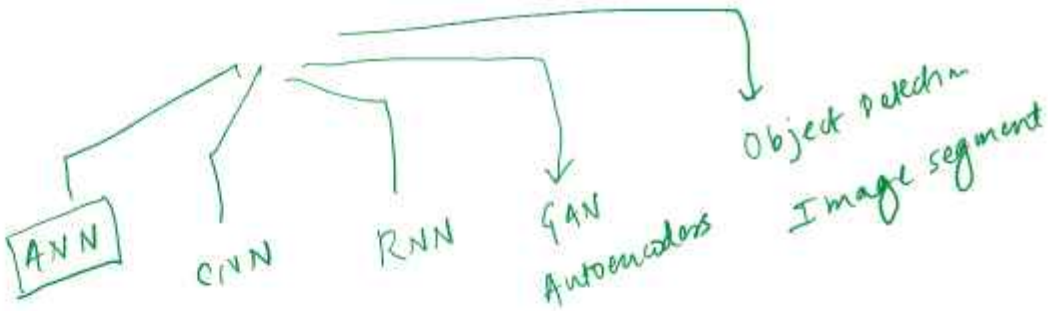
Curriculum

Tuesday, February 15, 2022 3:30 PM



Features

Tuesday, February 15, 2022 3:31 PM

1) Well Researched
2) Easy to consume
3) Well structured → 100 days
4) Tensorflow + Keras
5) Projects

Prerequisites

Tuesday, February 15, 2022 3:31 PM

1) Python →
2) Basics of ML
3) Linear Algebra → 3 Blue 1 Brown / Linear Alg → (5)

Extra Content

Tuesday, February 15, 2022 3:22 PM

1) Deep learning Roadmap
2) Deep learning Project ideas
3) Projects
4) ±interview questions

↳ Next video
↳ This

→ 10th Apr & → ANN → CNN
↳ 3-4 months

What is Deep Learning?

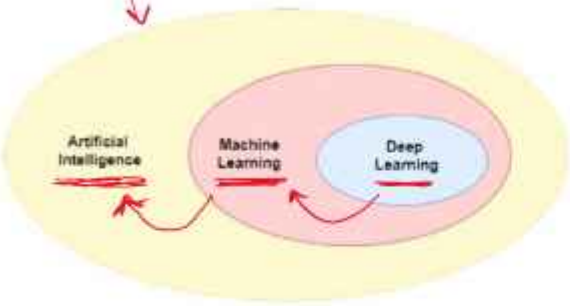
16 February 2022 06:32

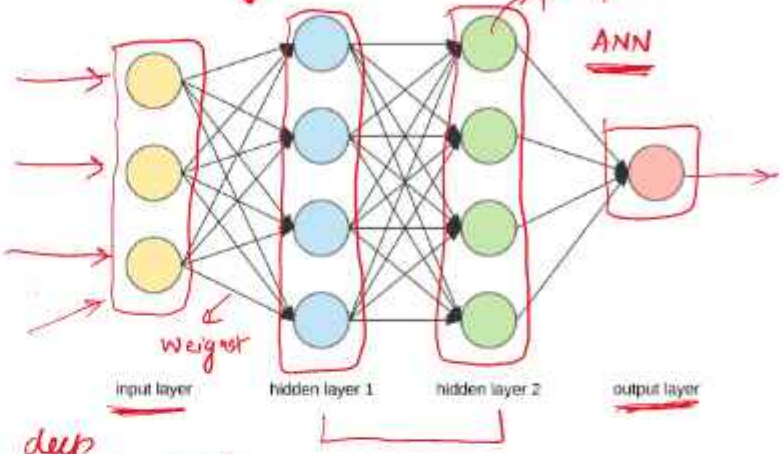
Deep Learning is a subfield of Artificial Intelligence and Machine Learning that is inspired by the structure of a human brain.
Deep learning algorithms attempt to draw similar conclusions as humans would by continually analyzing data with a given logical structure called Neural Network.

learn from data

input | output

statistical





deep

ANN →
CNN → convolutional NN ✓ image
RNN → Recurrent NN → speech / text
GAN → generate (text) images

why (dl) getting so famous?

1) Applicability →
2) Performance → state-of-the-art → human experts

Go → AlphaGo → AI
5 → (4)

Applications → self driving, drugs → chemistry

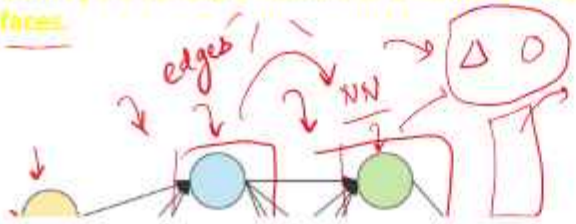
1) Representation

Deep learning is part of a broader family of machine learning methods based on artificial neural networks with representation learning.

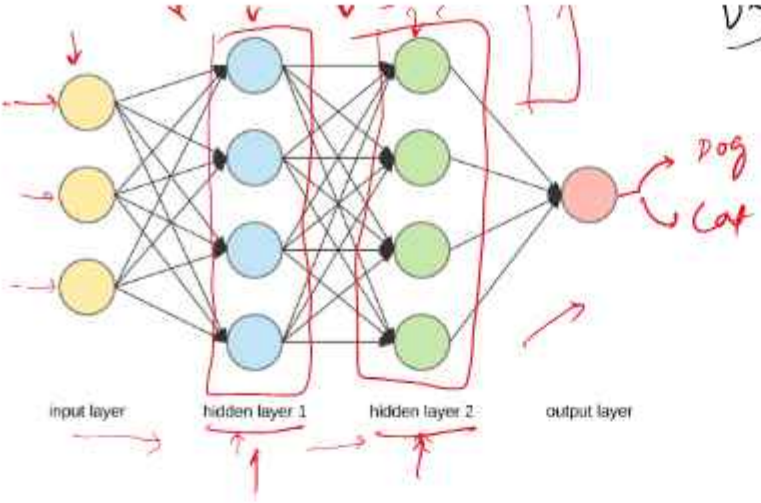
Deep Learning Algorithms uses multiple layers to progressively extract higher-level features from the raw input. For example, in image processing, lower layers may identify edges, while higher layers may identify the concepts relevant to a human such as digits or letters or faces.

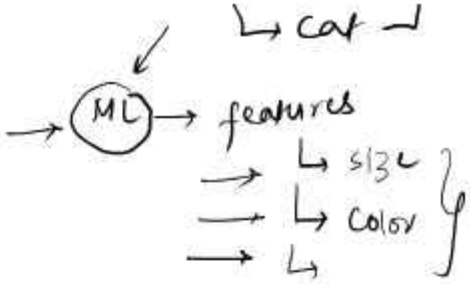
feature extraction engineering

features



Deep learning → Dog / cat → classification → Dog / cat


input layer
hidden layer 1
hidden layer 2
output layer
Dog
Cat


Cat
ML
features
size
Color

Deep Learning VS Machine Learning

16 February 2022 08:53

1. Data Dependency
2. Hardware Dependency
3. Training Time
4. Feature Selection
5. Interpretability

costly DL

CPU → run slow

Data ↑ hungry

Data ↑ performance ↑

matrix multiplication → GPU → powerful

Representation learning → features extract

complex

training time - high → weeks

ML → low minutes - hrs

less data

Mr. wins

Prediction

DL → fast

DL → Algorithm → feature

ML

KNN

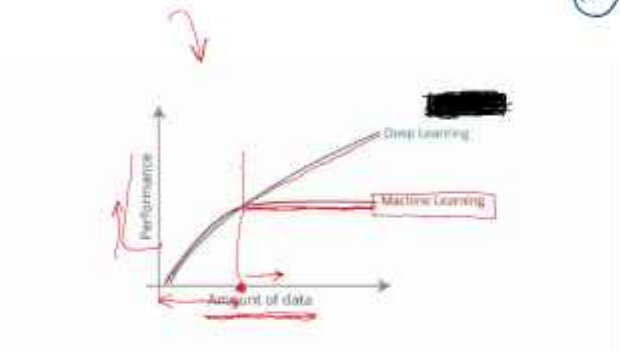
Resume based

→ predict → Y/N

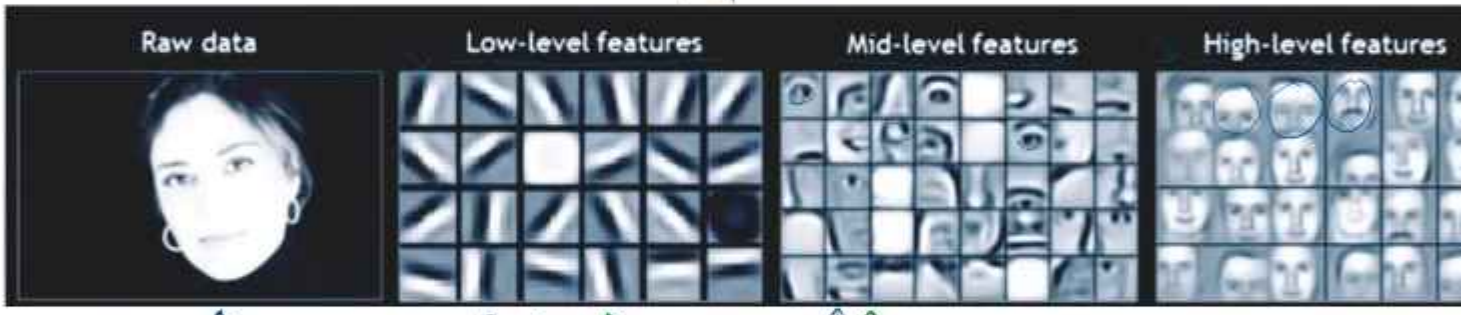
→ features

domain expert → 12th marks, 10th mars

teachers → No. of ach, No. of course, Quality of College



Deep learning



feature → interpre

Deep learning

comment → bank

user

DT

Dog

cat

Black box

logistic

cgpa | iq | placement

w_1 w_2 N

DT

flow chart

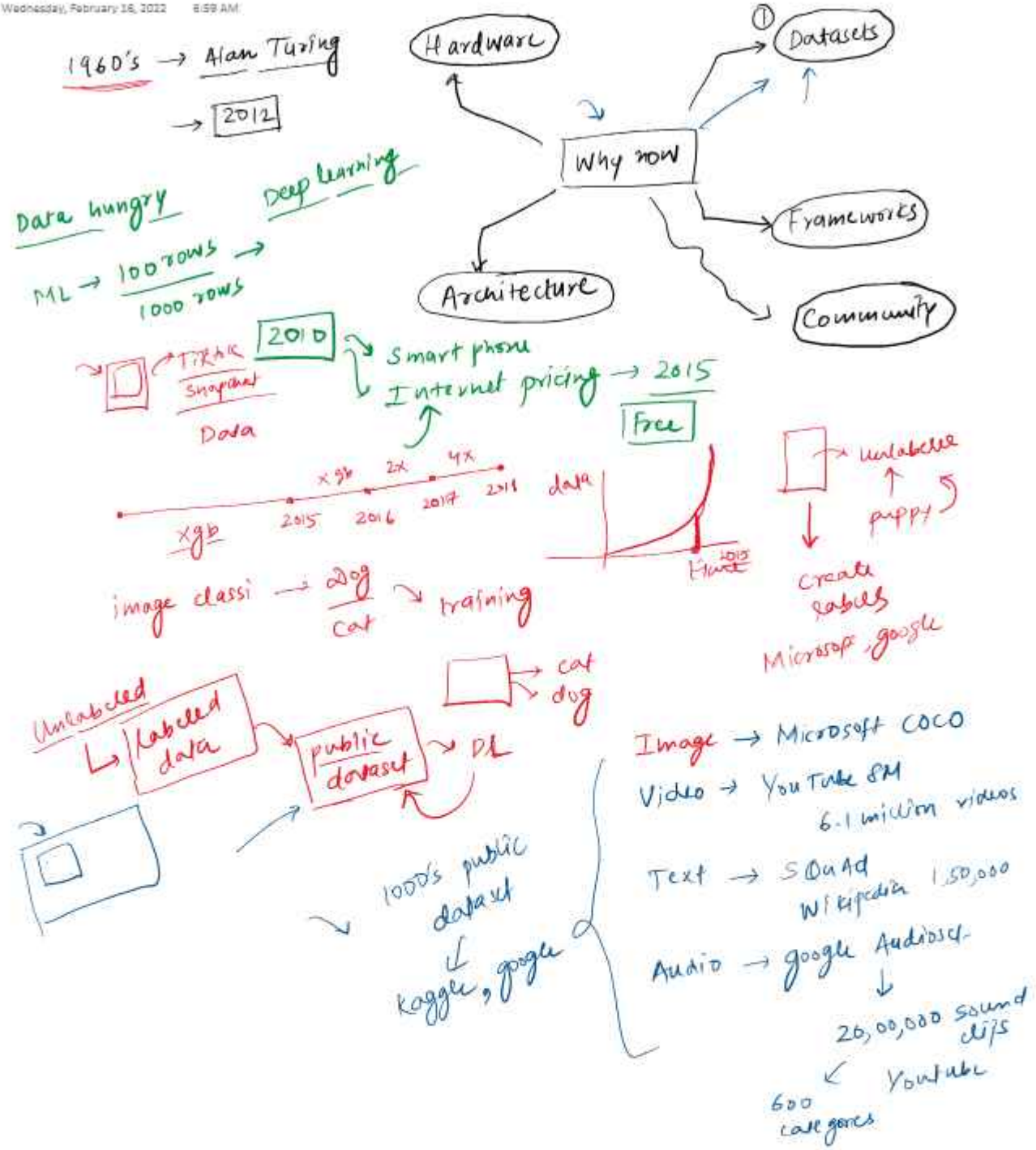
explainability

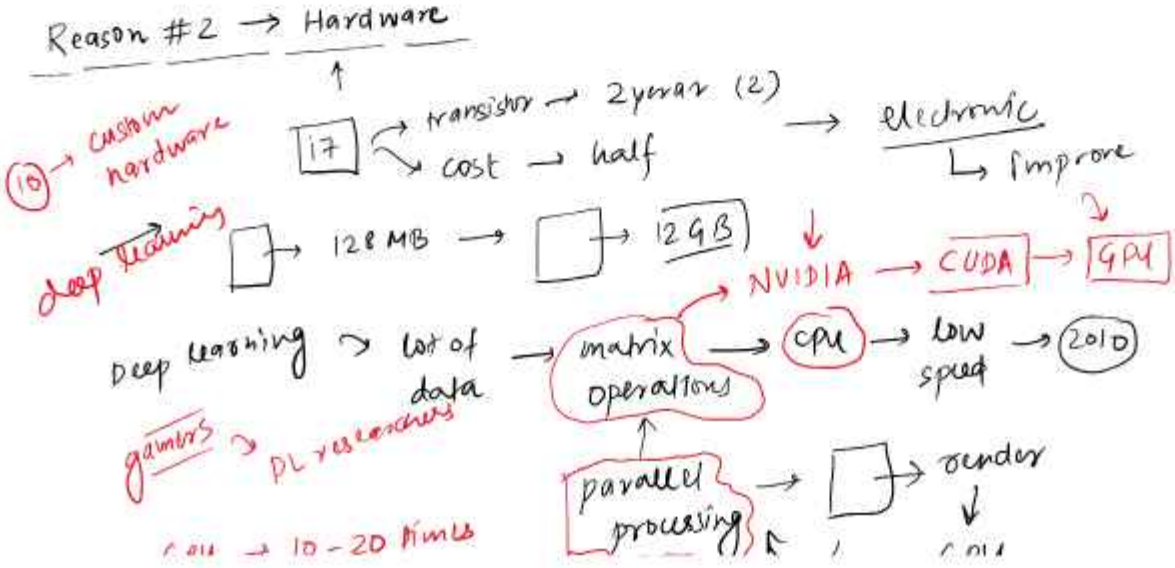
High ↑

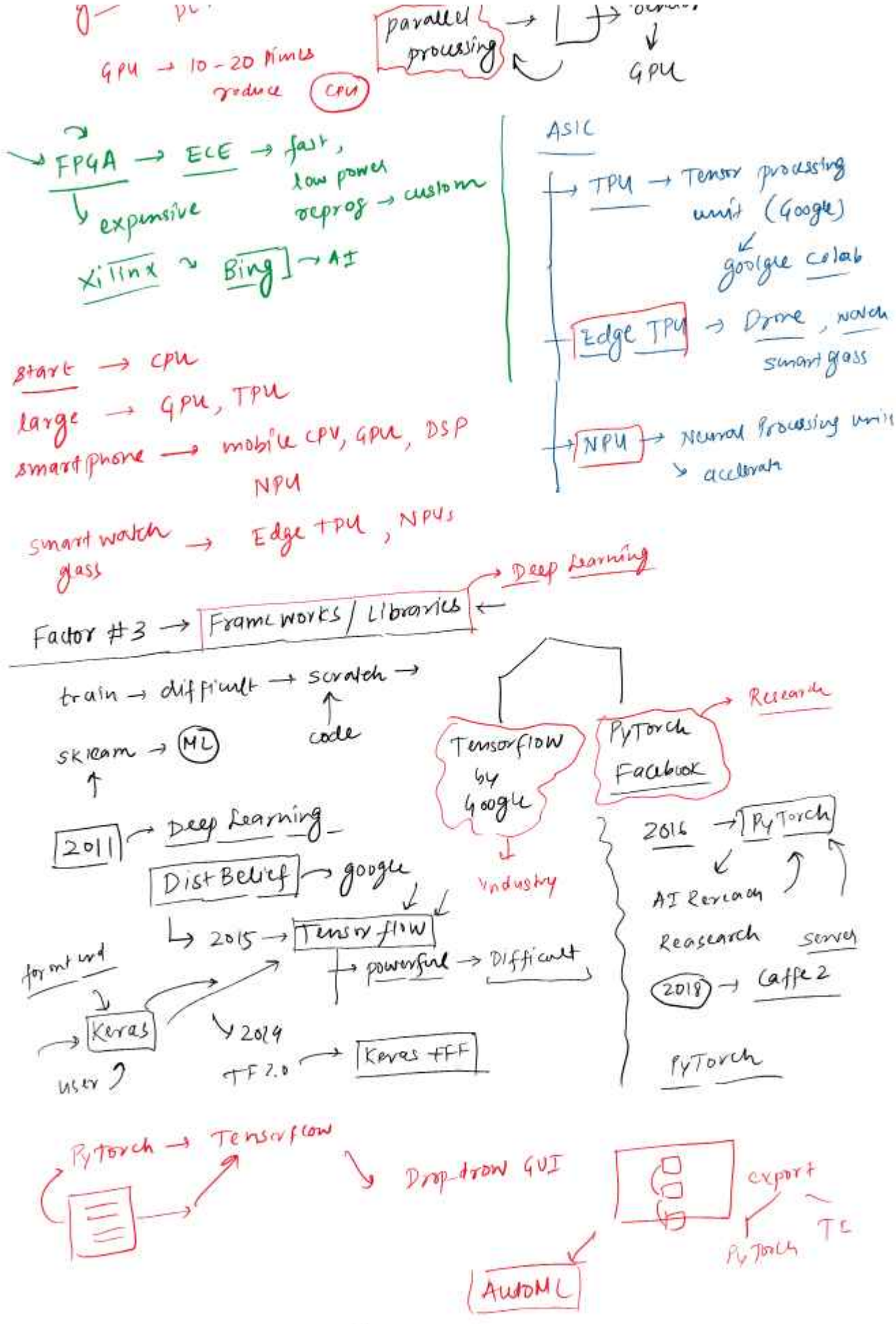
Machine Learning →

Why now?

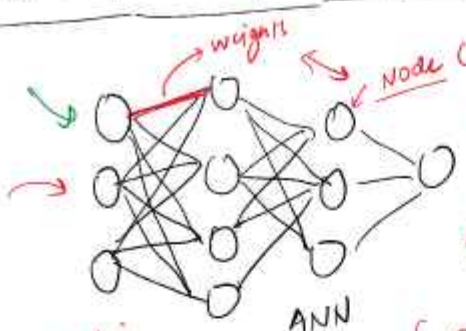
Wednesday, February 16, 2022 6:59 AM






parallel processing
GPU
GPU → 10 - 20 times reduce CPU
FPGA → ECE → fast, low power reprog → custom
expensive
Xilinx ~ Bing → AI
ASIC
TPU → Tensor processing unit (Google)
google colab
Edge TPU → Drone, watch smart glass
NPU → Neural Processing units → accelerate
start → CPU
large → GPU, TPU
smartphone → mobile CPU, GPU, DSP NPU
smart watch glass → Edge TPU, NPUs
Factor #3 → Frameworks / Libraries
Deep Learning
train → difficult → scratch → code
sklearn → ML
2011 → Deep Learning
DistBelief → google
2015 → Tensorflow
powerful → Difficult
Keras
front end
user
2019 TF 2.0 → Keras +FF
Tensorflow by Google
Industry
PyTorch Facebook
Research
2016 → PyTorch
AI Research
Research server
2018 → Caffe2
PyTorch
PyTorch → Tensorflow
Drop-down GUI
export
PyTorch
TF
AutoML
Factor #4 → Deep Learning Architectures

Factor #4 → Deep Learning Architectures



CNN RNN

DNN archi

Experiment
- time
- effort
- money

Transfer learning

State-of-the-art best accuracy

ANN → existing archite

NN → Archi
- files down
- Dataset → train → ready model

image classification → ResNET
Text classifi → BERT
Image segment → UNet
image gan → PIX2PIX

Object dect → YOLO
Speech gen → WaveNET

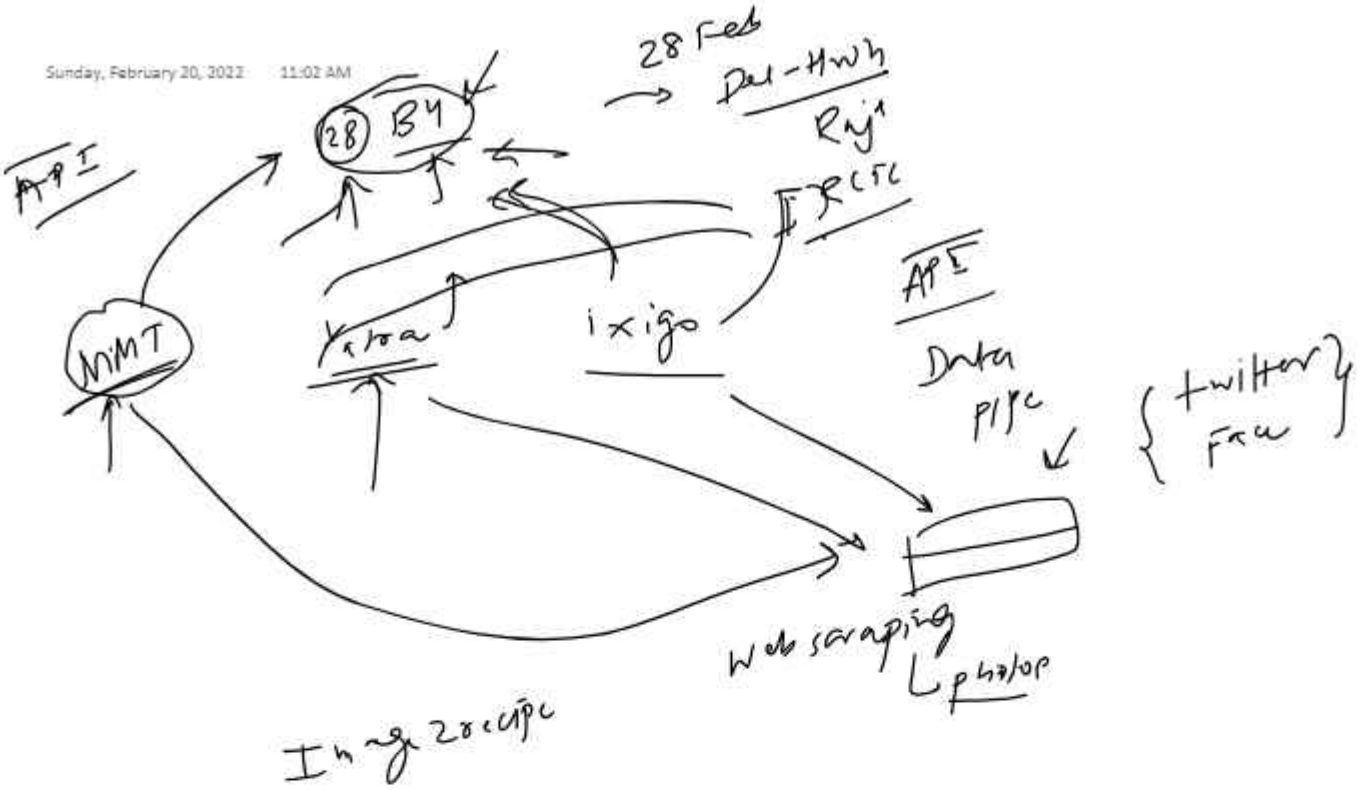
#5 → People / community

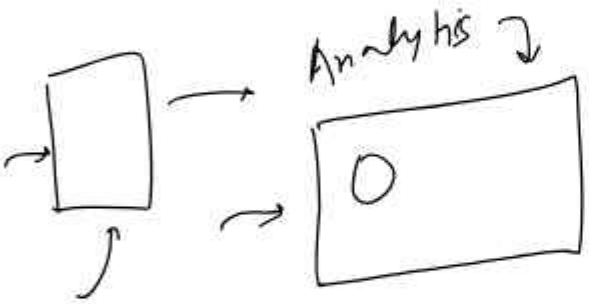
↳ Deep learning → 1960 → 2011 → PL →

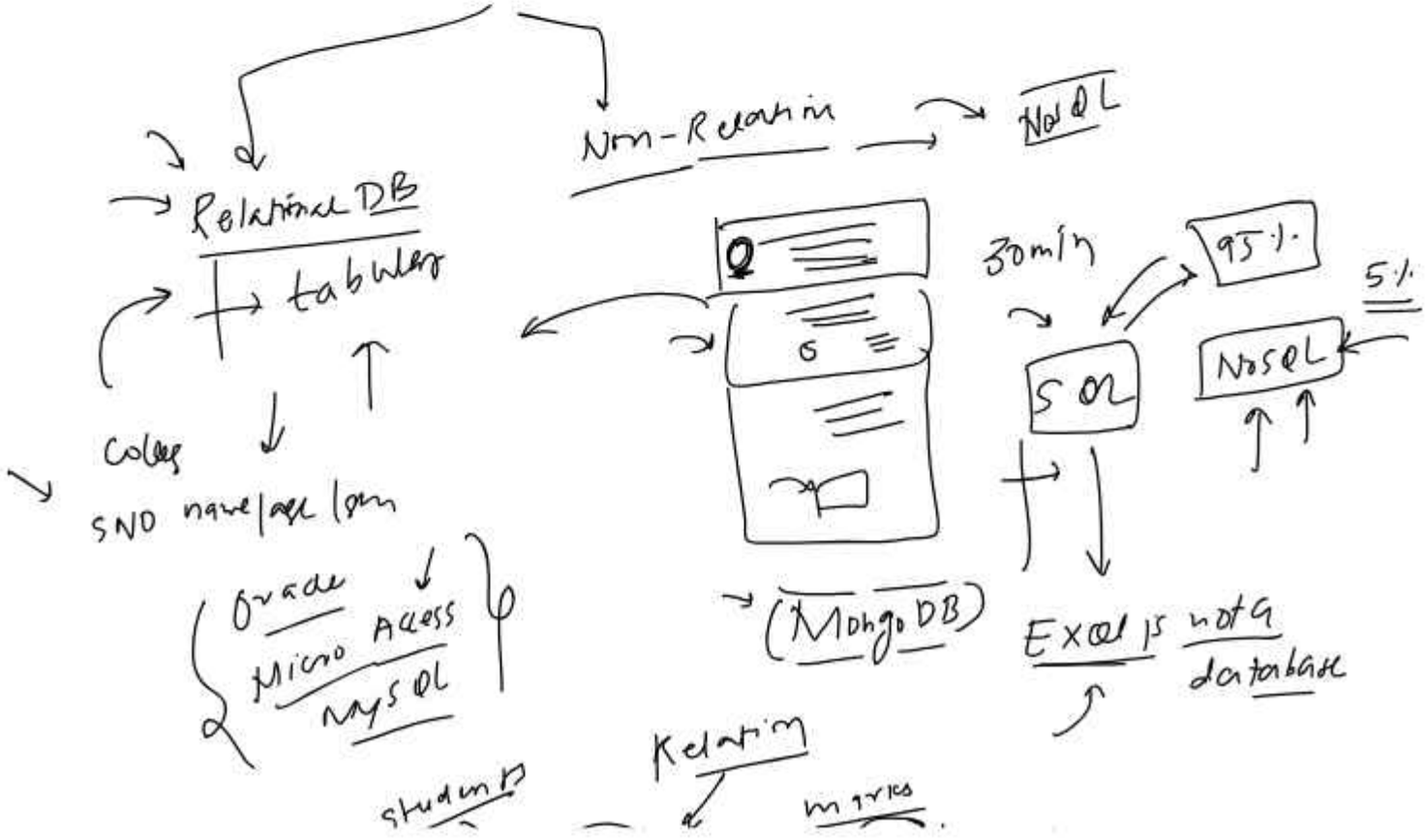
Research
↳ DL engin
↳ Teacher → student → kaggle

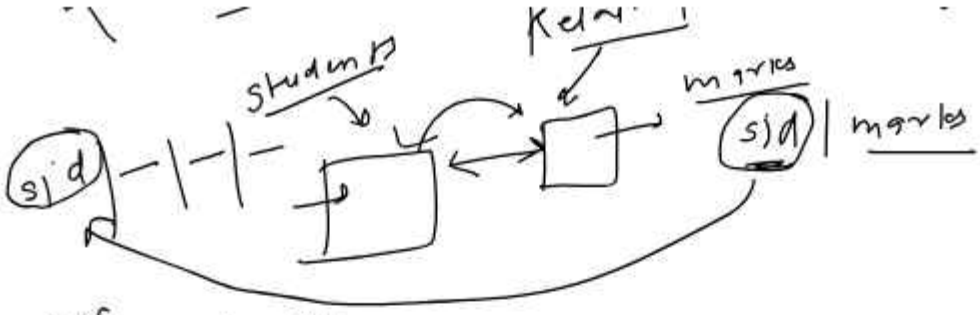
Code Example

Saturday, February 19, 2022 7:10 AM

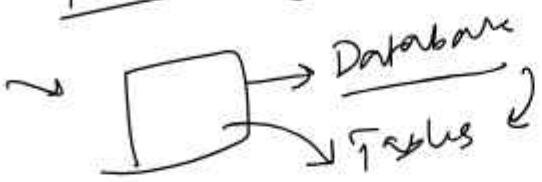

28 Feb
Del-Hwh
Raj
28
BY
API
IRCTC
API
MMT
ixigo
Data
Pipe
{ twitter
Fac }
Web scraping
Laptop
Image recipe

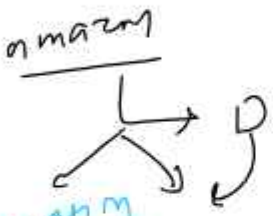

Analytics


Relational DB
tables
Non-Relation
NoSQL
30min
95%
5%
SQL
NoSQL
Colleges
SNO name age ...
Oracle
Micro Access
MySQL
(MongoDB)
Excel is not a database
Relation
student
marks



DBMS Oracle





Normalization

student-marks

DB → storage

sid	name	gender	age	course	marks
1	satyam	M	21	Eng	24
1	Satyam	M	21	Maths	32

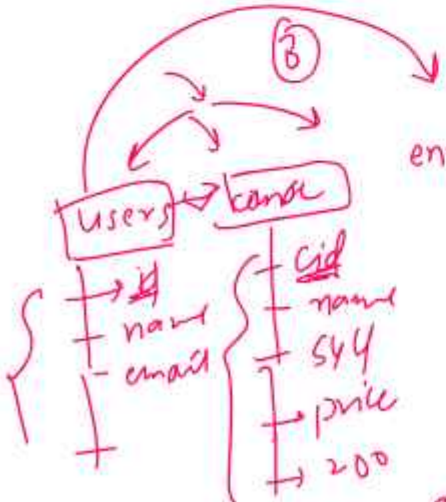
Denormalization

OLTP

OLAP

website

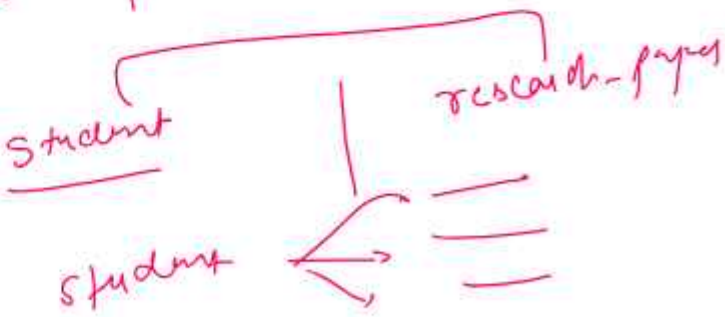
a

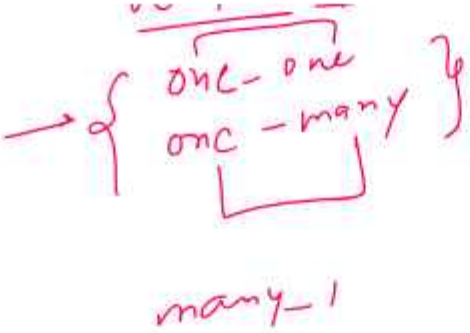




relationship

one-one

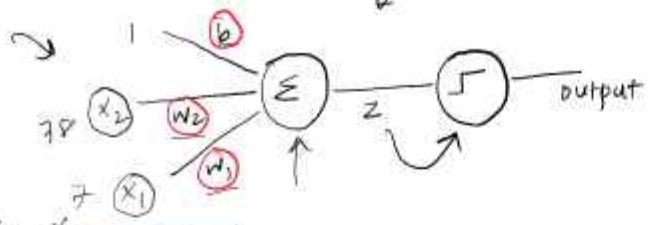



one-one
one-many
many-1

Recap

22 February 2022 14:12

$z = w_1x_1 + w_2x_2 + b$



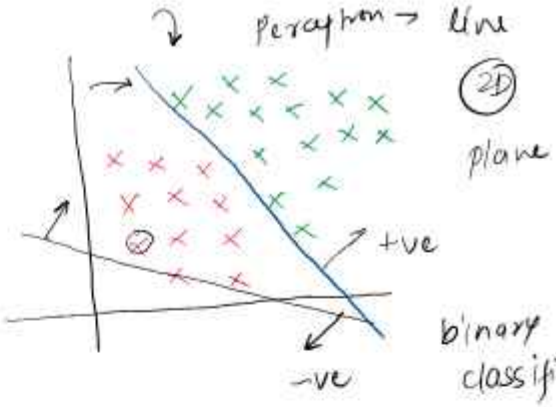
x_1 cgpa	x_2 iq	placed
7	78	1
6	61	0
9	92	1

1) mathematical model → neuron

$z \geq 0 \rightarrow 1$

$z < 0 \rightarrow 0$

Perceptron → line



binary classification

Prediction

→ Training → Perceptron Rule

1020 random

$A \rightarrow w_1$

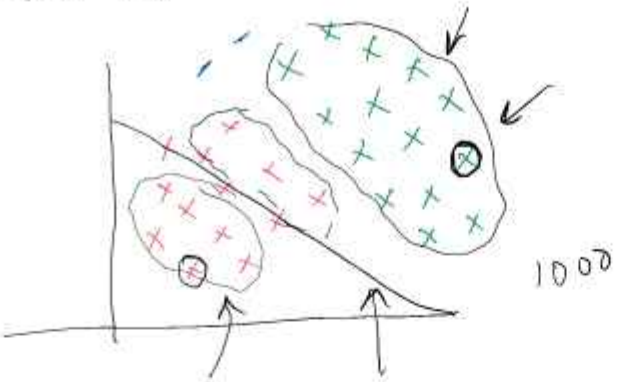
$B \rightarrow w_2$

$C \rightarrow b$

$Ax + By + C = 0$

Problem with Perceptron Trick

22 February 2022 16:10



1000

line → w_1, w_2, b

randomly per

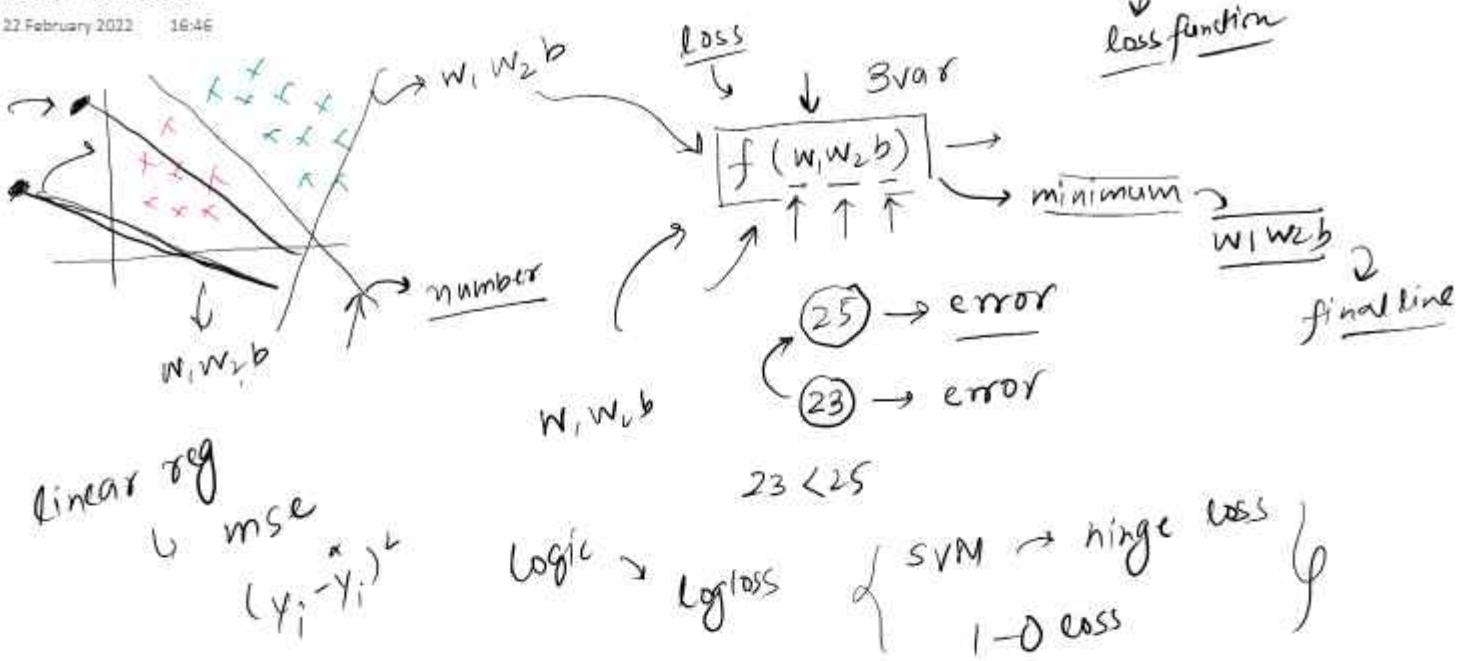
converge

quantify

w_1, w_2, b → loss functions

Loss Function

22 February 2022 16:46



linear reg

↳ mse $(y_i - \hat{y}_i)^2$

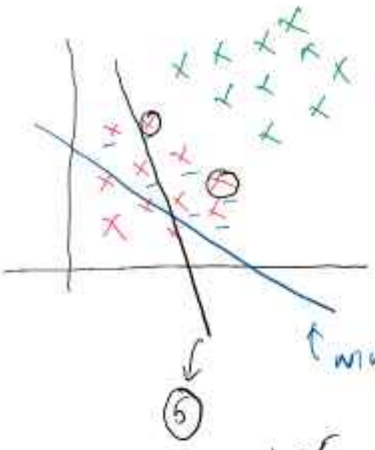
Logic ↘ Log loss { SVM → hinge loss, 1-0 loss }

Perceptron Loss Function

22 February 2022 17:47

$f(w_1, w_2, b)$ → number → error

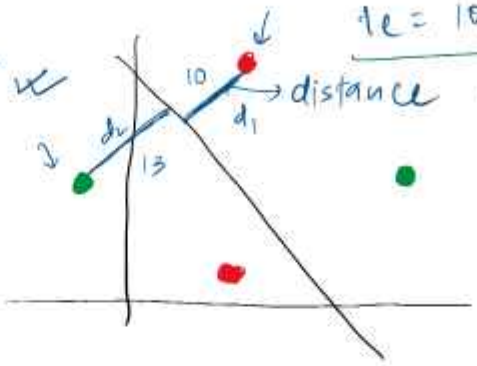
misclassified points



$w_1 w_2 b$ → 7

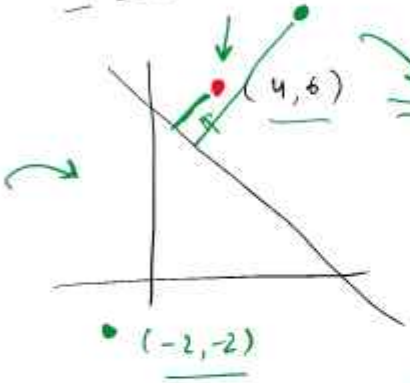
5

$1+1+1+1+1 = 5$



$10 + 13$

compute ↑ distance



$2(4) + 3(6) + 4 = 30$

↑ dot product

$30 + 6$

$2(-2) + 3(-2) + 4 = |-6| = 6$

Sklearn

$$L(w_1, w_2, b) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, f(x_i)) + \alpha R(w_1, w_2)$$

Regularization

categorical placement → y_i

$$L(y_i, f(x_i)) = \max(0, -y_i f(x_i))$$

$$f(x_i) = w_1 x_1 + w_2 x_2 + b$$

$z =$

$$L = \frac{1}{n}\sum_{i=1}^{n} \max(0, -y_i f(x_i))$$

hinge

$L(w_1, w_2, b)$

n: # rows in data

$w_1 w_2 b$

$$L = \underset{w_1, w_2, b}{\operatorname{argmin}} \frac{1}{n}\sum_{i=1}^{n} \max(0, -y_i f(x_i))$$

minimum

gradient descent

$L = \underset{w_1, w_2, b}{\operatorname{argmin}} \sum_{i=1}^{n}$

gradient descent

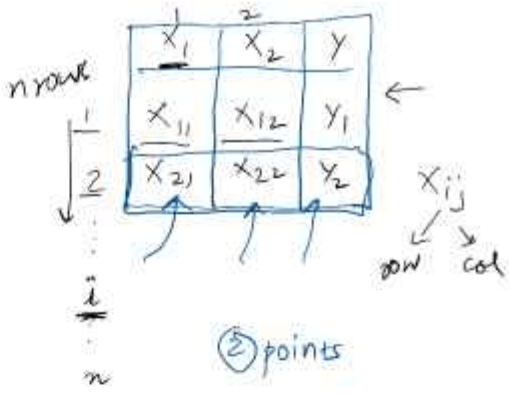
Explanation of Loss Function

23 February 2022 08:07

$$L = \frac{1}{n}\sum_{i=1}^{n} \max(0, -y_i f(x_i))$$

w_1, w_2, b

where $f(x_i) = w_1 x_{i1} + w_2 x_{i2} + b$



$\max(0, -y_i f(x_i)) = \max(0, x)$ → $= x$ if $x \geq 0$; $= 0$ if $x < 0$

$-y_i f(x_i) \geq 0$; $< 0 \Rightarrow = 0$

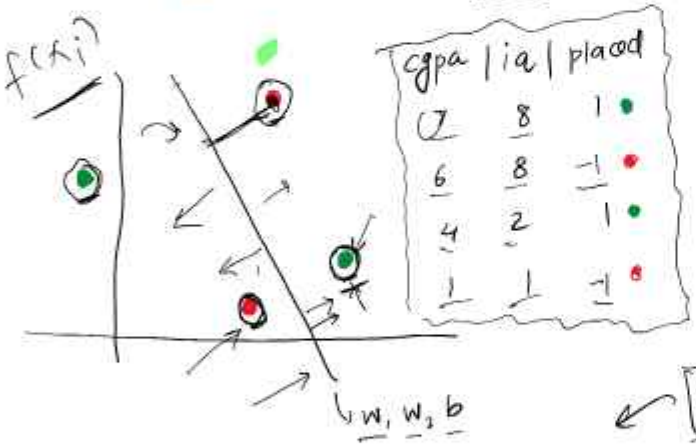
$f(x_2) = w_1 x_{21} + w_2 x_{22} + b$

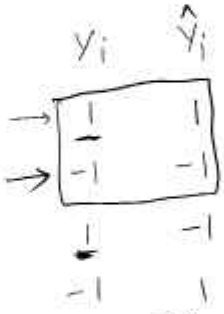
$y_i f(x_i)$?

$L = \frac{1}{2}\left[\max(0, -y_1 f(x_1)) + \max(0, -y_2 f(x_2))\right]$ (2 points, n points)

$= 0$ distance

$\max(0, -y_i f(x_i))$





$w_1 x_1 + w_2 x_2 + b \geq 0$

$\max(0, -ve) = 0$

1 $f(x_i) = +ve$

+ve +ve = −ve

$\max(0, -ve) = 0$

y_i $f(x_i) = +ve$

−1

−ve −ve −ve

y_i $f(x_i)$

+ve −ve = −ve = +ve

+1

$\max(0, +ve) = +ve$

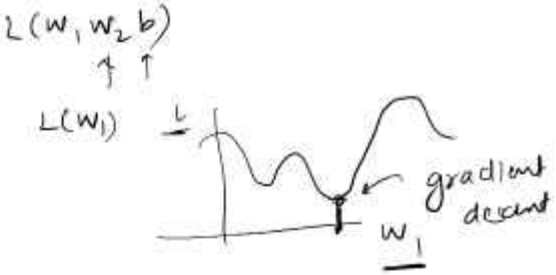
y_i $f(x_i)$

−ve +ve → −ve −1 = +ve

Gradient Descent

23 February 2022 08:50

$$L = \underset{w_1, w_2 b}{\operatorname{argmin}} \boxed{\frac{1}{n} \sum_{i=1}^{n} \max(0, -y_i f(x_i))}$$



where $f(x_i) = w_1 x_{i1} + w_2 x_{i2} + b$

100/1000

$\eta = 0.01$

$(w_1, w_2, b) = 1$

for i in epochs:

$$w_1 = w_1 + \eta \frac{\partial L}{\partial w_1}$$

$$w_2 = w_2 + \eta \frac{\partial L}{\partial w_2}$$

$$b = b + \eta \frac{\partial L}{\partial b}$$

$$\left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b} \right]$$

Loss Function Differentiation

23 February 2022 12:57

$$L = \frac{1}{n} \sum_{i=1}^{n} \max(0, -y_i f(x_i))$$

where $f(x_i) = w_1 x_{i1} + w_2 x_{i2} + b$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial f(x_i)} \times \frac{\partial f(x_i)}{\partial w_1}$$

cgpa is x_{i1}

$$\frac{\partial L}{\partial f(x_i)} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i & \text{if } y_i f(x_i) < 0 \end{cases} \qquad \frac{\partial f(x_i)}{\partial w_1} = x_{i1}$$

$$\frac{\partial L}{\partial w_1} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i x_{i1} & \text{if } y_i f(x_i) < 0 \end{cases}$$

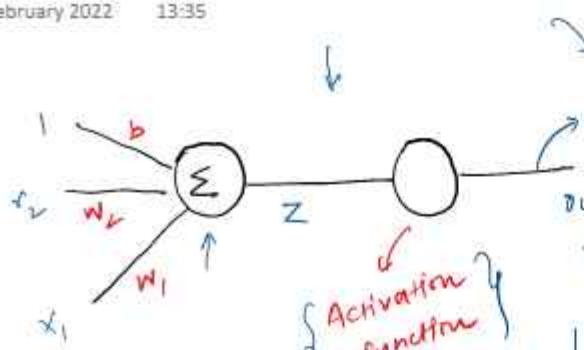
$$\frac{\partial L}{\partial w_2} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i x_{i2} & \text{if } y_i f(x_i) < 0 \end{cases}$$

→ code using gradient descent

$$\frac{\partial L}{\partial b} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i & \text{if } y_i f(x_i) < 0 \end{cases}$$

More Loss Functions

23 February 2022 13:35



mathematical model

flexible by design

Loss function

SGD

Loss function → binary cross entropy

$L = -y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)$

1, -1 prob

$z = w_1 x_1 + w_2 x_2 + b$

$\sigma(z) = \frac{1}{1+e^{-z}}$

SGD → output prob.

Perceptron == Logistic Regression

Activation: sigmoid

Loss func

multiclass (3)

Activation → softmax

$f = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$

Loss function → categorical cross entropy

$L = \sum_{j=1}^{M} y_j \log(\hat{y}_j)$

Softmax regression

$f = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$

$z \geq 0 \quad 1$

$z < 0 \quad -1$

(z) → (z)

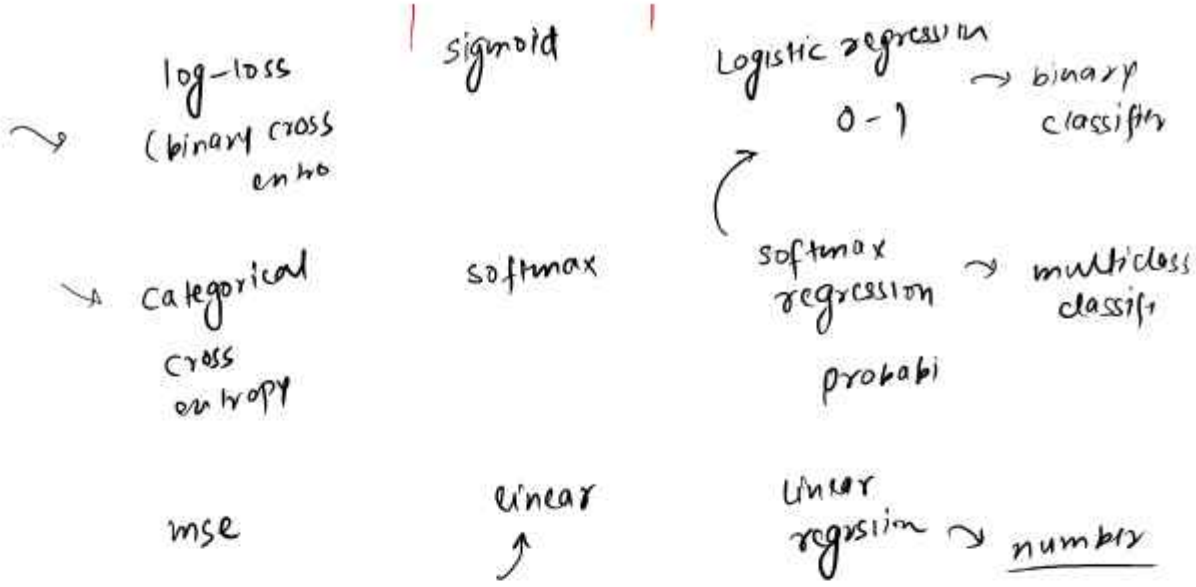
Regression

Activation → linear

Loss function → mse $(y_i - \hat{y}_i)^2$

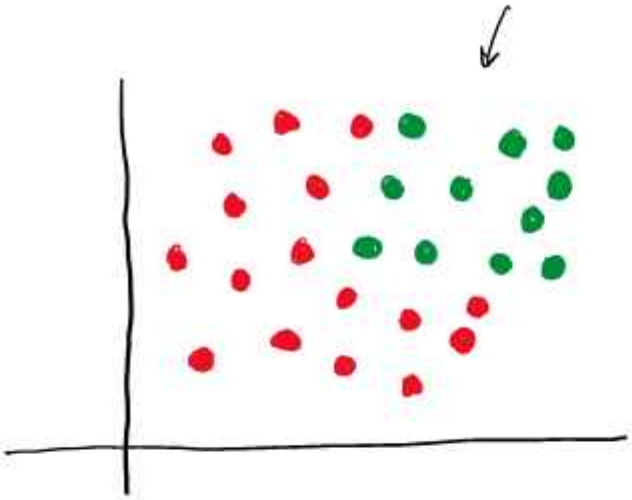
→ Linear regression

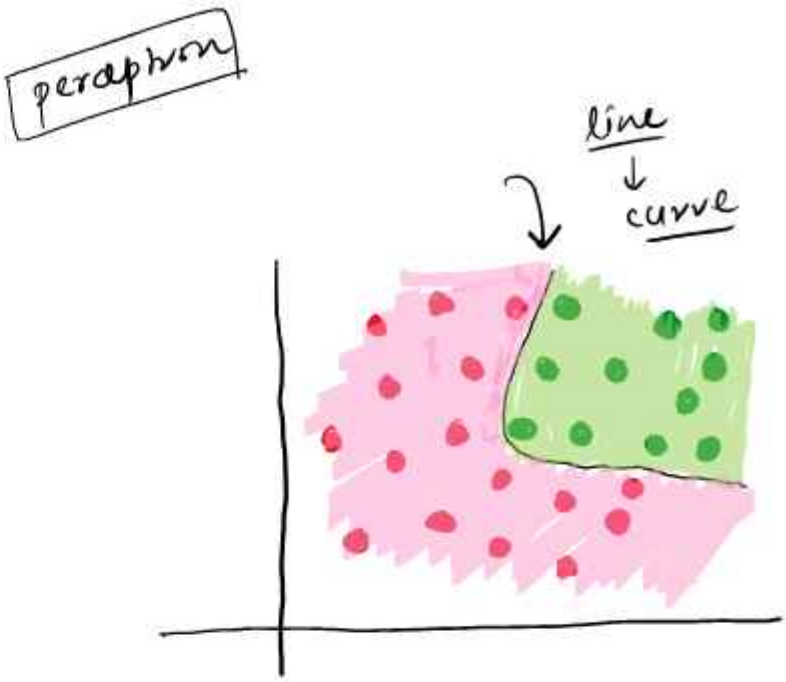
Loss function	Activation	output
Hinge loss	Step	Perceptron → binary classifier (-1, 1)
log-loss	sigmoid	Logistic regression → binary


log-loss
(binary cross entro
sigmoid
Logistic regression
0-1
binary classifi
softmax
softmax regression
multiclass classifi
probabi
categorical cross entropy
mse
linear
linear regresiin
numbr

The Problem

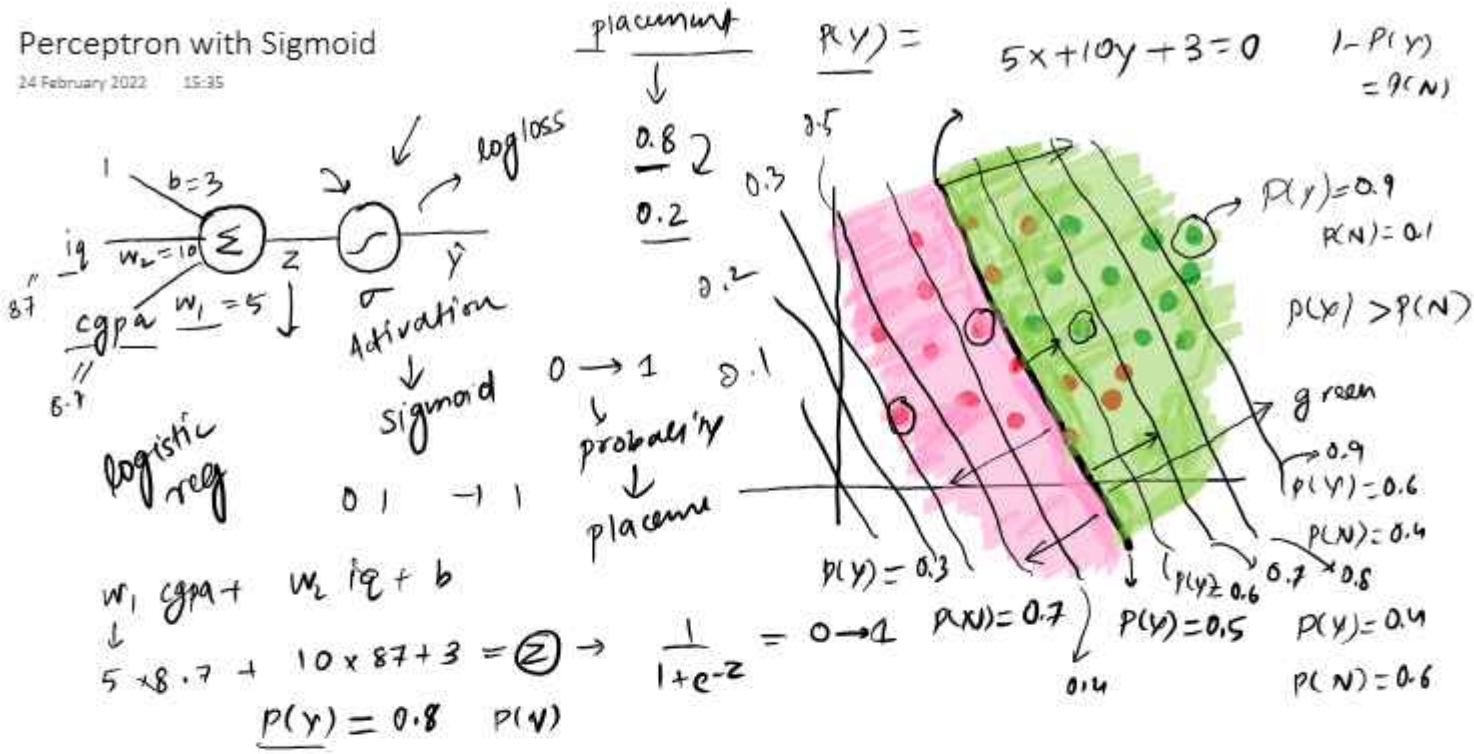
24 February 2022 13:41



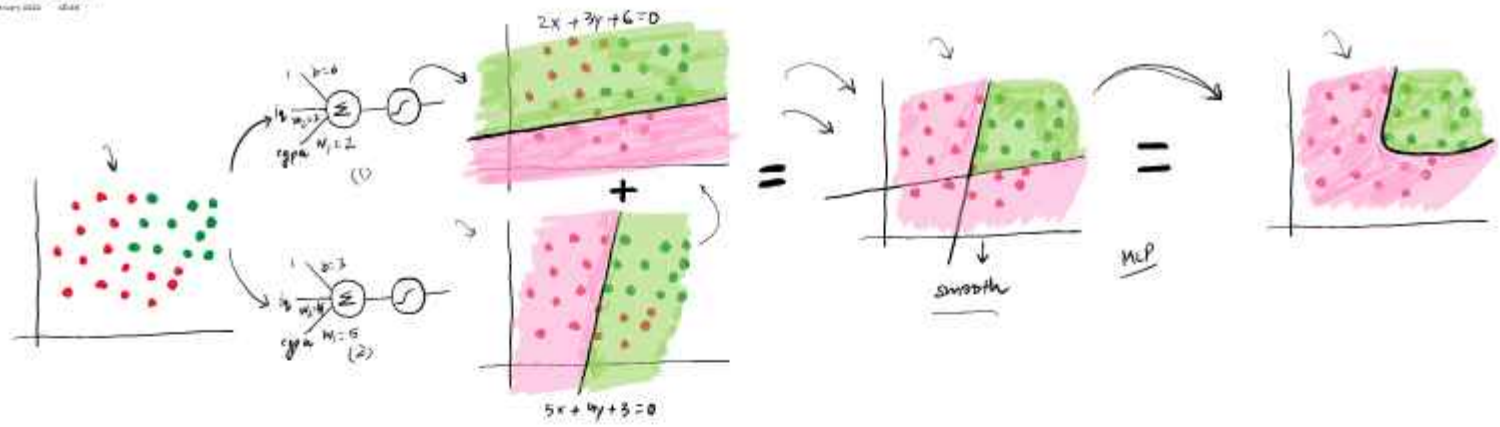


Perceptron with Sigmoid

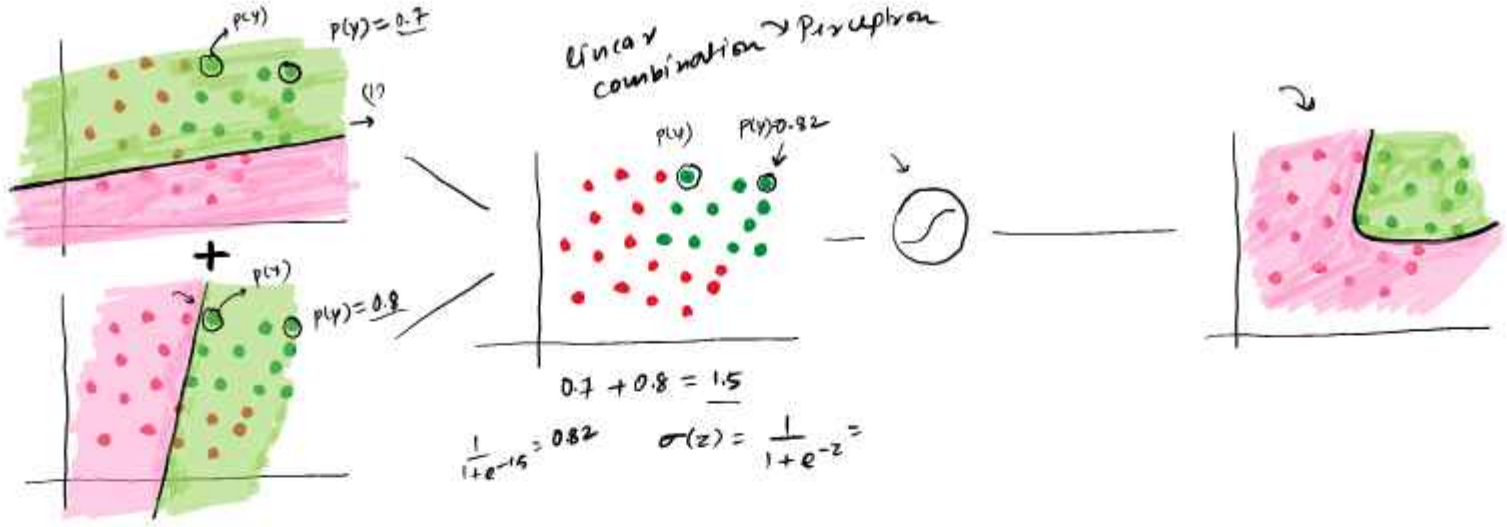
24 February 2022 15:35



Abstract Solution

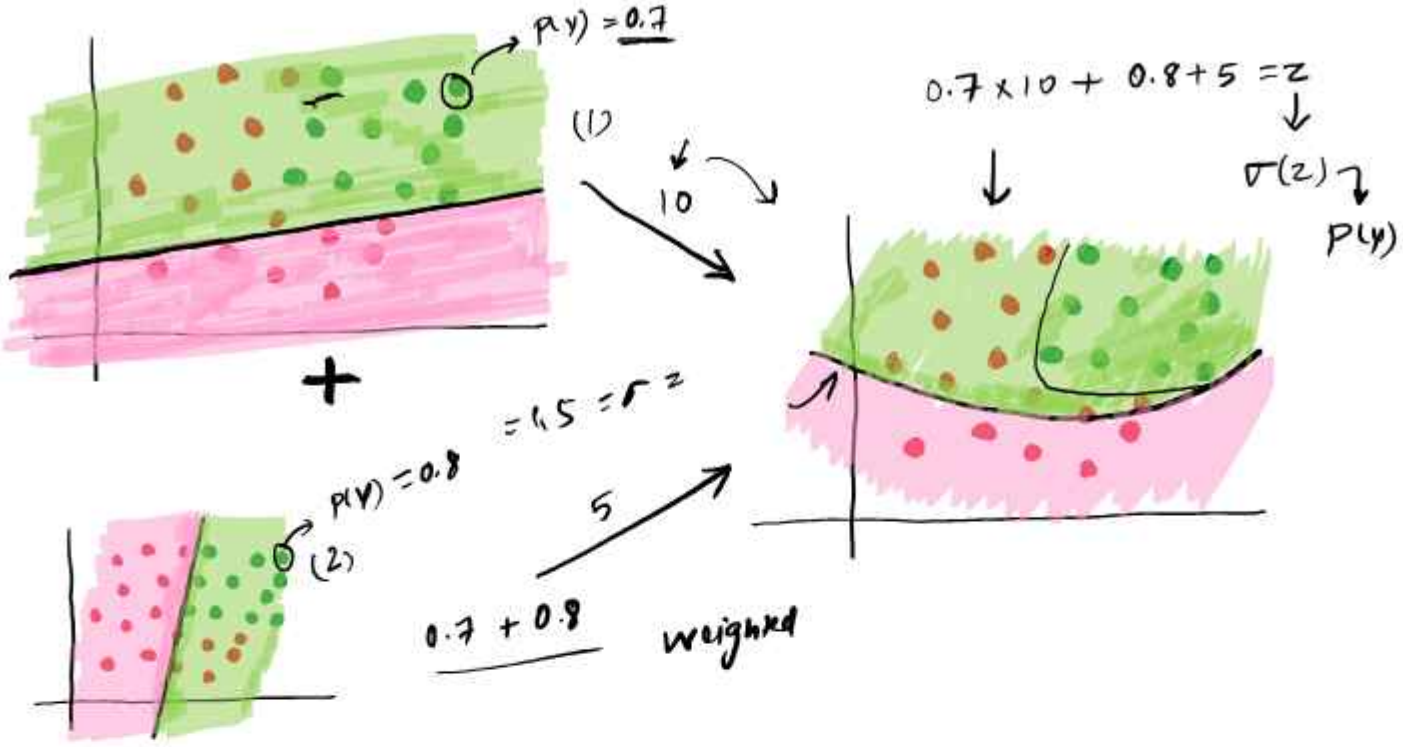


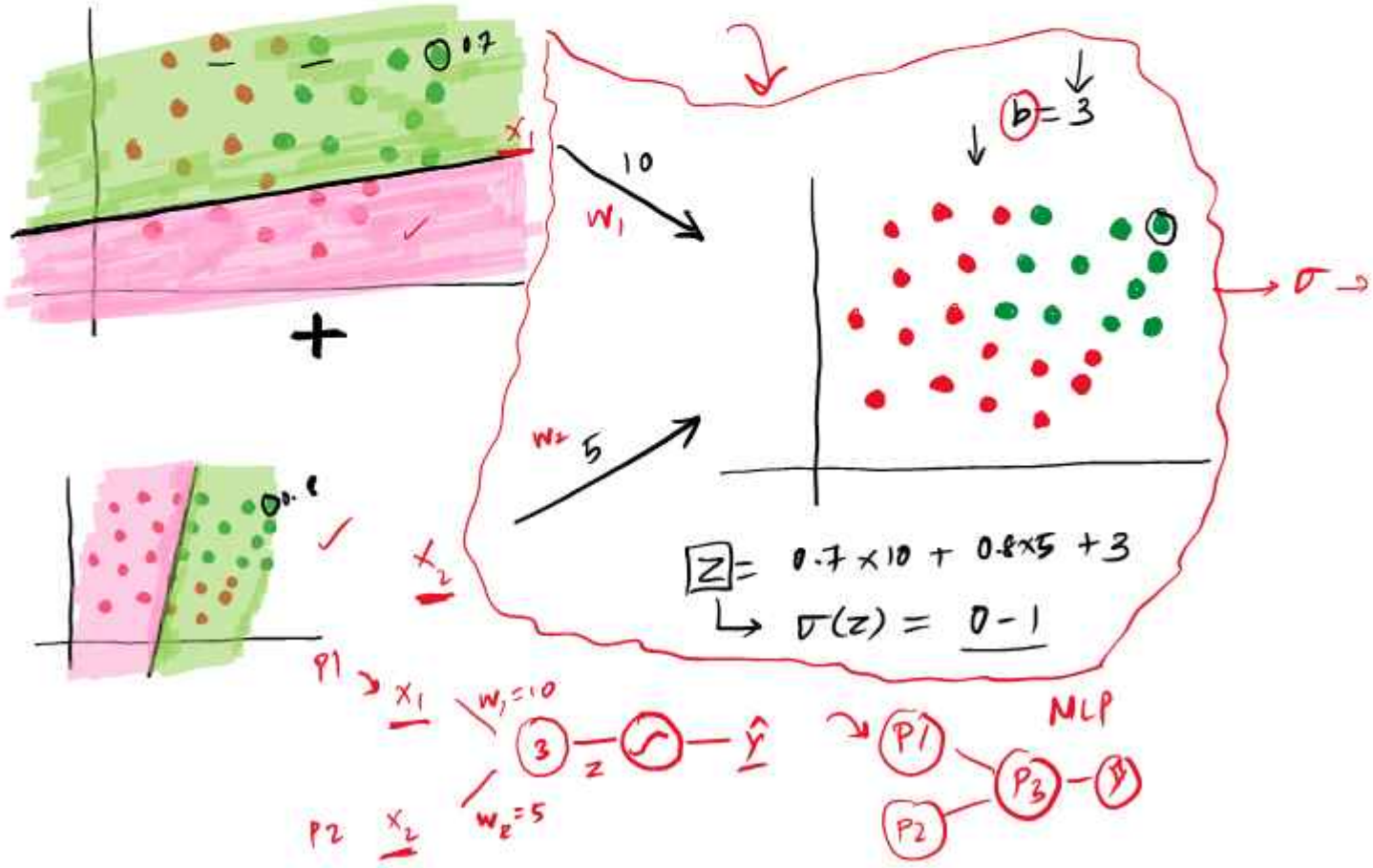
Mathematical Reasoning



Adding Weights

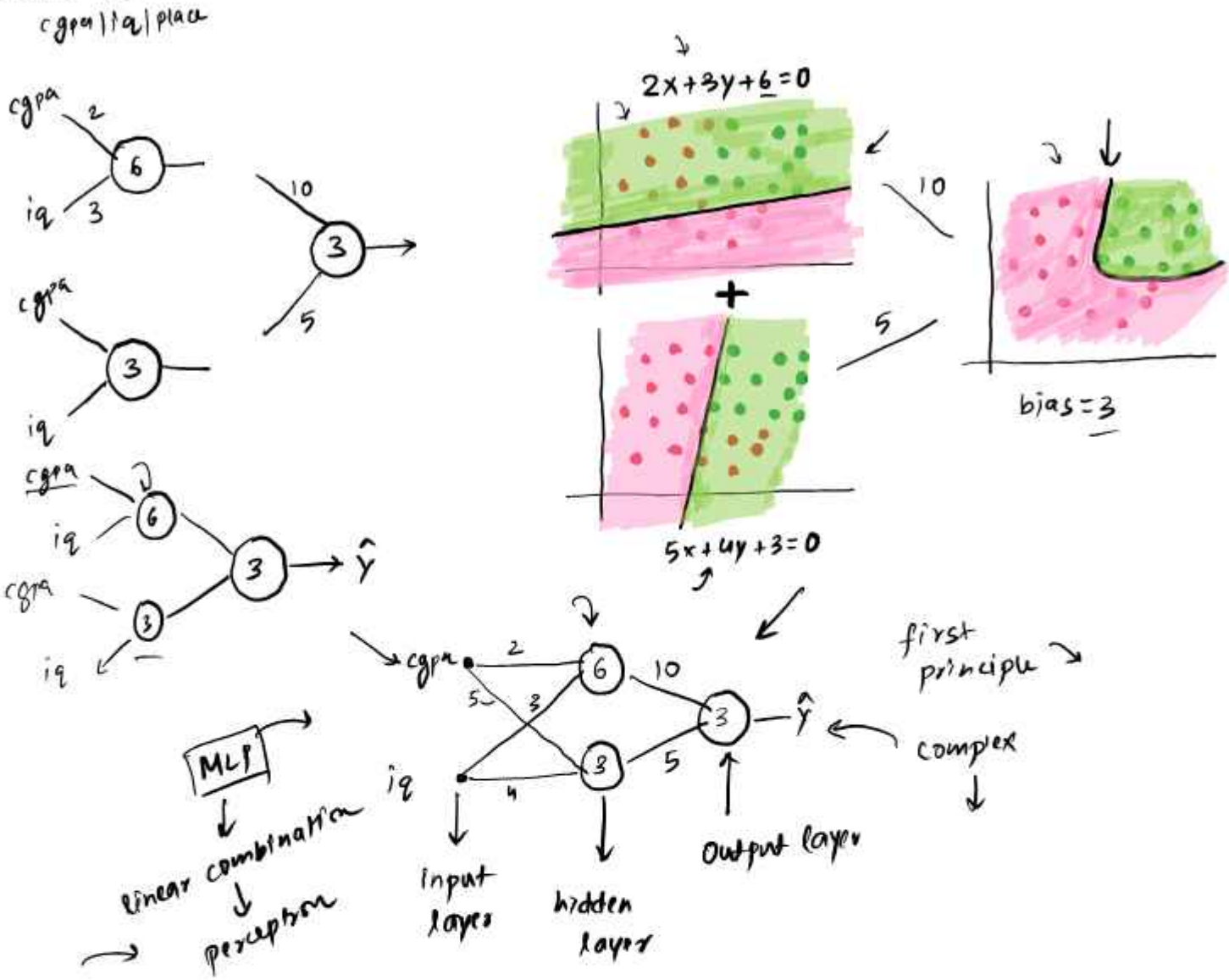
24 February 2022 15:37





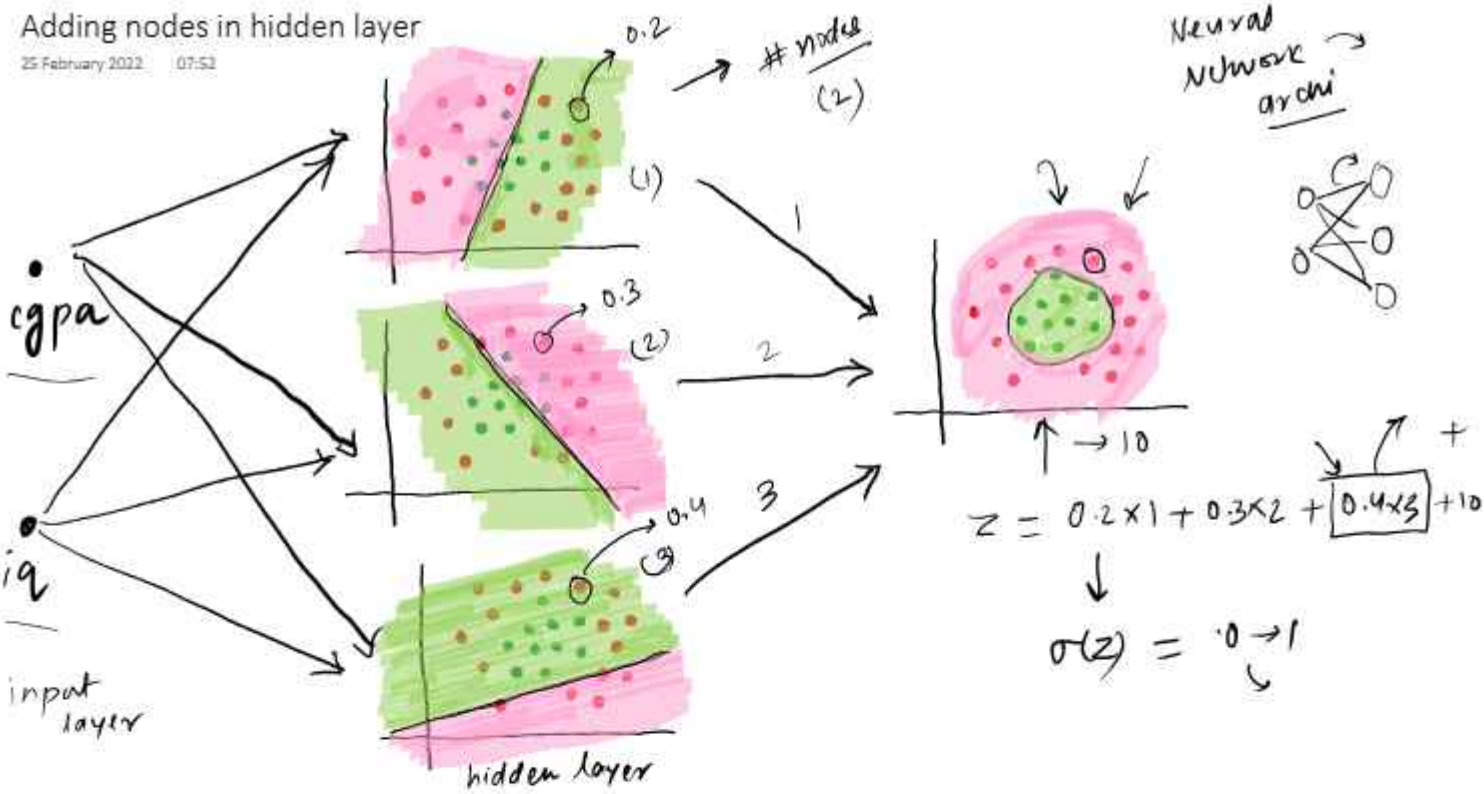
The Idea of MLP

24 February 2022 15:37



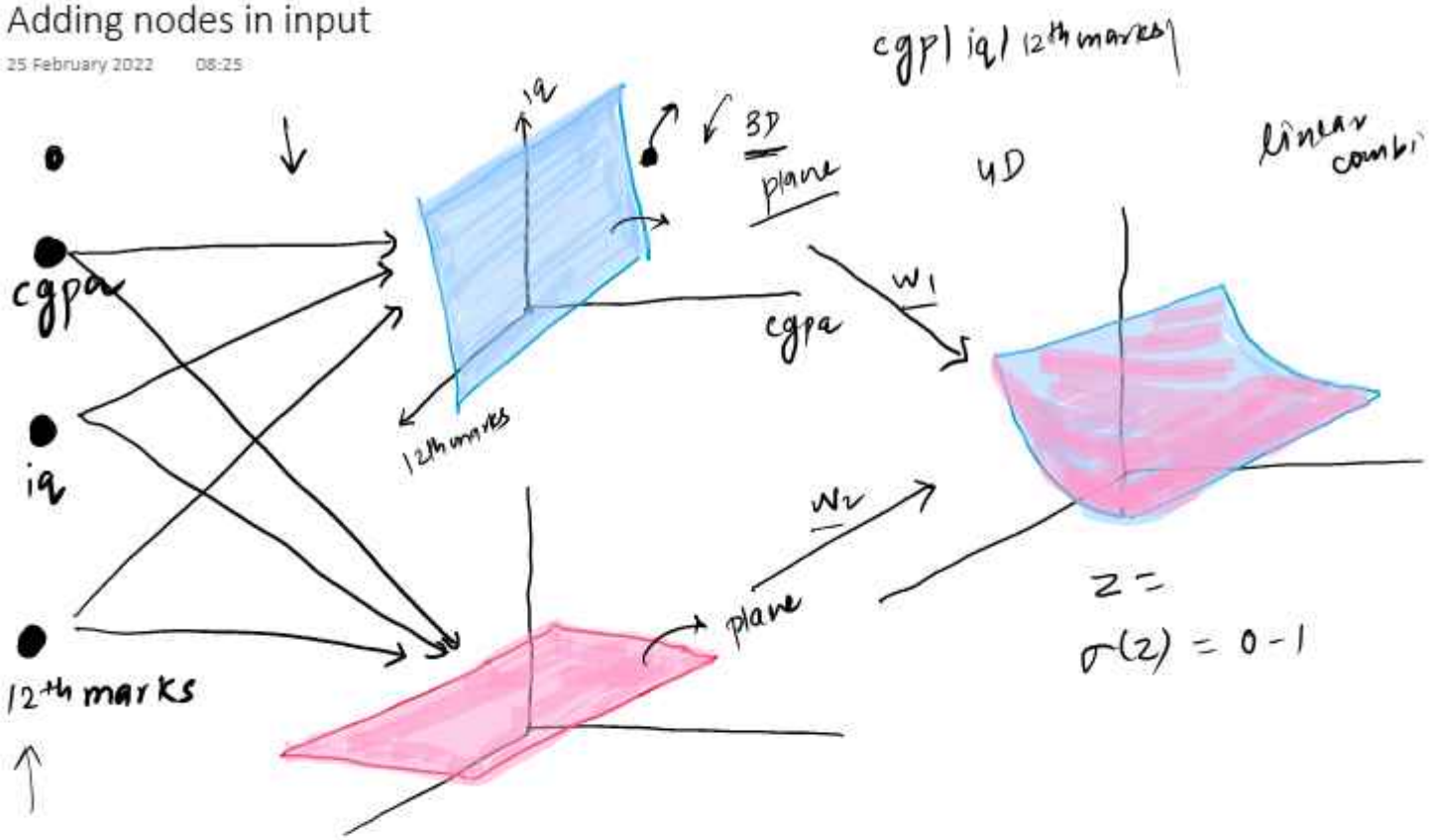
Adding nodes in hidden layer

25 February 2022 07:52



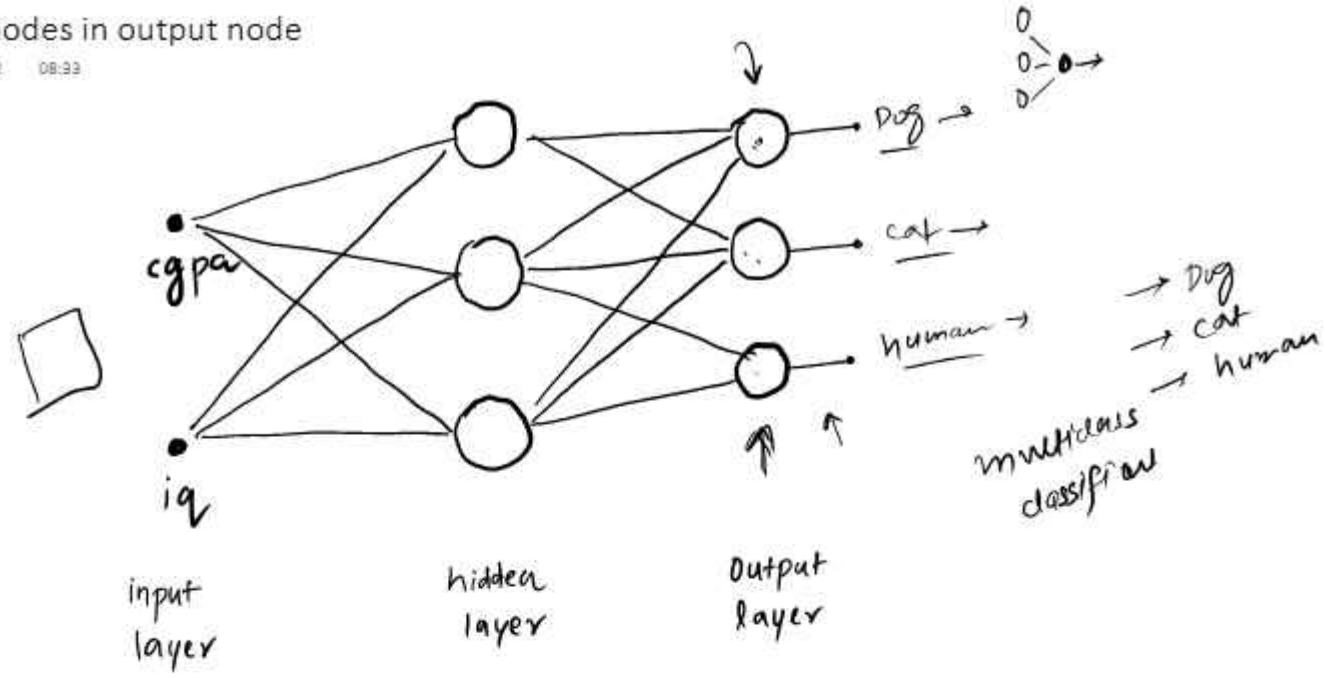
Adding nodes in input

25 February 2022 08:25



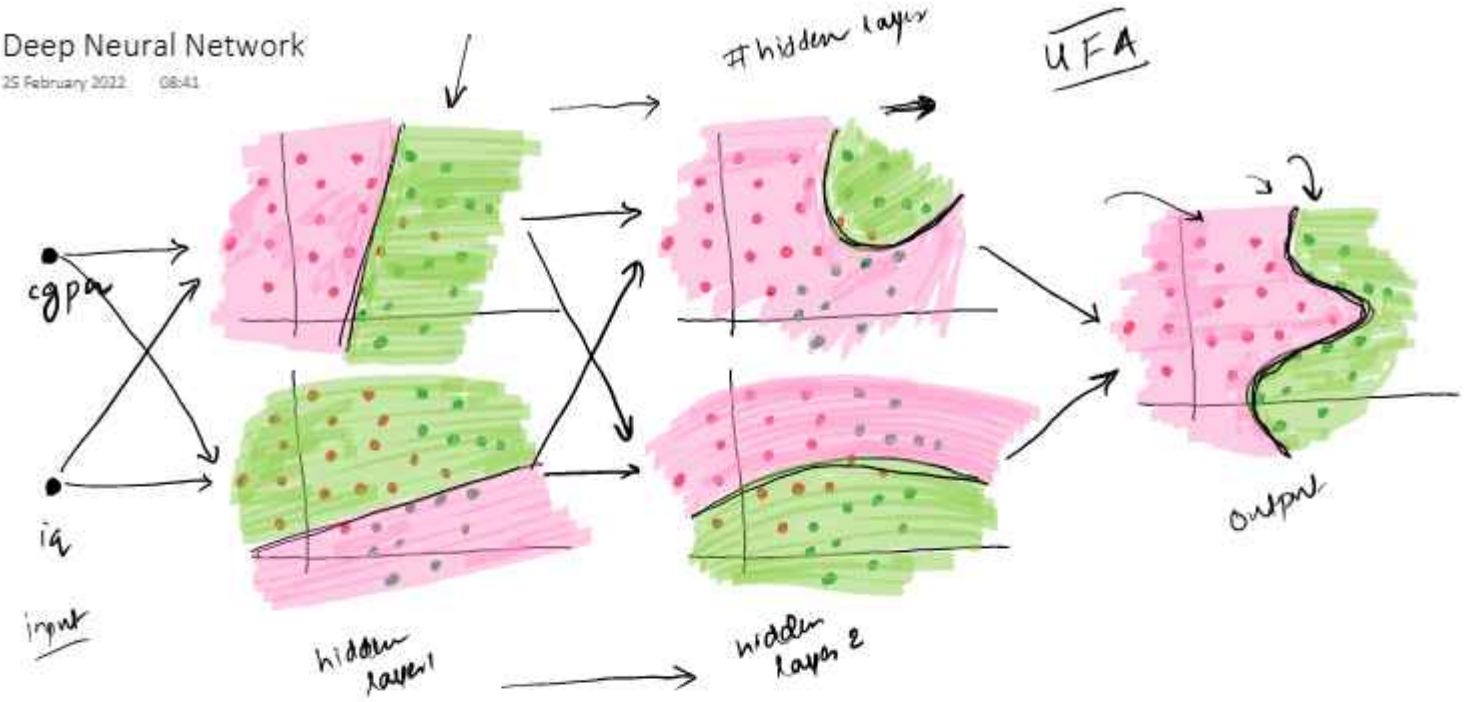
Adding nodes in output node

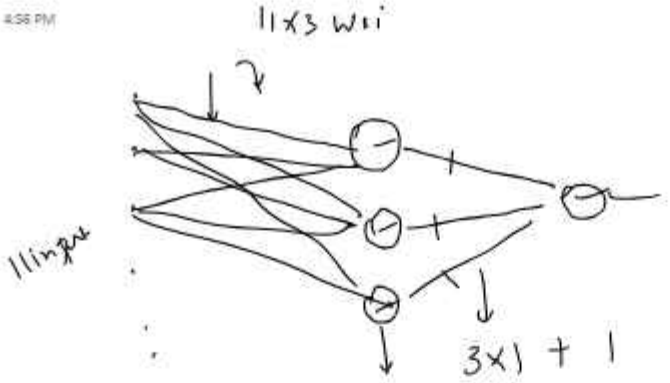
25 February 2022 08:33



Deep Neural Network

25 February 2022 08:41

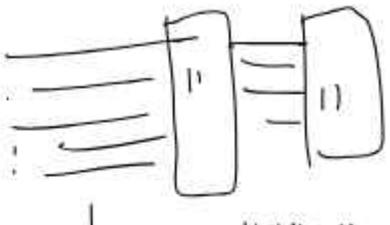




3x1 + 1

11x3 +3 4 = 40

36 +



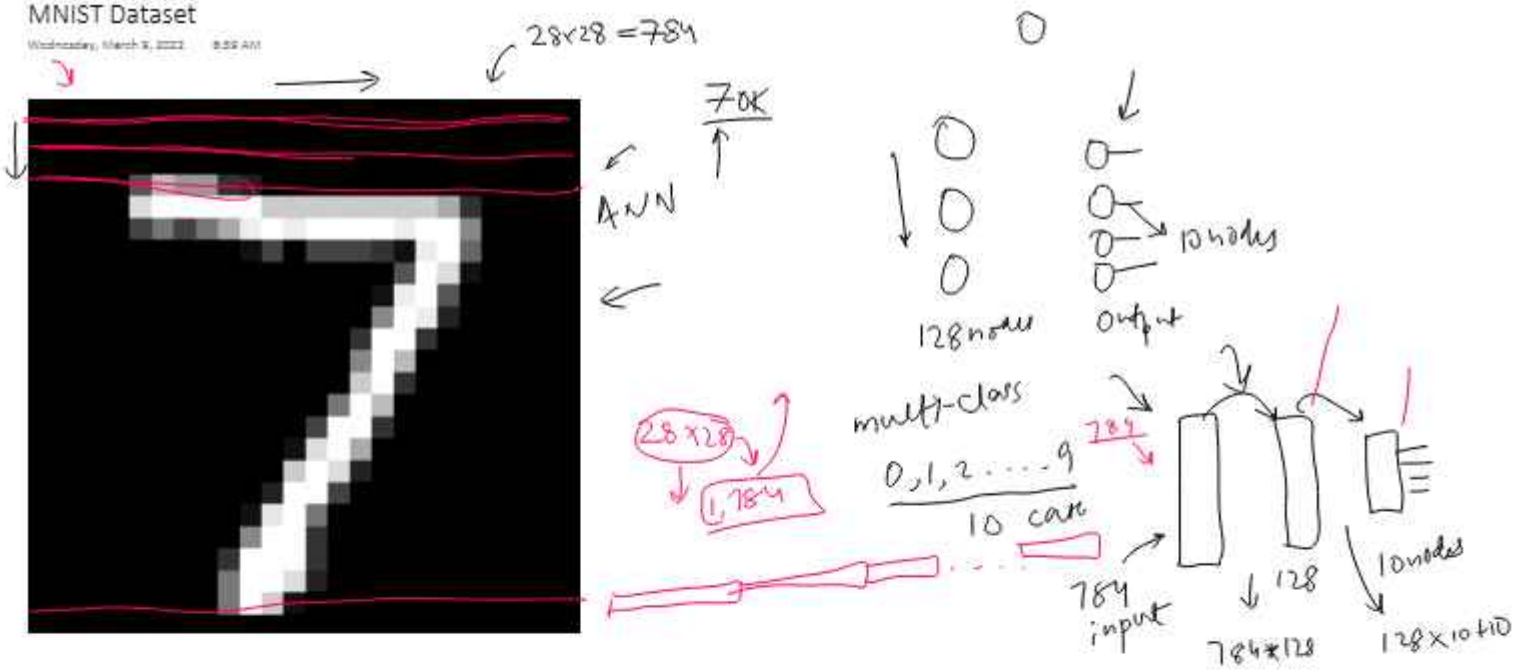
11x11+11 11x11+11

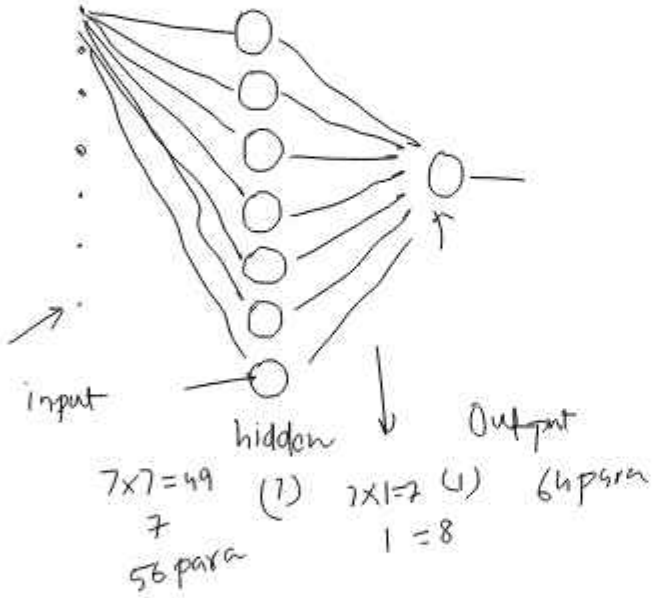
133 133

266 = 278
12

MNIST Dataset

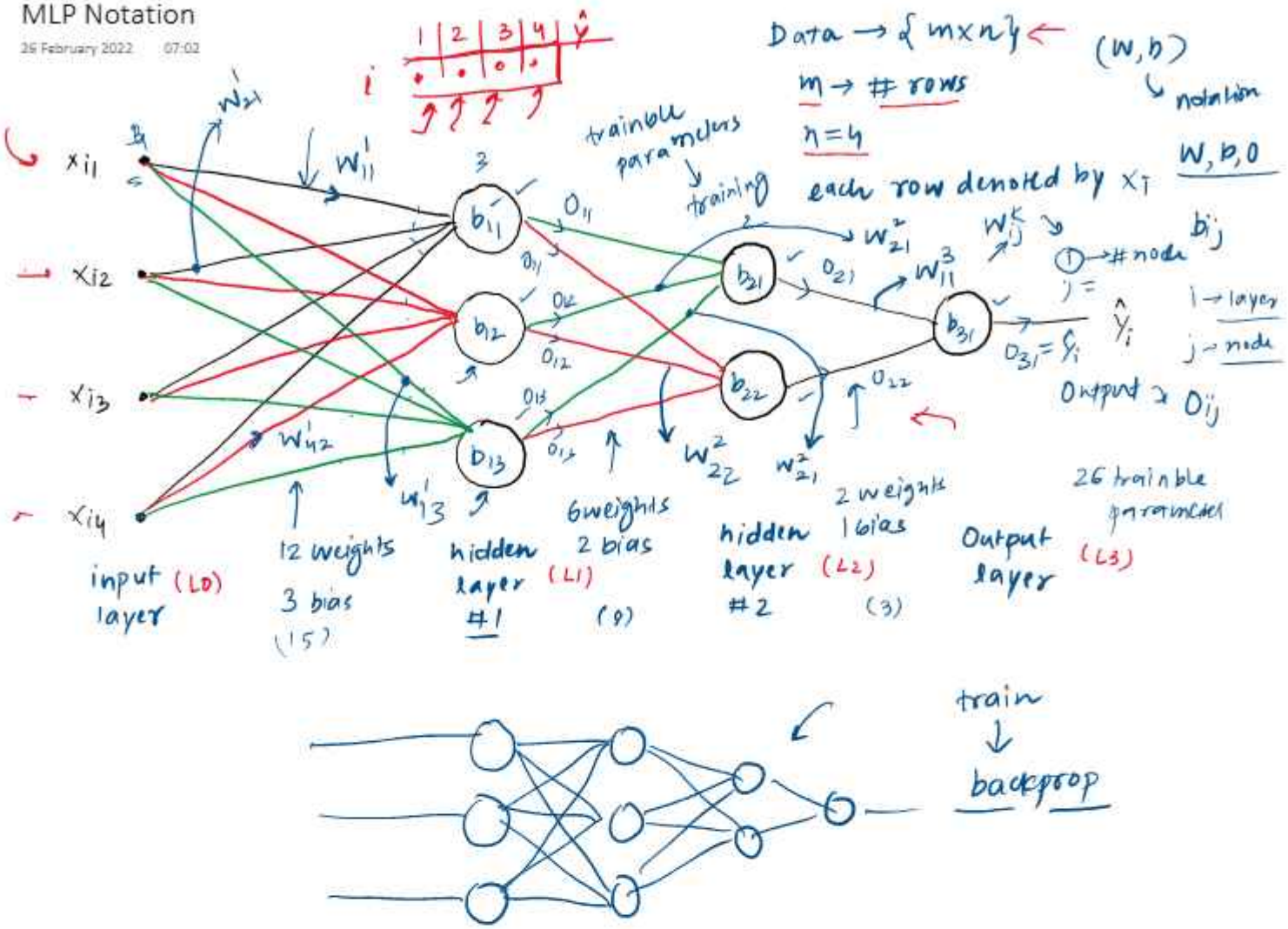
Wednesday, March 9, 2022 8:59 AM





MLP Notation

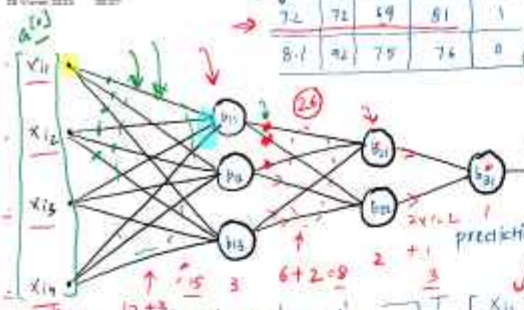
29 February 2022 07:02



Forward Propogation

CGPA	IQ	10th m	12th m	placed
7.2	72	69	81	1
8.1	92	79	76	0

$\hat{y}$ → # of trainable parameters



$$a^{[1]} = \sigma(a^{[0]} w^{[1]} + b^{[1]})$$

$$a^{[2]} = \sigma(a^{[1]} w^{[2]} + b^{[2]})$$

$$a^{[3]} = \sigma(a^{[2]} w^{[3]} + b^{[3]})$$

$$\sigma\left(\sigma\left(\left(\sigma(a^{[0]} w^{[1]} + b^{[1]})\right) w^{[2]} + b^{[2]}\right) w^{[3]} + b^{[3]}\right)$$

$a^{[1]}$ | $a^{[2]}$ | $a^{[3]} = \hat{y}$

Layer #1

$$\begin{bmatrix} w^1_{11} & w^1_{12} & w^1_{13} \\ w^1_{21} & w^1_{22} & w^1_{23} \\ w^1_{31} & w^1_{32} & w^1_{33} \\ w^1_{41} & w^1_{42} & w^1_{43} \end{bmatrix}^T \begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \end{bmatrix} + \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix}$$

4×3 → 3×4 | 3×1 | 4×1 | 3×1 → 3×1

$$= \begin{bmatrix} w^1_{11}x_{11} + w^1_{21}x_{12} + w^1_{31}x_{13} + w^1_{41}x_{14} \\ w^1_{12}x_{11} + w^1_{22}x_{12} + w^1_{32}x_{13} + w^1_{42}x_{14} \\ w^1_{13}x_{11} + w^1_{23}x_{12} + w^1_{33}x_{13} + w^1_{43}x_{14} \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix}$$

3×1

$$= \sigma\left(\begin{bmatrix} w^1_{11}x_{11} + w^1_{21}x_{12} + w^1_{31}x_{13} + w^1_{41}x_{14} + b_{11} \\ w^1_{12}x_{11} + w^1_{22}x_{12} + w^1_{32}x_{13} + w^1_{42}x_{14} + b_{12} \\ w^1_{13}x_{11} + w^1_{23}x_{12} + w^1_{33}x_{13} + w^1_{43}x_{14} + b_{13} \end{bmatrix}\right)$$

$$= \begin{bmatrix} O_{11} \\ O_{12} \\ O_{13} \end{bmatrix} \rightarrow a^{[1]}$$

Layer #2

$$\begin{bmatrix} w^2_{11} & w^2_{12} \\ w^2_{21} & w^2_{22} \\ w^2_{31} & w^2_{32} \end{bmatrix}^T \begin{bmatrix} O_{11} \\ O_{12} \\ O_{13} \end{bmatrix} + \begin{bmatrix} b_{21} \\ b_{22} \end{bmatrix} \rightarrow$$

3×2 → 2×3 | 3×1 | 2×1

$$= \sigma\left(\begin{bmatrix} w^2_{11}O_{11} + w^2_{21}O_{12} + w^2_{31}O_{13} + b_{21} \\ w^2_{12}O_{11} + w^2_{22}O_{12} + w^2_{32}O_{13} + b_{22} \end{bmatrix}\right) = \begin{bmatrix} O_{21} \\ O_{22} \end{bmatrix} \leftarrow a^{[2]}$$

Layer #3

$$\rightarrow \begin{bmatrix} w^3_{11} \\ w^3_{21} \end{bmatrix}^T \begin{bmatrix} O_{21} \\ O_{22} \end{bmatrix} + \begin{bmatrix} b_{31} \end{bmatrix}$$

1×1

2×1 → 1×2 | 2×1 | 1×1

$$= \sigma\left(\begin{bmatrix} w^3_{11}O_{21} + w^3_{21}O_{22} + b_{31} \end{bmatrix}\right) = \hat{y}_1 = \begin{bmatrix} a_{31} \end{bmatrix} \quad a^{[3]}$$

What is Loss Function?

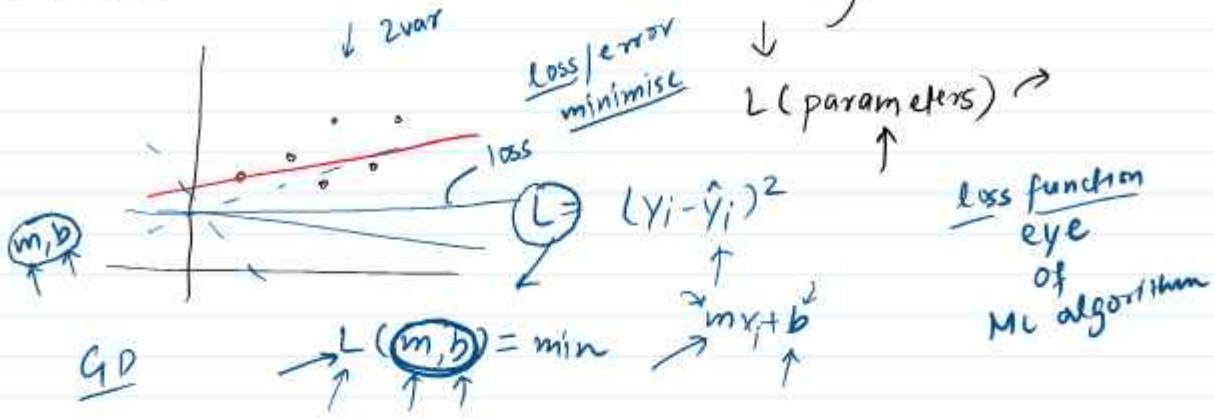
23 March 2022 07:15

Loss function is a method of evaluating how well your algorithm is modelling your dataset.

high → poor

small → great

$f(x) = x^2 + 2$



Why is Loss function important?

[You can't improve what you can't measure.]

Peter Drucker

Loss Function in Deep Learning?

multiple epochs

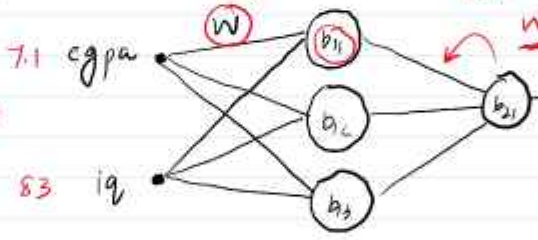
Backprop

random w, b → gradient descent

w, b

✓ cgpa | iq | package(lpa)

7.1	83	3.2
8.5	91	4.5
6.3	102	6.1
5.1	87	2.7

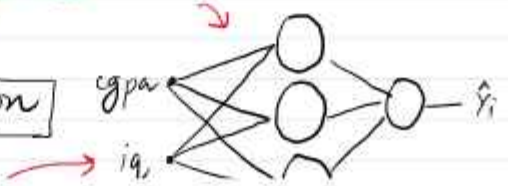


Keras

Loss functions in DL

- Object detection: focal loss
- Regression: mse, mae, huber loss
- Classification: binary crossentropy, categorical cross entro, hinge loss
- Autoencoders: KL divergence
- GAN: discriminator loss, min max gan loss
- Embedding: Triplet loss

Loss Function Vs Cost Function



Loss Function Vs Cost Function



Error function

Loss function → single training eg.

$y_i = 6.3 \quad \hat{y}_i = 6.1$

$(y_i - \hat{y}_i)^2$

$(6.3 - 6.1)^2 =$

batch Cost function

$\frac{1}{4}[(6.1-6.3)^2 + (4.1-4)^2 + (3.5-3.7)^2 + (7.2-7)^2] = CF$

$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$

(5) lpa $\sqrt{5}$ lpa $(lpa)^2$

cgpa	iq	y_i) package	$\hat{y}_i$ Prediction	$(y_i - \hat{y}_i)^2$ error
6.3	100	6.3	6.1	0.2
7.1	91	4.1	4	0.1
8.5	83	3.5	3.7	−0.2
9.2	102	7.2	7	0.2
8.1	100	50	8	Overall error

1. Mean Squared Error (MSE)

Squared loss L2 Loss

$(y_i - \hat{y}_i)^2$

(true − predict)^2

$(6.3-6.1)^2 = -$

$(y_i - \hat{y}_i)^2$

Advan — DSAdv

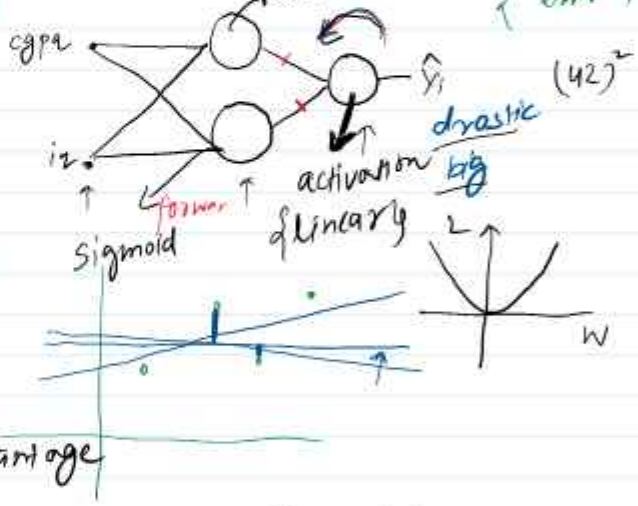
mae → $(y_i - \hat{y}_i)^2$ ← quadratic $\wedge 2$ punish



1 unit → 1 unit
2 unit → 4 unit
4 unit → 16 unit

$(y_i - \hat{y}_i)^2$

$(w, b) \rightarrow$ (L) min



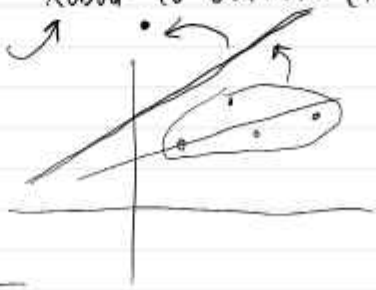
Advantages

1) Easy to interpret
2) Differentiable (GD)
3) 1 local minima

Disadvantage

1) Error unit (squared) → diff
2) Robust to Outliers (Not)

$CF = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$



mse

2. Mean Absolute Error (MAE) → L1 loss

2 lpa

cgpa | iq | package | $|(y_i - \hat{y}_i)|$ ↳ lpa

$L = |y_i - \hat{y}_i|$

2 | −abs

$C = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|^2$ | true − predict |

punish

$$C = \frac{1}{n}\sum_{i=1}^{n} |y_i - y_i|^2$$ (true - predict)

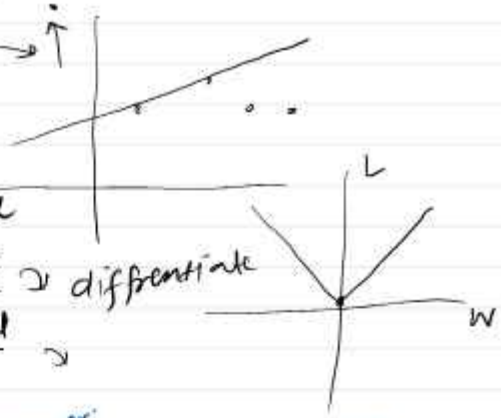
punish

Advantages

1) Intuitive and easy
2) Unit → same - y
3) Robust to outliers

Disadvantage

1) Not differentiable
   - GD → differentiate
   - Subgradient



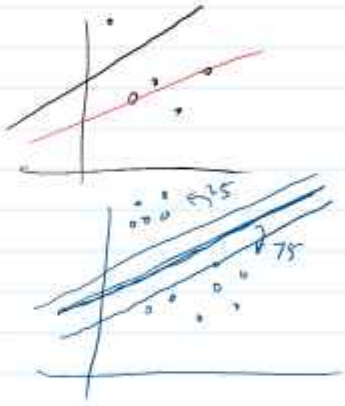
mae huber mse

3. Huber Loss ✓

$$L = \begin{cases} \frac{1}{2}(y-\hat{y})^2 & \text{for } |y-\hat{y}| \le \delta \\ \delta|y-\hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

mse, hyper, mae

mse – outliers ✓
mae – normal point ✓

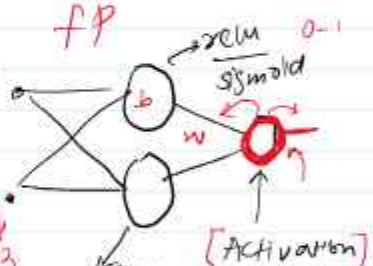


4. Binary Cross Entropy (log loss)

→ classification ✓
→ Two classes ✓

cgpa	iq	placement	100 days	10 GD	fp
8	80	1	0.73		
7	70	0	0.25		
6	60	0	0.13		

0.12



Loss function = $-y\log(\hat{y}) - (1-y)\log(1-\hat{y})$

Keras

y → actual value / target
$\hat{y}$ – NN prediction

$-(1-0)\log(1-0.25)$ $\quad -1\log(0.75)$ $\quad -0.12$

$-1\log(0.73)$

$-1 \times -0.13 = 0.13$

Cost function $= -\frac{1}{n}\left[\sum_{i=1}^{n} y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$

maximum likelihood

Logistic Reg

Advantage

* Differentiable
  - → multi local minima
  - → intuitive

Disadvan

5. Categorical Cross Entropy [used in Softmax Regression]

→ Multi-class classification ✓

cgpa	iq	placed?	OHE: Yes	No	Maybe

5. Categorical Cross Entropy [used in softmax Regression]

→ [Multi-class] classification

1 point

$$L = -\sum_{j=1}^{K} y_j \log(\hat{y}_j)$$

cgpa	iq	placed?
8	80	Yes 1
6	60	No 2
7	70	Maybe 3

Yes	No	Maybe	
1	0	0	Yes
0	1	0	No
0	0	1	Maybe

where K is # classes in the data

1 point

$$L = -(y_1 \log(\hat{y}_1) + y_2 \log(\hat{y}_2) + y_3 \log(\hat{y}_3))$$

[0.2 0.3 0.5]

[1 0 0]

$L = -1 \times \log(0.2)$

GD, WB

8, 6, 60, 80

forward prop

Activation

neurons

class ← categories

z_1 0.2 0-1 Yes 0.3

z_2 0.3 0-1 No 0.1

0.5 0-1 Maybe 0.1

$$f(z) = \frac{e^{z_1}}{e^{z_1}+e^{z_2}+e^{z_3}}$$

softmax

$$\frac{e^{z_3}}{e^{z_1}+e^{z_2}+e^{z_3}}$$

$$\frac{e^{z_2}}{e^{z_1}+e^{z_2}+e^{z_3}}$$

[0.3 0.6 0.1]

[0 1 0]

$-1 \times \log(0.6) =$ —

Sparse Categorical Cross Entropy

	cgpa	iq	placed	OHE
1/2/3	7	70	Yes	1
	8	80	No	2
	6	60	Maybe	3

[0.1 0.4 0.5]

L → $-1 \times \log(0.1)$

$-1 \times \log(0.4)$

$-1 \times \log(0.2)$

[0.1 0.4 0.6]

[0.6 0.2 0.2]

{SCE, CCE} Loss function

SCE → fast

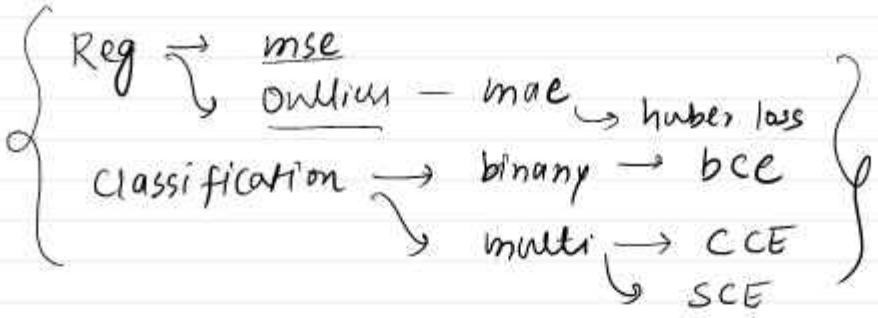
fast

OHE — fast

SCE

$$L = -\sum_{j=1}^{K} y_j \log(\hat{y}_j)$$ → Cost function

$$C = -\frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{K} y_{ij} \log(\hat{y}_{ij})$$


Reg
mse
outliers
mae
huber loss
classification
binary
bce
multi
CCE
SCE

23 March 2022 07:31

Backpropogation

Pre requisite

→ Gradient descent (v) →
→ Forward Propogation (v) →

Backprop → Algo → train nn (How)

training

cgpa	iq	lpa
8	80	8
7	70	7
?	?	?

weights, bias → correct value

predict

(W, b)

$\sigma(z)$

$O_{21} = W^2_{11} O_{11} + W^2_{21} O_{12} + b_{21}$

6 + 3 → 9 trainable

Backprop 4 students

iq	cgpa	lpa	error
80	8	3	225
60	9	5	
70	5	8	
120	7	11	

activation = linear

Linear

$\hat{y} = O_{21}$ (W, b) Random val

$W \to 1, b \to 0$

L min → $\hat{y} \to O_{21}$

Steps → 0) Init W, b
1) You select a point (row) → student → 18 lpa
2) Predict (lpa) → forward prop [not predicted]
3) Choose a loss function (mse) $L = (y - \hat{y})^2$ $(3-18)^2 = 225$ error
4) Weights and bias update → Gradient descent

Hierarchy: Backprop

$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$

$W^2_{11\,new} = W^2_{11\,old} - \eta \frac{\partial L}{\partial W^2_{11}}$ (01) 9 derivatives

$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b_{old}}$

$W^2_{21\,new} = W^2_{21\,old} - \eta \frac{\partial L}{\partial W^2_{21}}$

$b_{21\,new} = b_{21\,old} - \eta \frac{\partial L}{\partial b_{21}}$

derivative of loss wrt weight

$\frac{\partial L}{\partial W^2_{11}}$ partial derivative

derivative

$\frac{\partial L}{\partial W^2_{11}}, \frac{\partial L}{\partial W^2_{21}}, \frac{\partial L}{\partial b_{21}}$ | $\frac{\partial L}{\partial W^1_{11}}, \frac{\partial L}{\partial W^1_{21}}, \frac{\partial L}{\partial b_{11}}$ | $\frac{\partial L}{\partial W^1_{12}}, \frac{\partial L}{\partial W^1_{22}}, \frac{\partial L}{\partial b_{12}}$

$\hat{y} \to L$ (min)

$\frac{\partial L}{\partial W^2_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial W^2_{11}}$ → chain rule of diffrn

$\frac{dy}{dx}$

$\frac{\partial L}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} (y - \hat{y})^2 = -2(y - \hat{y})$

$\frac{\partial \hat{y}}{\partial W^2_{11}} = \frac{\partial}{\partial W^2_{11}} [O_{11} W^2_{11} + O_{12} W^2_{21} + b_{21}]$

$= O_{11}$

$\frac{\partial L}{\partial W^2_{11}} = -2(y - \hat{y}) O_{11}$

$\frac{\partial L}{\partial W^2_{21}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial W^2_{21}}$

$\frac{\partial \hat{y}}{\partial W^2_{21}} = \frac{\partial}{\partial W^2_{21}} [O_{11} W^1_{11} + O_{12} W^1_{21} + b_{21}]$

$= O_{12}$

$\frac{\partial L}{\partial W^2_{21}} = -2(y - \hat{y}) O_{12}$

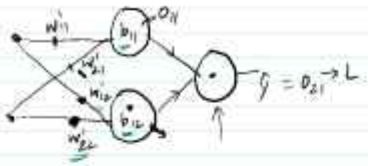
$\hat{y} \to b$

$\frac{\partial \hat{y}}{\partial b_{21}} = 1$

$$\frac{\partial L}{\partial w_{21}^2} = -2(y-\hat{y})\,o_{12}$$ 2

$$\frac{\partial L}{\partial b_{21}} = -2(y-\hat{y})$$ 3

$$\frac{\partial L}{\partial b_{21}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial b_{21}}, \qquad \frac{\partial \hat{y}}{\partial b_{21}} = 1$$



$$\frac{\partial L}{\partial w_{11}^1} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial o_{11}}\frac{\partial o_{11}}{\partial w_{11}^1} \rightarrow -2(y-\hat{y})\,w_{11}^2\,x_{i1}$$ 4

$$\frac{\partial L}{\partial w_{21}^1} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial o_{11}}\frac{\partial o_{11}}{\partial w_{21}^1} \rightarrow -2(y-\hat{y})\,w_{11}^2\,x_{i2}$$ 5

$$\frac{\partial L}{\partial b_{11}} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial o_{11}}\frac{\partial o_{11}}{\partial b_{11}} \rightarrow -2(y-\hat{y})\,w_{11}^2$$ 6

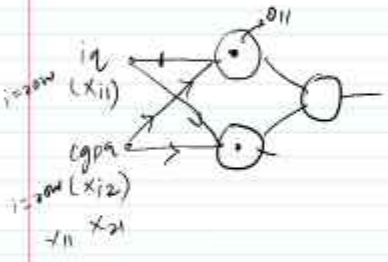


$$\frac{\partial L}{\partial w_{12}^1} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial o_{12}}\frac{\partial o_{12}}{\partial w_{12}^1} \rightarrow -2(y-\hat{y})\,w_{21}^2\,x_{i1}$$ 7

$$\frac{\partial L}{\partial w_{22}^1} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial o_{12}}\frac{\partial o_{12}}{\partial w_{22}^1} \rightarrow -2(y-\hat{y})\,w_{21}^2\,x_{i2}$$ 8

$$\frac{\partial L}{\partial b_{12}} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial o_{12}}\frac{\partial o_{12}}{\partial b_{12}} \rightarrow -2(y-\hat{y})\,w_{21}^2$$ 9

$\frac{\partial \hat{y}}{\partial o_{12}} = w_{21}^2$



$$\frac{\partial o_{11}}{\partial w_{11}^1} = \frac{\partial}{\partial w_{11}^1}\left[iq\,w_{11}^1 + cgpa\,w_{21}^1 + b_{11}\right] = (iq) = x_{i1}$$

$$\frac{\partial o_{11}}{\partial w_{21}^1} = x_{i2}$$

$$\frac{\partial o_{11}}{\partial b_{11}} = 1$$

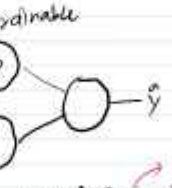
$$\frac{\partial o_{12}}{\partial w_{12}^1} = \frac{\partial}{\partial w_{12}^1}\left[iq\,w_{12}^1 + cgpa\,w_{22}^1 + b_{12}\right] = iq\,(x_{i1})$$

$$\frac{\partial o_{12}}{\partial w_{22}^1} = x_{i2}$$

$$\frac{\partial o_{12}}{\partial b_{12}} = 1$$

multiple chances

cgpa	iq	lpa $\hat{y}$
8	80	8
6	60	6
7	70	7
9	90	



$(y-\hat{y})^2$ → Backprop

epochs

Steps (once again)

0) weights/bias → init (random: w=1, b=0)

Loop - 100/120 times

1) for i in range(q):

1a) 1 student → forward prop → lpa) → /2 min

1b) Loss calculate (mse) →

1c) Adjust all weights and bias ↓.

4D → 9 times

$$w_{new} = w_{old} - \eta\left[\frac{\partial L}{\partial w_{old}}\right]$$

The How

Back propagation Algorithm

epochs = 5

for i in range(epochs):

    for j in range(X.shape[0]):

$\frac{\partial L}{\partial w_{11}^2} = -2(y-\hat{y})\,o_{11}$ Rough

$\frac{\partial L}{\partial w_{21}^2} = -2(y-\hat{y})\,o_{12}$

```
for i in range(epochs):
    for j in range(X.shape[0]):
        → Select 1 row (random)
        → Predict (using Forward propp)
        → Calculate loss (using Loss function → mse)
        ↑
        → Update weights and bias using GD
                W_n = W_0 - η ∂L/∂W
    → Calculate avg loss for the epoch
      Loss
```

$$\frac{\partial L}{\partial W^2_{21}} = -2(y-\hat{y})\,o_{12}$$

$$\frac{\partial L}{\partial b_{21}} = -2(y-\hat{y})$$

$$\frac{\partial L}{\partial W^1_{11}} = -2(y-\hat{y})\,W^2_{11}\,x_{i1}$$

$$\frac{\partial L}{\partial W^1_{21}} = -2(y-\hat{y})\,W^2_{11}\,x_{i2}$$

$$\frac{\partial L}{\partial b_{11}} = -2(y-\hat{y})\,W^2_{11}$$

$$\frac{\partial L}{\partial W^1_{12}} = -2(y-\hat{y})\,W^2_{21}\,x_{i1}$$

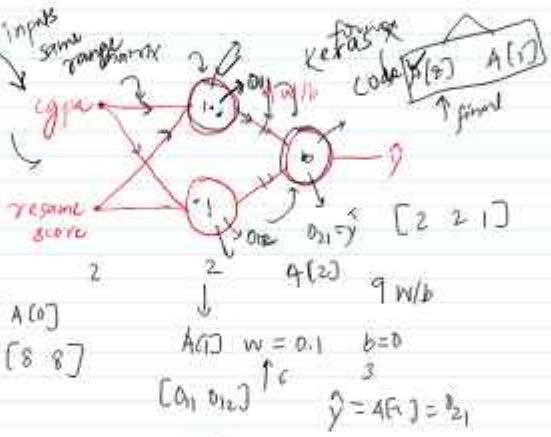
$$\frac{\partial L}{\partial W^1_{22}} = -2(y-\hat{y})\,W^2_{21}\,x_{i2}$$

$$\frac{\partial L}{\partial b_{12}} = -2(y-\hat{y})\,W^2_{21}$$

Regression Problem

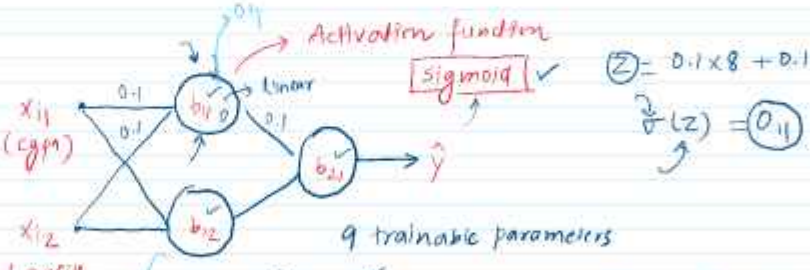
cgpa (X_{i1})	resume score (X_{i2})	package (lpa)
8	8	4
7	9	5
6	10	6
5	12	7

4 students



Classification Example

cgpa	profile score	placement
8	8	1
7	9	1
6	10	0
5	5	0



$$z = 0.1\times 8 + 0.1\times 8 + 0$$

$$\sigma(z) = o_{11}$$

Backpropagation Algorithm

```
epochs = 5
for i in range(epochs):  (rows)
    for j in range(X.shape[0]):
        → Select 1 row (random) → 1by1
        → Predict (using Forward propp) → predict
        → Calculate loss (using Loss function → bce) → {code convert}
        ↑
        → Update weights and bias using GD
```

Back propagation Algorithm

Cross entropy

epochs = 5 ✓

for i in range(epochs): (rows)

    for j in range(X.shape[0]):

        → select 1 row (random) → 1by1

        → Predict (using forward prop) → predict

        → Calculate loss (using loss function → ~~mse~~ bce) → code convert

        → Update weights and bias using GD

$$W_n = W_0 - \eta \left(\frac{\partial L}{\partial W}\right)$$ q w, b update

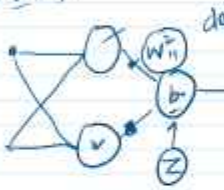
→ Calculate avg loss for the epoch

Loss

$z = w_{11}^2 o_{11} + w_{21}^2 o_{12} + b_{21}$

$\hat{y} = \sigma(z)$

$L = -y \log(\hat{y}) - (1-y)\log(1-\hat{y})$



der wrt L

$\hat{y} \to L$

$$\frac{\partial L}{\partial w_{11}^2} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w_{11}^2} = -(y-\hat{y})\, o_{11} \quad (3)$$

$$\frac{\partial L}{\partial w_{21}^2} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w_{21}^2} = -(y-\hat{y})\, o_{12}$$

$z_{final} = w_{11}^2 o_{11} + w_{21}^2 o_{12} + b_{21}$

$$\frac{\partial L}{\partial b_{21}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial b_{21}} = -(y-\hat{y})$$

$\hat{y} \to z_{final} \to w_{11}^2 \times w_{21}^2 \to L$

$$\frac{\partial L}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}}\left[-y\log(\hat{y}) - (1-y)\log(1-\hat{y})\right]$$

$$= -\frac{y}{\hat{y}} + \frac{(1-y)}{(1-\hat{y})} = \frac{-y(1-\hat{y}) + \hat{y}(1-y)}{\hat{y}(1-\hat{y})} = \frac{-y + y\hat{y} + \hat{y} - y\hat{y}}{\hat{y}(1-\hat{y})}$$

$$= \boxed{\frac{-(y-\hat{y})}{\hat{y}(1-\hat{y})}}$$

$\frac{\partial}{\partial z}\sigma(z) = \sigma(z)[1-\sigma(z)]$

$\hat{y} = \sigma(z)$

$\sigma(z) = \hat{y}$

$$\frac{\partial \hat{y}}{\partial z} = \frac{\partial}{\partial z}(\sigma(z)) = \sigma(z)[1-\sigma(z)] = \hat{y}(1-\hat{y})$$

$$\frac{\partial L}{\partial \hat{y}} = \frac{-(y-\hat{y})}{\hat{y}(1-\hat{y})} \qquad \frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y}) \Rightarrow \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} = \frac{-(y-\hat{y})}{\hat{y}(1-\hat{y})} \times \hat{y}(1-\hat{y})$$

$= -(y-\hat{y})$

$z_p = w_{11}^1 x_{11} + w_{21}^1 x_{12} + b_{11}$

$\frac{\partial z_p}{\partial w_{11}^1}$



$\frac{\partial L}{\partial w_{11}^1}, \frac{\partial L}{\partial w_{21}^1}, \frac{\partial L}{\partial b_{11}}$

$\to -(y-\hat{y})$

$z_f = w_{11}^2 o_{11} + w_{21}^2 o_{12} + b_{21}\, o_{11}(1-o_{11})$

$o_{11} = \sigma(z_{prev})$!



$\frac{\partial L}{\partial w^1_{11}}$, $\frac{\partial L}{\partial w^1_{21}}$, $\frac{\partial L}{\partial b_{11}}$

$z_f = w^2_{11} o_{11} + w^2_{21} o_{12} + b_{21}$ $o_{11}(1-o_{11})$

$o_{11} = \sigma(z_{prev})$

$= \sigma(z_{prev})[1-\sigma(z_{prev})]$

$L \to \hat{y} \to z_f \to o_{11} \to z_{prev} \to w^1_{11}$

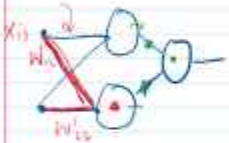
$$\frac{\partial L}{\partial w^1_{11}} = \boxed{\frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z_f}} \times \frac{\partial z_f}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial z_{prev}} \times \frac{\partial z_{prev}}{\partial w^1_{11}}$$

$\frac{\partial L}{\partial \hat{y}} = -(y-\hat{y})$; $\frac{\partial z_f}{\partial o_{11}} = w^2_{11}$; $\frac{\partial o_{11}}{\partial z_{prev}} = o_{11}(1-o_{11})$; $\frac{\partial z_{prev}}{\partial w^1_{11}} = x_{i1}$

$$\boxed{\frac{\partial L}{\partial w^1_{11}} = -(y-\hat{y}) w^2_{11} o_{11}(1-o_{11}) x_{i1}} \quad (3)$$

$$\boxed{\frac{\partial L}{\partial w^1_{21}} = -(y-\hat{y}) w^2_{11} o_{11}(1-o_{11}) x_{i2}}$$

$$\boxed{\frac{\partial L}{\partial b_{11}} = -(y-\hat{y}) w^2_{11} o_{11}(1-o_{11})}$$



$\frac{\partial L}{\partial w^1_{12}}$, $\frac{\partial L}{\partial w^1_{22}}$, $\frac{\partial L}{\partial b_{12}}$

$z_f = w^2_{11} o_{11} + w^2_{21} o_{12} + b_{21}$

$\frac{\partial z_f}{\partial o_{12}}$

$$\frac{\partial L}{\partial w^1_{12}} = \boxed{\frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z_f}} \times \frac{\partial z_f}{\partial o_{12}} \times \frac{\partial o_{12}}{\partial z_p} \times \frac{\partial z_p}{\partial w^1_{12}}$$

$L \to \hat{y} \to z_f \to o_{12} \to w^1_{12} \leftarrow z_p$

$\frac{\partial z_f}{\partial o_{12}} = w^2_{21}$; $\frac{\partial o_{12}}{\partial z_p} = o_{12}(1-o_{12})$; $\frac{\partial z_p}{\partial w^1_{12}} = x_{i1}$

$$\boxed{\frac{\partial L}{\partial w^1_{12}} = -(y-\hat{y}) w^2_{21} o_{12}(1-o_{12}) x_{i1}} \quad (3)$$

$$\boxed{\frac{\partial L}{\partial w^1_{22}} = -(y-\hat{y}) w^2_{21} o_{12}(1-o_{12}) x_{i2}}$$

$$\boxed{\frac{\partial L}{\partial b_{12}} = -(y-\hat{y}) w^2_{21} o_{12}(1-o_{12})}$$

Backpropagation → The why?

The intuition behind the algorithm ✓

Back propagation Algorithm

epochs = 5 ✓ # epochs × rows

```
for i in range(epochs):
    for j in range(X.shape[0]):
        -> select 1 row (random)
        -> Predict (using Forward prop)        ŷ  y
        -> Calculate loss (using loss function -> mse)   bce
        -> Update weights and bias using GD
            -> w_n = w_0 - η ∂L/∂w   rule
               (magic)
    -> Calculate avg loss for the epoch
       Loss
```

→ Loss function is a function of all trainable parameters

data → target

$L = (y - \hat{y})^2$

cgpa | profile | lpa

$mse = (y-\hat{y})^2$

$L = (y - \hat{y})^2$ → mathematical function

constant variable

$L(?)$ → $L(\hat{y}) =$ $y = f(x)$ something

$L(w, b)$ ↓

9 things → w, b

$\hat{y} = w_{11}^2 o_{11} + w_{21}^2 o_{12} + b_{21}$

$\hat{y} = w_{11}^2 [w_{11}^1 x_{11} + w_{21}^1 x_{12} + b_{11}] + w_{21}^2 [w_{12}^1 x_{11} + w_{22}^1 x_{12} + b_{12}] + b_{12}$

→ Concept of Gradient → Gradient Descent

↳ fancy word for derivative

$y = f(x) = x^2 + x$

$\frac{dy}{dx} = \frac{d}{dx}(fx) = \frac{d}{dx}(x^2 + x) = 2x + 1$

derivative

$y \to x \to$ derivative $\frac{d}{dx}$

$z = f(x, y) = x^2 + y^2$

$\frac{\partial z}{\partial x} = 2x$ $\frac{\partial z}{\partial y} = 2y$

gradient $\frac{\partial}{\partial x}$

gradient complex → $\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$

9 params

$L(w_{11}^1, w_{12}^1 \ldots w_{21}^2, b_{21}, b_{11}, b_{12})$

↳ 9D function → 9 diff slopes w.r.t each dim

3D → $z = x^2 + y^2$ $z = f(x, y)$

$y = f(x) = x^2$

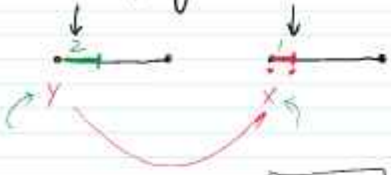


→ Concept of Derivative → Derivative at a point

↑ intuition

$\frac{dy}{dx}$ → (y) func x → (x)

rate of change ↑



$\frac{dy}{dx} = 2$ (+ve) ↓ sign (mag)

$\frac{dy}{dx} = -2$ (-ve)

$x \rightarrow$

$y \rightarrow$

derivative

$y = x^2 + 2x$ $\quad x=5$ → derivative

$\frac{dy}{dx} = (2x+1)_{x=5}$

$\left(\frac{dy}{dx}\right)_{x=5} = 11$ → slope

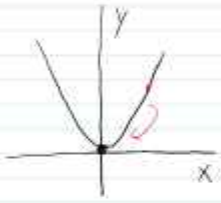
$\frac{\partial L}{\partial w_{11}^1}$

w_{11}^1 — 1 unit

$L \rightarrow ?$ (mag) (sign)

$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$

→ the concept of minima



$y = x^2$

$\frac{dy}{dx} = 2x = 0$

$x = 0$

$z = x^2 + y^2$

$(x,y) \rightarrow z$ min



$\frac{\partial z}{\partial x} = 0$ $x \rightarrow$ $\quad 2x = 0 \Rightarrow x = 0$

$\frac{\partial z}{\partial y} = 0$ $y \rightarrow$ $\quad 2y = 0 \Rightarrow y = 0$

L (q param) → (w, b) → min

L ↓ ↕ $\quad \frac{\partial L}{\partial w_{11}^1} \cdots \frac{\partial L}{\partial b_{12}} = 0$

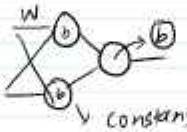
q dim → minima

→ Backprop Intuition

$\eta = 1$

$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$ ← q step

$w_{new} = w_{old} - \frac{\partial L}{\partial w}$



each parameters

$L = L(w,b)$ → q → constant

$L = (y - \hat{y})^2$ → L

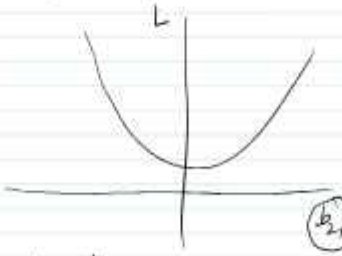
$L(b_{21})$ $\quad \hat{y} = (b_{21})$

$L(b_{21})$ $\quad L = b_{21}^2$

$b_{21} = b_{21} + \frac{\partial L}{\partial b_{21}}$ $\quad b_{21} = 5$

+ve

-ve



$\frac{\partial L}{\partial b_{21}}$ smart

L derivative wrt b_{21}

↑ ? $\quad$ ↑ 1 unit

$L \downarrow$ → $b_{21} \downarrow$

$b_{21} \downarrow$

$L \downarrow$ $\quad \frac{\partial L}{\partial b_{21}} = +ve$ → $b_{21} \uparrow$ $L \uparrow$

$b_{21} \uparrow$

$\frac{\partial L}{\partial b_{21}} = -ve$ $\quad b_{21} \uparrow$ $L \downarrow$

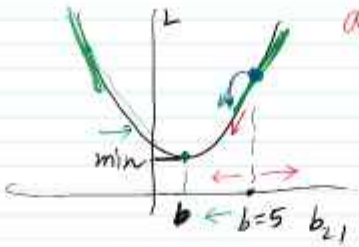
$b_{21} \uparrow$

$b_{21} = b_{21} - \frac{\partial L}{\partial b_{21}}$

game ... naive $\quad$ -ve of the gradient

game derivative (b_{21}) — -ve of the gradient

$$b_{21} = b_{21} - \frac{\partial L}{\partial b_{21}}$$



algo

$b = -5$

$\frac{\partial L}{\partial b}$ = slope

+ve

$\frac{\partial L}{\partial b}$ = -ve

-ve

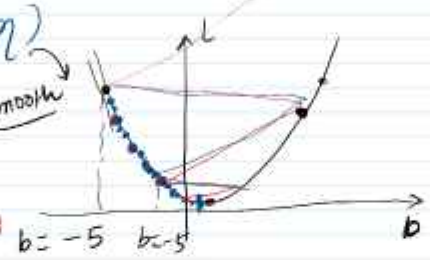
slope (gradient) b -ve

Slope

$$w_n = w_0 - \frac{\partial L}{\partial w} \quad \eta \quad w_n = w_0 - \eta \frac{\partial L}{\partial w}$$

→ Effect of Learning Rate (η)

$$w_n = w_0 - \eta \frac{\partial L}{\partial w}$$



smooth

0.1/0.01

train

$\eta = 1$

η = parameter ↓ 0.01

$\eta = 0.00001$

$\frac{\partial L}{\partial b} = -50$

$b = -5 - (-50) = 45$

$b = 45$

$\frac{\partial L}{\partial b} = 50$

$b = 45 - 50$

$b = -5 \quad \frac{\partial L}{\partial b} = -50 \quad \eta = 0.01$

$b = -5 + (0.01 \times 50) = 0.5$

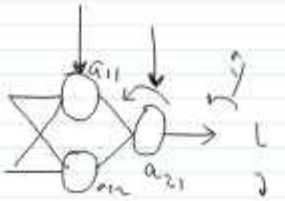
$= -4.5$

→ What is convergence?

$w_n = w_0 - \eta \frac{\partial L}{\partial w} = 0$

$w_n = w_0$

while → con → for epoch = 100 for 1000



$i = 1, 0$

$-(y - \hat{y})$

activation = ②

$\hat{y}(1 - \hat{y})\, o_{11}$

$-(y - \hat{y})\, o_{11}$

$-y \log(a_{21}) - (1-y) \log(1 - a_{21})$

$$\frac{\partial L}{\partial w_{11}^2} = \frac{\partial L}{\partial a_{21}} \times \frac{\partial a_{21}}{\partial w_{11}^2}$$

$$-\frac{y}{\hat{y}} + \frac{(1-y)}{(1-\hat{y})}$$

$\frac{\partial L}{\partial} =$

$$-\frac{y}{\hat{y}} + \frac{(1-y)}{(1-\hat{y})}$$

$$\frac{\partial L}{\partial w^2_{21}} =$$

curr_activa = a[1]

$$\frac{\partial L}{\partial b_{21}} =$$

$$\begin{cases} -(y-\hat{y})\ a_{21}(1-a_{21})\ a_{11} \\ -(y-\hat{y})\ a_{21}(1-a_{21})\ a_{12} \\ -(y-\hat{y})\ a_{21}(1-a_{21}) \end{cases}$$

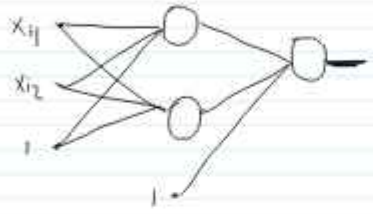
$$\text{delta} = (y-\hat{y})\ a^{(2)}(1-a^{(2)})$$

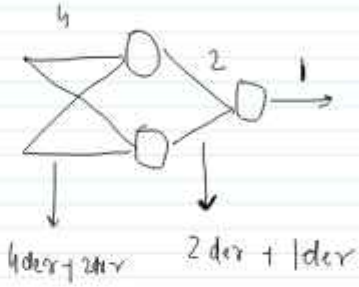
(1×1) (1×2)

W[1]

(1×2)

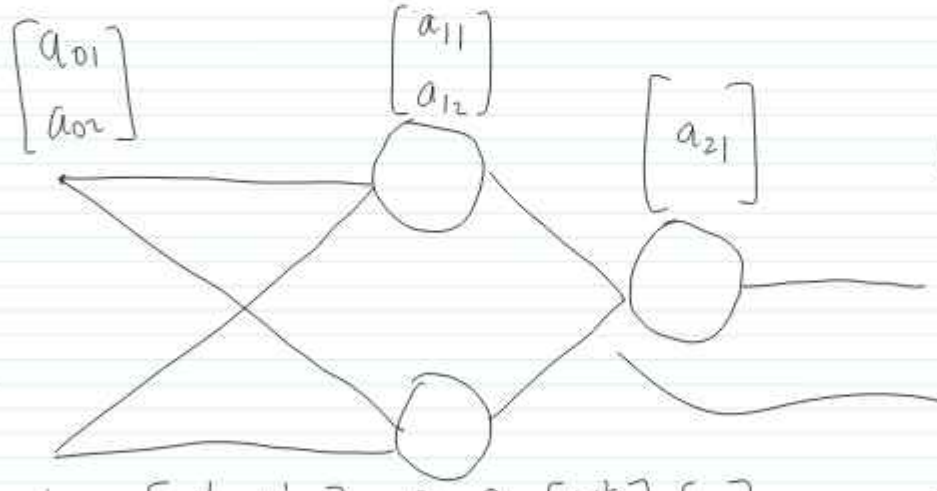
$$-(y-\hat{y})\ a_{21}(1-a_{21}) \times \begin{bmatrix} a_{11} \\ a_{12} \end{bmatrix} \quad a[1]$$





(1×1) (1×2)

(1×2) × (2×2)



L → a_2 → a_1 →

$$\frac{\partial L}{\partial a_2} \times \frac{\partial a_2}{\partial a_1} \frac{\partial a_1}{\partial w}$$

$$\begin{bmatrix} w^1_{11} & w^1_{12} \\ w^1_{21} & w^1_{22} \end{bmatrix} \quad \begin{bmatrix} b_{11} \\ b_{12} \end{bmatrix} \quad \begin{bmatrix} w^2_{11} \\ w^2_{12} \end{bmatrix} \quad \begin{bmatrix} b_{21} \end{bmatrix}$$

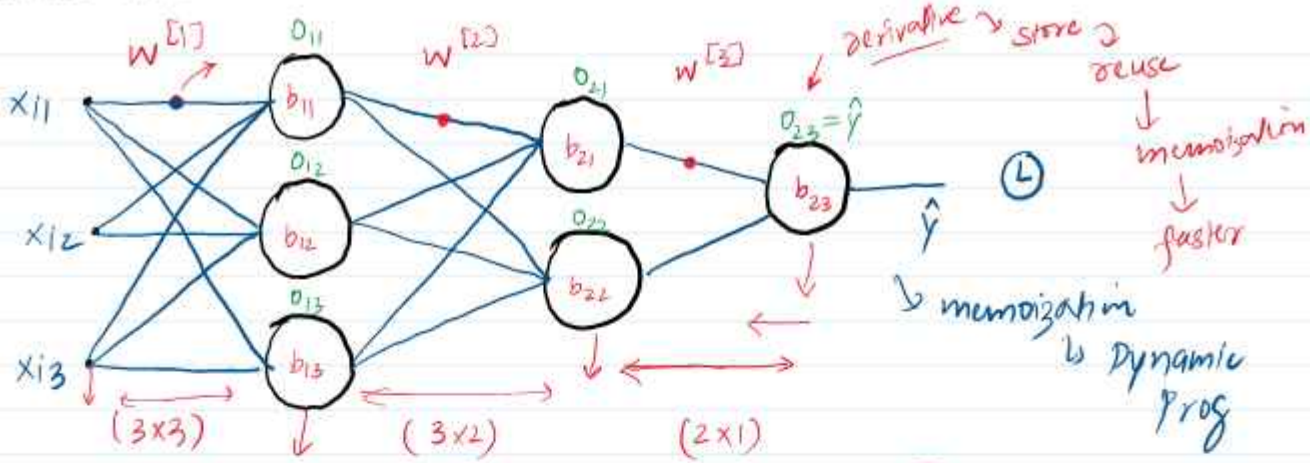
$$\frac{\partial L}{\partial w} = \left[\frac{\partial L}{\partial a_2}\right] \times \left[\frac{\partial a_2}{\partial w}\right]$$

$$\begin{bmatrix} \partial L/\partial w^1_{11} & \frac{\partial L}{\partial w^1_{12}} \\ \frac{\partial L}{\partial w^1_{21}} & \frac{\partial L}{\partial w^1_{22}} \end{bmatrix} \qquad \begin{bmatrix} \frac{\partial L}{\partial w^2_{11}} \\ \frac{\partial L}{\partial w^2_{12}} \end{bmatrix}$$

MLP Memoization

Thursday, April 7, 2022 7:45 AM

backprop update → w / b) deriv



derivative → store → reuse → memoization → faster

↳ memoization
↳ Dynamic Prog

Chain rule

$L \to \hat{y} \to o_{21} \to o_{11} \to w^1_{11}$

$$\frac{\partial L}{\partial w^3_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^3_{11}} \qquad \frac{\partial L}{\partial w^2_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial w^2_{11}}$$

$$x \to f(x),\ g(x) \to h(f(x), g(x)) \qquad \frac{\partial h}{\partial x} = \left[\frac{\partial h}{\partial f(x)} \times \frac{\partial f(x)}{\partial x}\right] + \left[\frac{\partial h}{\partial g(x)} \times \frac{\partial g(x)}{\partial x}\right]$$



$$\left[\frac{\partial L}{\partial w^1_{11}} = \frac{\partial L}{\partial \hat{y}}\left[\frac{\partial \hat{y}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w^1_{11}} + \frac{\partial \hat{y}}{\partial o_{22}} \times \frac{\partial o_{22}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w^1_{11}}\right]\right]$$

Backprop → Chain diff (rule) + Memorization

Keras TF

maths

(15)

Gradient Descent

Thursday, April 7, 2022 7:46 AM

Gradient descent is one of the most popular algorithms to perform optimization and by far the most common way to optimize neural networks.

Gradient descent is a way to minimize an objective function $J(\theta)$ parameterized by a model's parameters $\theta \in \mathbb{R}^d$ by updating the parameters in the opposite direction of the gradient of the objective function $\nabla_\theta J(\theta)$ w.r.t. to the parameters. The learning rate η determines the size of the steps we take to reach a (local) minimum. In other words, we follow the direction of the slope of the surface created by the objective function downhill until we reach a valley.



Back propagation Algorithm

epochs = 5 ↓ std

for i in range(epochs):

  for j in range(X.shape[0]): → 1 point → gradient descent

    → select 1 row (random)

    → Predict (using Forward prop)

    → Calculate loss (using loss function → mse)

    → Update weights and bias using GD

$$w_n = w_0 - \eta \frac{\partial L}{\partial w}$$

  → Calculate avg loss for the epoch

Loss

50 points
1 epoch
↳ 50 times
w, b update

Batch GD

epoch = 5 → current weight

↳ 50 points → predict J

↑ dot product

y_hat = np.dot(X, w) + b

↳ 50 predict

y = 50 actual occ.

y y_hat J loss

$$J = -\sum_{i=1}^{50}$$

Batch / Stochastic / mini batch

There are three variants of gradient descent, which differ in how much data we use to compute the gradient of the objective function. Depending on the amount of data, we make a trade-off between the accuracy of the parameter update and the time it takes to perform an update.

$\frac{\partial L}{\partial w}$ derivative → 3 types

accuracy → time → update

1 single (w, b)

Batch GD (Vanilla GD)

```
for i in range(no_epochs):
    params_grad = evaluate_gradient(loss_function, data, params)
    params = params - learning_rate * params_grad
```

entire dataset J update

epochs = # of updates

Stochastic GD

```
for i in range(no_epochs):
    np.random.shuffle(data)
    for example in data:
        params_grad = evaluate_gradient(loss_function, example, params)
        params = params - learning_rate * params_grad
```

50 x 10 = 500

epoch → 10 (50 rows)

for i in range(10): → shuffle

  for i in range(X.shape[0]) 50 rows

10 x 50 = 500 update

↳ 1 random point → bias eliminate

↳ y_hat → forward

↳ loss

↳ w, b update → $w_n = w_0 - \eta \frac{\partial L}{\partial w}$

avg loss print → for the epoch

5 epochs
↳ 5 times
↓
w, b update

frequency of weight update higher

Batch GD → 1 loop

epoch = 10 ← epoch

for in range(10): ← loop ← rows → total → 10 times w, b update

y_hat = np.dot(X, w) + b

50 values ← dot → manual replacement → loops

↳ vectorization → faster loop

Y = 50 values

50 values dot → smart replacement → loop
Y = 50 values ↳ vectorization → faster loop
y_hat, y → loss Optimized !
{ w, b update $w_n = w_0 - \eta \frac{\partial L}{\partial w}$
→ loss

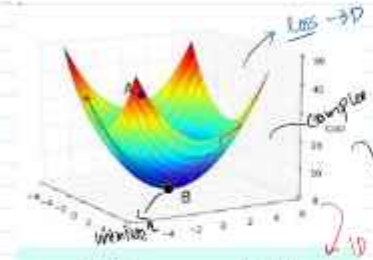
Which is faster (given same no. of epochs)

320 (m) 1

Which is the faster to converge (given some # epochs)

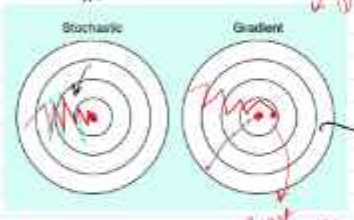
5

Loss function → 2 Algo



Stochastic → random → point → updates

Batch → dataset → all point → 1 update



Spikey SGD useful

Yes No

SGD
BGD

SGD → help the algo to move out of local minima

Local minima

Start solution

Approximate diff

faster → loops big dataset

Vectorization

np.dot (X, W) + b

X → dataset of 10 rows Add

Mini Batch Gradient Descent ← use

BGD ↔ SGD Best of both worlds

SGD → BGD

320 rows → batches → 32
In every epoch → 10 batches
10 update

$\frac{n}{x}$ = # of batches Keras batch-size = x

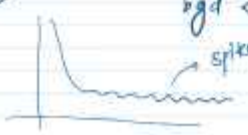
updates/epoch

for i in epochs →
for j in num of batch
1 batch
↳ y_pred / vector
↳ loss
↳ update

np.dot

bgd > mbgd > sgd
con
bgd < mbgd < sgd

MBGD



spikey SGD → Vectorization → Smaller batch

→ Why batch_size is provided in multiple of(2)?

2, 4, 8, 32, 64, 128, 256

10, 15

RAM effective → binary RAM → operations faster

→ What if batch_size doesn't divide # rows properly

e.g # of rows n = 400

batch_size = 150

of batch = $\frac{400}{150}$ = 2.66

150, 150, left 100

1 batch, 2 batch, 3rd batch

Vanishing Gradient Problem

Thursday, April 7, 2022 7:45 AM

In machine learning, the **vanishing gradient problem** is encountered when training artificial neural networks with gradient-based learning methods and backpropagation. In such methods, during each iteration of training each of the neural network's weights receives an update proportional to the partial derivative of the error function with respect to the current weight. The problem is that in some cases, the gradient will be vanishingly small, effectively preventing the weight from changing its value. In the worst case, this may completely stop the neural network from further training. ~~As one example of the problem cause, traditional activation~~

1) $0.1 \times 0.1 \times 0.1 \times 0.1 = 0.0001$ → VGP

2) Deep NN →

3) Sigmoid / tanh → Af → 0-1

Backprop

0 - 0.5

deep

sigmoid

sigmoid derivative

$L \to \hat{y} \to o_{11} \to w'_{11}$

$\hat{y} - y = L$

$w_n = w_0 - \eta \frac{\partial L}{\partial w}$ → derivative of L w.r.t weight

$w_0 = 1$ ↑ small change

$\frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w'_{11}}$

$0 < x < 1$ $0 < x < 1$ 0.1

0.0001

$w_n = 1 - 0.01 \times 0.0001$

did not change ← $w_n = 0.99999$ → vanishingly small

{ Sigmoid tanh }

$\frac{\partial L}{\partial w'_{11}} = 0.000001$

Backprop → model traing → x

80's, 90's → main reasons DL failed

0 - 100000

0 - 1

0 - 1

How to recognize?

1) Loss focus → epoch → no changes → VGP

2) weights → graph value

Tensorboard ↳ callbacks

w'_{11}

Keras

{ loss after epoch }

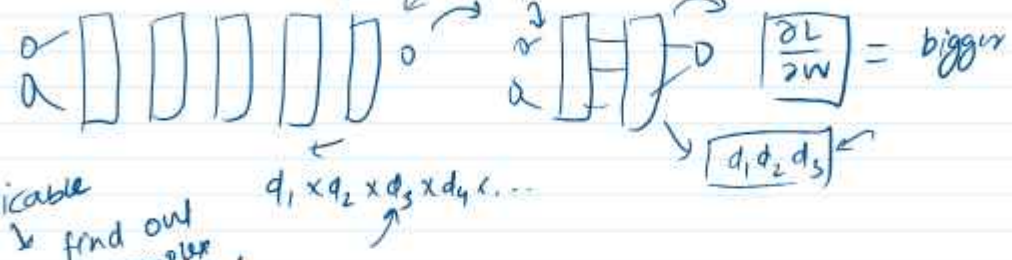
epoch

$w_n = w_0 - \eta \frac{\partial L}{\partial w}$ $\quad \frac{w_0 - w_n}{\eta} =$

$$w_n = w_0 - \eta \left[\frac{\partial L}{\partial w}\right] \qquad \left[\frac{w_0 - w_n}{\eta}\right] =$$

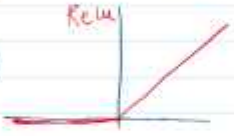
How to handle Vanishing Gradient Problem →

1) Reduce model complexity



Applicable
↳ find out complex patterns

2) Using [ReLU] Activation functions



max (0, z) → [0, 1]

{ -1000 - 1000 → squish
[0] - 1000
[0-1] }

z { 1 × 1 × 1 = 1 → VGP
dying relu → derivative $w_n = w_0$
{ leaky relu } }

3) Proper weight init → Global
↑ → xavier

4) Batch norm → layer →

5) Residual Network ← CNN → ResNET
↳ building block → ANN

Exploding Gradient Problem → [RNN]

10, 10, 10, 10 → 10000

big number

$\frac{\partial L}{\partial w_{11}^1} = d_1 \times d_1 \times d_2 \times d_3$

$w_n = w_0 - \eta \frac{\partial L}{\partial w}$ (0.1, 1000)

random

Loss↓ x

1 — 100 → (99) → 1000 →

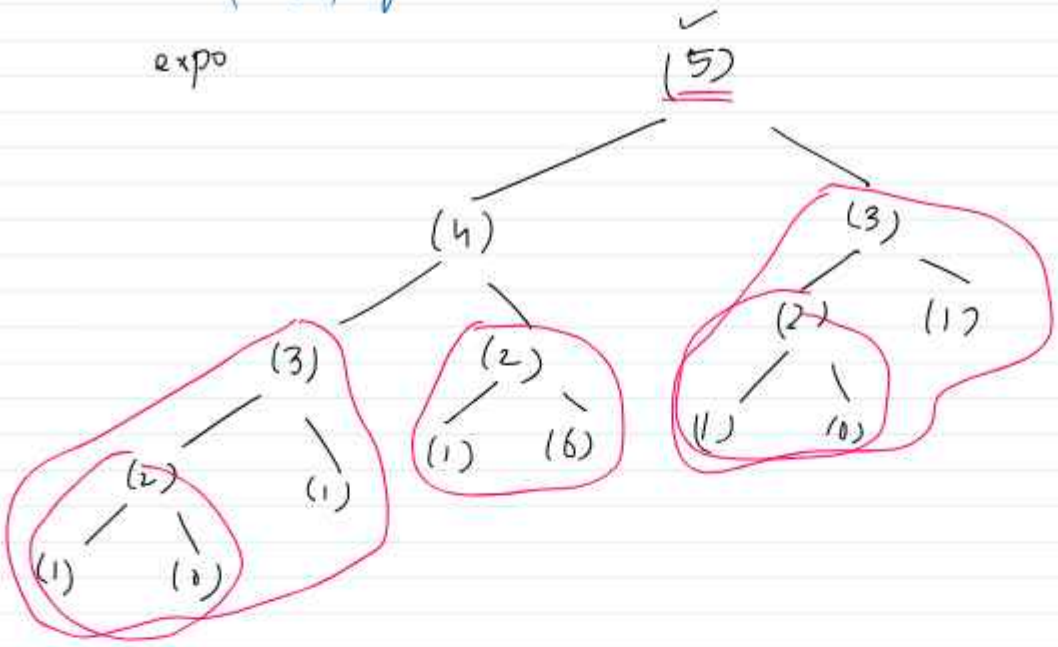
{ Gradient clipping }

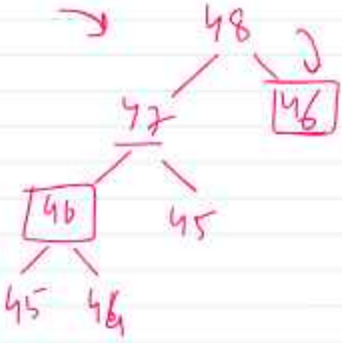
expo

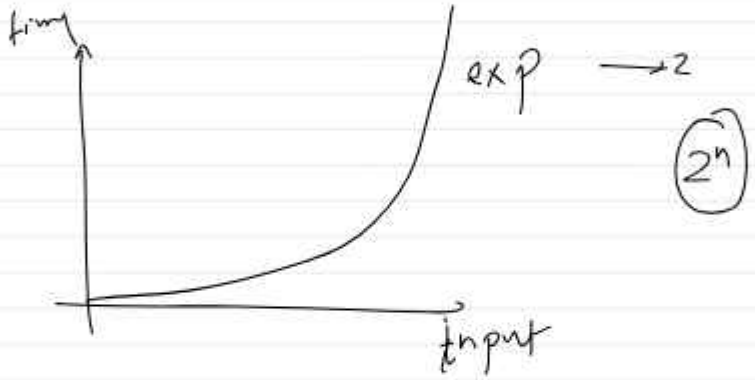
157

48

expo







Saturday, April 9, 2022 10:01 AM

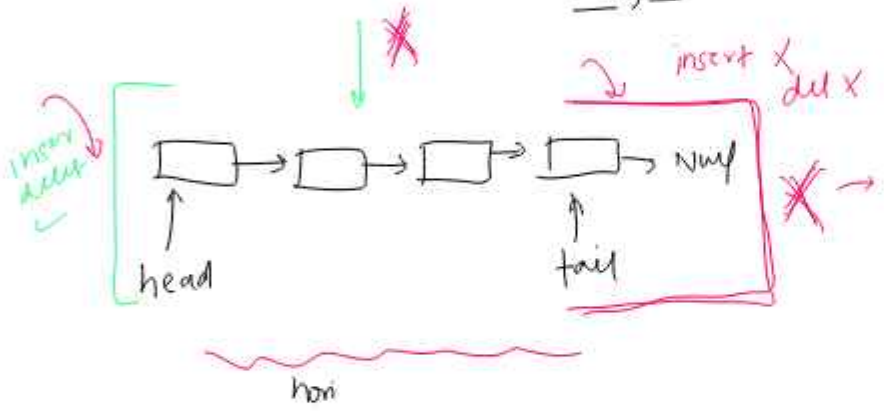
DS Algo → Time complex
→ Arrays → Dynamic Arrays
→ Linked list

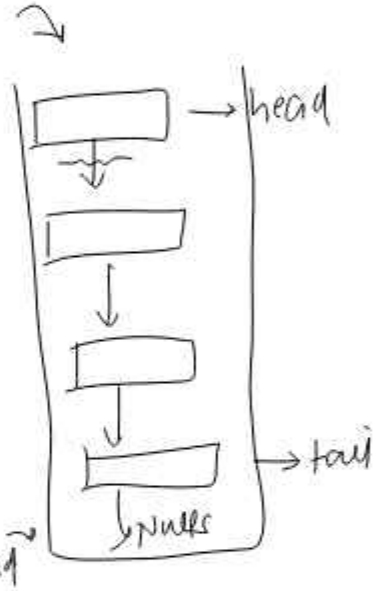
Today → Stacks → LL
→ Arrays

→ 3 → Tuesday
↓
last 3 videos

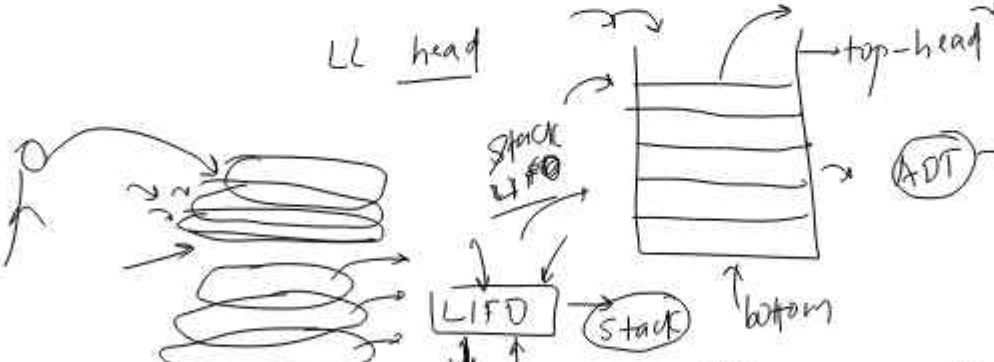
Next → Queues
→ Practice →

→ Stacks → DS
Usefull





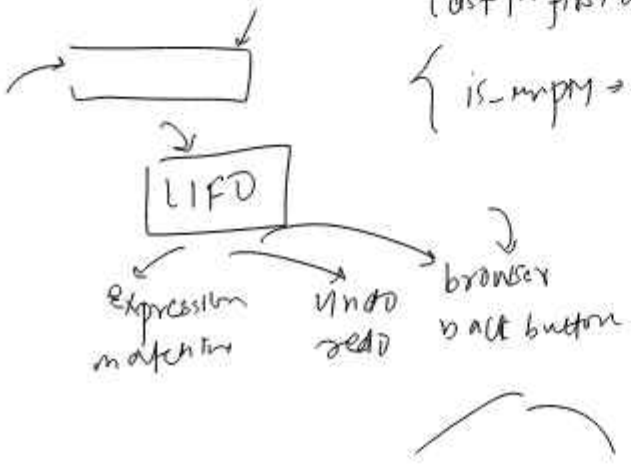
LL head



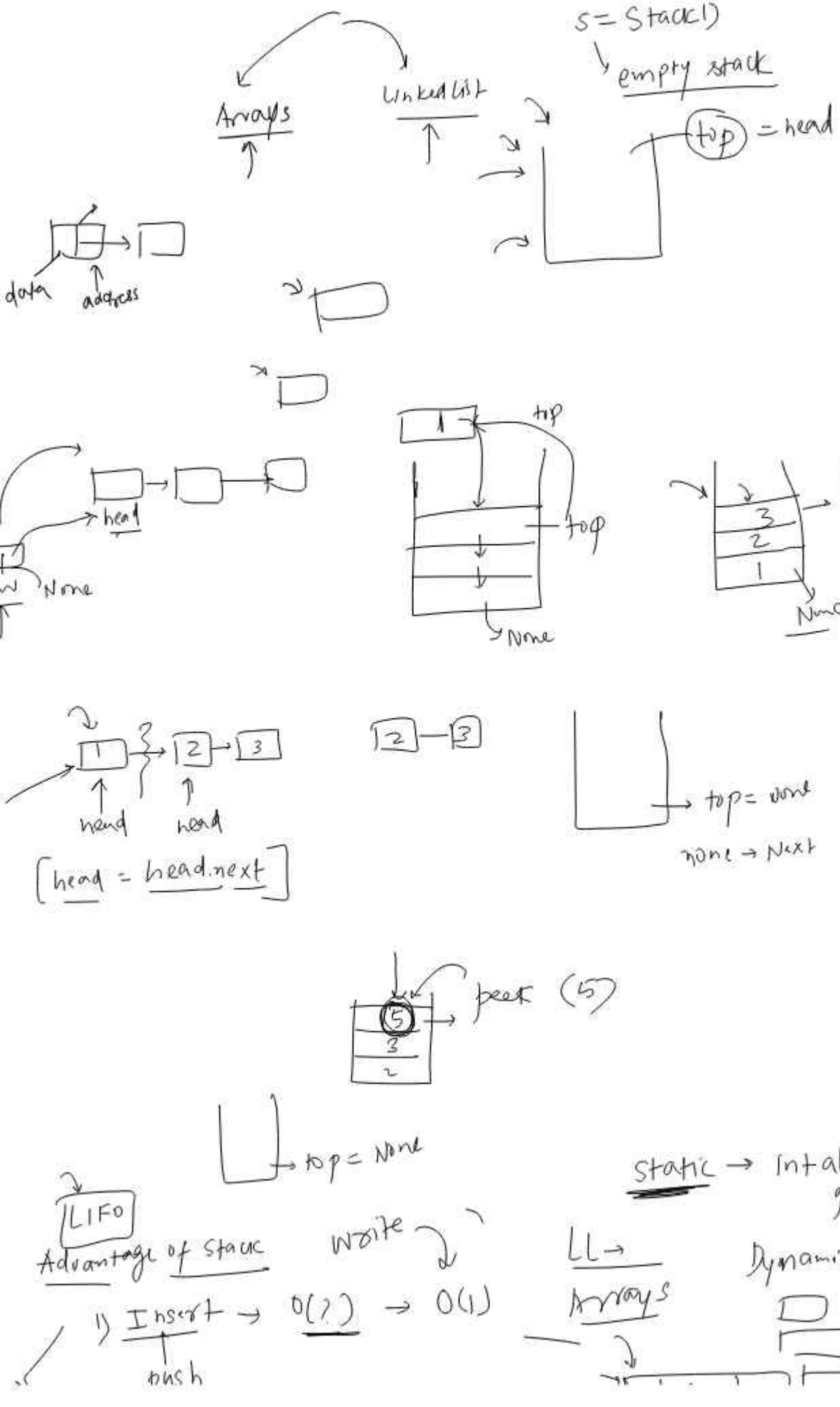
→ conceptual thing
↓
Normal data type

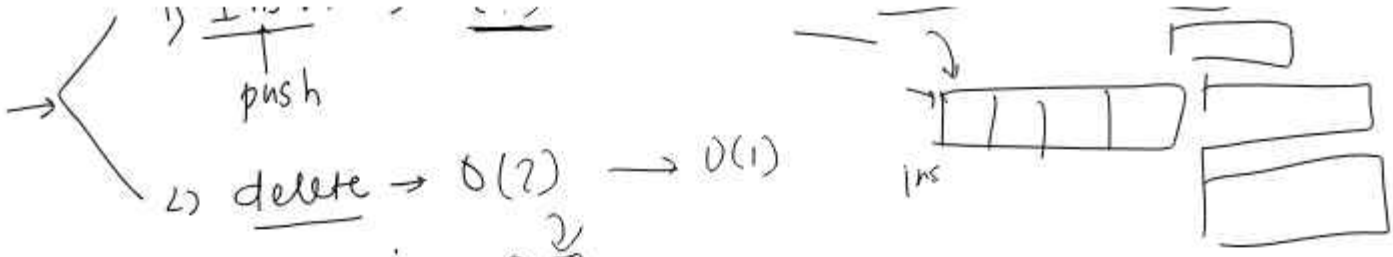
Last In first out

{ is_empty → empty
push → adding new node from top
pop → removing from top
peek → top node → value
size → size node



S = Stack()


Arrays
Linked list
s = Stack()
empty stack
top = head
data
address
head
new
None
top
None
3
2
1
None
head
head
[head = head.next]
top = None
none → Next
peek (5)
top = None
LIFO
Advantage of stack
1) Insert → O(?) → O(1)
push
write
LL →
Arrays
static → int a[50]
Dynamic



int a[5]

size = 4

top = -1 → 0 → 1 → 2 → 3

1, 2, 3, 4

-1 = 4-1 ×

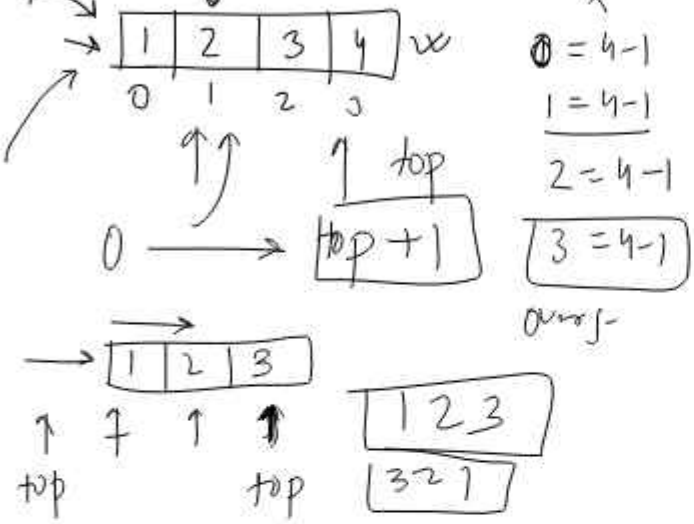
0 = 4-1

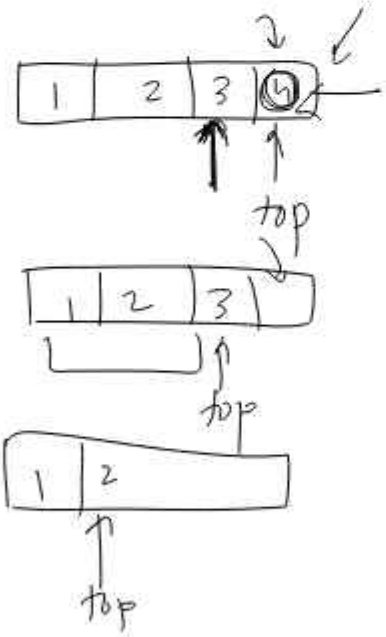
1 = 4-1

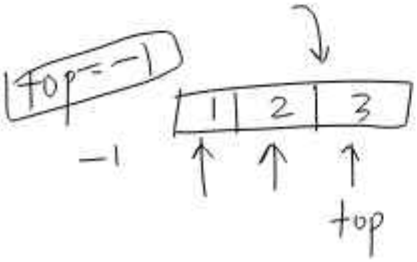
2 = 4-1

3 = 4-1

overf-



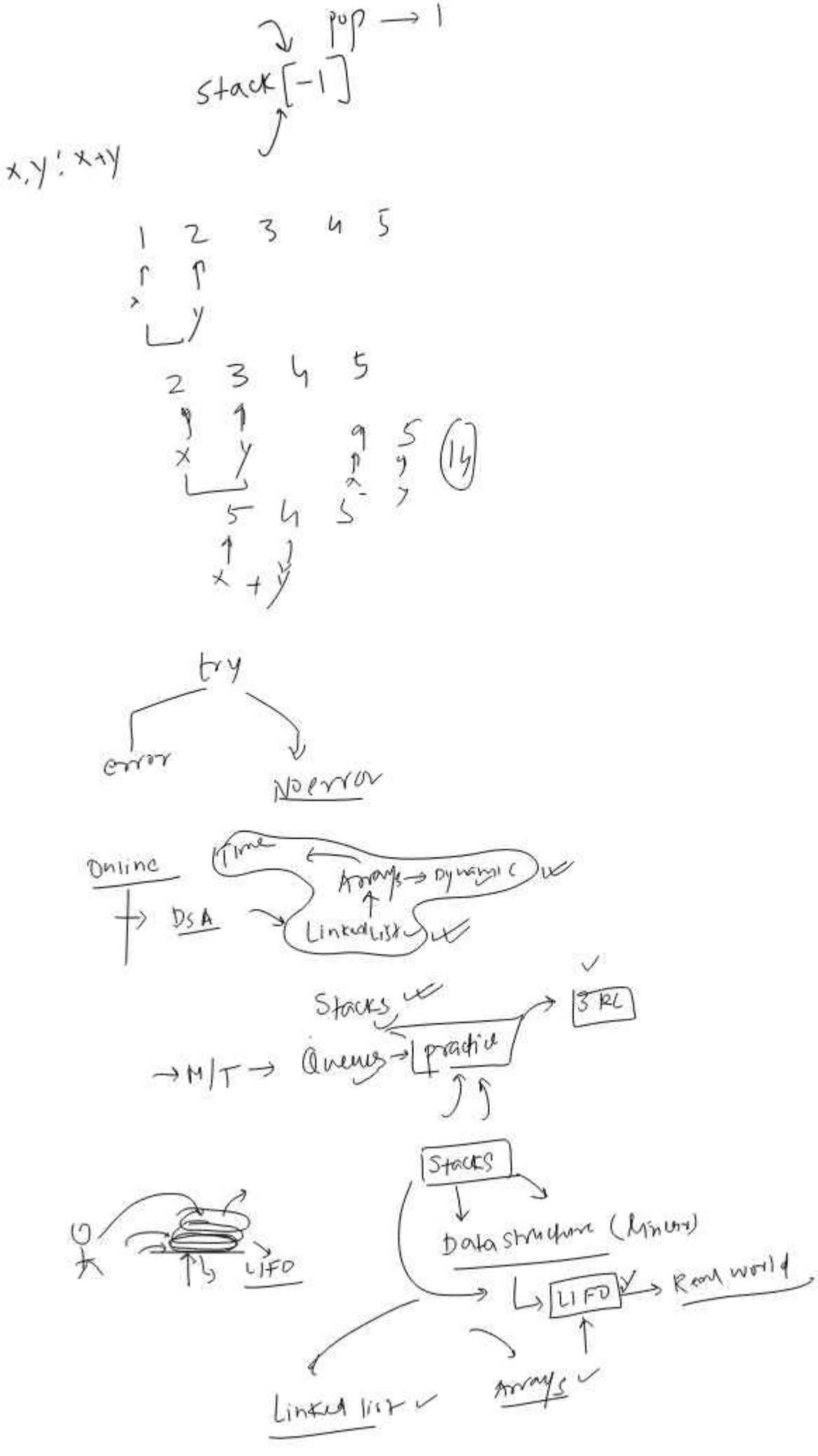


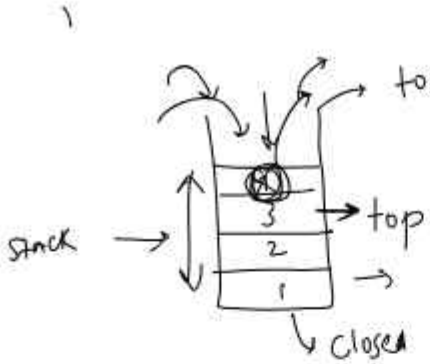


pop → 3

pop → 2

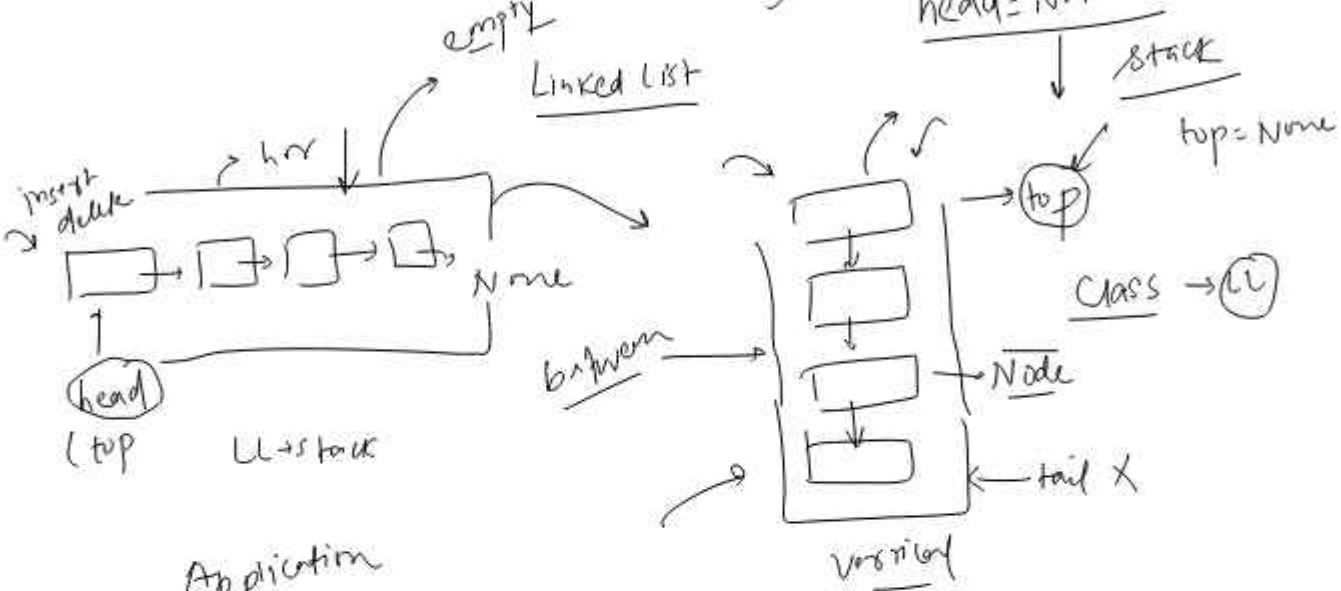
pop → 1


pop → 1
stack[-1]
x, y : x+y
1 2 3 4 5
2 3 4 5
14
try
error
No error
Online
Time
Arrays → Dynamic
DSA
LinkedList
Stacks
3 RL
→ M/T →
Queues
practice
Stacks
Data structure (Linear)
LIFO
LIFO
Real world
Linked list
Arrays



push
pop
peek
is empty
size

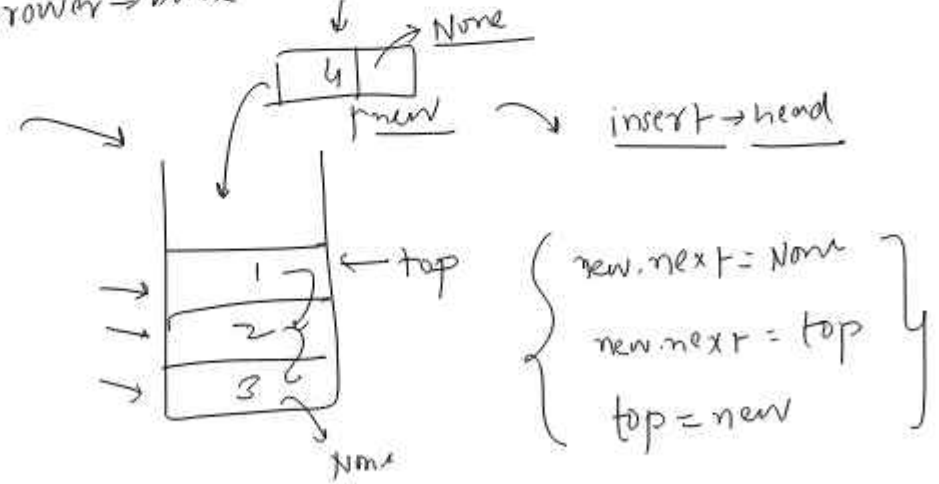
Simple



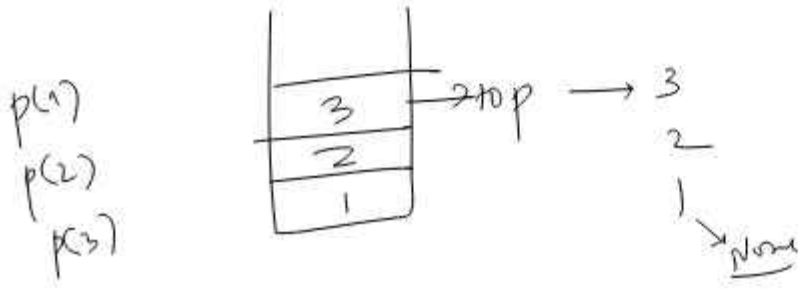
Application

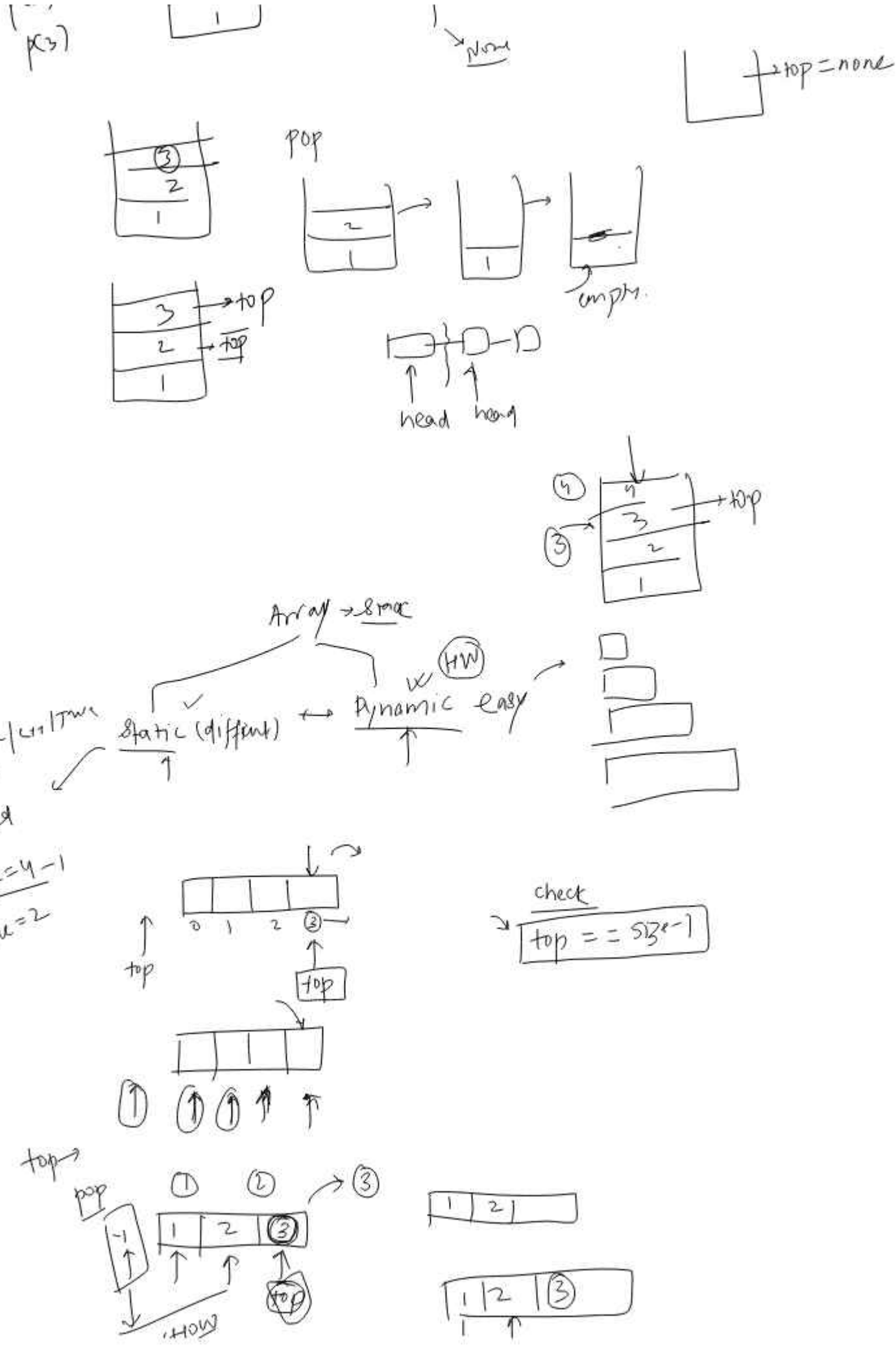
- expression
- undo/redo
- recursion
- browser → back

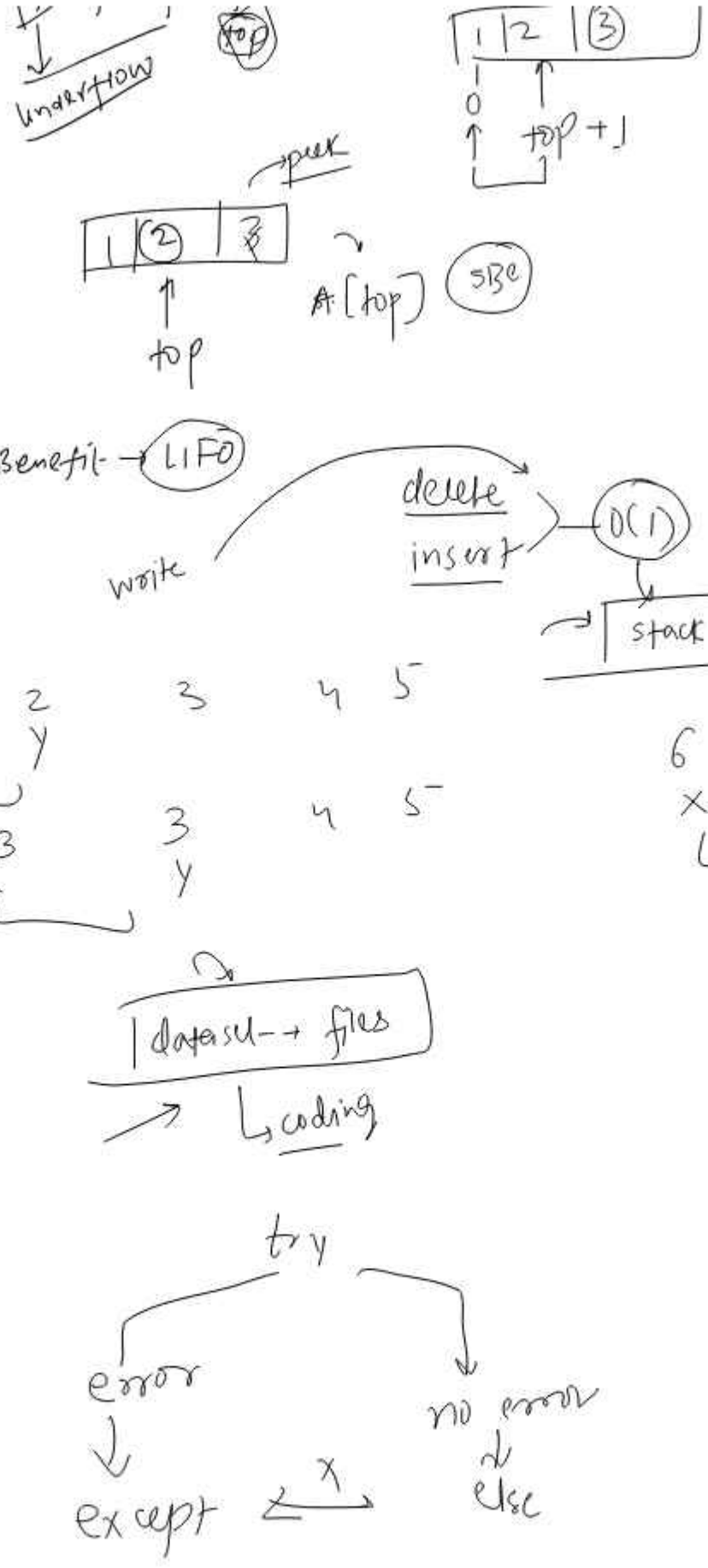
LIFO



123



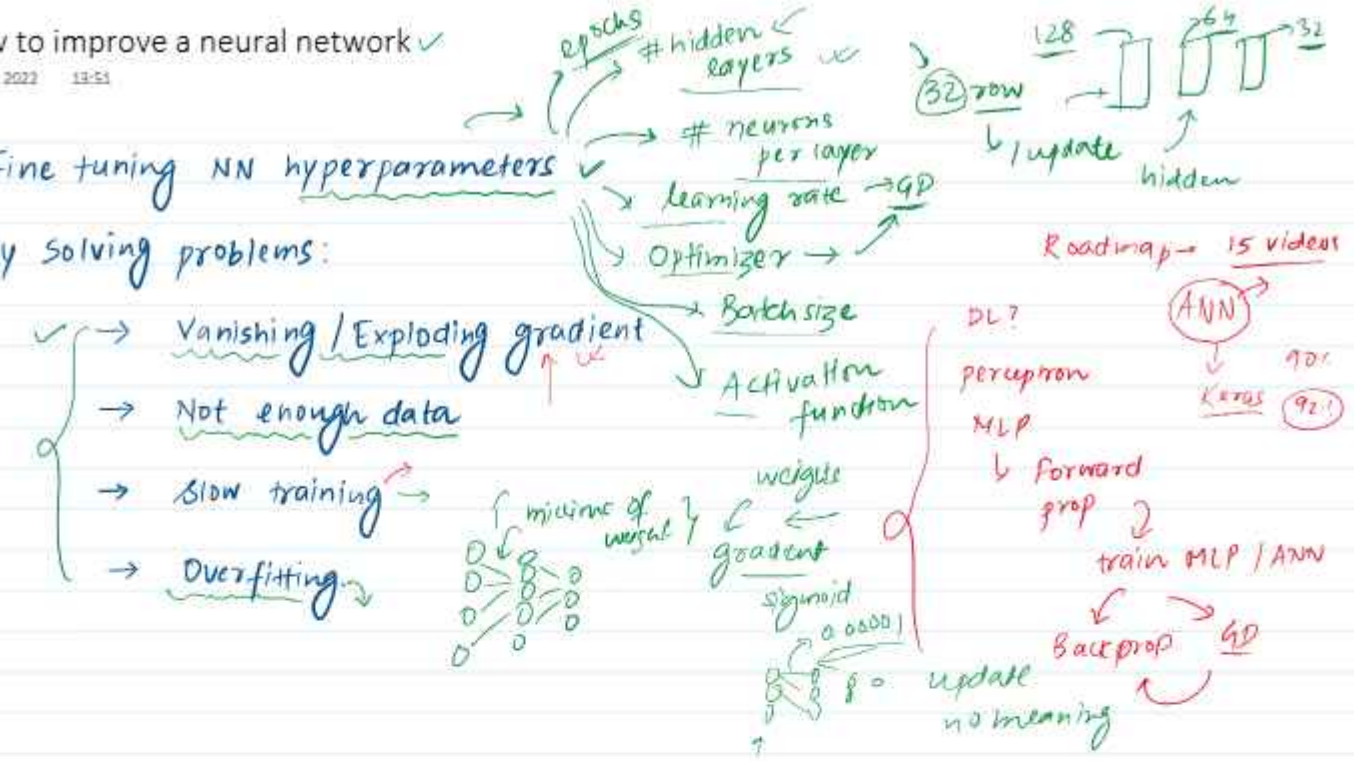

top = none
pop
empty
top
head
Array → Stack
Static (different)
Dynamic easy
HW
fixed
size = 4 - 1
value = 2
check
top == size - 1
pop
top
HOW

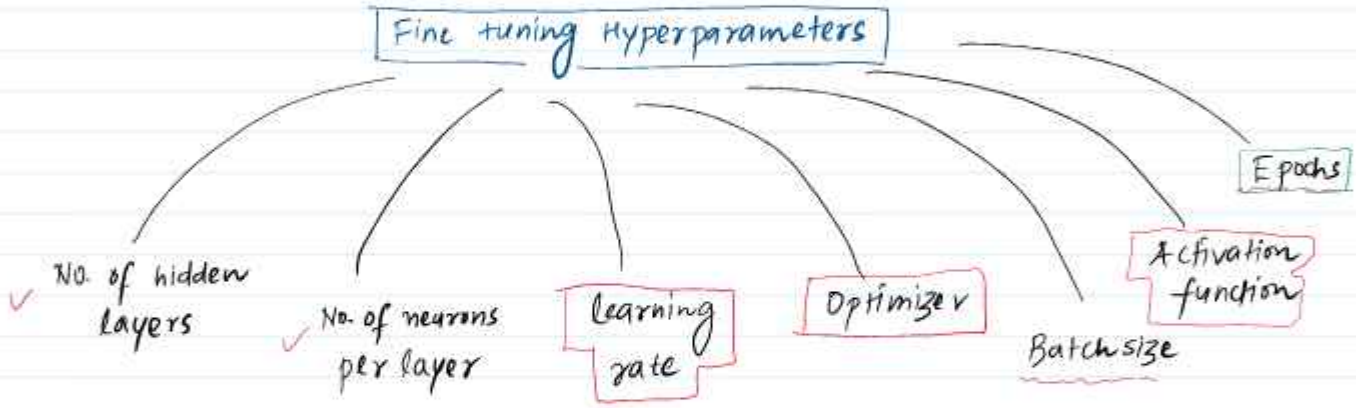

underflow
top
1 2 3
0
top + 1
peek
1 2 3
top
A[top]
size
Benefit → LIFO
write
delete
insert
O(1)
stack size
1 2 3 4 5
x y
3 3 4 5
x 3 y
6 4 5
x y
10
15
5
y
dataset → files
↳ coding
try
error
except
X
no error
else

How to improve a neural network

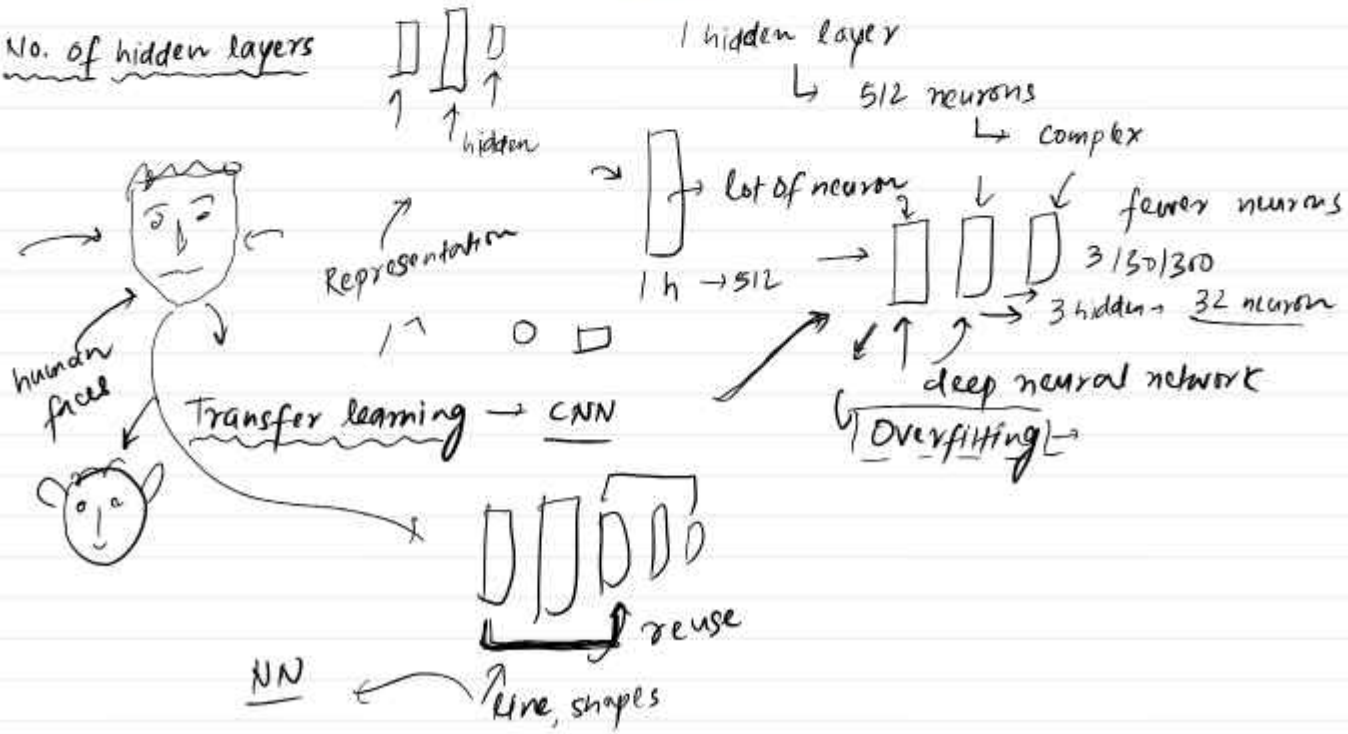
29 April 2022 13:51

1. Fine tuning NN hyperparameters
   - epochs
   - # hidden layers
   - # neurons per layer
   - learning rate → GD
   - Optimizer
   - Batch size
   - Activation function
2. By solving problems:
   - → Vanishing / Exploding gradient
   - → Not enough data
   - → Slow training
   - → Overfitting



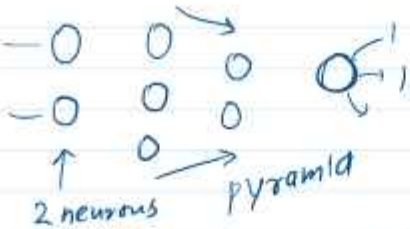


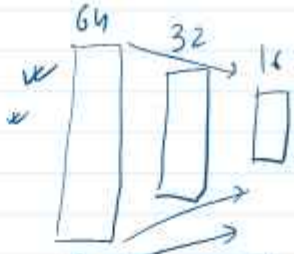
1 No. of hidden layers



2. Neuron / layer

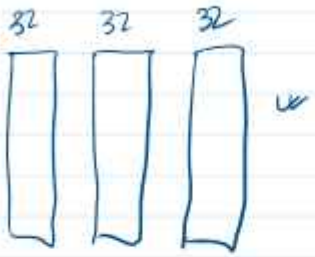
equal in place



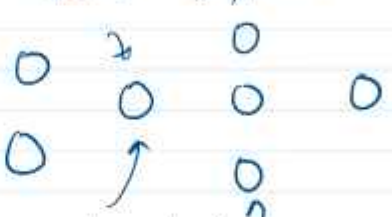


experiment



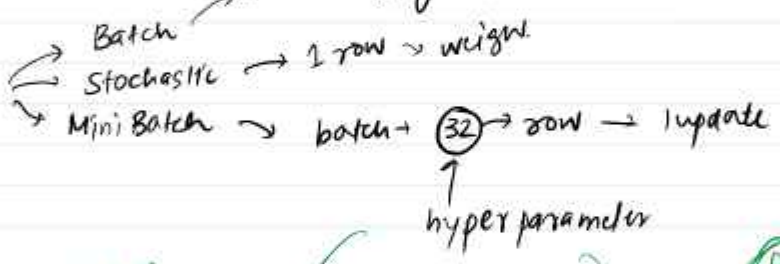


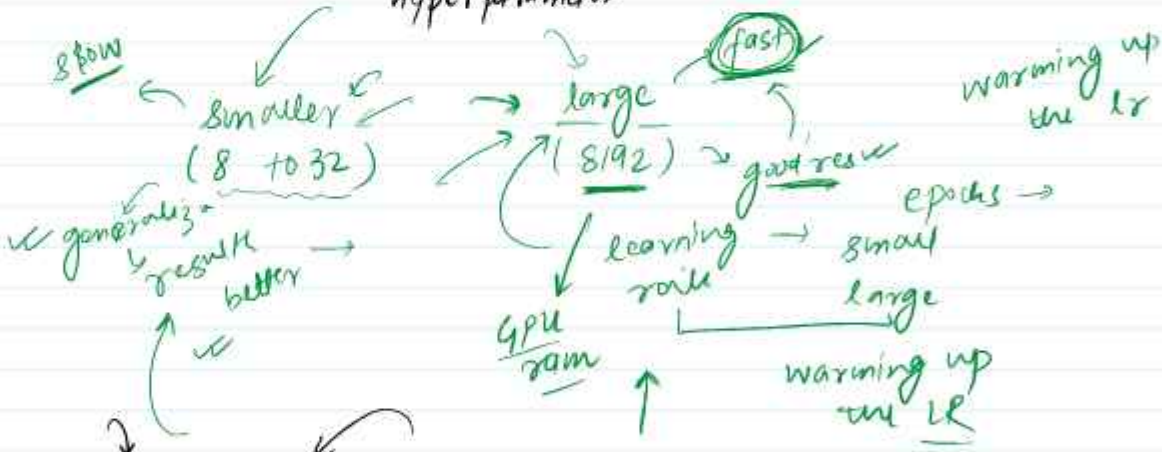
nodes sufficient



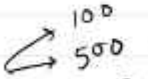


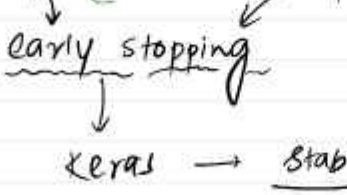
3. Batch size



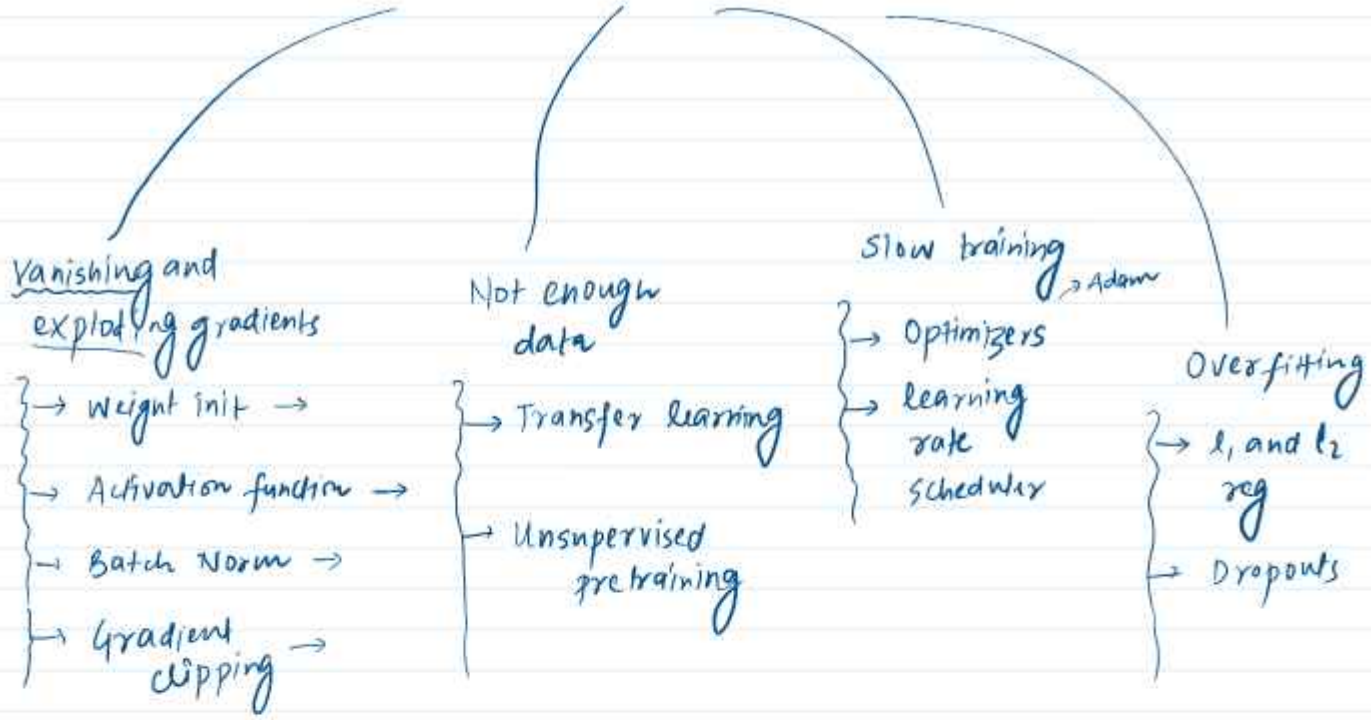


4. Epochs



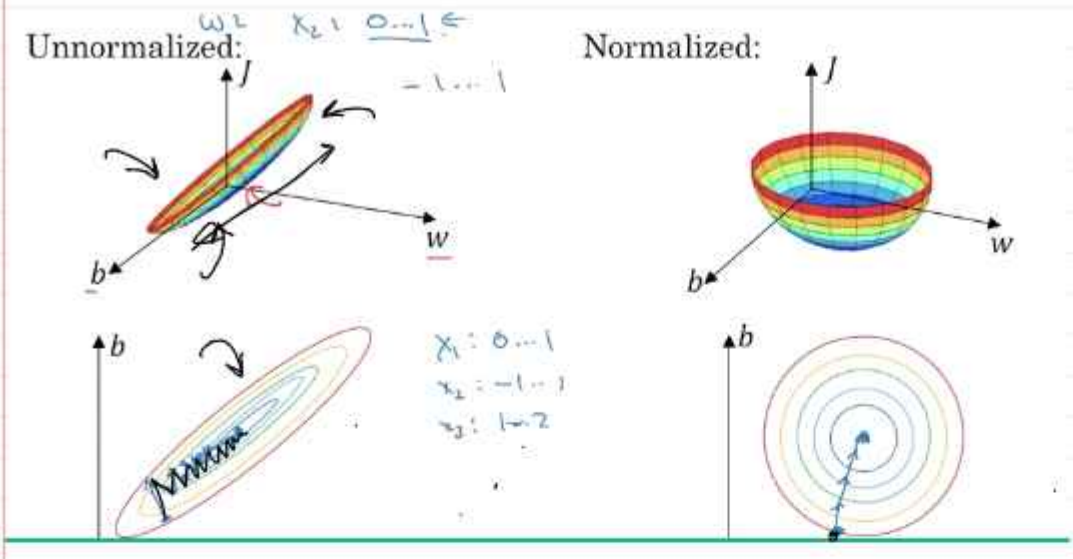


Problems with Neural Networks


Vanishing and exploding gradients
→ Weight init →
→ Activation function →
→ Batch Norm →
→ Gradient clipping →
Not enough data
→ Transfer learning
→ Unsupervised pretraining
Slow training
→ Adam
→ Optimizers
→ learning rate schedular
Overfitting
→ l_1 and l_2 reg
→ Dropouts

Feature Scaling

05 May 2022 17:37



(age) x_1 x_2 (salary)

w_1 w_2

$w_2 = w_2 - \eta \frac{\partial L}{\partial w_2}$

Standard

$\frac{x_j - \mu}{\sigma}$

Normalize

$\frac{x_j - x_{min}}{x_{max} - x_{min}}$

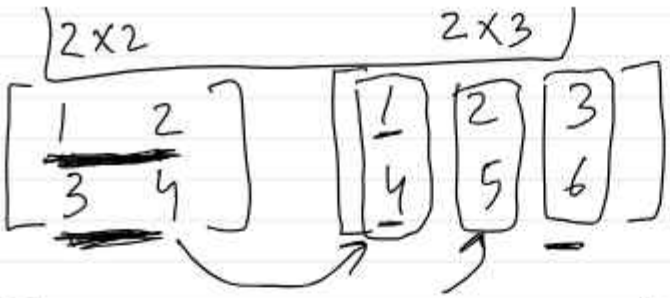
0 to 1

min = 0

max = 1

-1 to +1

2x2 2x3

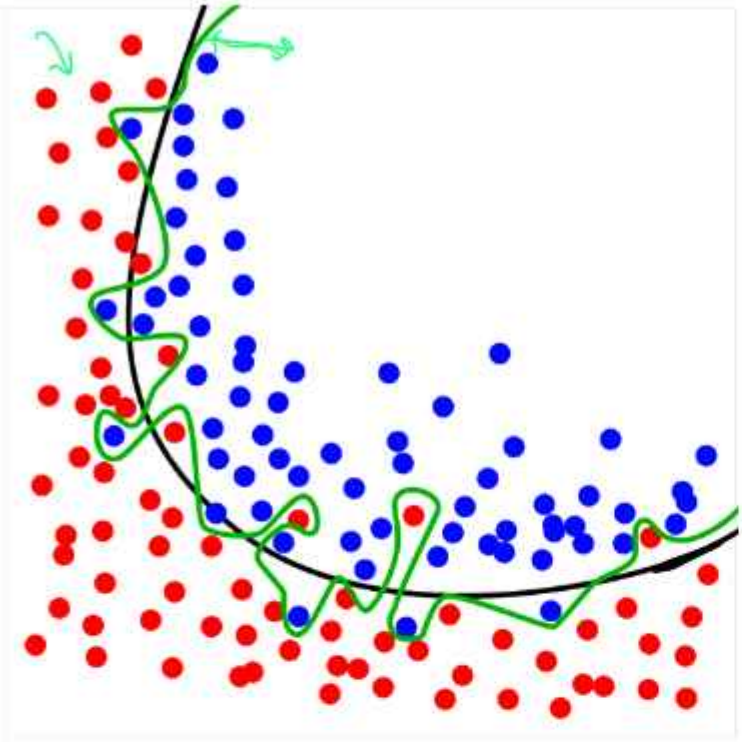


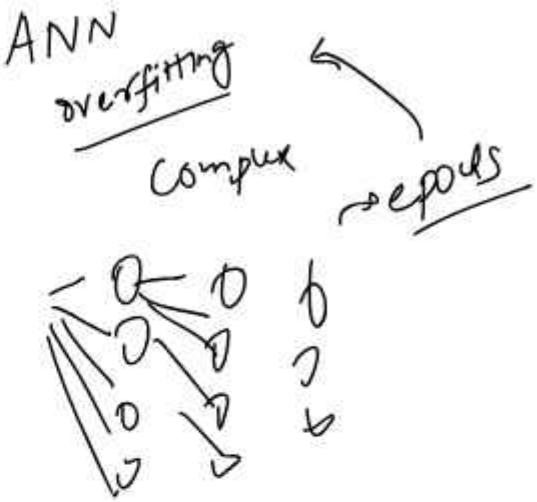
$$\begin{bmatrix} 1+8 & 2+10 & 3+12 \\ 3+16 & 6+20 & 9+24 \end{bmatrix}$$

$$\begin{bmatrix} 9 & 12 & 15 \\ 19 & 26 & 33 \end{bmatrix}$$

The Problem of Overfitting

12 May 2022 09:32





Possible Solutions

12 May 2022 09:33

1) Add more data

2) Reduce complexity

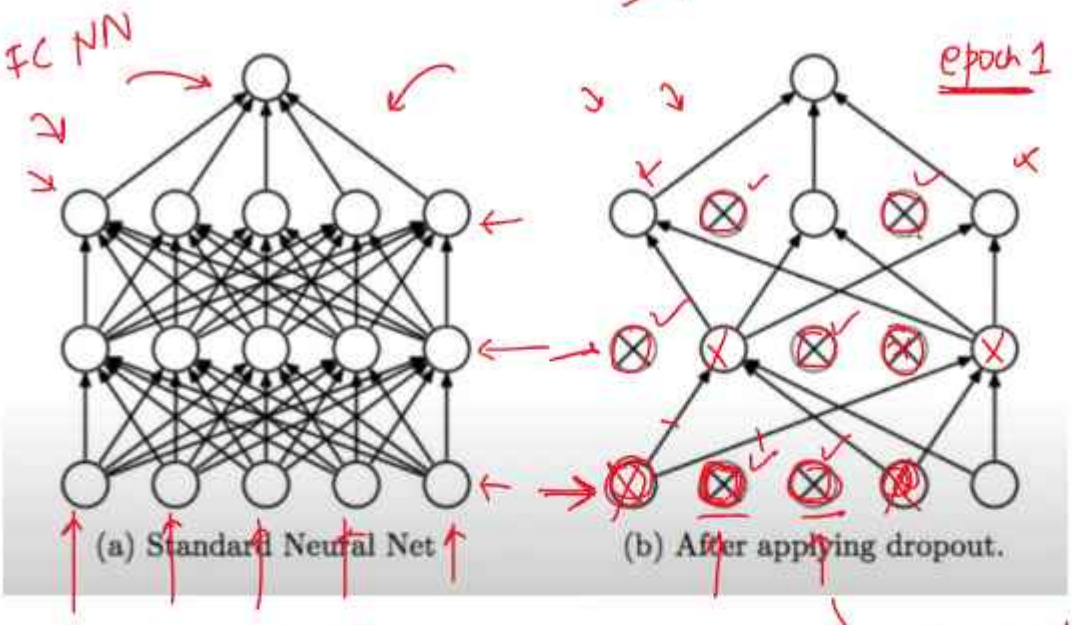
3) Early stopping

4) Regularization → L1, L2

5) Dropout

The concept of Dropouts

12 May 2022 09:33



Dropout

FC NN

epoch 1

x_1 x_2 x_3 x_4 x_5 y

input nodes

hidden drop

epoch = 10

epoch 2

epoch 3

2-1. → acc

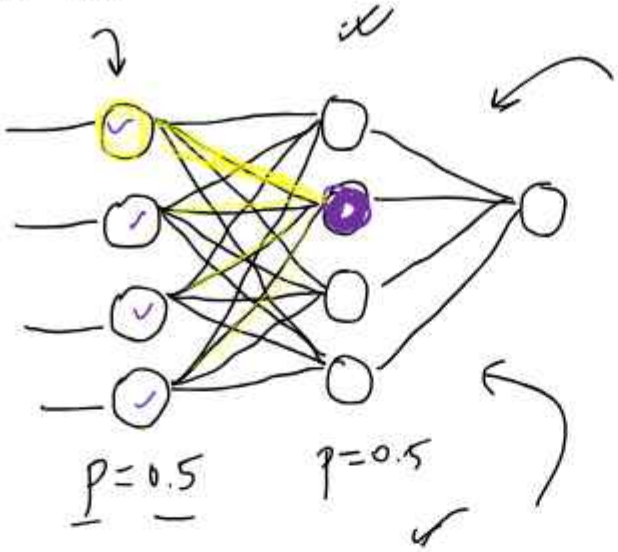
95 → 97

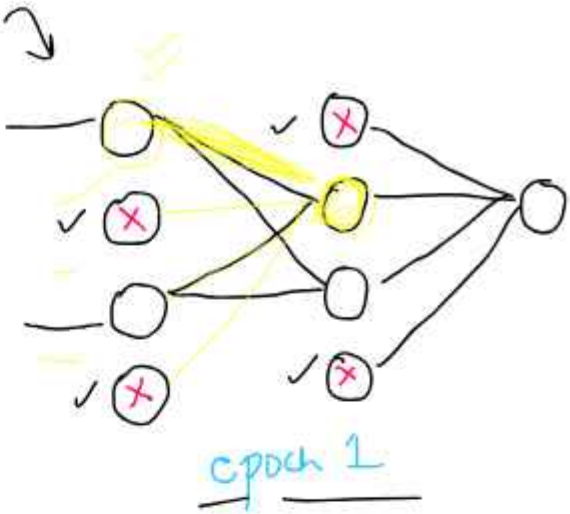
10 epochs

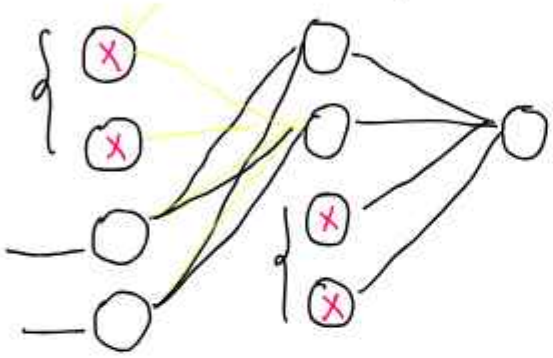
10 diff NNs

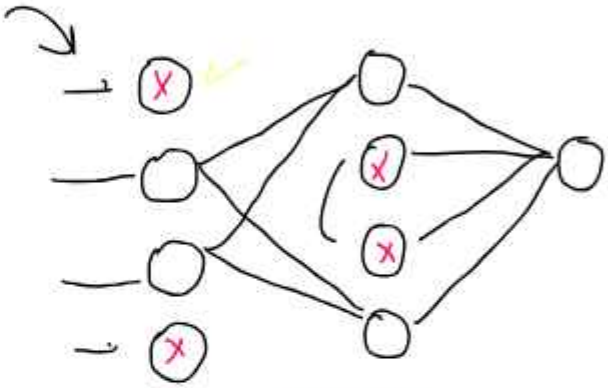
Why this works?

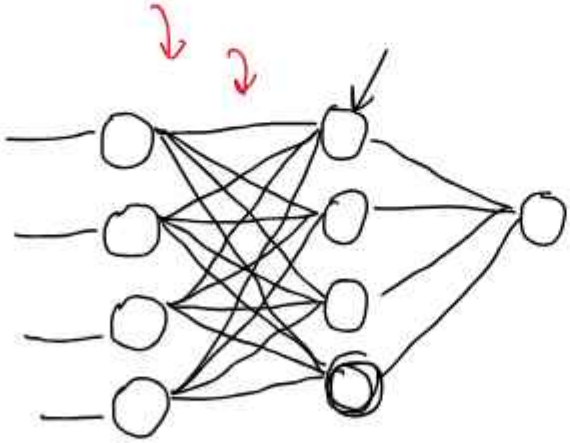
12 May 2022 09:33





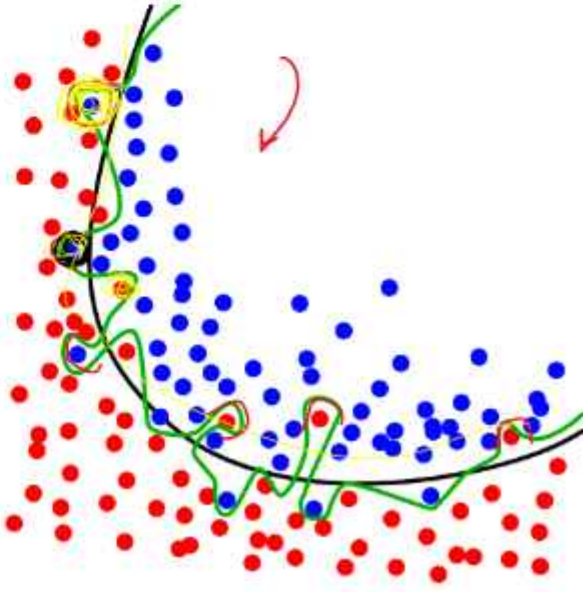






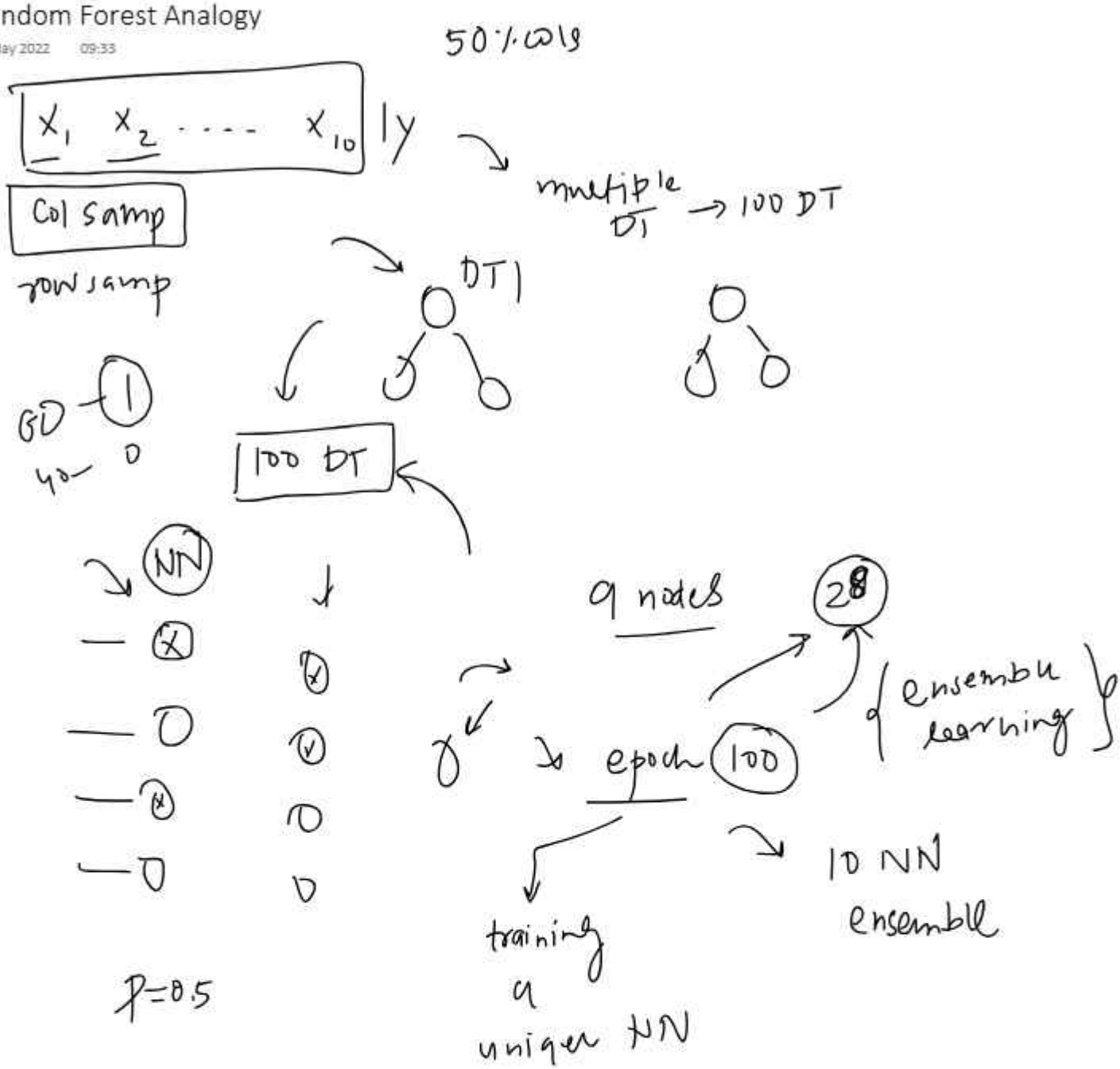
→ Reduce the # nodes

→



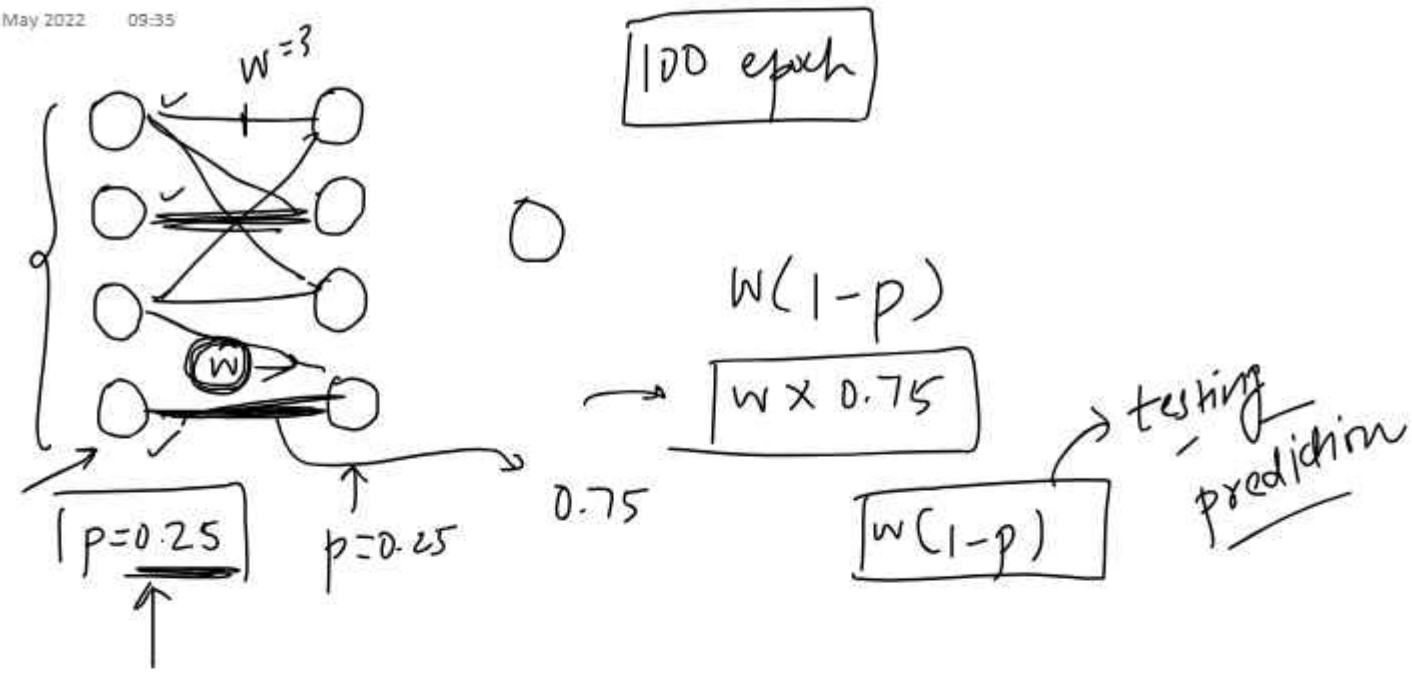
Random Forest Analogy

12 May 2022 09:33



How prediction works?

12 May 2022 09:35



Regression Code Example

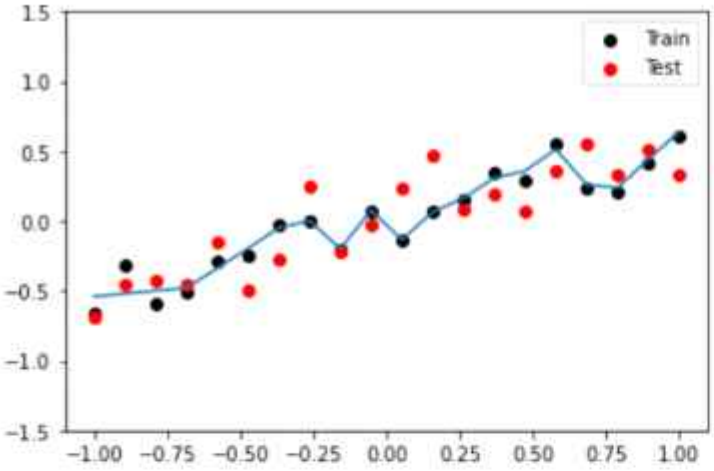
12 May 2022 09:34

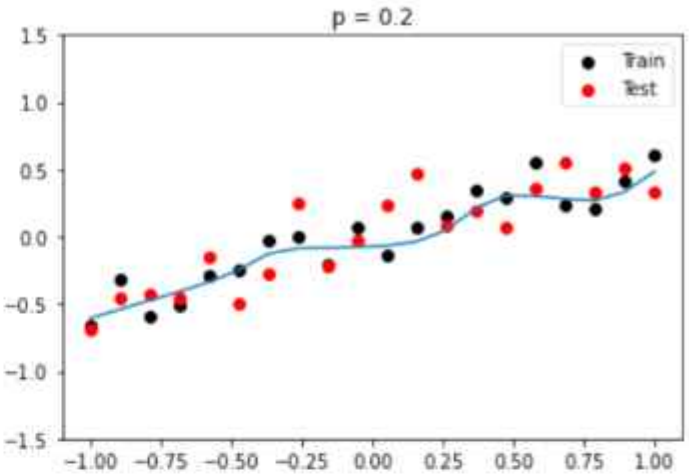
Classification Code Example

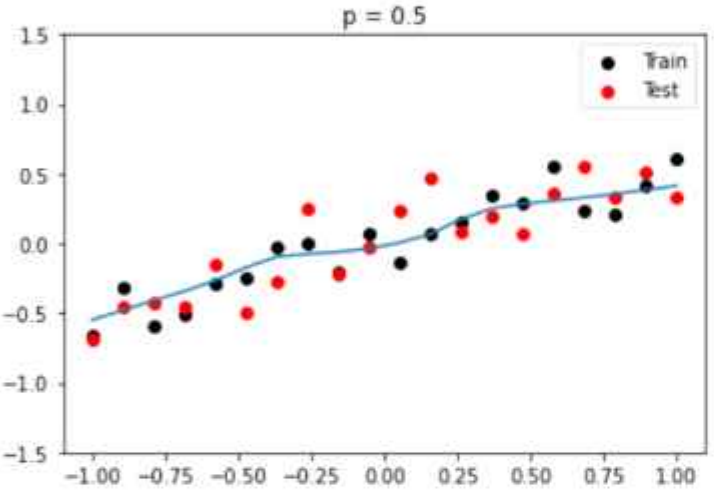
12 May 2022 09:34

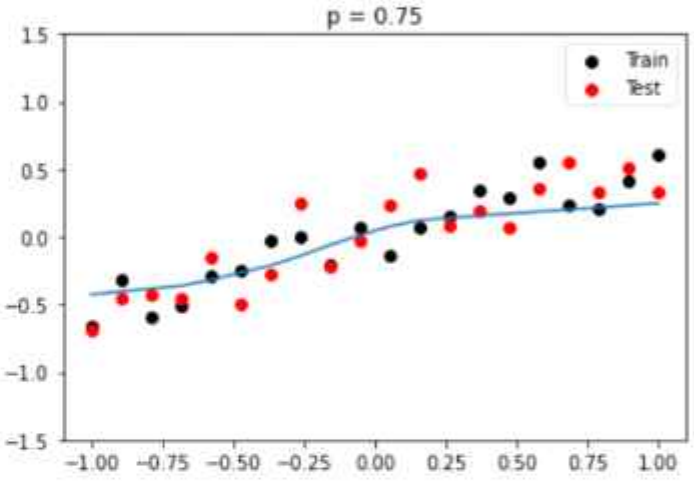
Effect of p

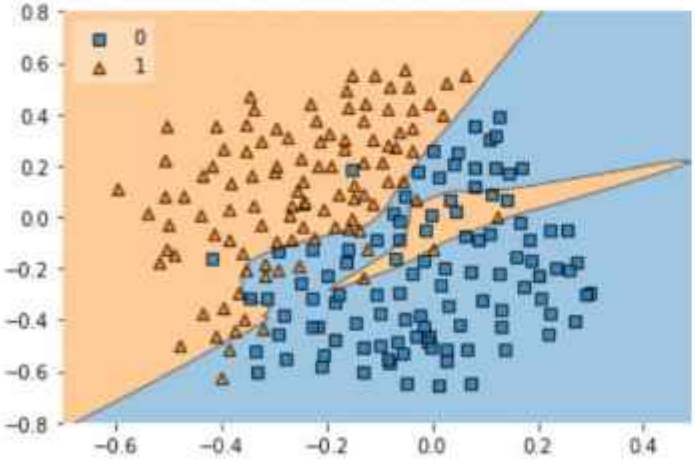
12 May 2022 10:06

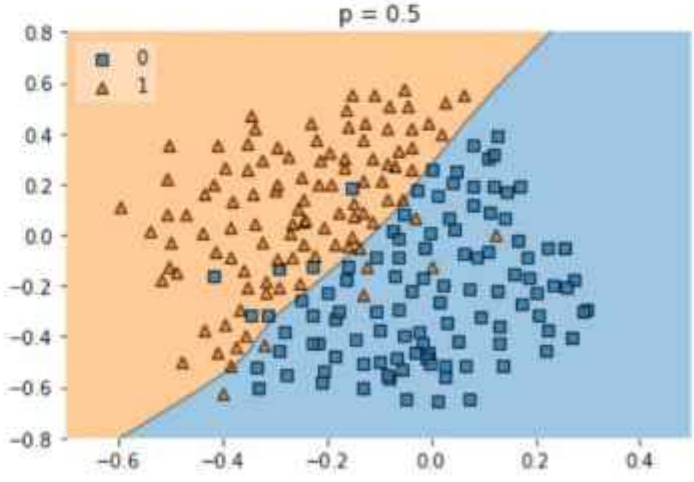










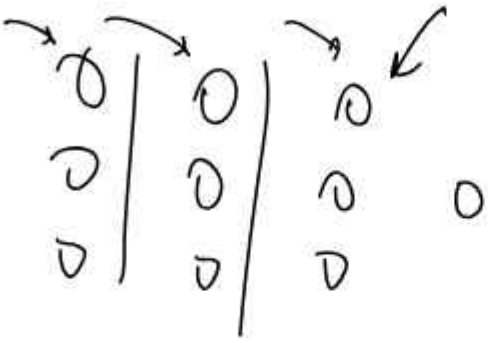


Practical Tips and Tricks

12 May 2022 09:34

1) Overfitting P↑, underfitting P↓

2) Last layer → dropout



3) CNN → 40-50 · (P) → ✓

20-30 RNN 50>

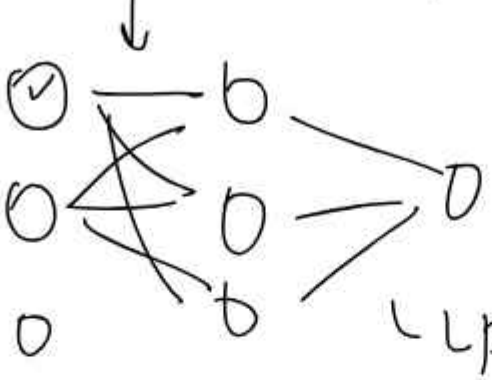
ANN → 10-50

Drawbacks

12 May 2022 09:34

1) convergence → delay

2)



{ loss function } value changes

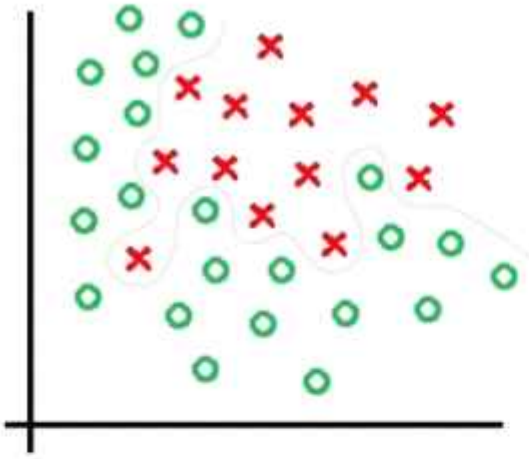
↳ LF – has to change

Resources

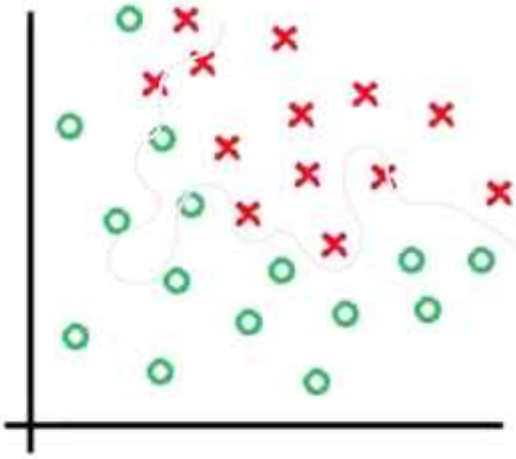
12 May 2022 12:38

Overfitting

20 May 2022 08:14



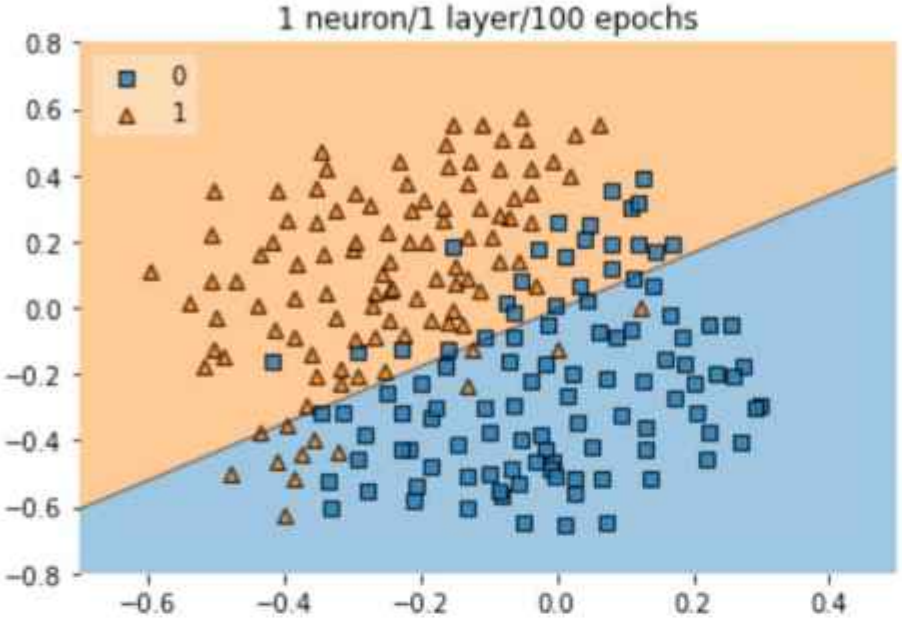
Train Dataset

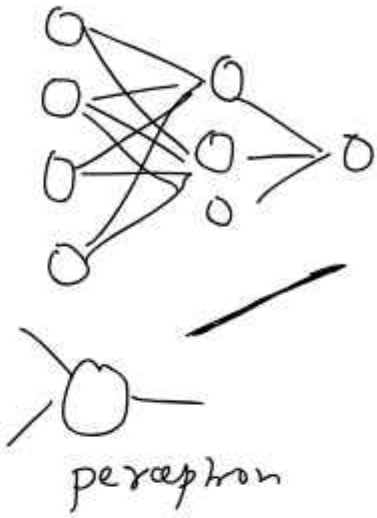


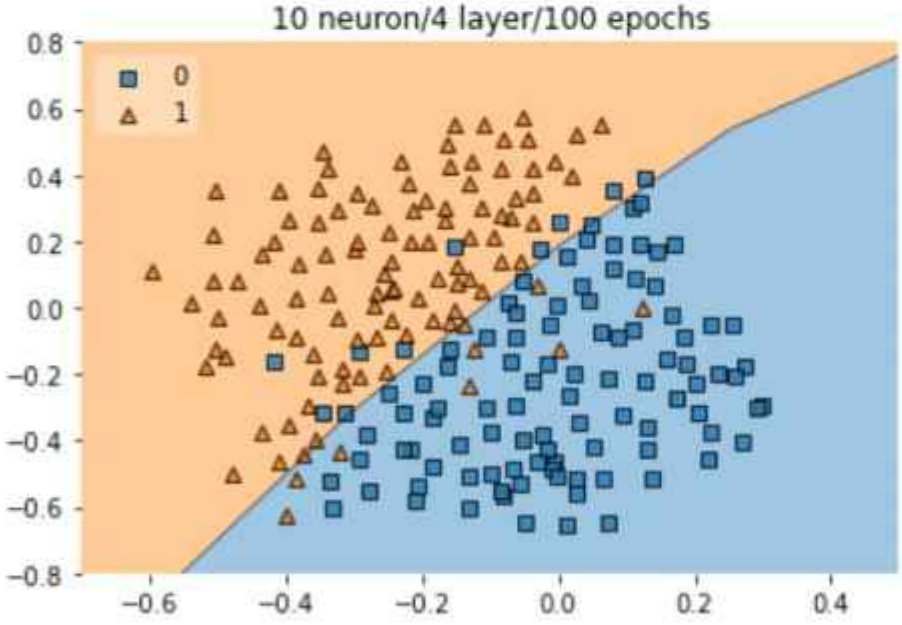
Test Dataset

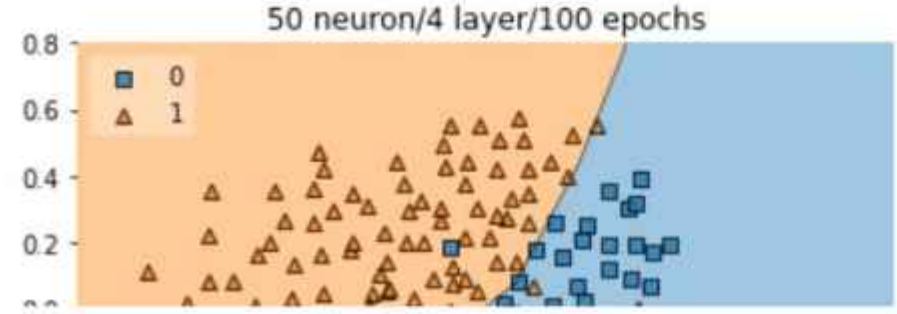
Why Neural Networks Overfit?

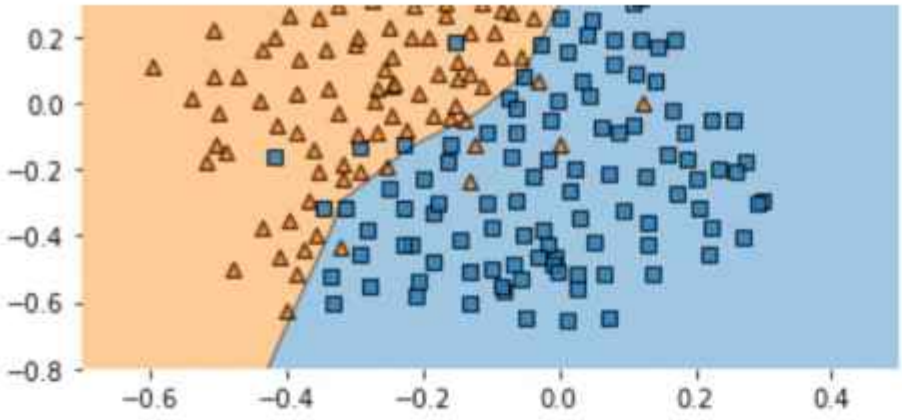
19 May 2022 07:34

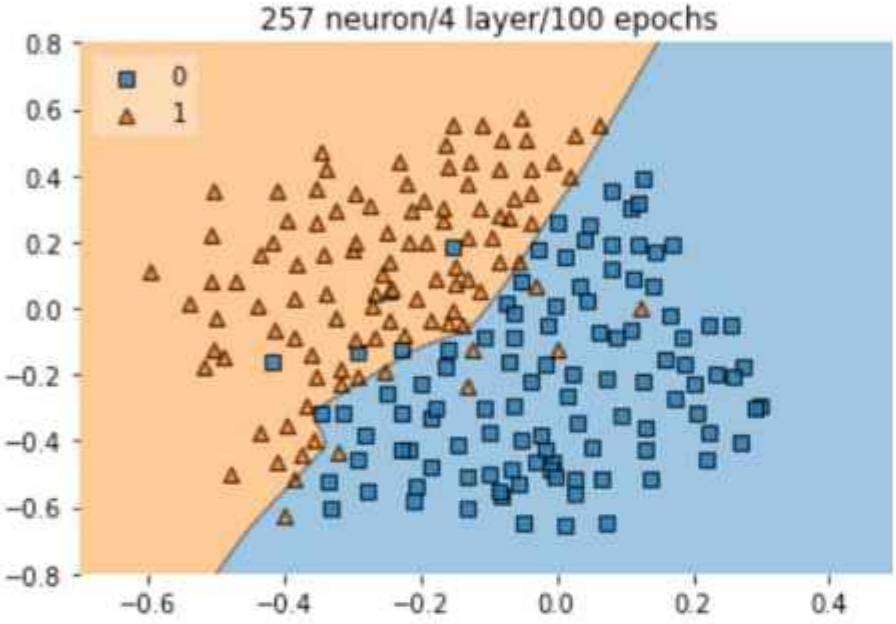


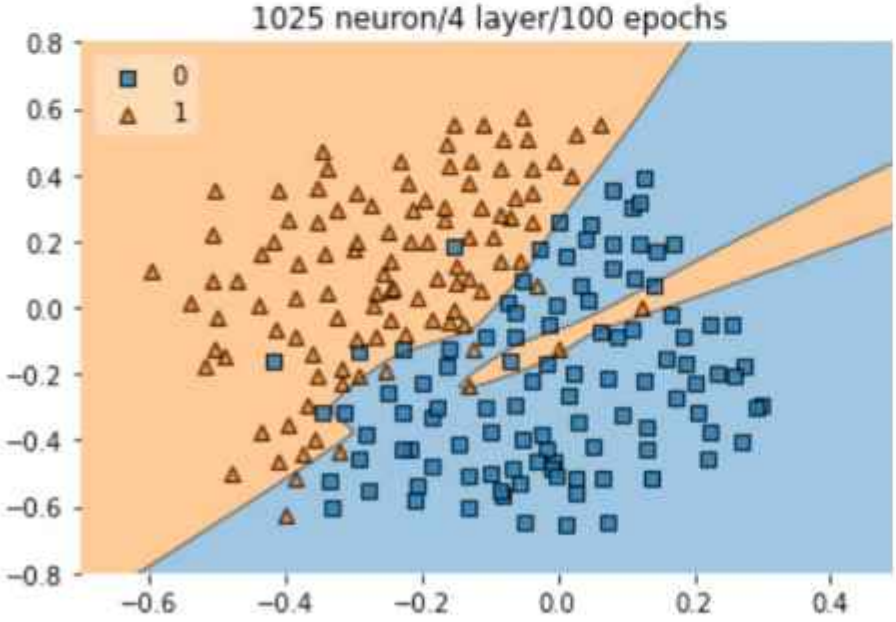


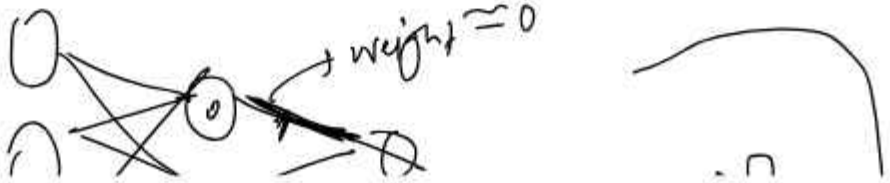


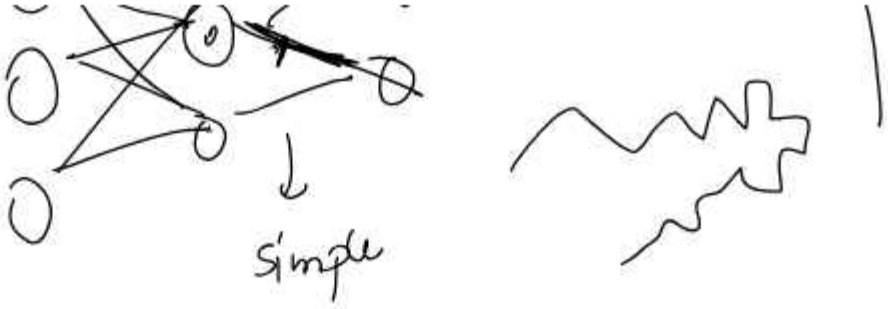





257 neuron/4 layer/100 epochs
0
1

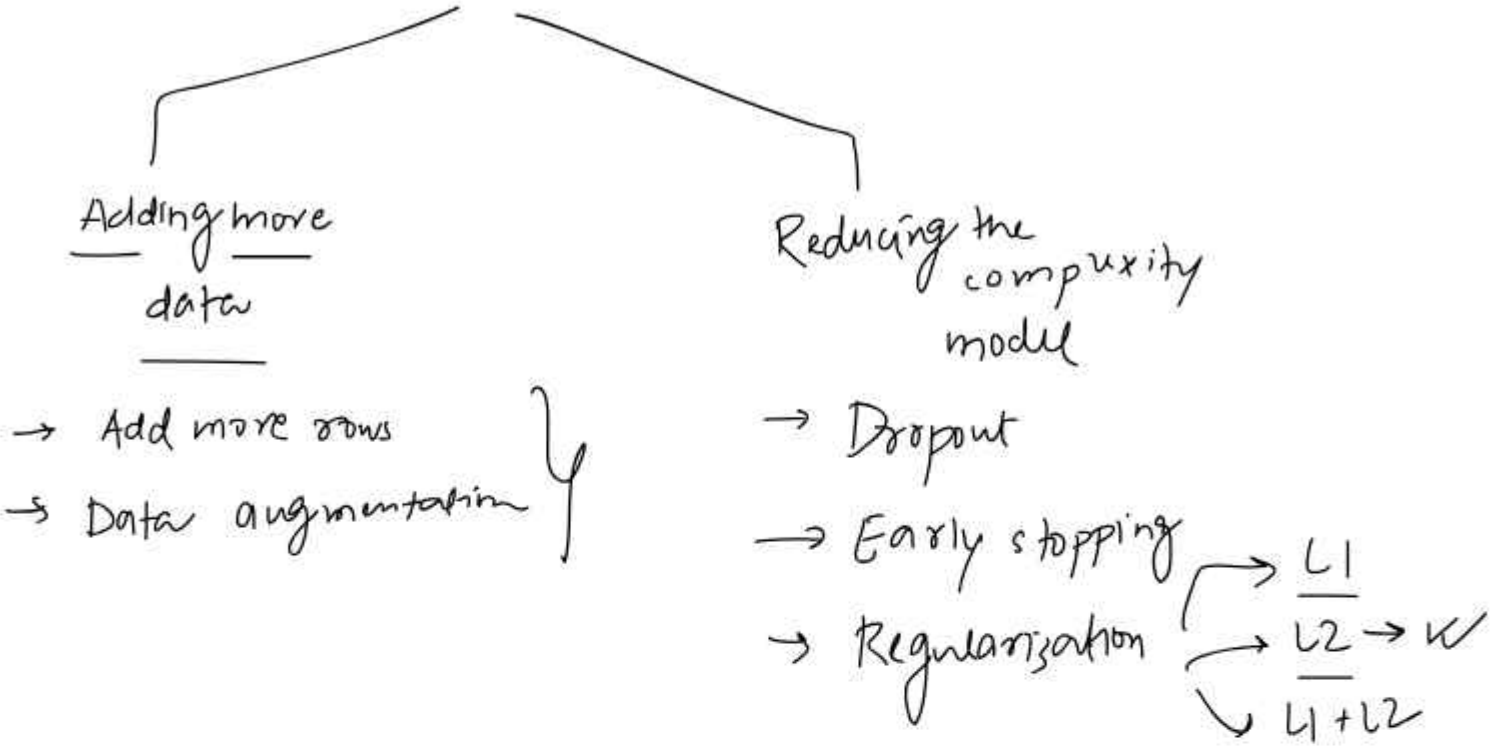

1025 neuron/4 layer/100 epochs
0
1


weight ≈ 0


simple

Ways to solve overfitting

20 May 2022 08:13



Regularization

20 May 2022 08:14

ANN → weights | bias

↳ min Loss function

L = mse

↳ binary

$$C = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{y}_i) + \text{penalty term}$$

$$C = L + \frac{\lambda}{2n}\sum_{i=1}^{k} \|w_i\|^2 \rightarrow \text{weightage}$$

L2

L1

$w_1 \rightarrow w_{10}$

$$\frac{\lambda}{2n}\left[w_1^2 + w_2^2 + \dots + w_{10}^2\right]$$

↑ λ = hyperparameter ↑

$\lambda = 0$

$$C = \sum L(y_i, \hat{y}_i) + P$$

L2 → (L1)

$P \rightarrow w \simeq 0$

→ L1 norm

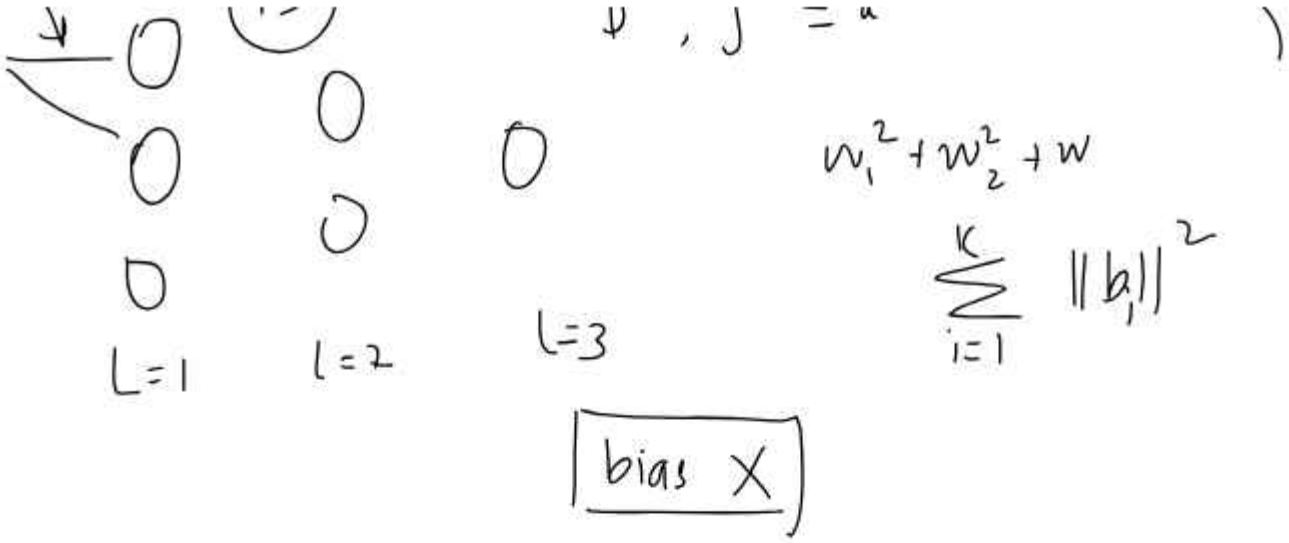
$$C = L + \frac{\lambda}{2n}\sum \|w_i\|$$

$$C = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{l=1}^{L}\sum_{i=1}^{}\sum_{j=1}^{} \|w_{ij}^{l}\|^2$$

$w \simeq 0$

$\rightarrow 0$ (15) $\cap$

$0, j = d$


L=1
L=2
L=3
$w_1^2 + w_2^2 + w$
$\sum_{i=1}^{K} \|b_i\|^2$
bias X

Intuition behind Regularization

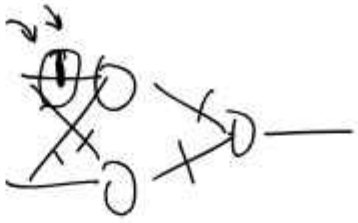
20 May 2022 08:15

$$w_n = w_0 - \eta \frac{\partial L}{\partial w_0}$$

$1 - \eta\lambda$

positive

L



$$L' = L + \frac{\lambda}{2} \sum \|w_i\|^2$$

$$w_2^1 + w_2^2 + w_3 + \dots$$

$2w$

$$\frac{\partial L'}{\partial w_0} = \frac{\partial L}{\partial w_0} + \frac{\lambda}{2} 2 w_0$$

$$= \frac{\partial L}{\partial w_0} + \lambda w_0$$

$$w_n = w_0 - \eta \left(\frac{\partial L}{\partial w_0} + \lambda w_0 \right)$$

$$w_n = w_0 - \eta\lambda w_0 - \eta \frac{\partial L}{\partial w_0}$$

$$w_n = (1 - \eta\lambda) w_0 - \eta \frac{\partial L}{\partial w_0}$$

$w_0 \ll w_0$

L2 reg → weight decay

weight decay ↑ w_0

Code Demo

20 May 2022 08:15

Further Resources

20 May 2022 08:15

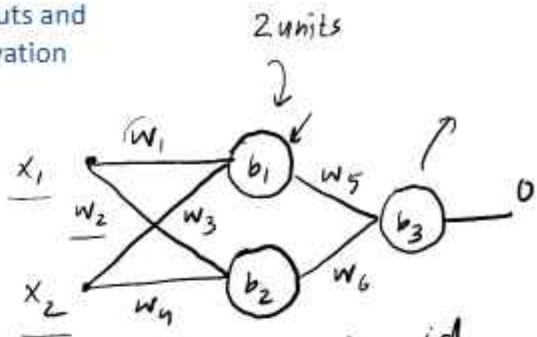
What are Activation Functions?

31 May 2022 14:49

In artificial neural networks, each neuron forms a weighted sum of its inputs and passes the resulting scalar value through a function referred to as an activation function or transfer function. If a neuron has n inputs then the output or activation of a neuron is

$$a = g(w_1x_1 + w_2x_2 + w_3x_3 + \dots w_nx_n + b)$$

This function g is referred to as the activation function.



$g(w_1x_1 + w_2x_2 + b_1)$ → sigmoid, relu, softmax

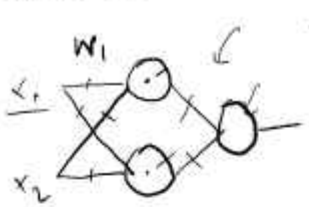
g → activation

input → output

gate → activated (Y/N) → how much

Why Activation Functions are needed?

31 May 2022 14:50



forward prop

$$Z_1 = W_1 A_0 + b_1$$

$$A_1 = g(Z_1) = Z_1$$

linear

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

sigmoid



ANN

↓

Non-linear

Non-linear

output = input

$$A_2 = g(W_2 A_1 + b_1)$$

$$= W_2 A_1 + b_2 = W_2 (W_1 A_0 + b_1) + b_2$$

$$= W_2 W_1 A_0 + W_2 b_1 + b_2$$

$$= W' A_0 + b'$$

output

input

$$A_2 = \hat{y} = W' A_0 + b'$$

linear

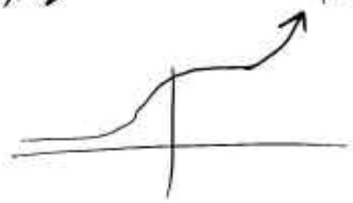
Ideal Activation Function

31 May 2022 14:50

$y = f(x)$ ✓ linear x non-linear

$\sigma(z) = \frac{1}{1+e^{-z}}$

1) Non-linear ✓ → Universal Approx Theorem



2) Differentiable ✓ → gradient des → derivatives → relu (0)

3) computationally inexpensive ✓ → derivative → simples easy fast → training slow

4) Zero centered → Zero cent normalized → mean = 0 → normalized data input → later layers → tanh

5) Non-saturating → Sigmoid → [0, 1] → faster → $f(x) = \max(0, x)$

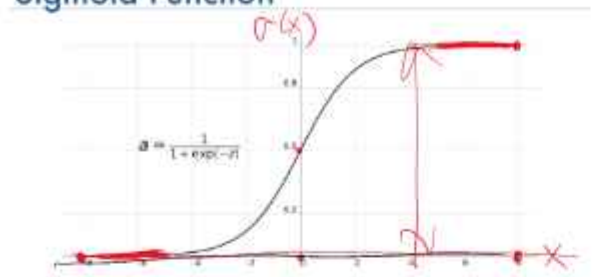
tanh → (-1, 1) Relu

update x [0, 1] → backprop → saturating AF → Vanishing Gradient Problem

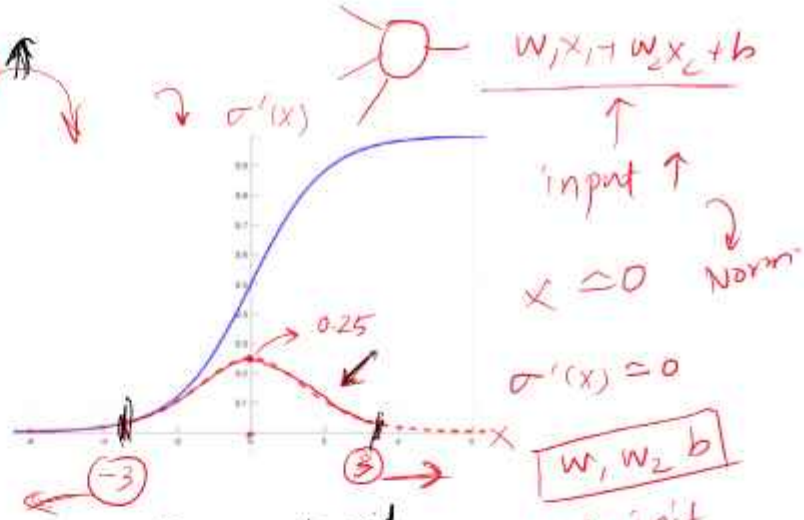
0.01 → 0.0001 ← 0.0001

Sigmoid Activation Function

31 May 2022 14:50



$$\sigma(x) = \frac{1}{1+e^{-x}}$$



$w_1x_1 + w_2x_2 + b$

input ↑ → Norm

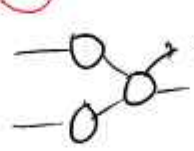
$x \simeq 0$

$\sigma'(x) \simeq 0$

w_1, w_2, b

Yes

NO



sigmoid → 0-1 — yes / No

Weight init

Advantages

1) [0,1] → probability → output layer → Binary classification

2) Non-linear → Non-linear data → good option

3) Differentiable → Backprop → $\frac{\partial L}{\partial w}$ ✓ / ✗

Disadvantages

1) Saturating function → $[-\infty, \infty] \rightarrow [0,1]$

↳ Vanishing gradient problem

Hidden layer — sigmoid — training ✗

output → binary classification

Backprop → update

$$w_n = w_0 - \eta \frac{\partial L}{\partial w}$$

$w_n = w_0$

training ✗

No update will take place

2) Non Zero centered

Normalized → mean = 0

+ve / -ve → Training slow

zero

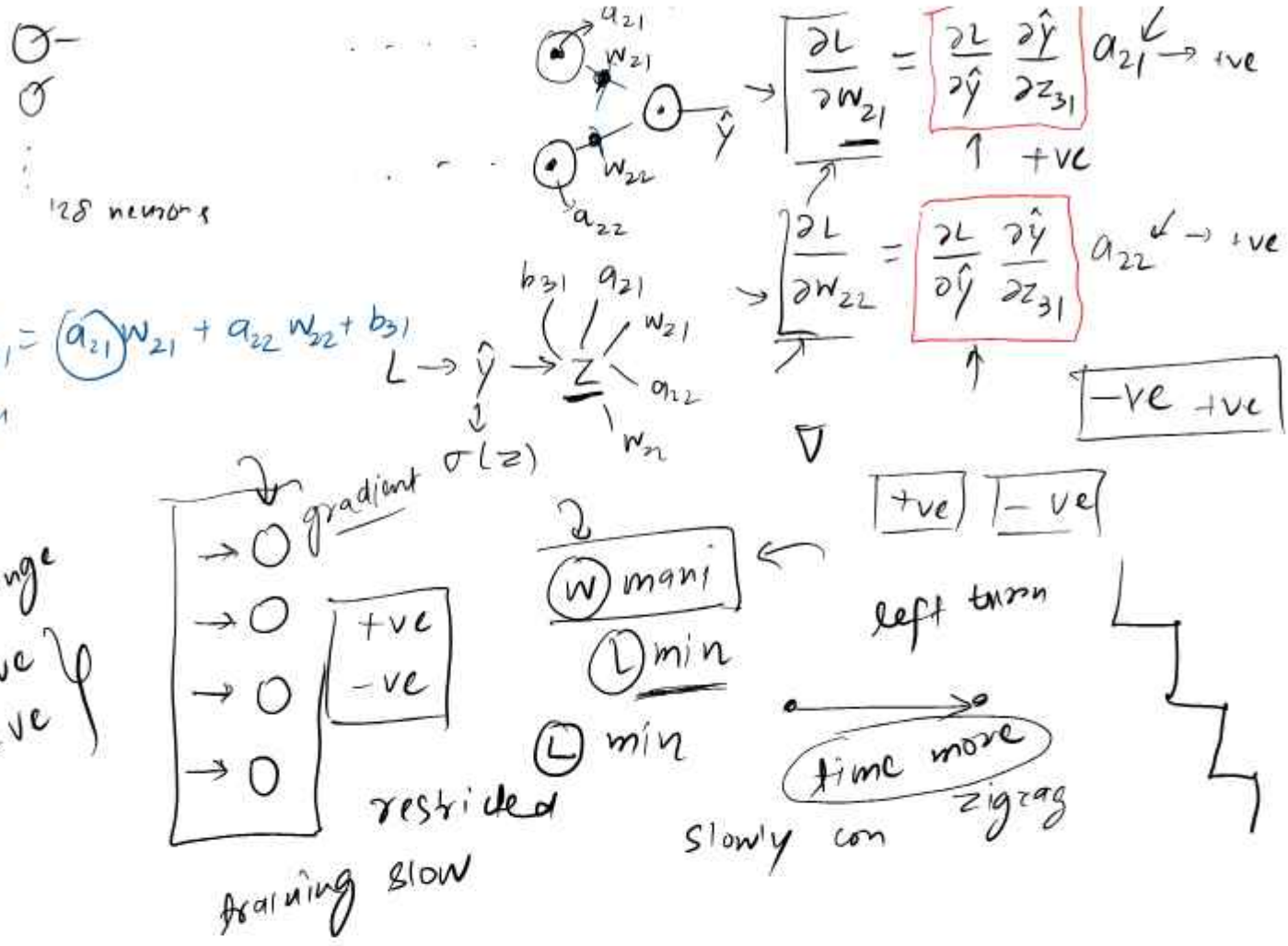
Convergence problem

+ve -ve

$0 \rightarrow 1$



$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} a_{21}$$ → +ve



3) $\sigma(x) = \frac{1}{1+e^{-x}}$ → computation time

exp

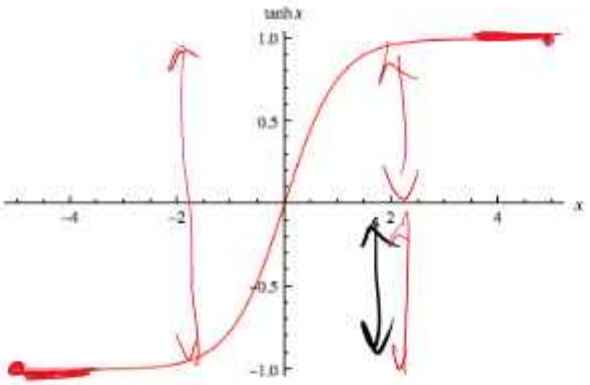
hidden layer → sigmoid → relu

output → b cp

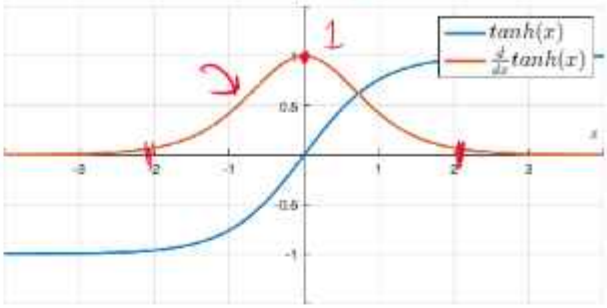
Tanh Activation Function

31 May 2022 14:50

(0,1) (-1,1)



0.65



$$f(x) = \frac{(e^{x} - e^{-x})}{ex + e^{-x}}$$

$$f'(x) = (1 - \tanh^2(x))$$

Advantages

1) Non-linear
2) Differentiable
3) zero centerid (+ve, -ve) training faster sigmoid

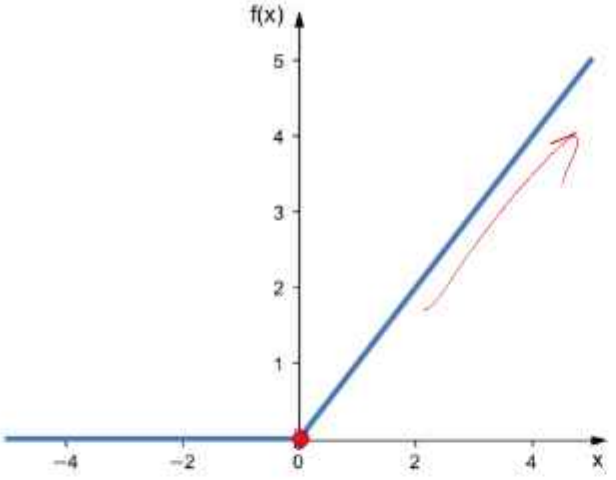
Disadvantage

→ saturating function → vanishing gradient prob

→ computationally exp [exp] slow

Relu Activation Function

01 June 2022 16:43



Most activation $f(x) = \max(0, x)$ $f(x) = x$

Advantage

$f(x) = \max(0, x+1) - \max(0, x-1)$

1) Non-linear
2) Not saturated in the +ve region
3) Computationally inexpensive
4) Converge → faster → sigmoid, ↘ tanh

Disadvantage

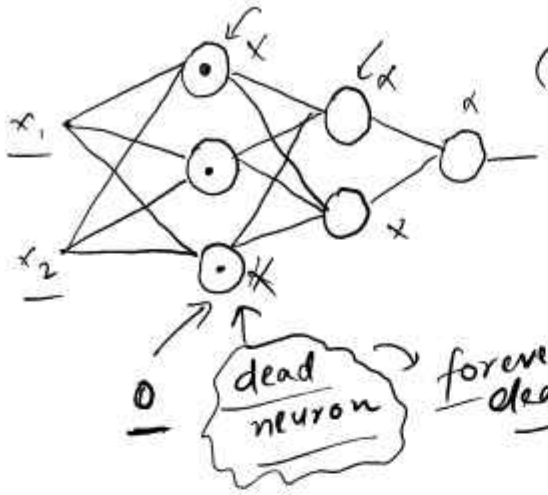
→ Differentiable $x < 0 \rightarrow 0$, $x \geq 0 \rightarrow 1$

→ Non Zero centered → sigmoid → Batch Normalization normalize →

↳ Dying ReLU problem

Dying Relu Problem

08 June 2022 22:53



Relu → Vanishing grad

↳ Best choice hidden layers

→ Dying Relu → 50% >

0 → dead neuron → forever dead (0)

100% dead → worst

modified data → low level regress

$a_1 = \max(0, z_1) = 0$

$z_1 = w_1x_1 + w_2x_2 + b_1 < 0$

$\frac{\partial a_1}{\partial z_1} = 0$

+ve

regression

$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1}$ −ve +ve

$w_2 = w_2 - \eta \frac{\partial L}{\partial w_2}$ −ve

$L \to \hat{y} \to b_2, w_3, a_1 \to z_1 \to w_1, w_2, x_1, x_2, b_1$

$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_1} \frac{\partial a_1}{\partial z_1} \frac{\partial z_1}{\partial w_1}$

$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_1} \frac{\partial a_1}{\partial z_1} \frac{\partial z_1}{\partial w_2}$

dying relu

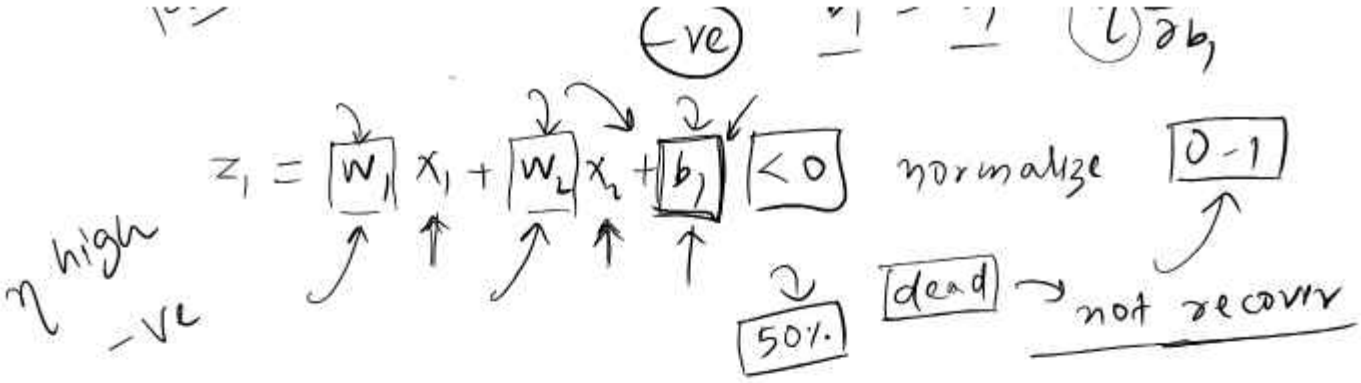
$z_1 = w_1x_1 + w_2x_2 + b_1 < 0$ (−ve)

$b_1 << 0$ (−ve)

dead

1) High learning rate (η)
2) High −ve bias

$b_1 = b_1 - \eta \frac{\partial L}{\partial b_1}$ (−ve)

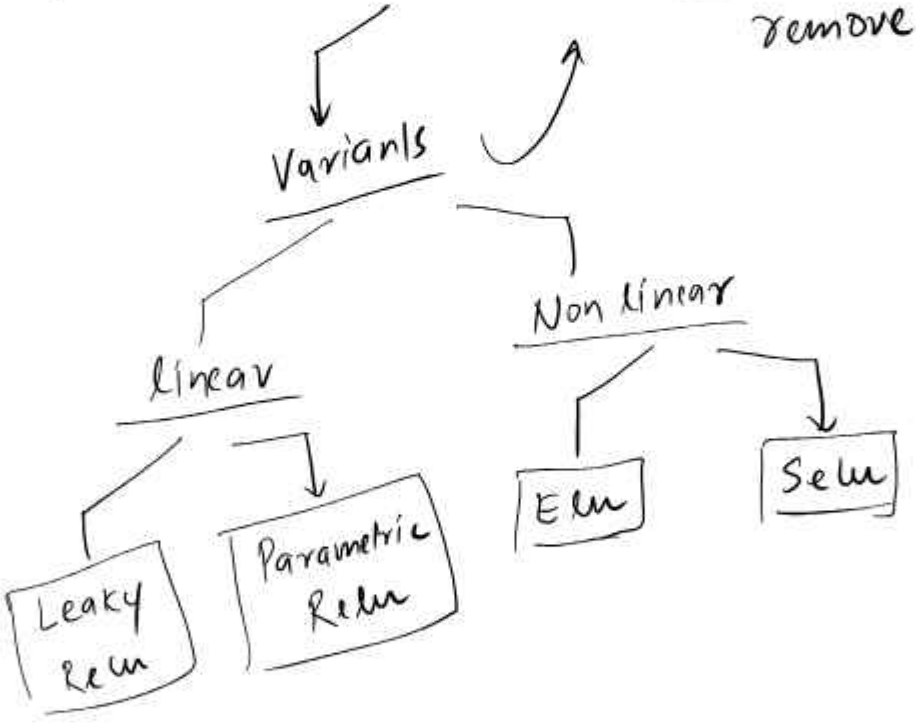


Solutions

→ Set low learning rate

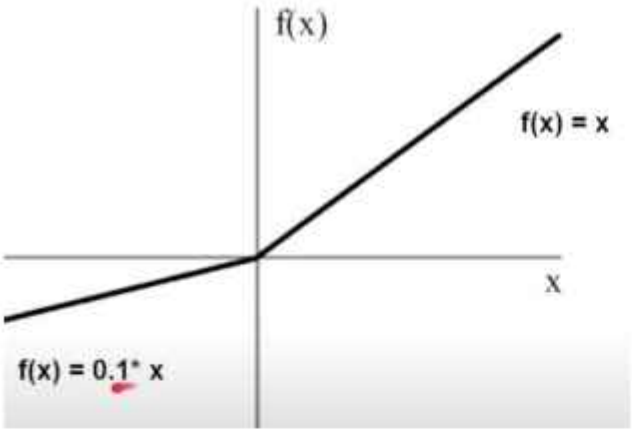
→ bias → +ve value → 0.01

→ Don't use relu → variants (+ve keep) / remove



Leaky Relu

08 June 2022 22:53



$f(z) = \max(0.01z, z)$

$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1}$

$z \geq 0 \rightarrow z$

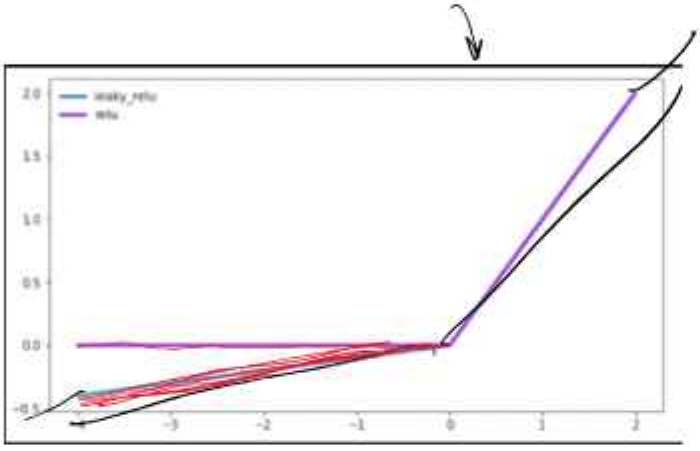
$z < 0 \rightarrow \frac{1}{100} z$ (fraction of z)

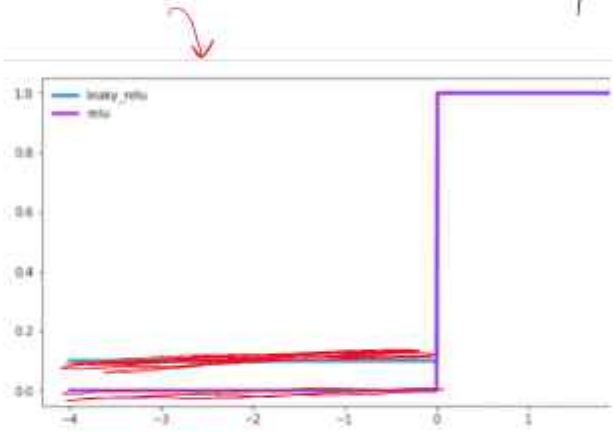
-ve (z) $\rightarrow \neq 0$ [0.01] z

↳ derivative

$f'(z)$ $z \geq 0 \rightarrow$ [1]

$z < 0 \rightarrow$ [0.01]





Advantages

→ Non-saturated → Unbounded

→ Easily computed

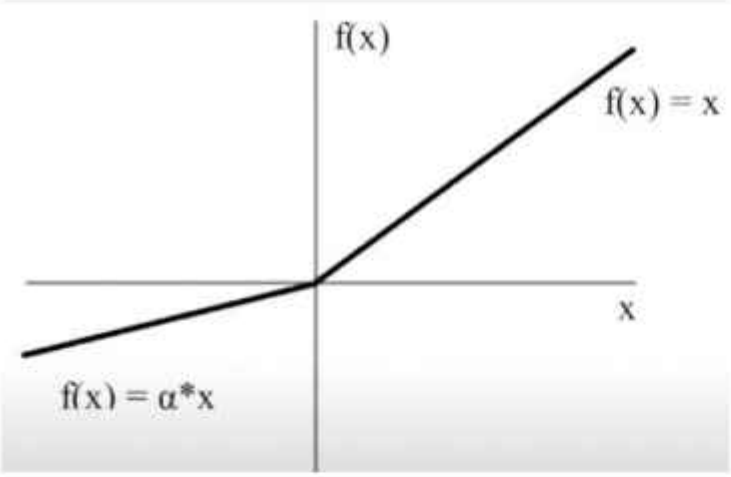
→ No dying relu problem

→ Close to 0 centered (-ve/+ve)

Disadv

Parametric Relu

08 June 2022 22:53



$$f(x) = \begin{cases} x & \text{if } x > 0 \\ ax & \text{otherwise} \end{cases}$$

$a \rightarrow$ trainable parameter

$0.01x$

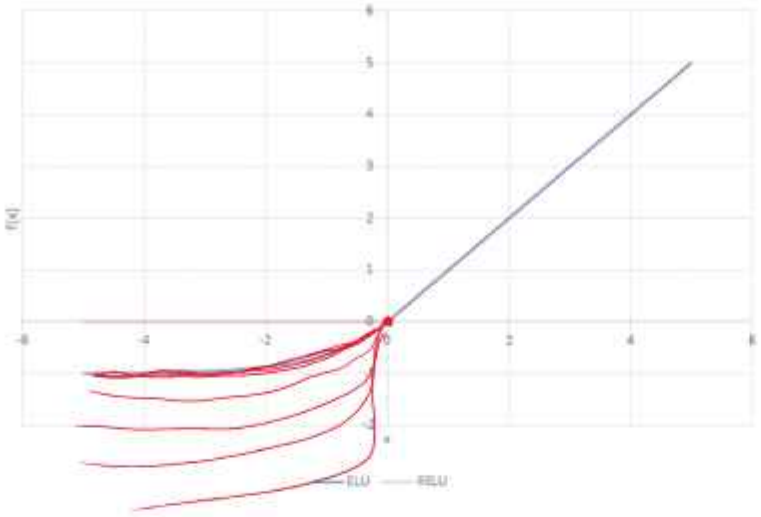
depend on data

flexibility

Elu - Exponential Linear Unit

09 June 2022 00:29

performance better Relu

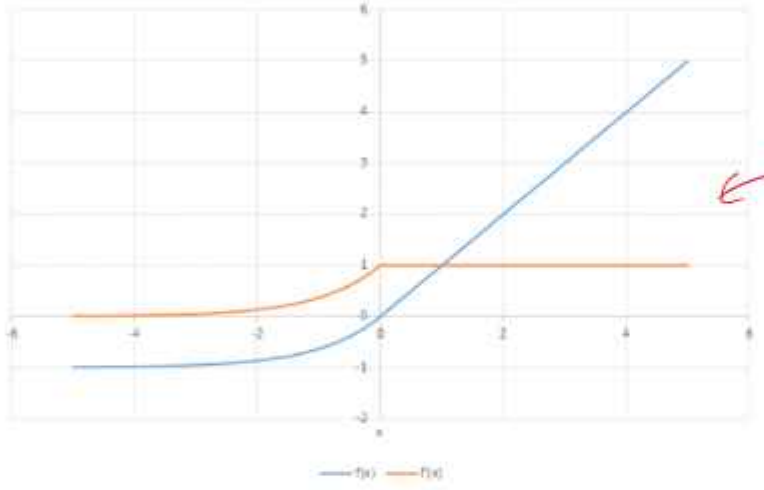


$$\mathrm{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x < 0 \end{cases}$$

constant

0.1 to 0.3

$$\mathrm{ELU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \mathrm{ELU}(x) + \alpha & \text{if } x \leq 0 \end{cases}$$



Advan

→ Close zero centered
↳ convergence faster

→ Better generalized

→ Dying relu

→ Always continous as well diffen

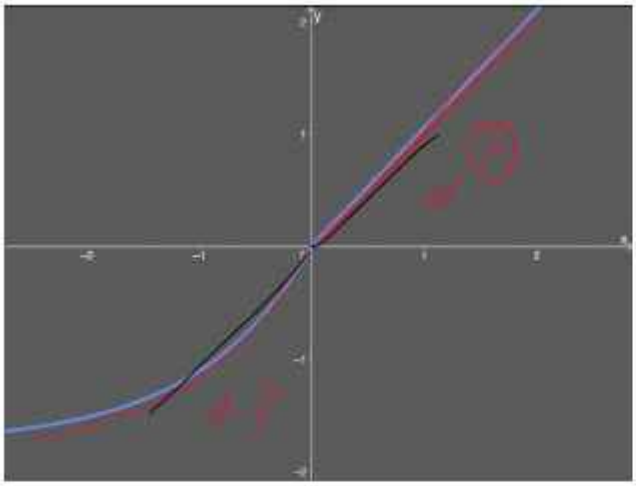
Disadv

→ computation expensive (ex)

Selu - Scaled Exponential Linear Unit

09 June 2022 00:29

Recent → new → 90+ pages



$$\text{SELU}(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ \alpha e^x - \alpha & \text{if } x \leq 0 \end{cases}$$

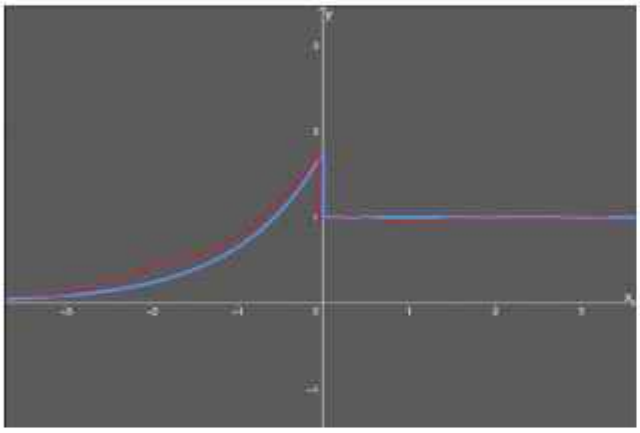
$$a \approx 1.67326324235437728481704299167 17$$

$$\lambda \approx 1.05070098735548049341933498529 46$$

→ fixed ✓

trainbu α

$$\text{SELU}'(x) = \lambda \begin{cases} 1 & \text{if } x > 0 \\ \alpha e^x & \text{if } x \leq 0 \end{cases}$$



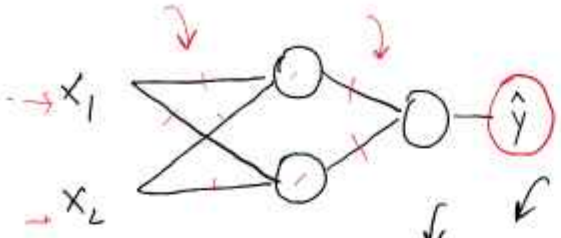
self-normalizing → activation → normalized

$m = 0 \quad \sigma = 1$

converge faster

Why Weight Initialization is Important?

22 June 2022 12:48



→ Vanishing GP
→ Exploding GP
→ Slow convergence

Wrong WI techniques

↓

What to do?

2000's Deep Learning

1. Initialize the parameters
2. Choose an *optimization algorithm*
3. Repeat these steps:
   1. Forward propagate an input
   2. Compute the cost function
   3. Compute the gradients of the cost with respect to parameters using backpropagation
   4. Update each parameter using the gradients, according to the optimization algorithm

Sigmoid Activ

Wrong weight init

What not to do?

22 June 2022 12:49

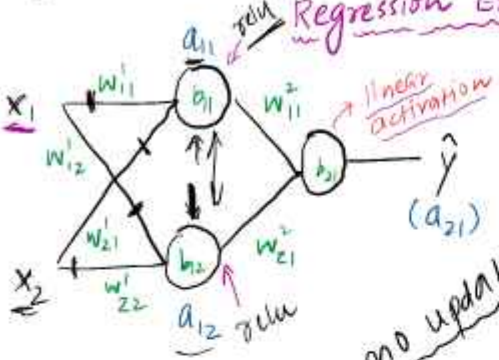
Case1 → Zero Initialization

relu
tanh
sigmoid

$W = 0, b = 0$

$a_{11} = \frac{e^{z_{11}} - e^{-z_{11}}}{e^{z_{11}} + e^{-z_{11}}} = 0$

Regression Example



$a_{11} = \max(0, z_{11}) = 0$

$z_{11} = w_{11}^1 x_1 + w_{21}^1 x_2 + b_{11} = 0$

$a_{12} = \max(0, z_{12}) = 0$

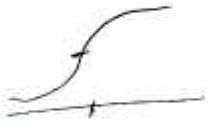
$z_{12} = w_{12}^1 x_1 + w_{22}^1 x_2 + b_{12} = 0$

$a_{11} = a_{12}$ → equal and 0

no update

$$w_{11}^1 = w_{11}^1 - \eta \frac{\partial L}{\partial w_{11}^1}$$

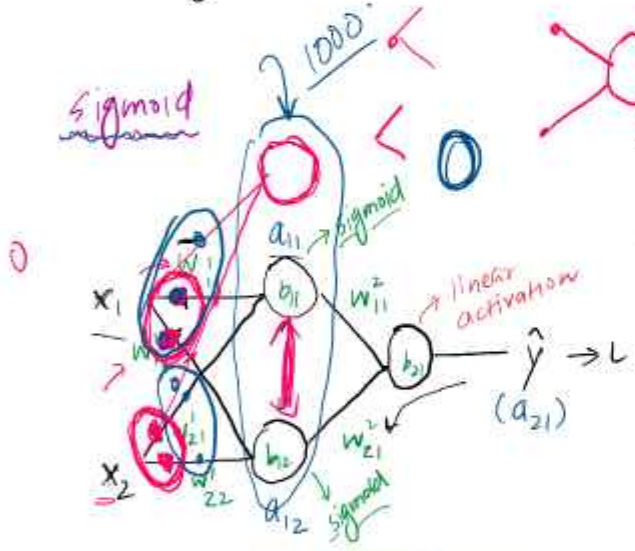
$W = 0$ → No training will place



0

Non-linear prop.

$$w = w - \eta \frac{\partial L}{\partial w}$$



$a_{11} = \sigma(z_{11}) = 0.5$

$z_{11} =$

$a_{12} = 0.5$

$a_{11} = a_{12}$

$a_{11} = a_{12}$

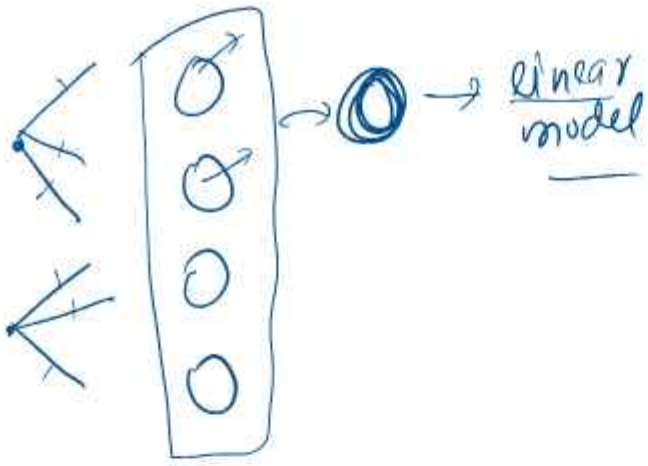
$z_{11} = z_{12}$

$$\frac{\partial L}{\partial w_{11}^1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{11}} \frac{\partial a_{11}}{\partial z_{11}} x_1$$

$$\frac{\partial L}{\partial w_{12}^1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{12}} \frac{\partial a_{12}}{\partial z_{12}} x_1$$

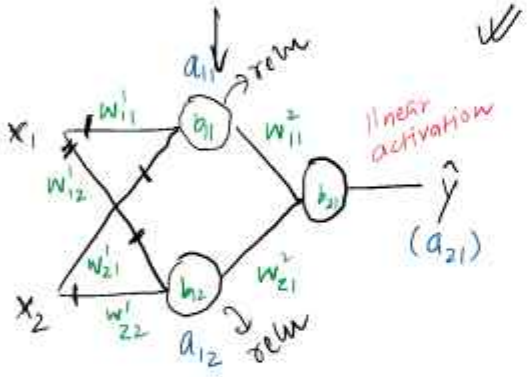
$$\frac{\partial L}{\partial w_{21}^1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{11}} \frac{\partial a_{11}}{\partial z_{11}} x_2$$

$$\frac{\partial L}{\partial w_{22}^1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{12}} \frac{\partial a_{12}}{\partial z_{12}} x_2$$



Case2 → Non-0 constant value

$w = 0.5 \quad b = 0.5$ sigmoid → 0



$a_{11} = \max(0, z_{11})$ → non-zero

$z_{11} = w^1_{11} x_1 + w^1_{21} x_2 + b_{11} \neq 0$

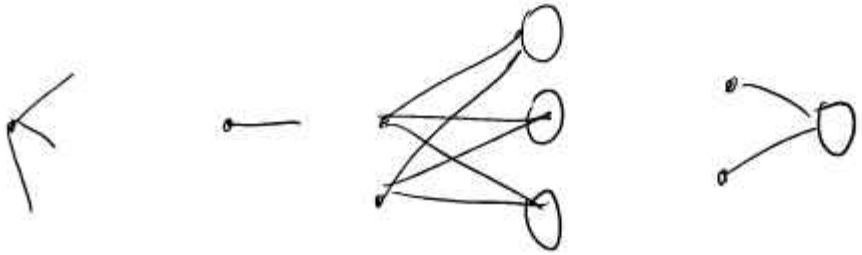
$a_{12} = \max(0, z_{12}) \neq 0$

$z_{12} = w^1_{12} x_1 + w^1_{22} x_2 + b_{12} \neq 0$

$z_{12} = z_{11}$ $\quad \boxed{a_{11} = a_{12}}$

$$\boxed{\frac{\partial L}{\partial w^1_{11}}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{11}} \frac{\partial a_{11}}{\partial z_{11}} x_1 \qquad \boxed{\frac{\partial L}{\partial w^1_{12}}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{12}} \frac{\partial a_{12}}{\partial z_{12}} x_1$$

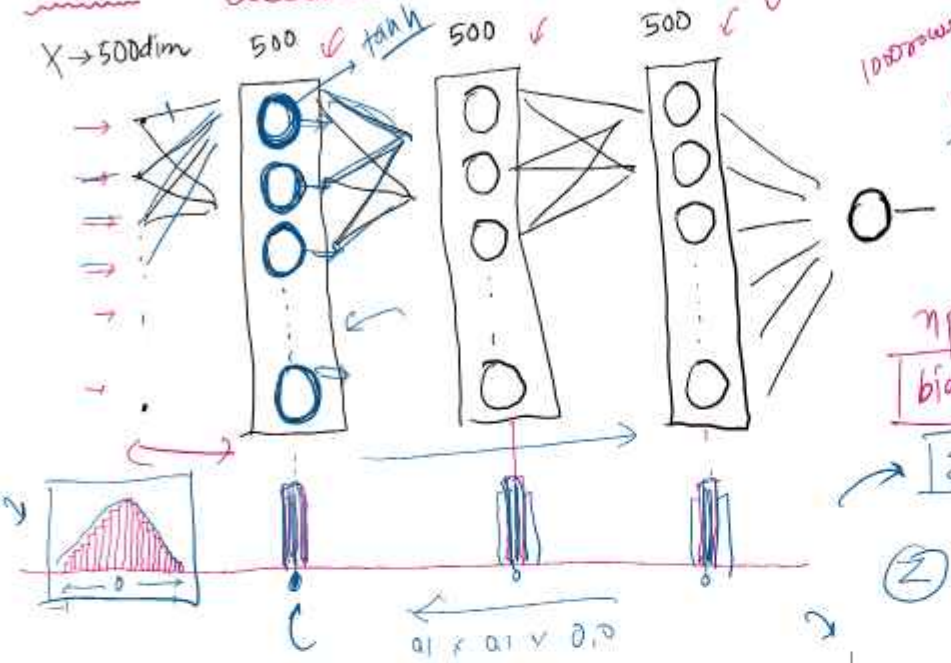
$$\frac{\partial L}{\partial w^1_{21}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{11}} \frac{\partial a_{11}}{\partial z_{11}} x_2 \qquad \frac{\partial L}{\partial w^1_{22}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{12}} \frac{\partial a_{12}}{\partial z_{12}} x_2$$

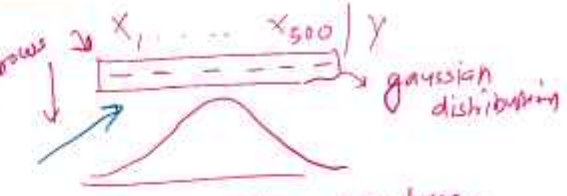


of random init ϕ

VGP sigmoid

Case 3 – Random Initialization → small / large → tanh / sigmoid (1000, 500)





weight init → random
↘ small values

np.random.randn (500, 500) * 0.01

bias = 0

x_i → 0-1 → -1 to 1

$\sum w_i x$ → 0.0007

small number tanh

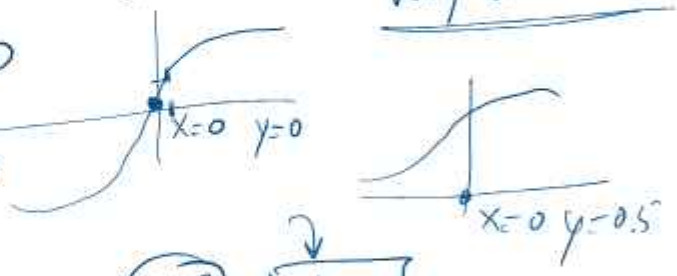
(Z) is a small

0.1 × 0.1 ∨ 0.2

small small = 0

very close to 0

$W = W - \eta \frac{\partial L}{\partial W} = 0$



vanishing gradient problem

VGP → deep

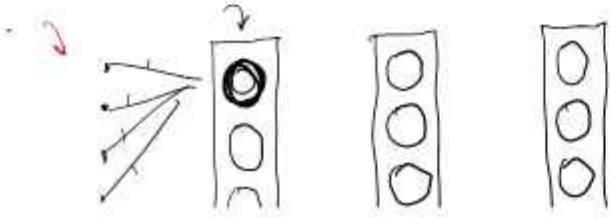
small weights → 0.0007

Relu → small values → slow ← convergence

tanh → VGP

sigmoid → VGP

Random Initialization (Large values)

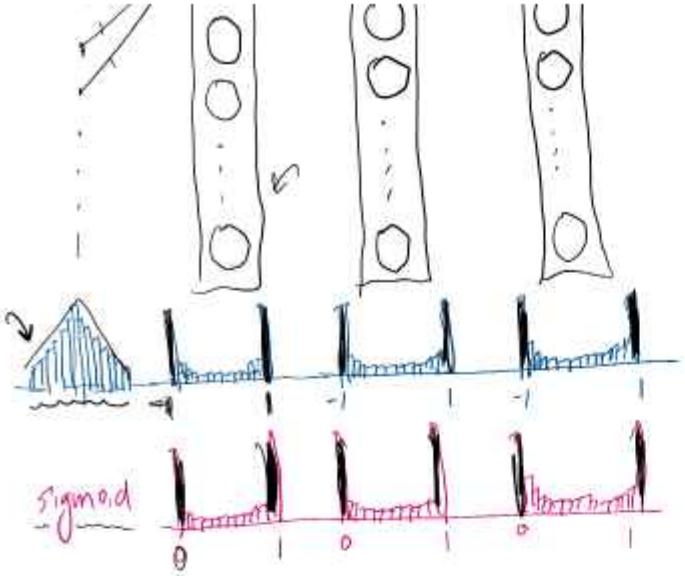


np.random.randn (500, 500) → [0-1]

$\sum w_i x_i$ → 500 → 100 → 500

x_i: 0-1

tanh / sigmoid



0-1 tanh/sigmoid

$g(z)$ → Saturation

$\sum w_i x_i \rightarrow 100$ Saturate

$\sum w_i x_i \rightarrow$ Relu → 250 → 250

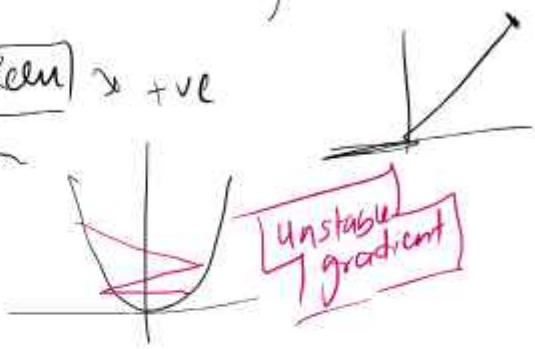
↓ gradients

↓ Slow training

↓ Vanishing gradient problem

$$w = w - \eta \frac{\partial L}{\partial w}$$

Relu → +ve



Unstable gradient

→ Zero ✗

→ non-Zero same ✗

→ small random ✓

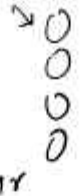
→ large random ✗

Key Insights

22 June 2022 12:49

Things not to do—

1) Zero Initialization → no training
2) Non-zero constant initialization → linear
3) Random initialization with small weights → vanish
4) Random initialization with large weights → exploding





np.random.randn(shape) * 0.01

(2,2)

-0.01 to 0.01

Vanishing grad

Slow convergence

random

np.random.randn(2,2)

-3 to 3

What can be done

→ Heuristics (Jugad) → practical solution

Xavier/Glorot Init (tanh, sigmoid) → Normal / Uniform

He Init (relu) → Normal / Uniform

Keras

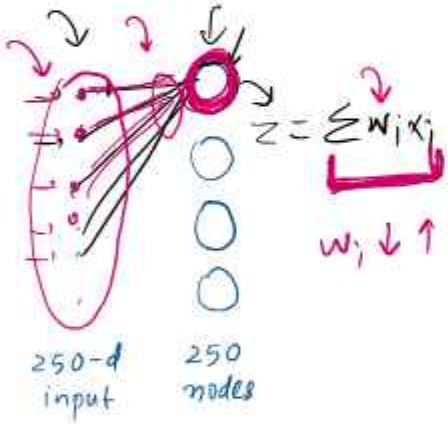
formulas

-1 to 1

0-1

-0.01 to 0.01

Intuition



$z = \sum w_i x_i$

$w_i \downarrow \uparrow$

small ←

np.random.randn(250,250) * 0.01

250

-250 to 250

large

np.random.randn(250,250) * 1 → -3 to 3

Variance = $\frac{1}{n} = \frac{1}{250}$ * $\sqrt{\frac{1}{250}}$ → sd

$n \rightarrow \uparrow \downarrow \frac{1}{n} \downarrow \uparrow w$

Xavier init (Normal)

$\sqrt{\frac{1}{fan_in}}$ ← # of inputs coming to the node

np.random.randn(2,2) * $\sqrt{\frac{2}{2}}$

$\sqrt{\frac{2}{fan_in + fan_out}}$

He Normal — relu

$$\sqrt{\frac{2}{fan_in}}$$

Uniform Distribution

Xavier Uniform

$[-limit, limit]$

$$limit = \sqrt{\frac{6}{(fan_in + fan_out)}}$$

Keras

He Uniform

$[-limit, limit]$ $\quad limit = \sqrt{\frac{6}{fan_in}}$

Weight Initialization Techniques

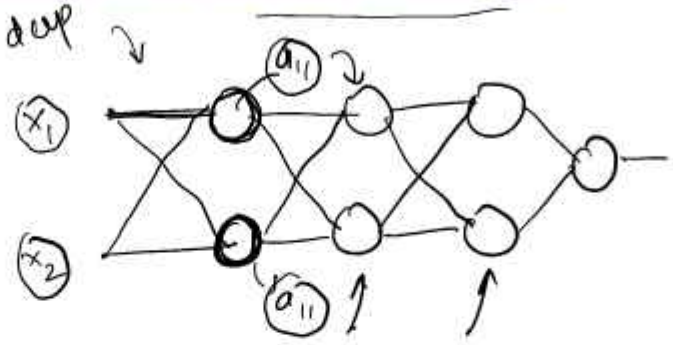
22 June 2022 12:49

What is Batch Norm?

27 June 2022 11:00

Batch-Normalization (BN) is an algorithmic method which makes the training of Deep Neural Networks (DNN) faster and more stable.

↪ It consists of normalizing activation vectors from hidden layers using the mean and variance of the current batch. This normalization step is applied right before (or right after) the nonlinear function.

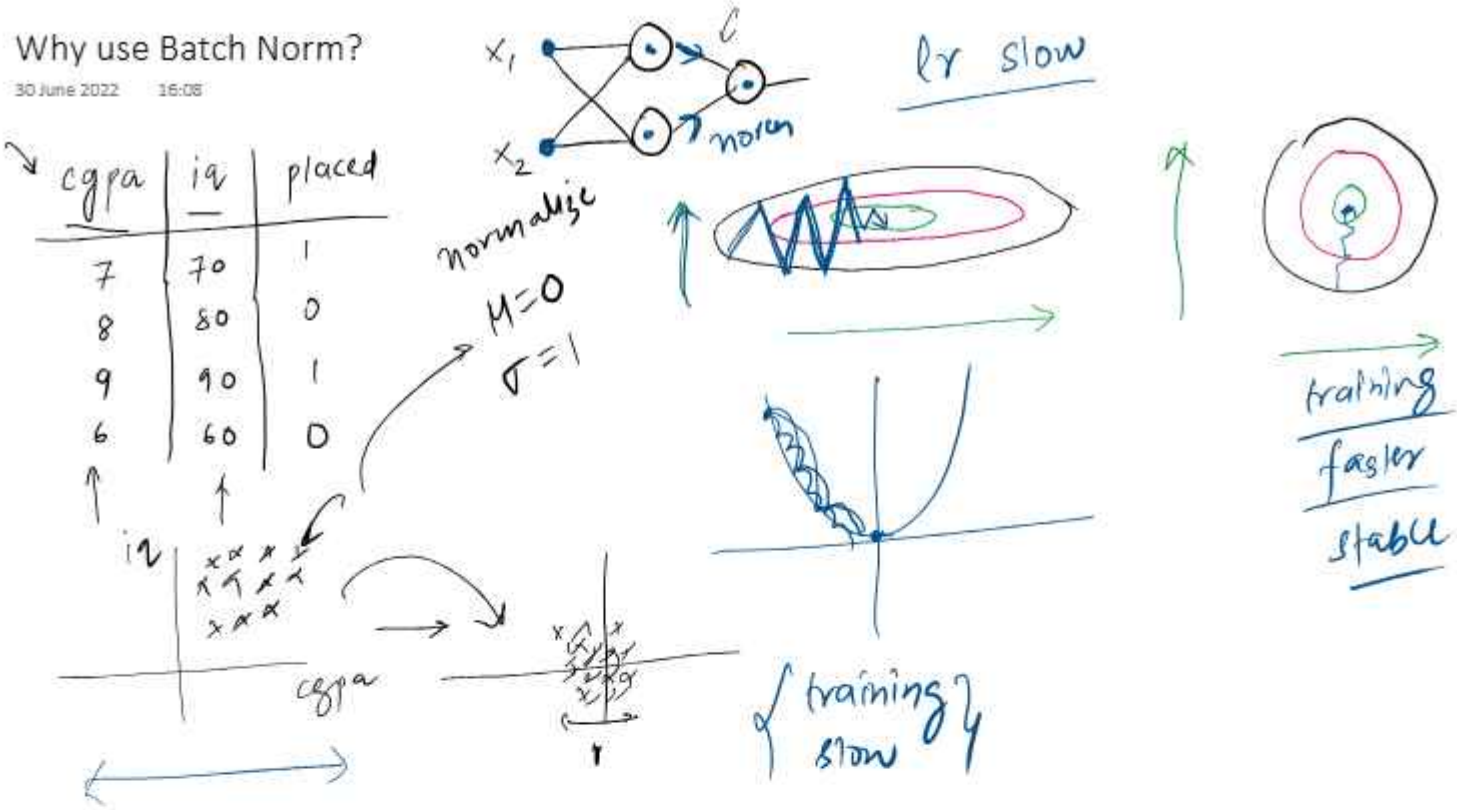


normalized
m=0
sd=1

normalize
m=0 sd=1

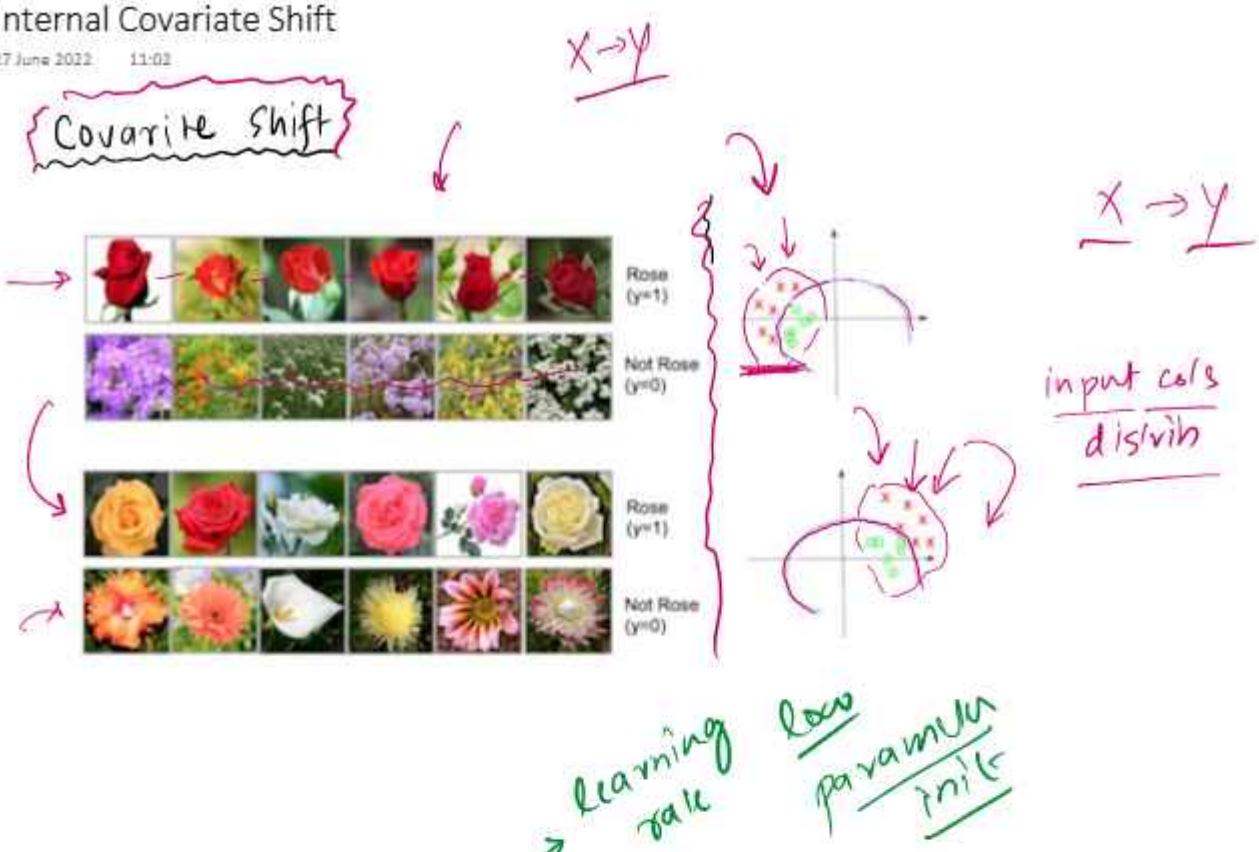
Why use Batch Norm?

30 June 2022 16:08



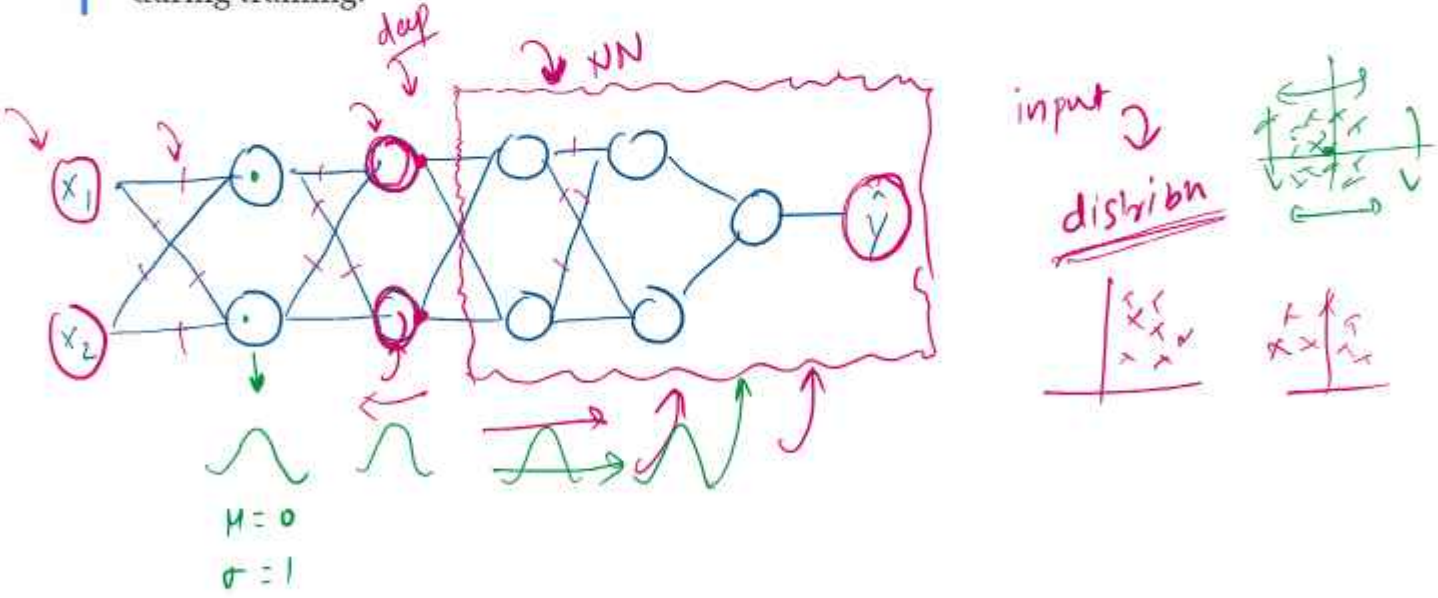
Internal Covariate Shift

27 June 2022 11:02



The authors' precise definition is:

> We define Internal Covariate Shift as the change in the distribution of network activations due to the change in network parameters during training.





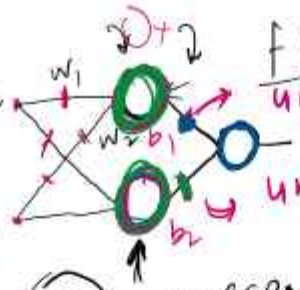

cheese
cheese
cheese

Batch Norm - The How

27 June 2022 11:02

cgpa	iq	placed
8.9	100	1
6.2	89	0
9.1	91	0
7.7	76	1
⋮	⋮	⋮
6.7	91	0

batch_size : 4

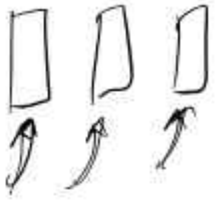


fp → 4 numbers

4 numbers → normalize $\mu=0$ $\sigma=1$

→ Mini-Batch GD

→ Layer by layer



$z_{11} = w_1 \text{cgpa} + w_2 \text{iq} + b$

$g(z_{11}) = a_{11}$

(4,2) * (2,2) + (1,2) → (4,2)

(1) $z_{11} \rightarrow z_{11}^N \rightarrow g(z_{11}^N) = a_{11}$

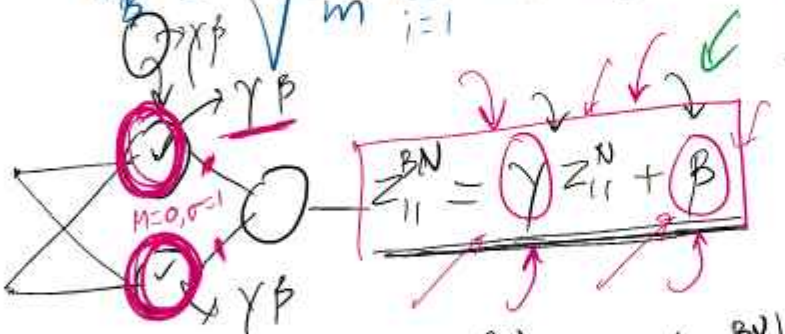
(2) $z_{11} \rightarrow g(z_{11}) = a_{11} \rightarrow a_{11}^N$

$$\frac{z_{11} - \mu}{\sigma} = z_{11}^N \quad (4,2)$$

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} z_{11}^i \qquad m = 4$$

$$z_{11}^i = \frac{z_{11}^i - \mu_B}{\sigma_B + \epsilon}$$

$$\sigma_B = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(z_{11}^i - \mu_B)}$$



0-1 $\mu=0$ $\sigma=1$

learnable parameters w, b

$\gamma=1$ $\beta=0$

$z_{11} \rightarrow z_{11}^N \rightarrow z_{11}^{BN} \rightarrow g(z_{11}^{BN}) = a_{11}$

$z_{11} \rightarrow$ normalize $\mu=0$ $\sigma=1$

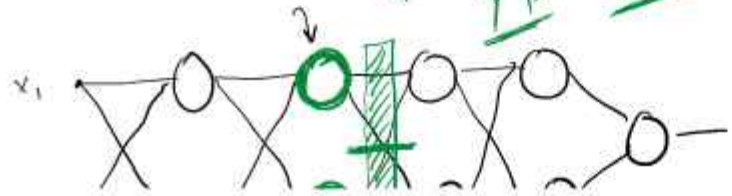
$\gamma = \sigma + \epsilon$ $\beta = \mu$

flexible

$z_{11} \rightarrow z_{11}^N \rightarrow z_{11}^{BN}$

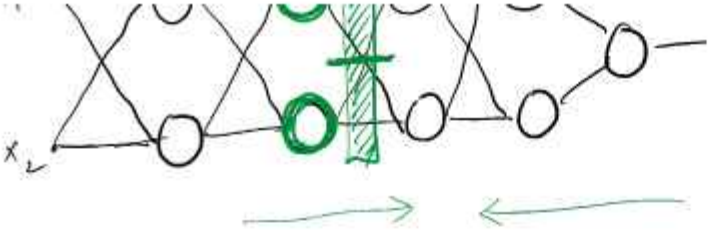
Batch Norm → Layer

4 learnable $\gamma\beta$ $\gamma\beta$



$$\gamma = \gamma - \eta \frac{\partial L}{\partial \gamma}$$

Keras

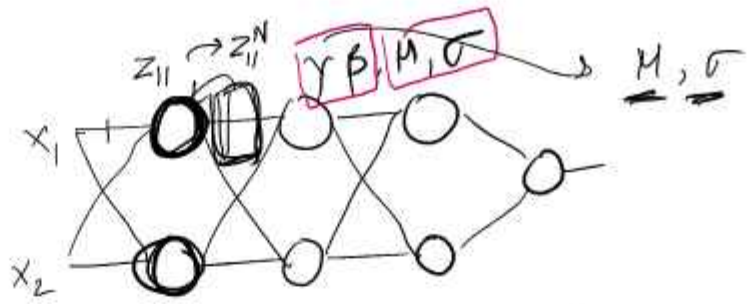


Batch Norm during test

27 June 2022 11:03

cgpa	iq	placed
8	80	1
7	70	0
6	60	1
⋮	⋮	⋮
9	90	1

100 students



testing / query point

Exp weighted moving avg.

EWMA

25 times

b = 4

1 batch

2

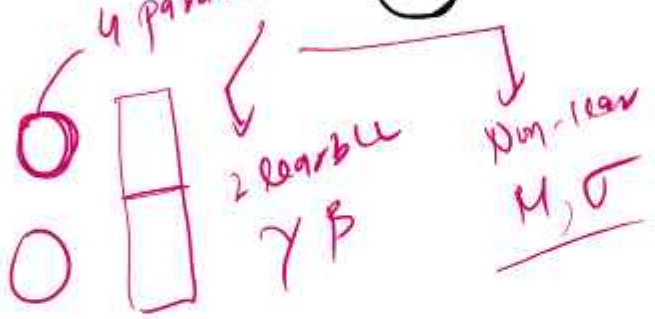
μ_{B1}, μ_{B2} . . . . μ_{B25} → μ

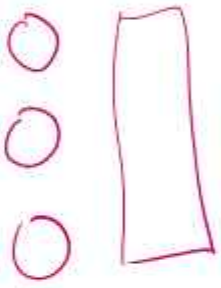
ex 7 WMA

4 parameters

2 learnable γ β

Non-learn μ, σ





$3 \times 4 = 12$

6 6

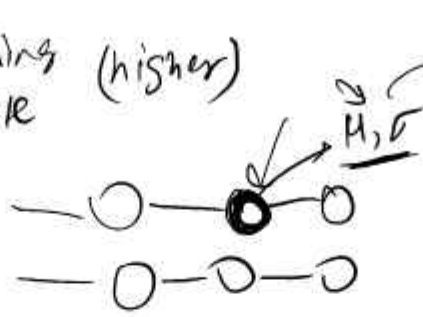
Advantages

27 June 2022 11:02

1) Stable → hyperpara → wider range of values

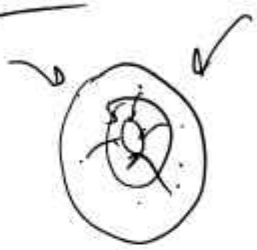
2) faster → learning rate (higher)

3) Regularizer → dropout



randomness ↓ rise

overfitting





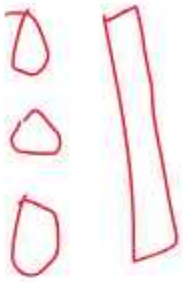
4) Weight init impact reduce

Keras Implementation

27 June 2022 11:03

```
model = Sequential()

model.add(Dense(3,activation='relu',input_dim=2))
model.add(BatchNormalization())
model.add(Dense(2,activation='relu'))
model.add(BatchNormalization())
model.add(Dense(1,activation='sigmoid'))
```

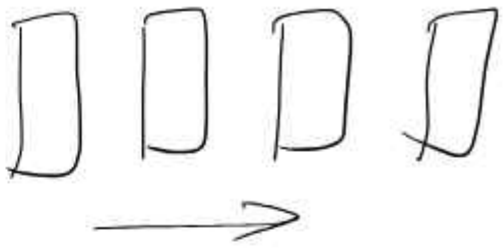


Introduction

05 July 2022 09:57

Performance of NN

↳ how the speed the training



{Optimizers} ✓

→ Weigh init

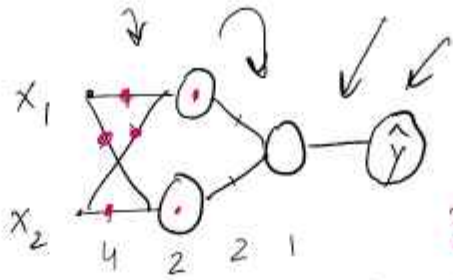
→ Batch Norm

→ Activation function

Role of Optimizer

05 July 2022 10:01

cgpa	iq	placed
8	80	1
9	90	1
7	70	0



9 parameters

2 weights

Optimization

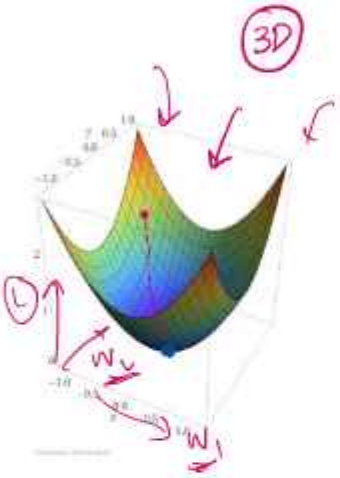
2 min

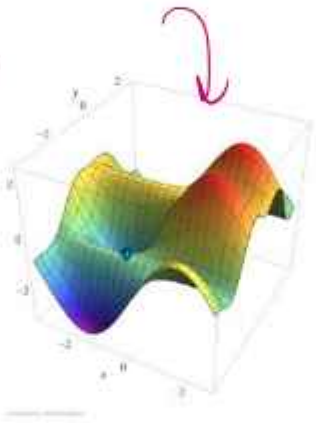
y $\hat{y}$ (w, b)

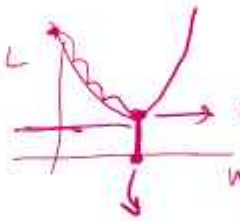
C/L()

9 para

(1) 3D L(-,-,-,-,-) → 10D







Gradient Descent

$$w_n = w_0 - \eta \frac{\partial L}{\partial w}$$

Types of Optimizers

05 July 2022 10:02

- Batch GD
- Stochastic GD
- Mini batch GD

(10)

epochs = (10)

$$W_n = W_0 - \eta \frac{\partial L}{\partial W}$$

500 rows → pred

↓

loss → weight update

10 × 500 → 5000

batch size = 100

5 batch → 10 × 5 times



Gradient Descent in Neural Networks | Batch vs...

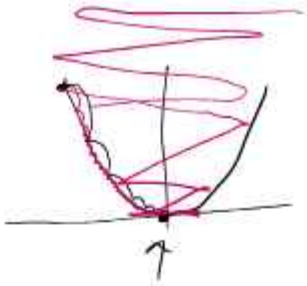
1.6K views • 2 months ago •

Challenges

05 July 2022 10:02

1) learning rate

$$w_n = w_0 - \eta \frac{\partial L}{\partial w_0}$$



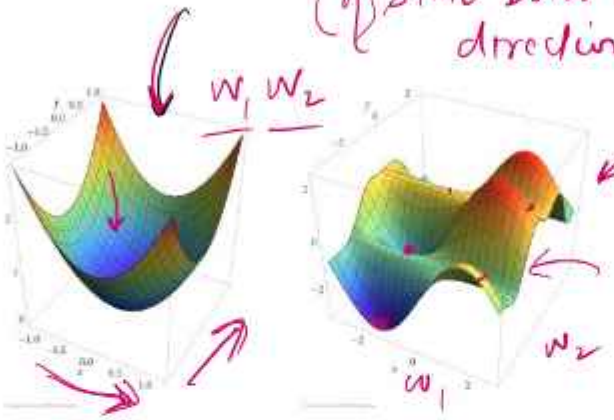
2) learning rate scheduling → pre define

3)

(1) same both the directions

(9) weights and bias 9 sep dim

10-D → 10 directions

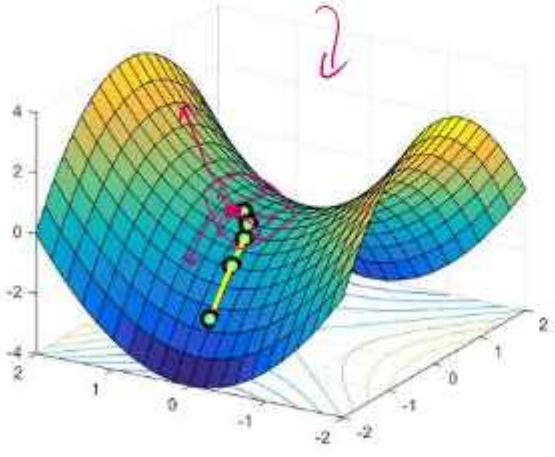


complex minima

sub optimum

4) local minima

5) saddle point

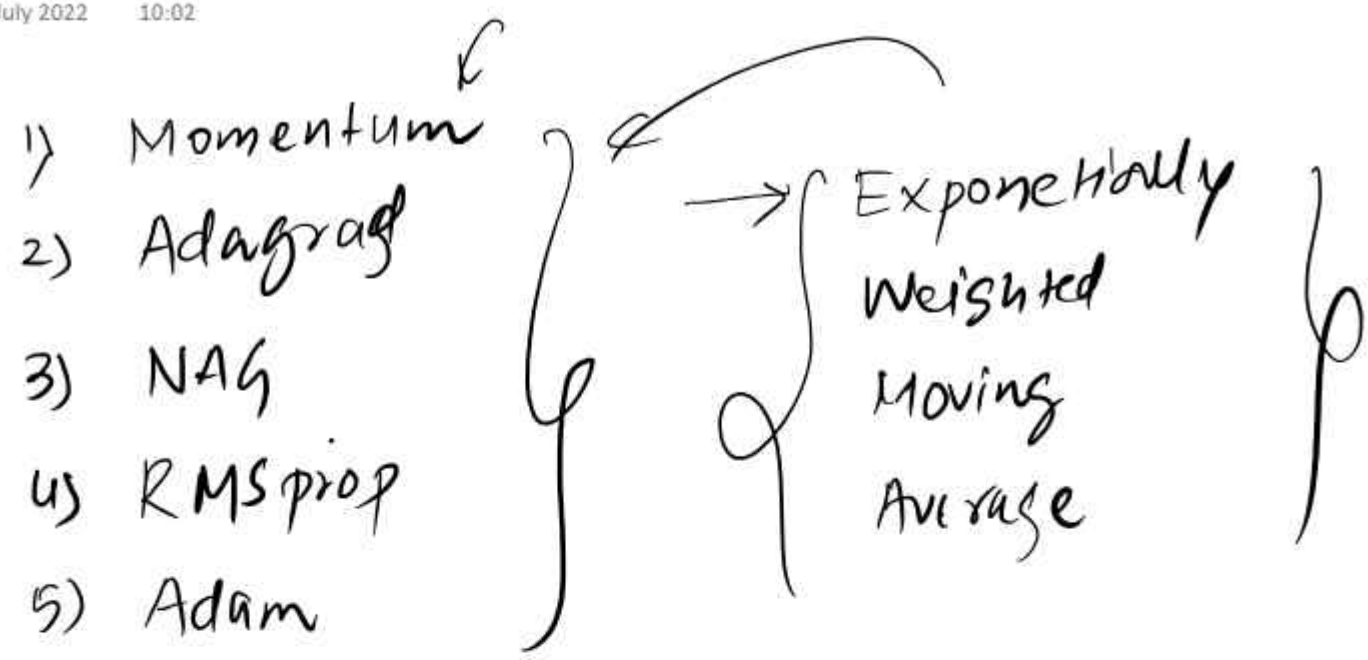


$$\frac{\partial L}{\partial w} = 0$$

$$w_n = w_0 - \eta \frac{\partial L}{\partial w}$$

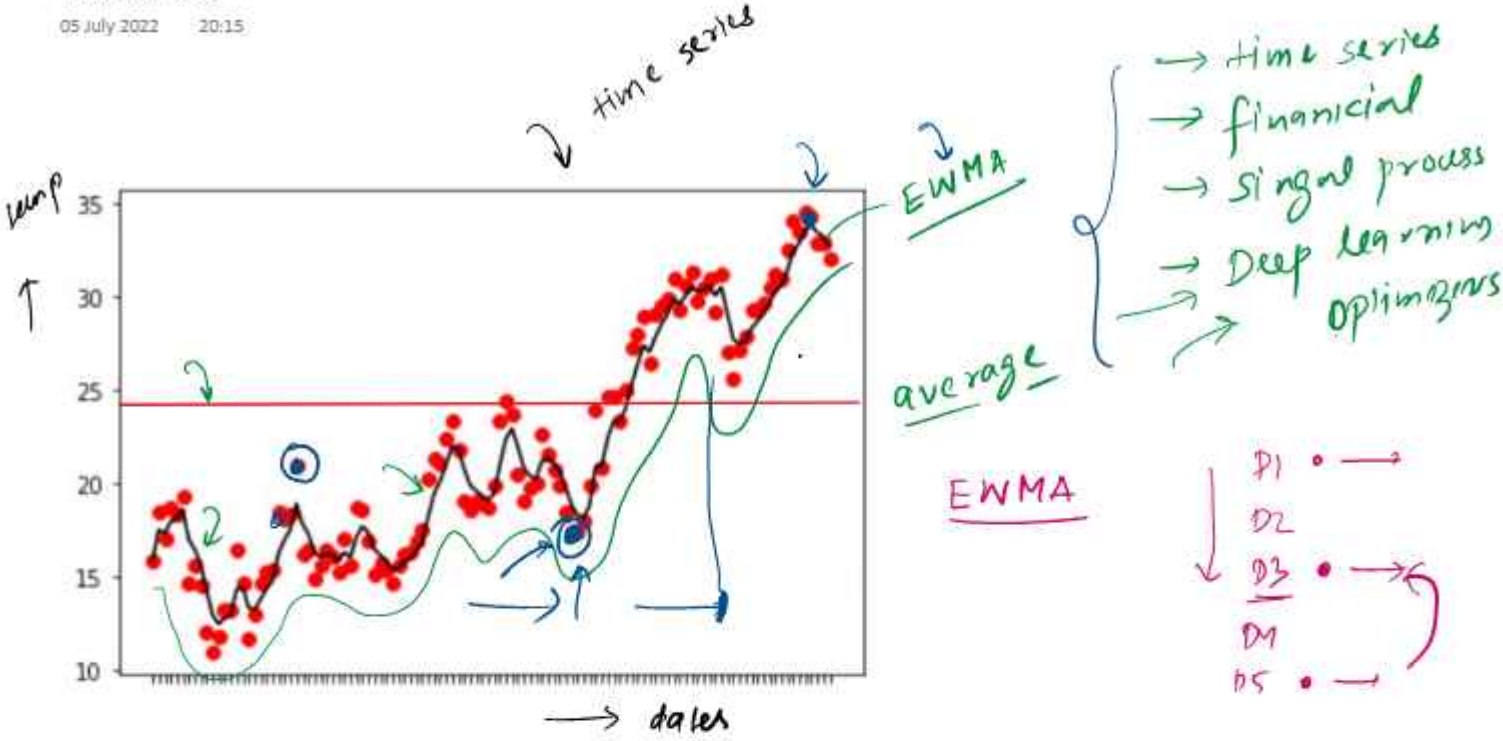
What next?

05 July 2022 10:02



The What

05 July 2022 20:15





Mathematical Formulation

05 July 2022 20:16

$$V_t = \beta V_{t-1} + (1-\beta)\theta_t$$

$\beta \rightarrow 0 < \beta < 1$

index	temp (θ)
D1	25
D2	13
D3	17
D4	31
→ D5	43

$\beta = 0.9$

$V_0 = 0$ $\quad V_0 = \theta_0$

$V_1 = 0.9 \times V_0 + 0.1 \times 13$

$= 1.3$

$V_2 = 0.9 \times 1.3 + 0.1 \times 17 =$

$\beta = 0.5$

$\frac{1}{1-0.5} = \frac{1}{0.5}$

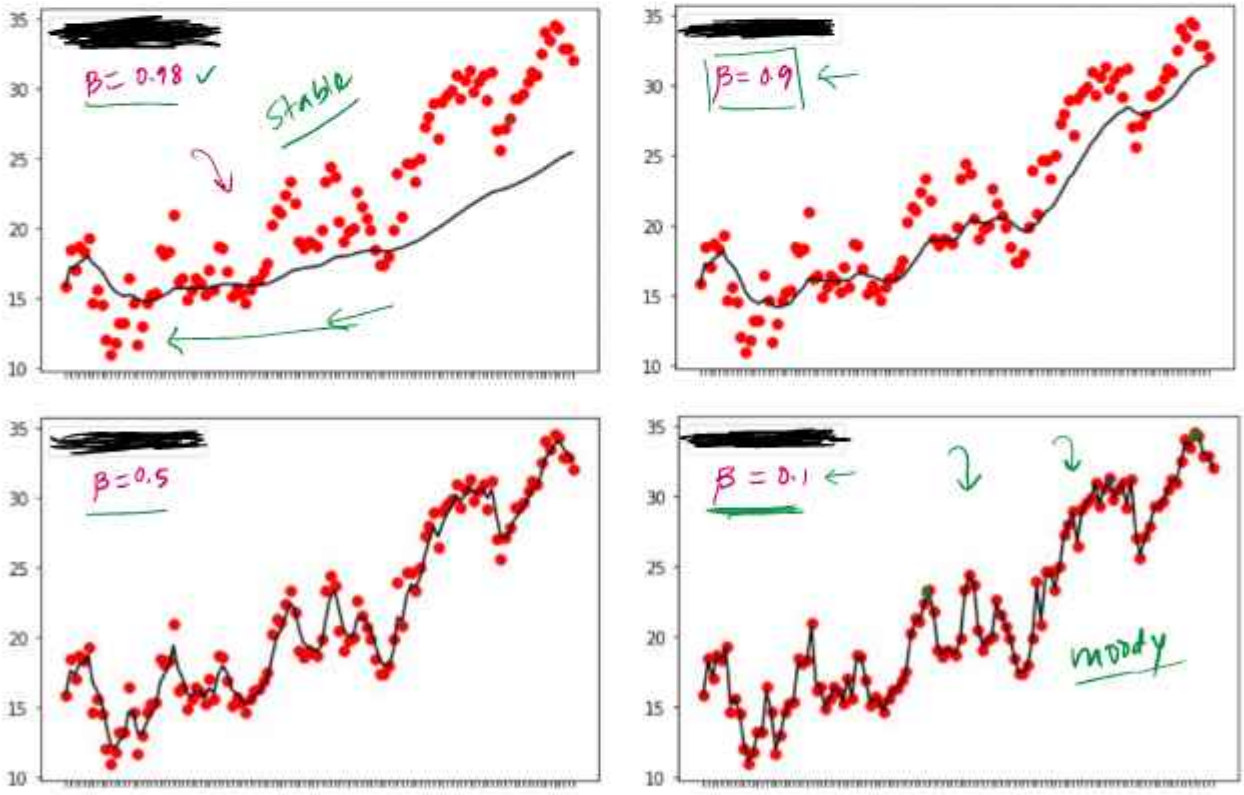
2 days

$\beta = 0.9$

$\beta \rightarrow \boxed{\frac{1}{(1-\beta)}}$

$\frac{1}{1-0.9} = \frac{1}{0.1} = 10$

Effect of β



Mathematical Intuition

$$V_t = \beta V_{t-1} + (1-\beta)\theta_t$$

$$V_4 = \beta V_3 + (1-\beta)\theta_4$$

$$V_4 = \beta^3(1-\beta)\theta_1 + \beta^2(1-\beta)\theta_2 + \beta(1-\beta)\theta_3 + (1-\beta)\theta_4$$

$$V_t = \beta V_{t-1} + (1-\beta)\theta_t$$

$V_0 = 0$

$V_1 = (1-\beta)\theta_1$

$V_2 = \beta V_1 + (1-\beta)\theta_2$

$= \beta(1-\beta)\theta_1 + (1-\beta)\theta_2$

$V_3 = \beta V_2 + (1-\beta)\theta_3$

$= \beta^2(1-\beta)\theta_1 + \beta(1-\beta)\theta_2 + (1-\beta)\theta_3$

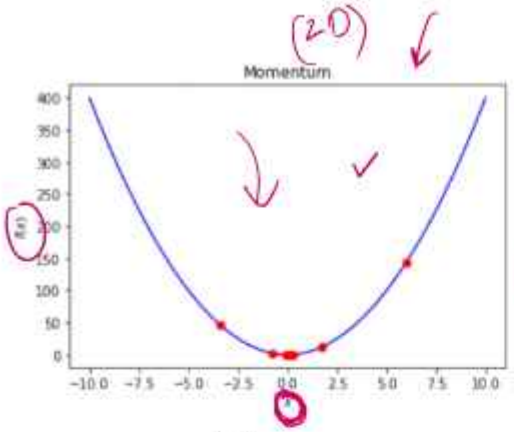
$V_4 = \beta^3(1-\beta)\theta_1 + \beta^2(1-\beta)\theta_2 + \beta(1-\beta)\theta_3 + (1-\beta)\theta_4$

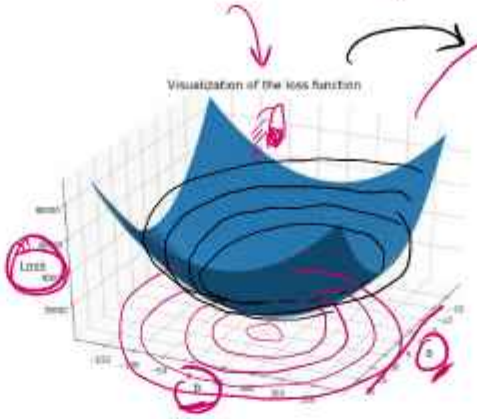
$= (1-\beta)[\beta^3\theta_1 + \beta^2\theta_2 + \beta\theta_3 + \theta_4]$

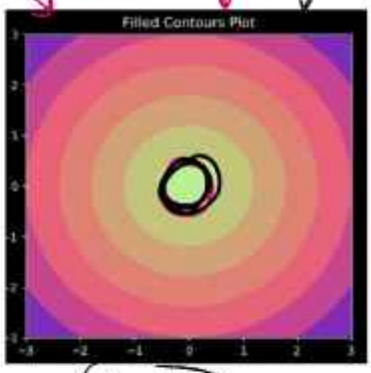
$\beta^3 < \beta^2 < \beta$ $\quad 0 < \beta < 1$

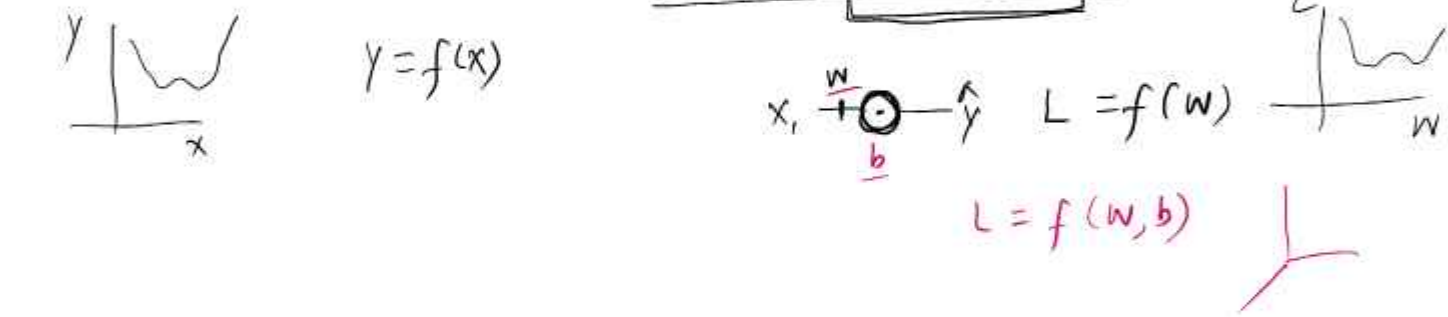
Understanding Graphs

20 July 2022 12:03



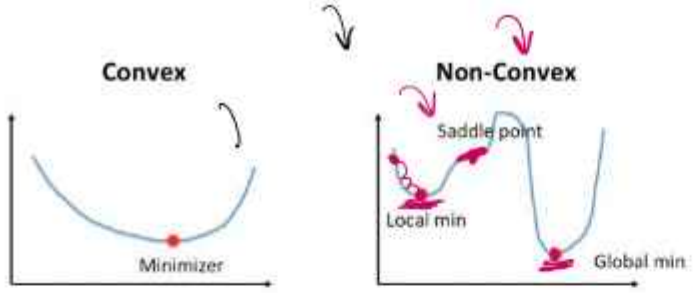






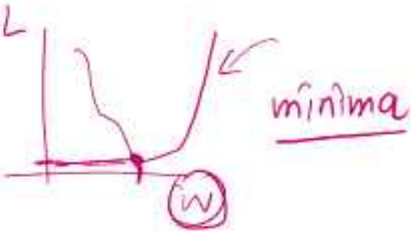
Convex Vs Non-Convex Optimization

20 July 2022 13:06

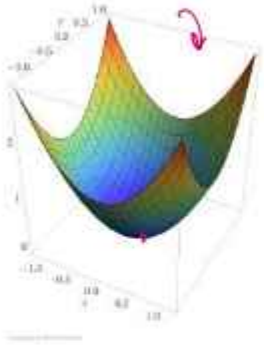


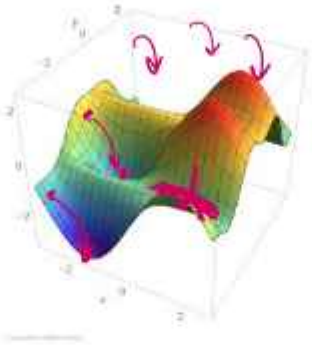


L (W)



minima





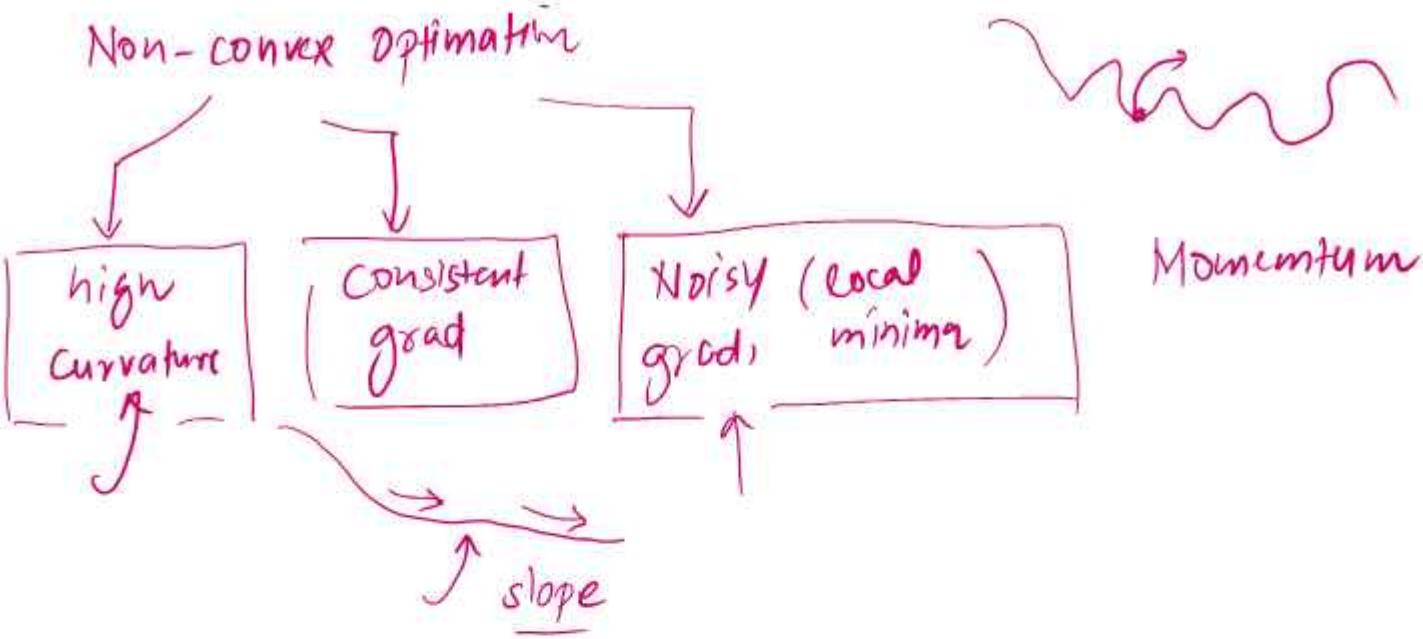
1) local minima
2) Saddle point
3) High curvature





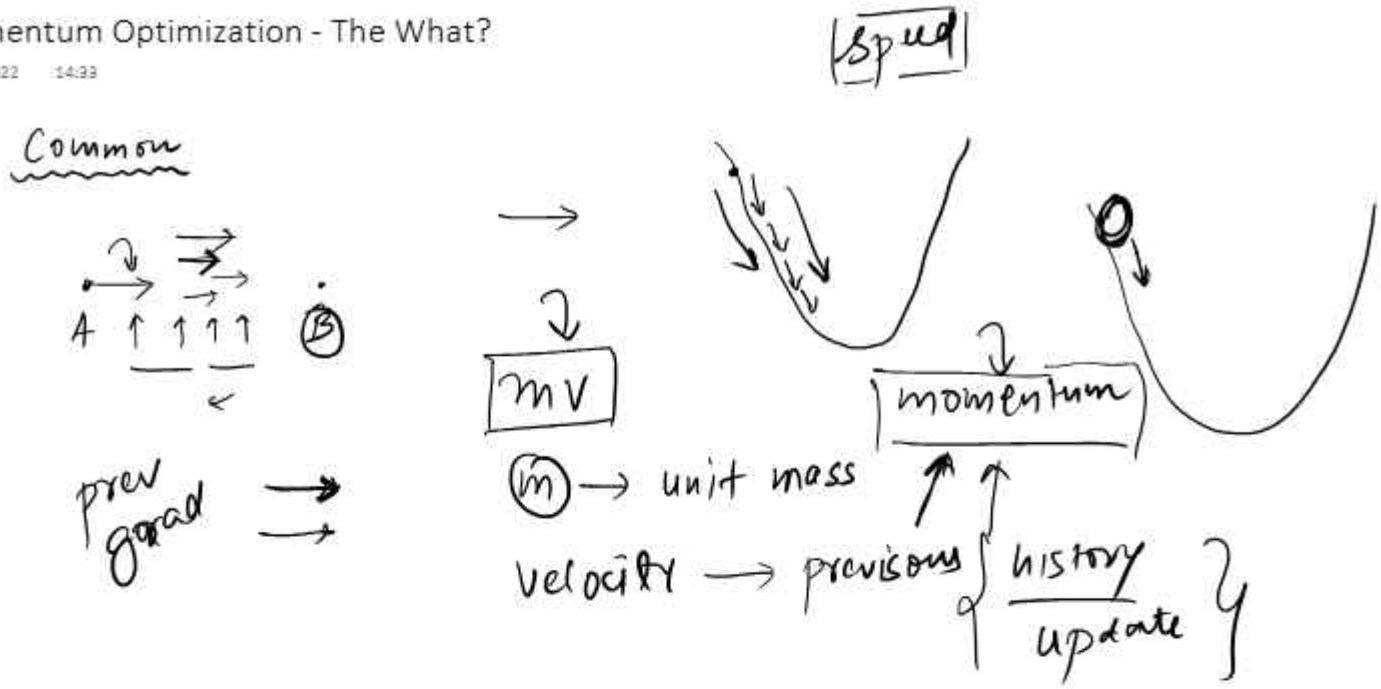
Momentum Optimization - The Why?

20 July 2022 14:28



Momentum Optimization - The What?

20 July 2022 14:33



Momentum Optimization - Mathematics(The How?)

20 July 2022 14:56

$$w_{t+1} = w_t - \eta \nabla w_t$$

$v \rightarrow$ velocity

$$w_{t+1} = w_t - v_t$$

$$v_t = \beta * v_{t-1} + \eta \nabla w_t$$

acceleration

momentum

0.9

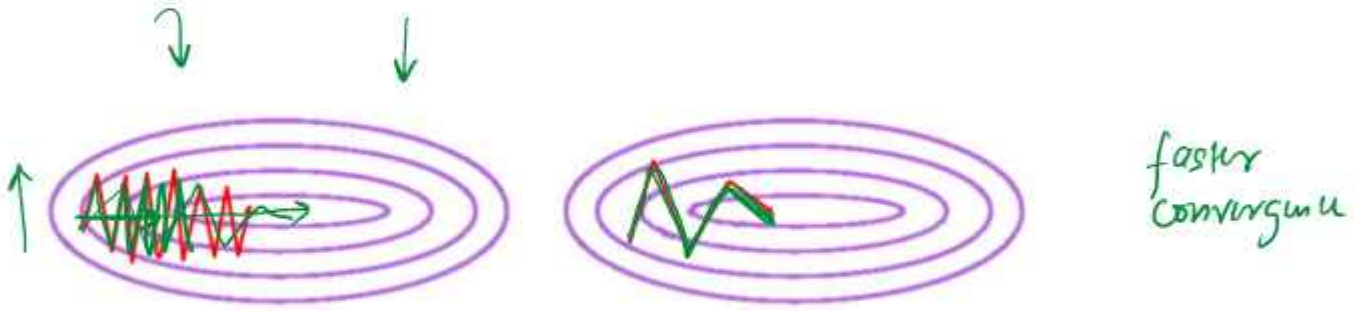
$0 < \beta < 1$

history of past velo- use

prev velocity

v_{t-2}

v_{t-3}



SGD

SGD momentum

faster convergence

Effect of beta

20 July 2022 15:09

$$w_{t+1} = w_t - \eta \nabla w_t$$

SGD

$$w_{t+1} = w_t - v_t$$

$$v_t = \beta v_{t-1} + \eta \nabla w_t$$

$\beta = 0$ ← $\beta = 0$ means → SGD

β = decaying factor

$\frac{1}{1-\beta}$ ↙ 0.9

$\frac{1}{1-0.9} = \frac{1}{0.1} = 1:10$

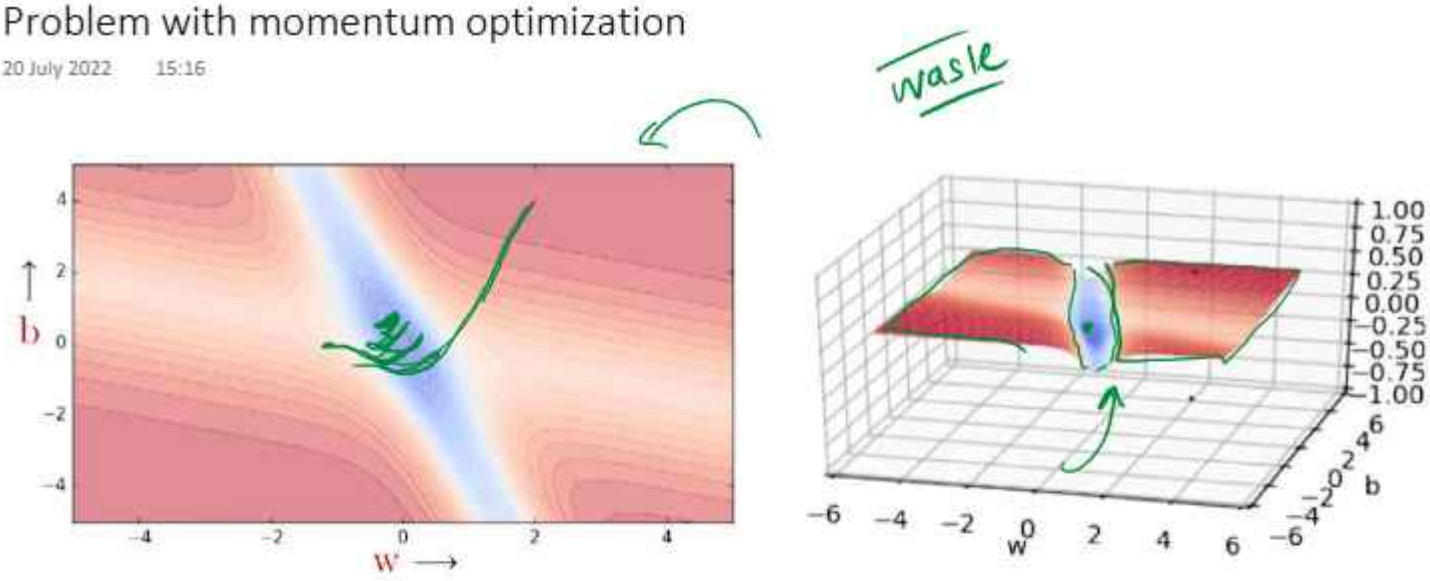
current avg past 10 velocities

up1 up2 . . - up3

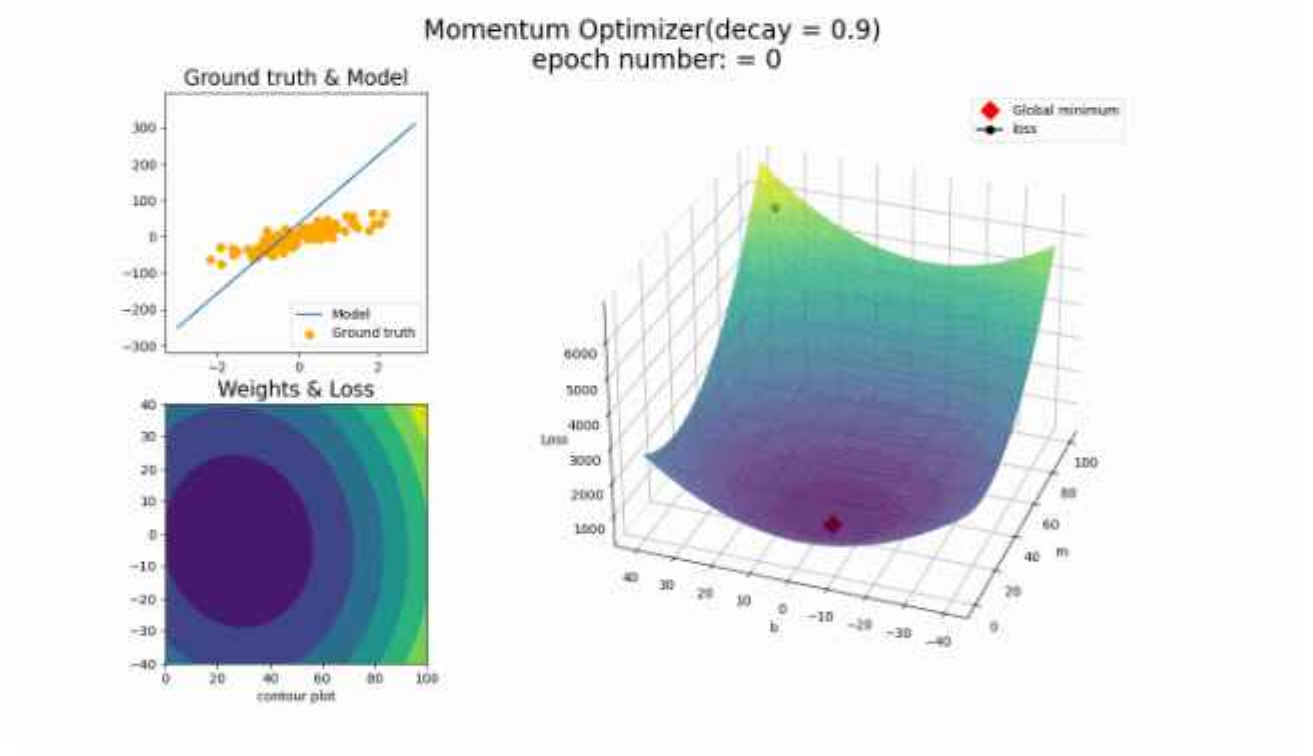
$\beta = 1$

Problem with momentum optimization

20 July 2022 15:16



decay 0.9



Demo

24 July 2022 13:17

Mathematical Intuition

24 July 2022 13:17

Momentum

$$w_{t+1} = w_t - v_t$$

where

$$v_t = \beta v_{t-1} + \eta \nabla w_t$$

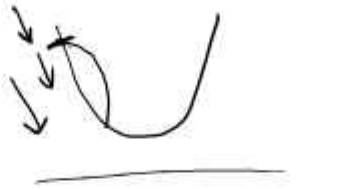
$\beta \rightarrow$ decay factor

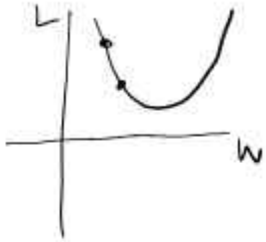
$$w_{t+1} = w_t - (\beta v_{t-1} + \eta \nabla w_t)$$

past velocity

gradient at that point

$$w_{t+1} = w_t - \eta \nabla w_t$$

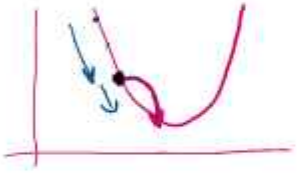




NAG

Nesterov Accelarated Gradient

better



history of velocity + gradient at that pt

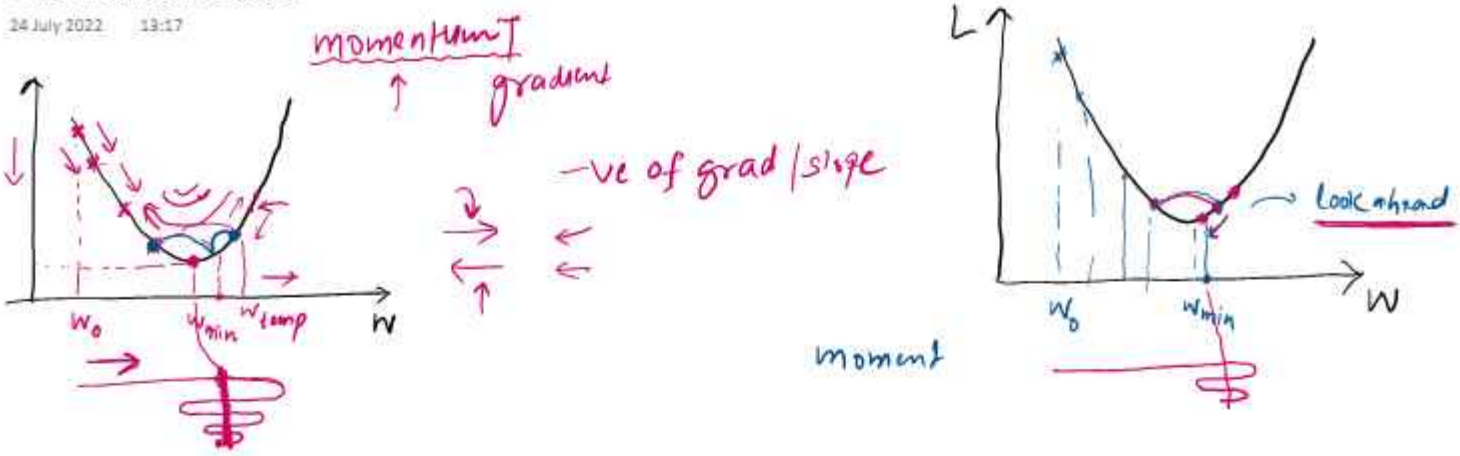


look ahead

$$\begin{cases} w_{la} = w_t - \beta v_{t-1} \\ v_t = \beta v_{t-1} + \eta \nabla w_{la} \\ w_{t+1} = w_t - v_t \end{cases} \rightarrow v_t = (w_t - w_{la}) + \eta \nabla w_{la}$$

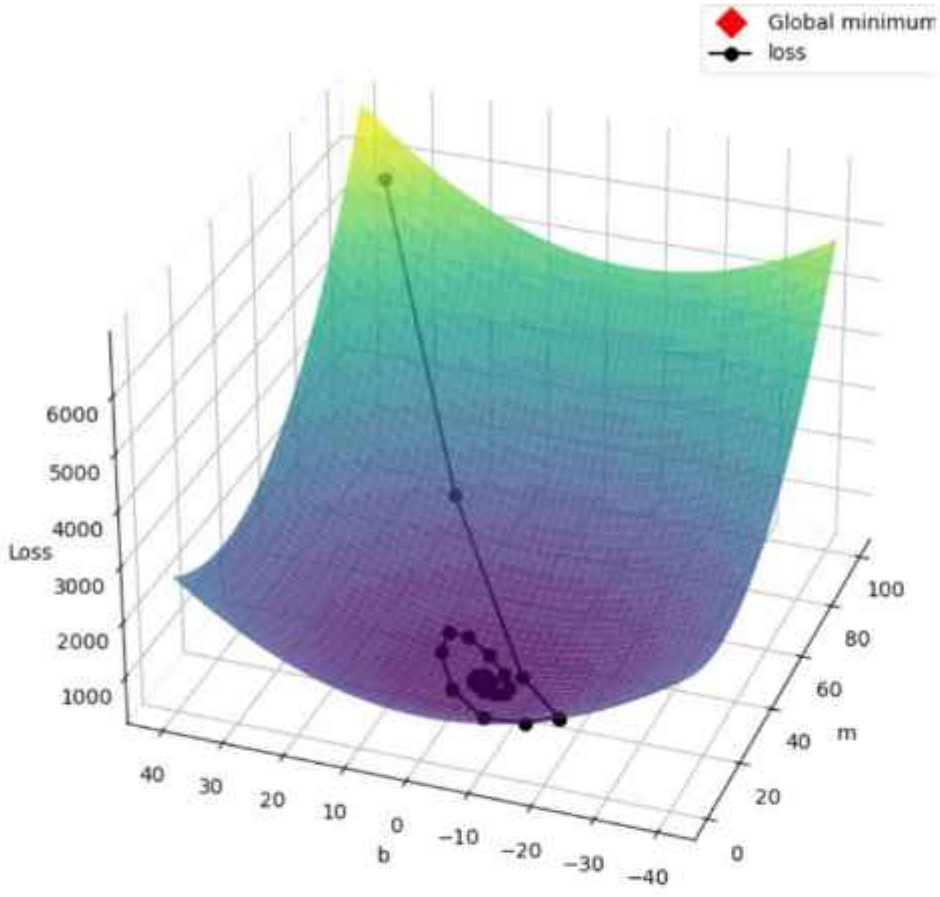
Geometric Intuition

24 July 2022 13:17



Disadvantage

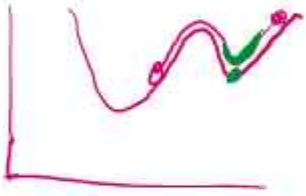
24 July 2022 13:18



Disadvantage

NAG → oscillation → dampen

local minima



Keras Code

24 July 2022 13:18

```
tf.keras.optimizers.SGD(
    learning_rate=0.01, momentum=0.0, nesterov=False, name="SGD", **kwargs
)
```

SGD ✓

Momentum ✓
momentum = 0.9
nesterov = False

NAG ✓
momentum = 0.9
nesterov = True

AdaGrad Intro

01 August 2022 18:45

Adaptive Gradient → learning rate fix X
change
adapt →

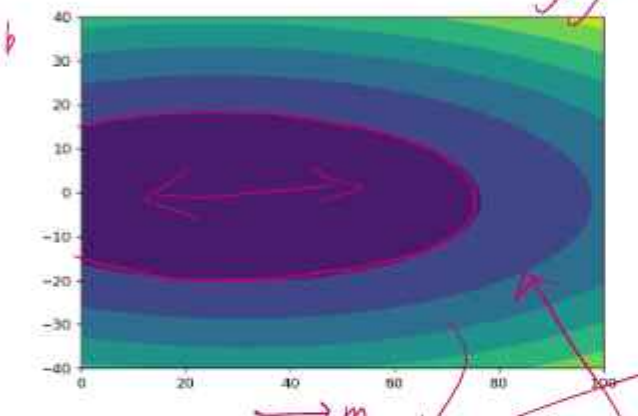
cgpa | salary | bank loan
0-10 ↓ 0-Lakh
↓
Normalize

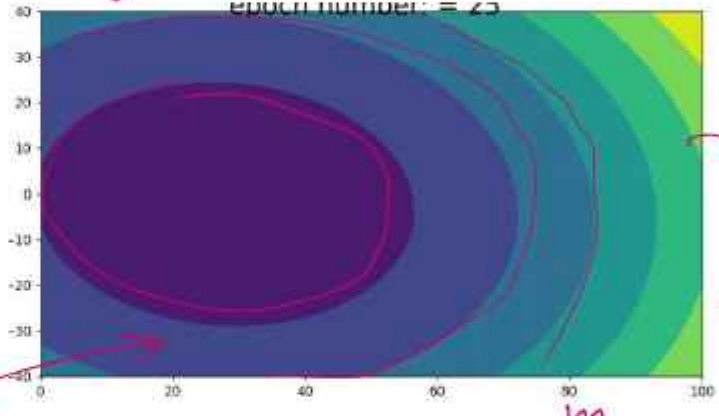
→ input features → scale
↳ different

⇒ features → [Sparse] → 0 (most)
↳ Adagrad

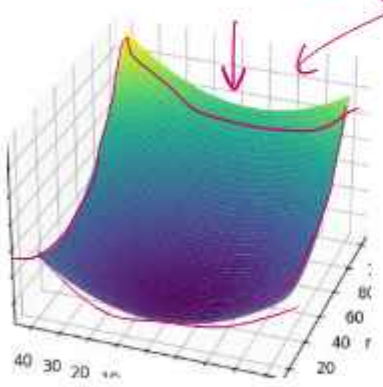
iq | cgpa | [from iit] | package
0 → ↘ 0,0,0,
1 →

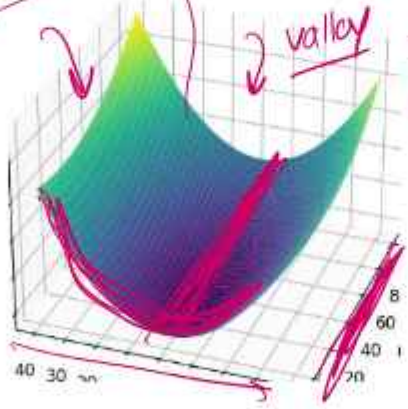
Elongated bowl problem





→ m





valley

Loss → w,b

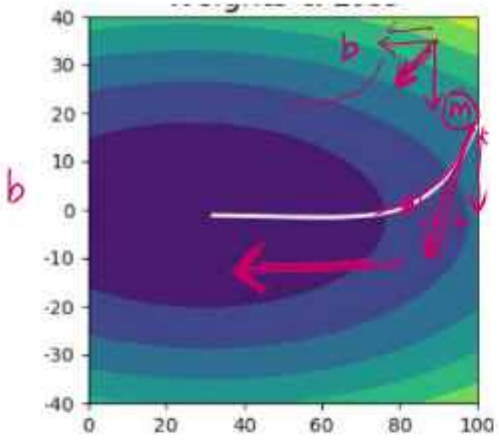
Bgd → [momentum] → NAG

[Adagrad]

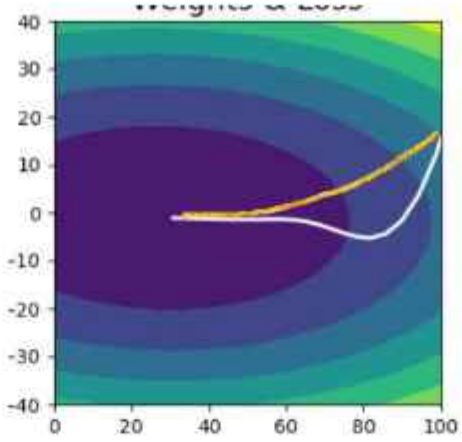
elongated
↓
slope → 1axis - along
2nd axis constant

How optimizers behave(Why?)

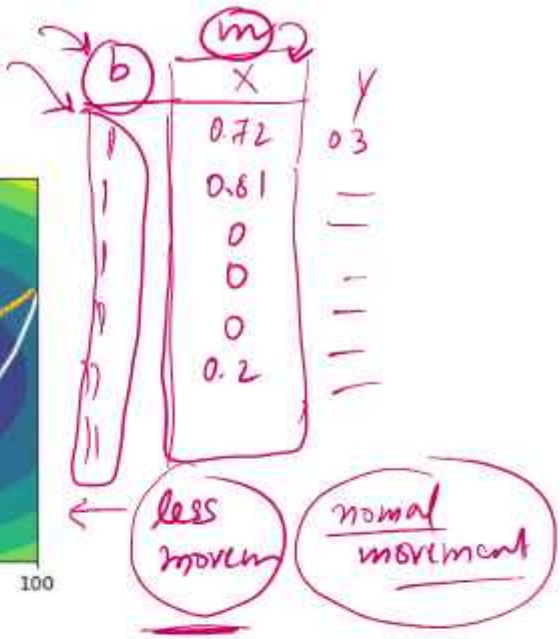
02 August 2022 18:44



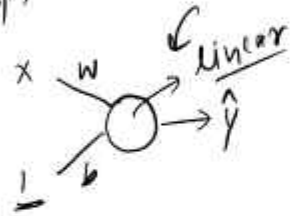
Batch GD



Momentum



$(y-\hat{y})^2$



BGD → sparse

for i in epochs:

$$w = w - \eta \left[\frac{\partial L}{\partial w}\right]$$

$$b = b - \eta \left[\frac{\partial L}{\partial b}\right]$$ → big update in every epoch

$$\left[\frac{\partial L}{\partial w}\right] = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w}$$

$$= -2(y-\hat{y})[x]$$ add

$$\frac{\partial L}{\partial b} = [-2(y-\hat{y}) * 1]$$ bignum

100 rows — 0 — X sparse

small number

small in every epoch

sparse col → small

nonsparse → normal/big



Adagrad mathematics + Intuition

02 August 2022 18:44



Adagrad

$W = W - \eta \frac{\partial L}{\partial W}$ update

$b = b - \eta \frac{\partial L}{\partial b}$

Optimization

lr → same

big lr → small update - small

Adagrad → diff learning rates

past gradients sum(squared)

lr → small → gradient → bg

epoch

$v_t = v_{t-1} + (\nabla w_t)^2$

v_t $\left(\frac{\partial L}{\partial W}\right)^2 \left(\frac{\partial L}{\partial b}\right)^2$

$\frac{\partial L}{\partial W}$

$w_{t+1} = w_t - \eta \nabla w_t$

η small $w_{t+1} = w_t - \eta \Delta w_t$

$\frac{\eta}{\sqrt{v_t + \epsilon}}$

ϵ = small num

$v_t = 0$

(-ve, +ve)

sparse data

Adagrad Demo

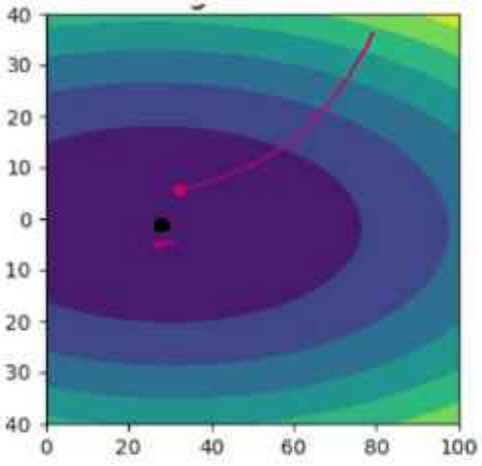
02 August 2022 18:44

Keras Implementation

02 August 2022 18:44

Disadvantage

02 August 2022 18:43



NN → use X → Linear regress

η small + decrease

$\sqrt{v_t}$

v_t

past updates → gradient

epochs

update small

global minima

RMSprop

Adam

The Why?

03 August 2022 14:03

RMSprop → Root Mean square Prop

↑ ↳ improvement

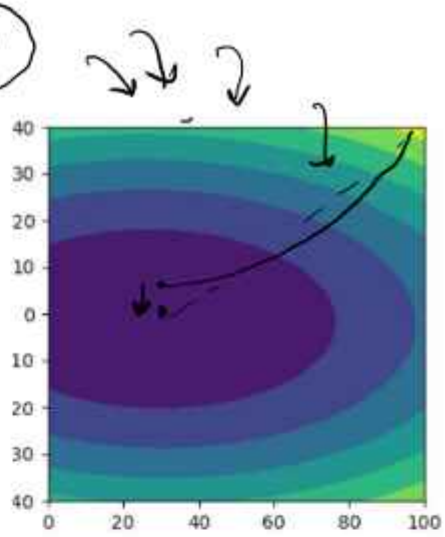
↓

Adagrad ← sparse data

BGD → momentum

Disadvantage → learning rate ↓





X_1	X_2	y
0		
0		
0		
1		
0		

↓

Update → small

Mathematical Formulation

03 August 2022 14:04

epochs

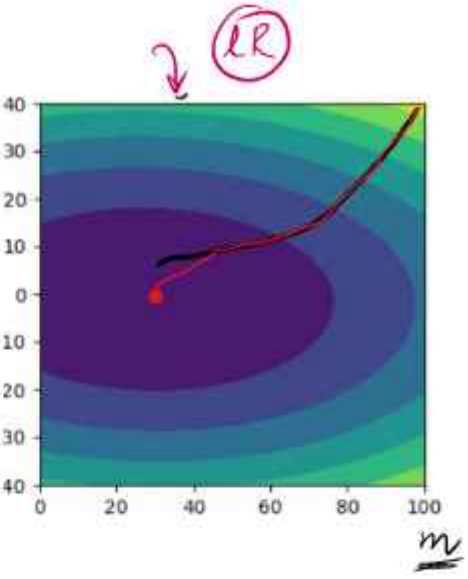
Adagrad

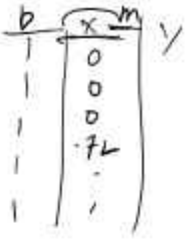
$$v_t = v_{t-1} + (\nabla w_t)^2$$

gradients

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$$

(b) small





↓ b move

non-sparse

RMSprop

$$v_t = \beta v_{t-1} + (1-\beta)(\nabla w_t)^2$$

exp decaying avg

old epoch

$\beta = 0.95$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$$

→ $v_0 = 0$

→ $v_1 = 0.95 \times 0 + 0.05 (\nabla w_1)^2$

→ $v_2 = 0.95 \times 0.05 (\nabla w_1)^2 + 0.05 (\nabla w_2)^2$

→ $v_3 = 0.95 \times 0.95 \times 0.05 (\nabla w_1)^2 + 0.95 \times 0.05 (\nabla w_2)^2 + 0.05 (\nabla w_3)^2$

smaller ← epoch 1 | epoch 2 | epoch 3 ← larger

v_t shoot → η small

Adagrad → NN → non-convex optimization

↳ convex optimiz → linear reg

↘ convex → xinav
optimiz ~~xinav~~ ~~ogg~~

global minima — RMSprop

Disadvantage

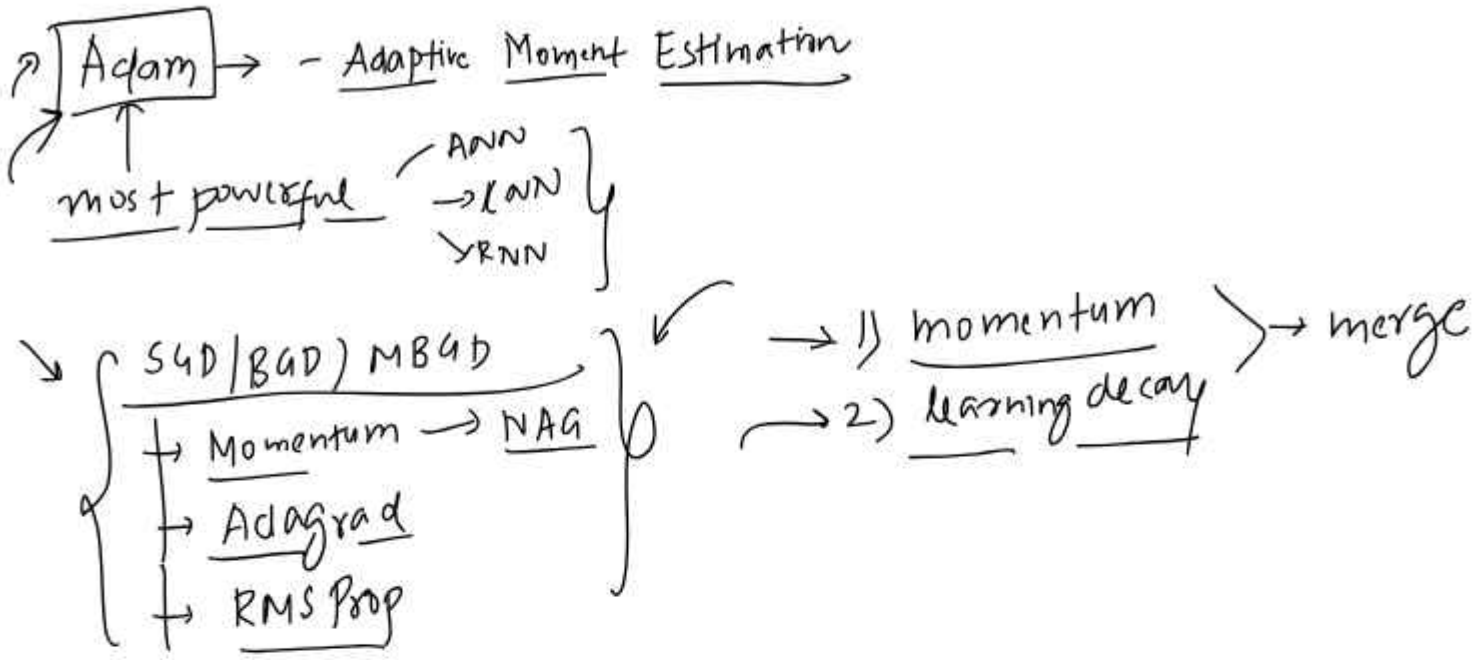
03 August 2022 14:54

No,
Optimization

↓
Adam

Introduction

04 August 2022 10:45



Mathematical Formulation

04 August 2022 10:45

Keras → Bias correction

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} * m_t$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

epoch #

$\beta_1 = 0.9$

$\beta_2 = 0.99$ → Keras

where

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla w_t$$ → momentum

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla w_t)^2$$ → Adagrad

$\eta = 0.1$

0.01

$m_0 = 0 \quad m_t = 0 \quad v_t = 0$

$v_0 = 0$

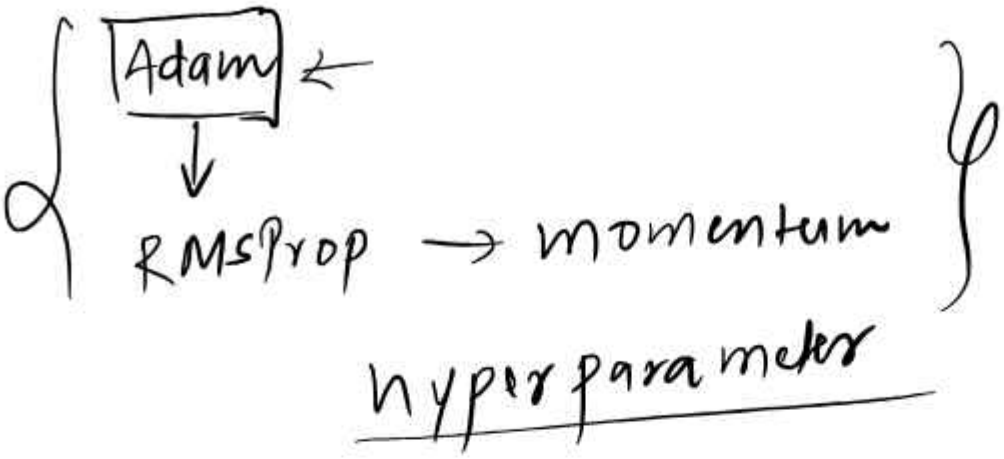
$0 \rightarrow$

Demo

04 August 2022 10:45

Verdict

04 August 2022 10:45

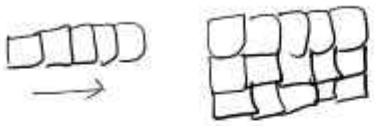


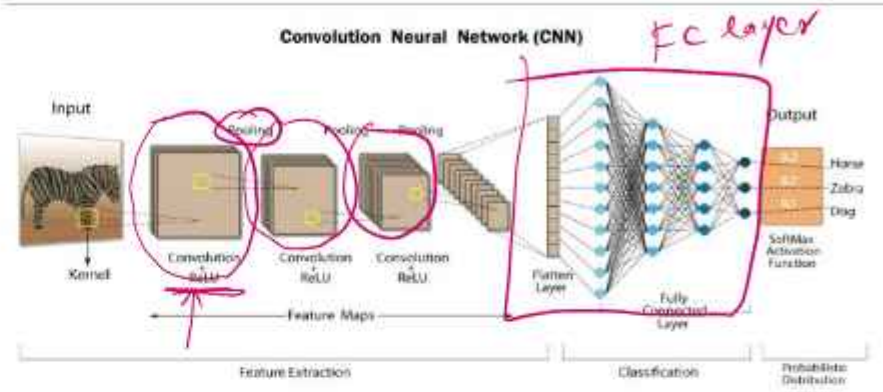
What is a CNN?

17 August 2022 06:47

Convolutional neural networks, also known as convnet, or CNNs, are a special kind of neural network for processing data that has a known grid-like topology like time series data(1D) or images(2D).

→ Convolution
→ Pooling
→ FC layer - ANN





→ ANN → Convolution → convolution ⊗

CNN → convolution

ANN → matrix mult.

Inspirations

↳ visual cortex

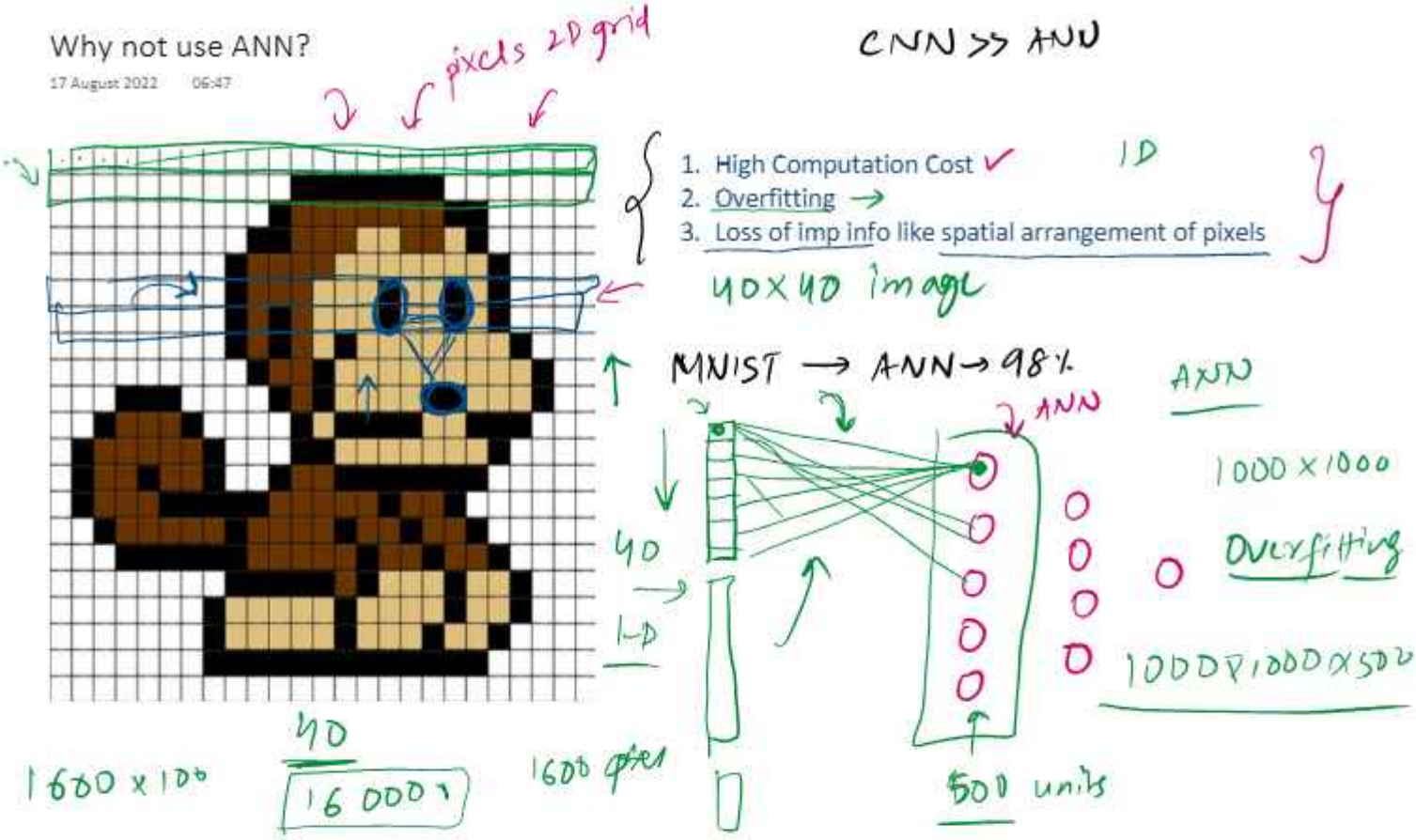
1998 Yann Lecun → AT&T →

↳ Microsoft → OCR hand writing recog } CNN

{ facial recog self driving } CNN RNN

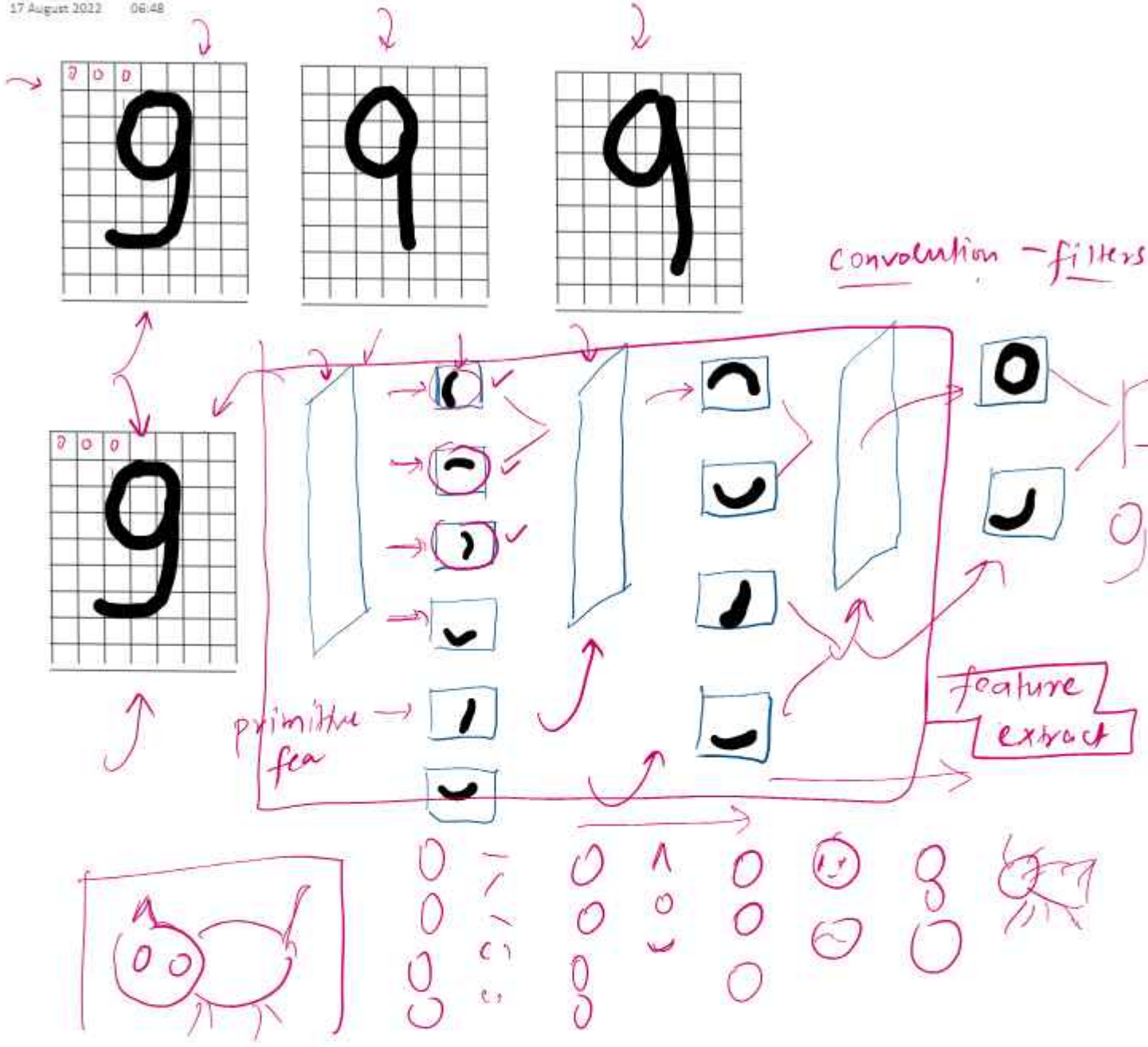
Why not use ANN?

17 August 2022 06:47



CNN Intuition

17 August 2022 06:48

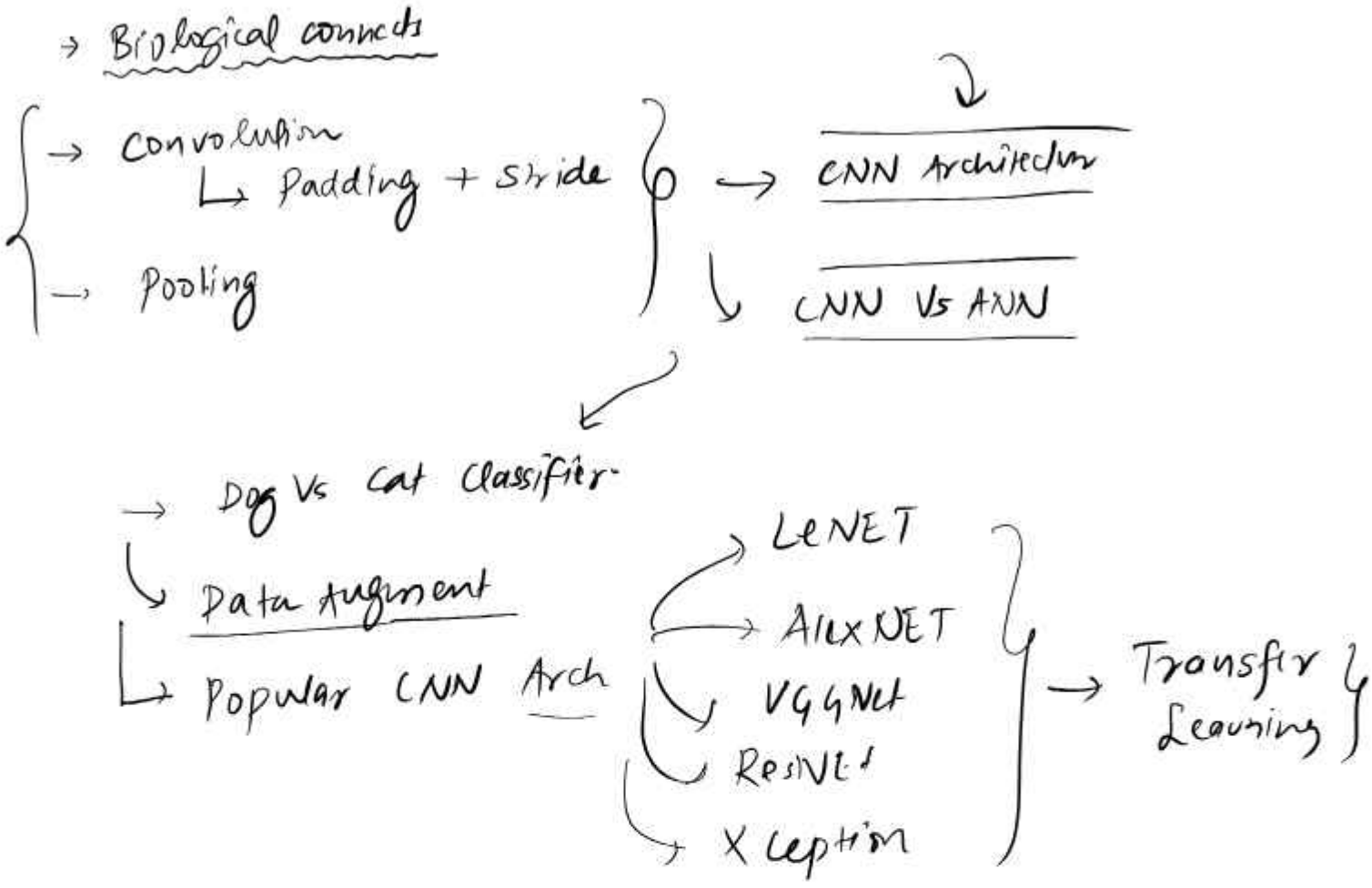


CNN Applications

17 August 2022 06:48

Roadmap

17 August 2022 06:48



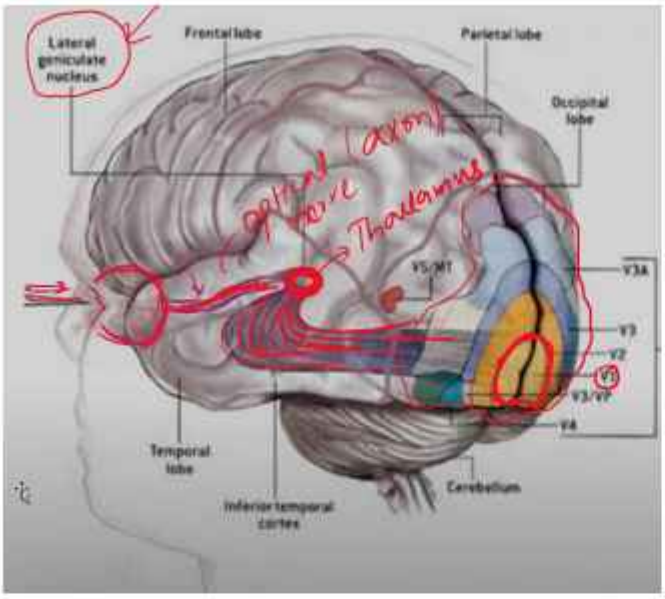
Human Visual Cortex

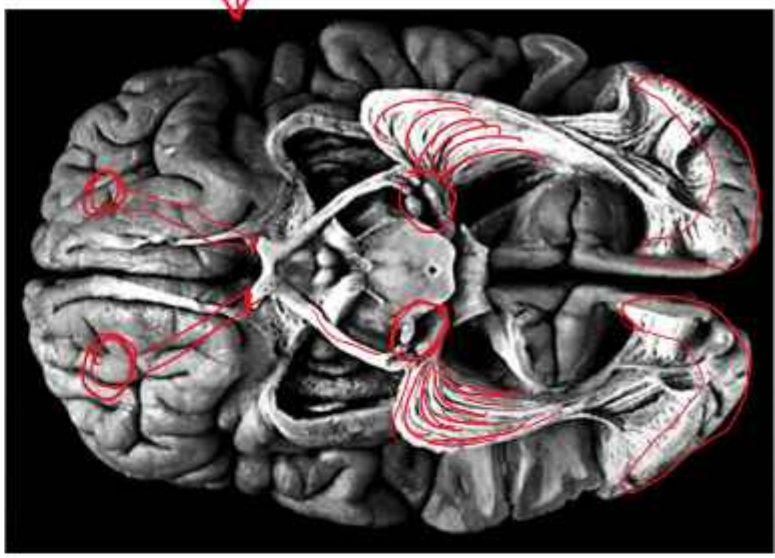
18 August 2022 16:09

2D sheet

primary Vcortex

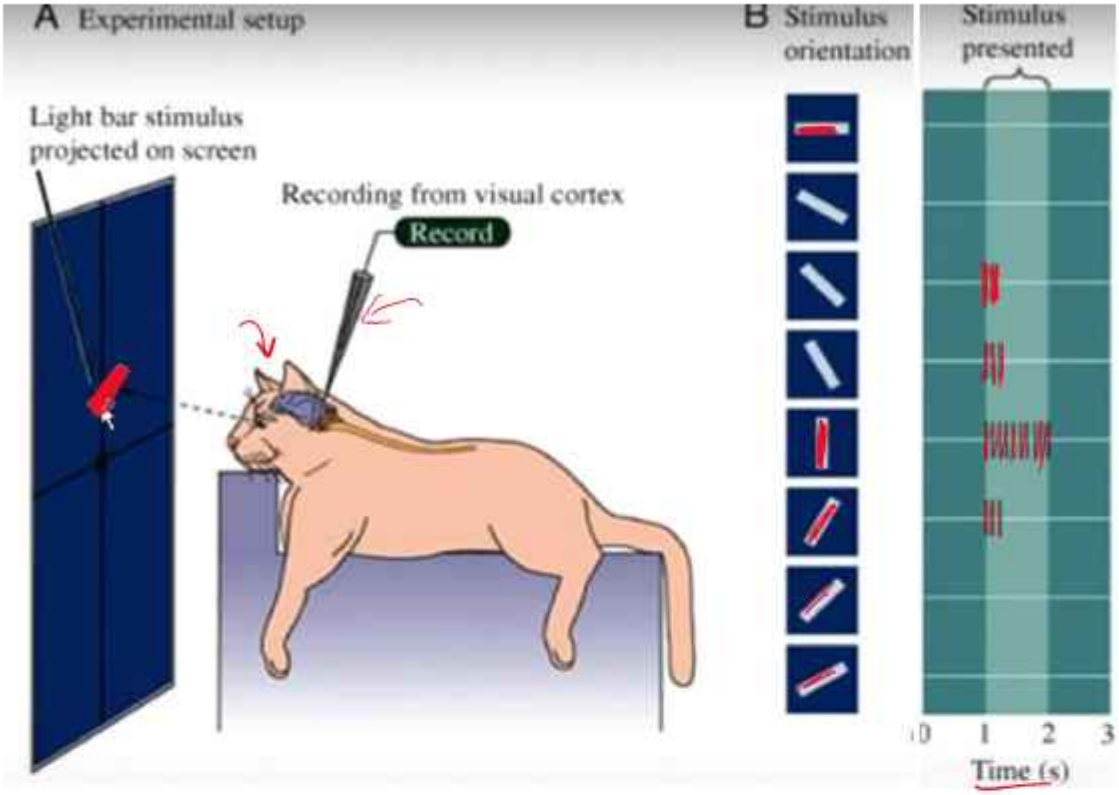
actual brain

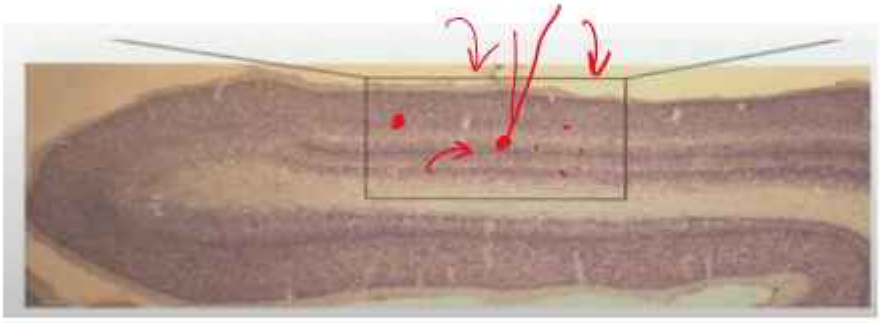




The Experiment

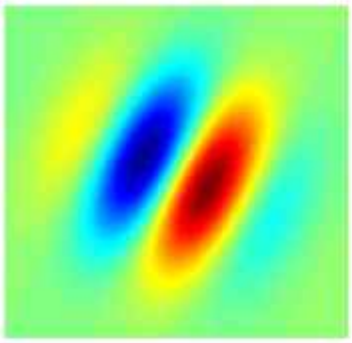
18 August 2022 16:09

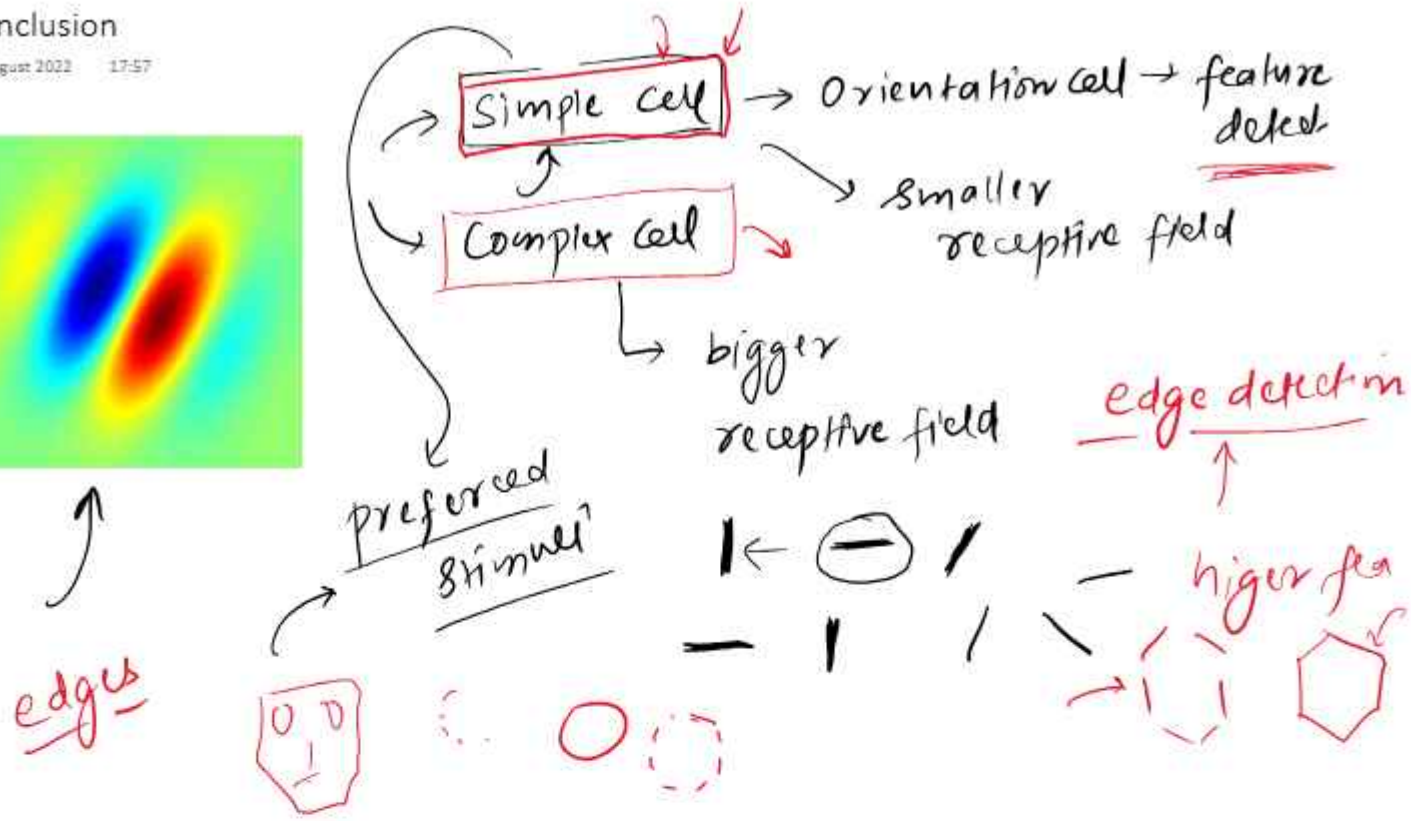




Conclusion

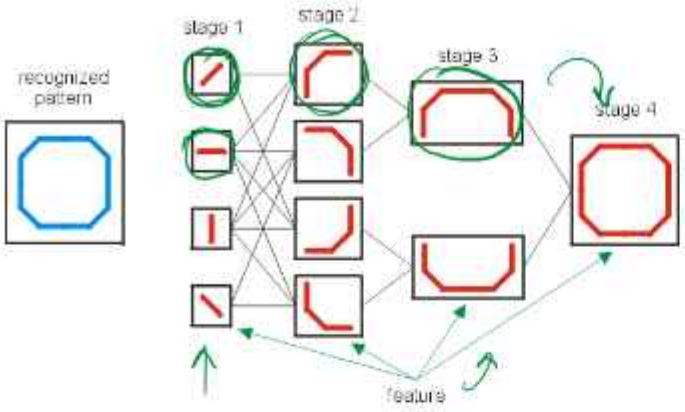
18 August 2022 17:57



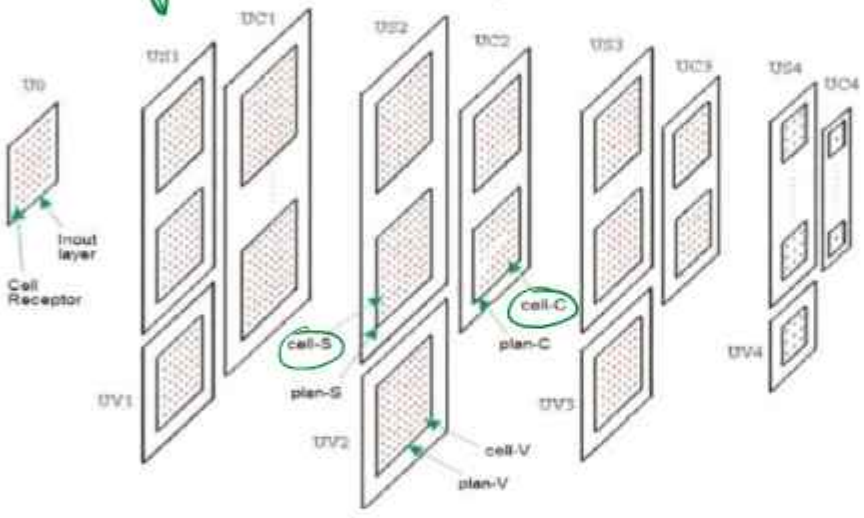


Development

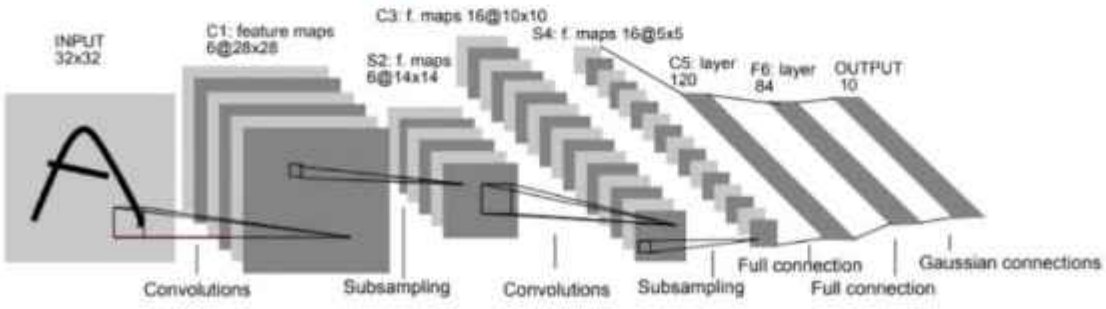
18 August 2022 16:15



Neocognitron → Fukushima



Yann Lecun → CNN → Backprop convolution

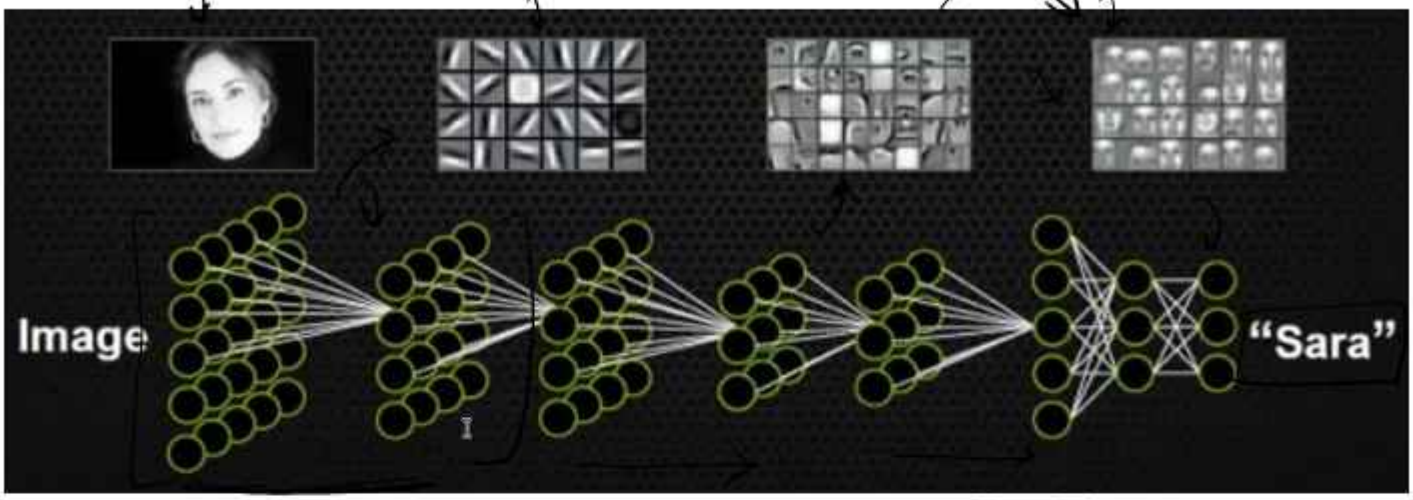


early CNN

2012 → AlexNET → ImageNET → CNNs

Introduction

19 August 2022 16:45



CNN → NN special → Convolution, Pooling, FC ← CNN

Basics of Images

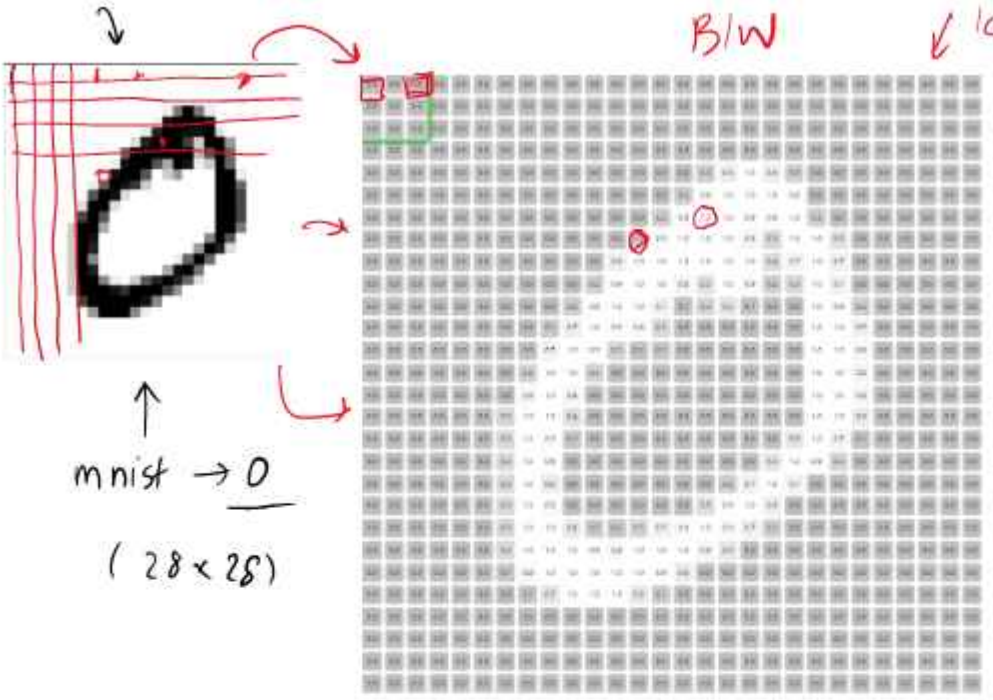
19 August 2022 16:53

0-255 → 0-1

B/W

1 channel – Greyscale → B/W

RGB → colored images



28 x 28 pixels

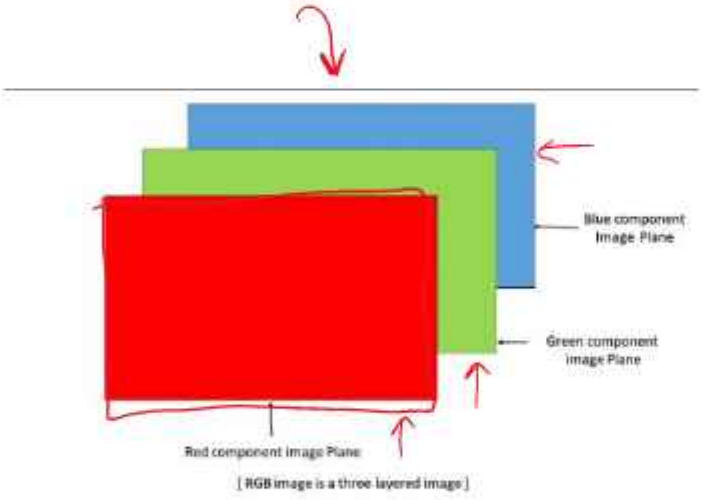
2D arrays →

2D numpy arrays

mnist → 0

(28 x 28)

3 channels → Red Green Blue (primary)

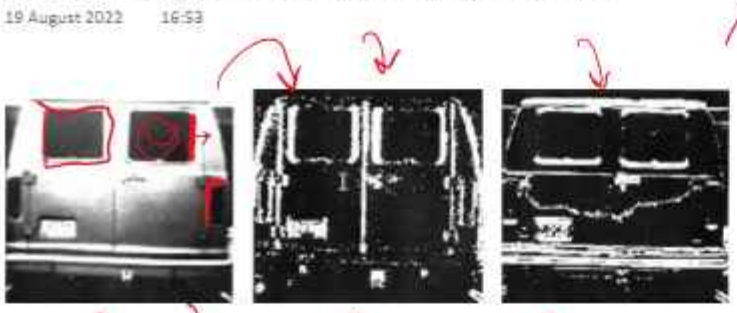


228 * 228 * 3

3 channels

Edge Detection (Convolution Operation)

19 August 2022 16:53



0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
255	255	255	255	255	255
255	255	255	255	255	255
255	255	255	255	255	255

image

6×6

-1	-1	-1
0	0	0
1	1	1

= ANN weights

filter / kernel

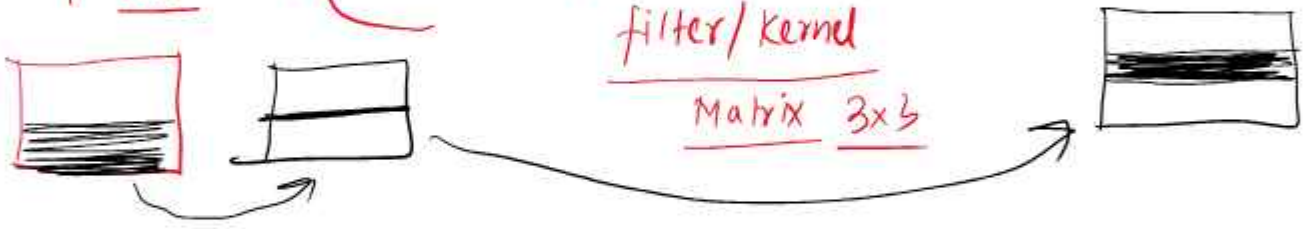
Matrix 3×3

255×1 + 255×1 + ... horizontal edge detection filter

-1×0 + -1×0 + -1×0 ...

0	0	0	0
255	255	255	255
255	255	255	255
0	0	0	0

→ feature map



0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
255	255	255	255	255	255
255	255	255	255	255	255
255	255	255	255	255	255

(6×6)

-1	-1	-1
0	0	0
1	1	1

(3×3)

= | | | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |

(4×4)

(28×28) → (26×26)

n×n

(3×3) → ? (26×26)

m×m → (n-m+1) × (n-m+1)

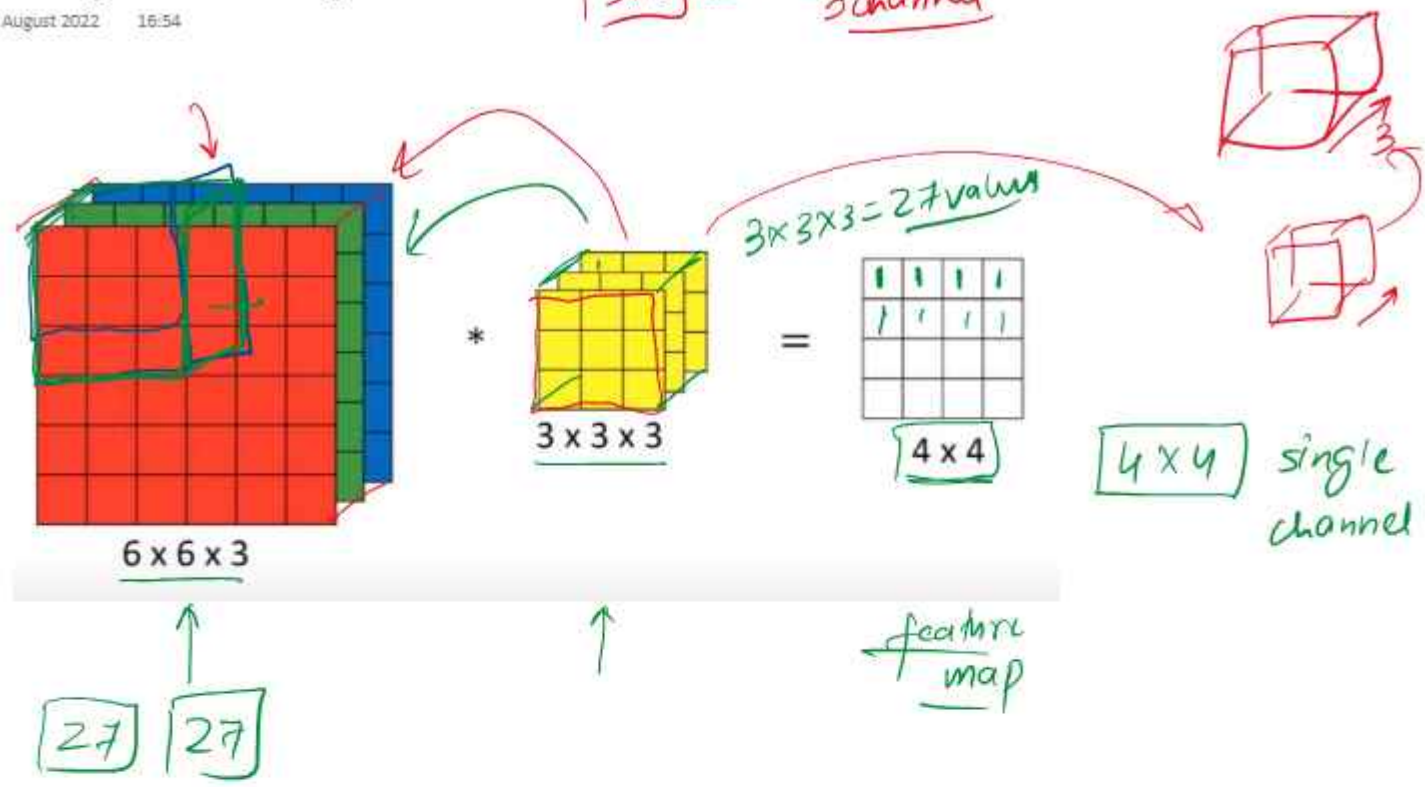
(64×64) (3×3) → (62×62)

Demo

19 August 2022 16:54

Working with RGB Images

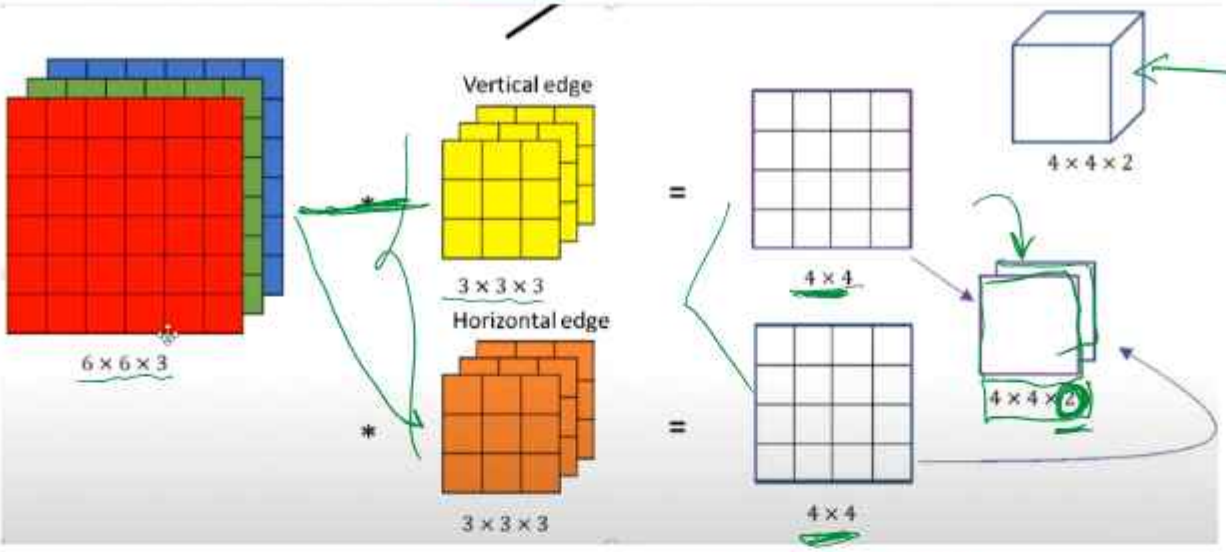
19 August 2022 16:54



$m \times m \times c$ $n \times n \times c$ $\longrightarrow$ $(m-n+1)(m-n+1)$ single channel

Multiple Filters

23 August 2022 08:24



#filters used

Image taken from Andrew NG's lecture

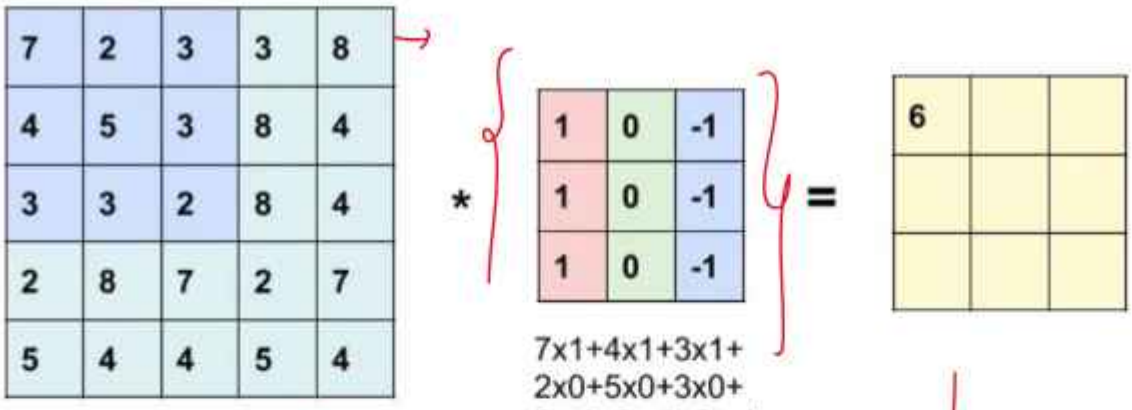
10 → 4 × 4 × 10

Problem with Convolution

26 August 2022 14:25

What is Padding?

26 August 2022 14:26



$n-f+1=n$

$n \times n$

5×5

7x7

$f \times f$

3×3

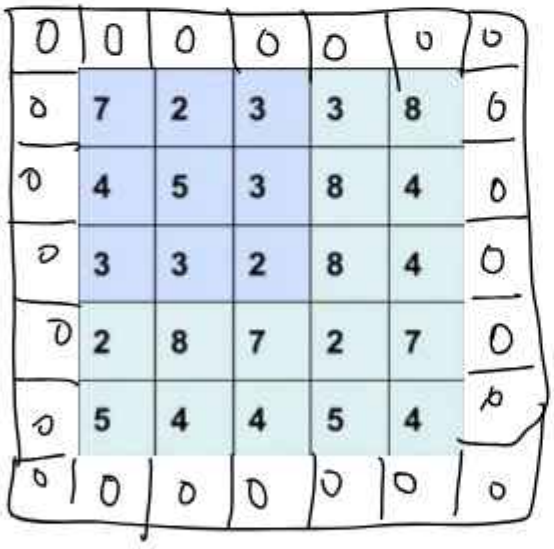
$\rightarrow (n-f+1)(n-f+1)$

$(5-3+1) = 3 \times 3$

$n-f+1 = 5$

$n-3+1=5 \Rightarrow n=8-1$

$n=7$



7x7

convolution

Zero padding

$5 \times 5 \rightarrow 3 \times 3$

$5 \times 5 \longrightarrow 3 \times 3$

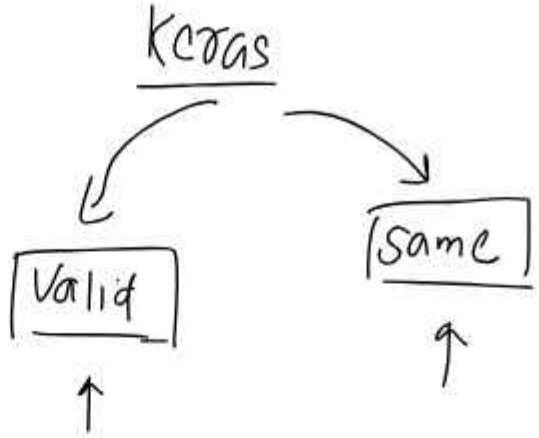
$(n - f + 1)$

↓

$(n + 2p - f + 1)$

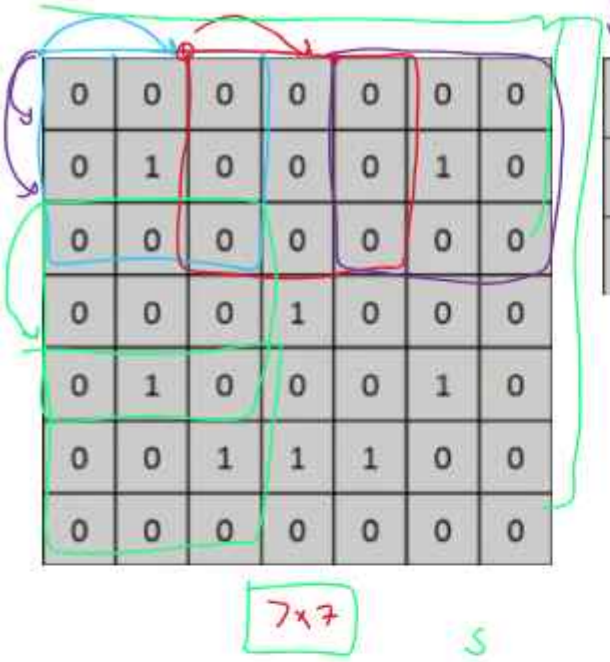
$5 + 2(1) - 3 + 1$

$= 7 - 3 + 1 = 5$



Strides

27 August 2022 08:23



Stride = 1



(1,1) 3x3

right

bottom

→ result feature

Stride = (2,2)

Stride = 2 →

$\frac{7-3}{2} + 1$

$2+1 = 3$

$(n-f+1) \longrightarrow \left[\frac{n-f}{s} + 1\right]$ ↘ $p = p$

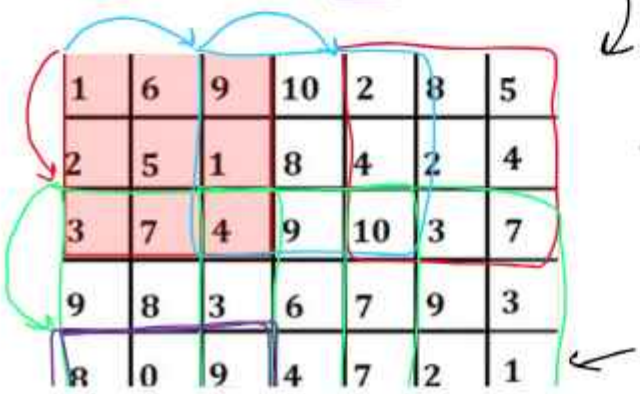
$\left[\frac{n+2p-f}{2} + 1\right]$ → strided convolution

$\frac{7+2-3}{2} + 1 = [4 \times 4]$

$\frac{n-f}{2}$ $\frac{6-3}{2}$ $1.5 = 1 + 1 = 2$

Special Case

Stride = 2



2x3

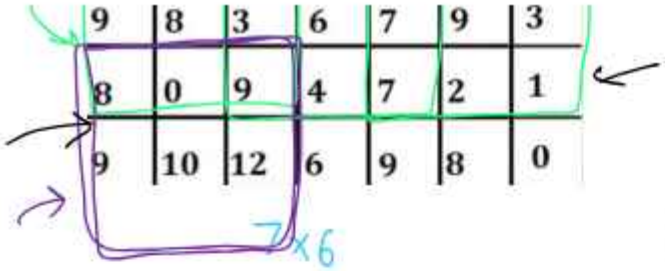
$\frac{6 \times 7}{3 \times 3}$

$\left[\frac{n-f}{s} + 1\right]$

19 → 1

1.1 → 1

floor



$$\left[\frac{\cdot}{5}+1\right]$$

floor

$$\left[2 \quad \frac{6-3}{2}+1 = 1.5+1\right]$$

$$1+1=2$$

$$\frac{7-3}{2}+1 = 2+1=3$$

Why Strides are required?

27 August 2022 08:24

1) High level features

2) Computing →

Keras → stride

$$\left[\frac{n+2p-f}{s}+1\right]$$

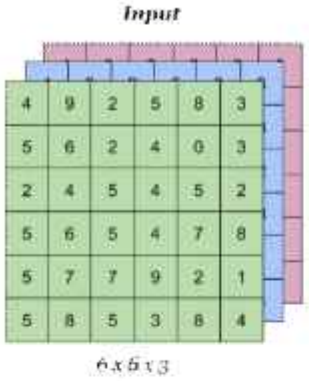
$$\frac{28+2-3}{2}+1$$

$$\frac{13.5}{}+1$$

$$13+1=14$$

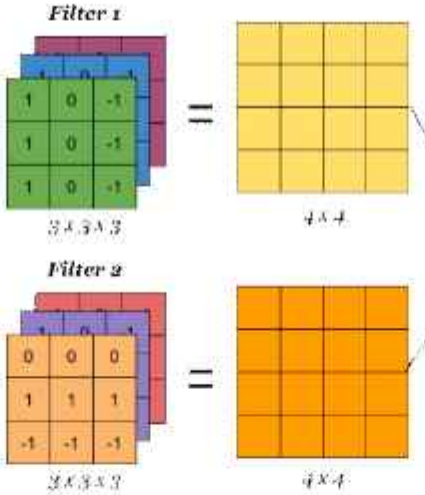
The Problem with Convolution

01 September 2022 09:55

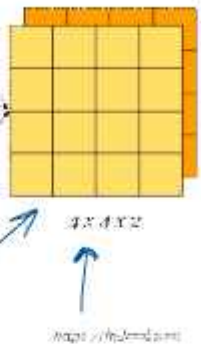


image

228 x 228 * 3



filters (100) → feature map



→ (226 * 226) * 100 * 32 → 19 MB → 1.5 GB

1 training → 100

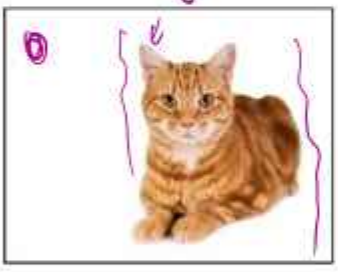
stride=2

1) Memory issue
2) Translation variance

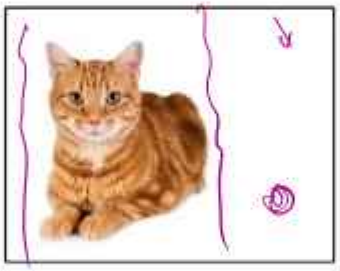
reduce size of feature map

pooling

Translation Variance



Cat



Cat

features

{ location dependent }

location

pooling { down sample your feature map }

Pooling

01 September 2022 09:55

0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
255	255	255	255	255	255
255	255	255	255	255	255
255	255	255	255	255	255

*

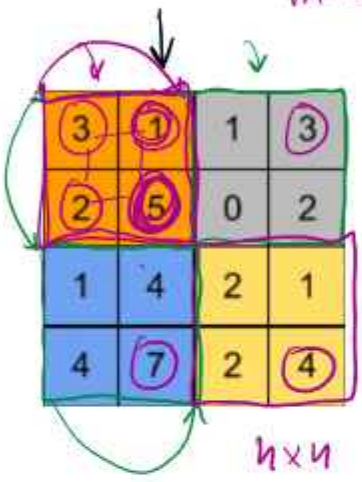
-1	-1	-1
0	0	0
1	1	1

= relu((4x4))

Max pooling
Min pooling
Avg pooling
L2 pooling
Global pooling

feature map (non-linear)

max pooling



4x4

(size) → (2,2)
stride → (2)
type → Max

receptive field

5	3
7	4

2x2

feature map

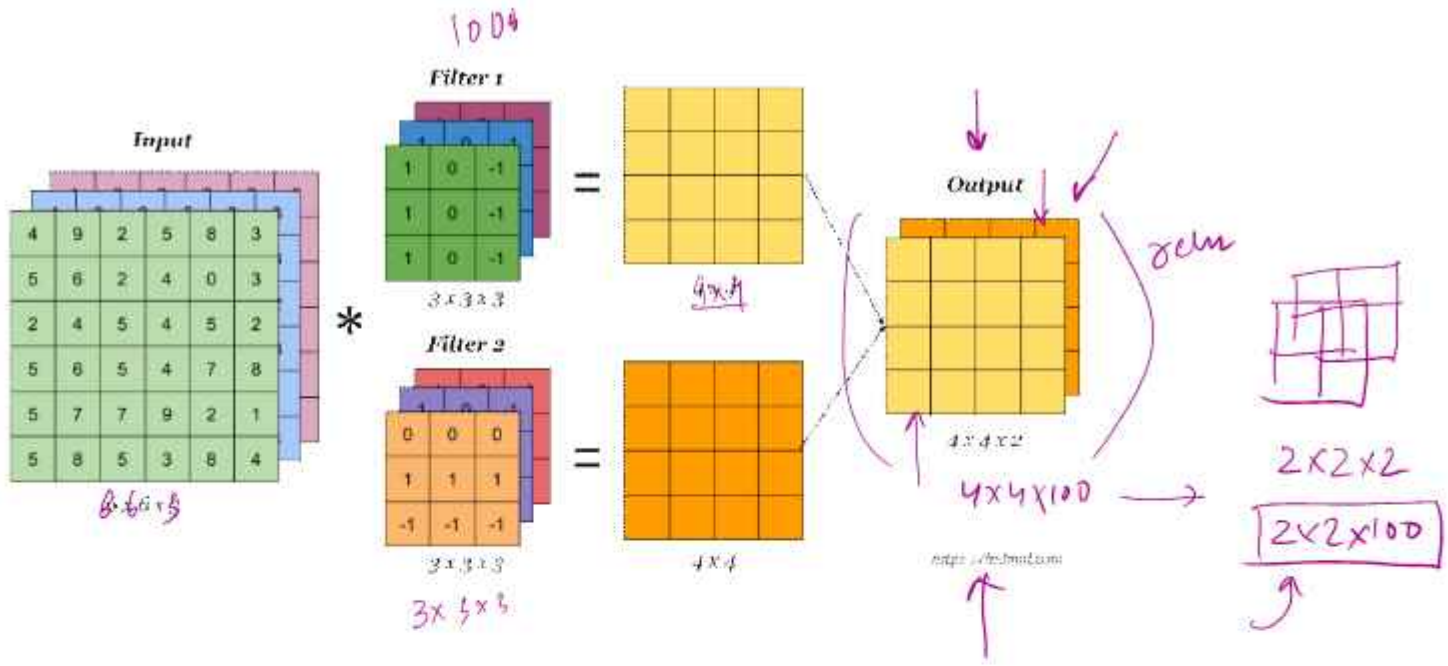
low level details eliminat

Demo

01 September 2022 09:57

Pooling on Volumes

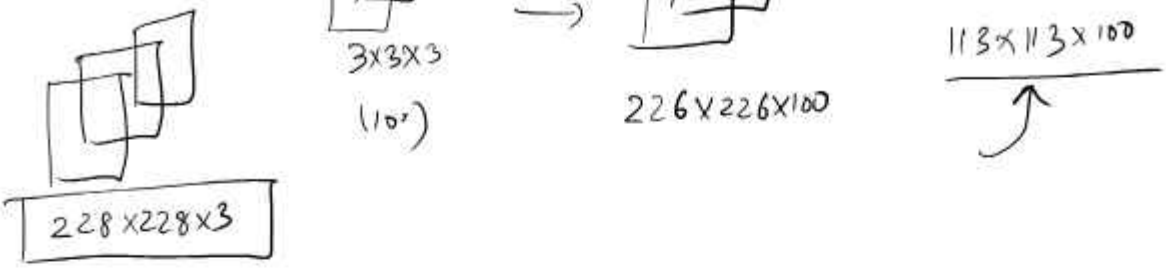
01 September 2022 09:56



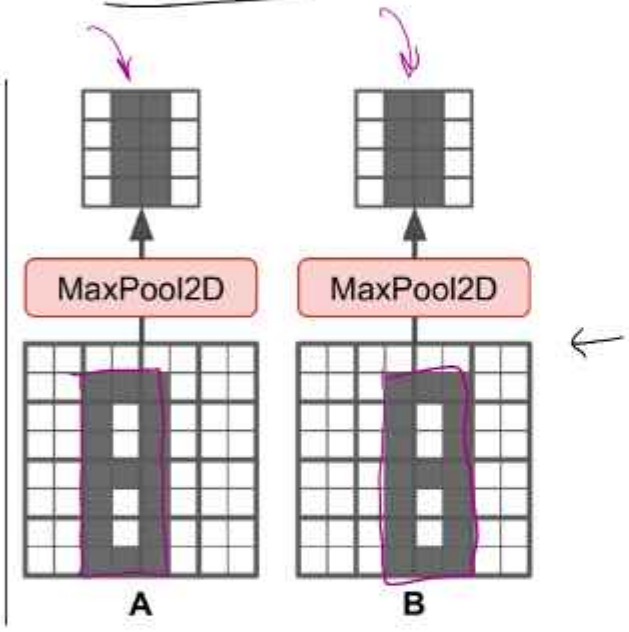
Advantages of Pooling

01 September 2022 09:56

1) reduced size

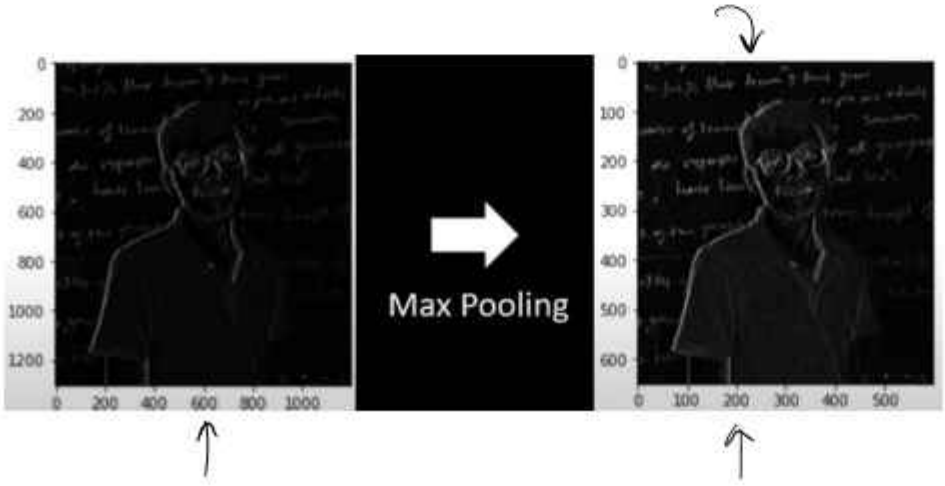


2) Translation invariance

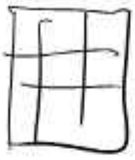


3) Enhanced features (only in case of Max pooling)



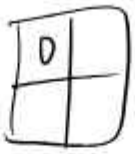


4) No need of training



min
max pooling → aggregate
avg pooling

faster



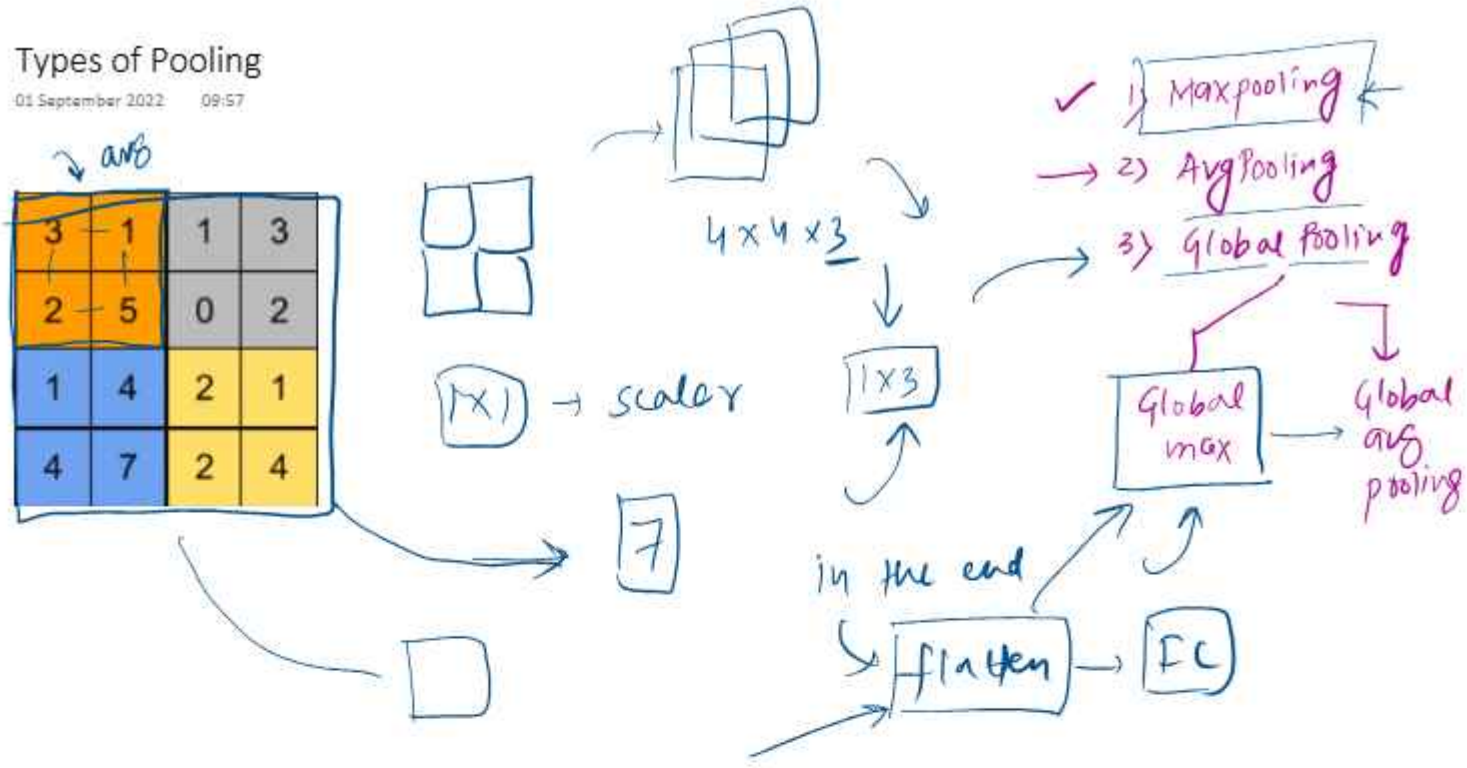
→ (2x2)
→ 2
→ type

Keras Code

01 September 2022 09:56

Types of Pooling

01 September 2022 09:57



Disadvantages of Pooling

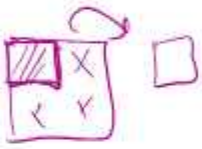
01 September 2022 09:57



lose a lot of info

4x4 → 16

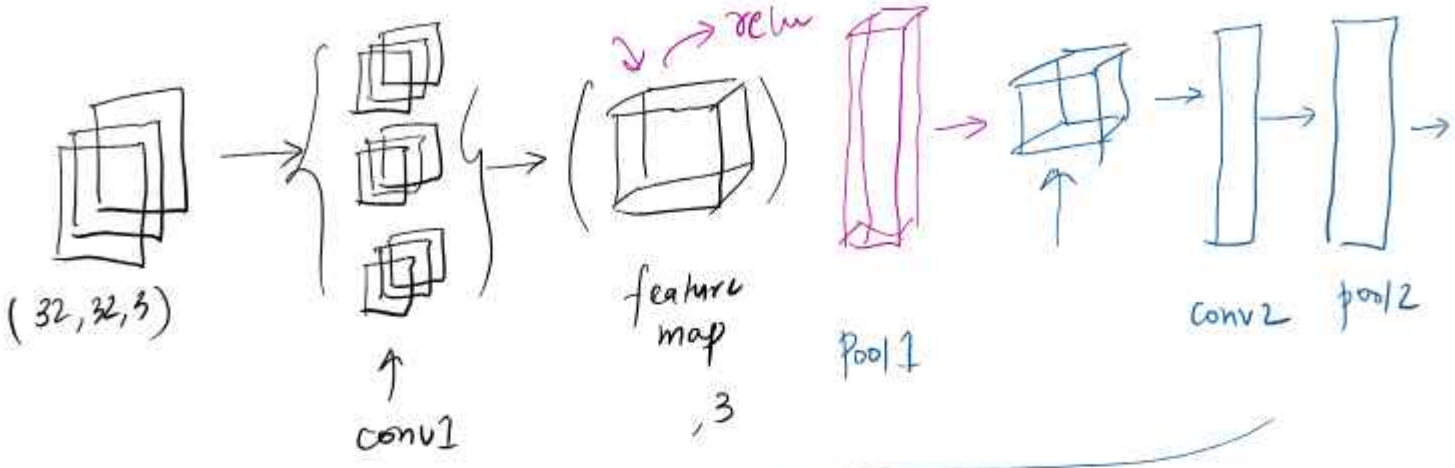
2x2 → 4

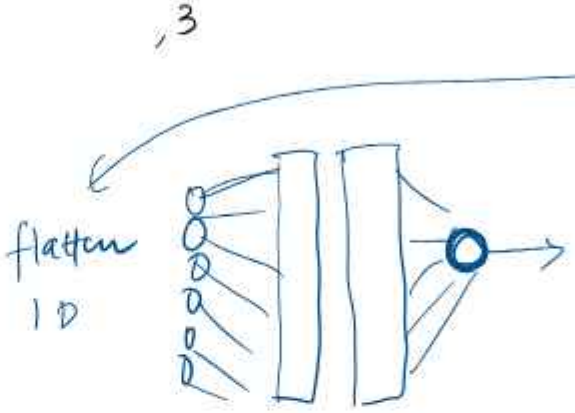


CNN Architecture

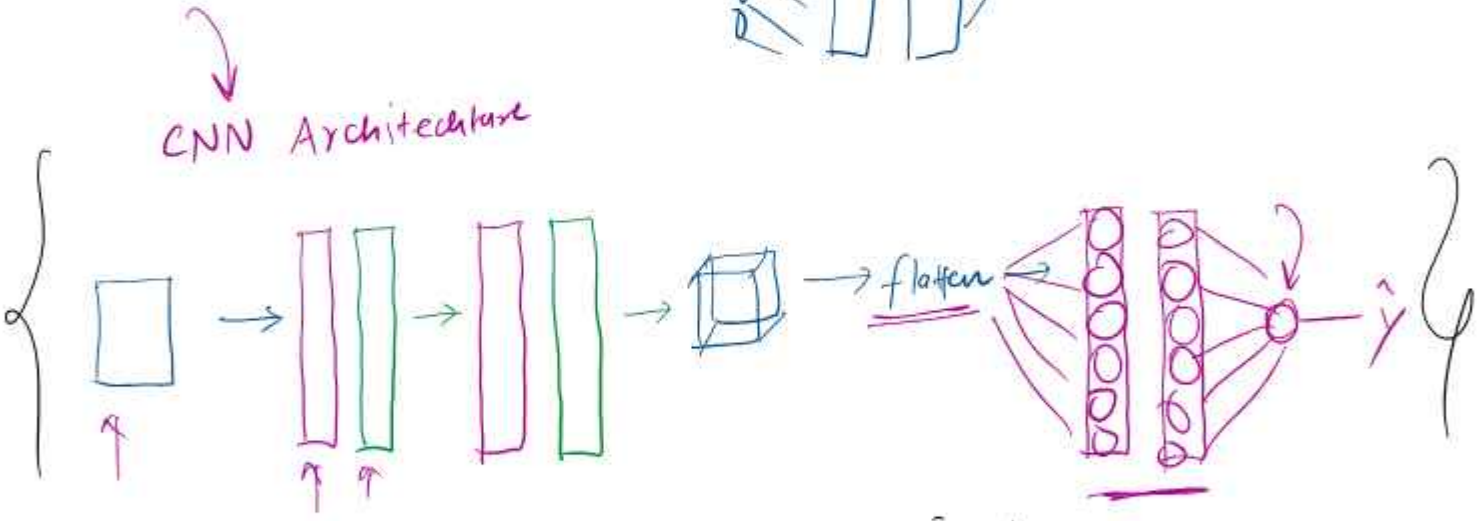
02 September 2022 10:55

1) Convolution 2) Padding / stride 3) Pooling





CNN Architecture



# conv # filters	stride padding	FC nodes # FC layer	activation dropout	Batch Norm

ImageNET

1) [LeNET] → Yann LeCunn
2) AlexNET
3) GoogLeNET
4) VggNET
5) ResNTE
6) Inception

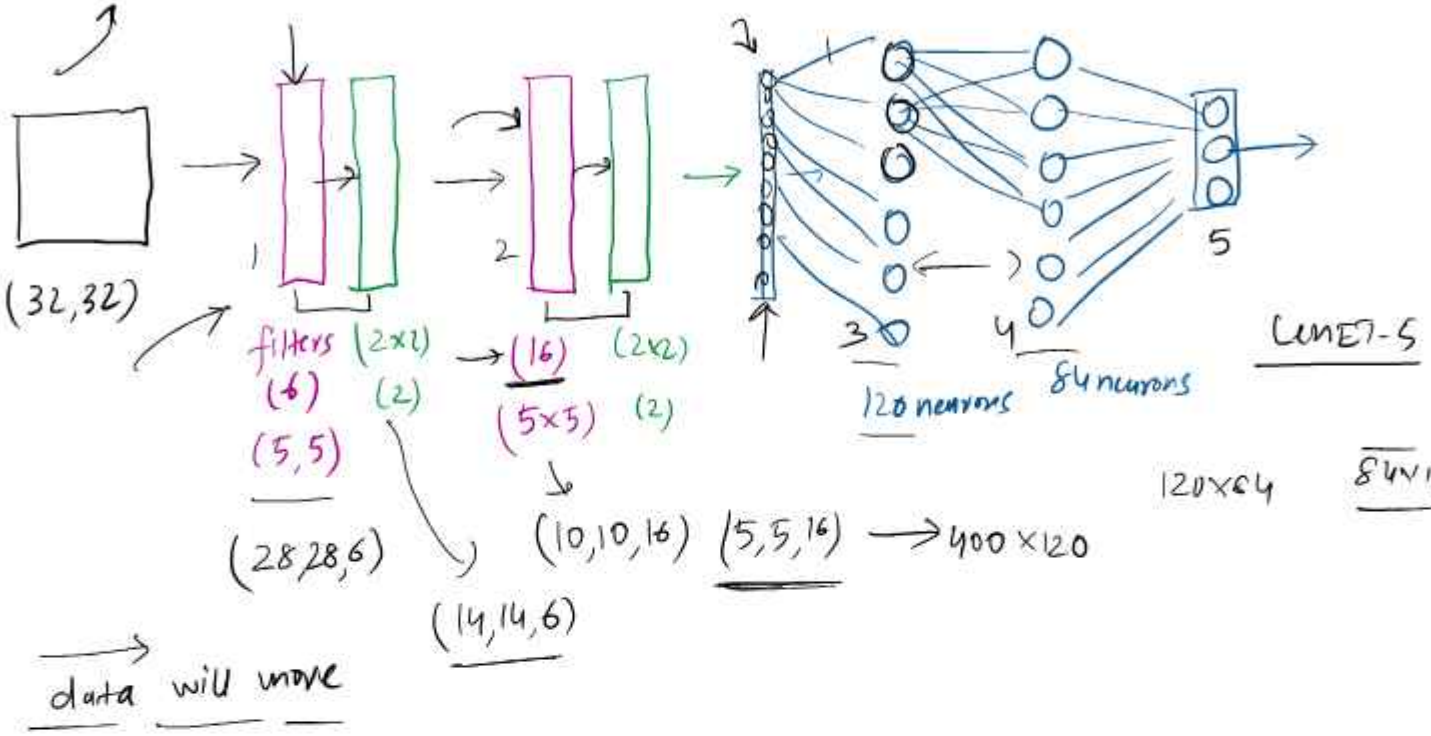
3) GoogLeNET

LeNet

02 September 2022 10:55

Yann LeCun → 1989 → 1998 → LeNET → CNN → US Navy Postal Service

LeNET-5



→ data will move

- AlexNET
- VggNET
- ResNET
- Inception

Guidelines

02 September 2022 10:58

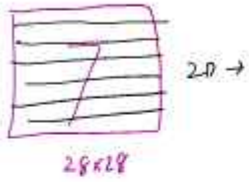
Keras Code

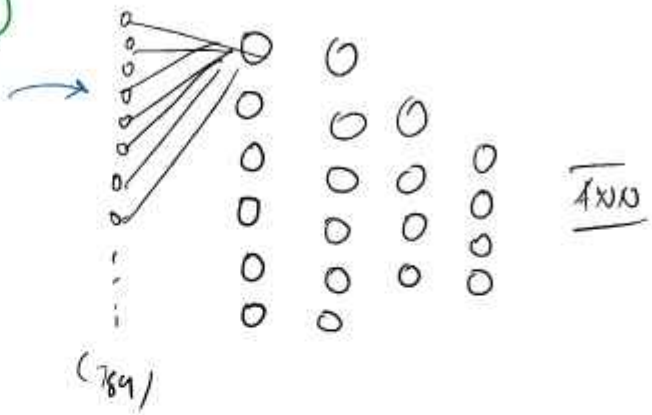
02 September 2022 14:58

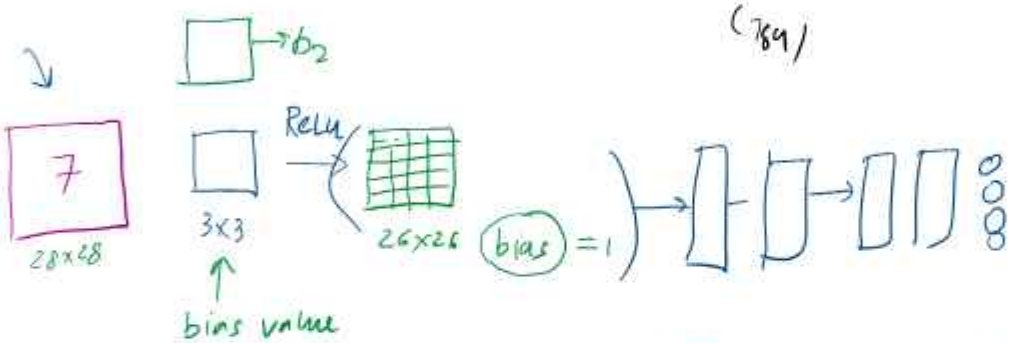
CNN Vs ANN

06 September 2022 10:00

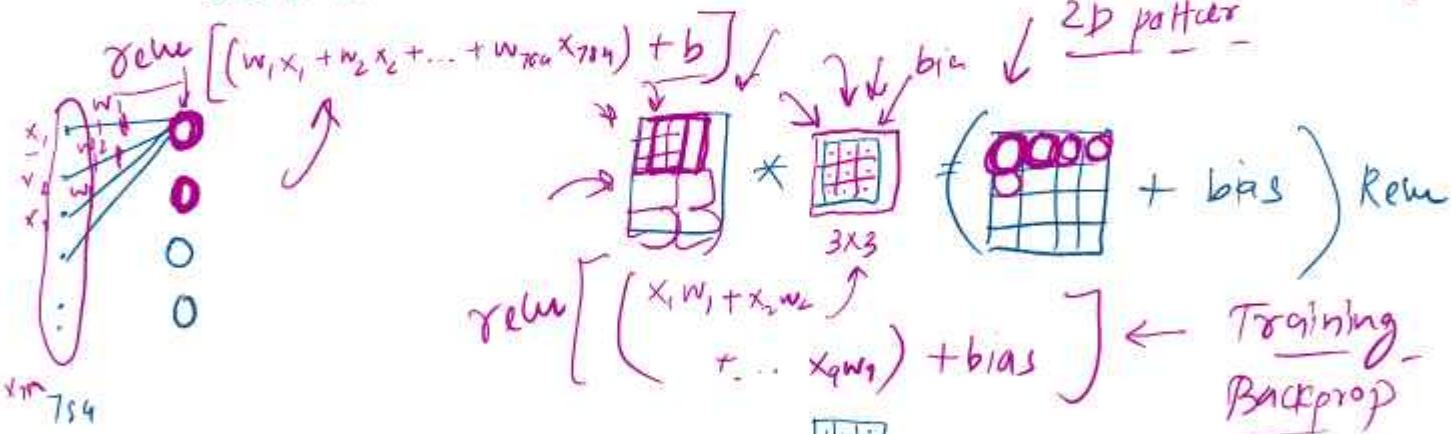
1) Computation Cost → ↓↓
2) Overfitting →
3) Loss of imp features like spatial arrangement of pixels

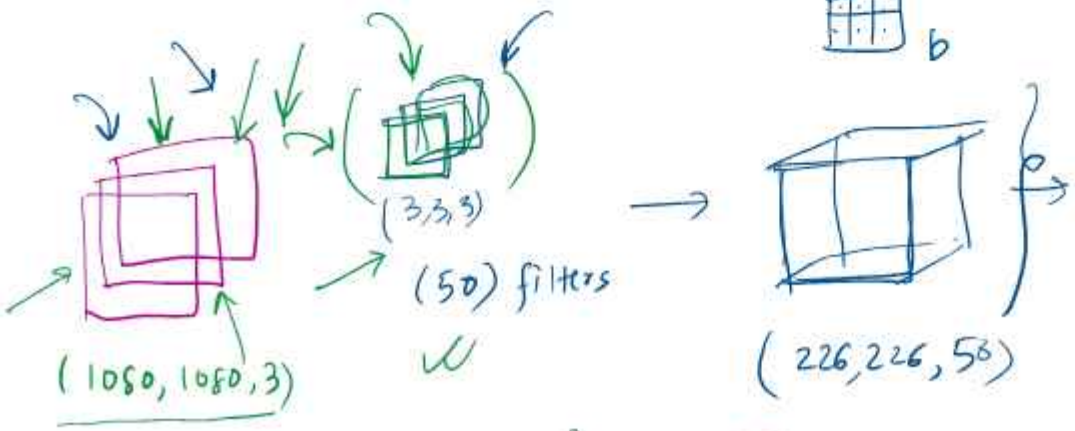






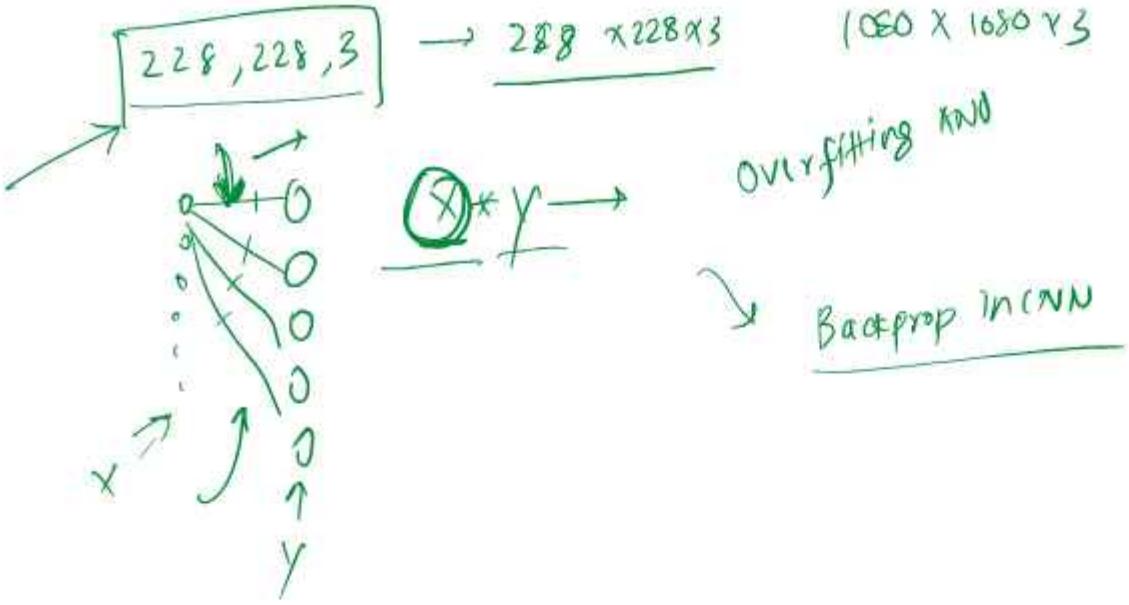
similarity ANN CNN





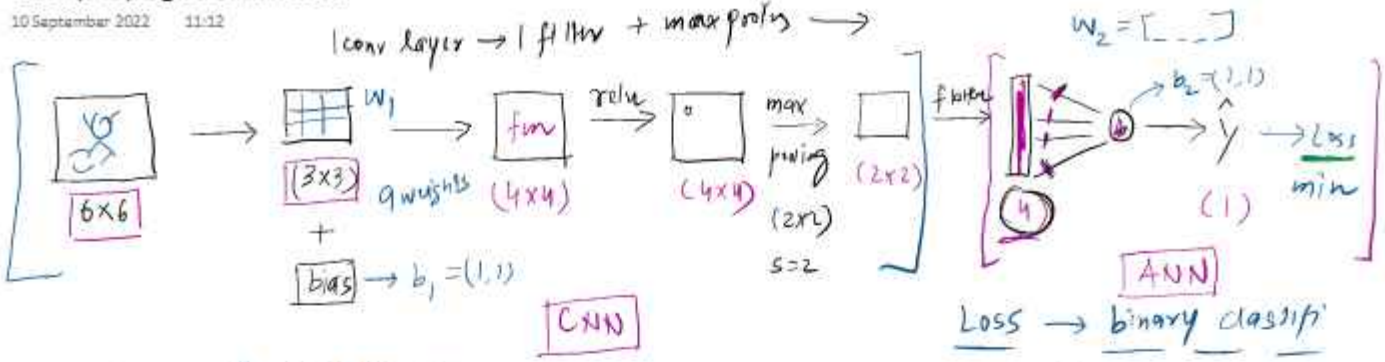
learnable parameters 1400

1350 wts
+50
1400



Backpropagation in CNN

10 September 2022 11:12



Loss → binary classification

Trainable Parameters

$W_1 = (3,3)$ $W_2 = (1,4)$ $= 15$ trainable parameters

$b_1 = (1,1)$ $b_2 = (1,1)$

$$L = -y_i \log(\hat{y}_i) - (1-y_i)\log(1-\hat{y}_i)$$

$A_2 = \hat{y}$

Logical Flow

Forward Prop

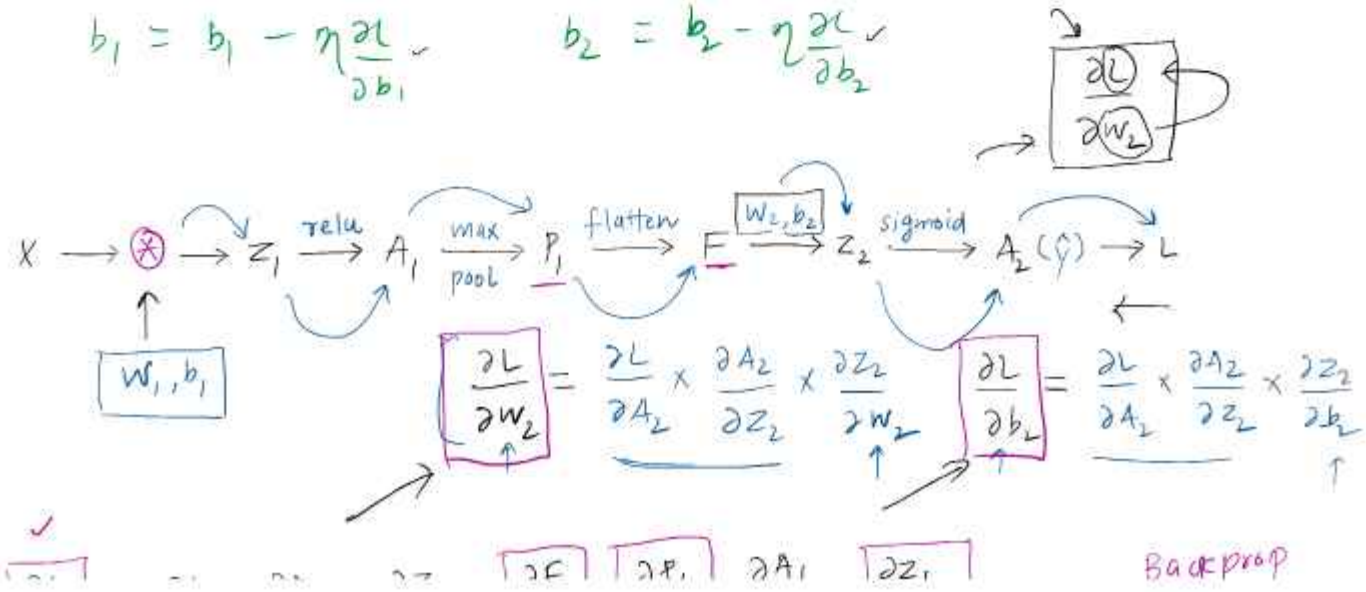
$$Z_1 = Conv(X, W_1) + b_1$$
$$A_1 = relu(Z_1)$$
$$P_1 = maxpool(A_1)$$
$$F = flatten(P_1)$$
$$Z_2 = W_2 F + b_2$$
$$A_2 = \sigma(Z_2)$$

Gradient Descent

$$W_1 = W_1 - \eta \frac{\partial L}{\partial W_1} \qquad W_2 = W_2 - \eta \frac{\partial L}{\partial W_2}$$

$$b_1 = b_1 - \eta \frac{\partial L}{\partial b_1} \qquad b_2 = b_2 - \eta \frac{\partial L}{\partial b_2}$$

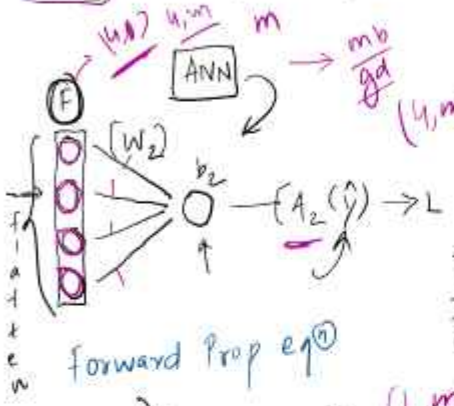
Loss is minimized



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial A_2} \times \frac{\partial A_2}{\partial Z_2} \times \frac{\partial Z_2}{\partial F} \times \frac{\partial F}{\partial P_1} \times \frac{\partial P_1}{\partial A_1} \times \frac{\partial A_1}{\partial Z_1} \times \frac{\partial Z_1}{\partial w_1}$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial A_2} \times \frac{\partial A_2}{\partial Z_2} \times \frac{\partial Z_2}{\partial F} \times \frac{\partial F}{\partial P_1} \times \frac{\partial P_1}{\partial A_1} \times \frac{\partial A_1}{\partial Z_1} \times \frac{\partial Z_1}{\partial b_1}$$

Backprop
- → Convolution
- → Flatten
- → Max pooling



(4,m)

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial A_2} \frac{\partial A_2}{\partial Z_2} \frac{\partial Z_2}{\partial w_2} \qquad \frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial A_2} \frac{\partial A_2}{\partial Z_2} \frac{\partial Z_2}{\partial b_2}$$

Forward Prop eq②

$$\begin{cases} Z_2 = W_2 F + b_2 \\ A_2 = \sigma(Z_2) \end{cases}$$

$$\frac{\partial L}{\partial a_2} = \frac{\partial}{\partial a_2}\left[-y_i \log(a_2) - (1-y_i)\log(1-a_2)\right]$$

(1,m) $Y = (1,m)$

$$= -\frac{y_i}{a_2} + \frac{(1-y_i)}{(1-a_2)} = \frac{-y_i(1-a_2) + a_2(1-y_i)}{a_2(1-a_2)}$$

$$\frac{\partial L}{\partial a_2} = \frac{-y_i + \cancel{y_i a_2} + a_2 - \cancel{a_2 y_i}}{a_2(1-a_2)} = \frac{(a_2 - y_i)}{a_2(1-a_2)}$$

$$\left[\frac{\partial A_2}{\partial Z_2} = \sigma(Z_2)[1-\sigma(Z_2)] = a_2[1-a_2]\right.$$

W_2 update

Shape =

$$\boxed{\frac{\partial Z_2}{\partial w_2} = F} \qquad \frac{\partial Z_2}{\partial b_2} = 1$$

(1,4)

$$\frac{\partial L}{\partial w_2} = \frac{(a_2 - y_i)}{\cancel{a_2(1-a_2)}} \times \cancel{a_2(1-a_2)} \times F = (a_2 - y_i) F = (A_2 - Y) F^T$$

(1,1) (1,4) (4,1)

(1,1) (1,4)

(1,4)

$$\frac{\partial L}{\partial b_2} = \frac{(a_2 - y_i)}{\cancel{a_2(1-a_2)}} \cancel{a_2(1-a_2)} \times 1 = (A_2 - Y)$$

m images

$$\boxed{\frac{\partial L}{\partial w_2} = (A_2 - Y) F^T} \qquad \boxed{\frac{\partial L}{\partial b_2} = (A_2 - Y)}$$

$\frac{\partial L}{\partial w_2} = (A_2 - Y)A_1^T$ | $\frac{\partial L}{\partial b_2}$ =

$(1,m) - (1,m)$

$(1,m)\ (m,n) \rightarrow (1,n)$

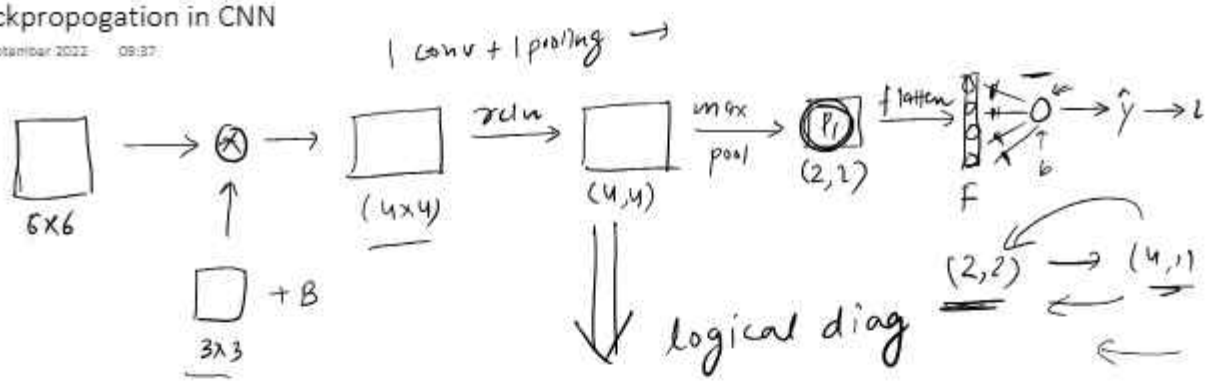
1 image

batch of image

$w_2 \rightarrow (1,n)$

Backpropogation in CNN

13 September 2022 09:37



Forward Prop

$Z_1 = conv(X, W_1) + b_1$

$A_1 = relu(Z_1)$

$P_1 = maxpool(A_1)$

$F = flatten(P_1)$

flatten

$Z_2 = W_2 F + b_2$

$A_2 = \sigma(Z_2)$

$L = \frac{1}{m} \sum_{i=1}^{m} [-y_i \log(A_2) - (1-y_i) \log(1-A_2)]$

$(A_2 - Y)$

$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial A_2} \times \frac{\partial A_2}{\partial Z_2} \frac{\partial Z_2}{\partial F} \frac{\partial F}{\partial P_1} \frac{\partial P_1}{\partial A_1} \frac{\partial A_1}{\partial Z_1} \frac{\partial Z_1}{\partial W_1}$ (5)

$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial A_2} \frac{\partial A_2}{\partial Z_2} \frac{\partial Z_2}{\partial F} \frac{\partial F}{\partial P_1} \frac{\partial P_1}{\partial A_1} \frac{\partial A_1}{\partial Z_1} \frac{\partial Z_1}{\partial b_1}$ (6)

reshape(P_1.shape)

6 derivatives

$\left[\frac{\partial Z_2}{\partial F}\right] = W_2$ →

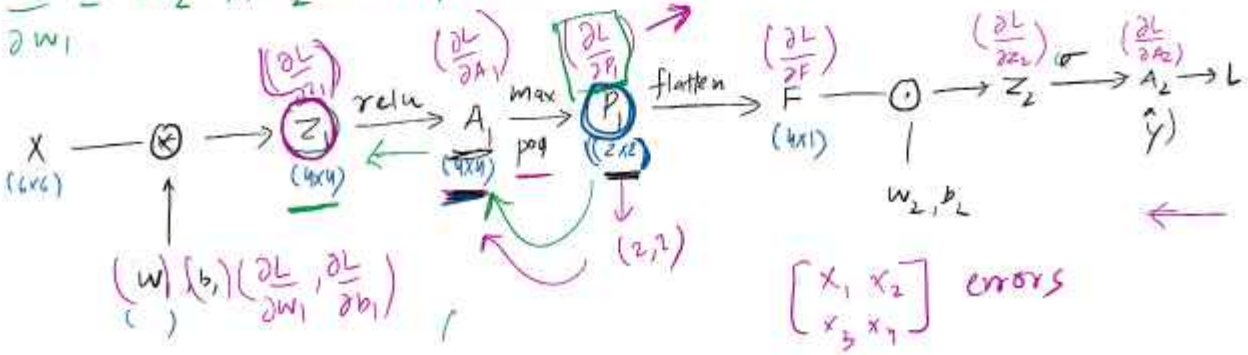
Shape ? → (F)

$\frac{\partial F}{\partial P_1}$ no trainable parameters

$conv(X, \frac{\partial L}{\partial Z_1})$

$\frac{\partial L}{\partial W_1} = (A_2 - Y) W_2 .\text{reshape}(P_1.\text{shape})$

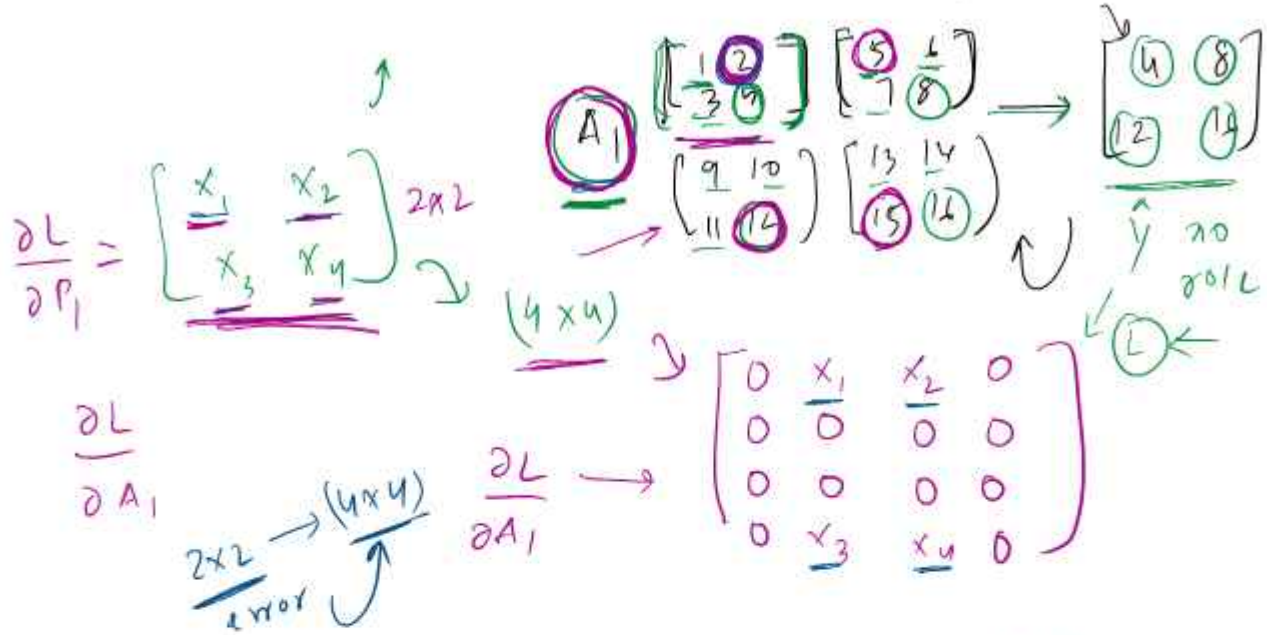
$\begin{bmatrix} x_1 & x_2 \\ x_3 & x_4 \end{bmatrix}$

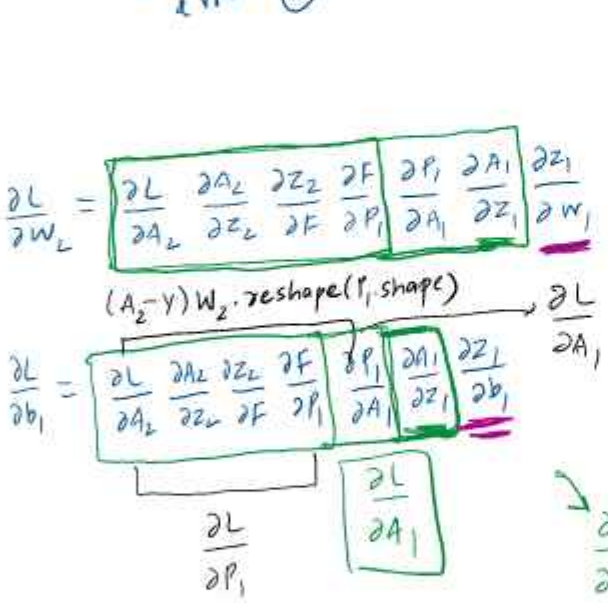


$\frac{\partial L}{\partial A_1} = (4,4)$

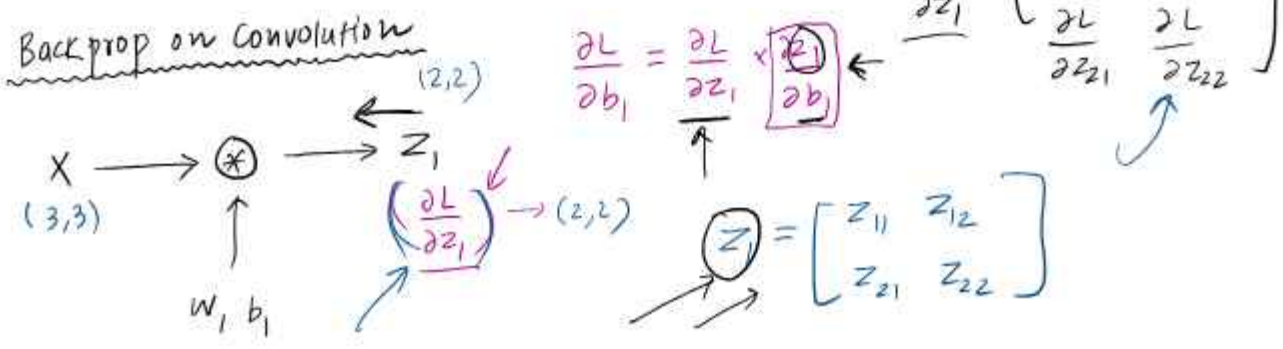
$\frac{\partial L}{\partial A_1} = (4,4)$

Flatten → no trainable parameters





Convolution → Backprop → max pooling → Flatten





$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} \circledast W_1 = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} + b_1$$

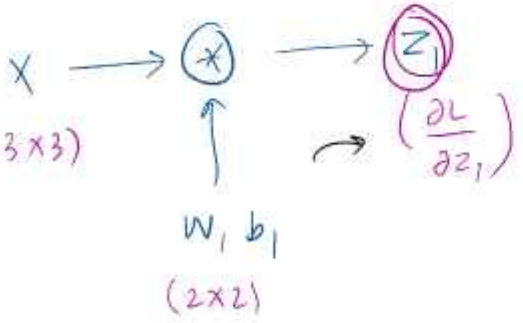
$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1} \times \frac{\partial z_1}{\partial b_1} = \frac{\partial L}{\partial z_{11}} \times \frac{\partial z_{11}}{\partial b_1} + \frac{\partial L}{\partial z_{12}} \times \frac{\partial z_{12}}{\partial b_1} + \frac{\partial L}{\partial z_{21}} \times \frac{\partial z_{21}}{\partial b_1} + \frac{\partial L}{\partial z_{22}} \times \frac{\partial z_{22}}{\partial b_1}$$

$$= \left(\frac{\partial L}{\partial z_{11}} + \frac{\partial L}{\partial z_{12}} + \frac{\partial L}{\partial z_{21}} + \frac{\partial L}{\partial z_{22}} \right) = \text{sum}\left(\frac{\partial L}{\partial z_1} \right)$$

$$\frac{\partial L}{\partial b_1} = \text{sum}\left(\frac{\partial L}{\partial z_1} \right) \rightarrow \text{scalar}$$

bias

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} \quad W_1 = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} + b$$



$$\frac{\partial L}{\partial W_1} = \begin{bmatrix} \frac{\partial L}{\partial w_{11}} & \frac{\partial L}{\partial w_{12}} \\ \frac{\partial L}{\partial w_{21}} & \frac{\partial L}{\partial w_{22}} \end{bmatrix} \qquad \frac{\partial L}{\partial z_1} = \begin{bmatrix} \frac{\partial L}{\partial z_{11}} & \frac{\partial L}{\partial z_{12}} \\ \frac{\partial L}{\partial z_{21}} & \frac{\partial L}{\partial z_{22}} \end{bmatrix}$$

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} \times \frac{\partial z_1}{\partial W_1}$$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_{11}} \times \frac{\partial z_{11}}{\partial w_{11}} + \frac{\partial L}{\partial z_{12}} \times \frac{\partial z_{12}}{\partial w_{11}} + \frac{\partial L}{\partial z_{21}} \times \frac{\partial z_{21}}{\partial w_{11}} + \frac{\partial L}{\partial z_{22}} \times \frac{\partial z_{22}}{\partial w_{11}}$$

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial z_{11}} \times \frac{\partial z_{11}}{\partial w_{12}} + \frac{\partial L}{\partial z_{12}} \times \frac{\partial z_{12}}{\partial w_{12}} + \frac{\partial L}{\partial z_{21}} \times \frac{\partial z_{21}}{\partial w_{12}} + \frac{\partial L}{\partial z_{22}} \times \frac{\partial z_{22}}{\partial w_{12}}$$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial z_{11}} \times \frac{\partial z_{12}}{\partial w_{21}} + \frac{\partial L}{\partial z_{21}} \times \frac{\partial z_{21}}{\partial w_{21}} + \frac{\partial L}{\partial z_{22}} \times \frac{\partial z_{22}}{\partial w_{21}}$$

$$\frac{\partial L}{\partial w_{12}} \quad \frac{\partial z_{11}}{} \quad \frac{\partial}{\partial w_{12}} \quad - \quad - \quad - \quad -$$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial z_{11}} \times \frac{\partial z_{11}}{\partial w_{21}} + \frac{\partial L}{\partial z_{12}} \times \frac{\partial z_{12}}{\partial w_{21}} + \frac{\partial L}{\partial z_{21}} \times \frac{\partial z_{21}}{\partial w_{21}} + \frac{\partial L}{\partial z_{22}} \times \frac{\partial z_{22}}{\partial w_{21}}$$

$$\frac{\partial L}{\partial w_{22}} = \frac{\partial L}{\partial z_{11}} \times \frac{\partial z_{11}}{\partial w_{22}} + \frac{\partial L}{\partial z_{12}} \times \frac{\partial z_{12}}{\partial w_{22}} + \frac{\partial L}{\partial z_{21}} \times \frac{\partial z_{21}}{\partial w_{22}} + \frac{\partial L}{\partial z_{22}} \times \frac{\partial z_{22}}{\partial w_{22}}$$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_{11}} x_{11} + \frac{\partial L}{\partial z_{12}} x_{12} + \frac{\partial L}{\partial z_{21}} x_{21} + \frac{\partial L}{\partial z_{22}} x_{22}$$

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial z_{11}} x_{12} + \frac{\partial L}{\partial z_{12}} x_{13} + \frac{\partial L}{\partial z_{21}} x_{22} + \frac{\partial L}{\partial z_{22}} x_{23}$$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial z_{11}} x_{21} + \frac{\partial L}{\partial z_{12}} x_{22} + \frac{\partial L}{\partial z_{21}} x_{31} + \frac{\partial L}{\partial z_{22}} x_{32}$$

$$\frac{\partial L}{\partial w_{22}} = \frac{\partial L}{\partial z_{11}} x_{22} + \frac{\partial L}{\partial z_{12}} x_{23} + \frac{\partial L}{\partial z_{21}} x_{32} + \frac{\partial L}{\partial z_{22}} x_{33}$$

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} \qquad \frac{\partial L}{\partial z_1} = \begin{bmatrix} \frac{\partial L}{\partial z_{11}} & \frac{\partial L}{\partial z_{12}} \\ \frac{\partial L}{\partial z_{21}} & \frac{\partial L}{\partial z_{22}} \end{bmatrix}$$

$$\frac{\partial L}{\partial w_1} = \text{conv}\left(X, \frac{\partial L}{\partial z_1}\right)$$

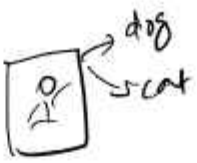
$$\frac{\partial L}{\partial w_1} = \text{conv}\left(X, \frac{\partial L}{\partial z_1}\right)$$

$$\frac{\partial L}{\partial b_1} = \text{sum}\left(\frac{\partial L}{\partial z_1}\right)$$

Why use Pretrained models?

09 October 2022 12:52

1) Data hungry → image data → labelled data → cat dog → google



pinterest

labelling

10000 photos

manually

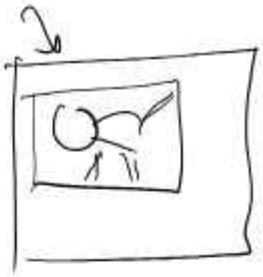
2) Time → model → training

Pretrained model

ImageNET Dataset

03 October 2022 12:36

Visual Database of images (Why What and How)



Why

2006 → Fei fei Li → model and algorithms

1.4 million images → 1.4 crore image → 20,000 categories → labelled

1 million images → to label → bounding box → dog, breed, white

↳ object local

How → Crowd sourcing → Amazon mechanical Turk

↳ Dataset → Deep learning

ImageNET challenge

ILSVRC

09 October 2022 12:38

Imagenet challenge → 2010 → Dataset → subset → 1 million → 10 → 20,000 classes → 1000 classes

14 crore

100 image

Machine learning → feature extraction → SIFT, HOG

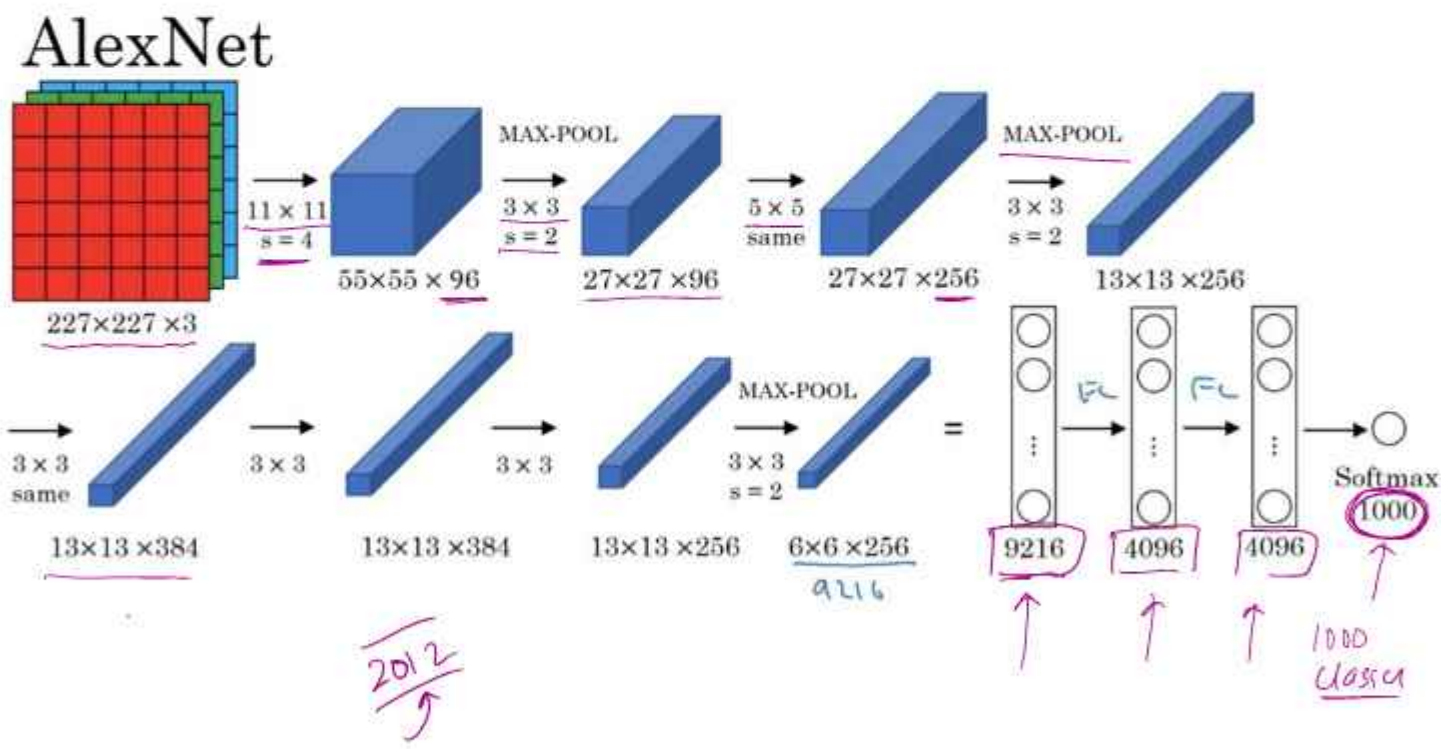
2010 → 28%.

2011 → 25%.

2012 → 16%. → CNN

2012 → DL → AlexNET → CNN → Deep, GPU, relu

2013 →

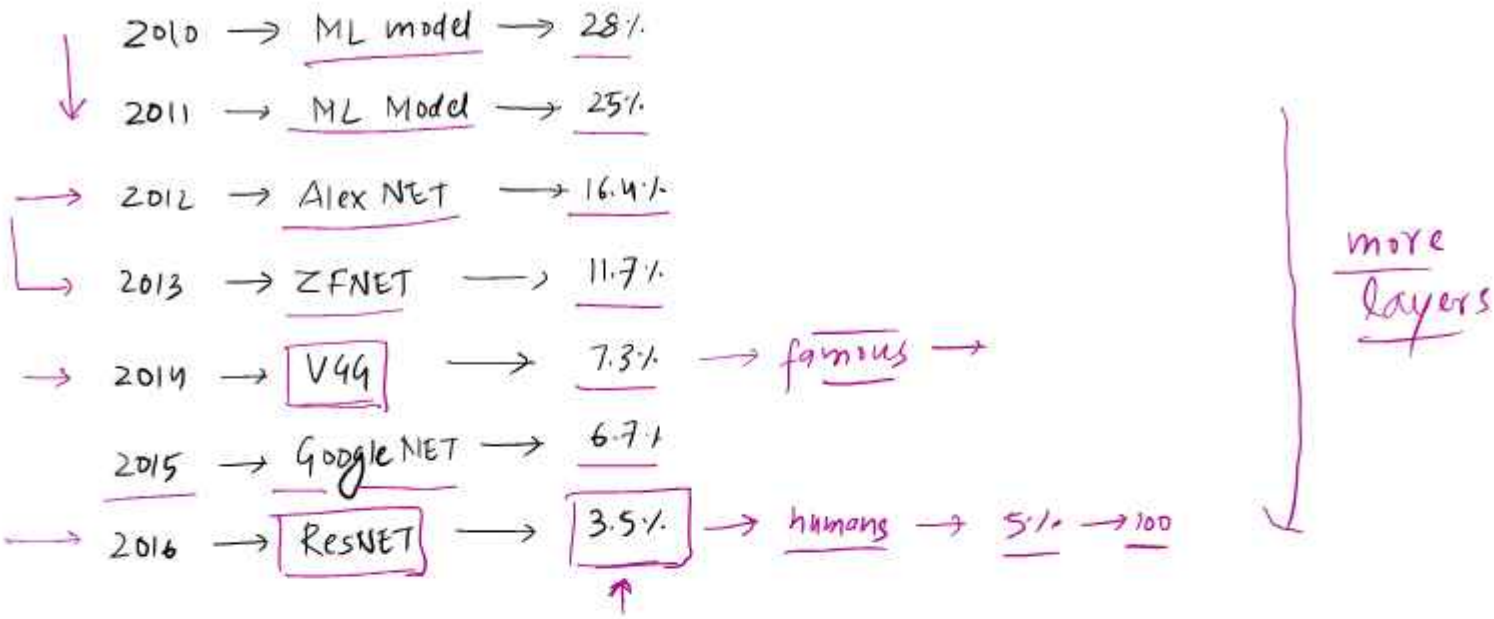


[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

Andrew Ng

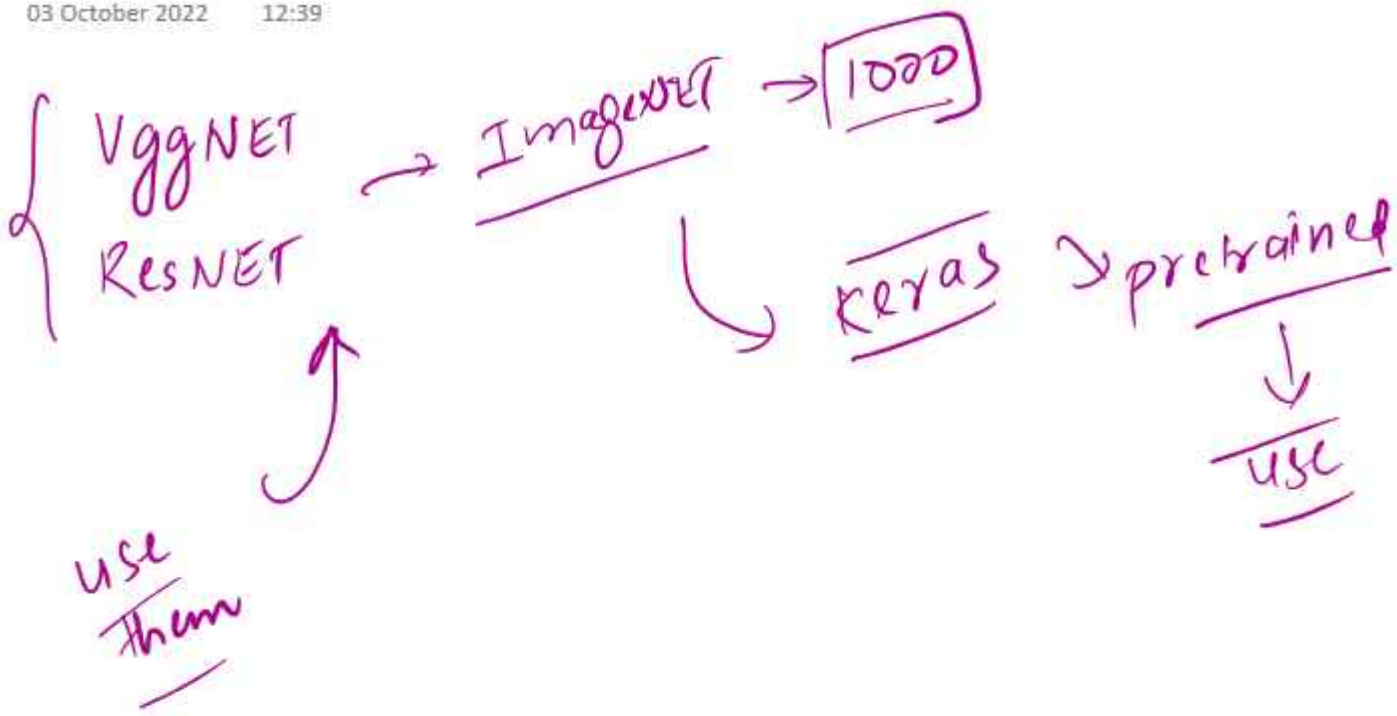
Famous Architectures

03 October 2022 12:39



Idea of Pretrained Models

03 October 2022 12:39



Keras Demo

03 October 2022 12:40

Problem with training your own model

10 October 2022 10:48

1) Data hungry → labelled → 10,000 → google

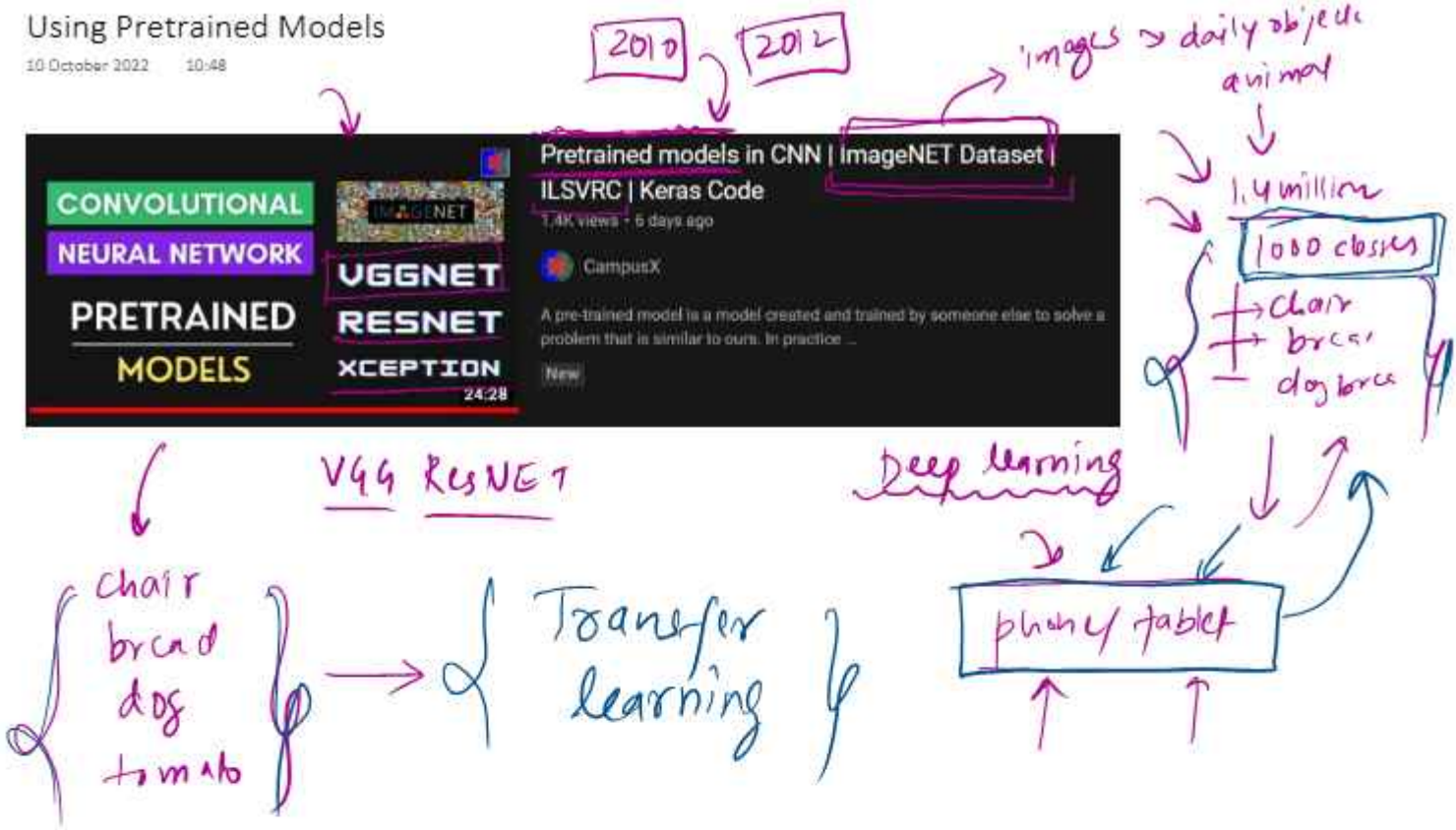
↓

cat/dog → manual labour

2) lot of time → days/weeks/

Using Pretrained Models

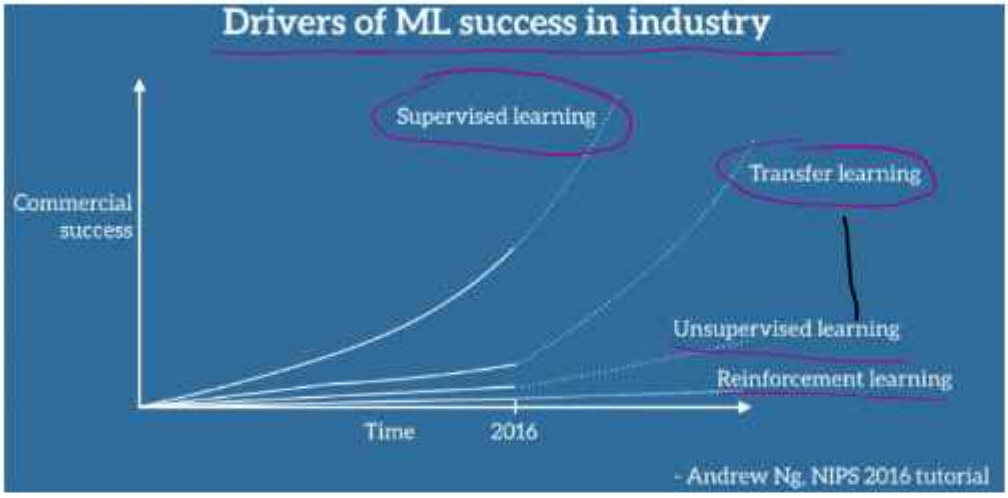
10 October 2022 10:48

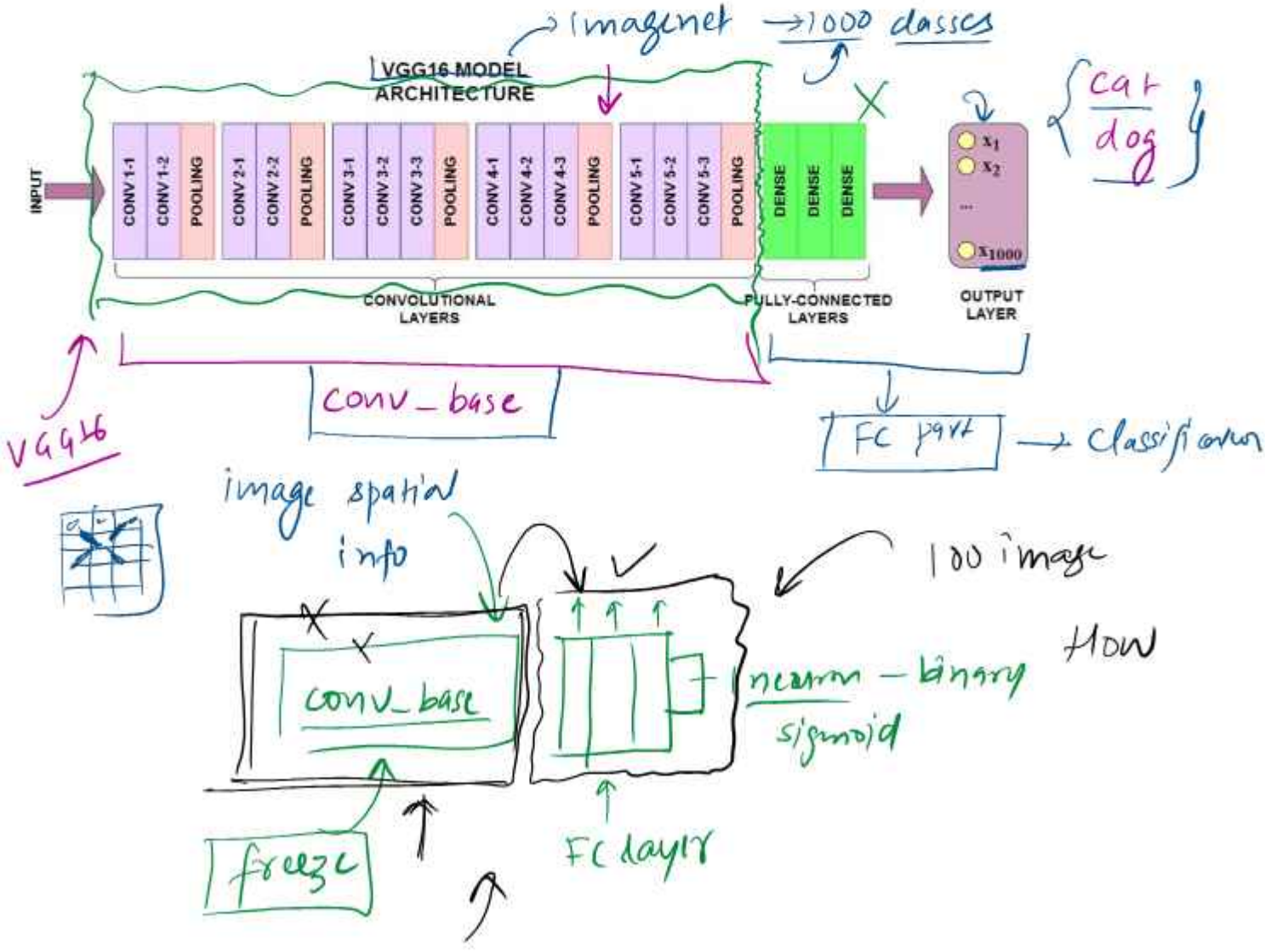


Transfer Learning

10 October 2022 10:49

Transfer learning is a research problem in machine learning that focuses on storing knowledge gained while solving one problem and applying it to a different but related problem.

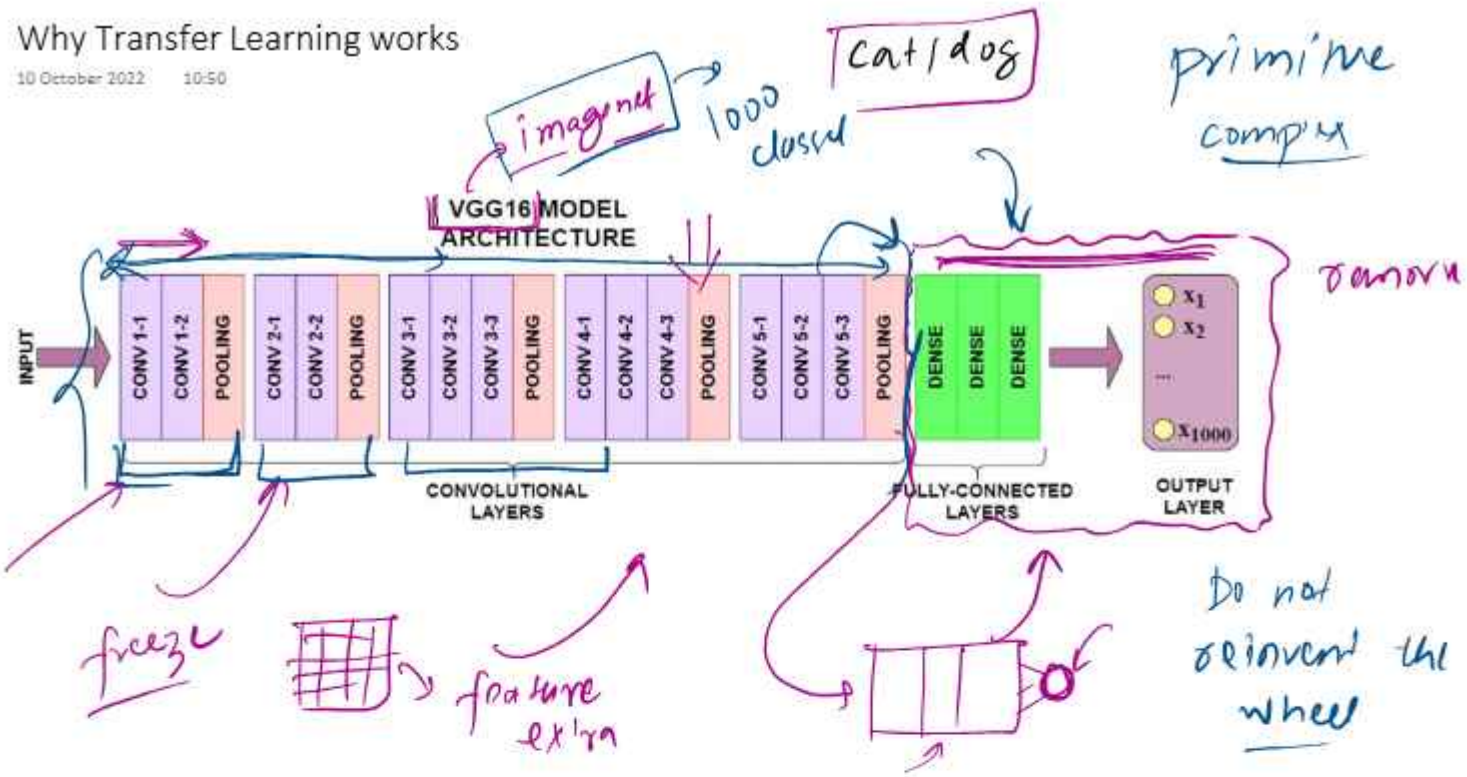






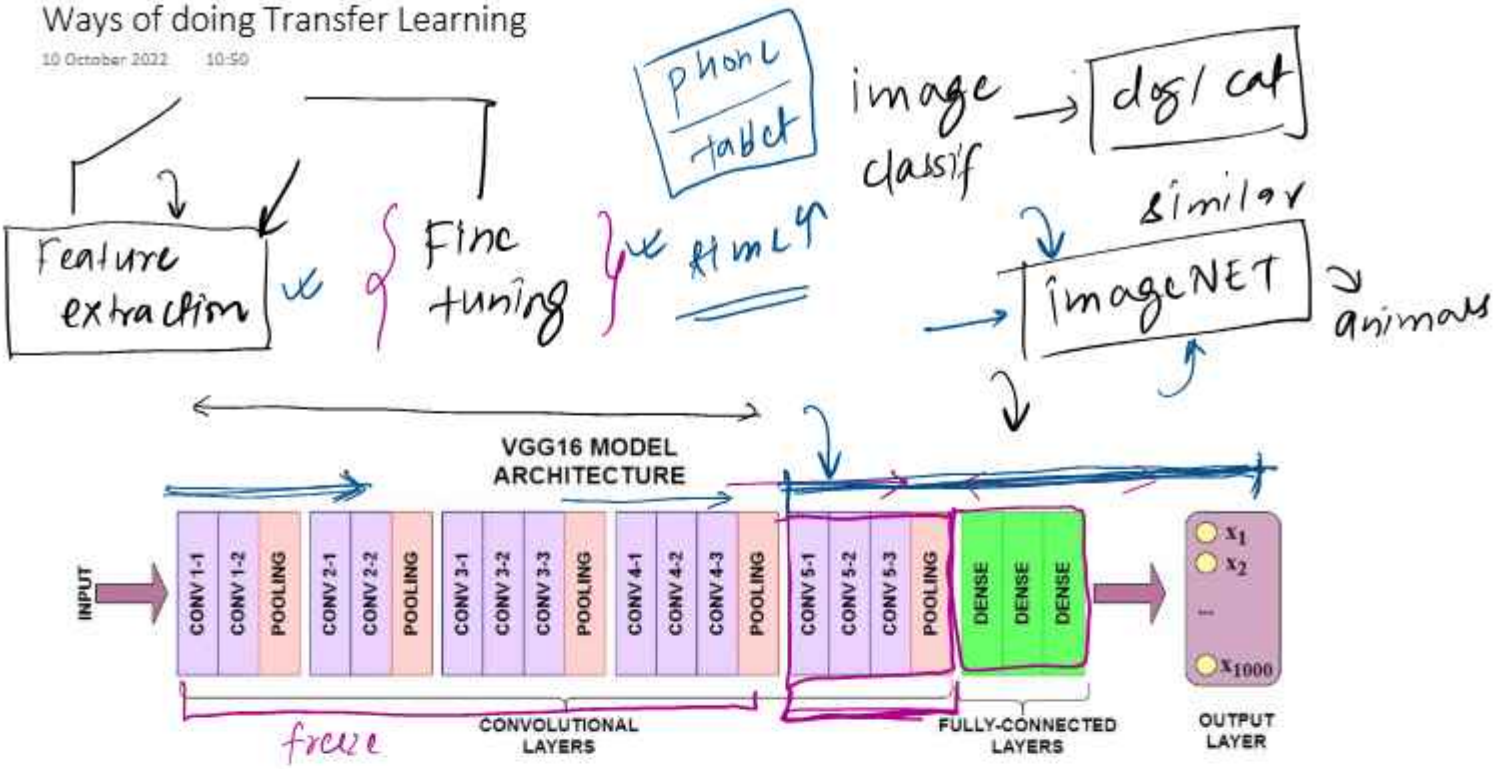
Why Transfer Learning works

10 October 2022 10:50



Ways of doing Transfer Learning

10 October 2022 10:50



Code

10 October 2022 10:50

Problem with Sequential Model

14 October 2022 16:00

ANN ⇔ CNN

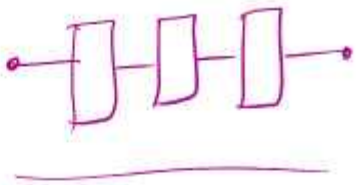
↳ Sequential – Keras

→ 1 input

→ 1 output

→ linear

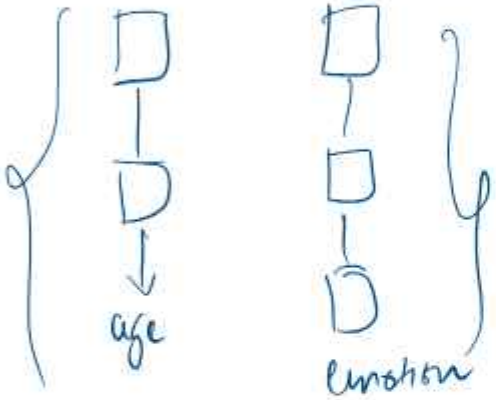
↘ linear topology

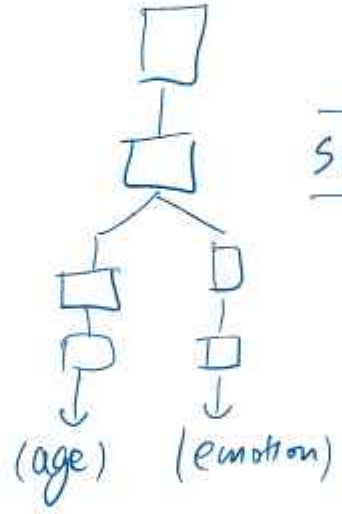


Example →  → image dataset → human faces → 25000 images

→ age?

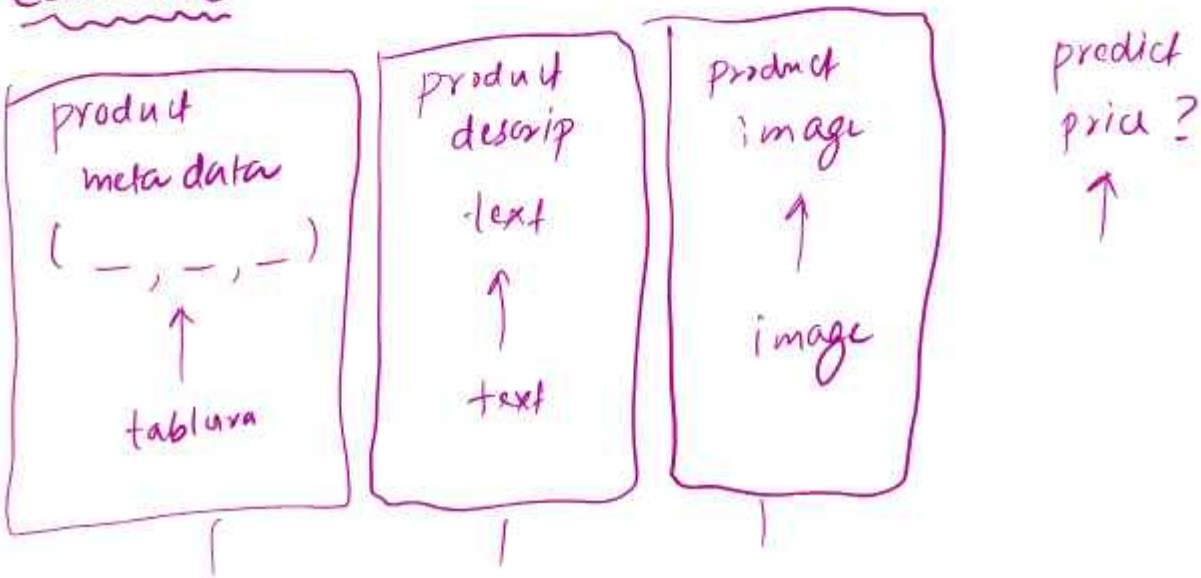
→ emotion → happy / sad / angry

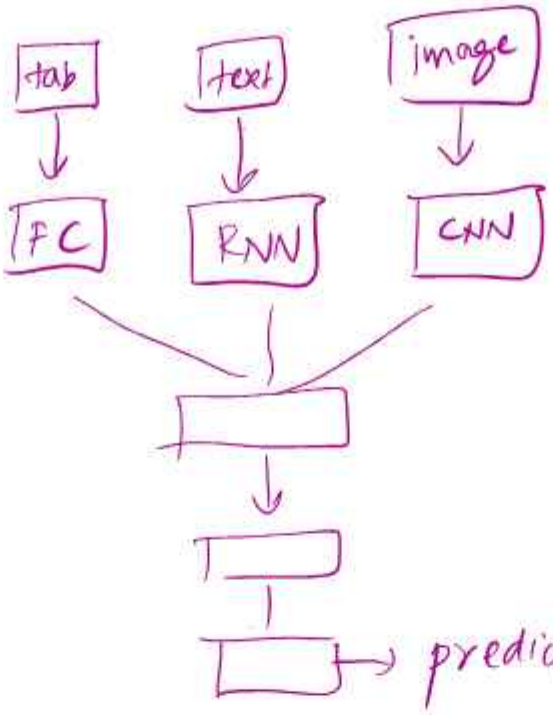




Sequential API

ecommerce





non-linear

Sequential X

Non-linear

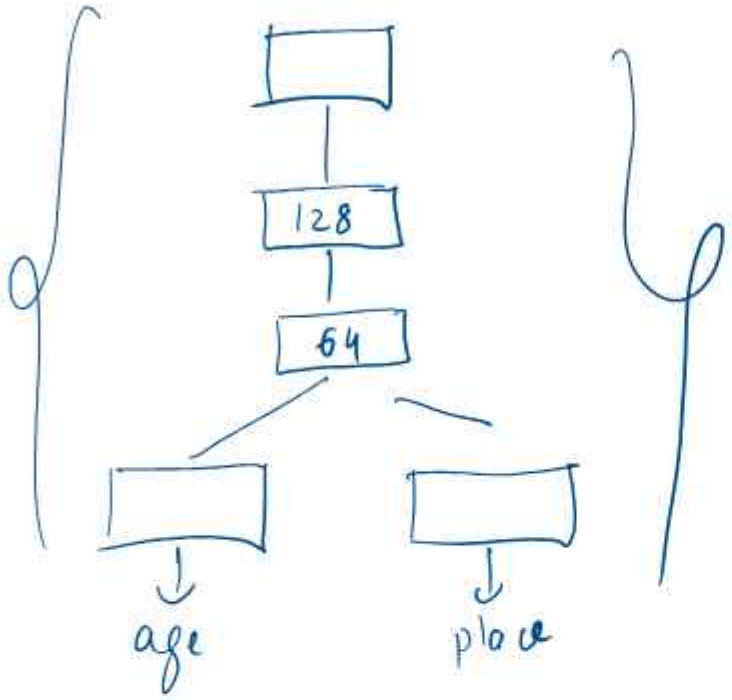
Functional API

A Simple Example

14 October 2022 16:01

| yearly salary | height | marital status |

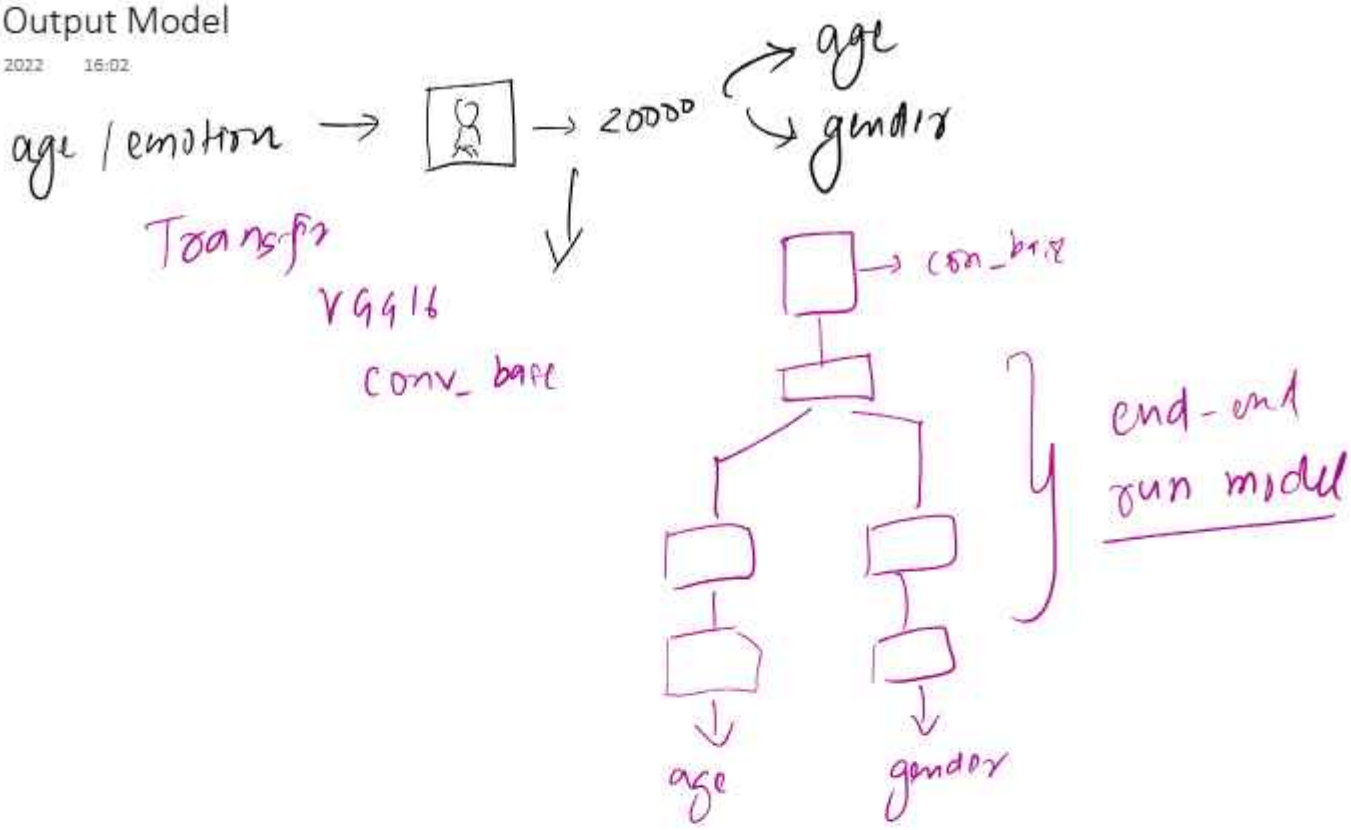
3 cols → age
→ delhi/ mumbai



keras functional api

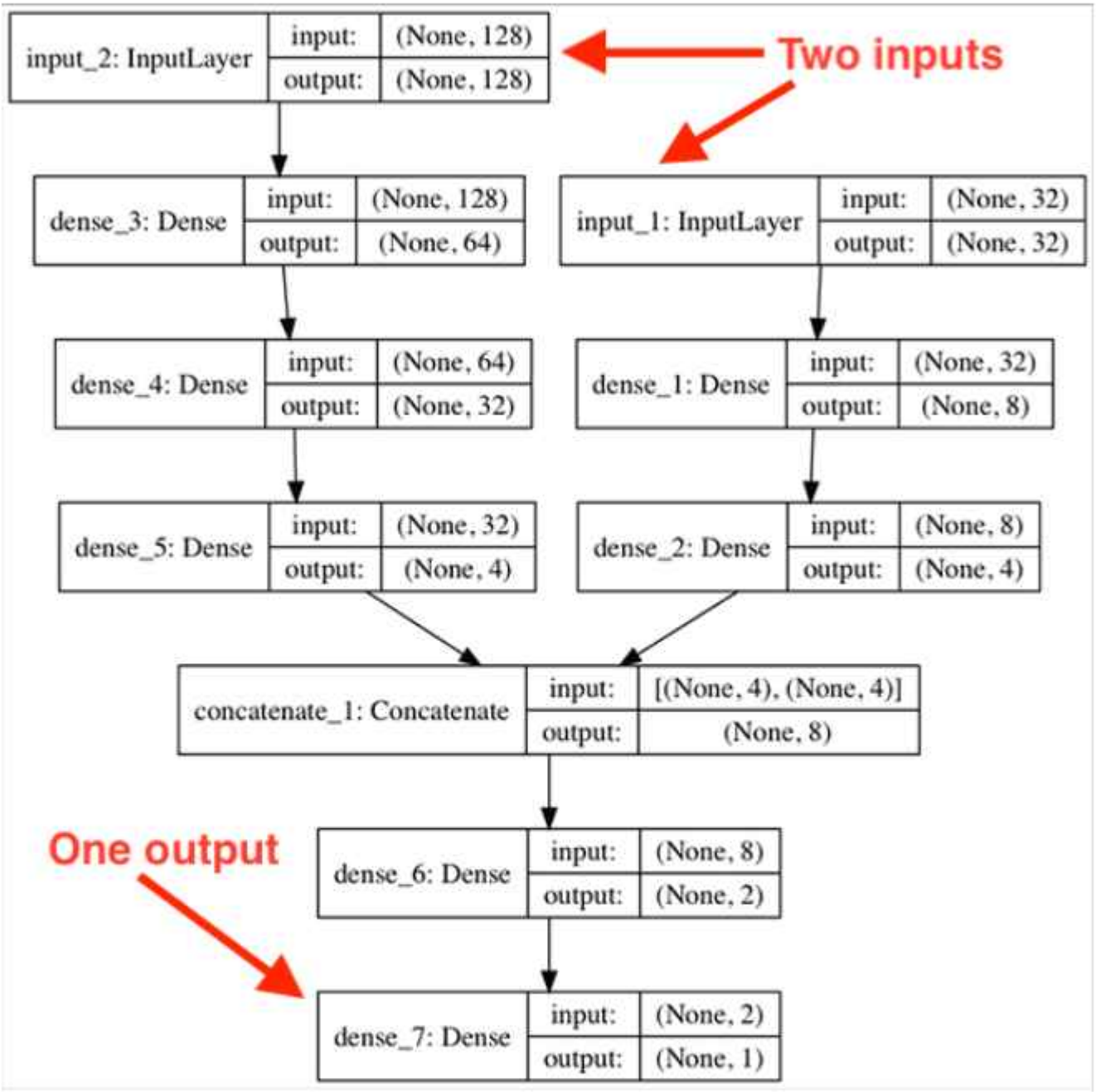
Multi Output Model

14 October 2022 16:02



Multi Input Model

14 October 2022 16:02



Shared Layers Model

14 October 2022 16:02

Sequential Data

22 October 2022 10:09

ANN → tabular data

CNN → images

RNN → Recurrent NN

↳ is type of sequential model to work on sequential data

iq | marks | gender | placement

↳ does not matter

iq, marks, gender → 0 0 0 → No

RNN → NLP → ML

NLP → RNN

CNN → images → computer vision

eg → text → sequential data

Hi my name is Nitish

Time Series → sequential

2001 2002 2003

Speech → sequential

DNA sequence

RNN

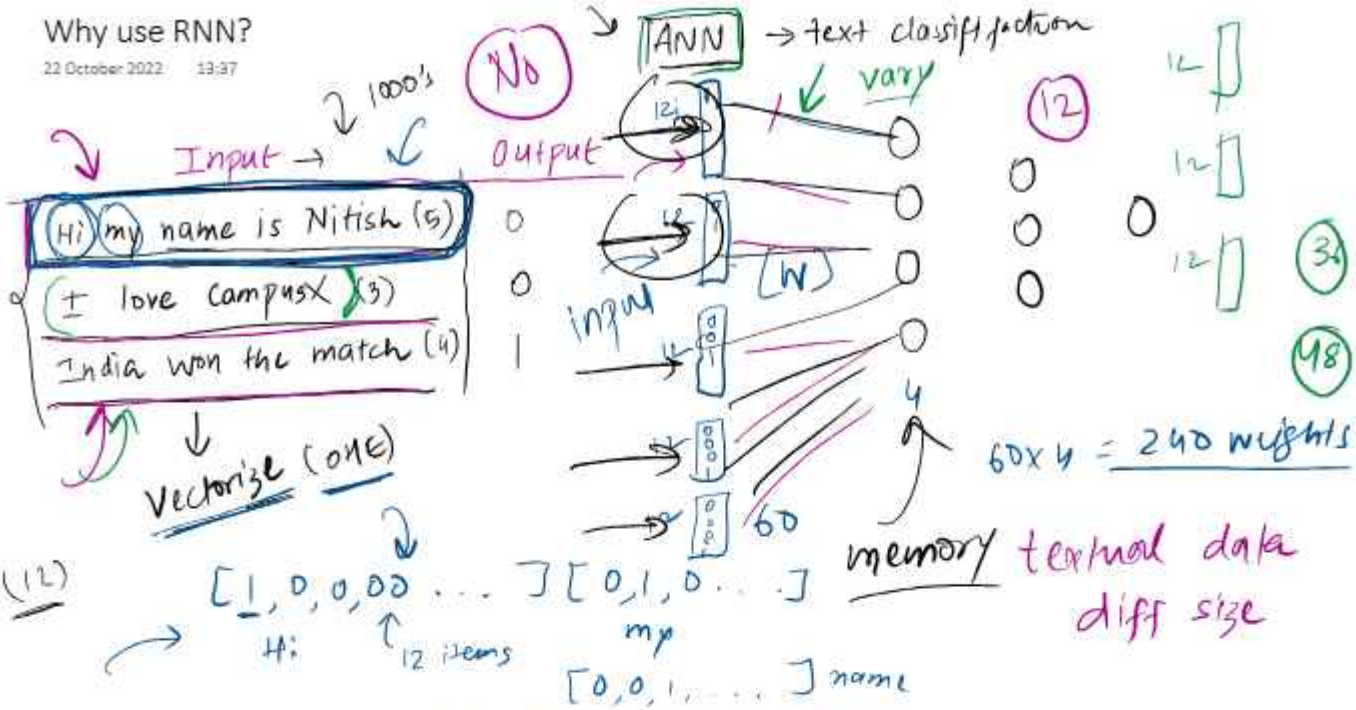
→ RNN Why? → ANN/CNN

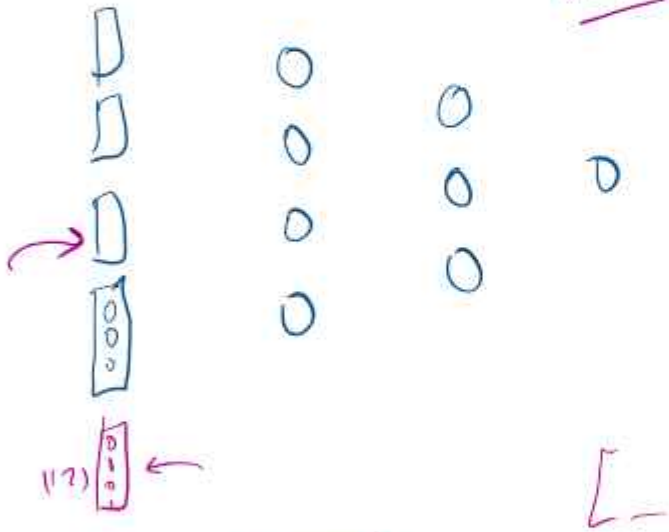
→ Application → RNN

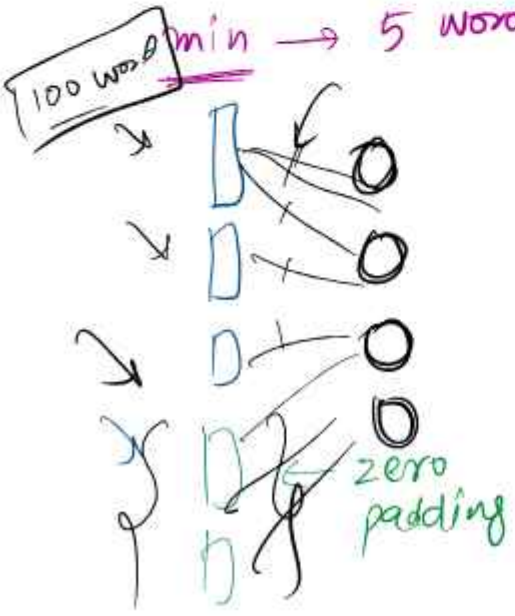
→ Roadmap

Why use RNN?

22 October 2022 13:37







10000 x 100

10000000

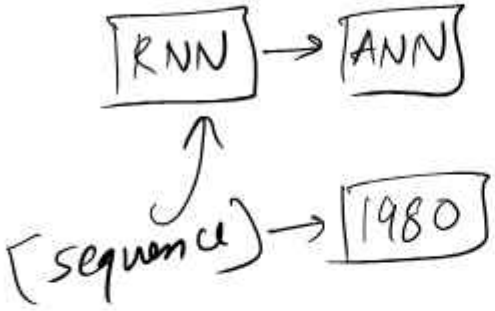
unneccessary computation

fail

prediction ↓ 200 words

n .. input → varying size

1) text input → varying size
2) Zero padding → unnecessary computation
3) Prediction problem
4) Totally disregarding the sequence info

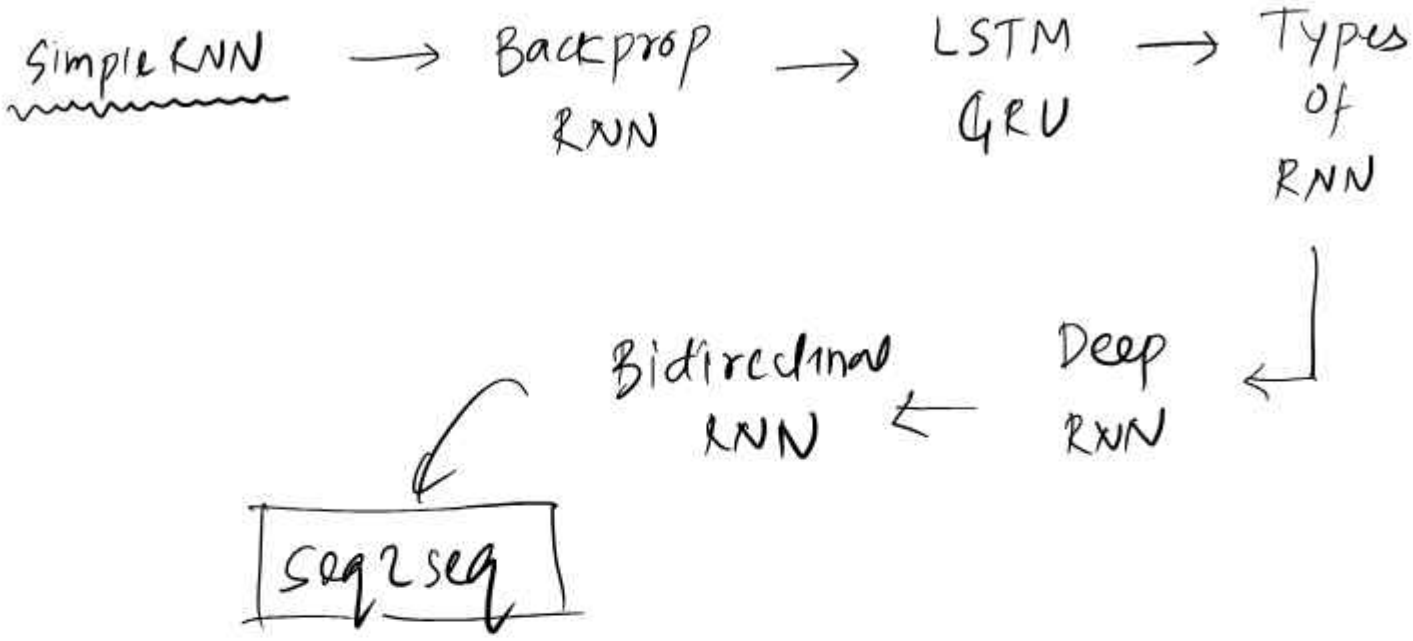


RNN Applications

22 October 2022 13:37

Roadmap

22 October 2022 13:37



Why RNNs?

29 October 2022 13:30

zero padding → cost of compu ↑

1) Sequence → any length → ANN → fixed input size

movie reviews

RNNs

same

NNs

2) sequence contains some meaning

ANN → input

memory feature

RNN architecture

RNN forward prop → prediction

↳ input → output

Codes → solidify

Data for RNN

29 October 2022 13:30

[(timesteps, input_features)]

t=1, t=2, t=3

Review	Sentiment
movie was good	1
movie was bad	0
movie was not good	0

(4)

review 1

[[10000], [01000], [00100]]

vocab → 5 word

movie was good

[10000] [01000] [00100]

[00010] [00001]

bad not

(3,5) → (timesteps, # of features)

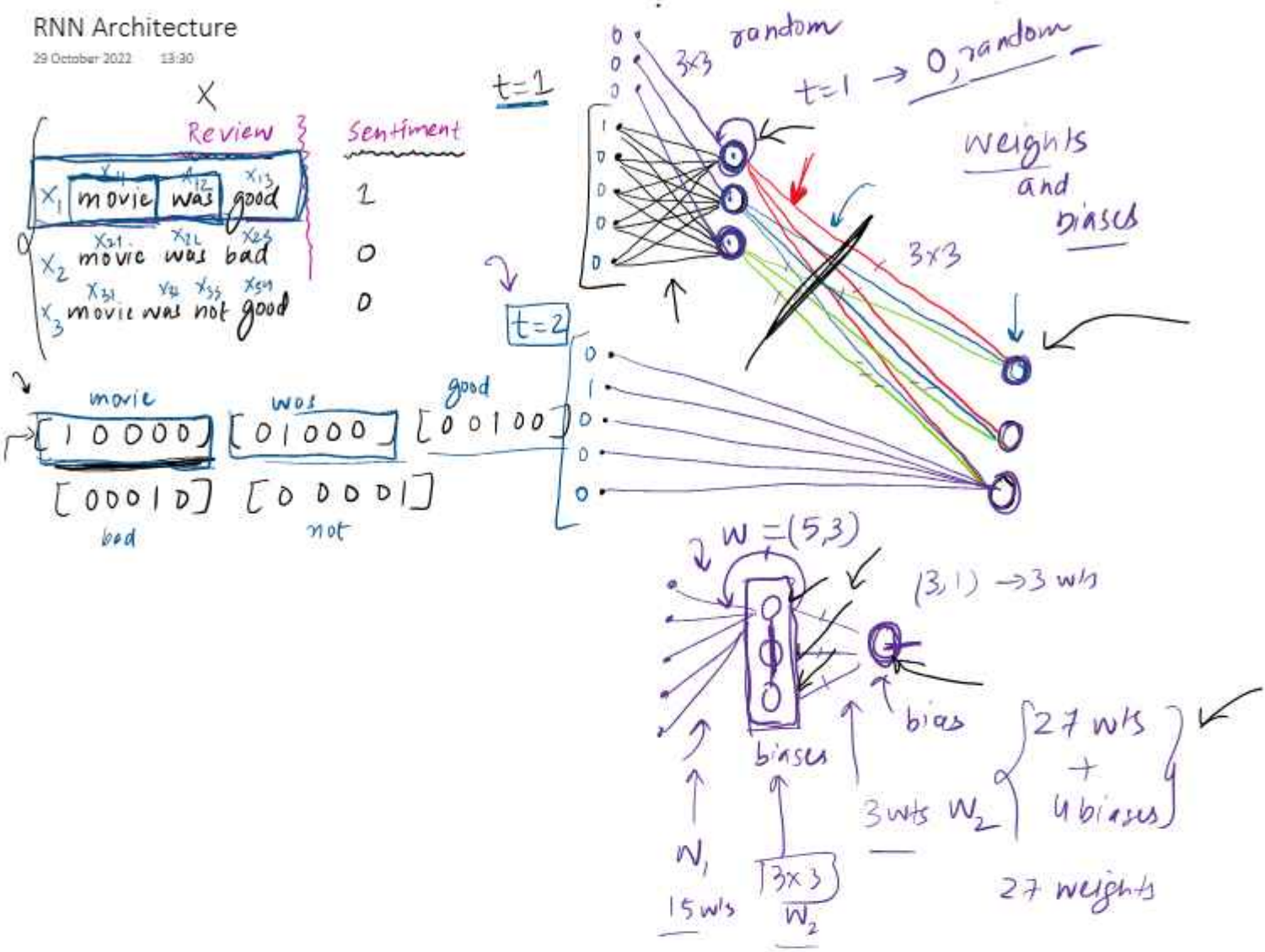
(3,5) (4,5)

Keras → SimpleRNN → batch size, timesteps, input_features

(3, 4, 5) → 3D tensor → RNN

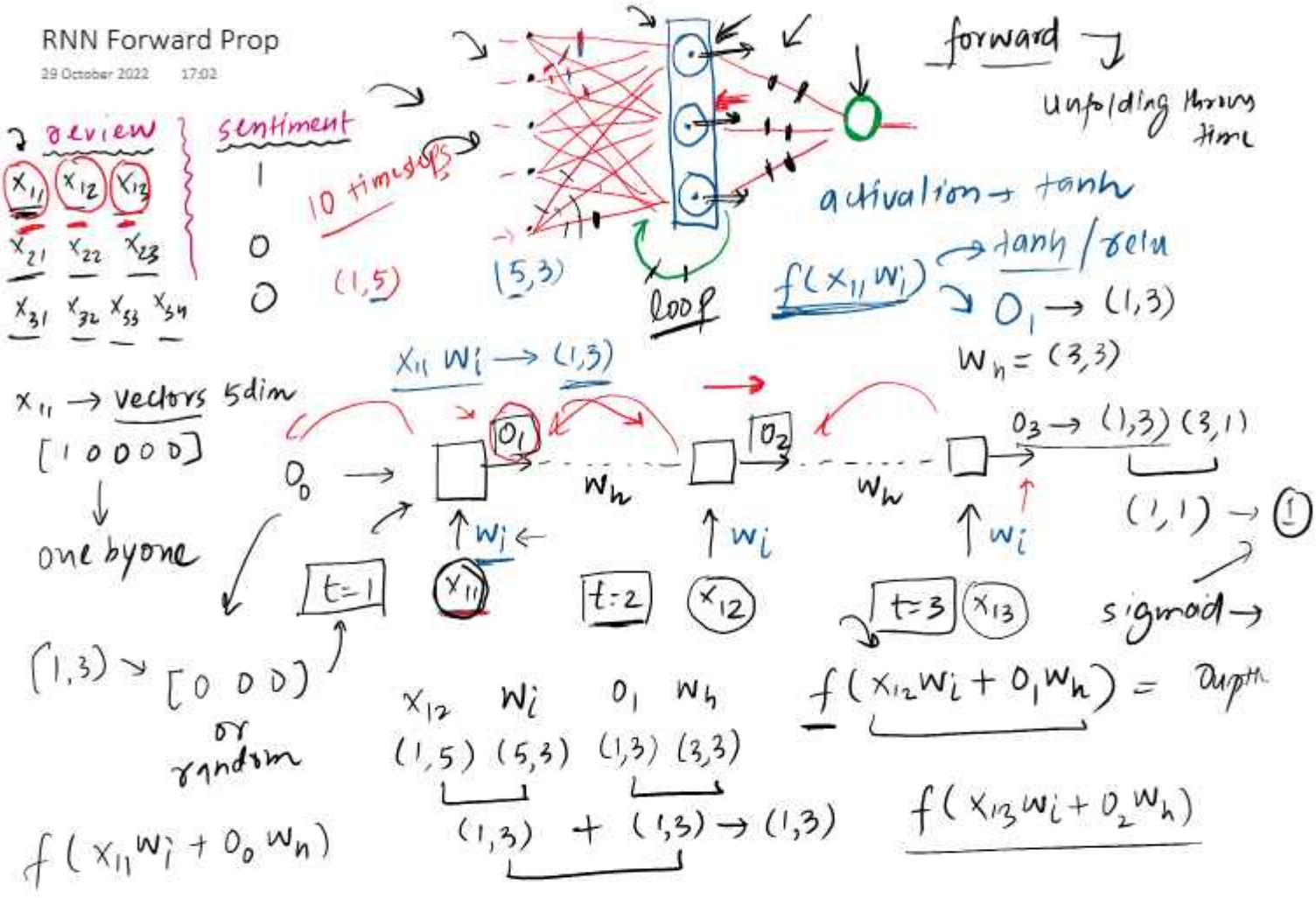
RNN Architecture

29 October 2022 13:30



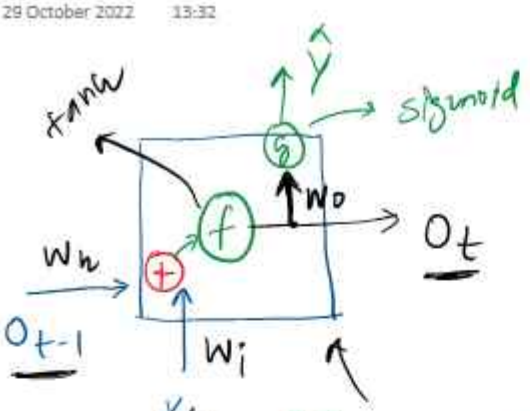
RNN Forward Prop

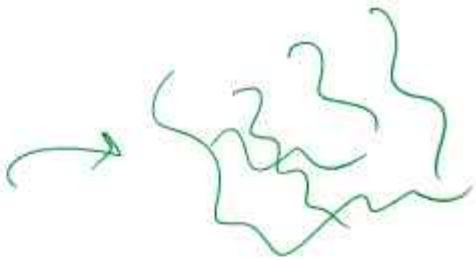
29 October 2022 17:02



Simplified Representation

29 October 2022 13:32





$\hat{y} = g(o_t w_o)$

$f(x_{it} w_i + o_{t-1} w_h) \rightarrow$ vector $\rightarrow o_t$

i = row #

t = time step

$x_{11} \rightarrow$

Hello how are you?

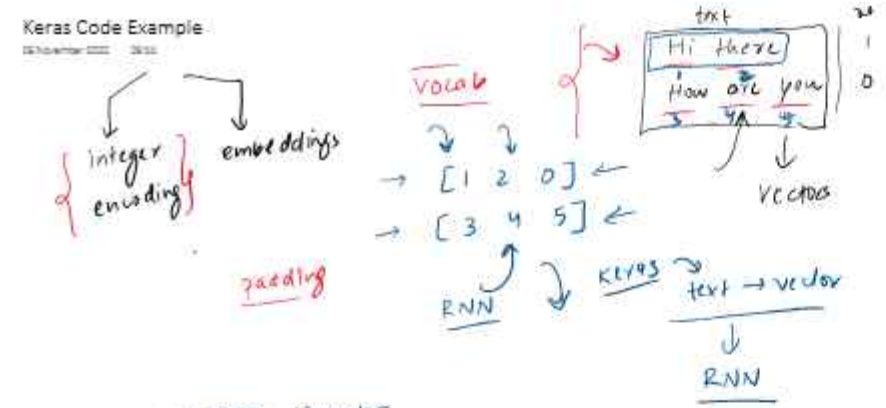
Code

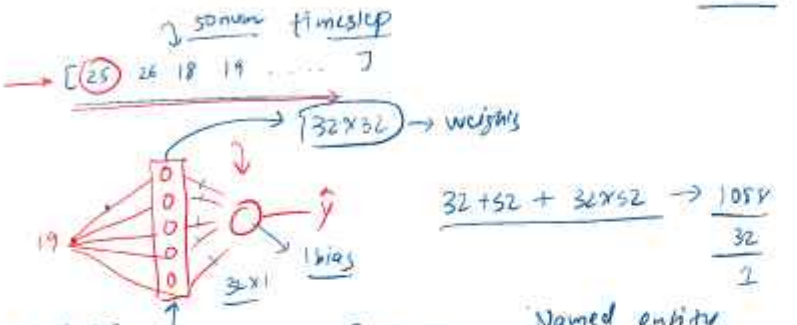
29 October 2022 13:32

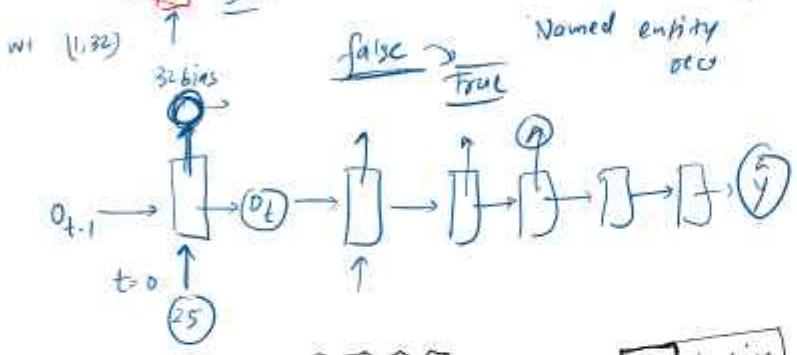
State and Memory

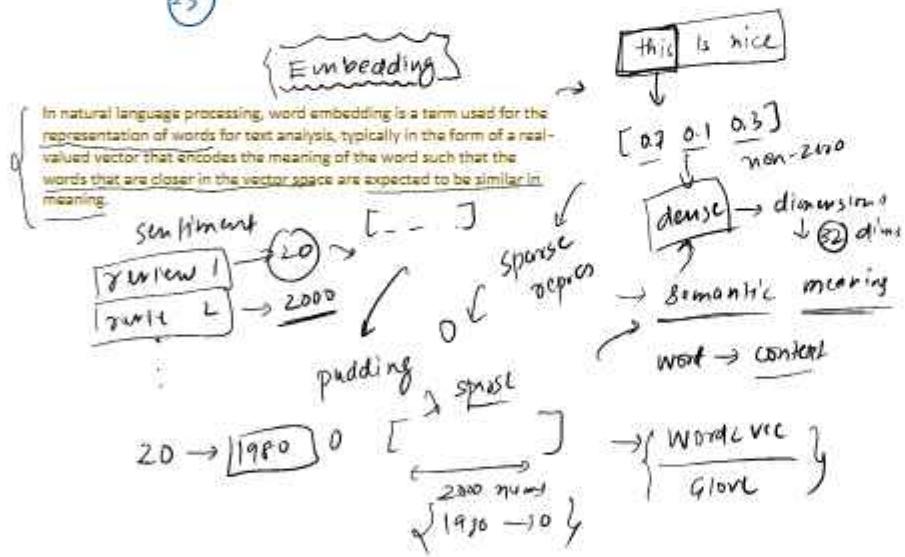
29 October 2022 13:33

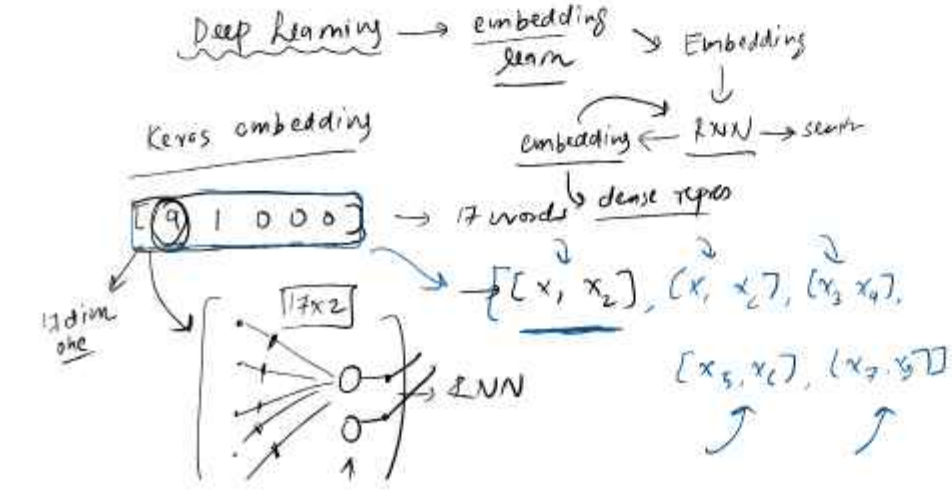
Keras Code Example







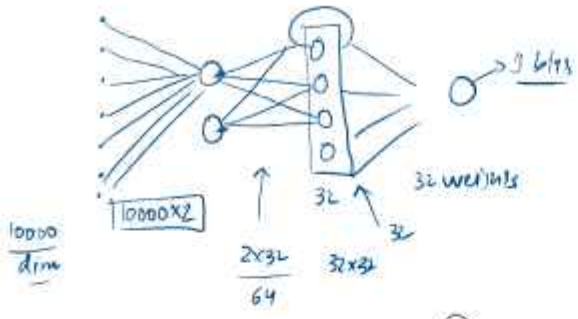








If nodes



→ population in 9th year = x

x + 10% of x = 10000 t(n-1)

x + 0.1x = 10000

1.1x = 10000

x = 10000 / 1.1

$\frac{x-1}{x} + \frac{1}{2}\left(\frac{x-1}{x}\right)^2 + \frac{1}{2}\left(\frac{x-1}{x}\right)^3 + \frac{1}{2}\left(\frac{x-1}{x}\right)^4 +$

+ 2 +

+ (n-1)

50 → 500

A

A[35] → t sec

A[35] →

35

1 x 4 x35

x

1 25 50 100

12

O(n)

O(n²) → nested loops

input → 10 100 1000

time (4)² 8 4

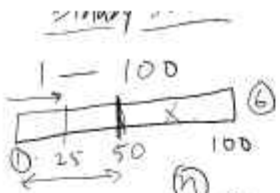
O(log(n)) ↓

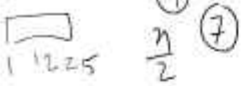
Binary search

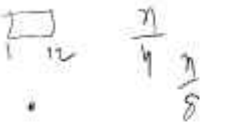
1 — 100

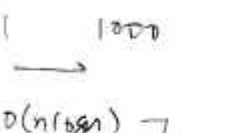
→ (A)

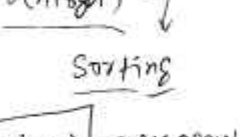
$\sqrt{(x_1$

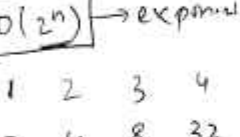

1 — 100
25
50
100

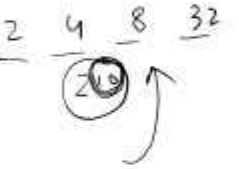


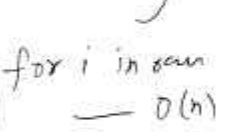


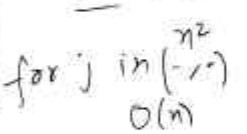


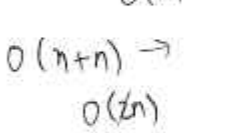


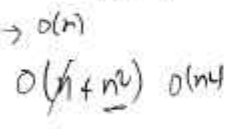


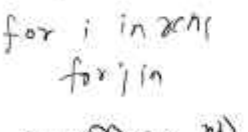


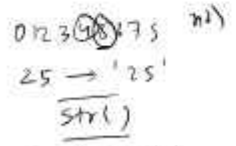


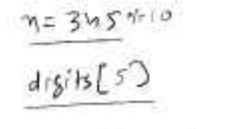


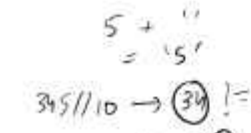


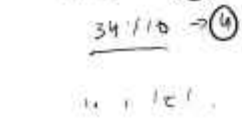












34 // 10 → (4)

4 + '5' →
45

34 // 10 → (3)

3 != 0

3 // 10 => 3

digits [3]

3 + '45'
= '345'

Loop n → 10 times

345 = (n) ⇒ 345 (2)

(3) → (4)

inp → 10
time → 1 → log

O(n) → O(n+n)
O(n) O(2n) → O(n)

O(n) → O(n*n)
O(n) O(n²)

O(n)
O(n)
O(1000000)

n² 1000000
×
O(n²)

log(L)/2
1 → n/2

4 n/2 24

O(n) n=100

50, 100 n/2
↳
2 → 100=n n/2

j=1 → (2)
j=2 → 4 2-100
j=3 → 8
j=4 → 32

n/2 × log n
n log n

O(1) → constant

n = 345

3+4+5 → 12

5

3 4

3 3

0

3450 → 4

log

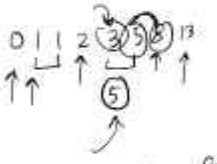
inp → 10 100

out → 1 2 3

fibonacci

↳ function

↳ recursion



n=1 n=0 → 1

fib(n)

↓

function calls

input → 7 → 10

#fcalls →

fib(3) n=3

fib(2) fib(1) → 1

fib(1) fib(0)

1 1

exponential → bad



O(2^n) 15 20 50

input 1 2 3 4

1 2 4 8 16

$O(2^n)$ (50)

input 1 2 3 4

ti 2 4 8 16

$n = 50, 100, 500$

↑ weeks

exponential

↳ days/ weeks

Subset → O(?)

power set

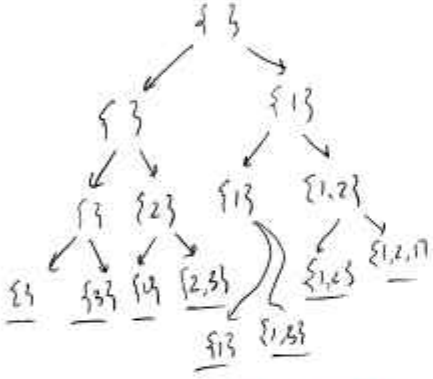
$\{1,2\} \rightarrow \{\{\},\{1\},\{2\},\{1,2\}\}$

$\{1,2,3\} \rightarrow$

$\{\},\{1\},\{2\},\{3\},\{1,2\}\{2,3\}$

$\{3,1\}\{1,2,3\}$

$\{1,2,3\}$



reduce → divide → log

Increase → double → exp

→ exponential

$2^2 = 4$

$\{1,2\} \rightarrow (4)$ $2^3 = 8$

$\{1,2,3\} \rightarrow 8$ $2^4 = 16$

$O(2^n)$

O(?)

$$T(n) = \begin{cases} 3T(n-1) & \text{if } n>0 \\ 1, & \text{otherwise} \end{cases}$$

$n>0$

$T(n) = 3T(n-1)$

$= 3[3T(n-2)]$

$= 3^2 T(n-2)$

$= 3^2[3T(n-3)]$

$= 3^3 T(n-3)$

$= 3^n T(n-n)$

$= 3^n T(0)$

$T(n) = \boxed{3^n} \rightarrow O(3^n)$

$$T(n) = \begin{cases} 2T(n-1)-1 & \text{if } n \geq 0 \\ 1, & \text{otherwise} \rightarrow \text{constant} \end{cases}$$

$=$

$T(n) = 2T(n-1)-1$

$= 2[2T(n-2)-1]-1$

$= 2^2 T(n-2) - 2 - 1$

$= 2^2[2T(n-3)-1] - 2 - 1$

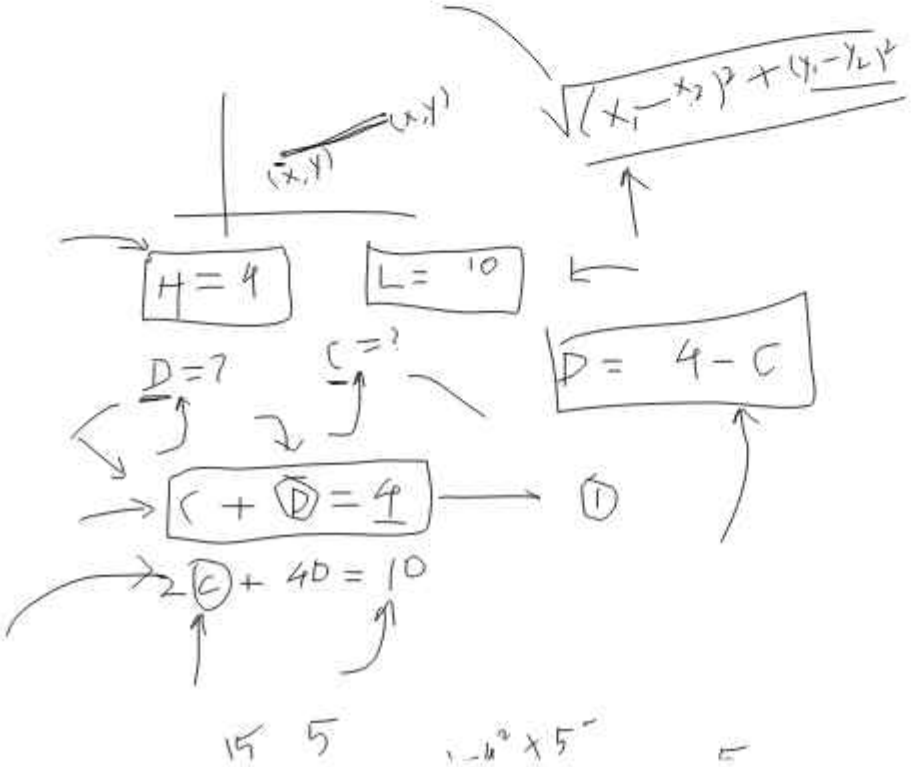
$= 2^3 T(n-3) - 2^2 - 2^1 - 2^0$

$= 2^n T(n-n) - 2^{n-1} - 2^{n-2} \ldots - 2^1 - 2^0$

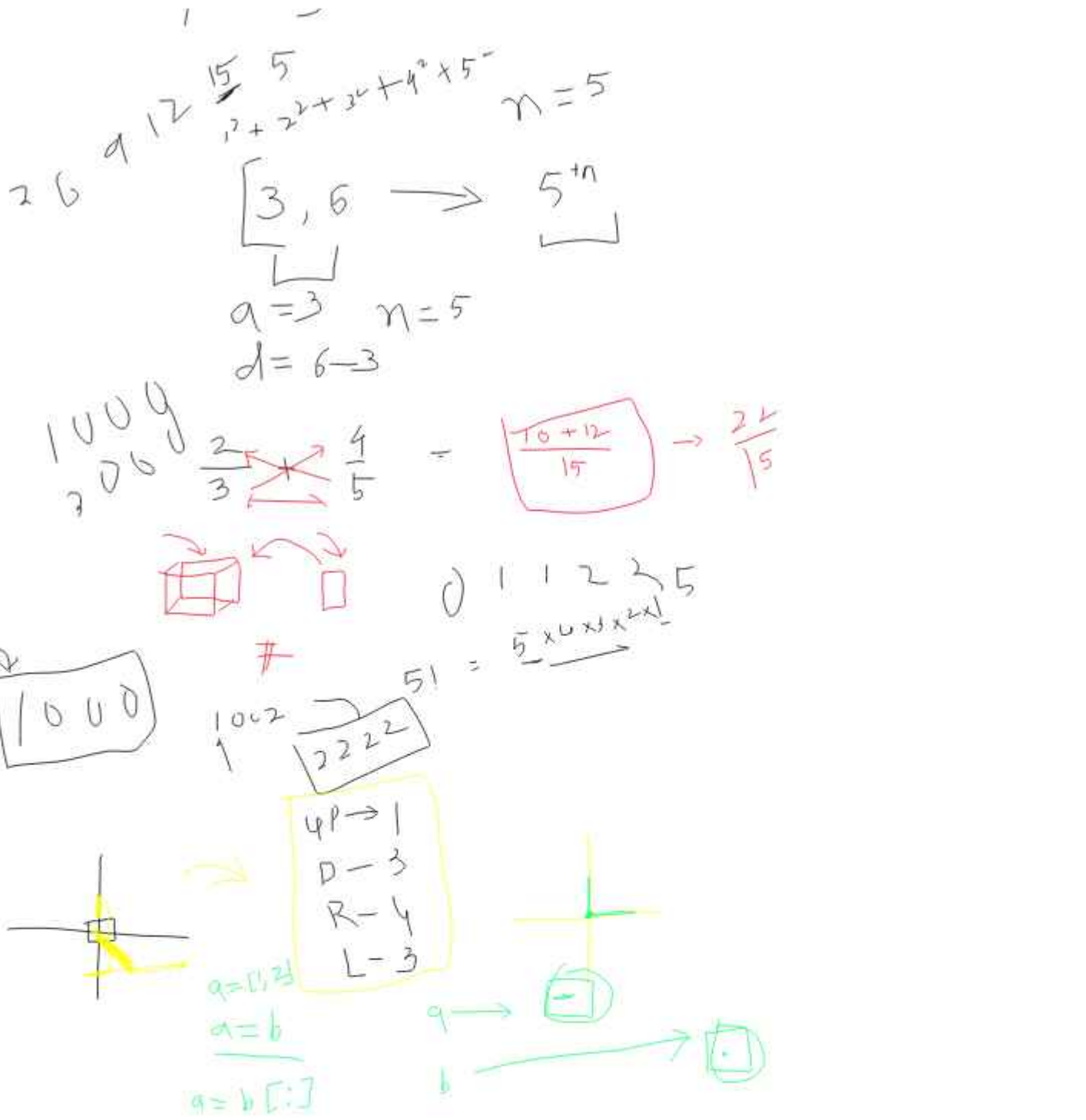
$= 2^n - [2^{n-1} + 2^{n-2} + \ldots + 2^1 + 2^0]$

$= 2^n - [2^n - 1] = 2^n - 2^n + 1$

$O(1) \rightarrow$ constant

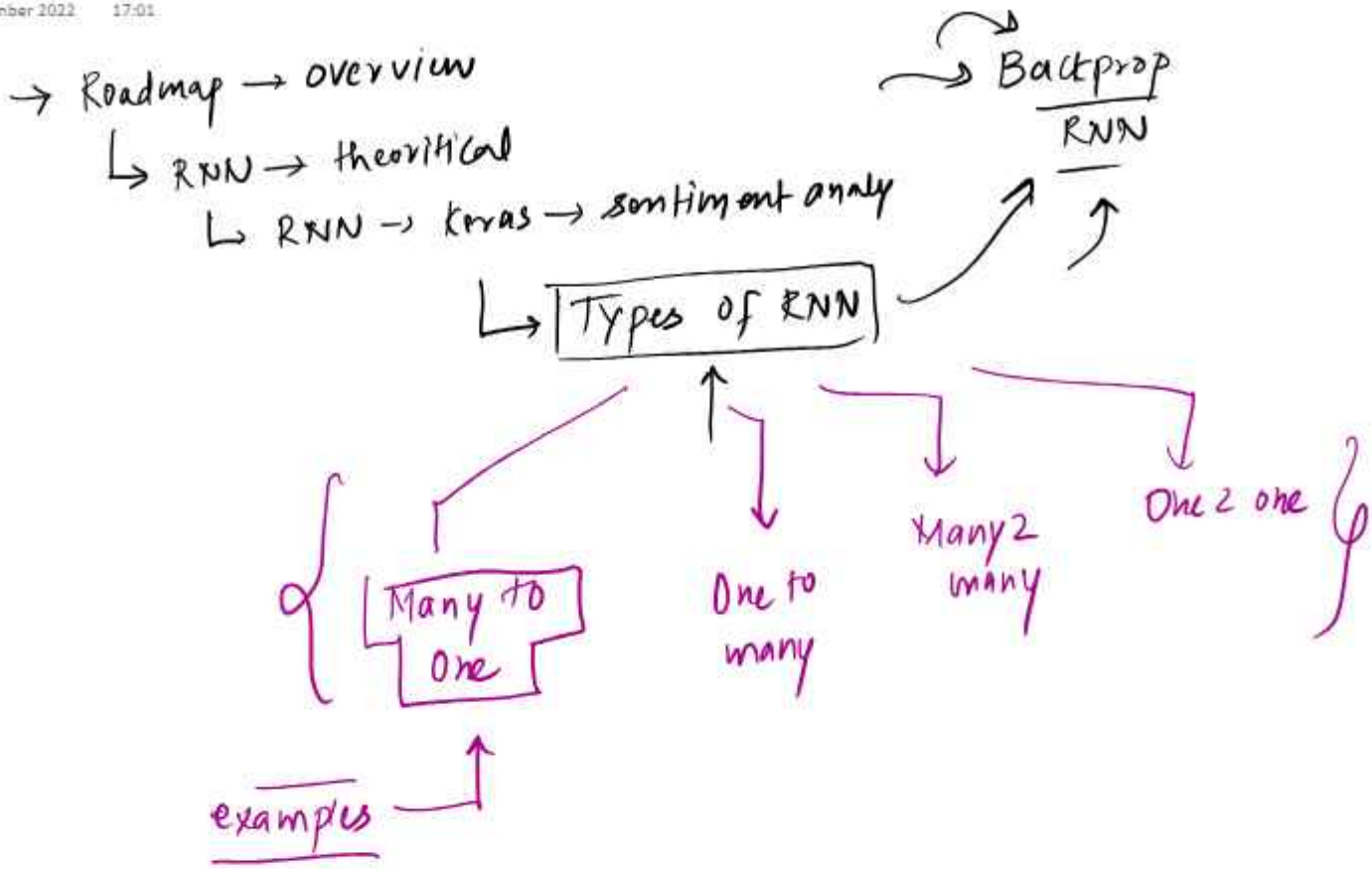


15 5


n = 5
5^n
q = 3
n = 5
d = 6 − 3
10 + 12
15
21
15
1000
1002
2222
51
4P → 1
D − 3
R − 4
L − 3
q = [1, 2]
a = b
a = b [:]
q
b

Till Now

17 November 2022 17:01



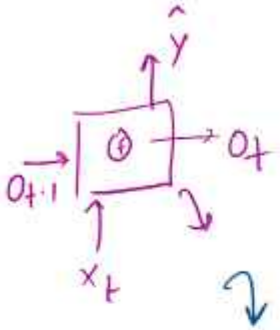
Many to One

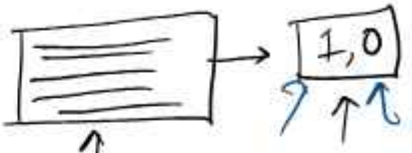
17 November 2022 17:02

input → sequence → sent, chord, time series

out → non-sequence → int/num → scalars → (1,0)

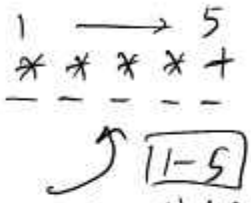
↳ sentiment → many





sequence

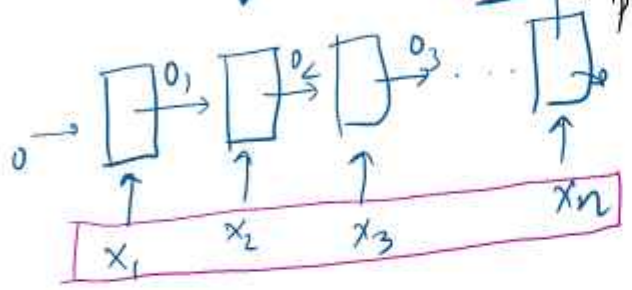




multiclass classif.



Rating prediction



recurrent → simple RNN

many to one

One to Many

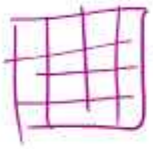
17 November 2022 17:02

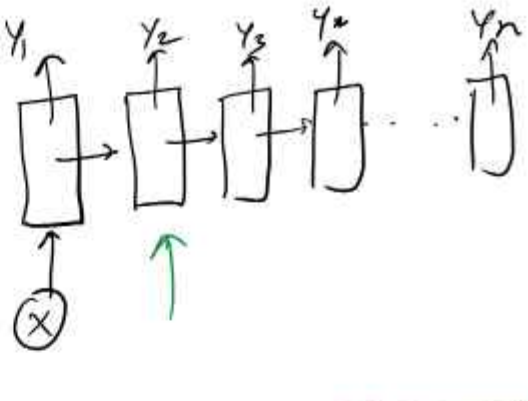
→ normal non sequential

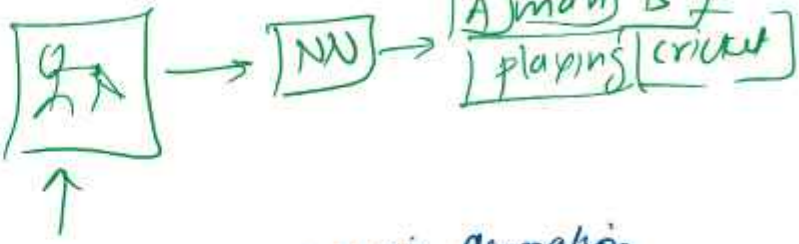
↳ 

→ Output → sequentials

image captioning

 → textual depres





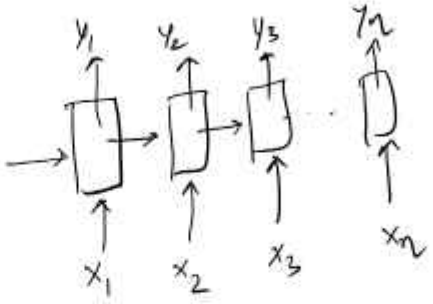
→ music generation

Many to Many

17 November 2022 17:02

input → sequent
out → sequen
→ seq2 seq

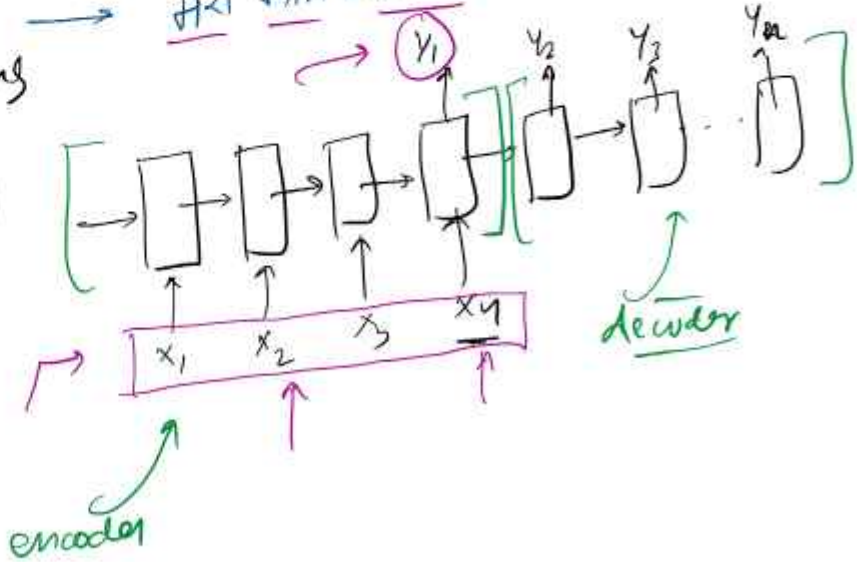
- Same length
  - input seq == output seq
  - pos tagging → NLP
    - 1 My → pronoun
    - 2 name → noun
    - 3 is → verb
    - 4 Nitish → Noun
  - NER
    - Lets meet at 7pm at the airport
- Variable length
  - machine trans
    - 1 lang → 2 lang
    - google translate



same length many2many RNN

Variable length

Hi my name is Nitish → मेरा नाम नीतिश है

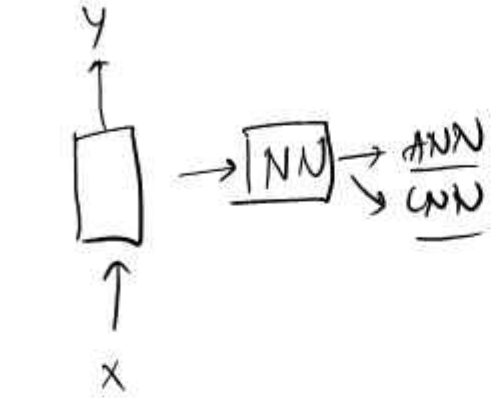


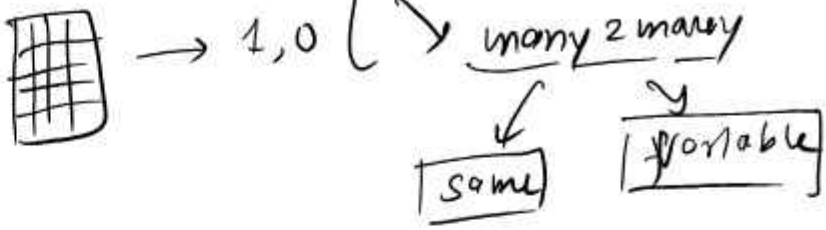
One to One

17 November 2022 17:02

one → non sequential → image classif'

out → non sequ





Backpropagation in RNN

01 December 2022 18:45

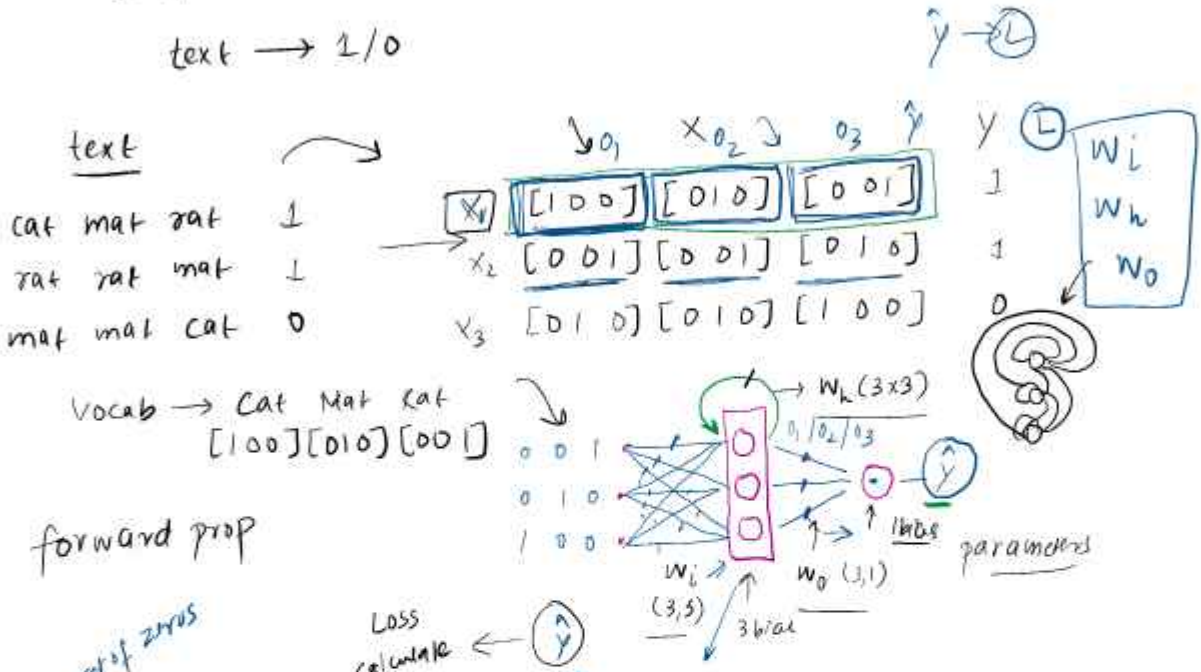
RNN Intro → RNN → practical → types of RNN

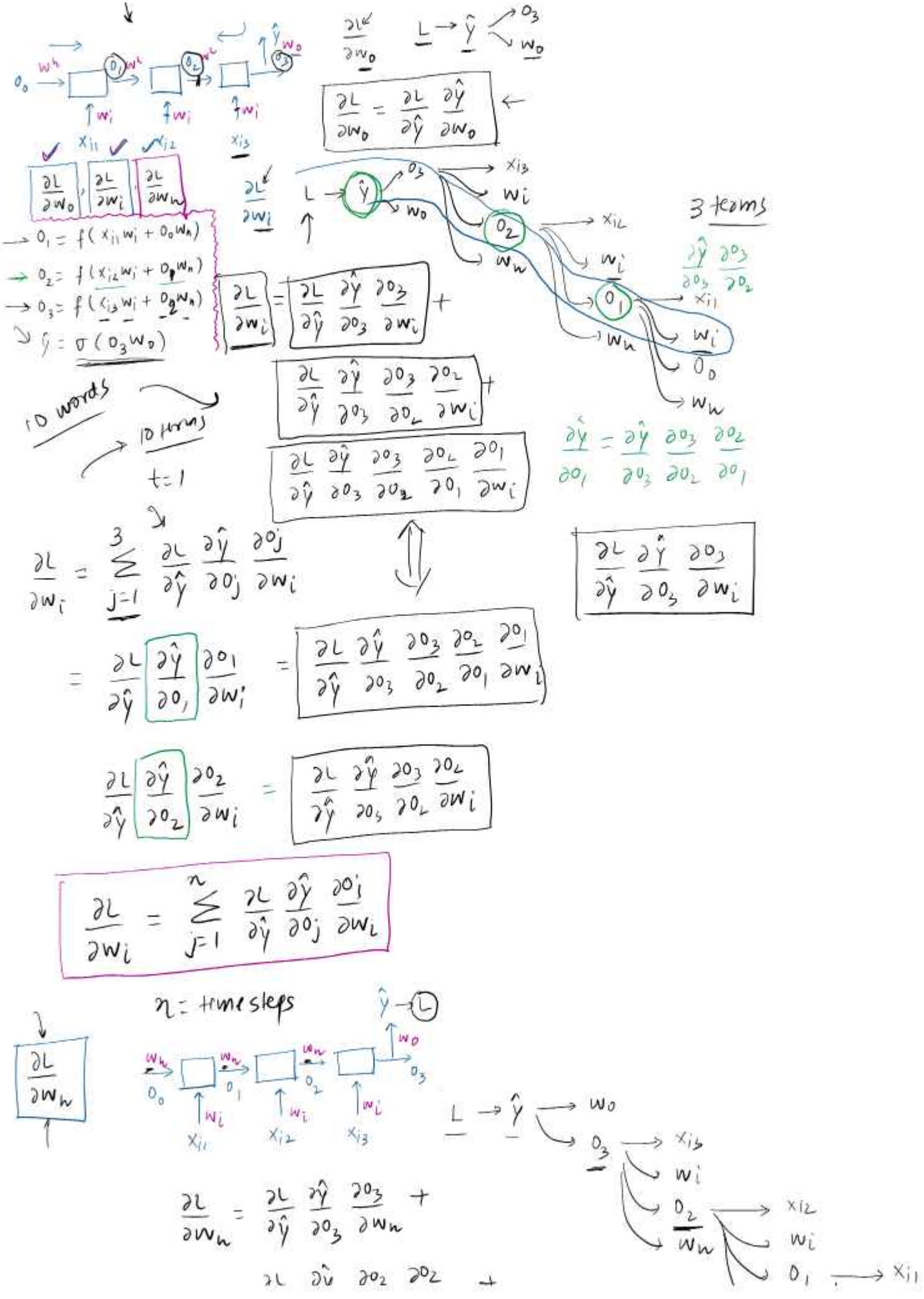
→ RNN → Backpoop → BPTT

Many to One RNN

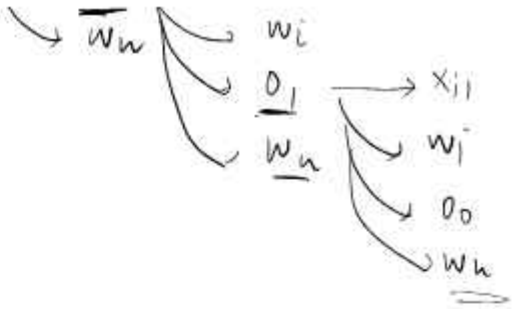
Sentiment Analysis

text → 1/0




3 terms
10 words
10 terms
t=1
n = timesteps

$\frac{\partial L}{\partial W_h}$ $\frac{\partial y}{\partial o_3}$ $\frac{\partial o_3}{\partial W_h}$ ...



$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial o_2} \frac{\partial o_2}{\partial W_h} +$$

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial o_2} \frac{\partial o_2}{\partial o_1} \frac{\partial o_1}{\partial W_h}$$

$$\frac{\partial L}{\partial W_h} = \sum_{j=1}^{n} \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_j} \frac{\partial o_j}{\partial W_h}$$

n = timesteps

for j = 3

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial W_h} \rightarrow \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial o_1} \frac{\partial o_1}{\partial W_h}$$

for j = 10

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_{10}} \frac{\partial o_{10}}{\partial o_1} \frac{\partial o_1}{\partial W_h}$$

$$\prod_{t=2}^{j} \frac{\partial o_t}{\partial o_{t-1}}$$

$$\prod_{t=2}^{t=3} \frac{\partial o_t}{\partial o_{t-1}} = \frac{\partial o_2}{\partial o_1} \frac{\partial o_3}{\partial o_2}$$

$$o_t = f(x_{it} W_{inp} + o_{t-1} W_h)$$

$$\frac{\partial o_t}{\partial o_{t-1}} = \prod_{t=2}^{j} f'(x_{it} W_{inp} + o_{t-1} W_h) W_h$$

$\downarrow$

$[0-1]$

Problem with RNN

19 December 2022 16:33

RNNs → sequential data → textual, time series

suffer (2) major problems → LTSMs (Why)

RNN

→ Problem of long term dependency
→ Unstable gradients stagnated badly } → Unstable gradients

Applications next word predictor

→ RNN

timesteps

Marathi is spoken in Maha

short

RNN

{ Vanishing gradient prob } → long term → RNN → keypad

Maha is a beautiful place I went there last year, but I could not enjoyed properly bcz I don't understand marathi

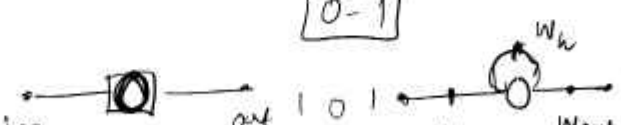
{ Exploding gradient problem } → RNN

possible solⁿ → remove

Problem # 1 → Problem of long term dependency → Vanishing

inp	out
1 0 1	1
0 0 1	0
0 0 0	0
1 1 1	1

0-1

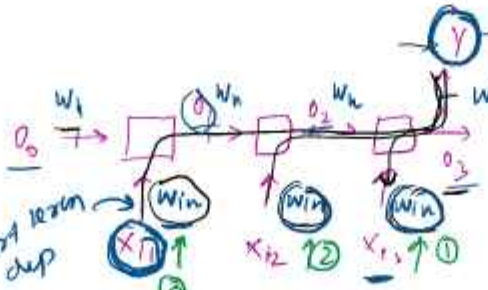


→ min

W_h, W_{in}, W_{out}

↓

gradient des

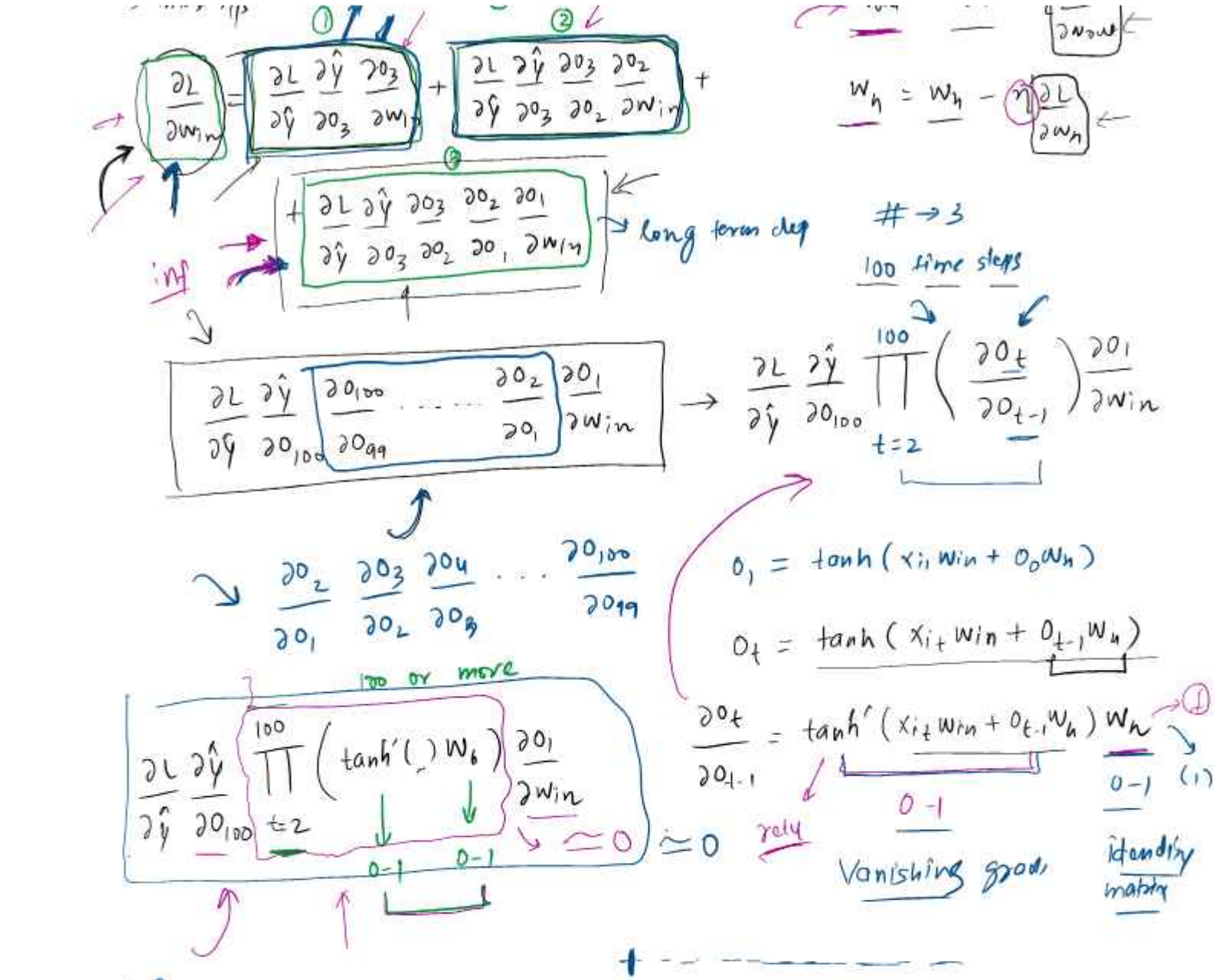


3 time stamps



$$W_{in} = W_{in} - \eta \frac{\partial L}{\partial W_{in}}$$

$$W_{out} = W_{out} - \eta \frac{\partial L}{\partial W_{out}}$$



Soln

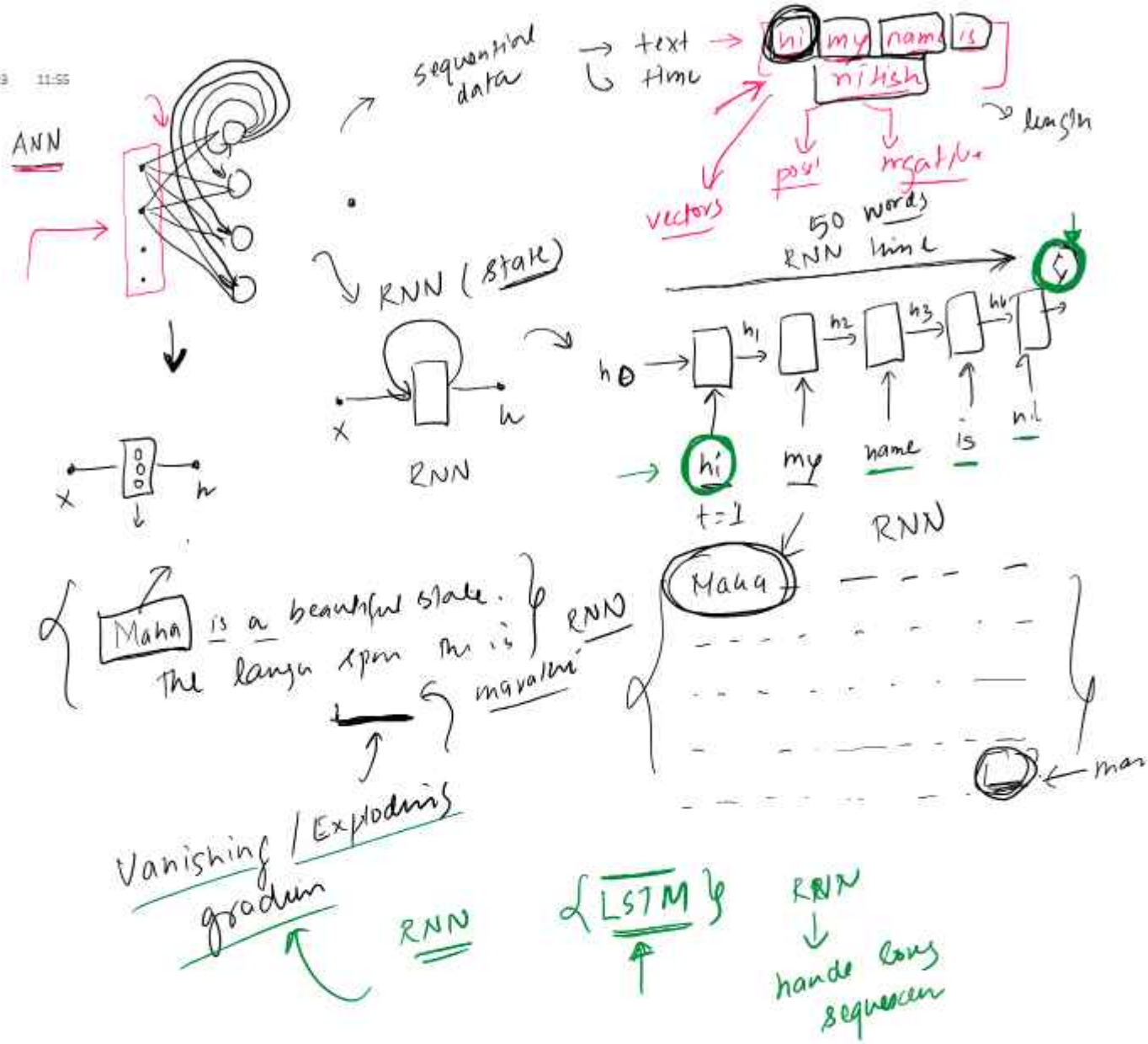
1) Diff activation → relu / leaky relu
2) Better weight init
3) Skip conns
4) LSTM

Problem #2 → Unstable Training (Exploding gradients)

1) Gradient Clipping
2) Controlled learning rate
3) LSTM

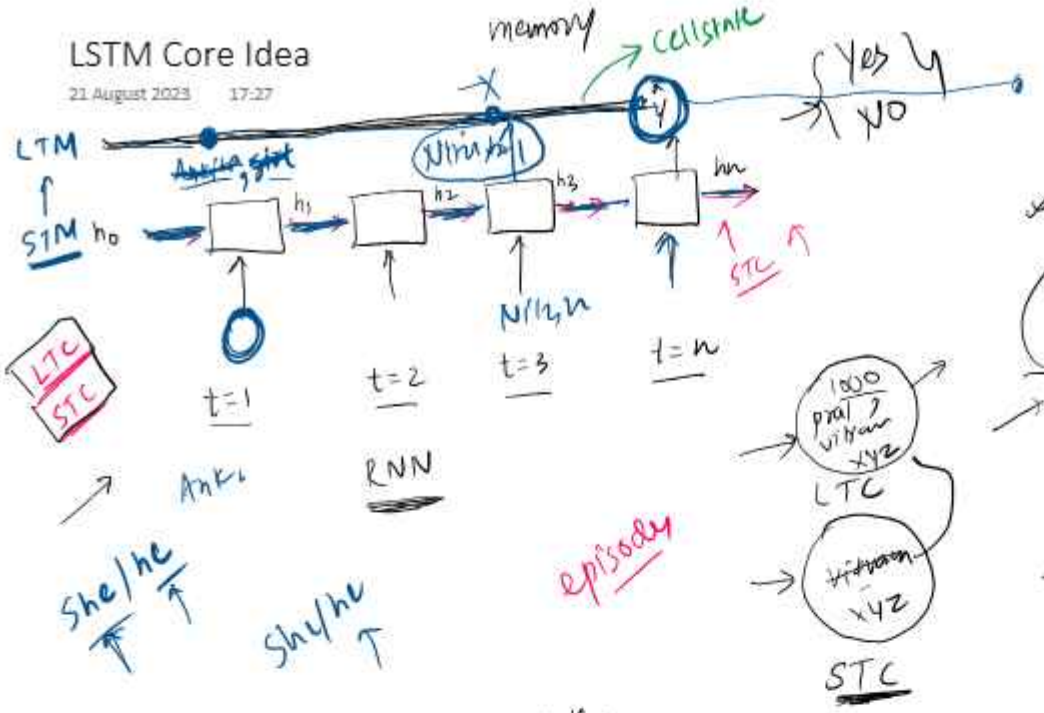
Recap

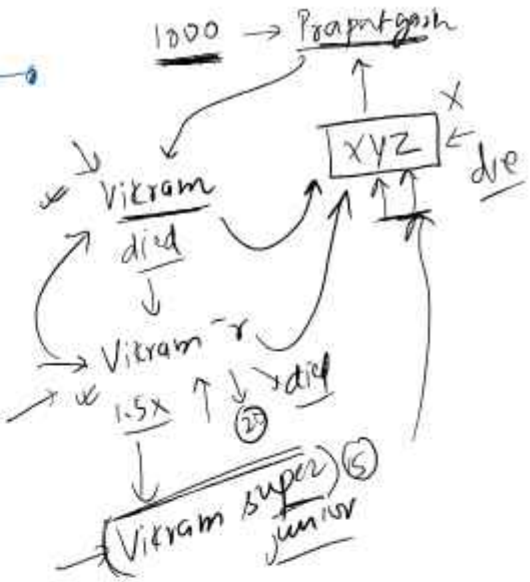
21 August 2023 11:55

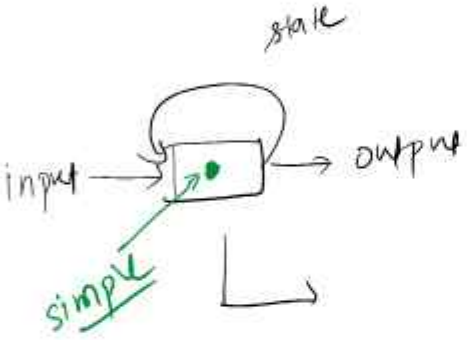


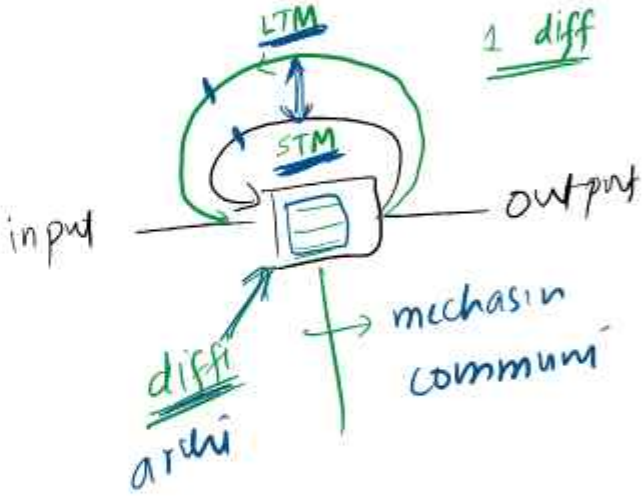
LSTM Core Idea

21 August 2023 17:27



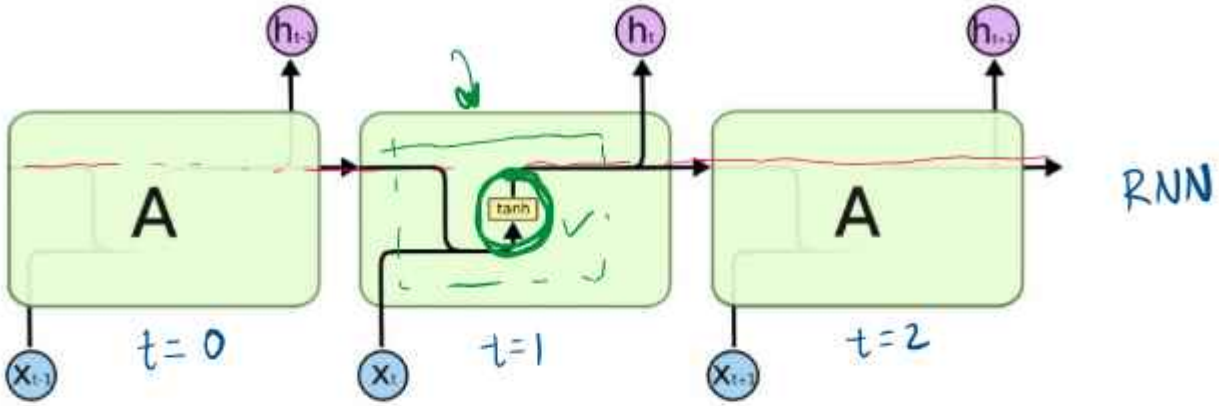


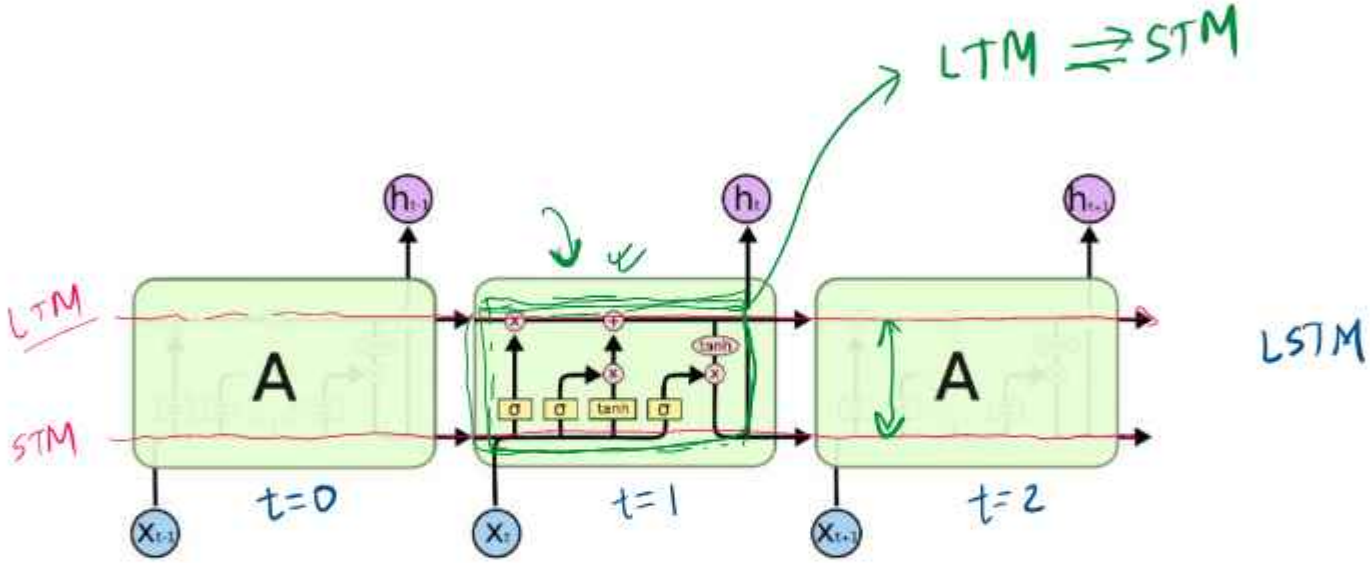


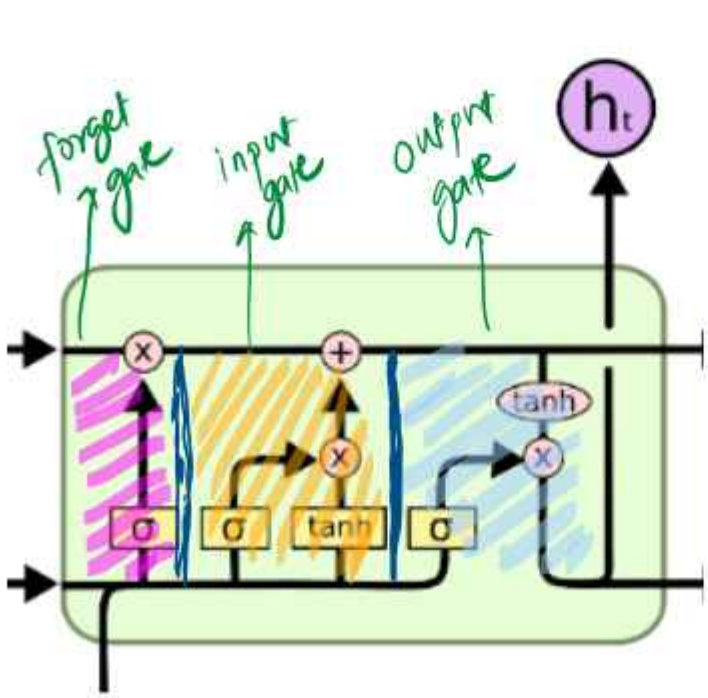


LSTM Architecture

21 August 2023 18:41



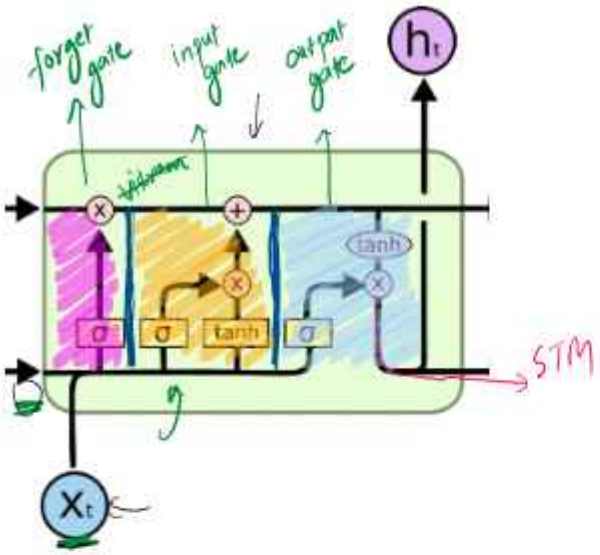





X_t

LSTM Gates

21 August 2023 19:06

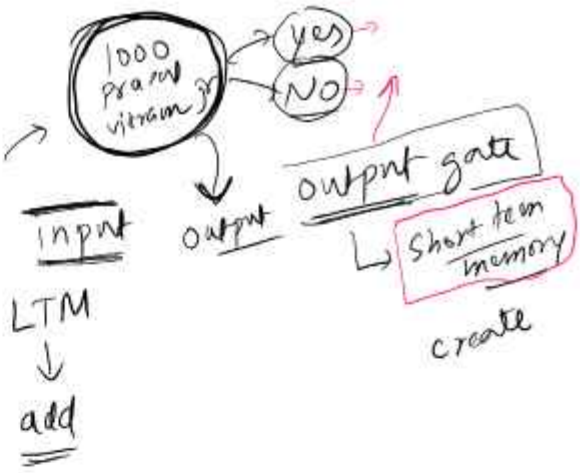


forget

input

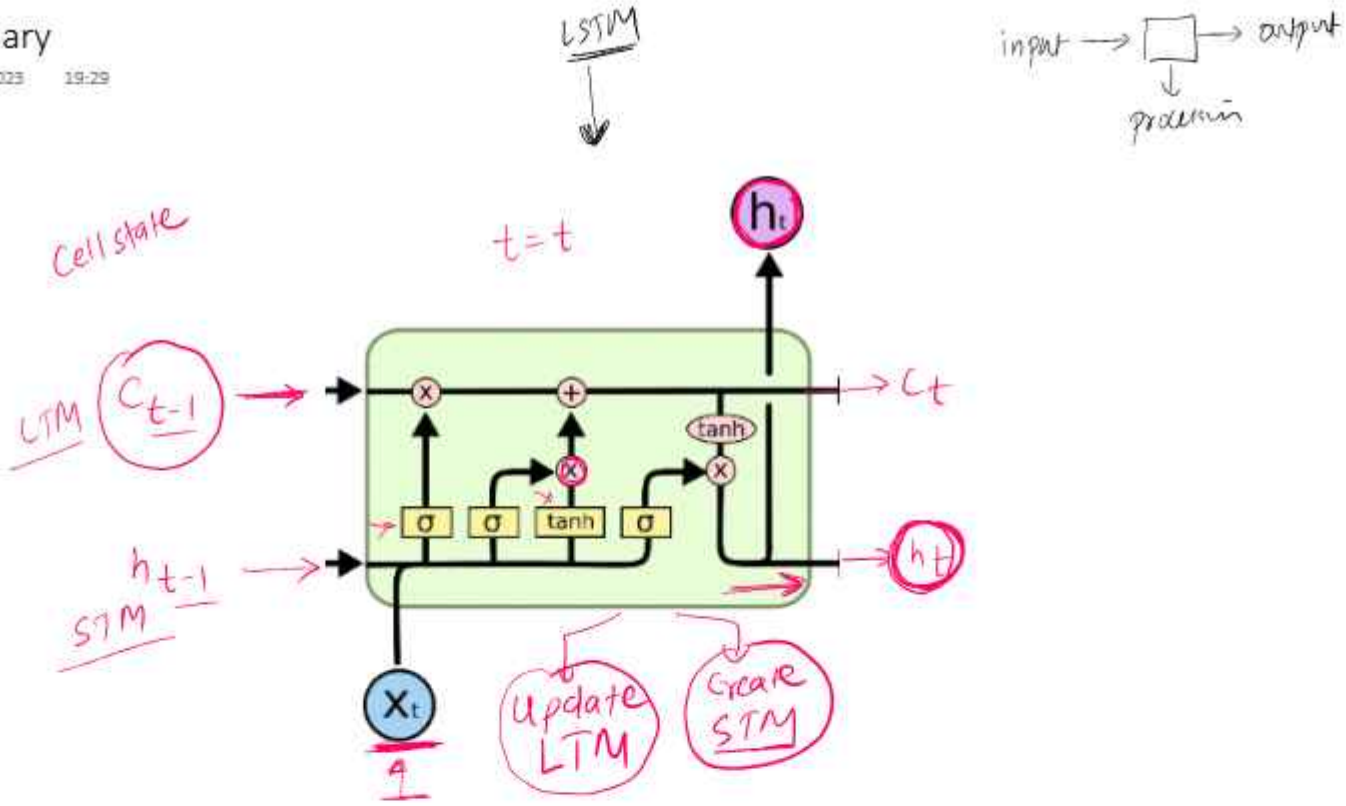
LTM

remove



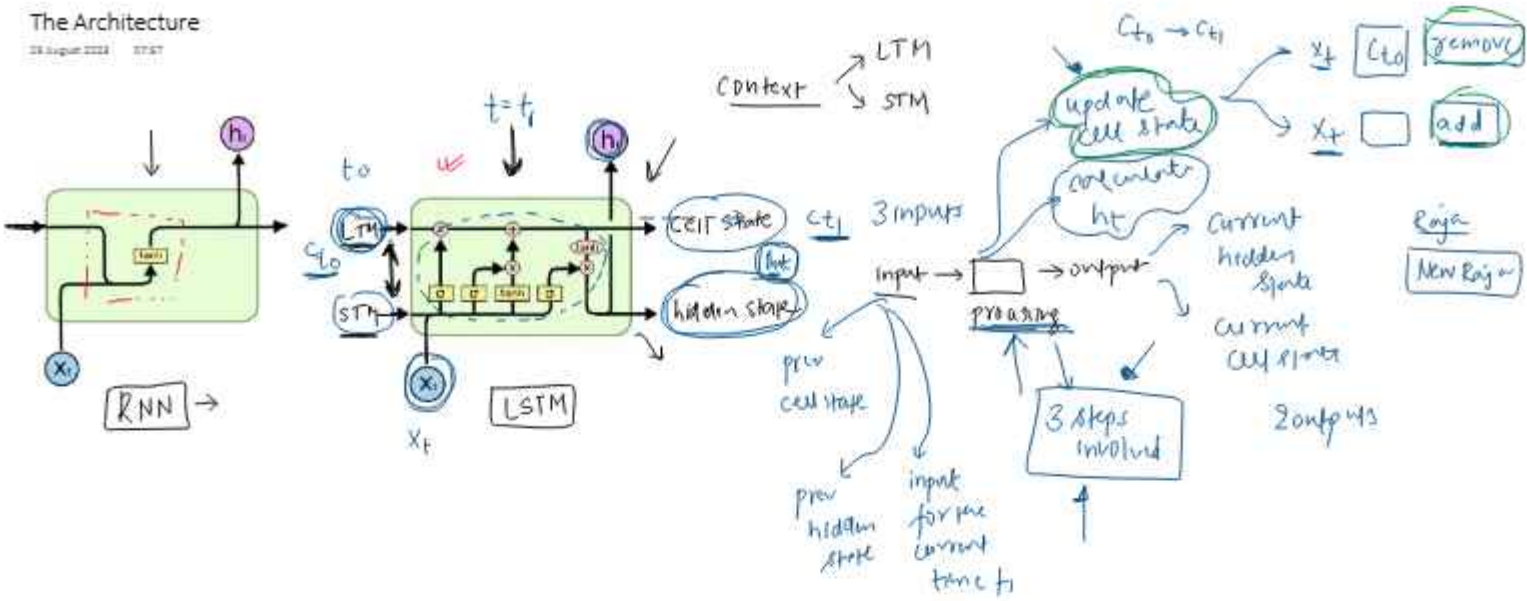
Summary

21 August 2023 19:29



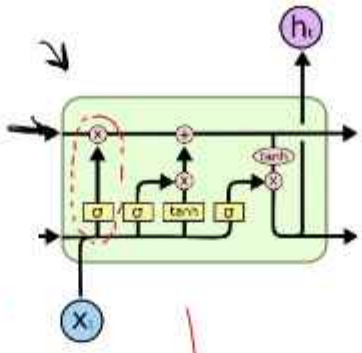
The Architecture

28 August 2024 07:07

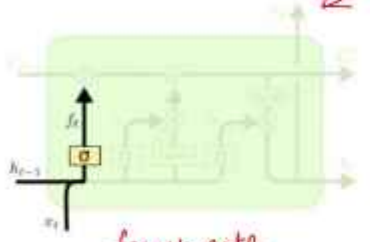


The Gates

29 August 2023 05:09



3 gates

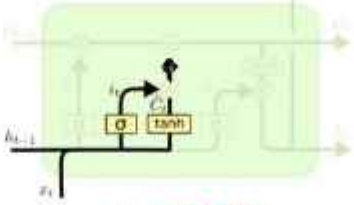


forget gate

to remove something from C_t

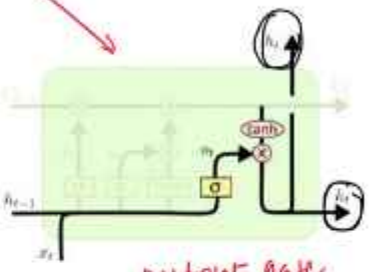
update C_t

+



input gate

to add something to the C_t

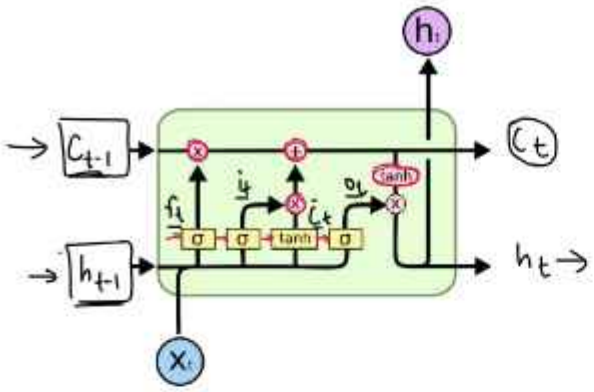


output gate

Calculat hidden state h_t

What are C_t and h_t

29 August 2023 08:08



h_t C_t → [0.1 0.3 0.9]

vectors

3d vector

full

same

h_t C_t dim equal

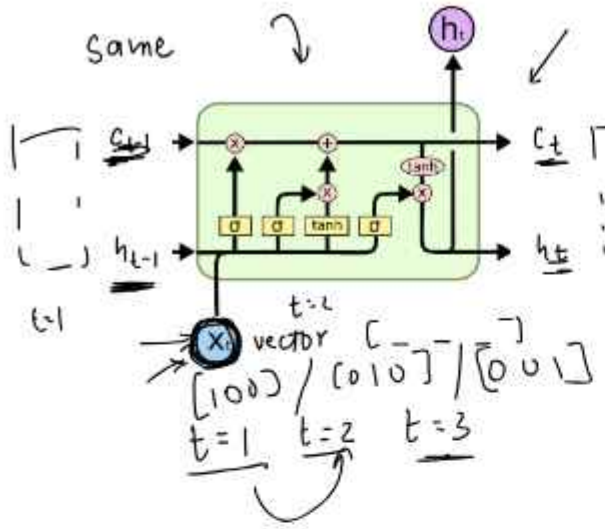
h_t [0.1 0.45 0.6]

C_t [0.55 0.6 0.0]

What is X_t

29 August 2023 17:40

RNN



sentiment

vectorization

cat mat rat	0 ←
cat rat rat	0
mat mat cat	1

[100] [010] [001]

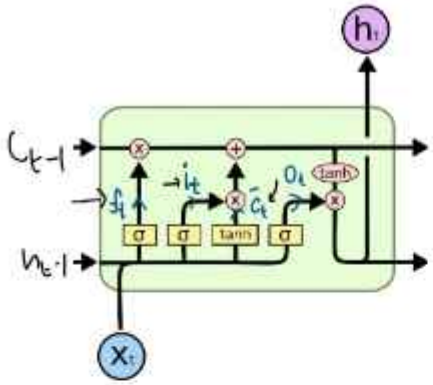
word vector

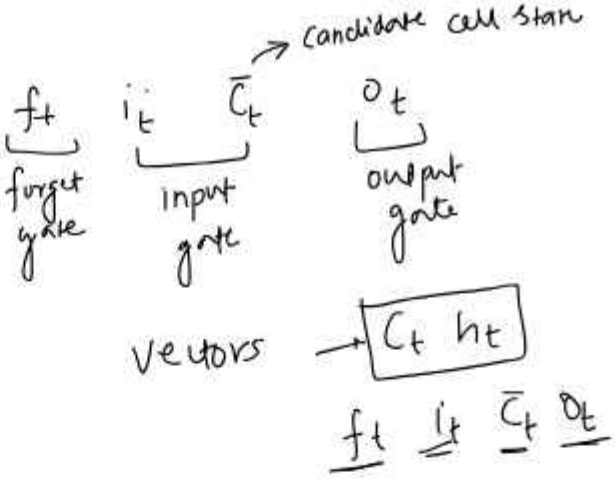
OHE

cat	mat	rat
1	0	0
0	1	0
0	0	1

What are f_t, i_t, o_t and $\bar{C}t$

29 August 2023 08:09

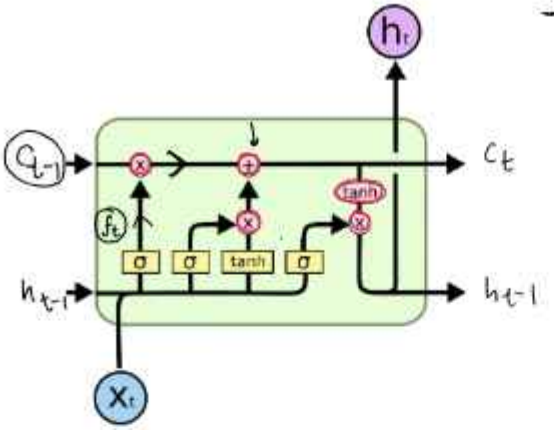


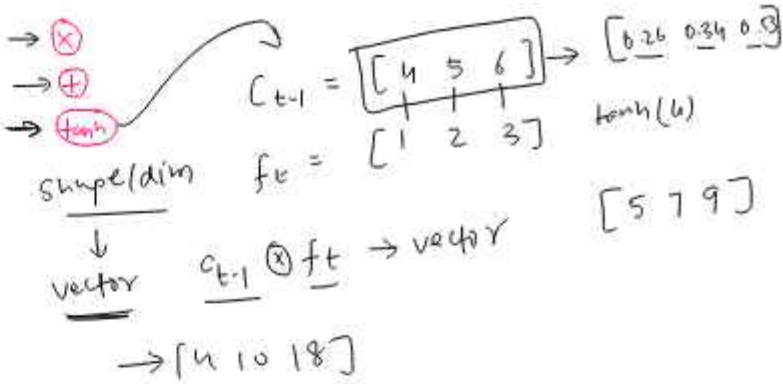


[x y z]
[ ]

Pointwise Operations

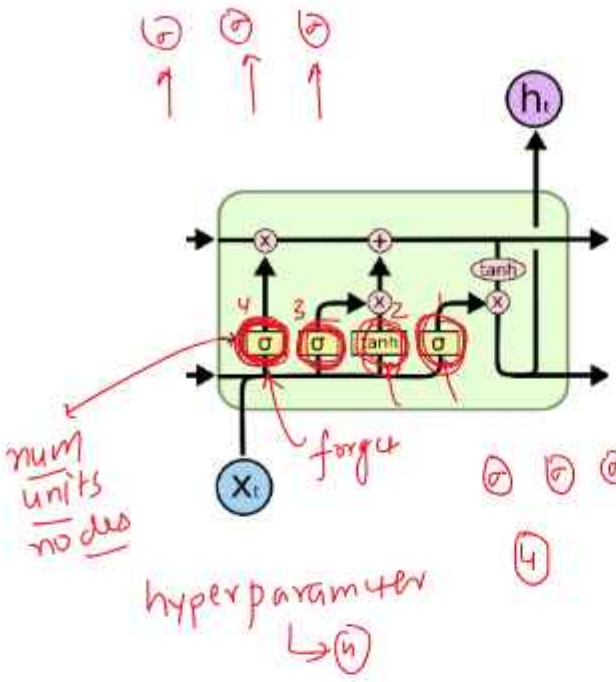
29 August 2023 18:08



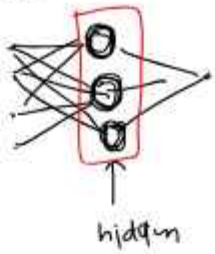


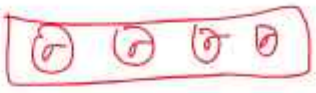
→Neural Network Layers

29 August 2023 18:34



ANN









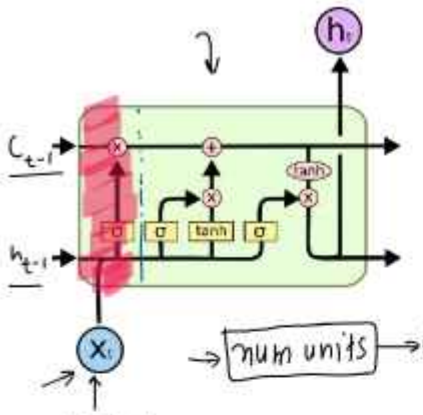


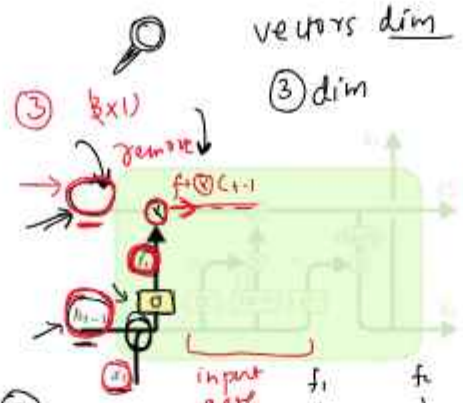
4

hyperparameter

↳ 4

The Forget Gate





vectors dim

③ dim

③ (3x1)

→ num units → 3

4 dim value

$[x_{i1}\ x_{i2}\ x_{i3}\ x_{in}]$

input

output → cell state → remove

(3x1) ⊗ (3x1)

→ 1> Calculate f_t

2> $C_{t-1} \otimes f_t$

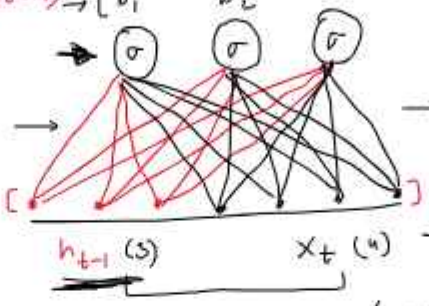
σ σ σ

f_1 f_2 f_3 → $[f_1\ f_2\ f_3]$ → f_t (3)

b_1 b_2 b_3 (3x1)

(3x7) (7x1)

→ (3x1) + (3x1) = (3x1)



→ W_{3x7} (21 weights) 21 wts

biases 4 biases

h_{t-1} (3) X_t (4) → 7x1

forget gate ← 0%, 50%, 100% remove

$[h_{t-1}, X_t]$

long term

f_t

$[4\ 5\ 6]$

C_{t-1} → $[0\ 0\ 0]$ ⊗

f_t → $[\frac{1}{2}\ \frac{1}{2}\ \frac{1}{2}]$

$[1\ 1\ 1]$

$[2\ 2.5\ 3]$

$[4\ 5\ 6]$

$[0\ 0\ 0]$ → 100% remove

$f_t = \sigma\left(W_f[h_{t-1}, X_t] + b_f\right)$

(3x1) (3x7) (7x1) (3x1) + (3x1) → (3x1) → 8x1

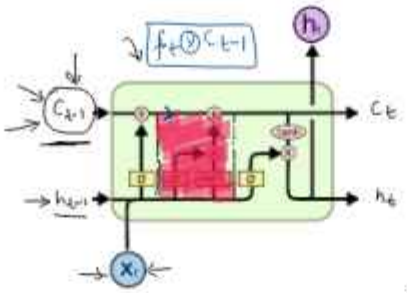
removal

$f_t \otimes C_{t-1}$ → (3x1)

(C_{t-1}) → remove

The Input Gate

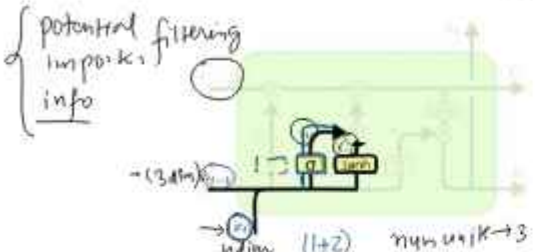
add some new imp info to the C_t

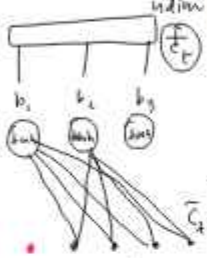


Stages

1) $\bar{C}_t$ candidate cell state
2) i_t
3) C_t → current cell state

potential filtering important info





$\bar{C}_t$ = (3x1)

W_c (3x7)

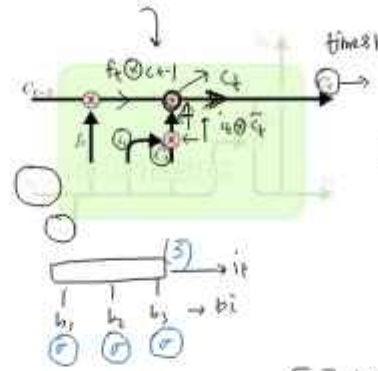
$\bar{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$

(3x7) (7x1) (3x1)

(3x1) (7x1)

(3x1)

h_{t-1} x_t



timestep

pointwise $i_t \otimes \bar{C}_t \rightarrow \bar{C}_t$ (filtered candidate cell state)

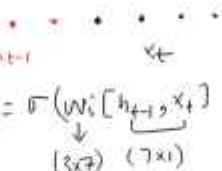
(3x1) (3x1)

[0.05 0.05 0.5] [4 5 6]

1 1 1 ↓ 50% or not

[2, 2.5 3]

$C_t = f_t \otimes C_{t-1} \oplus i_t \otimes \bar{C}_t$



→ [ ] b_i

h_{t-1} x_t

$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$

(3x7) (7x1) (3x1)

(3x1)

σ(3x1) → (3x1)

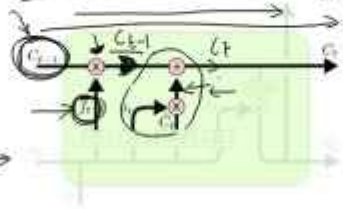
i_t - filter

$\bar{C}_t \rightarrow C_t$

10%

[4 5 6] → [4 5 6]

t=t-1 t=t



$f_t = [1\ 1\ 1]$

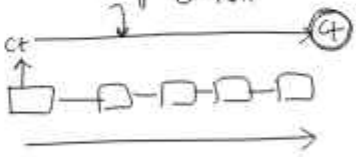
$i_t \otimes \bar{C}_t = [0\ 0\ 0]$

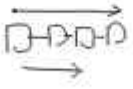
[4 5 6] [0 0 0]

↓

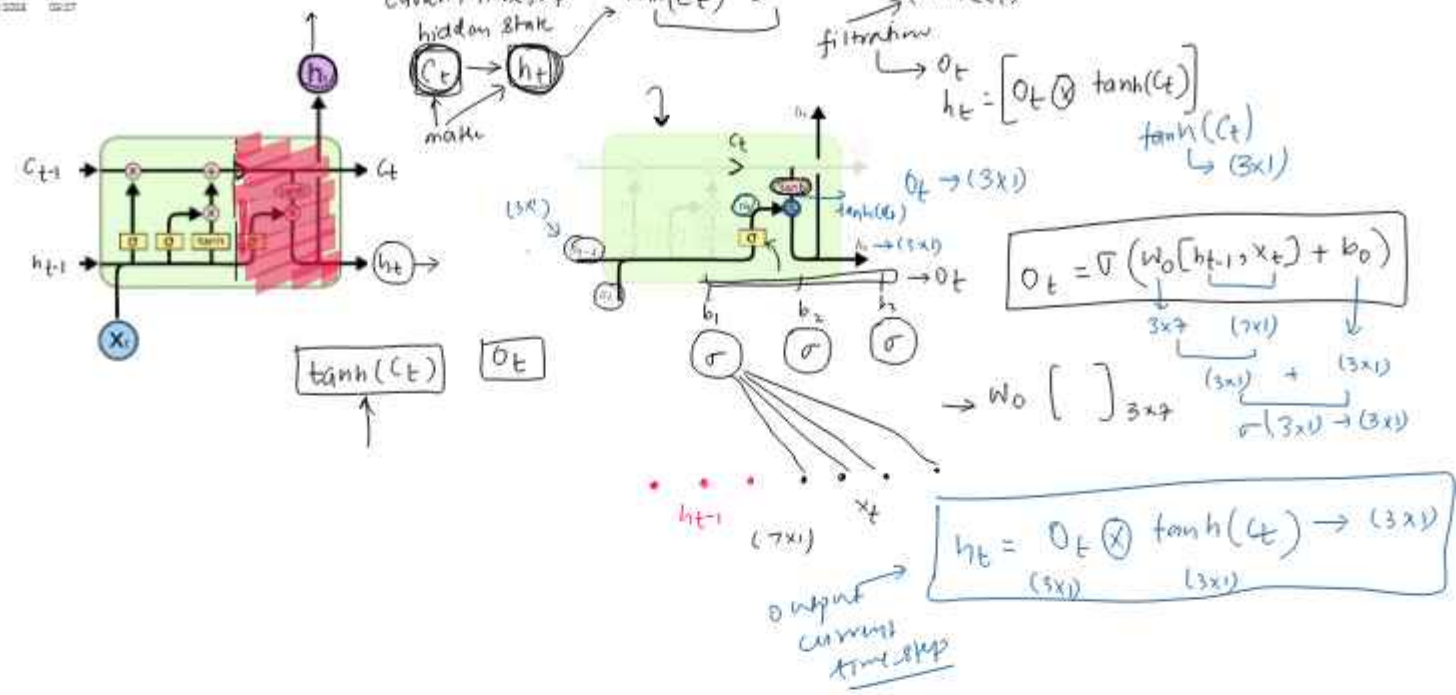
$C_t = [4\ 5\ 6]$

long term context





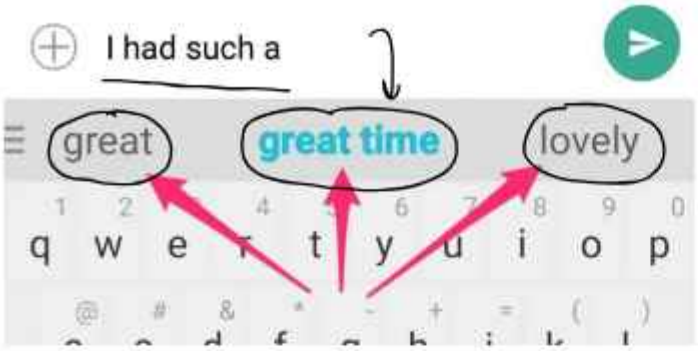
The Output Gate

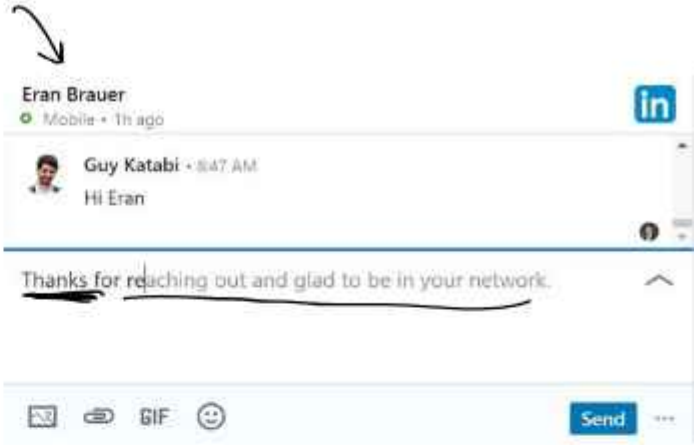


What is a Next Word Predictor

08 September 2023 08:50

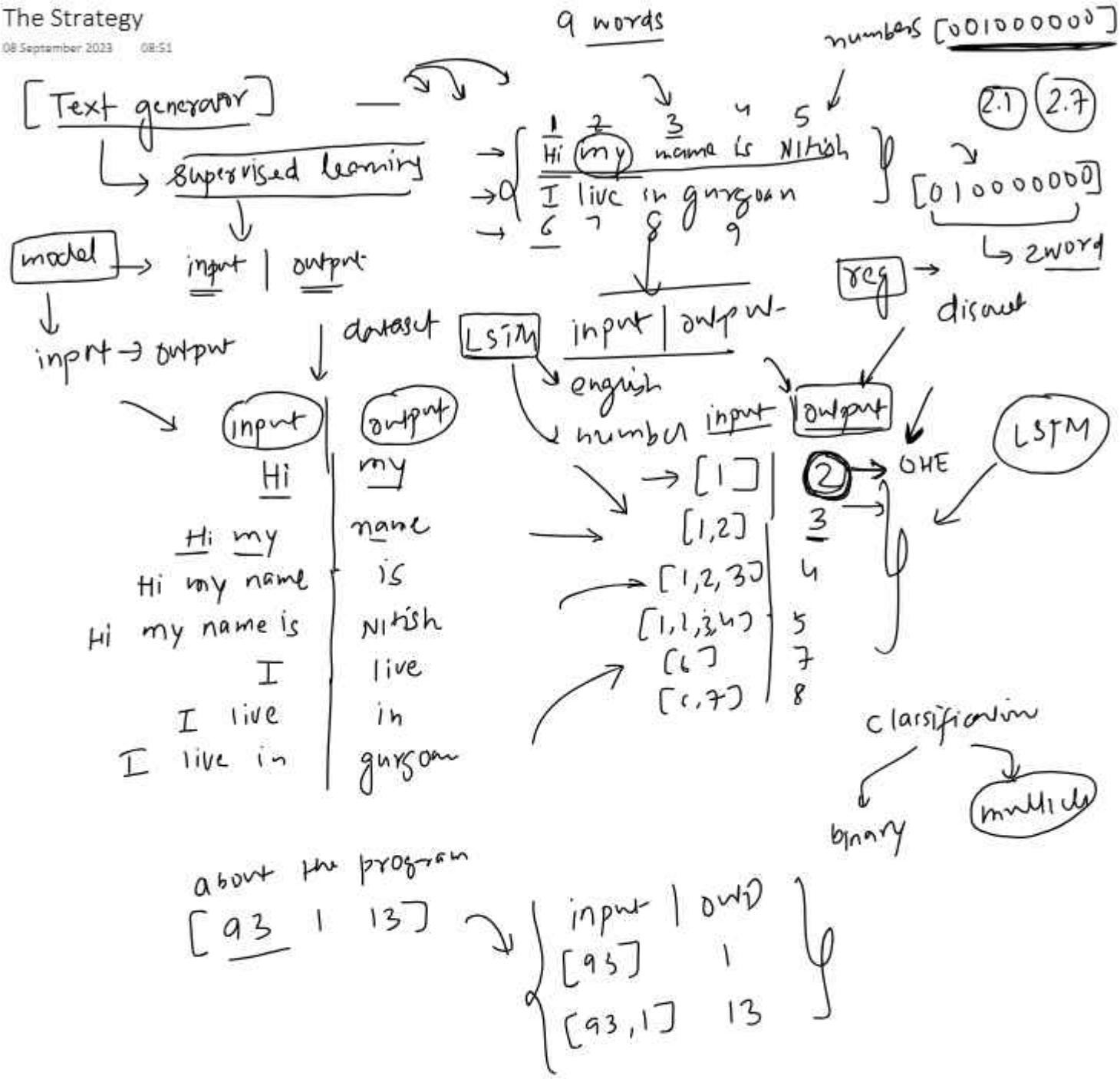
code





The Strategy

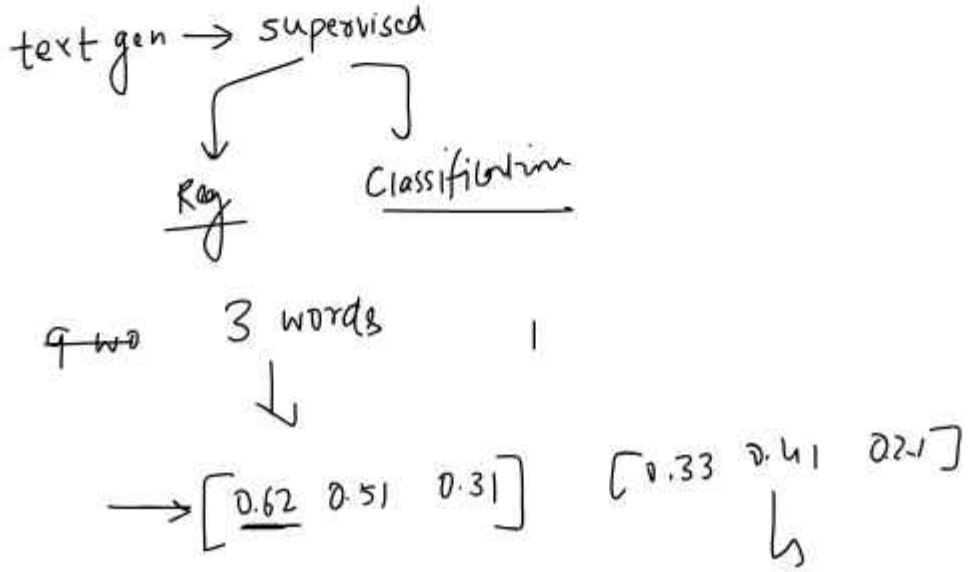
08 September 2023 08:51

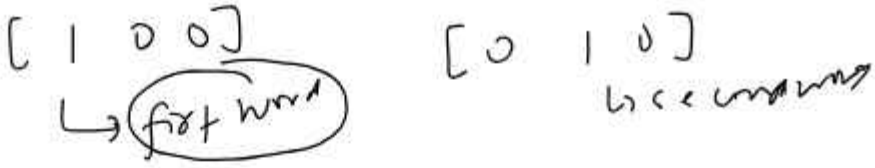


about the program

[93 1 13]

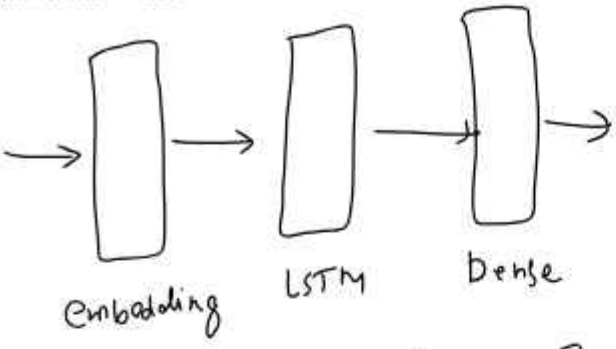
input	out
[93]	1
[93,1]	13

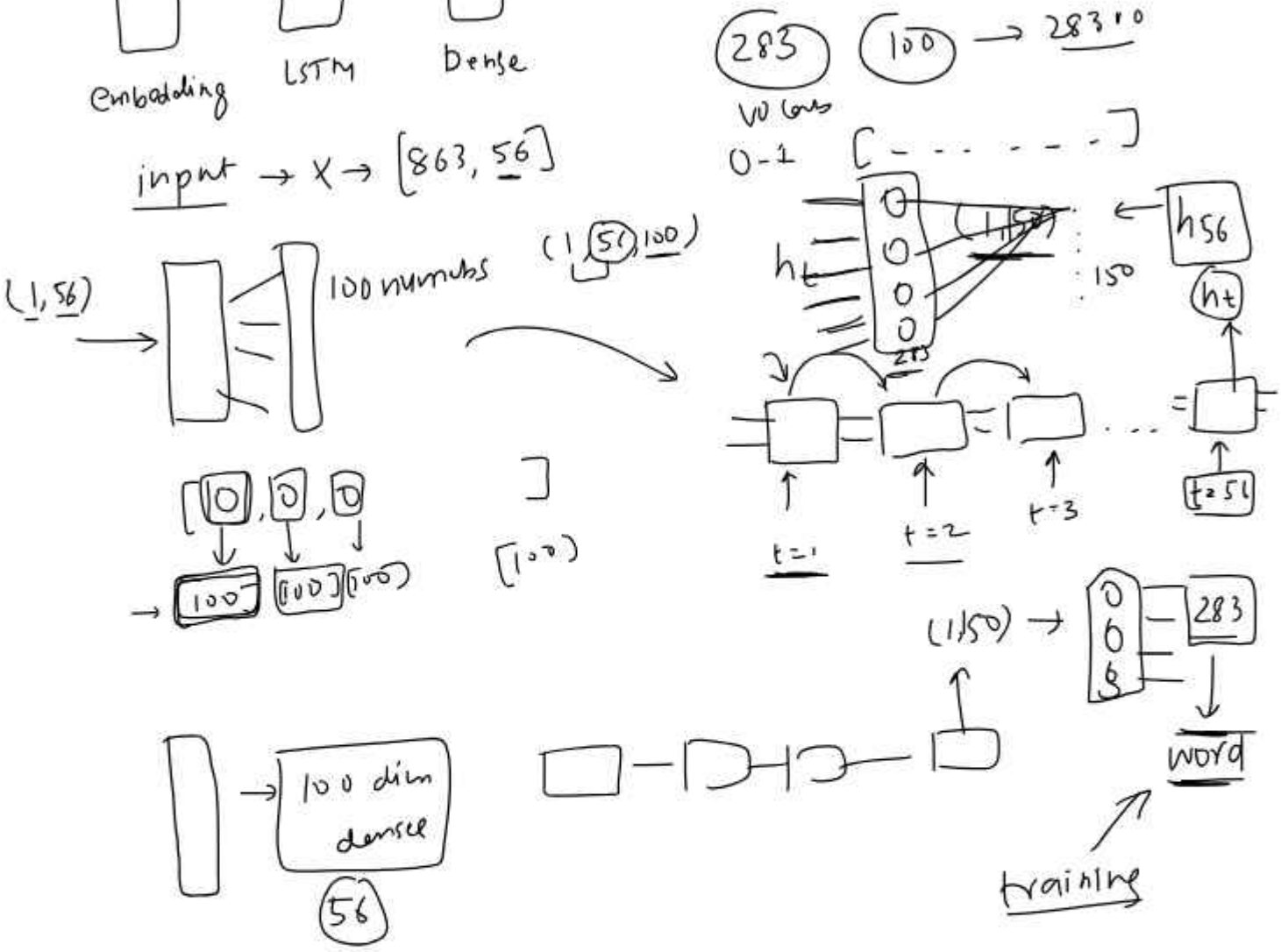



[1 0 0]
first word
[0 1 0]

The Architecture

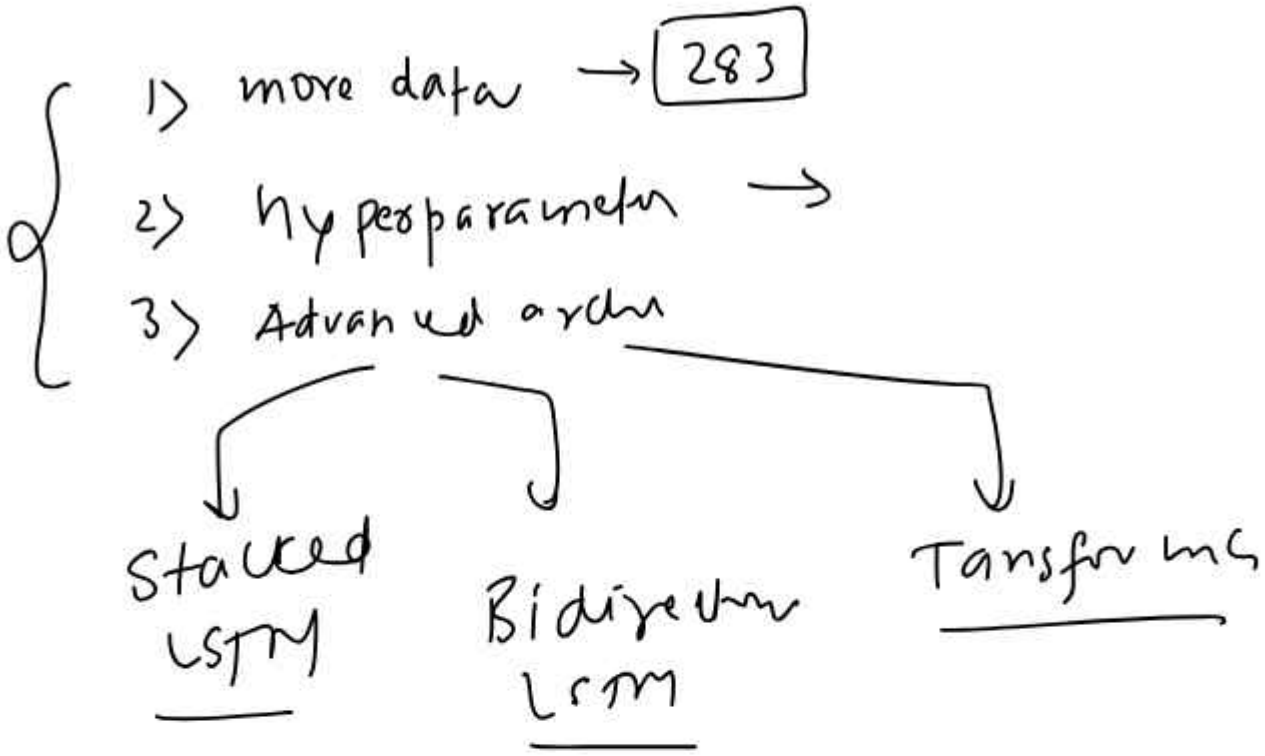
08 September 2023 08:55





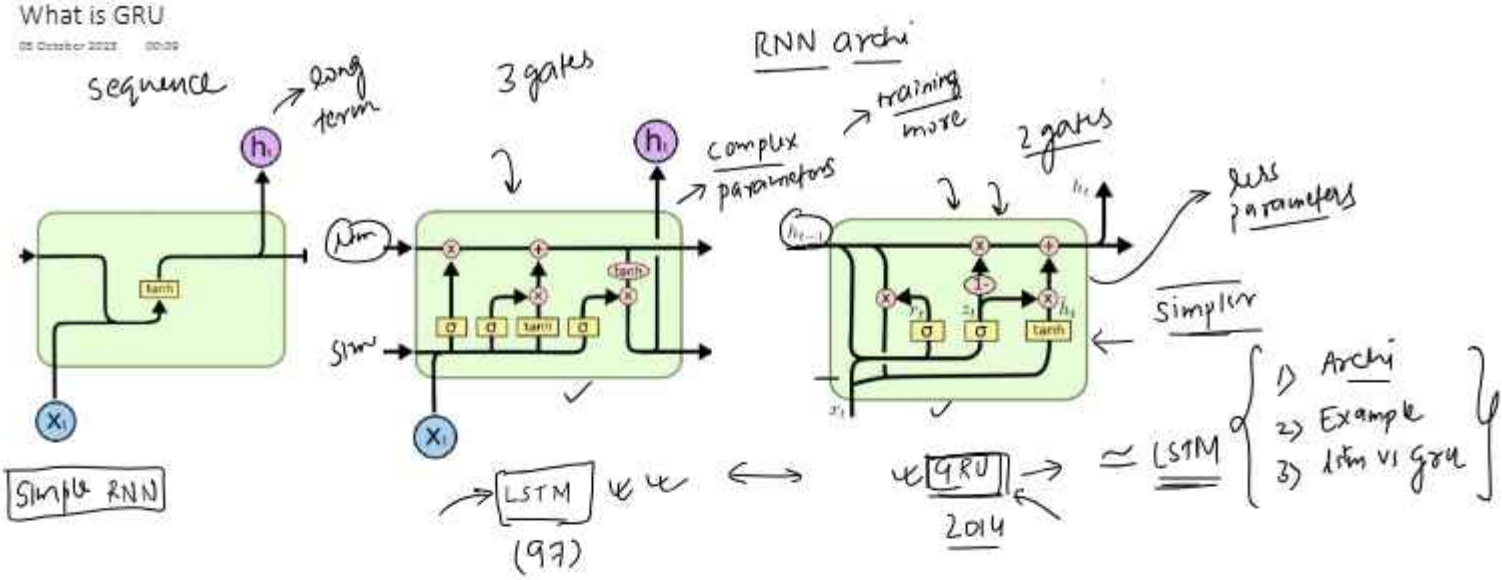
How to improve performance?

08 September 2023 08:51



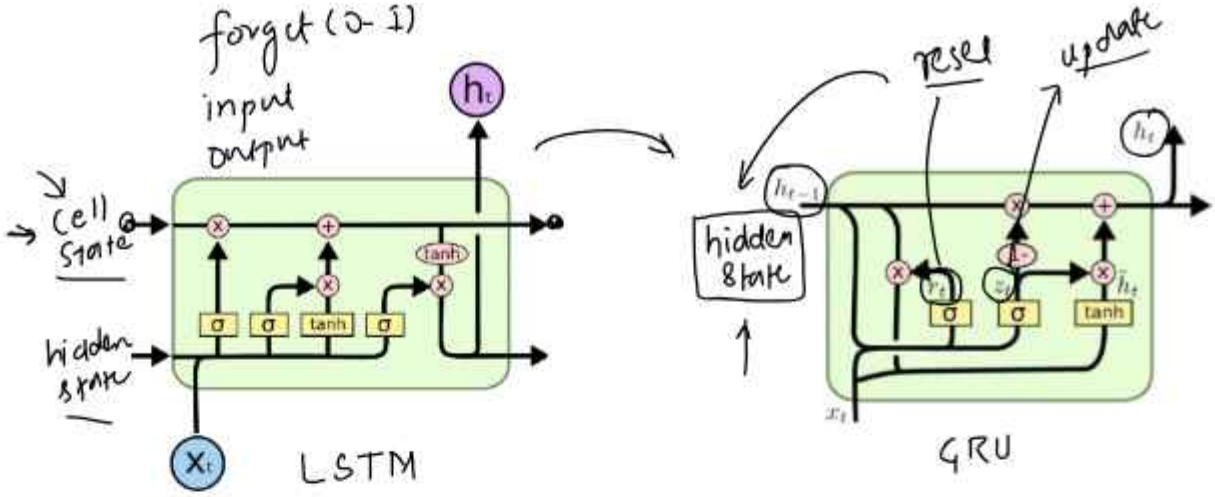
What is GRU

08 October 2022 00:09

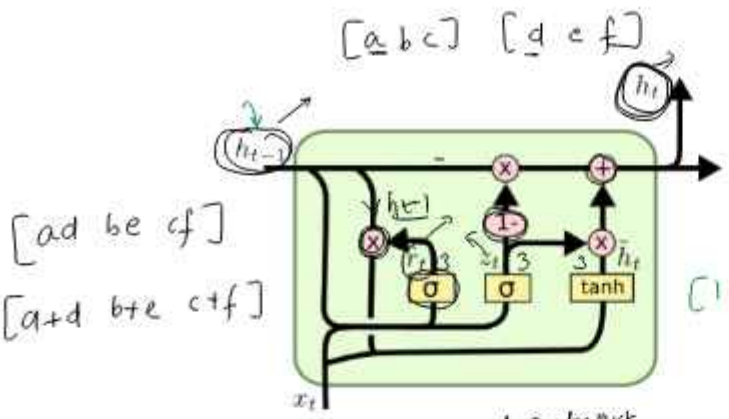


The Big Idea Behind GRU

05 October 2023 00:47



The Setup



[a b c] [d e f]

[ad be cf]

[a+d b+e c+f]

neural network layers (FC layers)

→ Advise → LSTM/GRU → confusing

Goal → t

prev hidden state | curr hidden state | input

h_{t-1} x_t → h_t

vectors

candidate hidden state

h_{t-1} h_t x_t r_t z_t $\tilde{h}_t$?

σ tanh ?

reset gate

update gate

[1 2 3 4]

X + (1-) ?

[1 2 3 4] update same

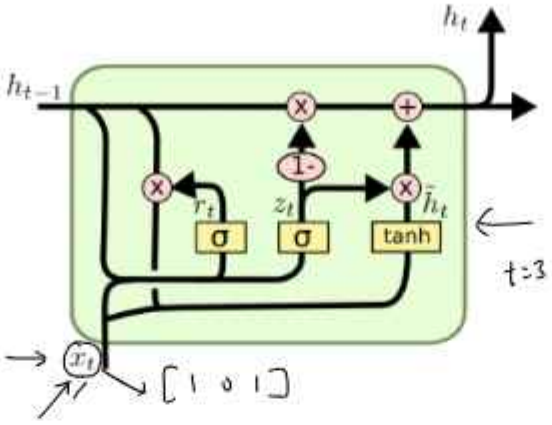
number of = 5 6

10

σ → σ σ σ σ σ

The Input Xt

28 October 2023 01:52



t input → vector

OHE / BOW / Word level

Sentiment

text	review
cat mat rat	1
cat rat rat	0
mat rat mat	1

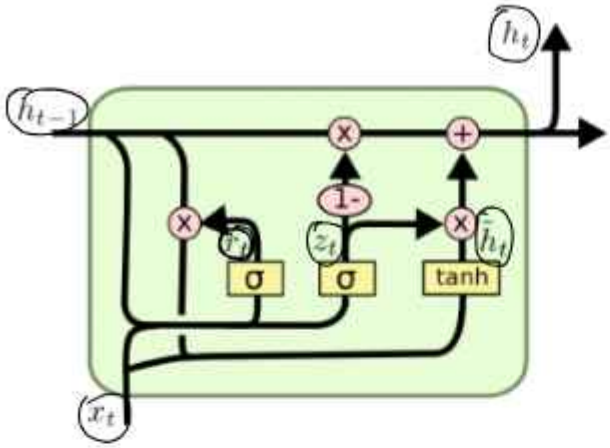
S1 [1 0 0] [0 1 0] [0 0 1]

S2 [1 0 0] [0 0 1] [0 0 1]

S3 [0 1 0] [0 0 1] [0 1 0]

Architecture

05 October 2023 02:10



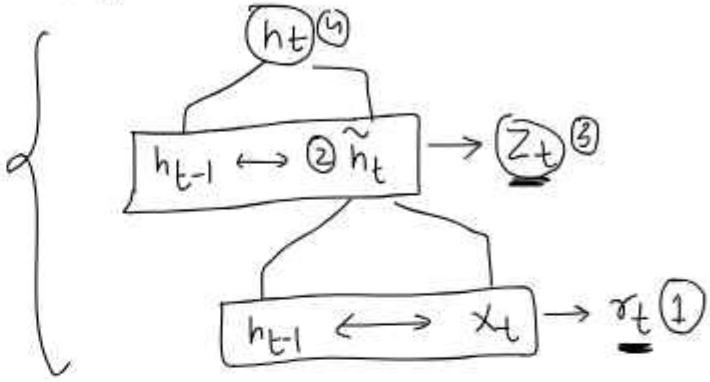
h_{t-1} , x_t ⟶ h_t

→ 1) Calculate r_t (reset gate)

→ 2) Calculate $\tilde{h}_t$ (candidate hidden state)

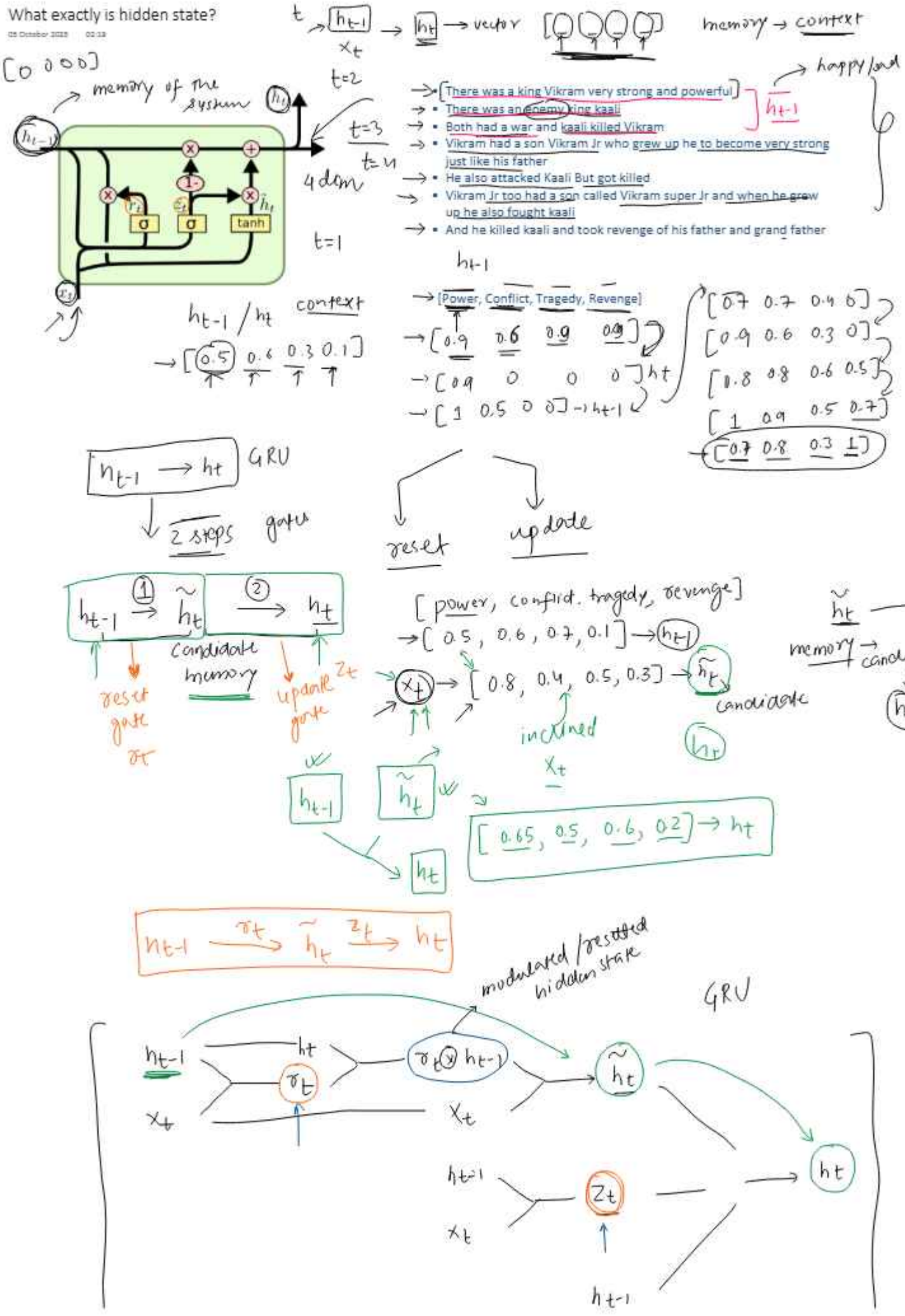
→ 3) Calculate z_t (update gate)

→ 4) Calculate h_t (current hidden state)



What exactly is hidden state?

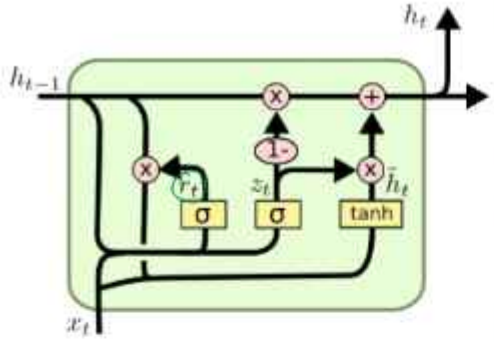
05 October 2023 02:13




h_{t-1}

Calculating the reset gate

08 October 2023 14:24



reset

r_t → vector → h_{t-1}

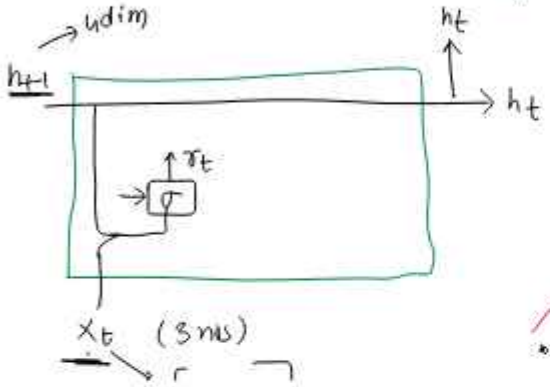
↳ gate

$h_{t-1} = [0.6, 0.6, 0.7, 0.1]$ lower lower higher

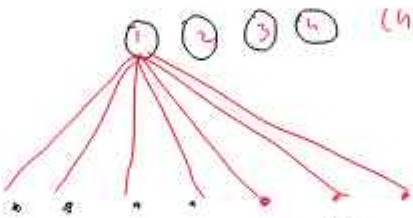
$r_t = [0.8, 0.2, 0.1, 0.9]$

gate — 80 percent 20% — 90%

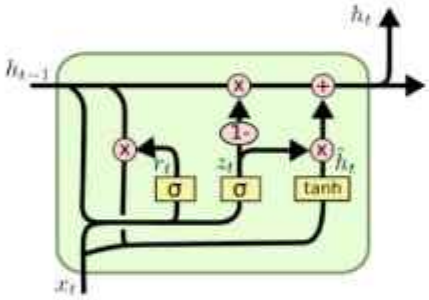
reset

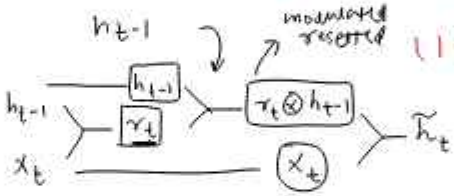


$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$



← (7x4) = 28 + 4





(1x7) (7x4)

(1x4) + (1x4) → ((1x4))

↓

(1x4) → r_t

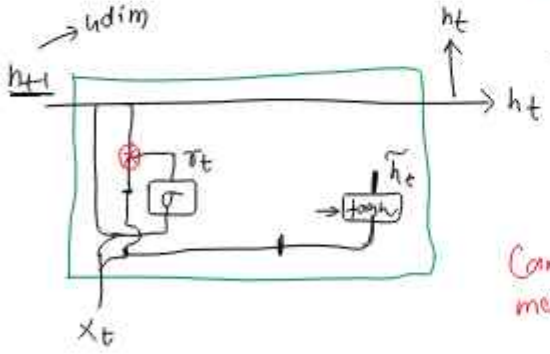
$h_{t-1} = [0.6, 0.6, 0.7, 0.1]$

$r_t = [0.8, 0.2, 0.1, 0.9]$ 90% power content tragedy revenge

$\tilde{h}_t = [0.7\ 0.2\ 0.1\ 0.2]$ ↑ x_t

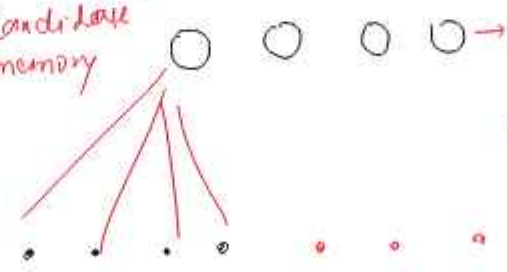
$h_{t-1} \otimes r_t = [0.48, 0.12, 0.07, 0.09]$ → x_t

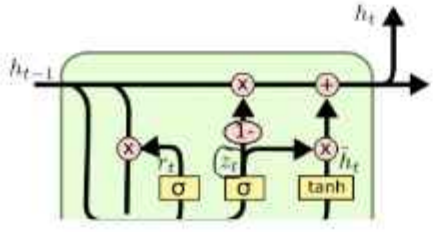
h_{t-1} / $\tilde{h}_t$ → z_t

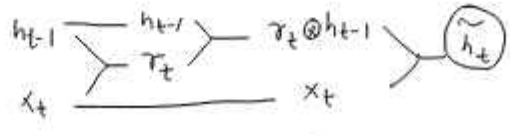


$\tilde{h}_t = \tanh(W_c[h_{t-1} \otimes r_t, x_t] + b_c)$

Candidate memory



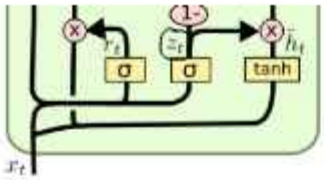




$\tilde{h}_t = [0.7\ 0.2\ 0.1\ 0.2]$

$h_{t-1} = [0.6\ 0.6\ 0.7\ 0.1]$

h_{t-1} z_t



h_{t-1} x_t → z_t

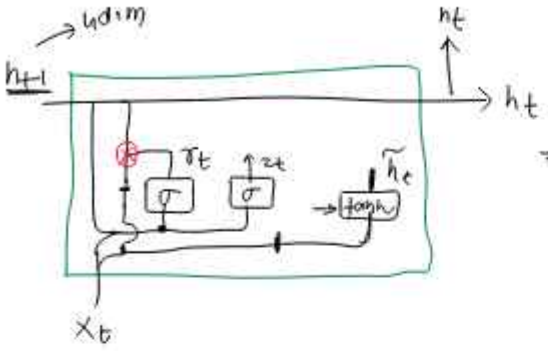
$[[- - - -]]$ (0-1)

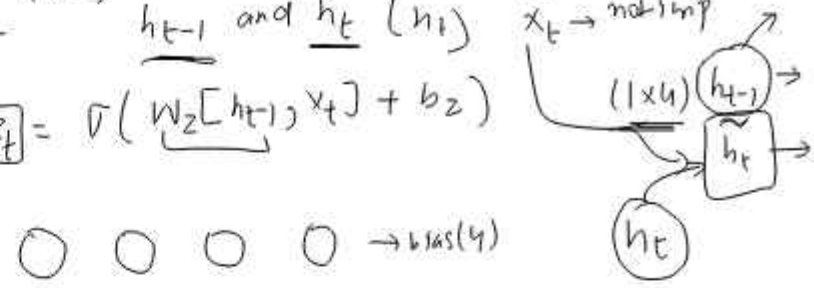
x_t → important
x_t → not imp

gate

h_{t-1} and $\tilde{h}_t$ (h_t)

$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$

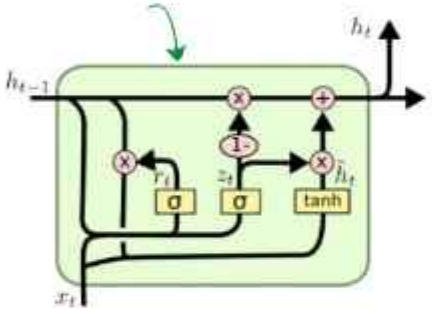




○ ○ ○ ○ → bias(4)

→ (7×4)

h_{t-1}



$\tilde{h}_t = [0.7\ 0.2\ 0.1\ 0.2]$

$h_{t-1} = [0.6\ 0.6\ 0.7\ 0.1]$

$z_t = [0.1\ 0.7\ 0.8\ 0.2]$

h_t [0.07 0.14 0.08 0.04]

0.1 0.1

$h_t = (1 - z_t) \otimes h_{t-1} \oplus z_t \otimes \tilde{h}_t$

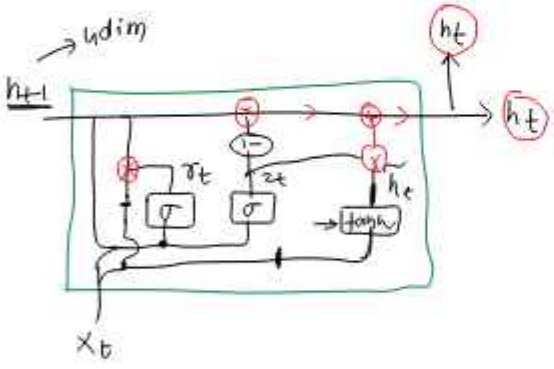
$(1-z_t) \to 0.9$

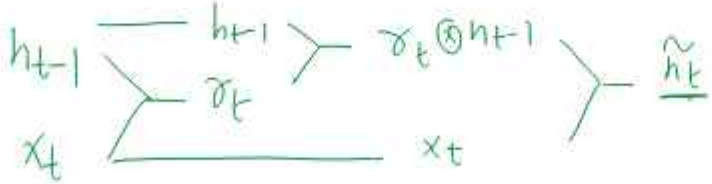
$(1-z_t)$ [0.9 0.3 0.2 0.8] [0.1 0.6 0.7 0.1]

[0.54 0.18 0.14 0.08]

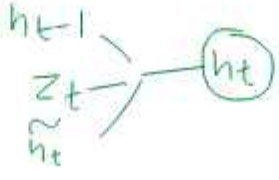
h_t [0.61 0.32 0.22 0.12]

power angle tragedy revenge









LSTM vs GRU

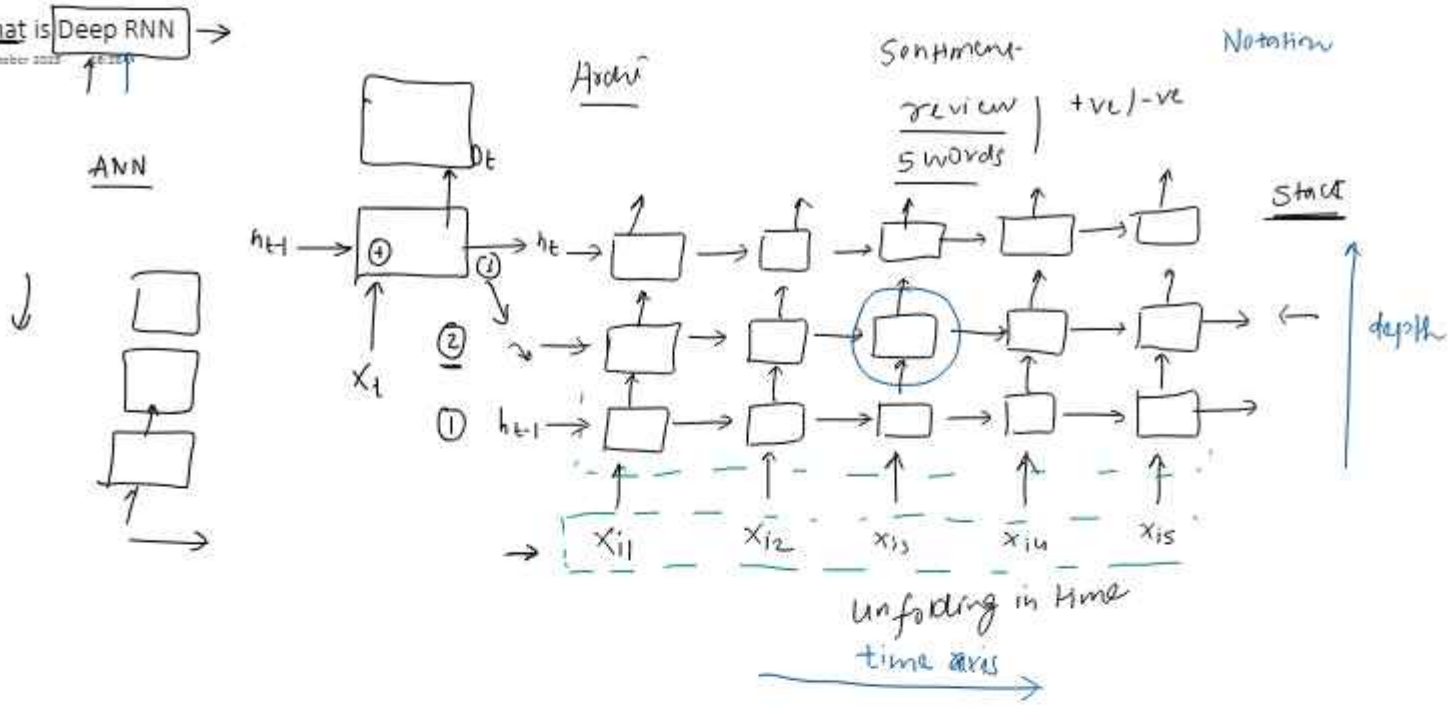
05 October 2023 16:45

Here are the main differences between LSTM and GRU:

1. Number of Gates:
   - LSTM: Has three gates — input (or update) gate, forget gate, and output gate.
   - GRU: Has two gates — reset gate and update gate.
2. Memory Units:
   - LSTM: Uses two separate states - the cell state (ct) and the hidden state (ht). The cell state acts as an "internal memory" and is crucial for carrying long-term dependencies.
   - GRU: Simplifies this by using a single hidden state (ht) to both capture and output the memory.
3. Parameter Count: —
   - LSTM: Generally has more parameters than a GRU because of its additional gate and separate cell state. For an input size of d and a hidden size of h, the LSTM has 4 ×((d×h)+(h×h)+h)4×((d×h)+(h×h)+h)parameters.
   - GRU: Has fewer parameters. For the same sizes, the GRU has 3×((d×h)+(h×h)+h)3 ×((d×h)+(h×h)+h)parameters.
4. Computational Complexity:
   - LSTM: Due to the extra gate and cell state, LSTMs are typically more computationally intensive than GRUs.
   - GRU: Is simpler and can be faster to compute, especially on smaller datasets or when computational resources are limited.
5. Empirical Performance:
   - LSTM: In many tasks, especially more complex ones, LSTMs have been observed to perform slightly better than GRUs.
   - GRU: Can perform comparably to LSTMs on certain tasks, especially when data is limited or tasks are simpler. They can also train faster due to fewer parameters.
6. Choice in Practice:
   - The choice between LSTM and GRU often comes down to empirical testing. Depending on the dataset and task, one might outperform the other. However, GRUs, due to their simplicity, are often the first choice when starting out.

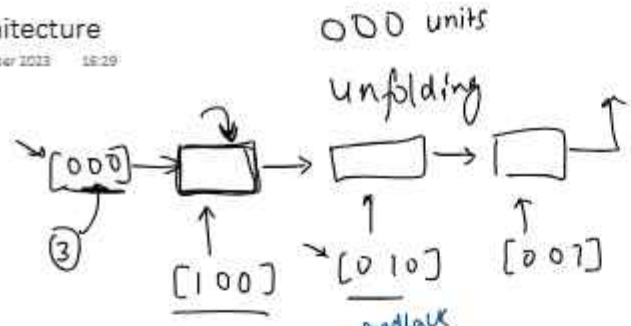
What is Deep RNN →

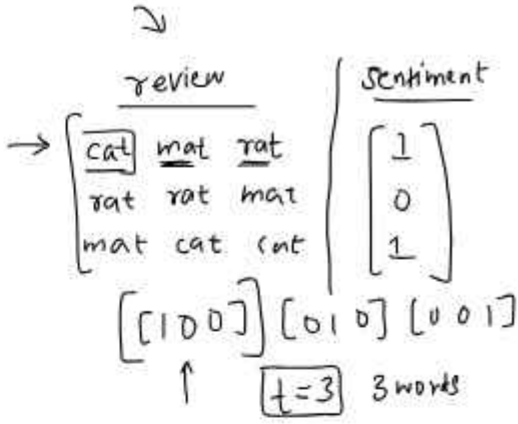
17 October 2022 8:28 PM

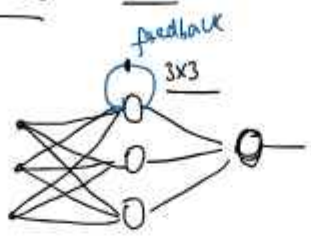


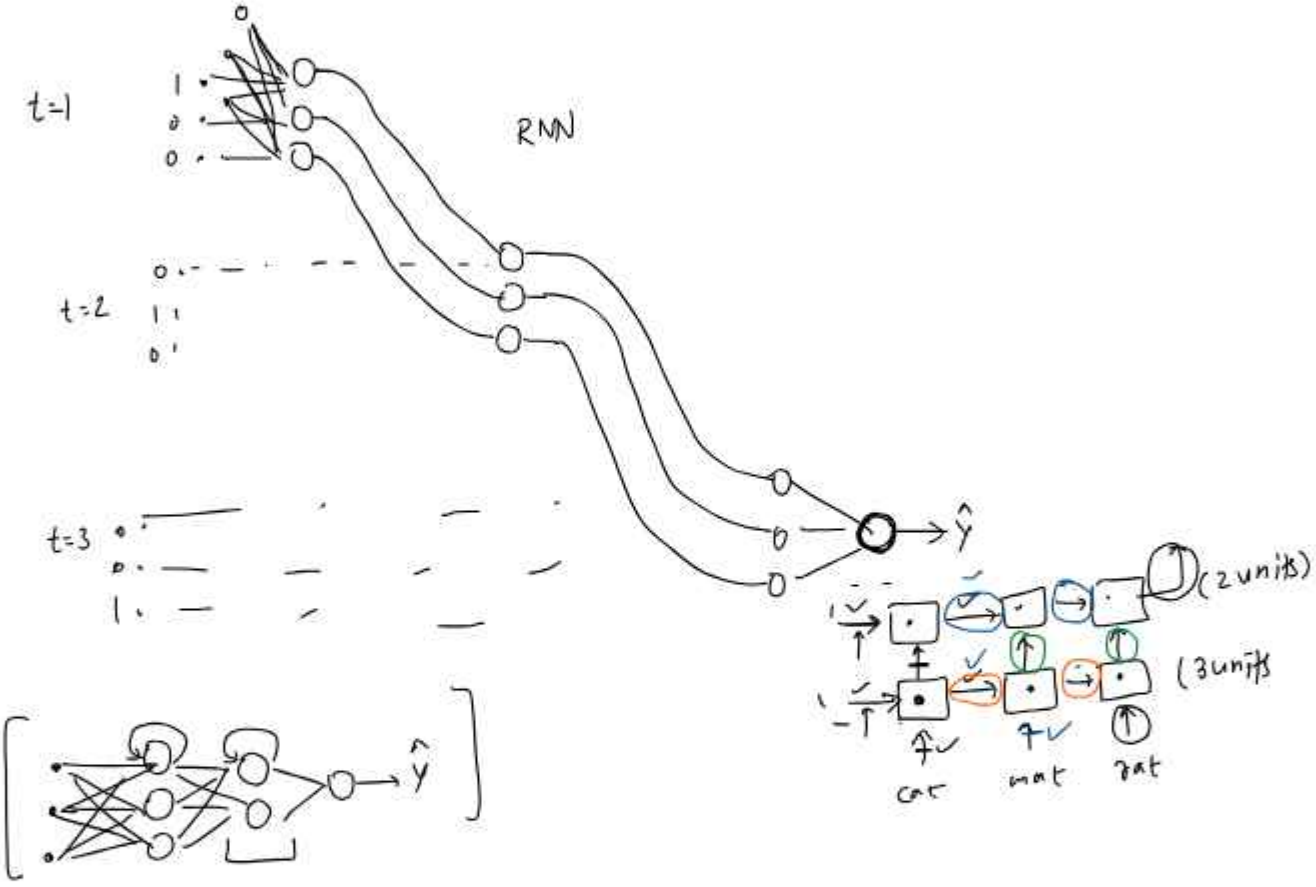
Architecture

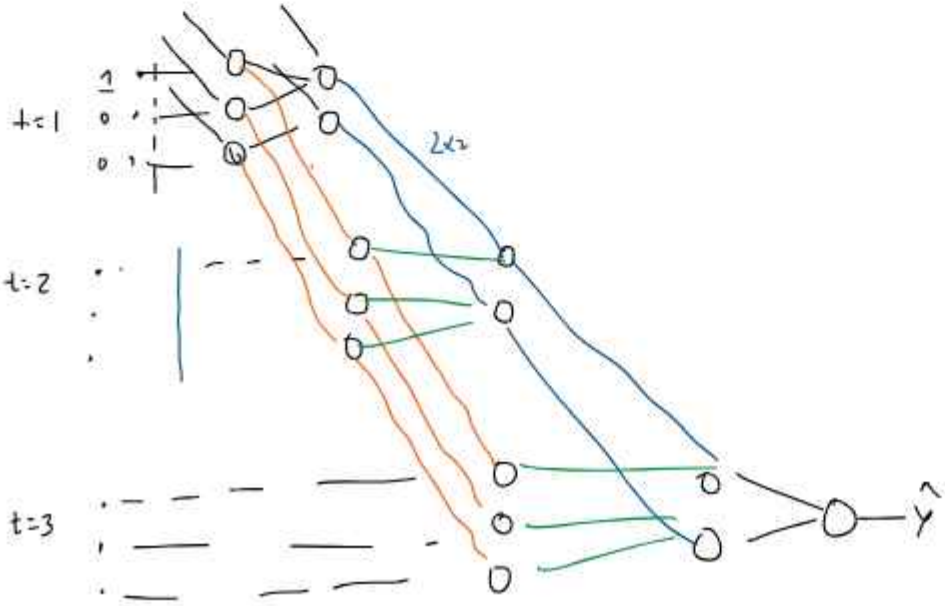
17 October 2023 16:29

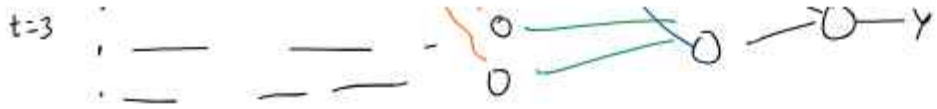






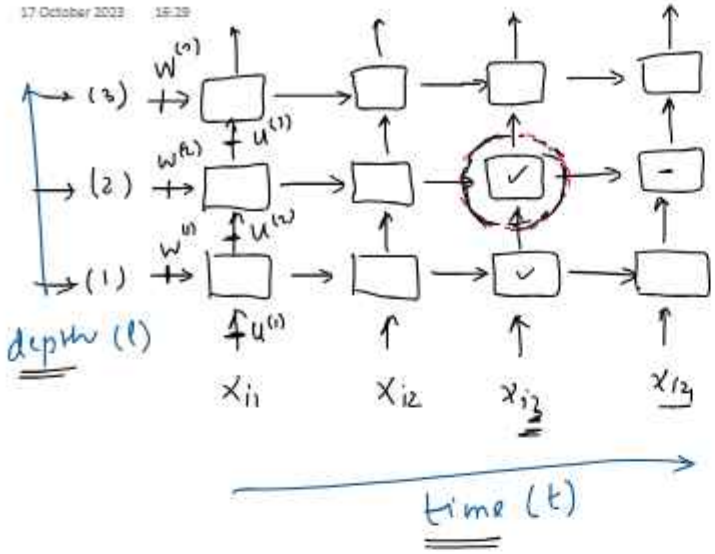


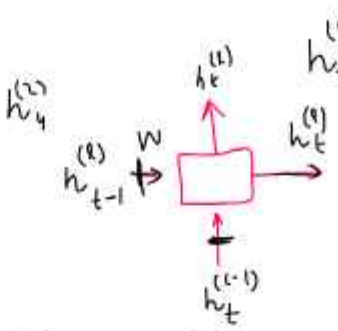



t=3
y

Notation

17 October 2023 16:39





$$h_t^{(l)} = \tanh\left(\right.$$

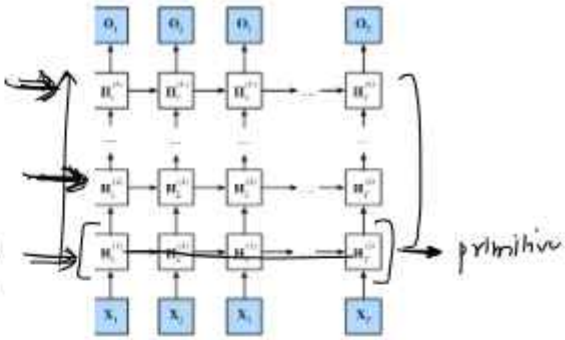
$$\left[h_t^{(l)} = \tanh\left(W^{(l)} h_{t-1}^{(l)} + U^{(l)} h_t^{(l-1)} + b \right) \right]$$

Why and When to use?

17 October 2023 16:28

1. Hierarchical Representation ✓
2. Customization for Advanced Tasks

deep RNN — stack — product — sentence



encoder-decoder → machine → deep RNN

love hate amazing terrible

deep RNN

Audio is bad

Display is great

Overall I am happy

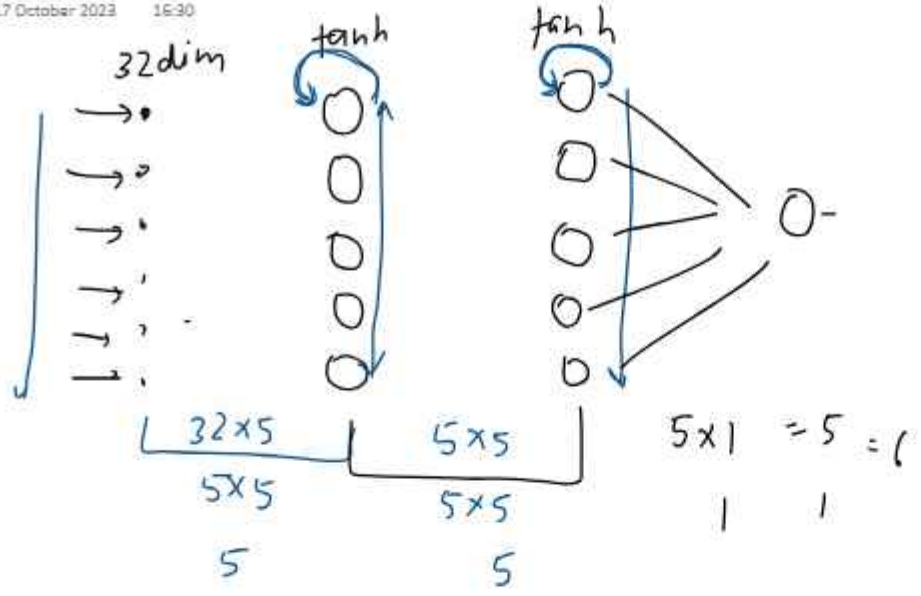
→ sentiment

When to use Deep RNNs?

- Complex tasks
  - Speech recog
  - Machine translation
- Large datasets
  - Overfitting
- Computational
- Simpler Models → Deep RNN

Code Example

17 October 2023 16:30



→ embedded

10000 x 32 = 3,20,000

simpleRNN 1

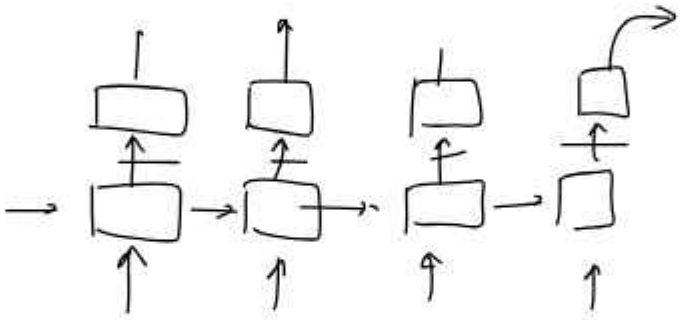
190

simpleRNN 2

55

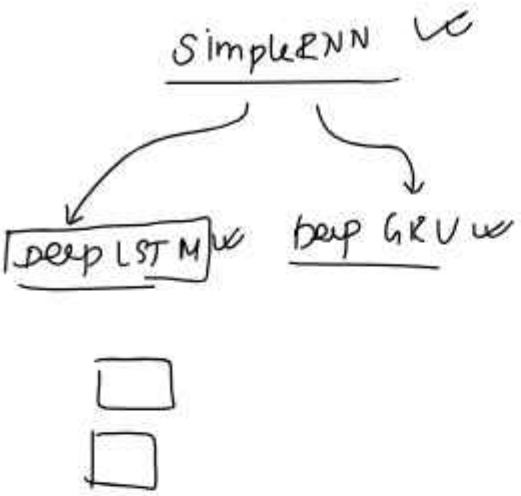
Dense

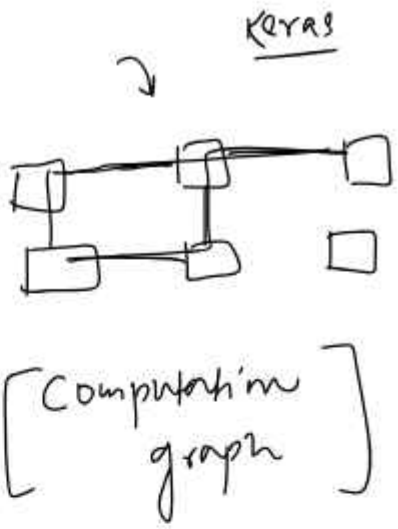
6



Variants

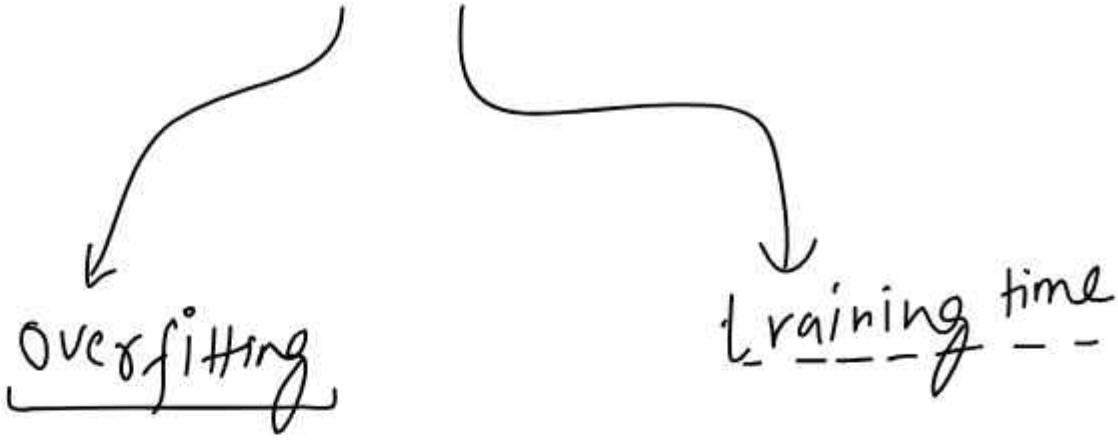
17 October 2023 16:30





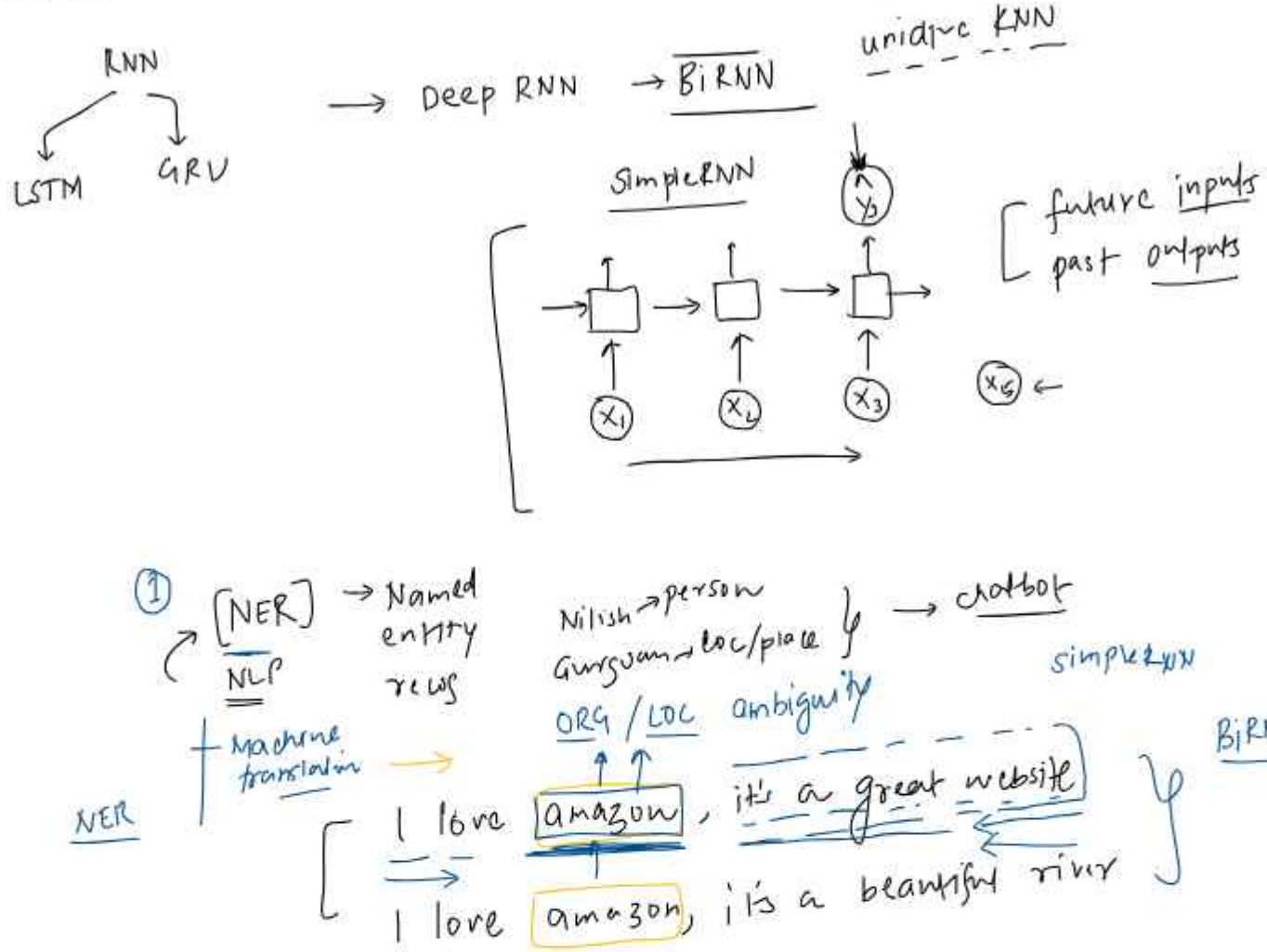
Disadvantages

17 October 2023 16:30



The Why? BiRNN

28 October 2023 15:19



Bidirectional RNN Architecture

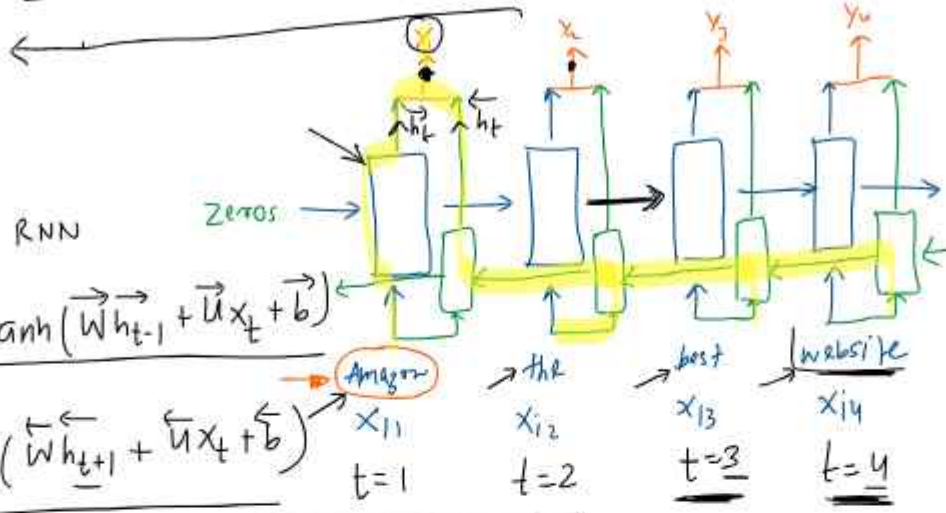
28 October 2023 15:18

Amazon the best website.

Amazon the beautiful river.

loc / org

2 RNNs ✓ forward RNN, ← RNN → concatenate



website → org

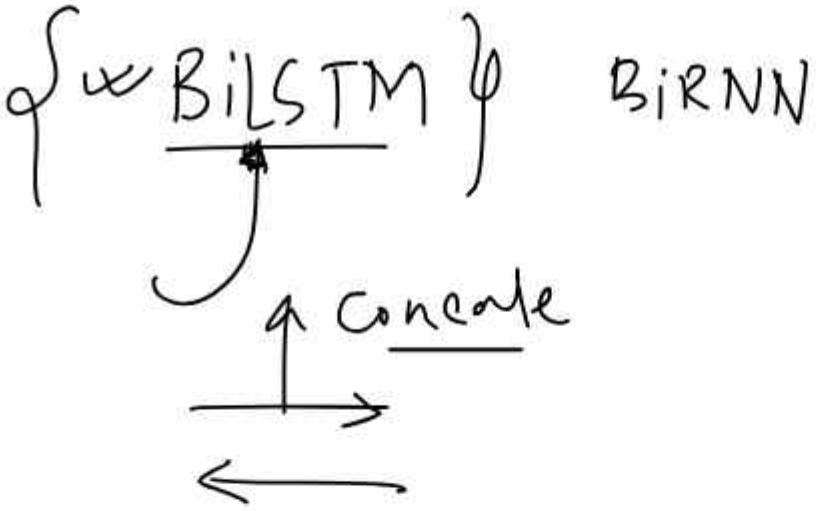
$$\overrightarrow{h_t} = \tanh(\overrightarrow{W}\overrightarrow{h_{t-1}} + \overrightarrow{U}x_t + \overrightarrow{b})$$

$$\overleftarrow{h_t} = \tanh(\overleftarrow{W}\overleftarrow{h_{t+1}} + \overleftarrow{U}x_t + \overleftarrow{b})$$

$$y_t = \sigma(V[\overrightarrow{h_t}, \overleftarrow{h_t}] + b)$$

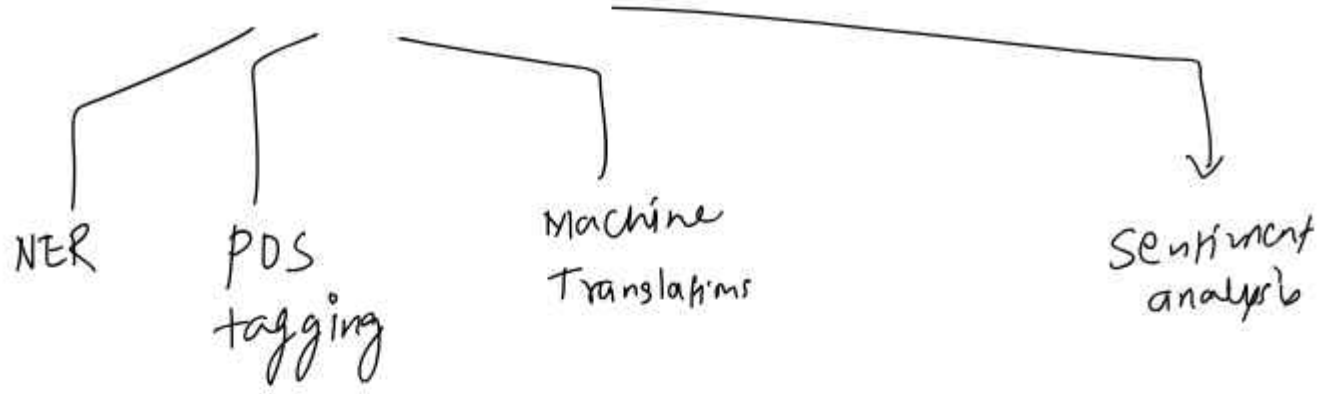
Code

26 October 2023 15:21



Applications and Drawbacks

26 October 2023 15:21



[Time series forecasting]

→ ←

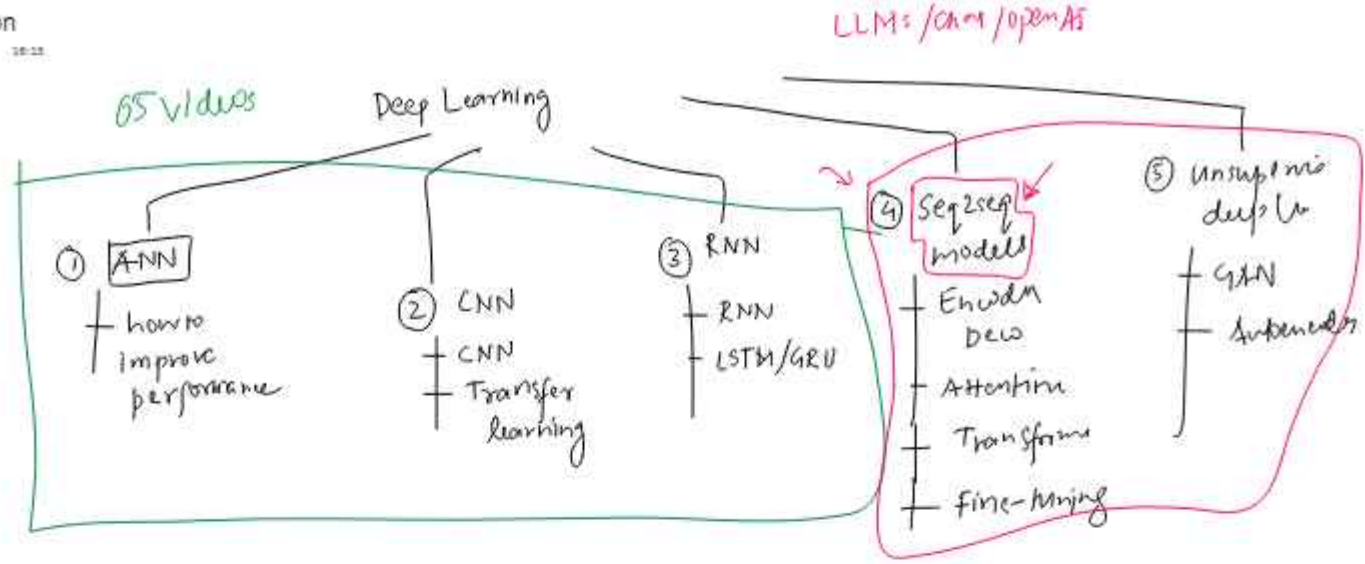
→ Complexity → 190 → 380 → training → overfitting
double

→ → ← [Speech recog] → bi rnn

↳ latency → slow

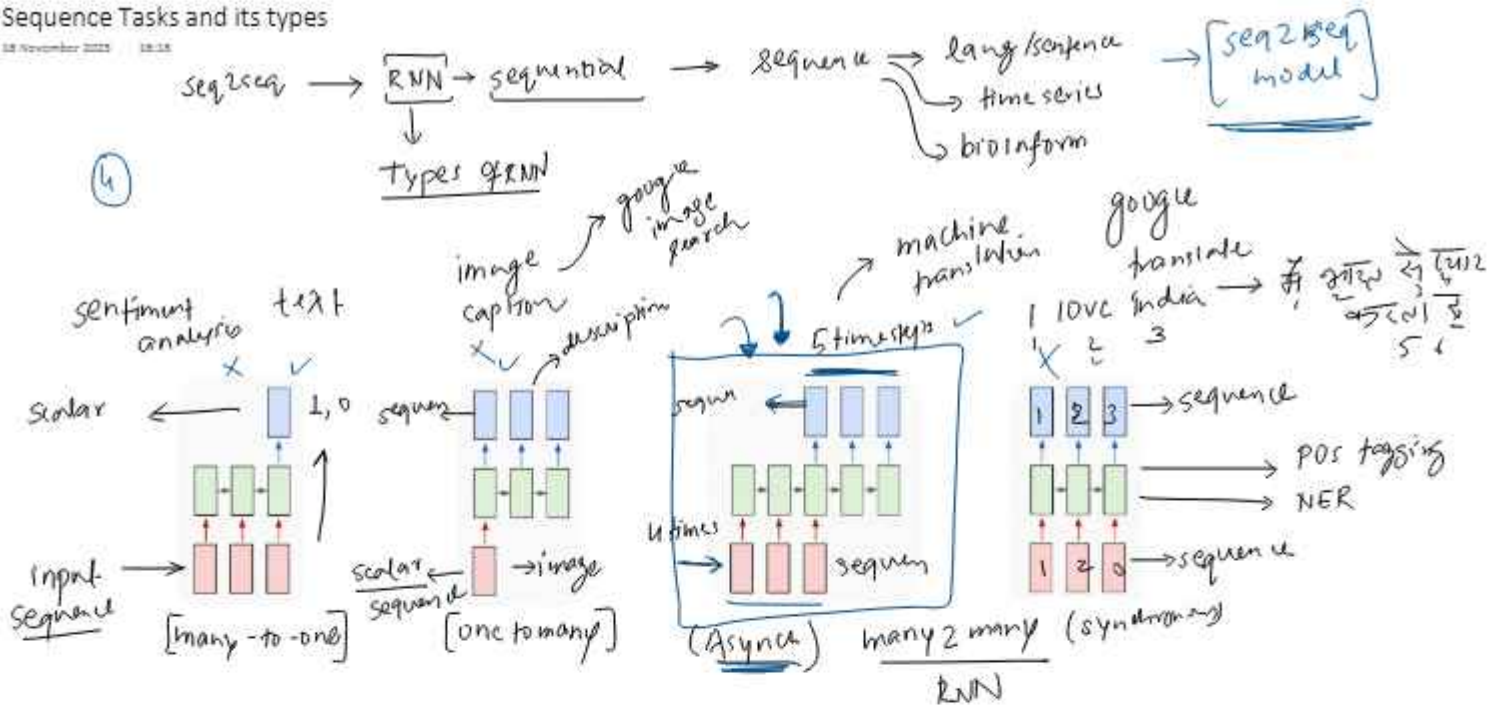
Introduction

18 November 2023 18:18



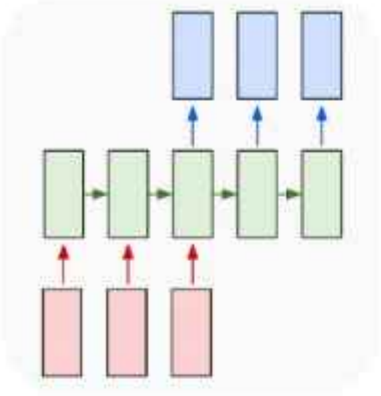
Sequence Tasks and its types

18 November 2023 18:18



Seq2Seq Tasks

16 November 2023 16:16



NLP

Seq2seq → machine trans

1) text summary → seq → seq

2) Question answer → seq → seq (Knowledge base)

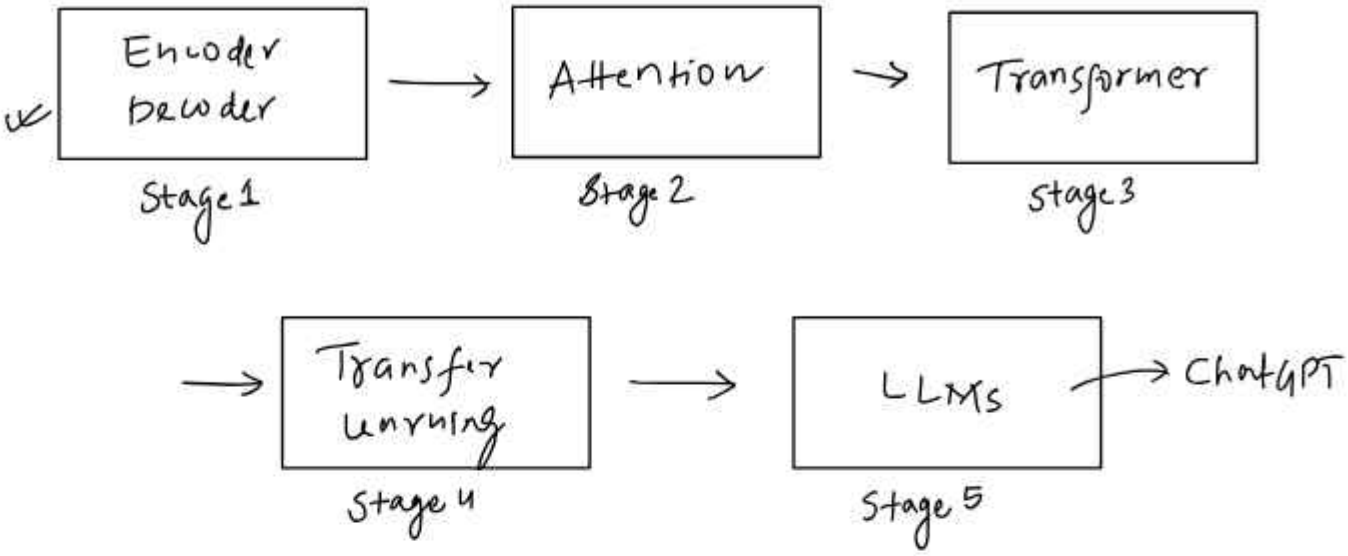
3) chatbot → input (text) → output (text)

4) speech-to-text → seq seq → seq2seq → chatgpt

History of Seq2Seq Models

16 November 2023 16:16

ChatGPT



Stage 1 - Encoder Decoder Architecture

18 November 2023 18:16

2014 Seminal

Sequence to Sequence Learning with Neural Networks

Ilya Sutskever
Google
ilyasu@google.com

Oriol Vinyals
Google
vinyals@google.com

Quoc V. Le
Google
qvl@google.com

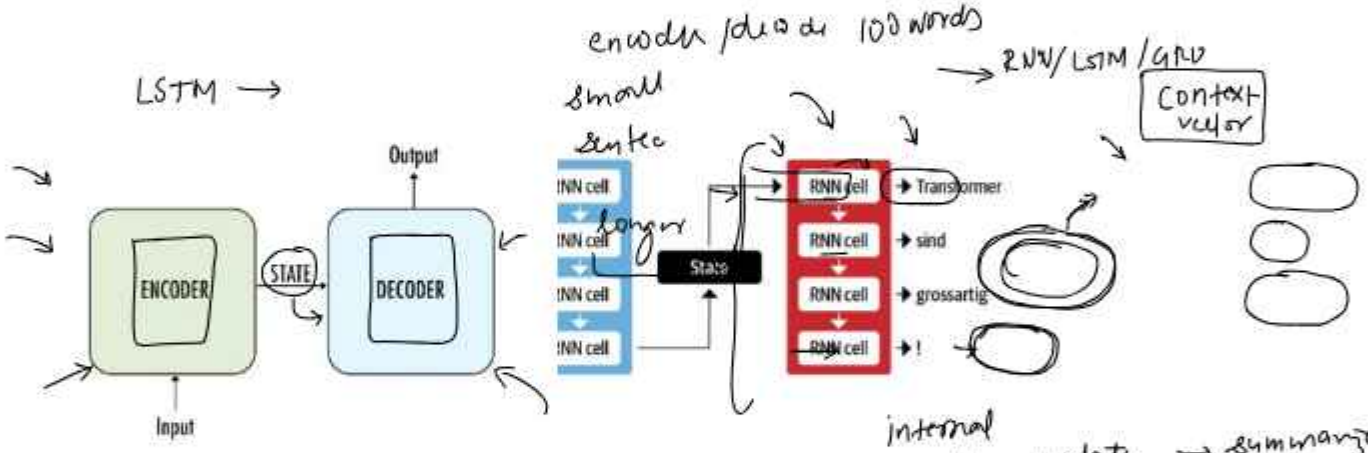
Abstract

Deep Neural Networks (DNNs) are powerful models that have achieved excellent performance on difficult learning tasks. Although DNNs work well whenever large labeled training sets are available, they cannot be used to map sequences to sequences. In this paper, we present a general end-to-end approach to sequence learning that makes minimal assumptions on the sequence structure. Our method uses a multilayered Long Short-Term Memory (LSTM) to map the input sequence to a vector of a fixed dimensionality, and then another deep LSTM to decode the target sequence from the vector. Our main result is that on an English to French translation task from the WMT'14 dataset, the translations produced by the LSTM achieve a BLEU score of 34.8 on the entire test set, where the LSTM's BLEU score was penalized on out-of-vocabulary words. Additionally, the LSTM did not have difficulty on long sentences. For comparison, a phrase-based SMT system achieves a BLEU score of 33.3 on the same dataset. When we used the LSTM to rerank the 1000 hypotheses produced by the aforementioned SMT system, its BLEU score increases to 36.5, which is close to the previous best result on this task. The LSTM also learned sensible phrase and sentence representations that are sensitive to word order and are relatively invariant to the active and the passive voice. Finally, we found that reversing the order of the words in all source sentences (but not target sentences) improved the LSTM's performance markedly, because doing so introduced many short term dependencies between the source and the target sentence which made the optimization problem easier.



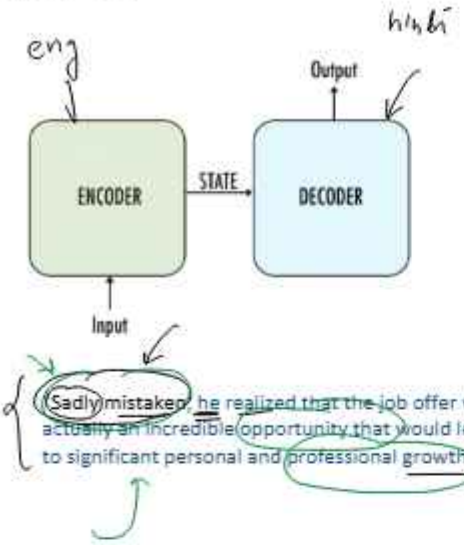
seq2seq
↓
diff
↳ encoder decoder

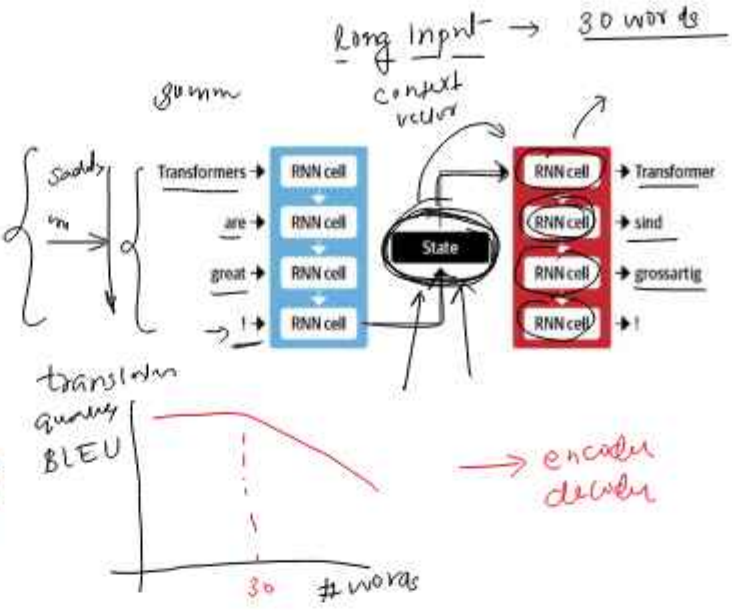
[Ilya Sutskever] → cofounder openAI



Stage 2 - Attention Mechanism

22 November 2023 10:59



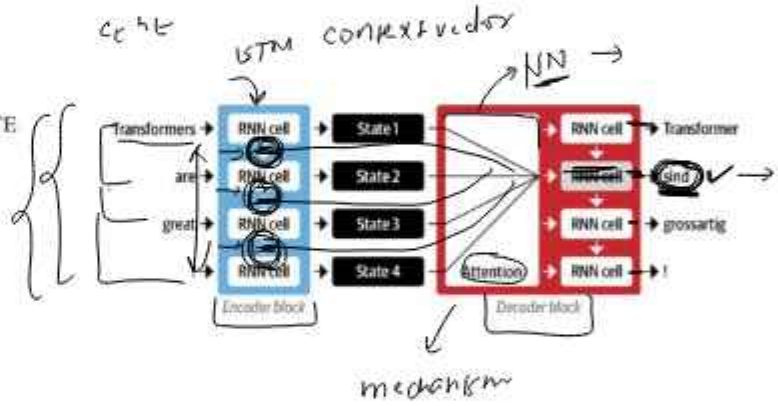


2015 Attention

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho **Yoshua Bengio***
Université de Montréal



ABSTRACT

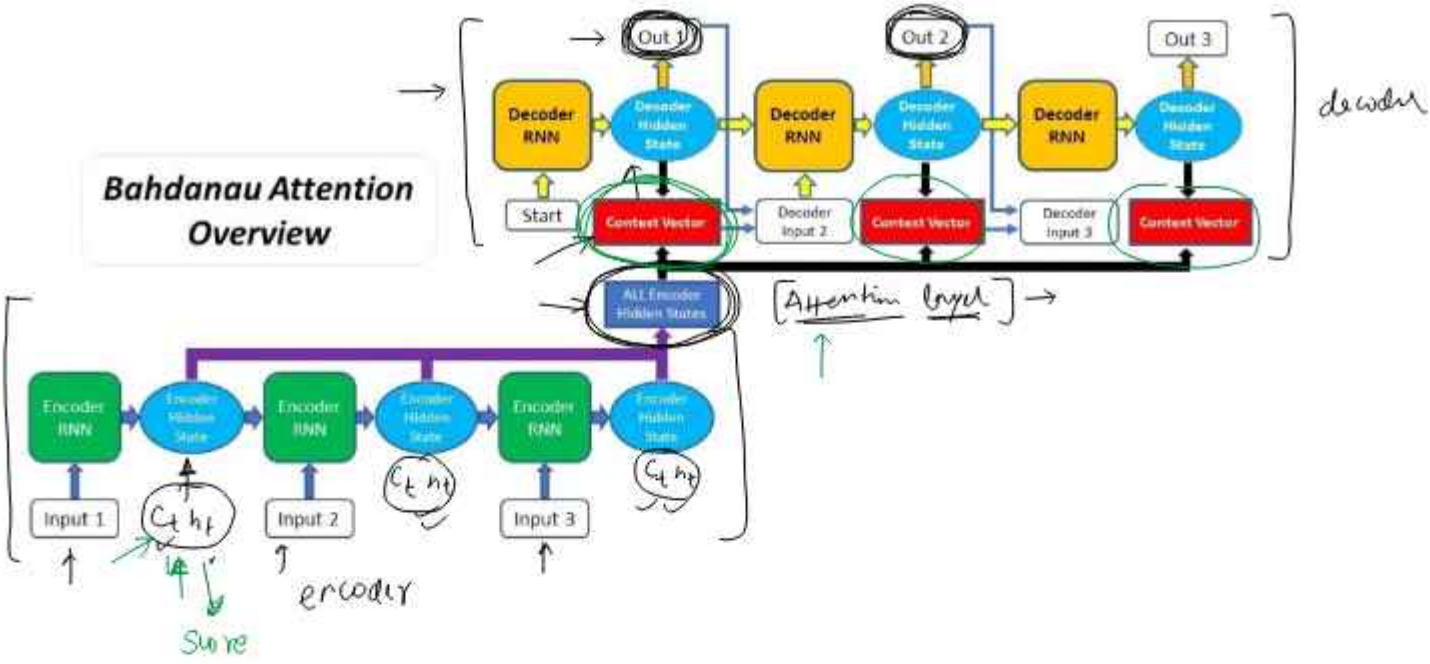
Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder–decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder–decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

1 INTRODUCTION

Neural machine translation is a newly emerging approach to machine translation, recently proposed by Kalchbrenner and Blunsom (2013), Sutskever *et al.* (2014) and Cho *et al.* (2014b). Unlike the traditional phrase-based translation system (see, e.g., Koehn *et al.*, 2003) which consists of many small sub-components that are tuned separately, neural machine translation attempts to build and train a single, large neural network that reads a sentence and outputs a correct translation.

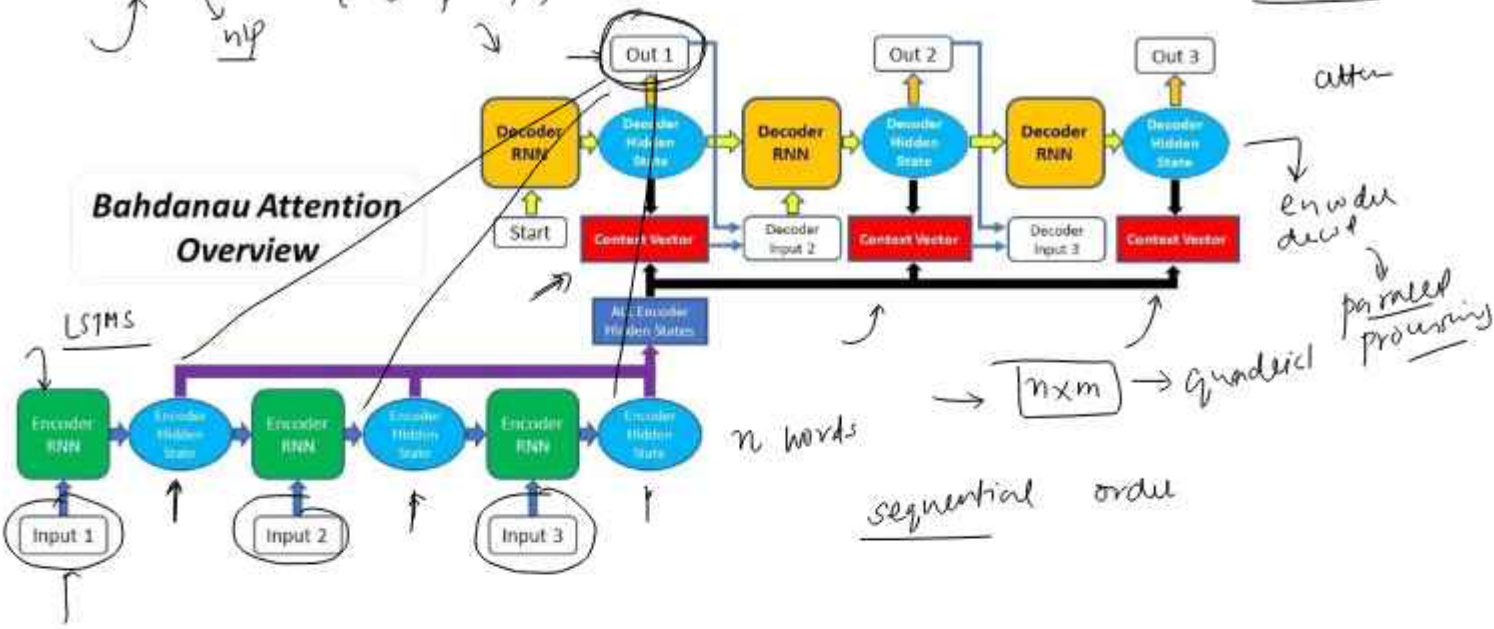
Most of the proposed neural machine translation models belong to a family of *encoder–decoders* (Sutskever *et al.*, 2014; Cho *et al.*, 2014a), with an encoder and a decoder for each language, or involve a language-specific encoder applied to each sentence whose outputs are then compared (Hermann and Blunsom, 2014). An encoder neural network reads and encodes a source sentence into a fixed-length vector. A decoder then outputs a translation from the encoded vector. The whole encoder–decoder system, which consists of the encoder and the decoder for a language pair, is jointly trained to maximize the probability of a correct translation given a source sentence.

A potential issue with this encoder–decoder approach is that a neural network needs to be able to compress all the necessary information of a source sentence into a fixed-length vector. This may make it difficult for the neural network to cope with long sentences, especially those that are longer than the sentences in the training corpus. Cho *et al.* (2014b) showed that indeed the performance of a basic encoder–decoder deteriorates rapidly as the length of an input sentence increases.


Bahdanau Attention Overview
Out 1
Out 2
Out 3
Decoder RNN
Decoder Hidden State
Start
Context Vector
Decoder Input 2
Decoder Input 3
All Encoder Hidden States
Encoder RNN
Encoder Hidden State
Input 1
Input 2
Input 3
decoder
encoder

Stage 3 - Transformers

30 November 2022 12:16





Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Lukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* † illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

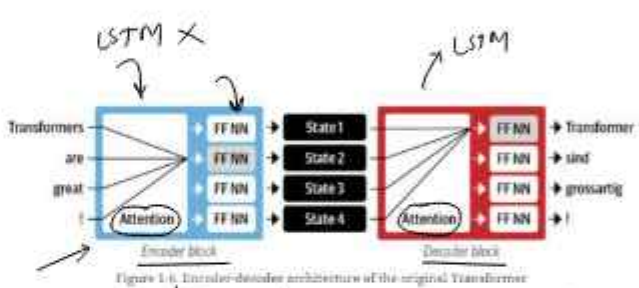


Figure 1.6: Encoder-decoder architecture of the original Transformer

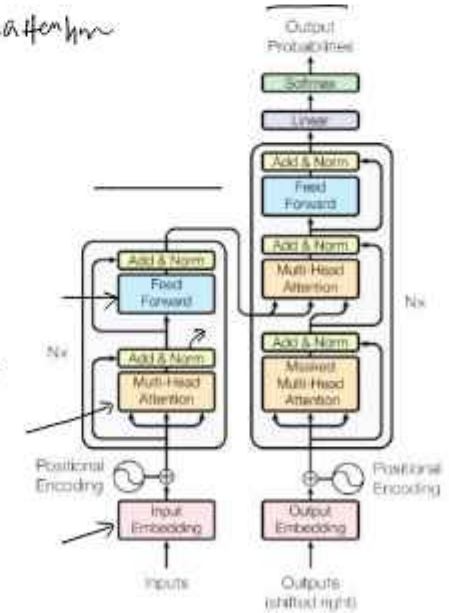


Figure 1: The Transformer - model architecture.

Stage 4 - Transfer Learning

22 November 2023 15:32

1) Hardware → GPUs
2) Time →
3) Data → a lot

→ 100 , 1000

↓

Transformer → Transfer learning

ULMFiT

Universal Language Model Fine-tuning for Text Classification

Jeremy Howard*
fast.ai
University of San Francisco
j@fast.ai

Sebastian Ruder*
Insight Centre, NUI Galway
Aylien Ltd., Dublin
sebastian@ruder.io

primary pretrain

Abstract

Inductive transfer learning has greatly impacted computer vision, but existing approaches in NLP still require task-specific modifications and training from scratch. We propose Universal Language Model Fine-tuning (ULMFiT), an effective transfer learning method that can be applied to any task in NLP, and introduce techniques that are key for fine-tuning a language model. Our method significantly outperforms the state-of-the-art on six text classification tasks, reducing the error by 18-24% on the majority of datasets. Furthermore, with only 100 labeled examples, it matches the performance of training from scratch on 100× more data. We open-source our pretrained models and code[1].

1 Introduction

Inductive transfer learning has had a large impact on computer vision (CV). Applied CV models (including object detection, classification, and segmentation) are rarely trained from scratch, but instead are fine-tuned from models that have been pretrained on ImageNet, MS-COCO, and other datasets (Sharif Razavian et al., 2014; Long et al., 2015a; He et al., 2016; Huang et al., 2017).

Text classification is a category of Natural Language Processing (NLP) tasks with real-world applications such as spam, fraud, and bot detection (Jindal and Liu, 2007; Ngai et al., 2011; Chu et al., 2012), emergency response (Caragea et al., 2011), and commercial document classification, such as for legal discovery (Roitblat et al., 2010).

While Deep Learning models have achieved state-of-the-art on many NLP tasks, these models are trained from scratch, requiring large datasets, and days to converge. Research in NLP focused mostly on *inductive* transfer (Blitzer et al., 2007). For *inductive* transfer, fine-tuning pretrained word embeddings (Mikolov et al., 2013), a simple transfer technique that only targets a model's first layer, has had a large impact in practice and is used in most state-of-the-art models. Recent approaches that concatenate embeddings derived from other tasks with the input at different layers (Peters et al., 2017; McCann et al., 2017; Peters et al., 2018) still train the main task model from scratch and treat pretrained embeddings as fixed parameters, limiting their usefulness.

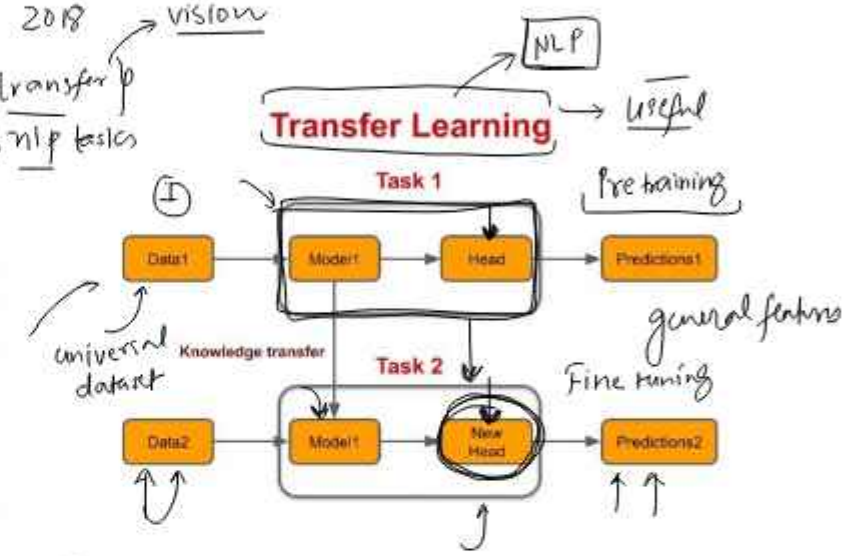
In light of the benefits of pretraining (Erhan et al., 2010), we should be able to do better than *randomly initializing* the remaining parameters of our models. However, inductive transfer via fine-tuning has been unsuccessful for NLP (Mou et al., 2016). Dai and Le (2015) first proposed fine-tuning a language model (LM) but require millions of in-domain documents to achieve good performance, which severely limits its applicability.

We show that not the idea of LM fine-tuning but our lack of knowledge of how to train them effectively has been hindering wider adoption. LMs overfit to small datasets and suffered catastrophic forgetting when fine-tuned with a classifier. Compared to CV, NLP models are typically more shallow and thus require different fine-tuning methods.

We propose a new method, Universal Language Model Fine-tuning (ULMFiT) that addresses these issues and enables robust inductive transfer learning for any NLP task, akin to fine-tuning ImageNet

2018 → vision
→ transfer learning
→ NLP tasks

ULMFiT



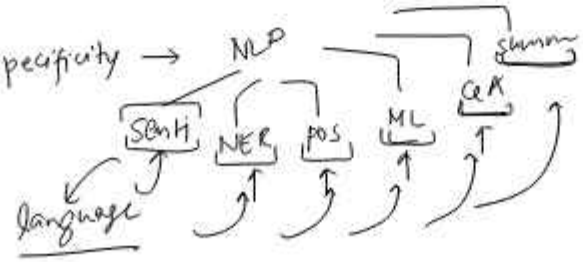
Transfer learning (TL) is a technique in which knowledge learned from a task is re-used in order to boost performance on a related task.

For example, for image classification, knowledge gained while learning to recognize cars could be applied when trying to recognize trucks.

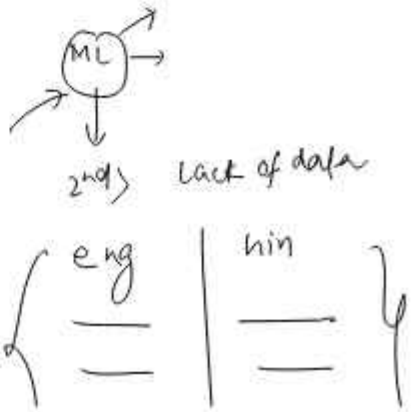
imagenet → dataset → Millions of
↳ CNN → ResNet
↳ layers
↳ pretrained
edges
↳ shapes
↓
fine-tune → cat vs dog classif → 100

2018

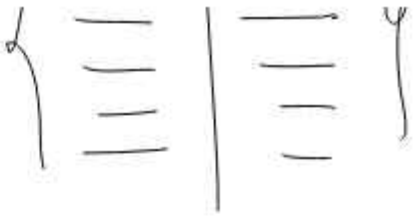
1) Task specificity → NLP



NLP → transfer learning



NLP → transfer learning



Pretraining → Machine transl X

→ Language modelling → pretraining → successful → (2)

NLP task → NLP/DL model next word pred

I live in India. and the capital is New Delhi

Language modelling as a Pretraining task

→ unsupervised pretra. task

1) Rich feature learning

The hotel was exceptionally clean, yet the service was bad / pathetic

→ known trans

↓

text class / quest. | textsum | NER / pos

mt (labels → supervised labeled

eng | nlm

→ unsupervised task

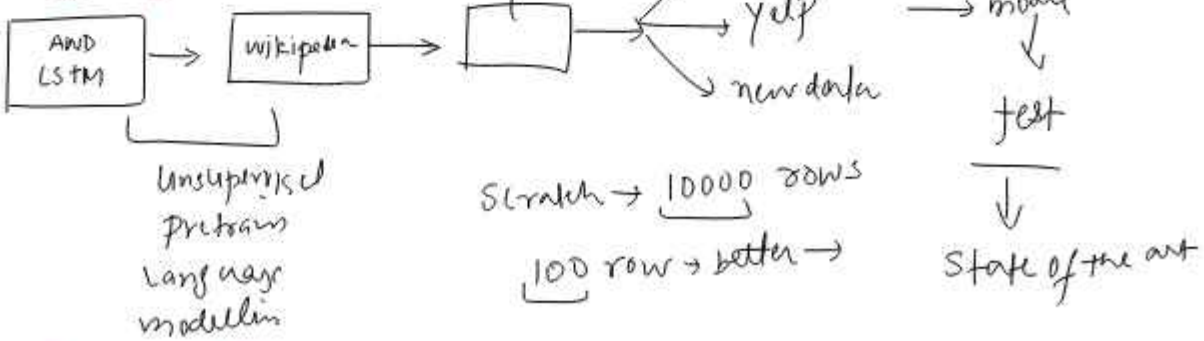
2) Huge avail of data

pdf → dataset labelling

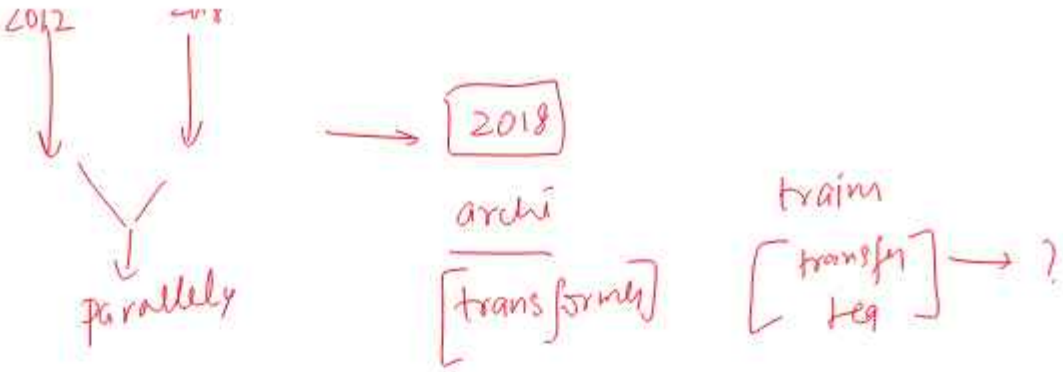
[ULMFiT]

fine tuning

X transformer

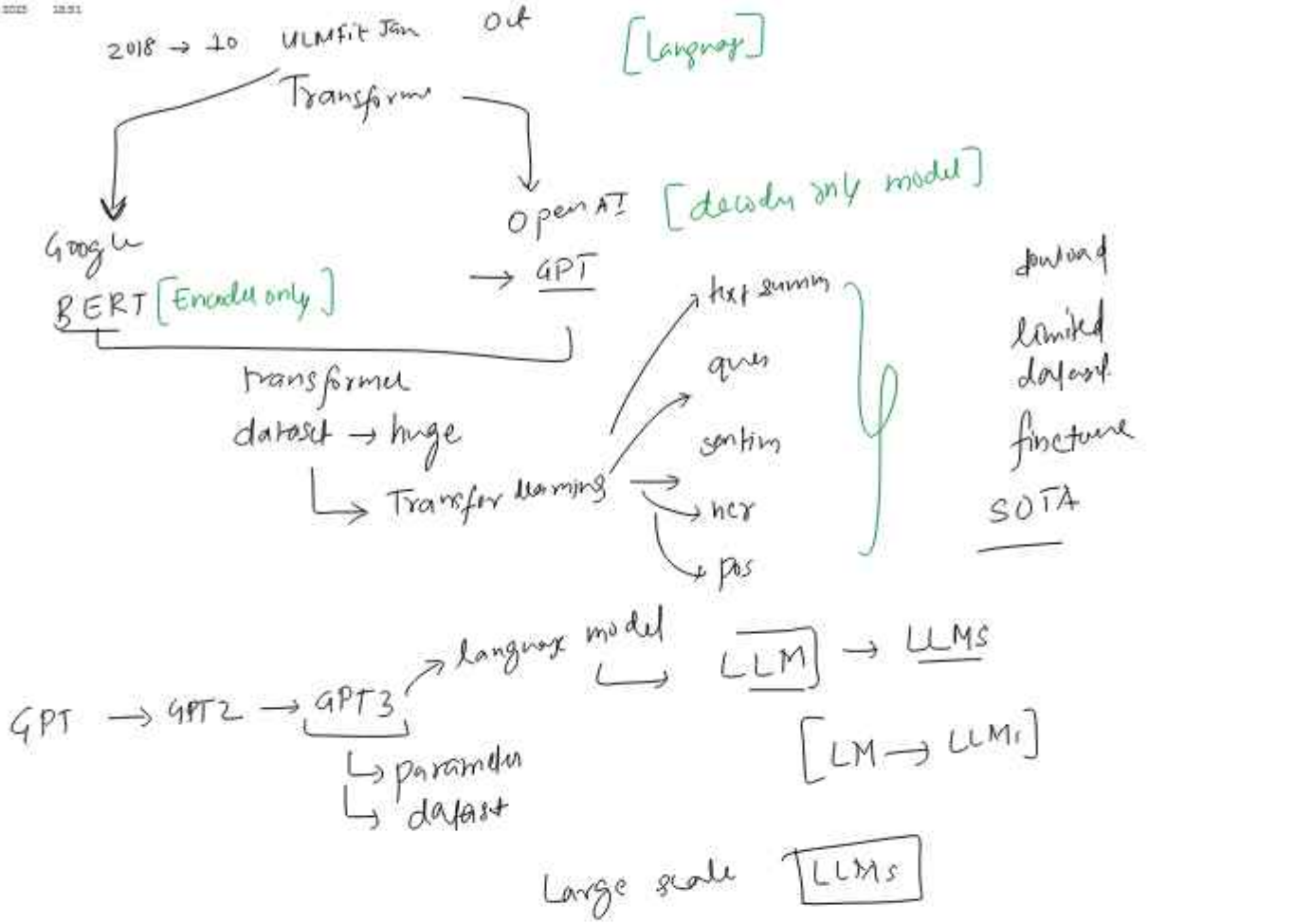


2012 2018


2012
2018
Parallelly
2018
archi
transformer
train
transfer
Eng
?

Stage 5 - LLMs

20 November 2025 18:51



Qualities of LLMs

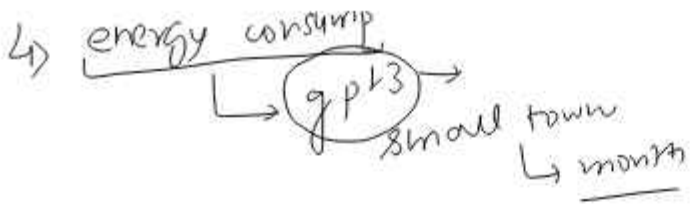
1) Data → billions → GPT3 → 45 TBs
   - ↳ book, websites, internet
   - ↳ diversity → bias

2) Hardware → Cluster of GPU → GPT3 → Supercomputer → 1000s NVIDIA GPU

3) Training → days to weeks

4) Cost → hardware + elect + infra + experts → individual
   - ↳ millions → companies, govt, institutes

4) energy consump (gpt3) → ...


energy consump
gpt3
small town
month

The Grand Finale - ChatGPT

22 November 2023 10:13

NOV

GPT3 → chatGPT

[diff]

GPT → model

[chatGPT] → application

Chat-bot

hp → chatgpt → Bard, Jasper

intel → GPT

dell

ASUS

GPT3 → [chatGPT]

1) RLHF → Reinf. Lea from human feed

- 1 Supervised Fine tuning → dataset
- 2 Reinforce → prompt produces
  - response
  - human → response rank

2) Incorporate safety and ethical guideline
- minize biases

3) improvent in contextual point

context retain → maintain context → dialogue Conver

4) Dialogue specific training
- conversation
- better unders → dialogue lang → pattern

5) ChatGPT continous imp → human feedback usu

train → refining → GPT4 → GPT3



Seq2Seq Data

08 December 2022 17:12

cgpa | iq | placement

DNN

1) Tabular → [ANN] ×

Encoder Decoder Arch

3) Sequential → textual
→ timeseries

RNN

LSTMs GRUs

2) Image data → → CNN <

dog / cat

[2d grid]

4) Seq2seq data diffficult

[input seq → output seq]

machine translation

nice to meet you → आप से मिल कर अच्छा लगा

input → sent → eng → variable length

output → sent → hindi → variable len

→ variable length

[lstm / gru] → input

[output]

3 words → 3 words

eng

Before Starting

08 December 2023 19:24

Prerequisite

- RNN / LSTM

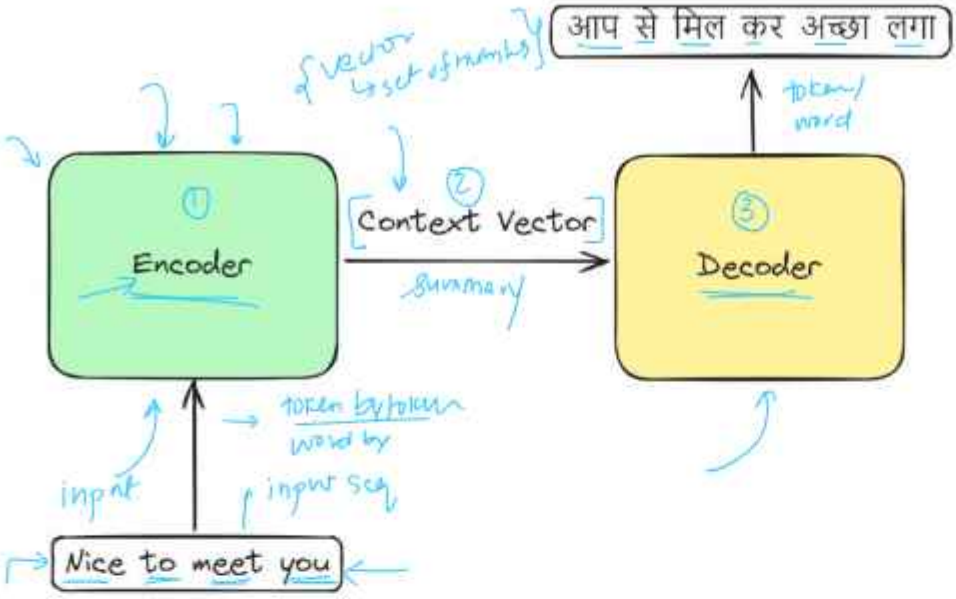
Plan of attack

→ simple version
→ deep
→ improvements

High Level Overview

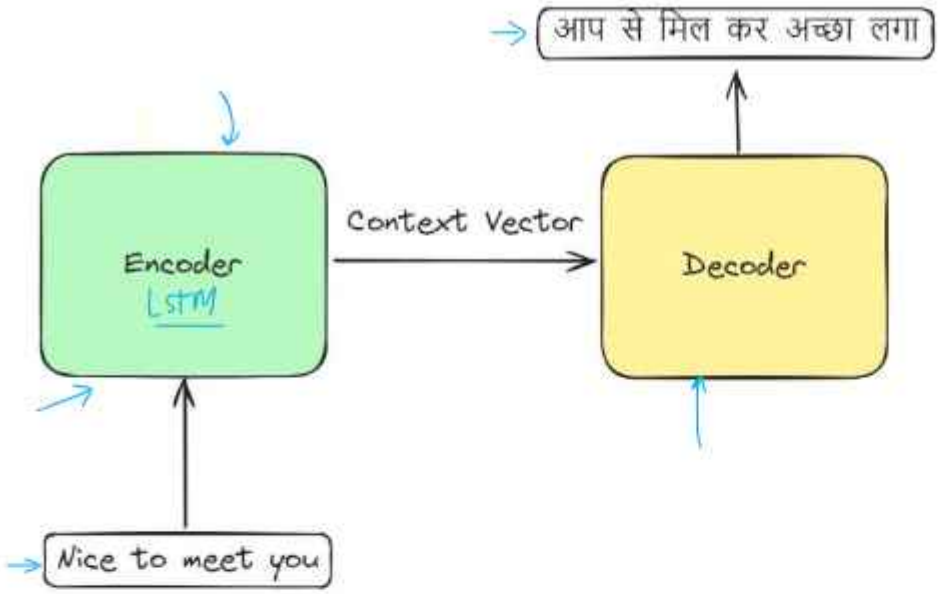
08 December 2023 17:14

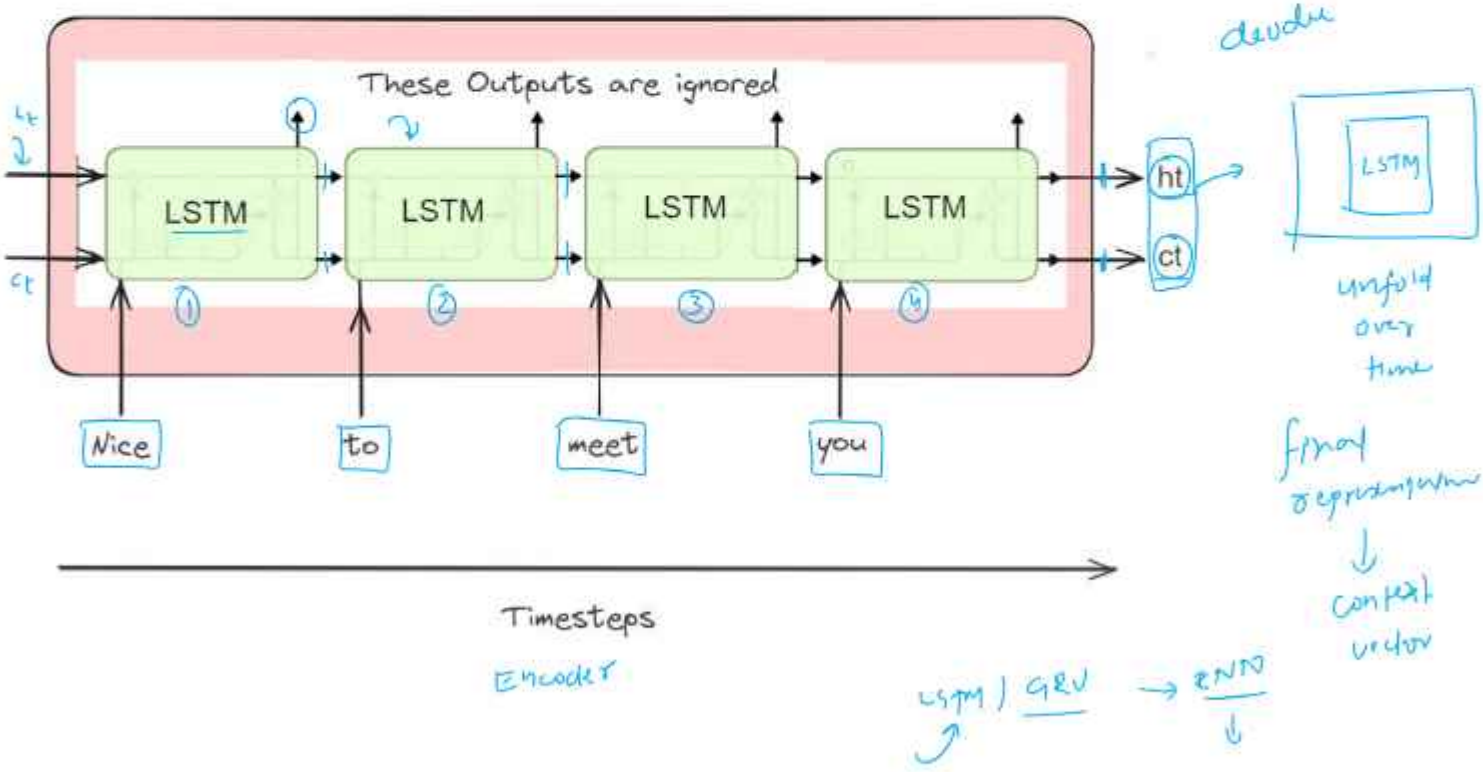
Machine tran → exam

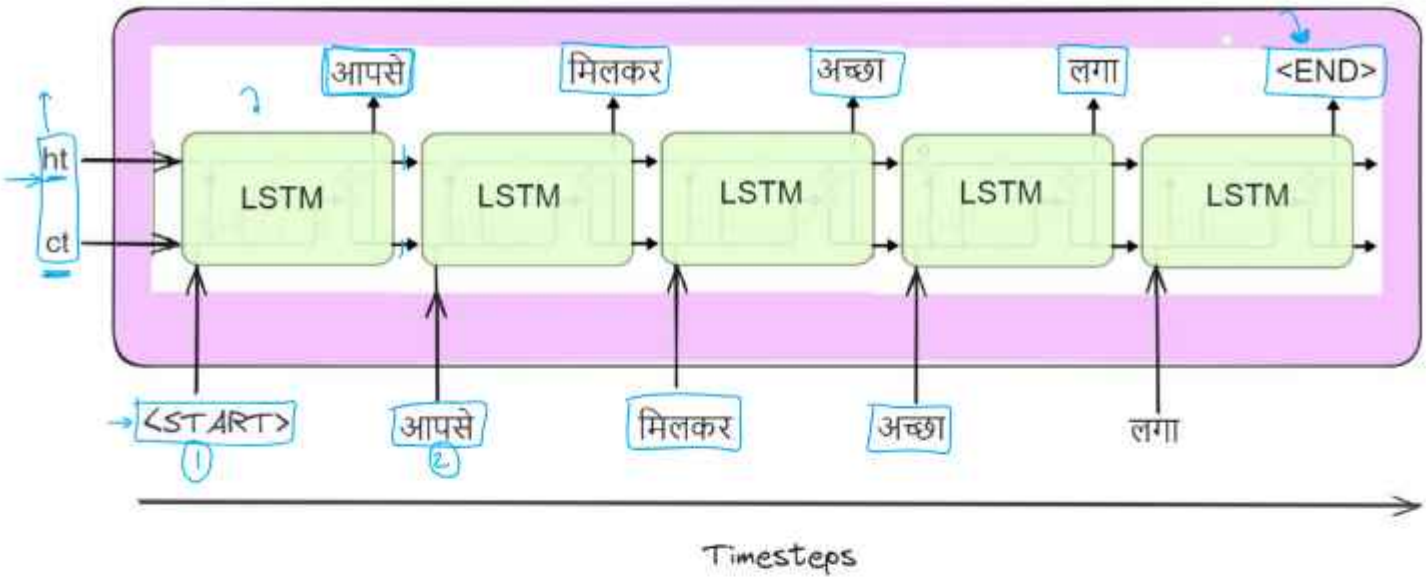


What's under the hood?

08 December 2023 17:14

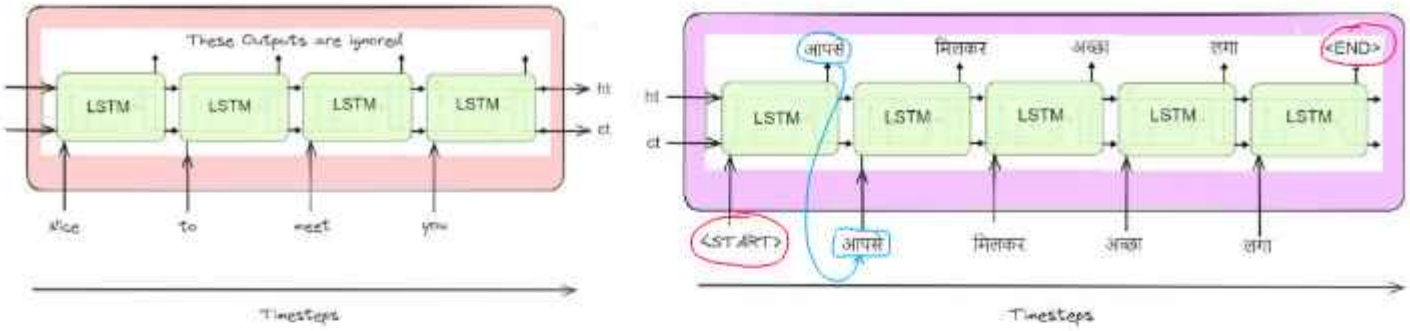


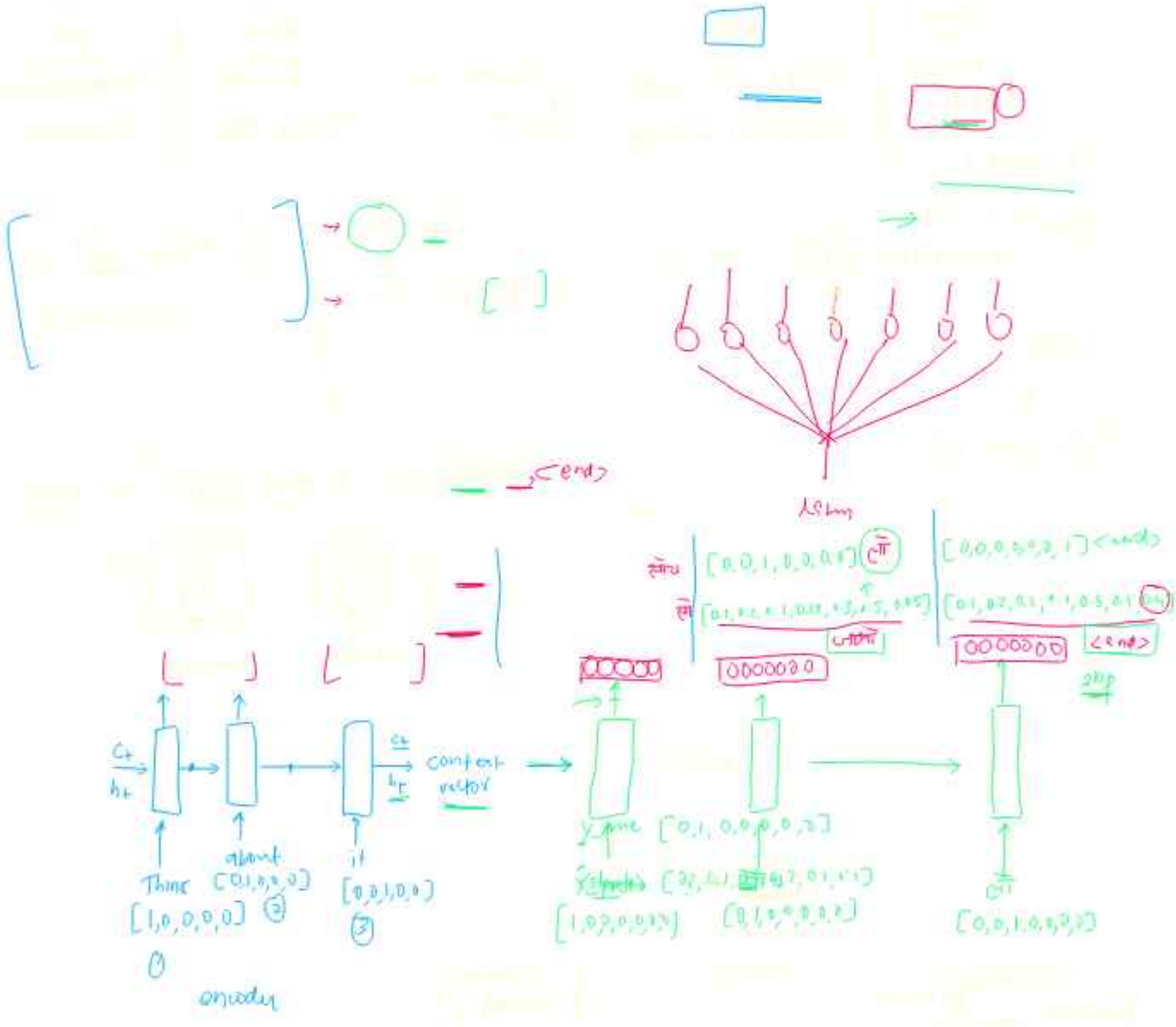



ht
ct
LSTM
LSTM
LSTM
LSTM
LSTM
आपसे
मिलकर
अच्छा
लगा
<END>
<START>
आपसे
मिलकर
अच्छा
लगा
Timesteps

Training the Architecture using Backpropagation

08 December 2023 17:13

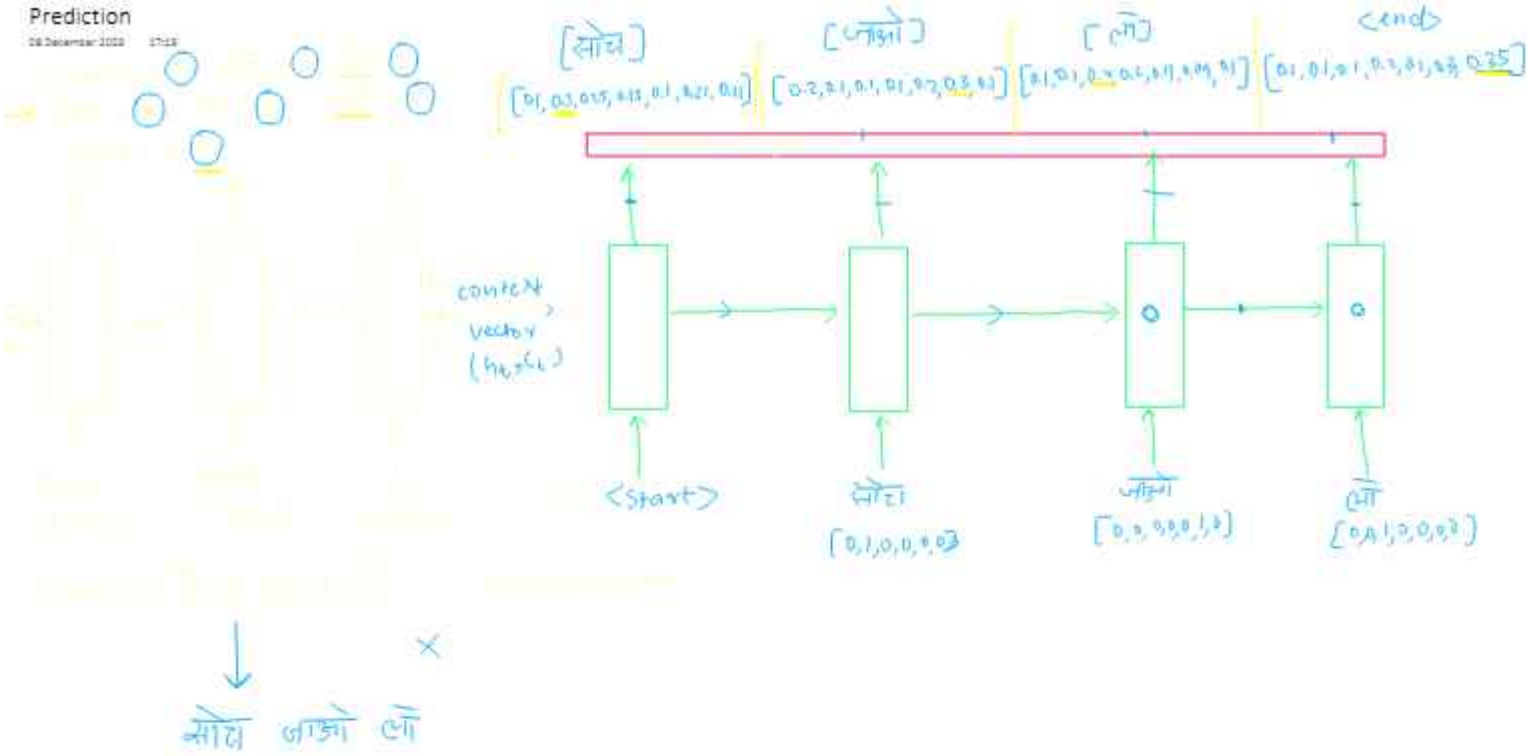






Prediction

08 December 2022 07:13

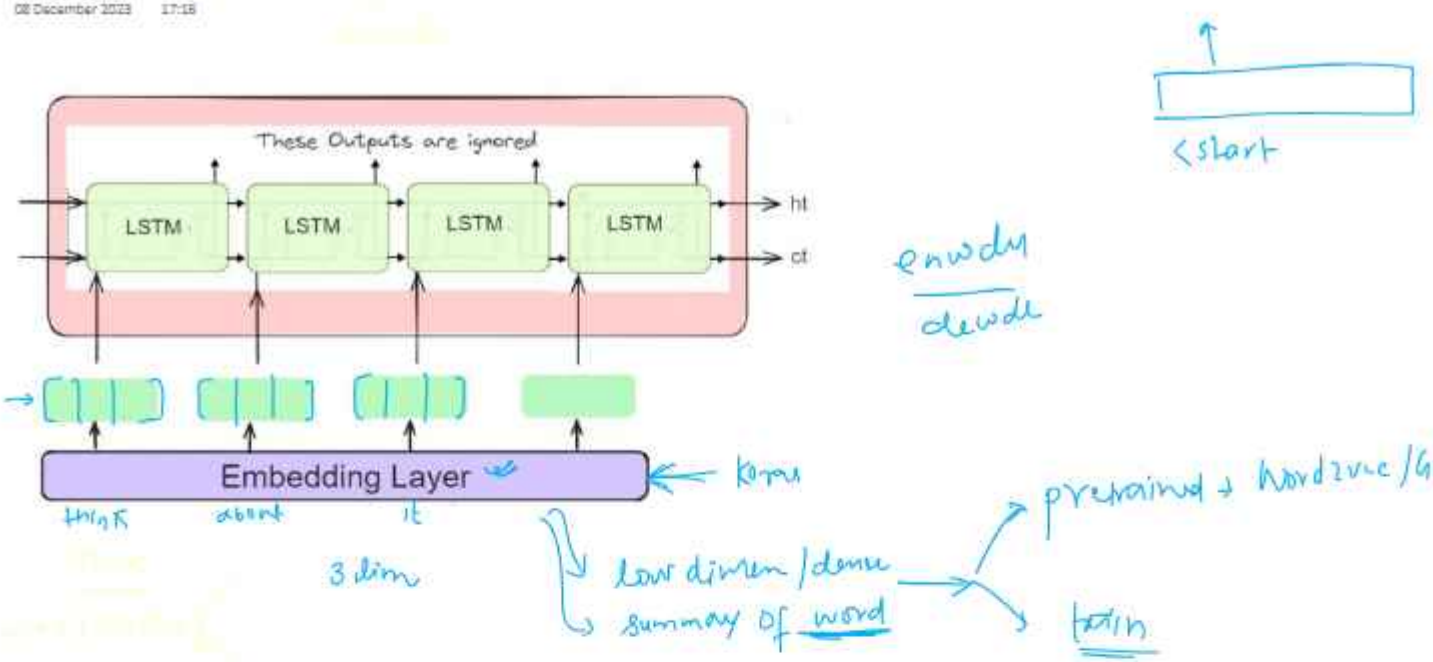


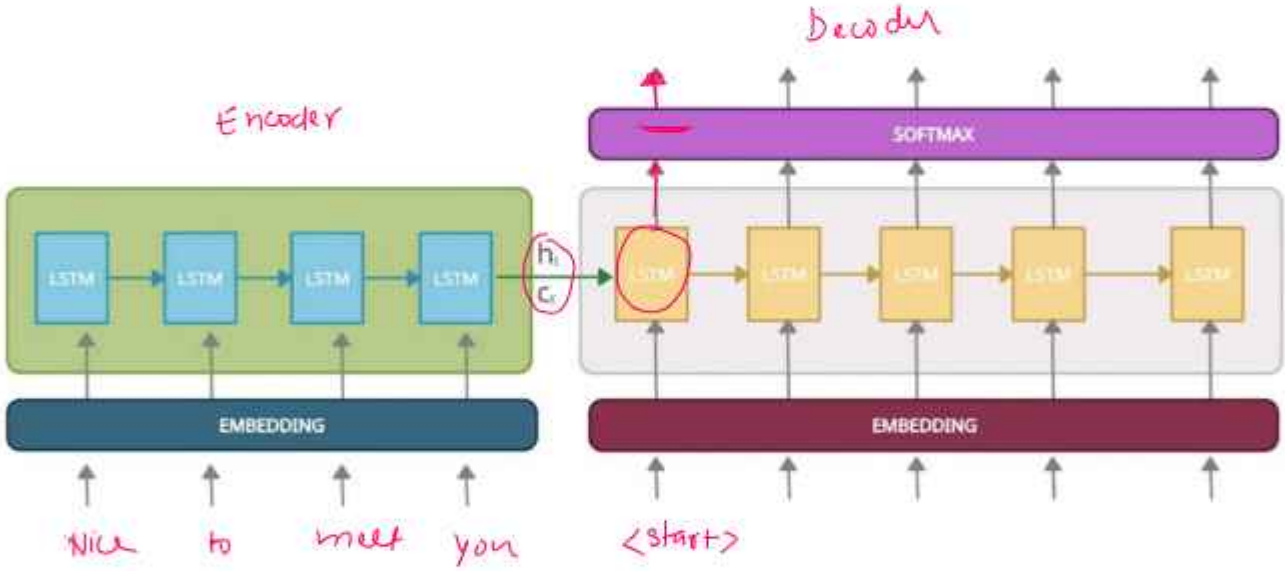
↓

소리 나라이 이

Improvement 1 - Embeddings

08 December 2023 17:18

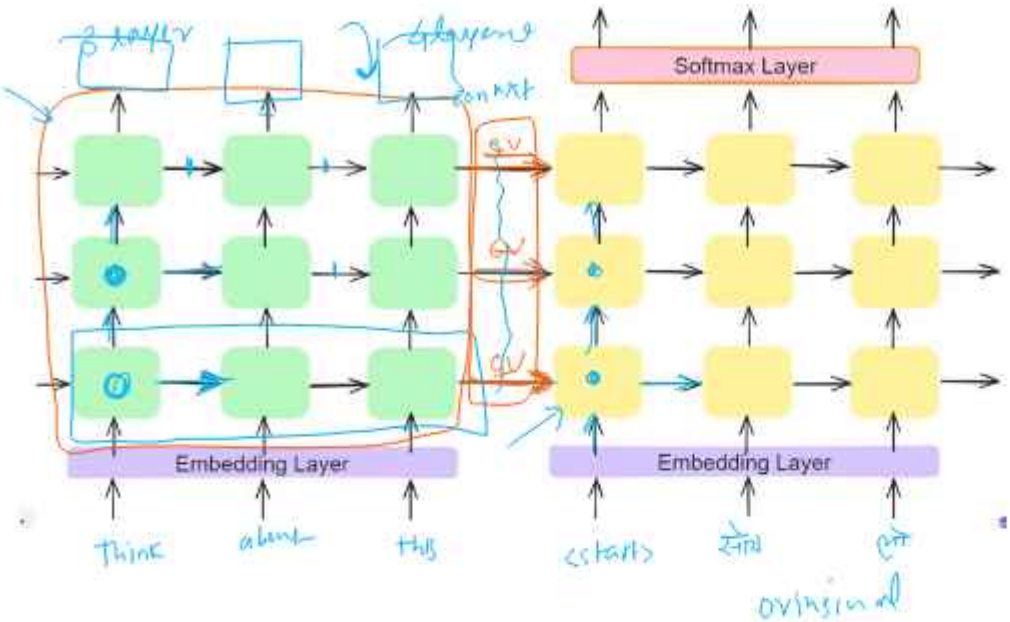




Improvement 2 - Deep LSTMs

08 December 2023 17:16

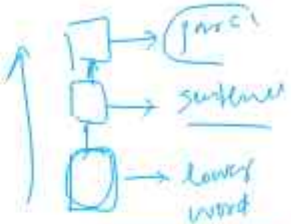
long para single layer



3) deep learning NN learning

1) long term dependency

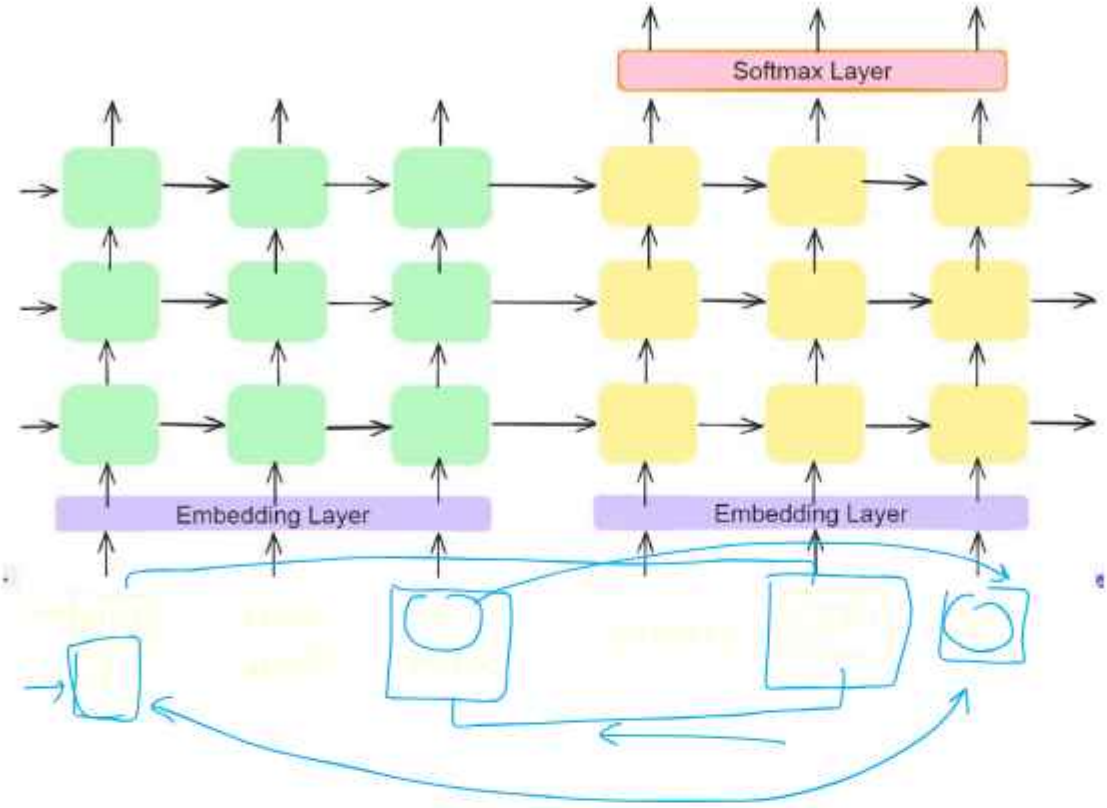
2) layered representation



the phone battery is bad but the overphone is great

Improvement 3 - Reversing the Input

08 December 2023 17:16



language
↓
'initial context
eng - french

The Sutskever Architecture

08 December 2023 17:18

Application to Translation: The model focused on translating English to French, demonstrating the effectiveness of sequence-to-sequence learning in neural machine translation.

Special End-of-Sentence Symbol: Each sentence in the dataset was terminated with a unique end-of-sentence symbol ("<EOS>"), enabling the model to recognize the end of a sequence.

Dataset: The model was trained on a subset of 12 million sentences, comprising 348 million French words and 304 million English words, taken from a publicly available dataset.

Vocabulary Limitation: To manage computational complexity, fixed vocabularies for both languages were used, with 160,000 most frequent words for English and 80,000 for French. Words not in these vocabularies were replaced with a special "UNK" token.

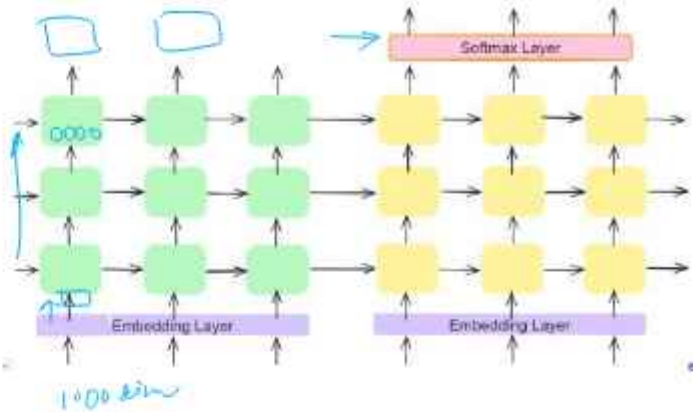
Reversing Input Sequences: The input sentences (English) were reversed before feeding them into the model, which was found to significantly improve the model's learning efficiency, especially for longer sentences.

Word Embeddings: The model used a 1000-dimensional word embedding layer to represent input words, providing dense, meaningful representations of each word.

Architecture Details: Both the input (encoder) and output (decoder) models had 4 layers, with each layer containing 1,000 units, showcasing a deep LSTM-based architecture.

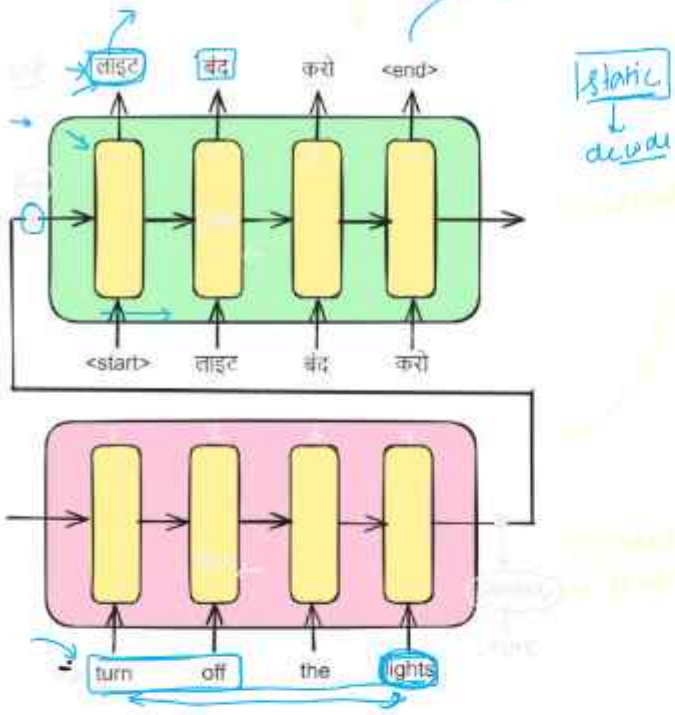
Output Layer and Training: The output layer employed a Softmax function to generate the probability distribution over the target vocabulary. The model was trained end-to-end with these settings.

Performance - BLEU Score: The model achieved a BLEU score of 34.81, surpassing the baseline Statistical Machine Translation (SMT) system's score of 33.30 on the same dataset, marking a significant advancement in neural machine translation.



The Why

10 December 2023 13:55



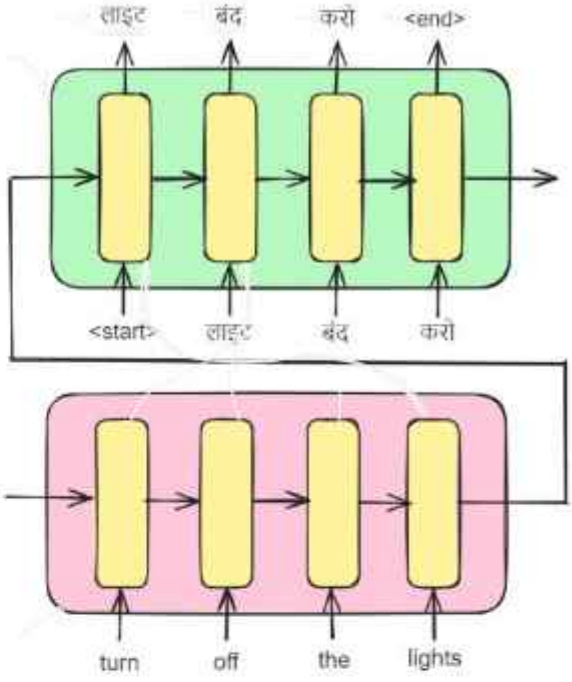
Once upon a time in a small Indian village, a mischievous monkey stole a turban from a sleeping barber, wore it to a wedding, danced with the bewildered guests, accidentally got crowned the 'Banana King' by the local kids, and ended up leading a vibrant, impromptu parade of laughing villagers, cows, and street dogs, all while balancing a stack of mangoes on its head, creating a hilariously unforgettable spectacle and an amusing legend that the village still chuckles about every monsoon season.

The Solution

20 December 2023 17:32

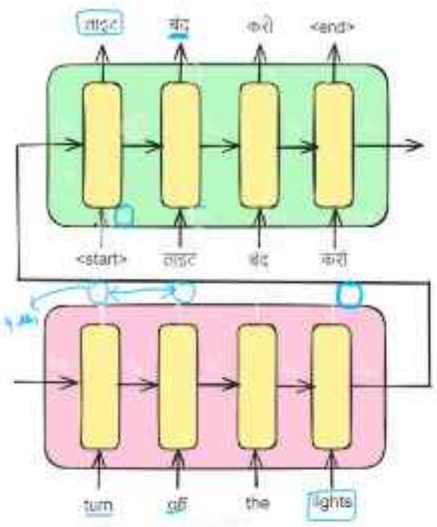


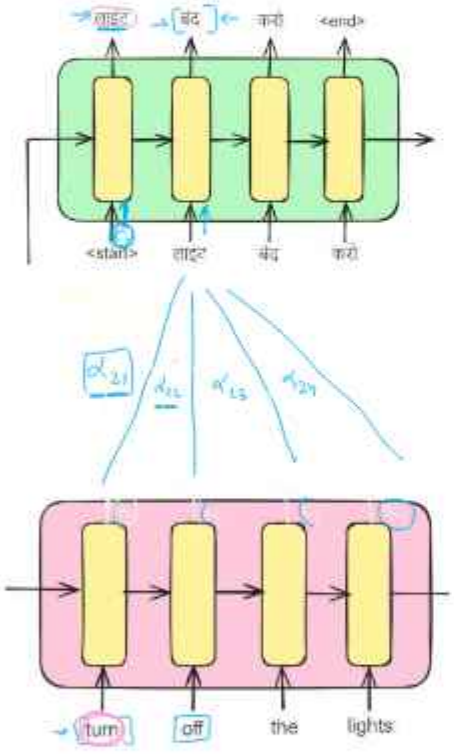
Once upon a time in a small Indian village, a mischievous monkey stole a turban from a sleeping barber, wore it to a wedding, danced with the bewildered guests, accidentally got crowned the 'Banana King' by the local kids, and ended up leading a vibrant, impromptu parade of laughing villagers, cows, and street dogs, all while balancing a stack of mangoes on its head, creating a hilariously unforgettable spectacle and an amusing legend that the village still chuckles about every monsoon season.

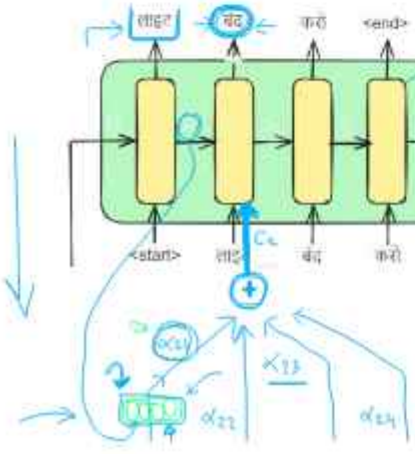


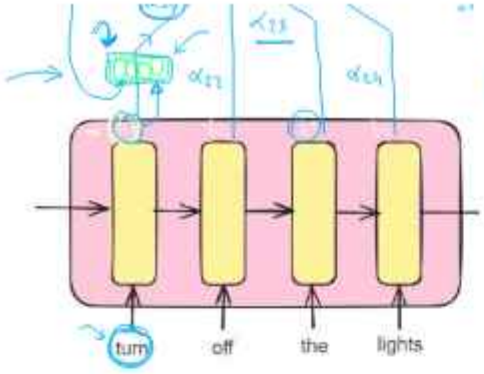
The What

11 December 2018 08:04



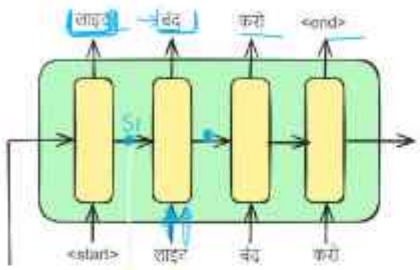




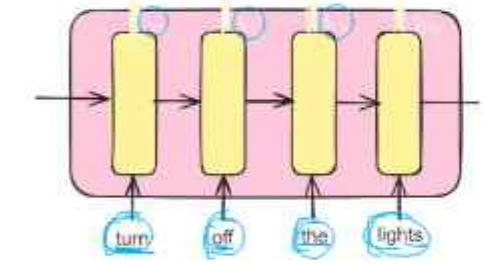


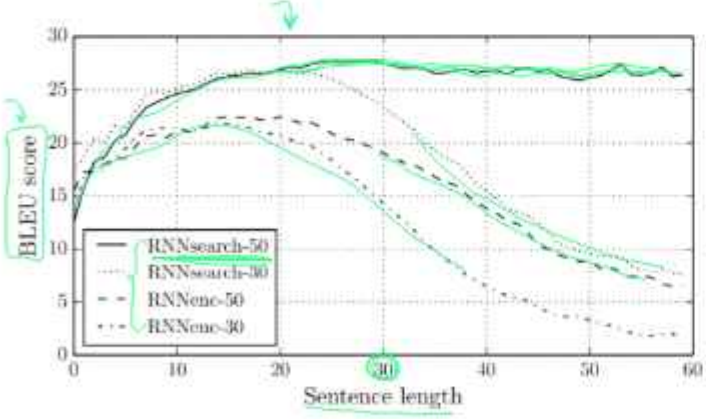
$\alpha_{21} \rightarrow f(h_1, s_1)$

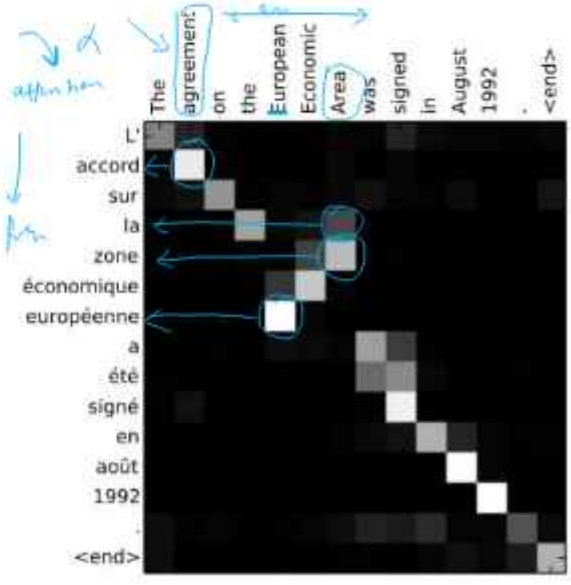
$\alpha_{ij} = f(h_j, s_{i-1})$



$c_2 = \alpha_{21} h_1 + \alpha_{22} h_2 + \alpha_{23} h_3 + \alpha_{24} h_4$

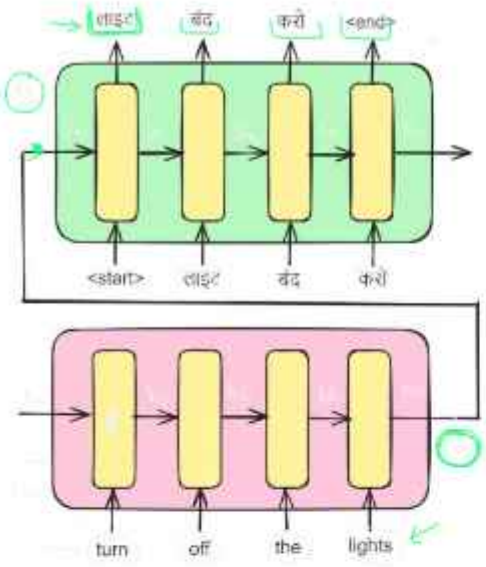






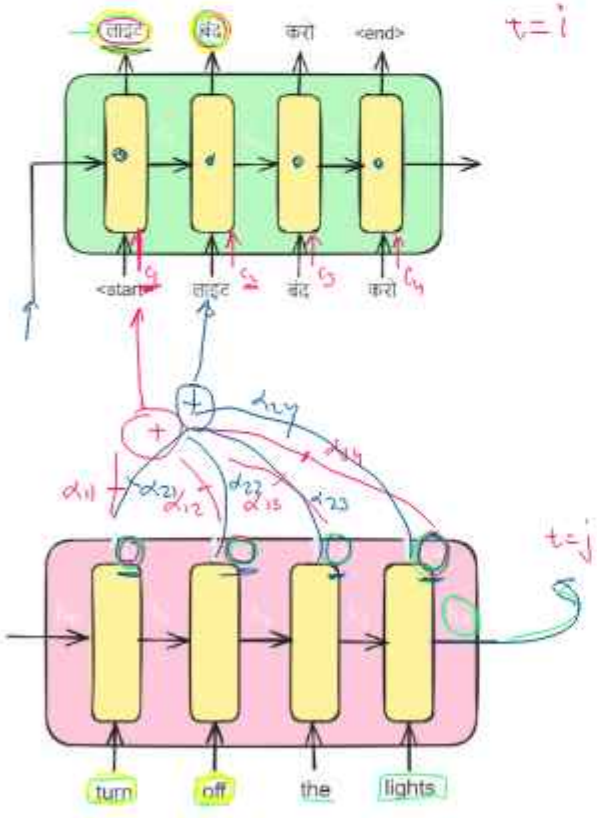
(a)

Recap



sentence > 30 words
paragraph
document

translation bottleneck → Attention mechanism



$t = i$

$c_1\ c_2\ c_3\ c_4$

$4 \times 4 = 16$

weighted sum

$$c_i = \sum_{j=1}^{4} \alpha_{ij} h_j$$

$\alpha \rightarrow$ alignment score

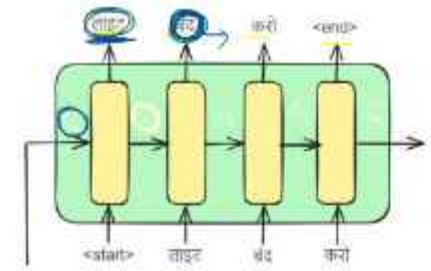
$$c_1 = \alpha_{11} h_1 + \alpha_{12} h_2 + \alpha_{13} h_3 + \alpha_{14} h_4$$

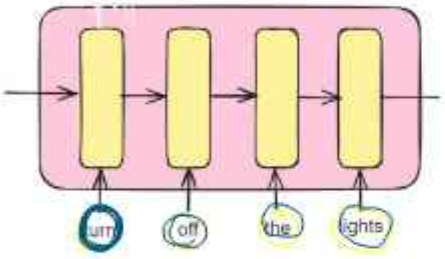
$$c_2 = \alpha_{21} h_1 + \alpha_{22} h_2 + \alpha_{23} h_3 + \alpha_{24} h_4$$

$t = j$

Bahdanau Attention

18 January 2024 18:11





given

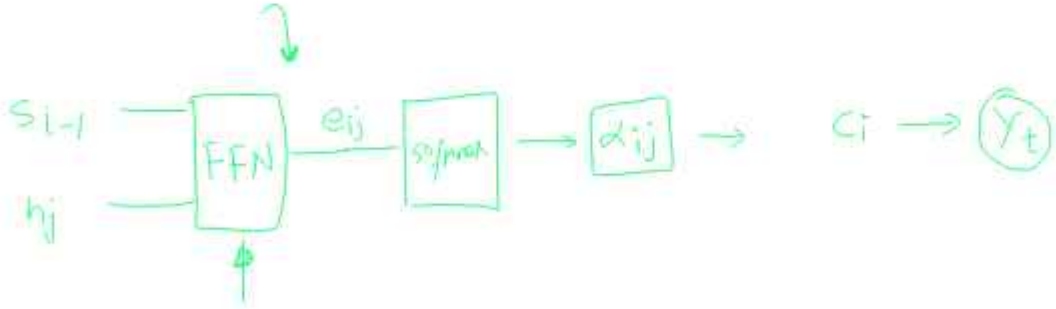
α

α_{ij} = approximate

FFN → ANN → dFA

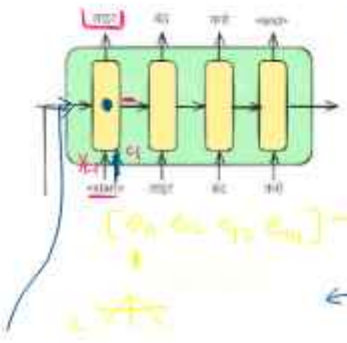
α_{ij} ← softmax ← e_{ij}

s_{i-1}, h_j



s_0 y_{t-1} c_1 → LSTM → y_t (output) s_1

$i=1$ $t=2$

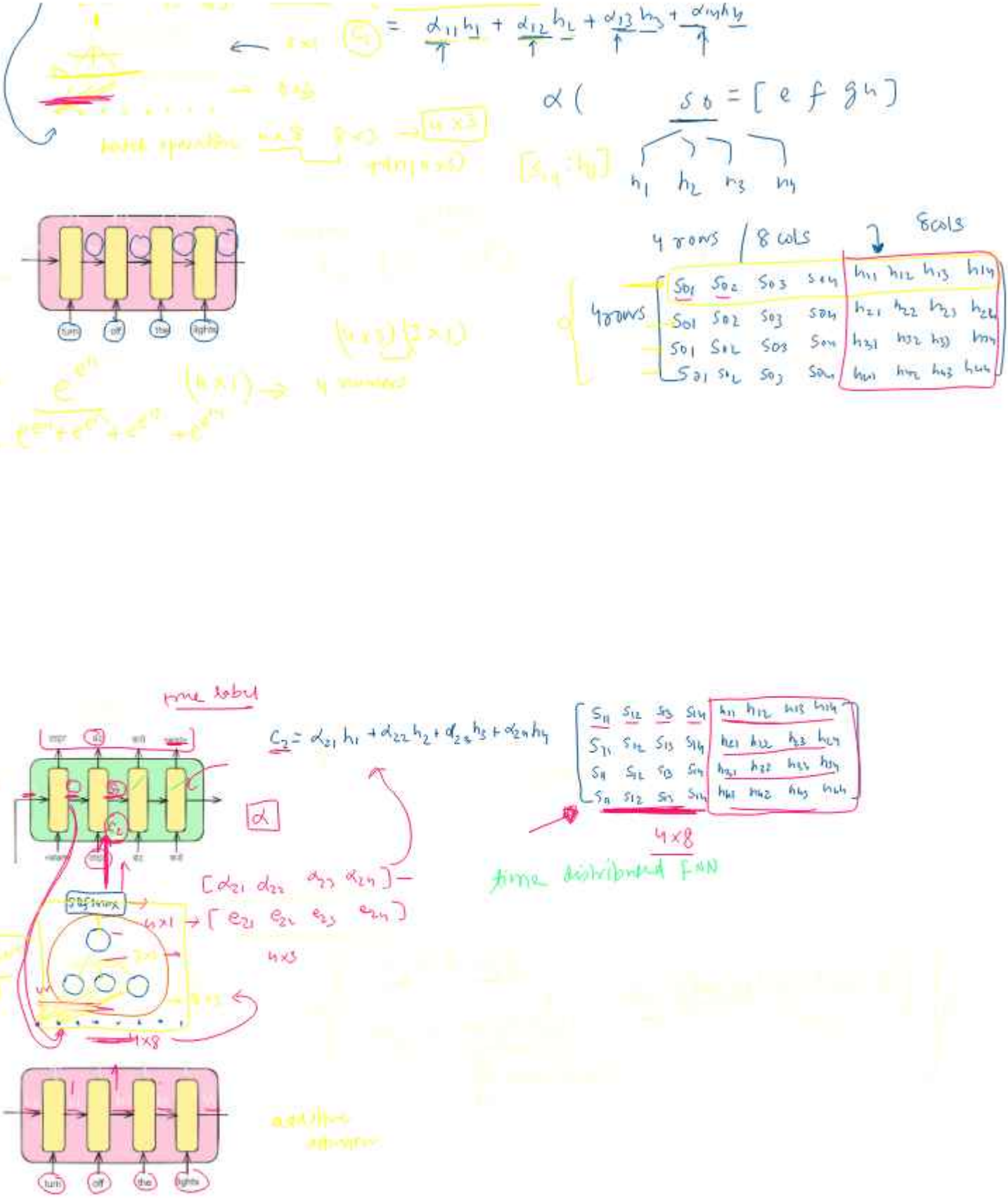


h_1 h_2 h_3 h_4

s_{i-1} h_j

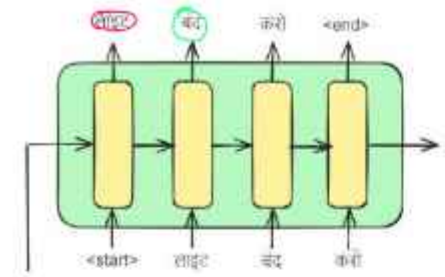
$c_1 = \sum \alpha_{ij} h_j$

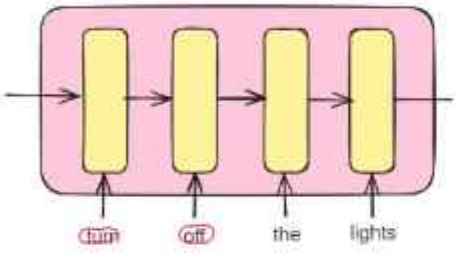
$= \alpha_{11} h_1 + \alpha_{12} h_2 + \alpha_{13} h_3 + \alpha_{14} h_4$

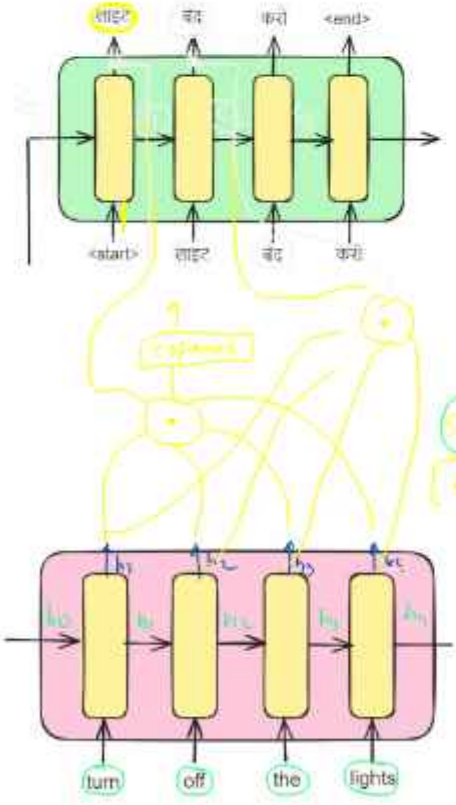

turn
off
the
lights
time distributed FNN
4x8
4x3
4x1
softmax

Luong Attention

17 January 2024 00:09



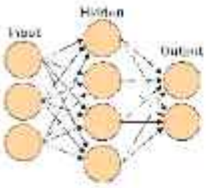




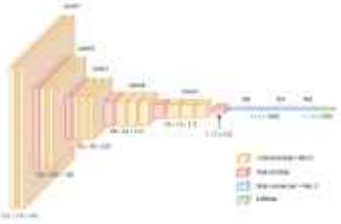
What is Transformer?

27 January 2024 18:41

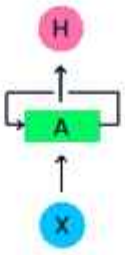
→ self attention scalable



ANN
tabular



CNN
Image

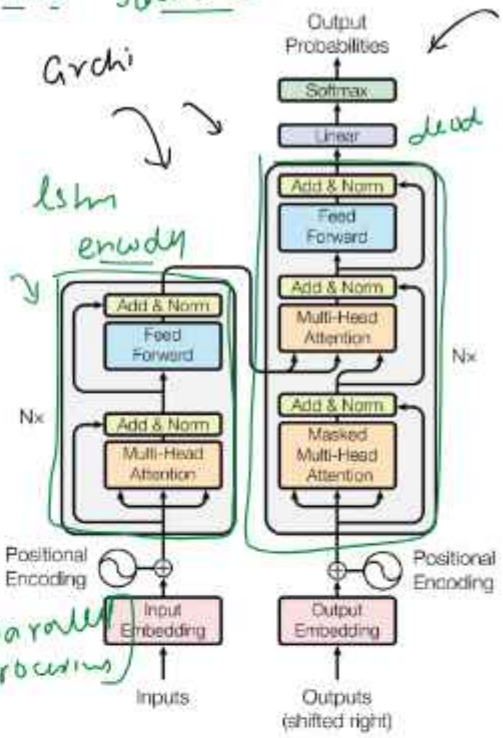


RNN
sequence
↳ text

Archi

lstm

encody



decod

parallel processing

→ [transformer] → seq2seq task

NN arch

- machine translation
- question ans
- text summary

Attention Is All You Need

2017

↳ Google Brain

Deep learni

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Lukasz Kaiser* Google Brain lukaszkaiser@google.com

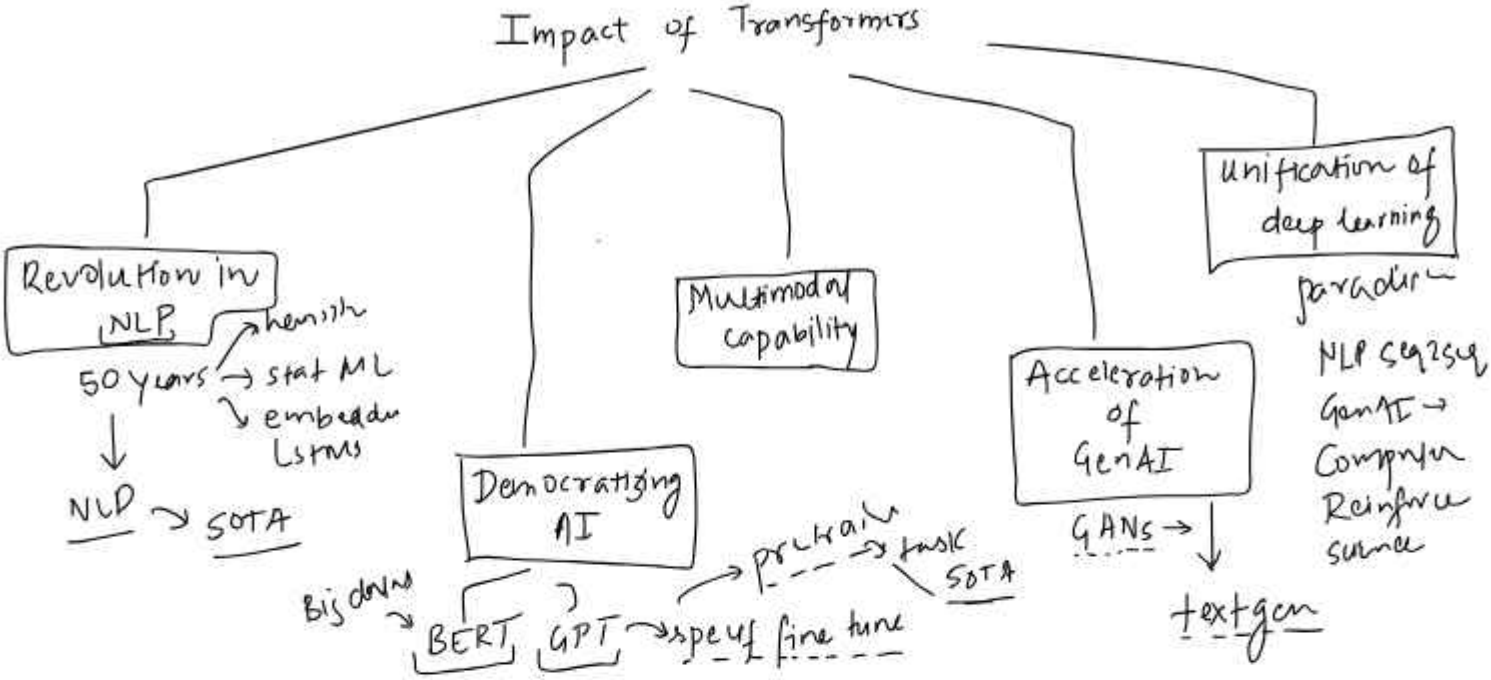
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Impact of Transformers

27 January 2024 20:03



The Origin Story!

27 January 2024 22:38

2014-15 → seq2seq machine

Sequence to Sequence Learning with Neural Networks

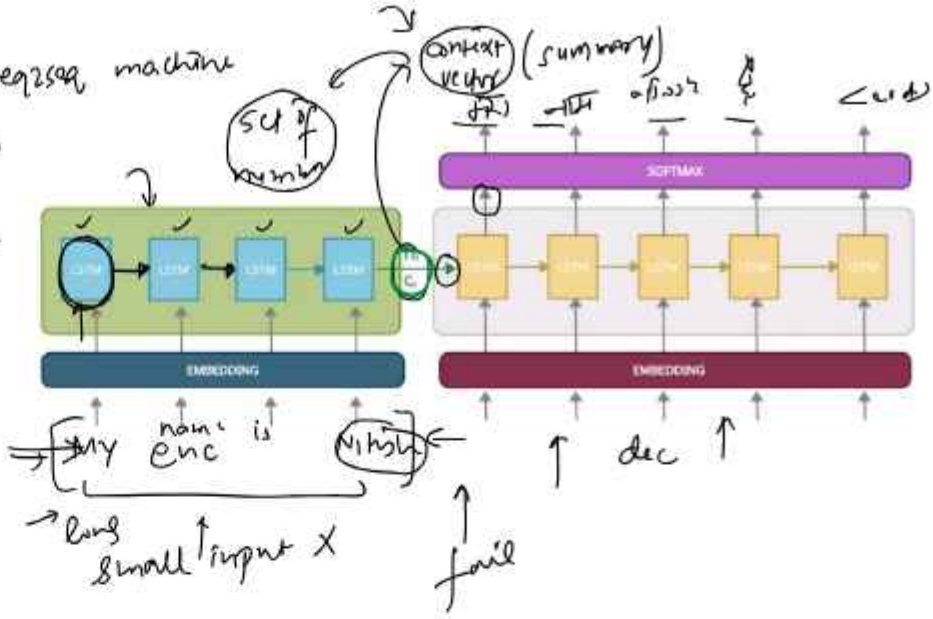
Ilya Sutskever
Google
ilyasu@google.com

Oriol Vinyals
Google
vinyals@google.com

Quoc V. Le
Google
qvl@google.com

Abstract

Deep Neural Networks (DNNs) are powerful models that have achieved excellent performance on difficult learning tasks. Although DNNs work well whenever large labeled training sets are available, they cannot be used to map sequences to sequences. In this paper, we present a general end-to-end approach to sequence learning that makes minimal assumptions on the sequence structure. Our method uses a multilayered Long Short-Term Memory (LSTM) to map the input sequence to a vector of a fixed dimensionality, and then another deep LSTM to decode the target sequence from the vector. Our main result is that on an English to French translation task from the WMT'14 dataset, the translations produced by the LSTM achieve a BLEU score of 34.8 on the entire test set, where the LSTM's BLEU score was penalized on out-of-vocabulary words. Additionally, the LSTM did not have difficulty on long sentences. For comparison, a phrase-based SMT system achieves a BLEU score of 33.3 on the same dataset. When we used the LSTM to rerank the 1000 hypotheses produced by the aforementioned SMT system, its BLEU score increases to 36.5, which is close to the previous best result on this task. The LSTM also learned sensible phrase and sentence representations that are sensitive to word order and are relatively invariant to the active and the passive voice. Finally, we found that reversing the order of the words in all source sentences (but not target sentences) improved the LSTM's performance markedly, because doing so introduced many short term dependencies between the source and the target sentence which made the optimization problem easier.



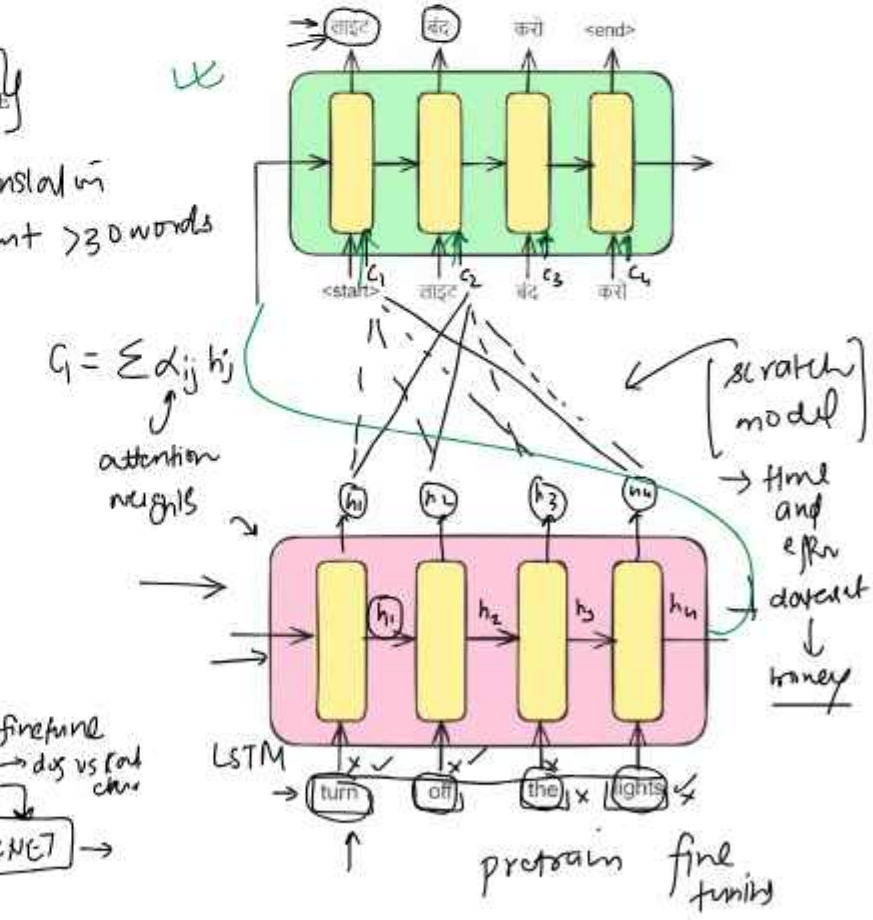
Neural Machine Translation by Jointly Learning to Align and Translate

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho Yoshua Bengio*
Université de Montréal

Abstract

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.



sequential training → slow → huge dataset

Transfer learning → CNN → finetune → dog vs cat → ImageNET

scratch model → time and effort → dataset → money

research → incremental → time travel → stream

→ RNN/self → parallely
→ stable
→ hyperfine

NLP → BERT / GPT → fine tune

Attention Is All You Need → 2017

Attention Is All You Need

Ashish Vaswani[*]
Google Brain
avaswani@google.com

Noam Shazeer[*]
Google Brain
noam@google.com

Niki Parmar[*]
Google Research
nikip@google.com

Jakob Uszkoreit[*]
Google Research
usz@google.com

Llion Jones[*]
Google Research
llion@google.com

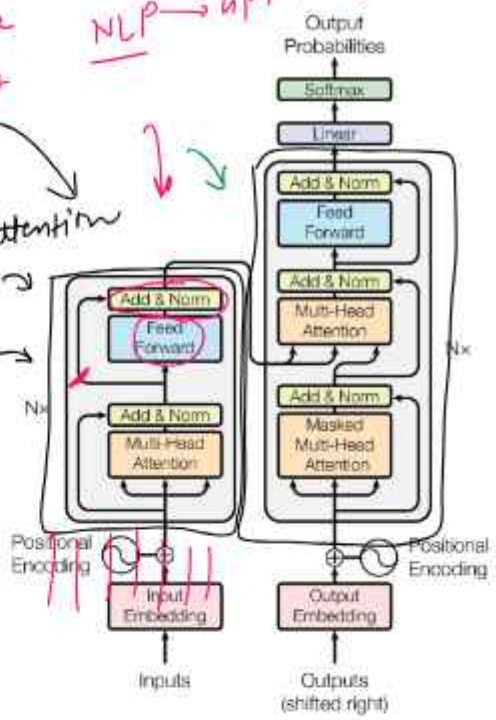
Aidan N. Gomez[*] [†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser[*]
Google Brain
lukaszkaiser@google.com

Illia Polosukhin[*] [‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.



The Timeline

28 January 2024 00:55

2000 - 2014 → RNNs / LSTMs



2014 → Attention



2017 → Transformer

2018 → BERT / GPT (Transfer learning)

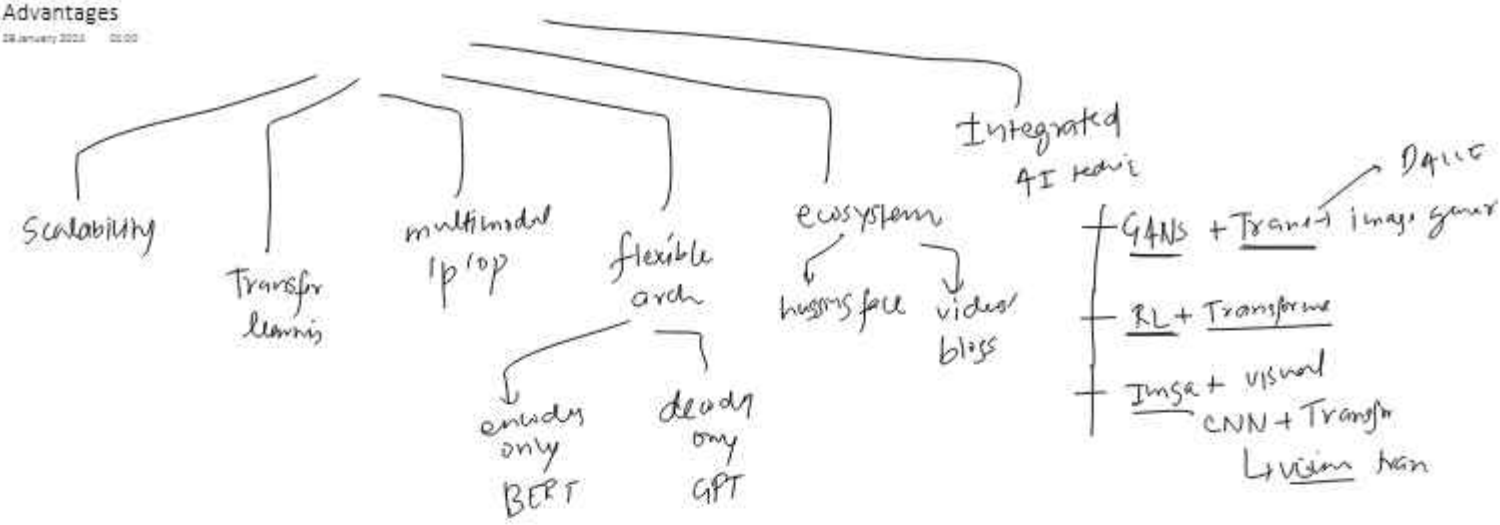
2018 - 2020 → Vision Transfor / Alphafold-2

2021 → Gen AI

2022 → ChatGPT / Stable Diffusion

Advantages

28 January 2023 01:00

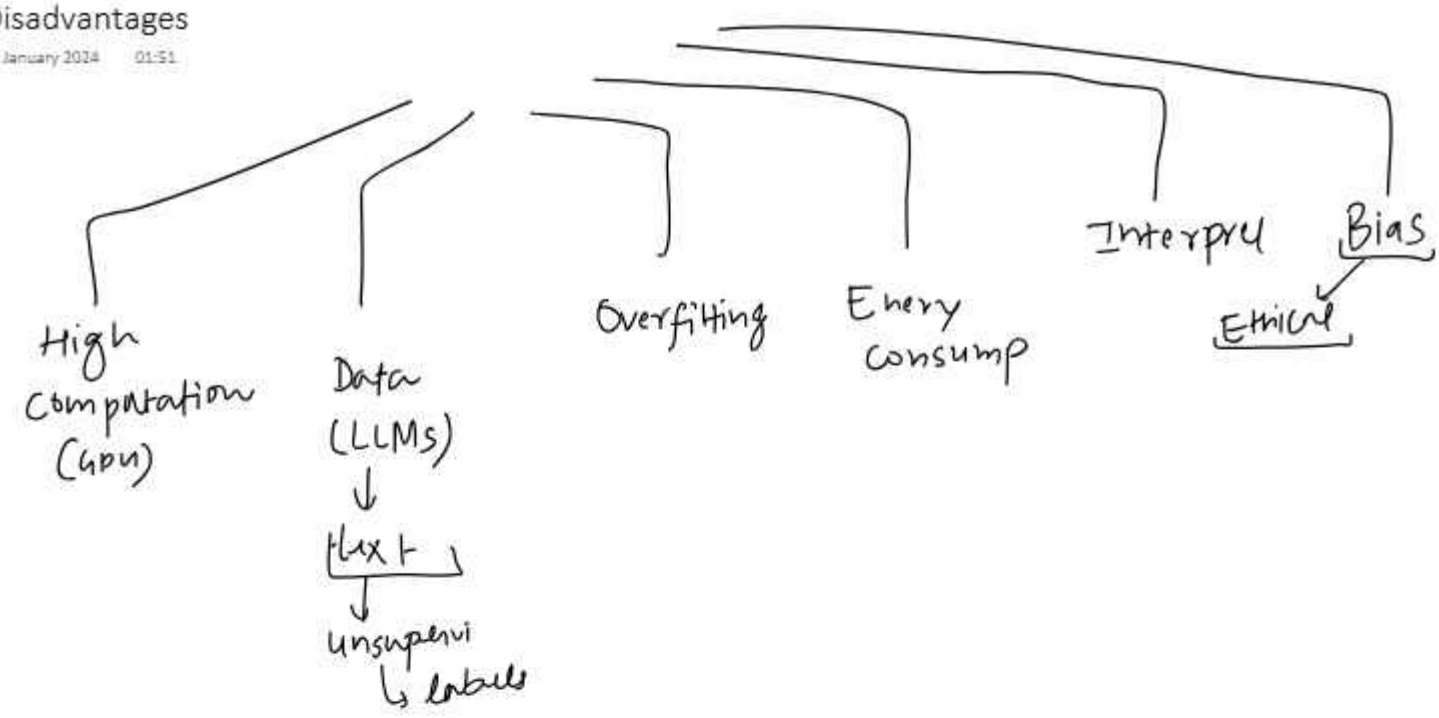


Famous Applications

28 January 2024 01:17

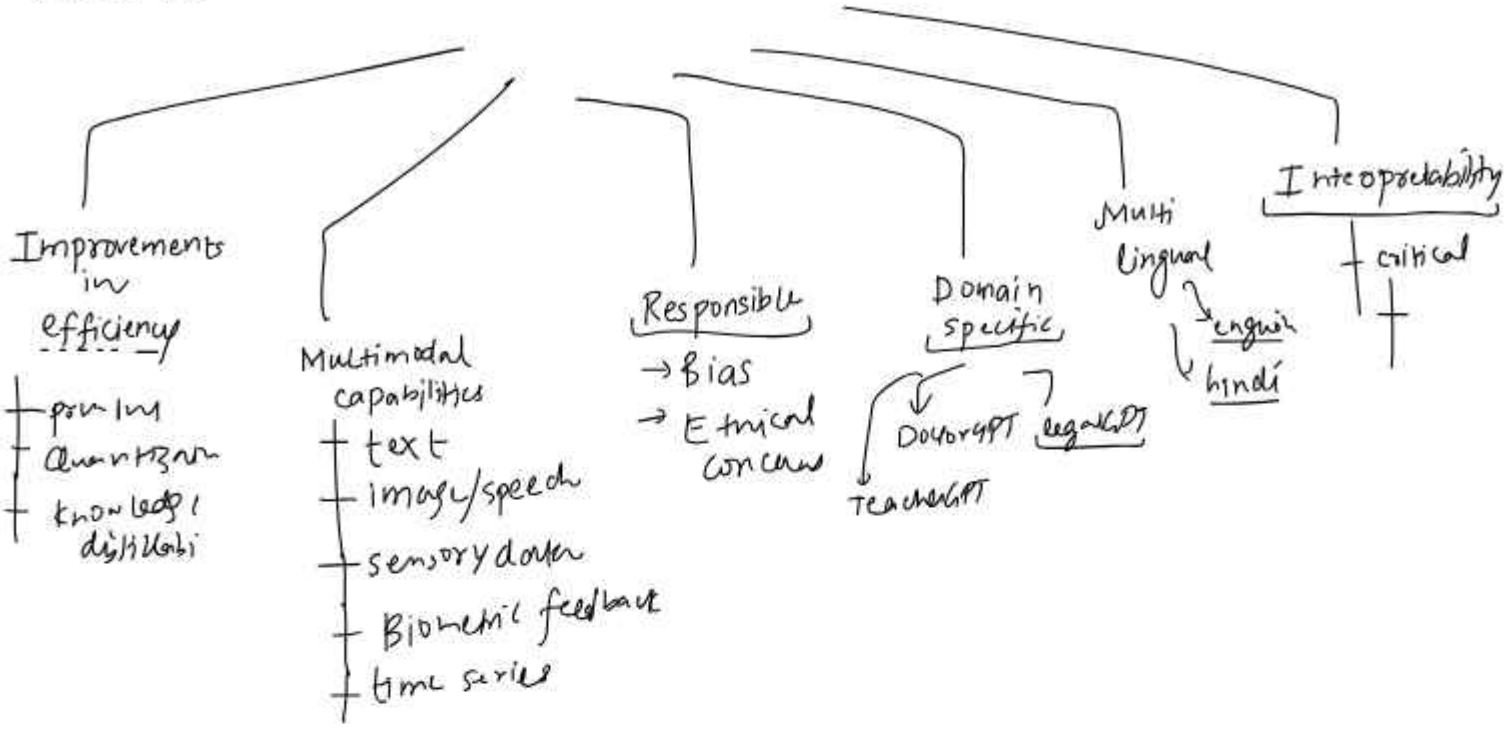
Disadvantages

28 January 2024 01:51



Future

28 January 2024 02:09



Self attention computer ✗

NLP → words → numbers → vectorization

OHE

1) mat cat mat
2) cat rat rat

	mat	cat	rat
mat	1	0	0
cat	0	1	0
rat	0	0	1

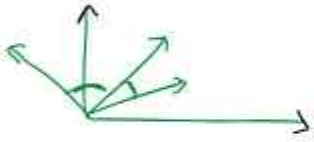
num → [1 0 0] [0 1 0] [1 0 0]

BOW

	mat	rat	cat
S1	[2	0	1]
S2	→ [0	2	1]

Tf Idf → word embedding → number → Semantic meaning

training wiki → 5 dim



Cricket [0.2 0.9 0.5 0.90 ... n-dim vector] 256, 512, 64

king → [0.9 0.1 1 0 0.9]

queen → [0.9 0.2 0.4 1 0]

similar (king, queen) → similar

royalty

Apple → vector

meaning → vector

The problem of "Average Meaning"

dataset

1) An apple a day keeps the doctor away
2) Apple is healthy
3) Apple is better than orange
4) Apple makes great phones

10000 words
9000 fruit
1000 phone

9000 tea 1000 fruit

[x y]

taste technology

2 dim

[0.8 0.2] → [0.9 0.3]

tea

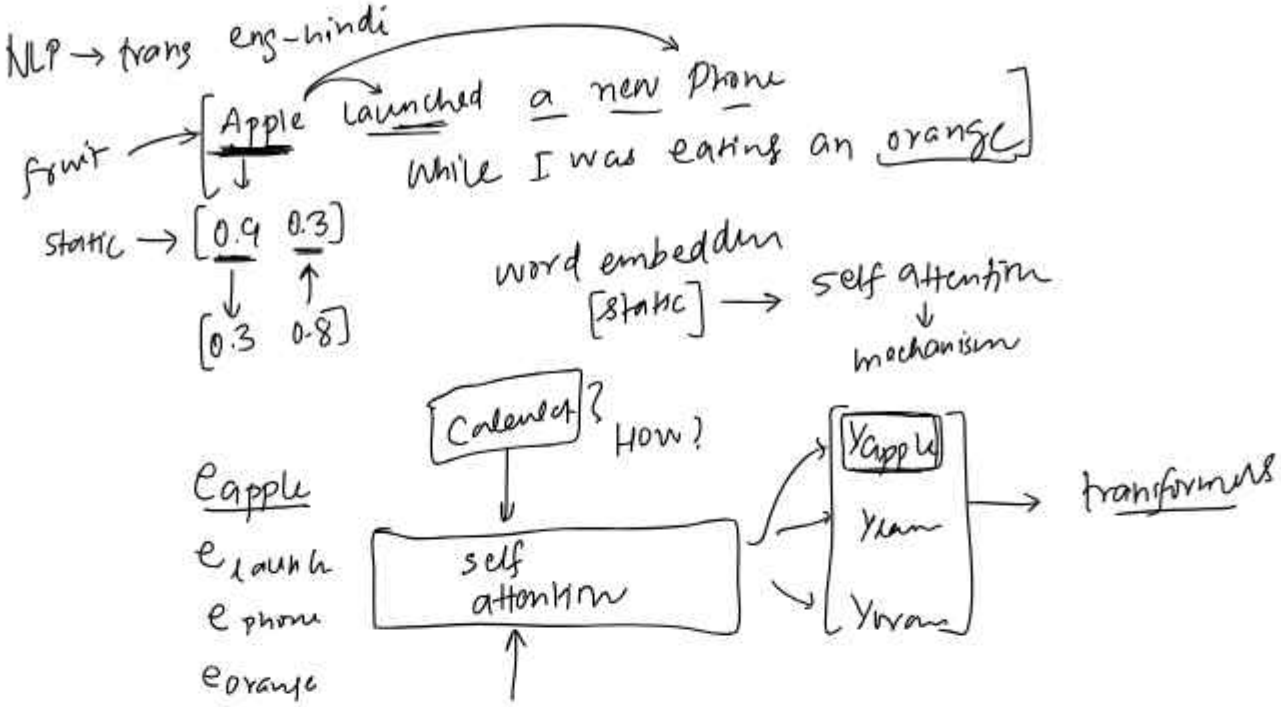
taste

word embeddings

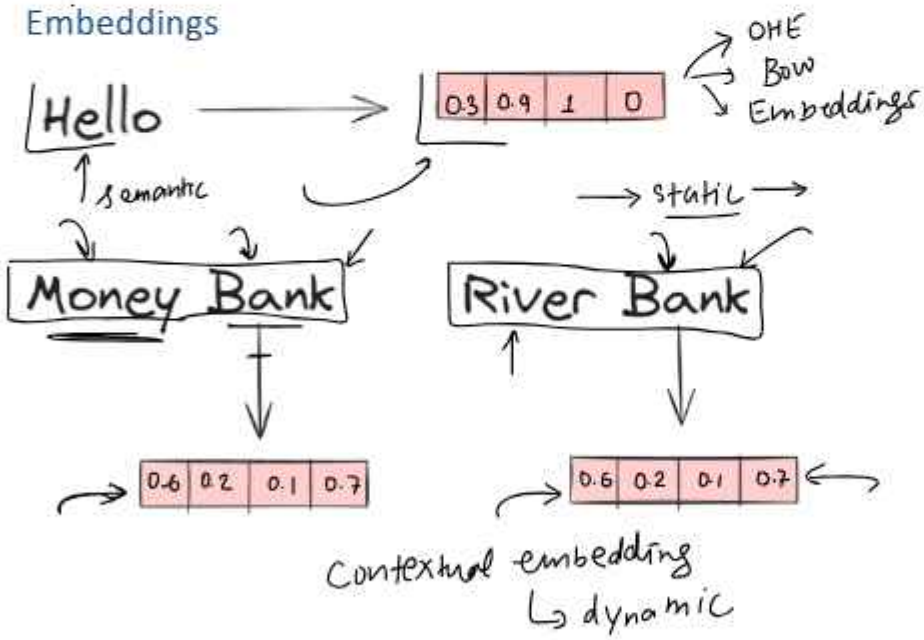
create → use → static

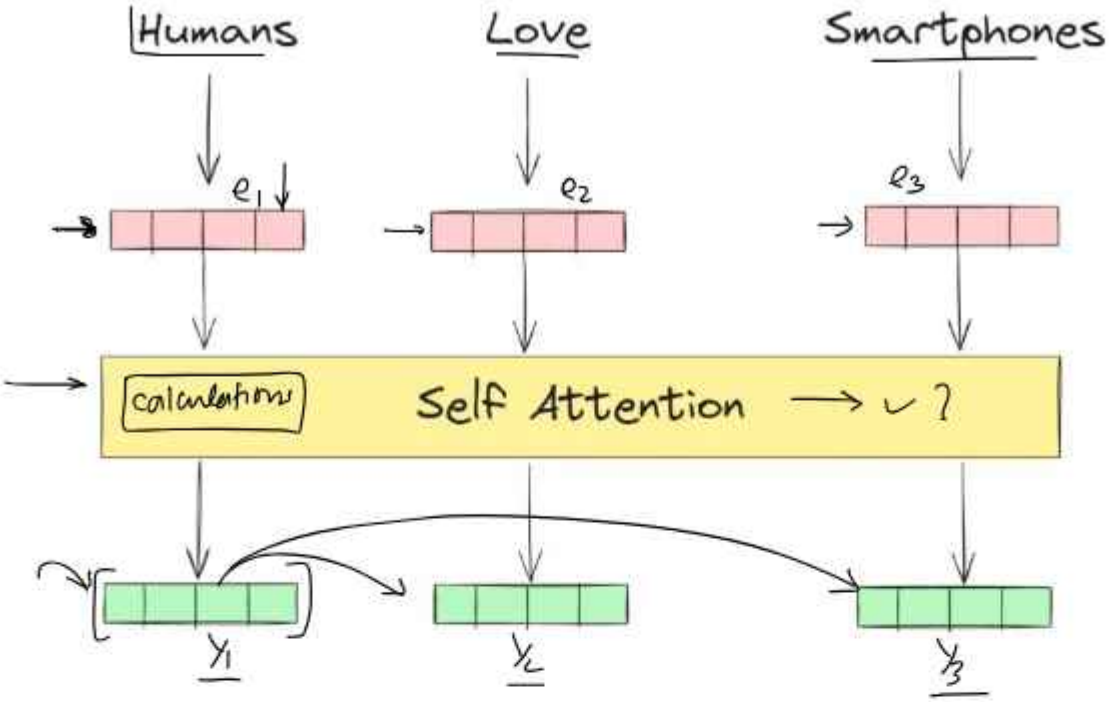
static embedding → [contextual embedding] ← smart

t = 0 ... trans eng-hindi →


NLP → trans eng-hindi
fruit
Apple launched a new Phone
while I was eating an orange
static → [0.9 0.3]
[0.3 0.8]
word embeddin [static] → self attention mechanism
Calendar? How?
eapple
elaunch
ephone
eorange
self attention
transformers

Embeddings


Hello
0.3 0.9 1 0
OHE
BoW
Embeddings
semantic
static
Money Bank
River Bank
0.6 0.2 0.1 0.7
0.6 0.2 0.1 0.7
Contextual embedding
↳ dynamic
Self Attention
embeddings → dynamic contextual


Humans
Love
Smartphones
e1
e2
e3
calculations
Self Attention
y1
y2
y3

First Principle Approach

04 February 2024 17:03

money bank grows

river bank flows

bank → bank

bank → 0.3 money + 0.7 bank + 0.1 grows

bank = 0.5 river + 0.4 bank + 0.1 flows

S1 word embe

S2

money = 0.7 money + 0.2 bank + 0.1 grows

bank = 0.25 money + 0.7 bank + 0.05 grows

grows = 0.1 money + 0.2 bank + 0.7 grows

river = 0.8 river + 0.15 bank + 0.05 flows

bank = 0.2 river + 0.78 bank + 0.02 flows

flows = 0.4 river + 0.01 bank + 0.59 flows

n dim

$$e^{(new)}_{money} = 0.7\, e_{money} + 0.2\, e_{bank} + 0.1\, e_{grows}$$

$$e^{(new)}_{bank} = 0.25\, e_{money} + 0.7\, e_{bank} + 0.05\, e_{grows}$$

$$e^{(new)}_{grows} = 0.1\, e_{money} + 0.2\, e_{bank} + 0.7\, e_{grows}$$

similarity

e_{money} e_{money}

e_{money} e_{bank}

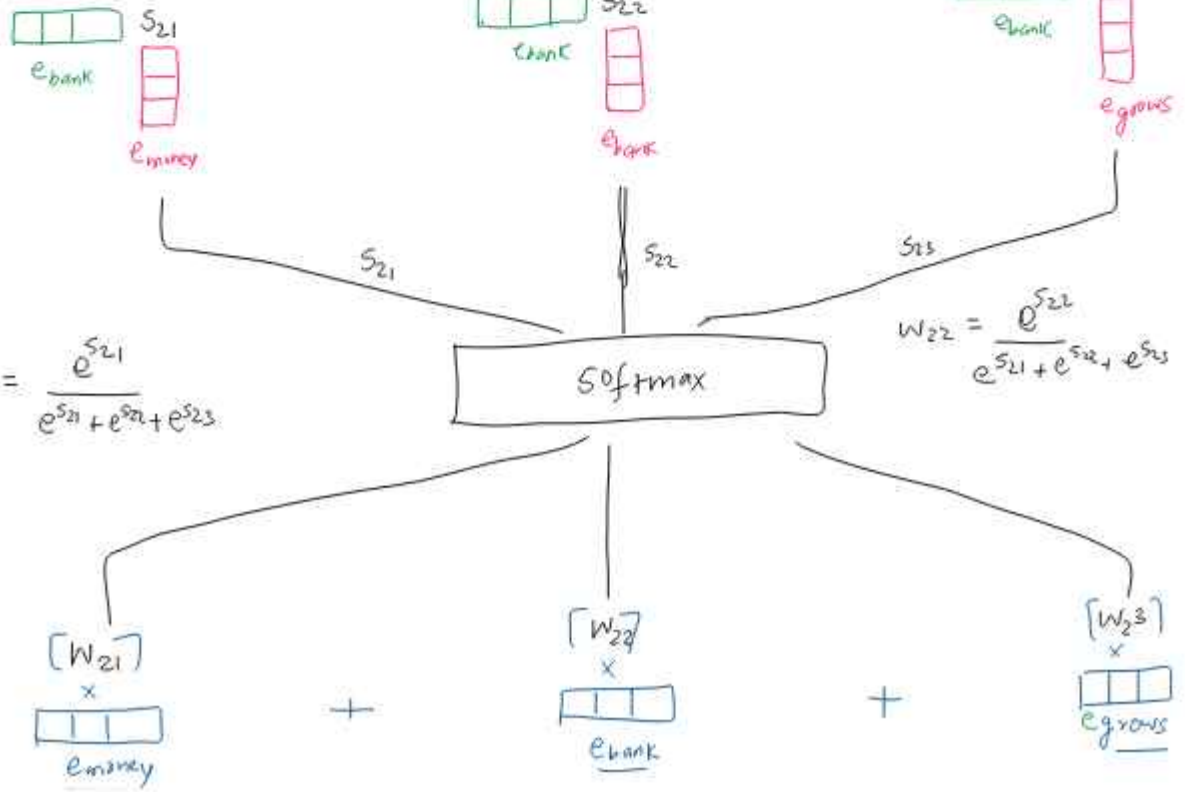
e_{money} e_{grow}

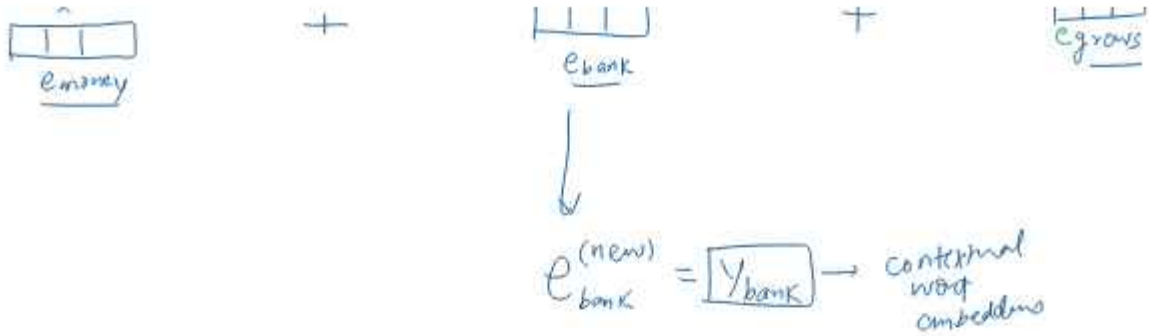
e_{grows} e_{money}

dot products

normalised

$$e^{(new)}_{bank} = [e_{bank} \cdot e^{T}_{money}]\, e_{money} + [e_{bank} \cdot e^{T}_{bank}]\, e_{bank} + [e_{bank} \cdot e^{T}_{grows}]\, e_{grows}$$

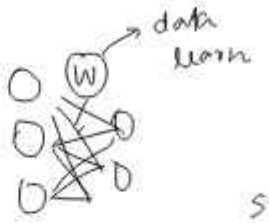


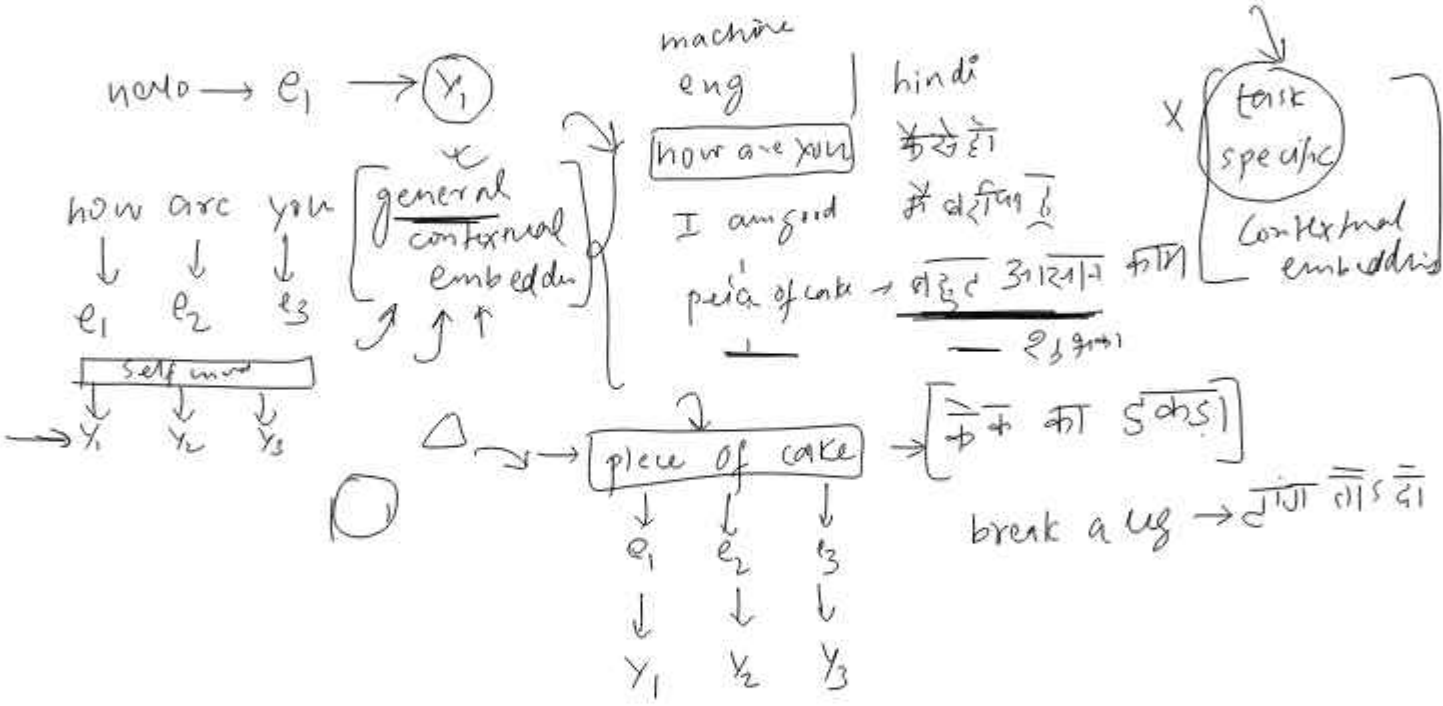


Points to consider

→ This operation is a parallel operation

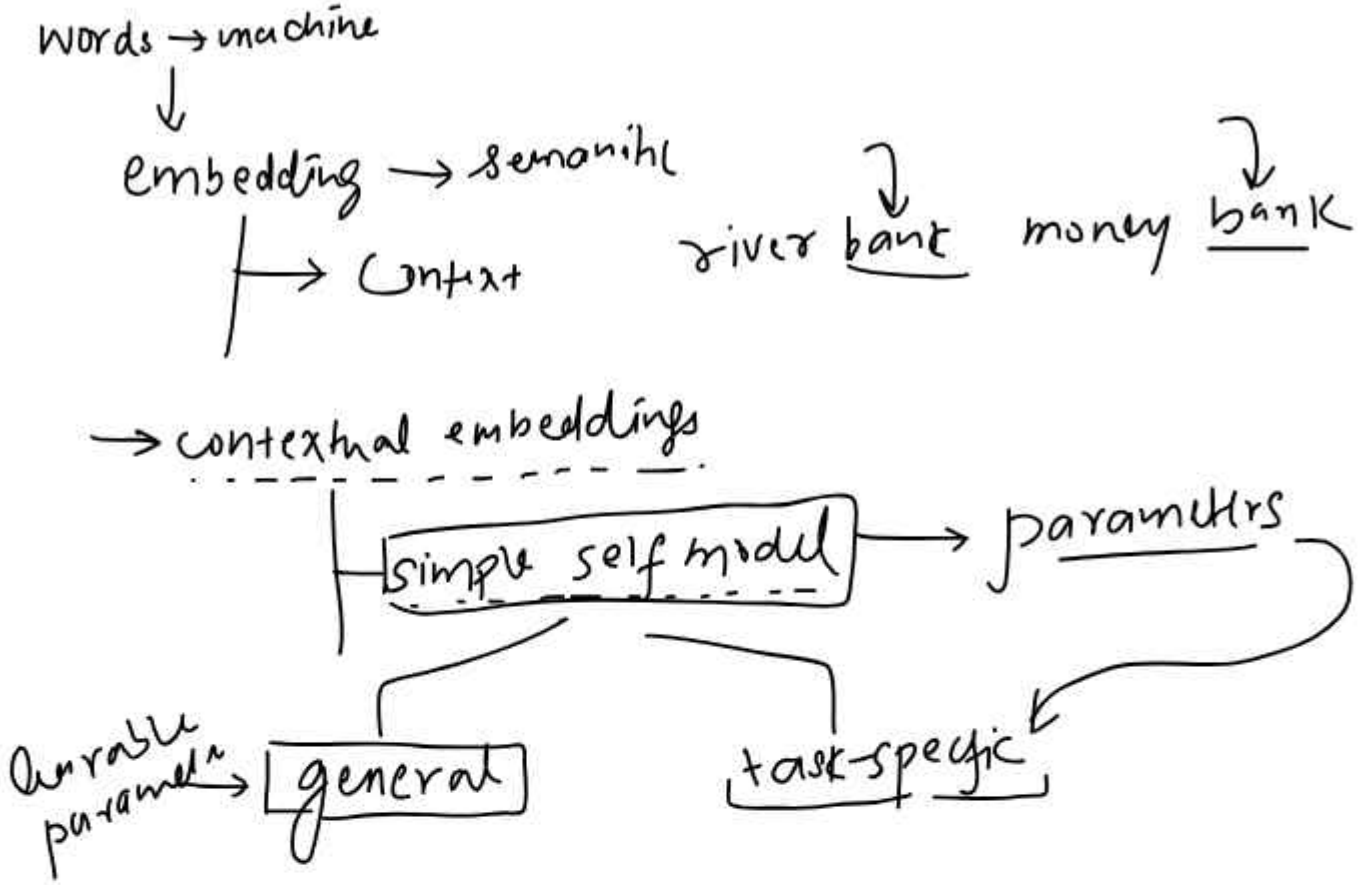
→ There are no parameters involved





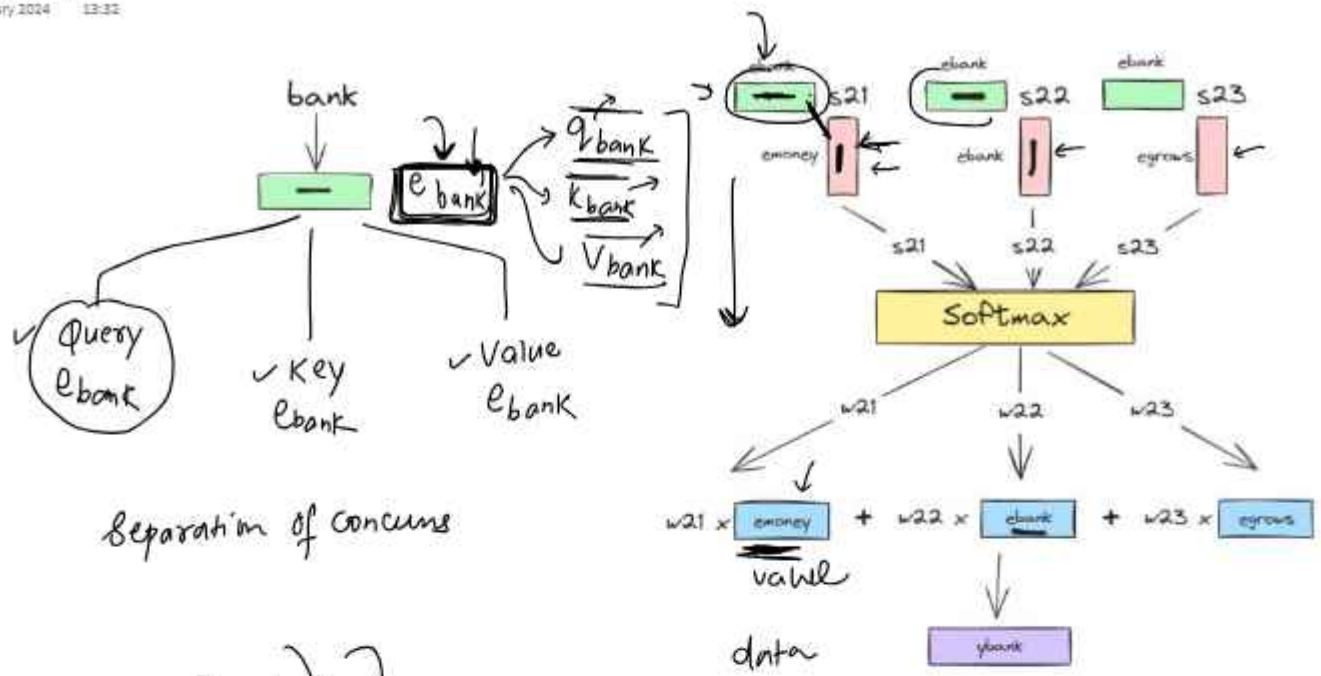
Progress

06 February 2024 00:42

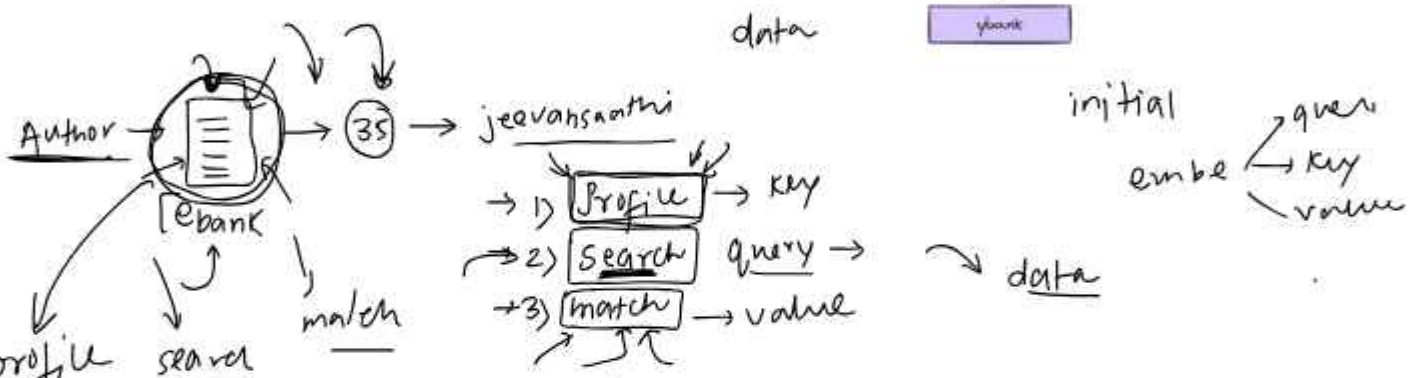


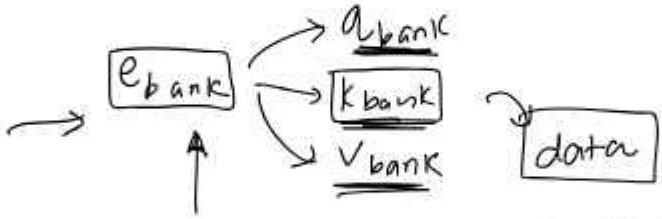
Query, Key & Value Vectors

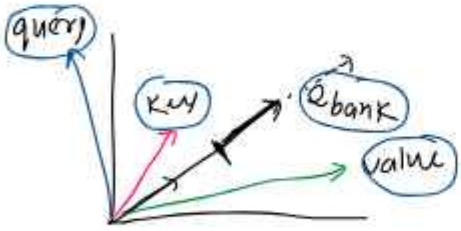
06 February 2024 13:32



Separation of concerns



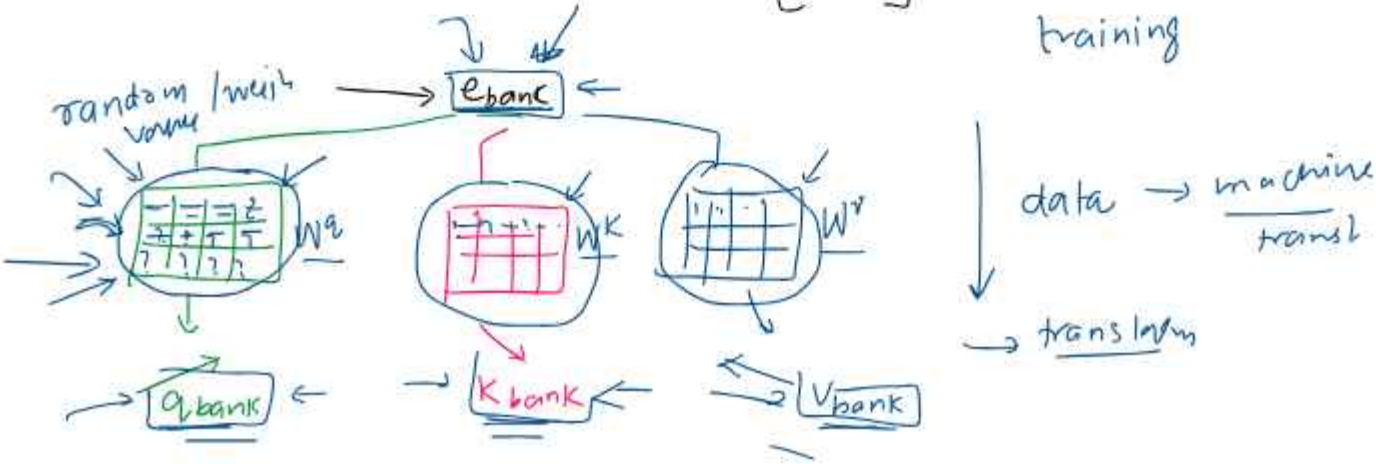


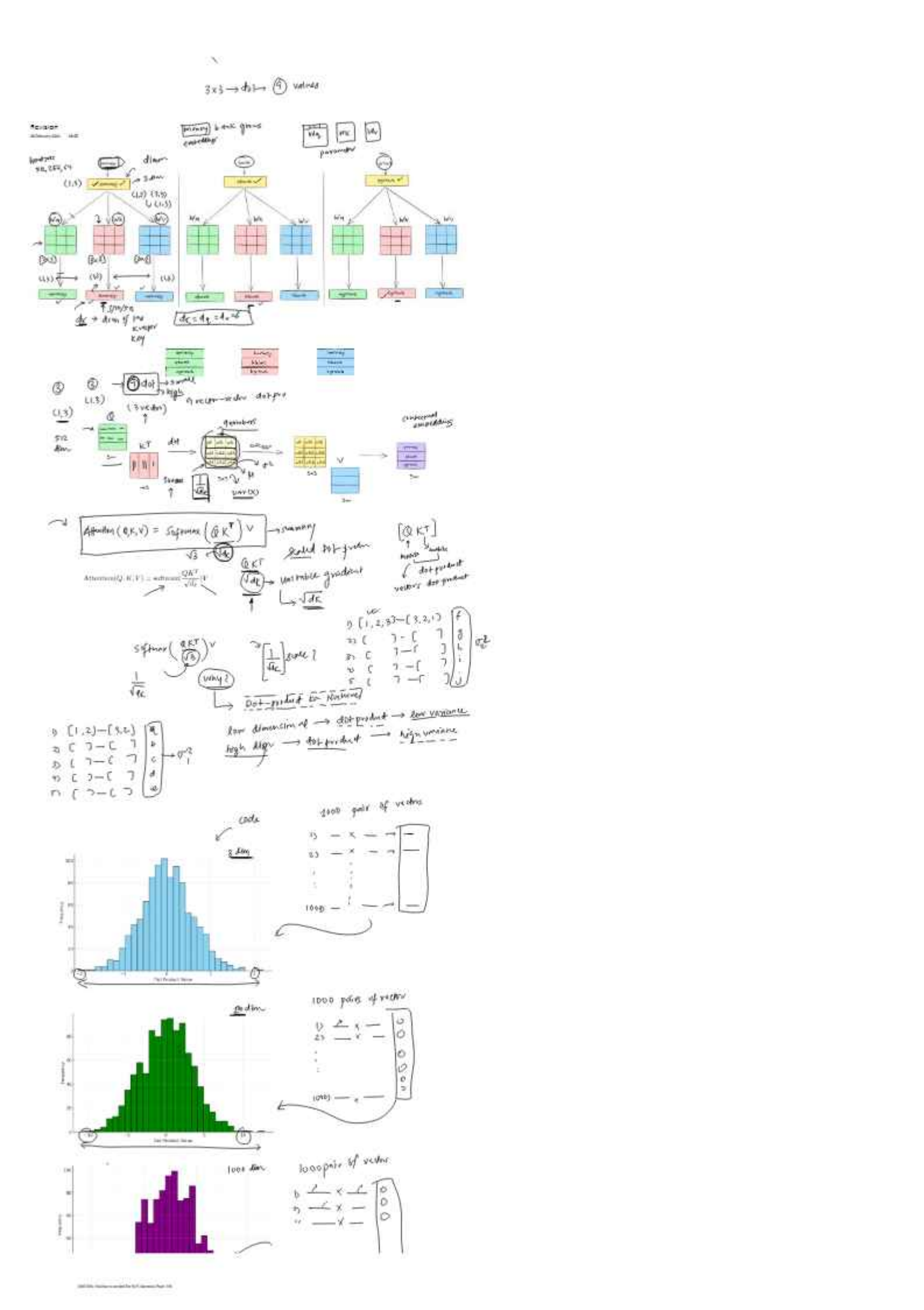


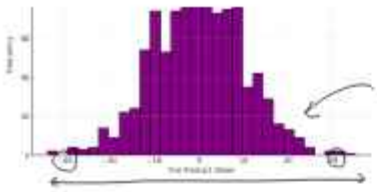
d magnitude (scaling)

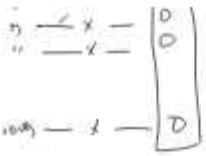
→ linear transform

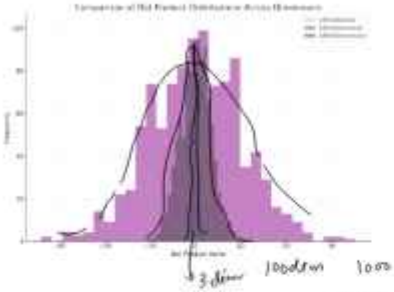
→ [ ]

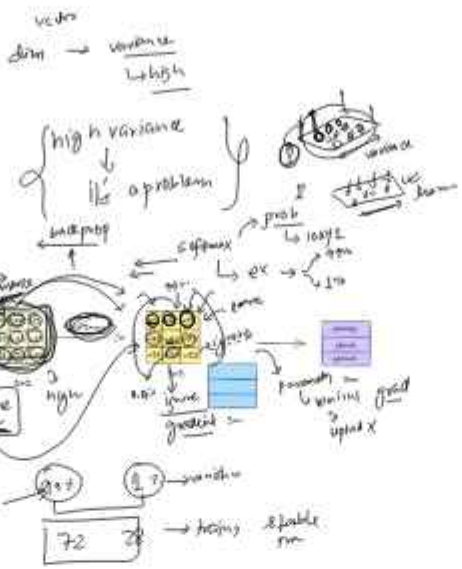



3x3 → dot → 9 values
Revision
parameters
dim
dk = dq = dv
Attention(Q,K,V) = Softmax(QK^T/√dk)V
summary
Scaled Dot-product
Unstable gradient
dot-product
vectors dot product
Softmax(QKT/√dk)V
why?
Dot-product for Variance
low dimension of → dot product → low variance
high dim → dot product → high variance
code
8 dim
1000 pair of vectors
100 dim
1000 pair of vector

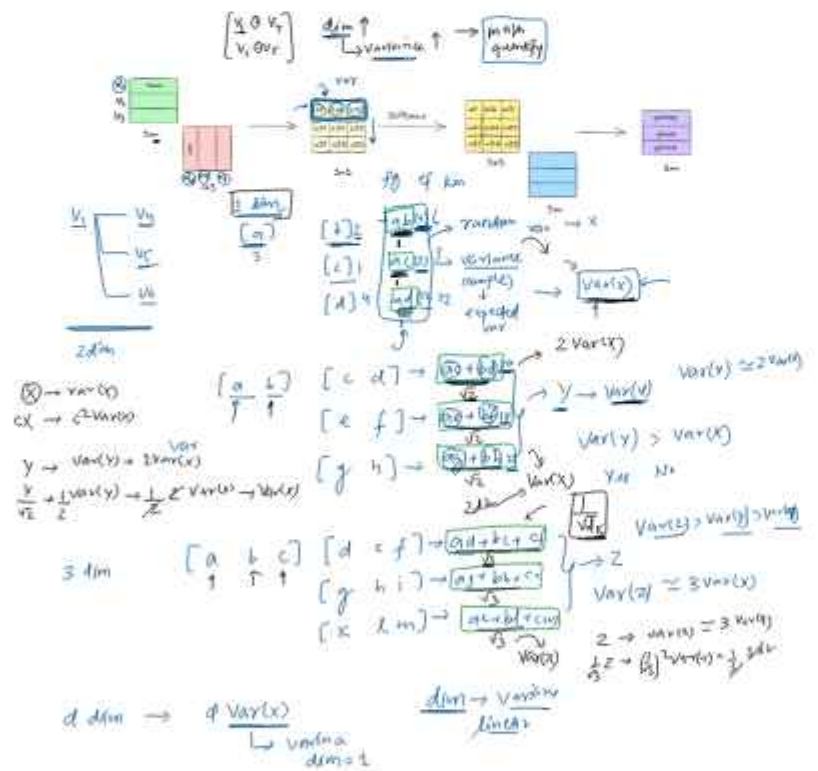


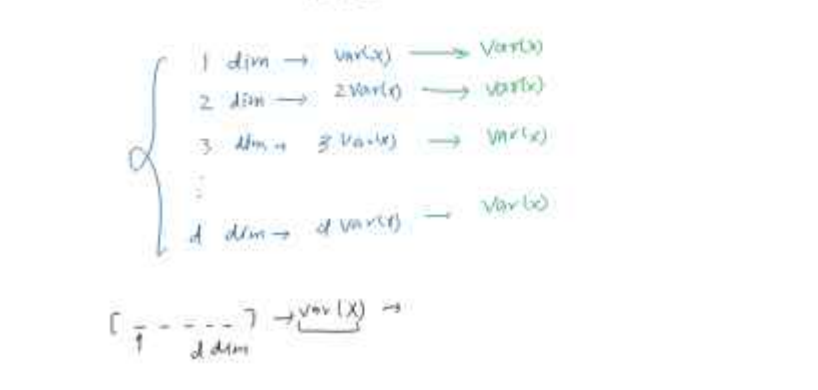




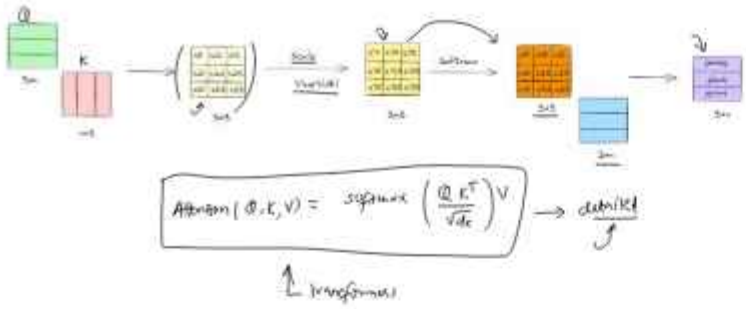












$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right)V$$

→ decoder

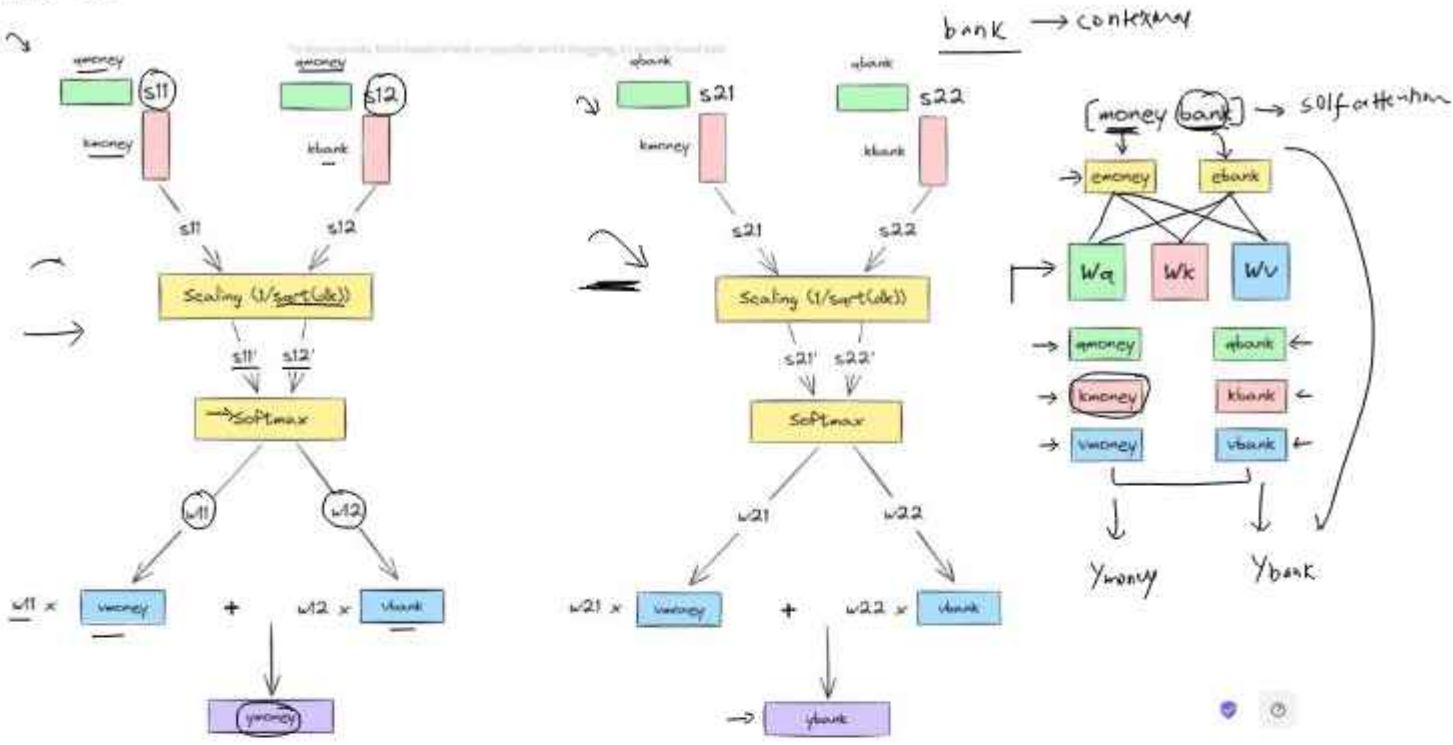
↑ transformer

What is d_k

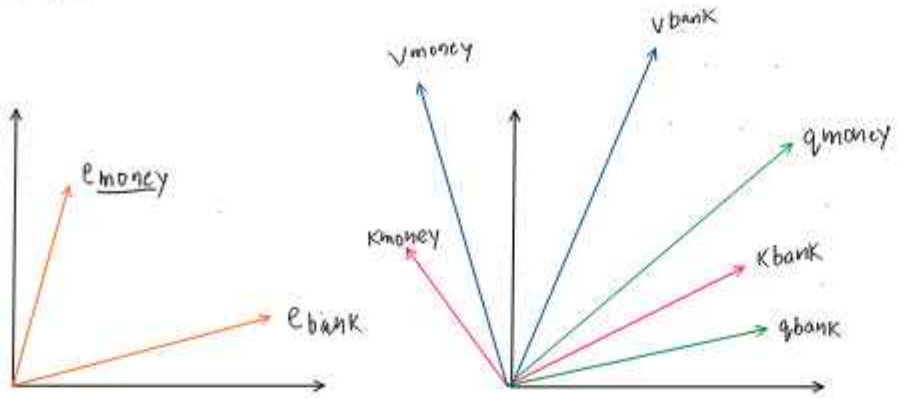
28 February 2024 16:59

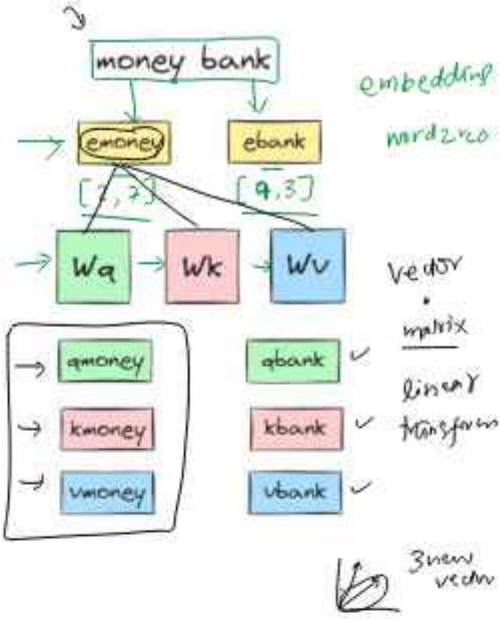
Recep

08 March 2024 15:14



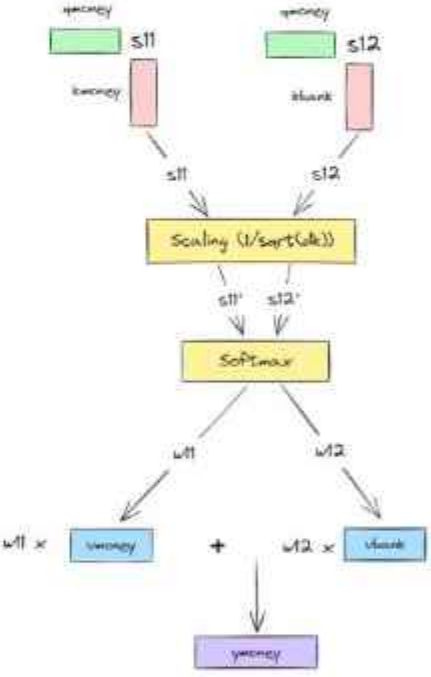
Geometric Intuition

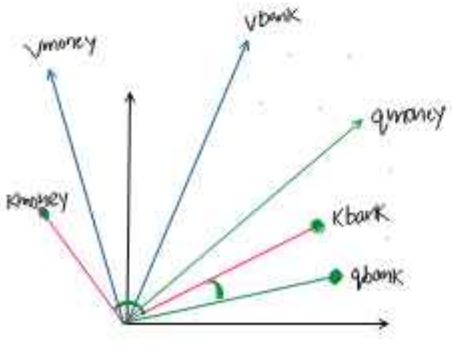




$W_q = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ (2×2) $\quad W_k = \begin{bmatrix} 3 & 4 \\ 5 & 1 \end{bmatrix}$ (2×2) $\quad W_v = \begin{bmatrix} 4 & 1 \\ 2 & 1 \end{bmatrix}$ (2×2)

* All values are hypothetical





Dot Product

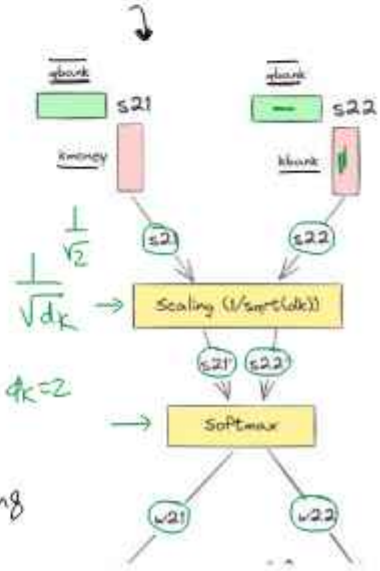
$S_{21} = 10$ $\quad S_{22} = \frac{32}{\sqrt{2}}$

Scaling:

$S'_{21} = \frac{10}{\sqrt{2}} = 7.09$ $\quad S'_{22} = 22.69$

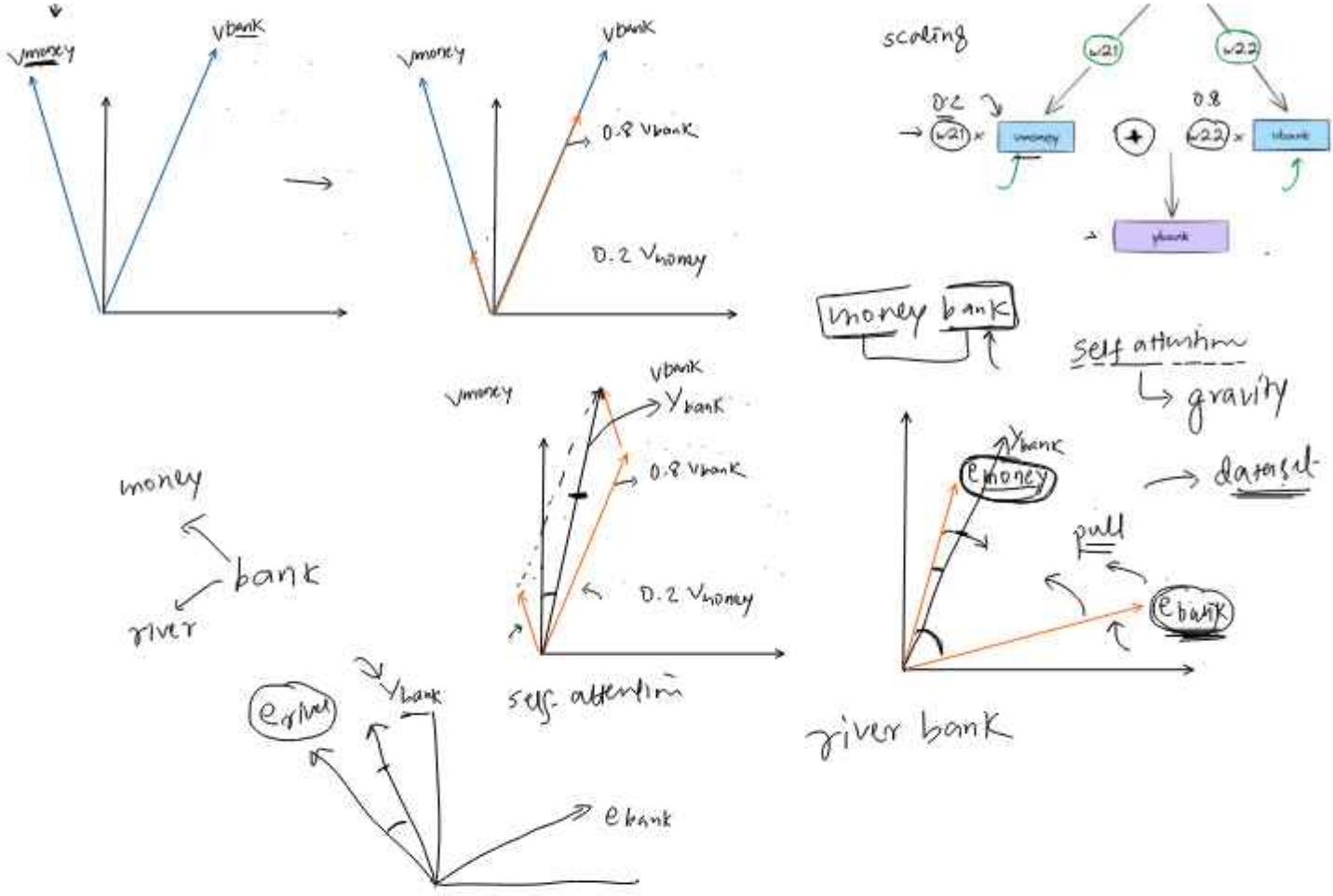
Softmax

$w_{21} = 0.2$ $\quad w_{22} = 0.8$



scaling



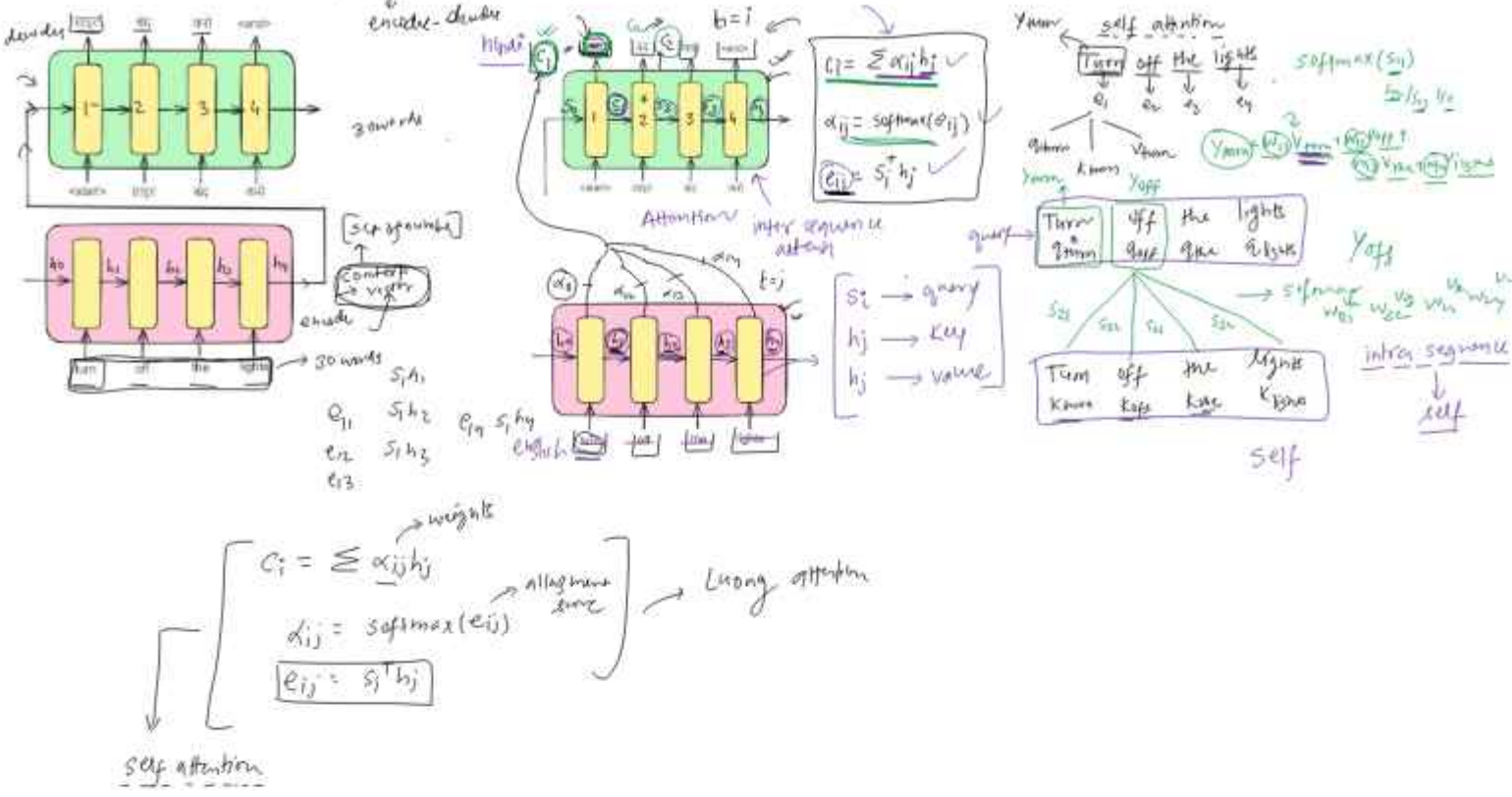

vmoney
vbank
0.8 vbank
0.2 vmoney
scaling
w21
w22
0.2
0.8
money bank
self attention
gravity
dataset
pull
money
bank
river
ybank
ebank
self attention
river bank

Recap of Attention

long brans → NN

encoder-decoder

Attention



$c_i = \sum \alpha_{ij} h_j$ (weights)

$\alpha_{ij} = \text{softmax}(e_{ij})$ (alignment score)

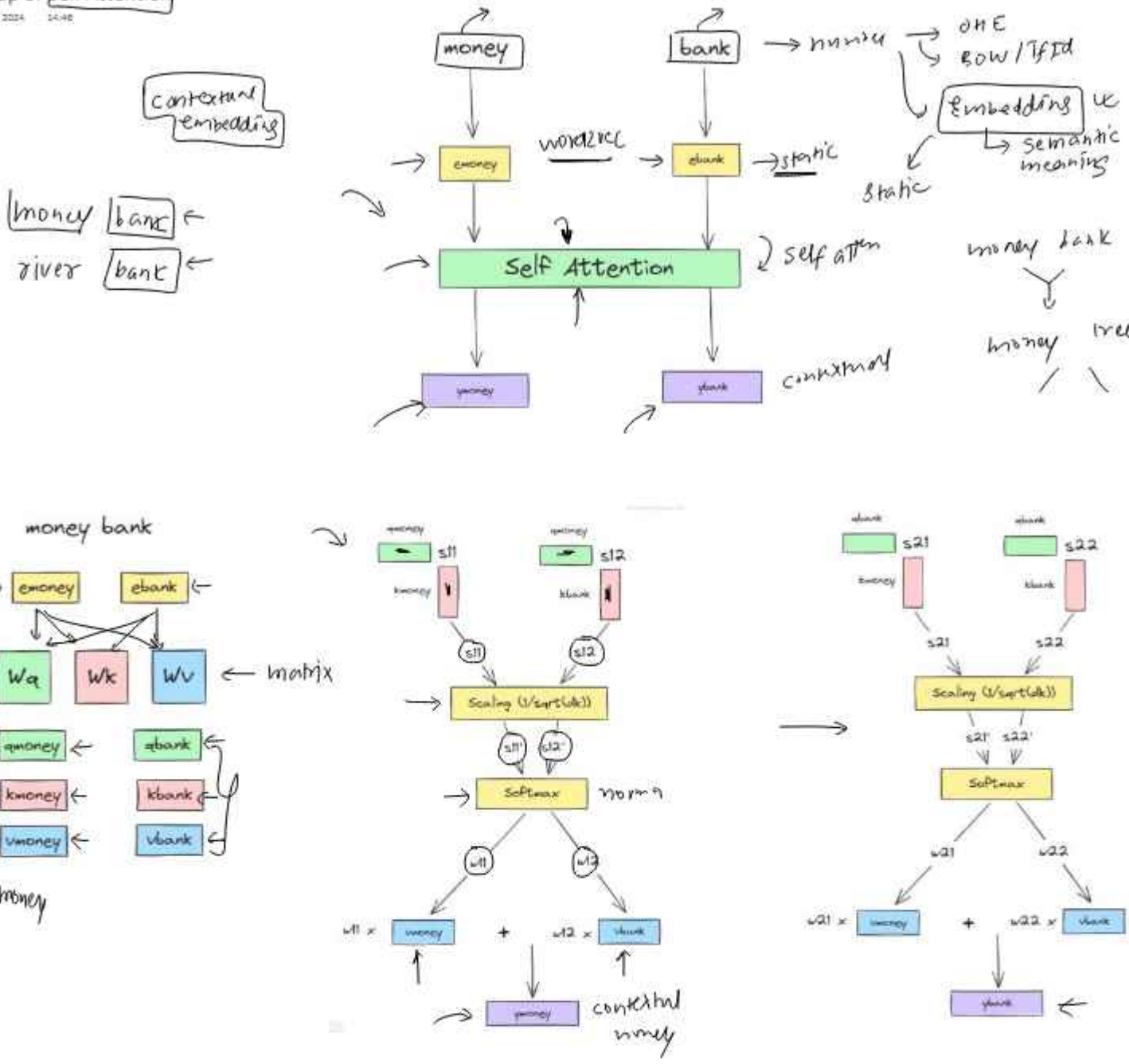
$e_{ij} = s_i^T h_j$

→ Luong attention

self attention

Recap of Self Attention

08 April 2024 14:48



Problem with Self Attention

08 April 2024 14:47

The man saw the astronomer with a telescope

Multihead attention

table

self

single perspective

	The	man	saw	the	astronomer	with	a	telescope
The								
man								
saw								
the								
astronomer								
with								
a								
telescope								

doc summarization

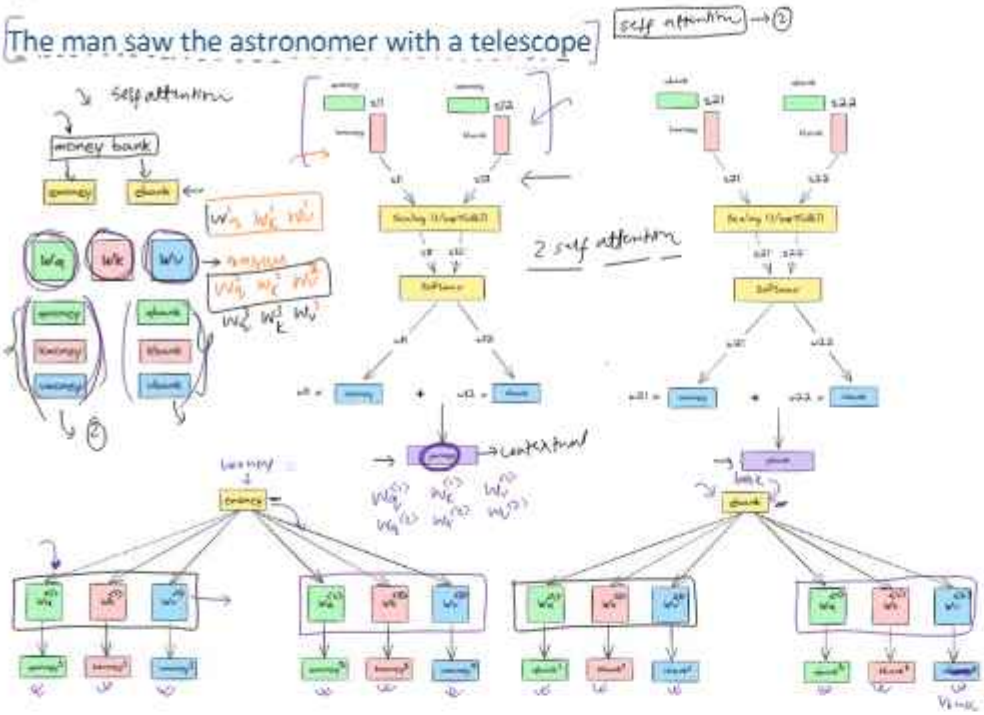
self attention

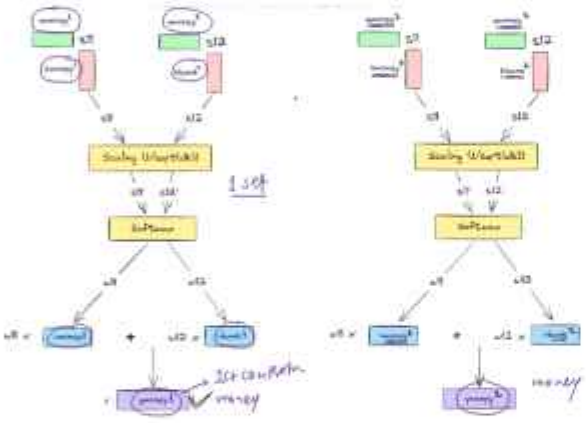
The future of AI in India presents a dynamic and promising landscape, marked by rapid advancements and a burgeoning ecosystem of innovation. With a robust talent pool of engineers and IT professionals, India is poised to become a significant player in the global AI arena. The government's proactive stance on AI, exemplified by initiatives like the National Strategy for Artificial Intelligence, aims to harness AI's potential across various sectors, including healthcare, education, agriculture, and urban infrastructure. Indian startups and tech giants are increasingly incorporating AI to solve complex societal challenges, improve efficiency, and enhance service delivery. Moreover, India's focus on ethical AI and data security aims to create a sustainable and responsible growth trajectory. As AI becomes more integrated into daily life and industry, India's unique blend of technological prowess, entrepreneurial spirit, and societal needs will likely shape a distinctive path in the AI domain, fostering innovation that is not only technologically advanced but also socially inclusive and impactful.

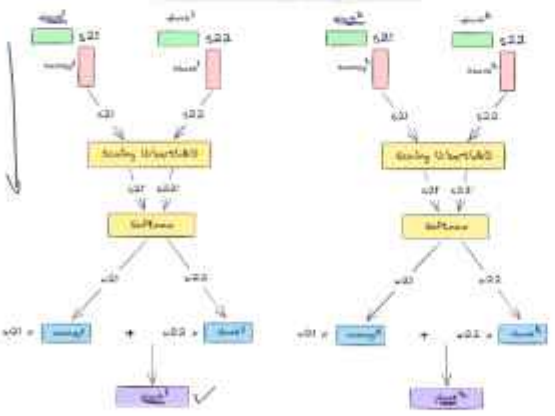
India is poised to become a key player in the AI domain, leveraging its skilled workforce and government initiatives to apply AI across various sectors like healthcare and education. With a focus on innovation, ethical AI, and data security, India aims to integrate AI to address societal challenges, enhance efficiency, and promote inclusive growth. This approach positions India to uniquely contribute to global AI advancements while ensuring sustainable and responsible development within its own borders.

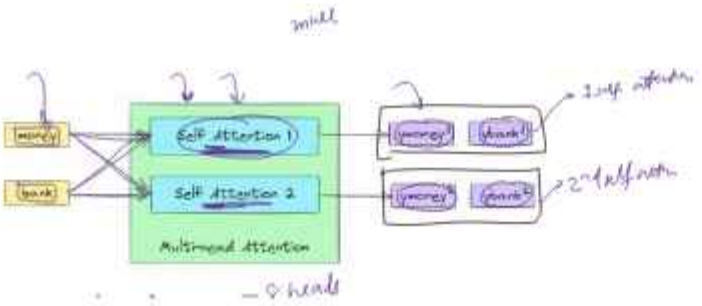
India's AI future holds promise as it harnesses a burgeoning talent pool and government initiatives to pioneer AI-driven innovation. With a focus on sectors like healthcare and education, India aims to leverage AI for societal development. The emphasis on ethical AI and data security underscores India's commitment to responsible technological advancement. This approach positions India not only as a global AI hub but also as a trailblazer in addressing societal challenges through cutting-edge technology.

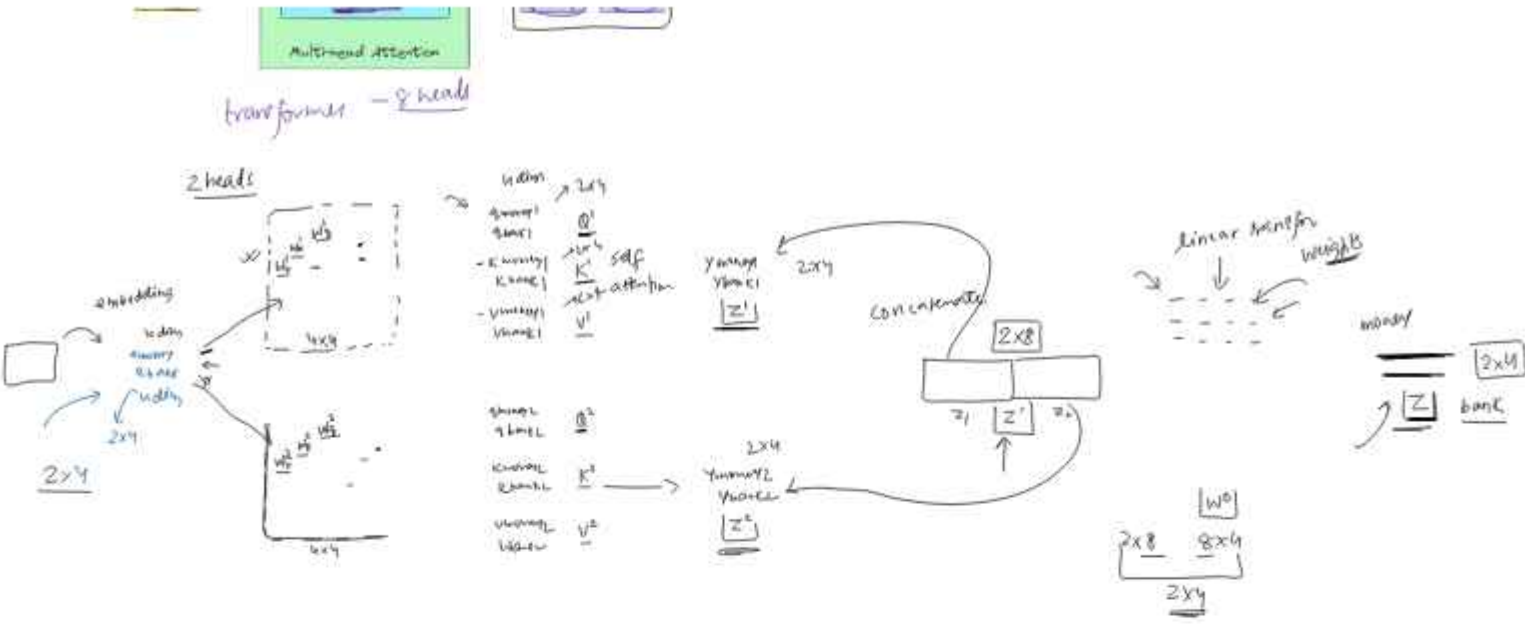
Multi-head Attention

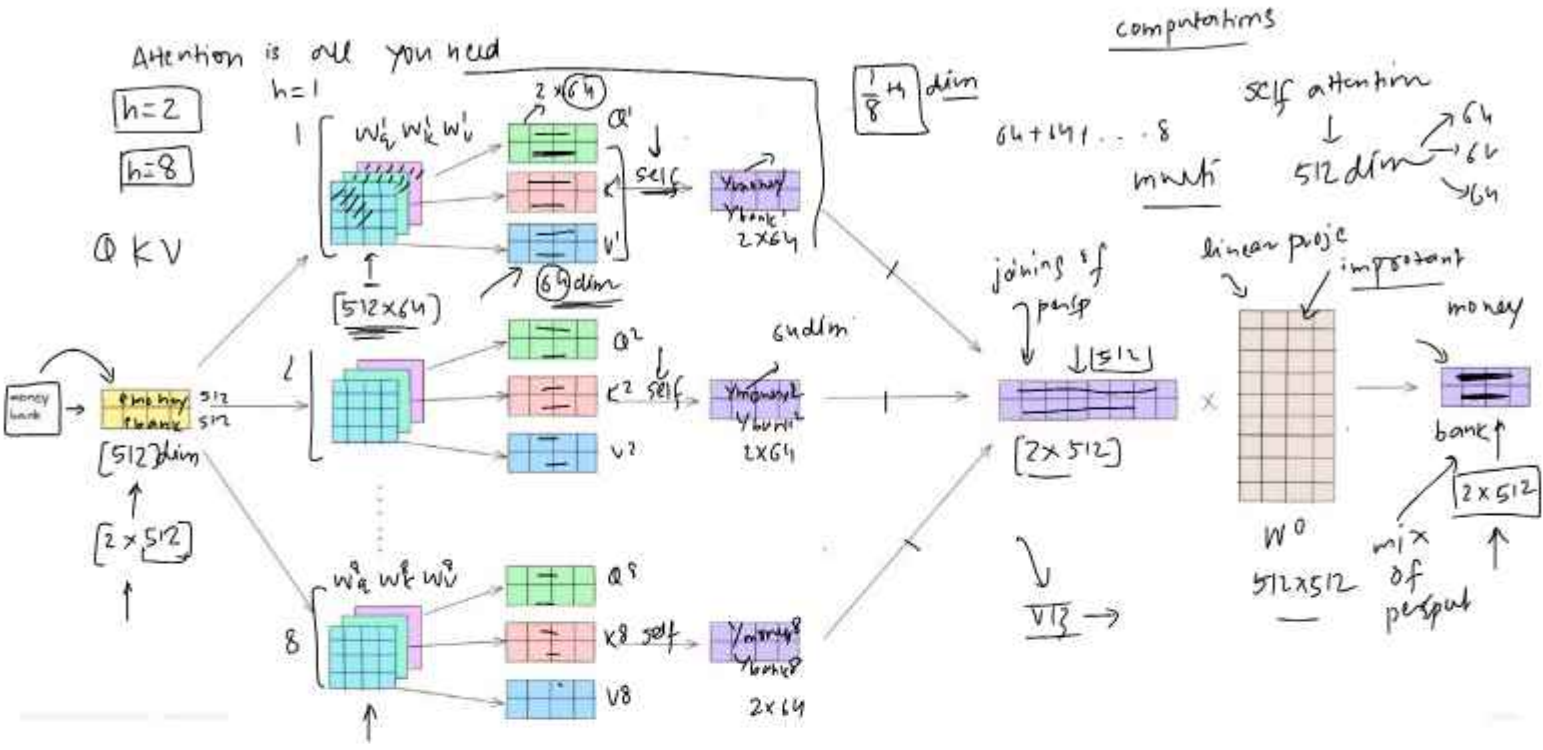








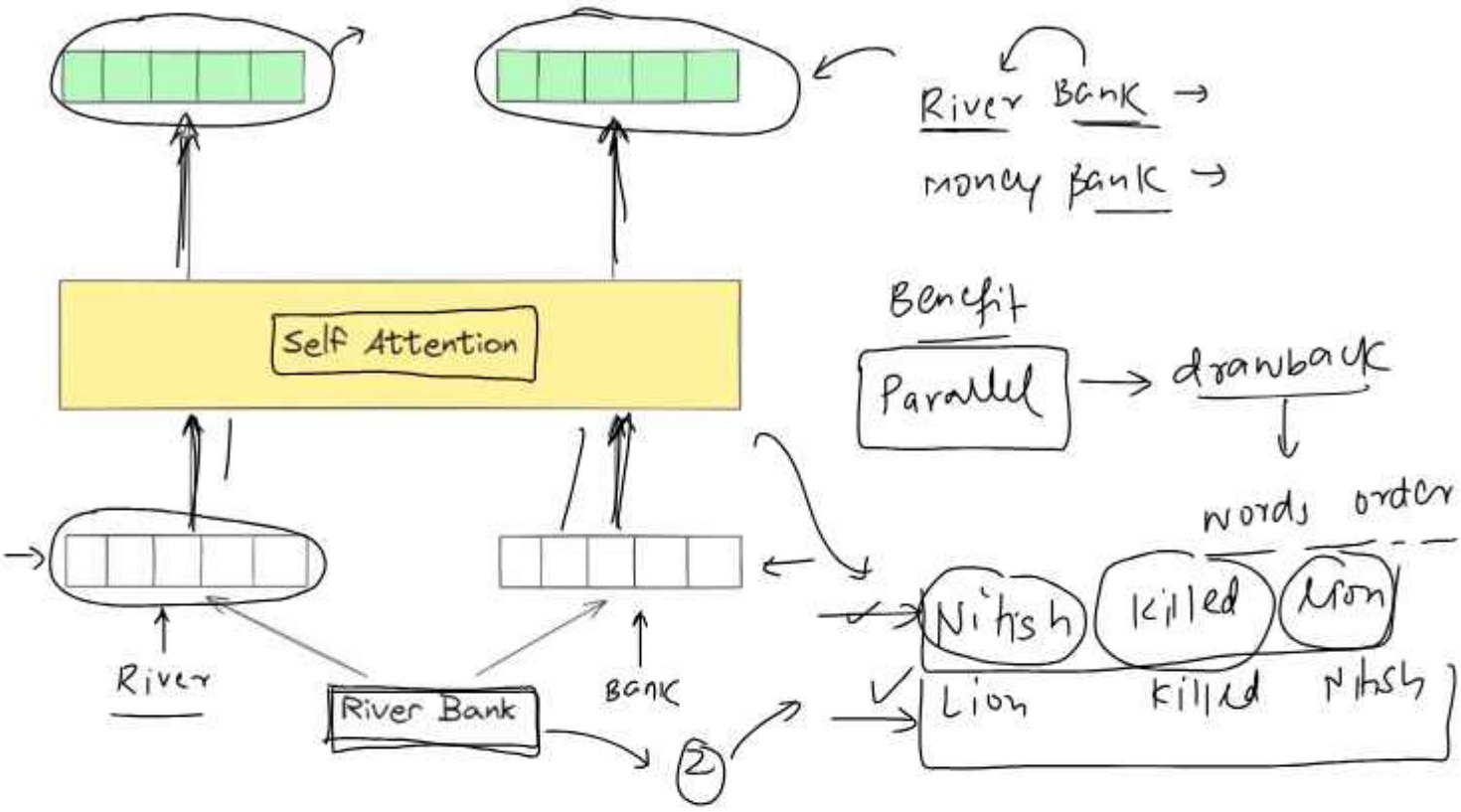

Multi-Head Attention
transformer - 8 heads
2 heads
embedding
2x4
4x8
4x4
self attention
2x4
2x8
concatenate
linear transform
weights
2x4
bank
8x4
2x8
2x4


Attention is all you need
h=1
h=2
h=8
Q K V
computations
self attention
512 dim
64
64
64
multi
64+64 . . . 8
1/8 64 dim
joining of persp
linear proje
important
money
bank
mix of persp
W0
512x512
[2x512]
[512]dim
[2x512]
money bank
512
512
2x64
6dim
self
[512x64]
2x64
2x64

The Why

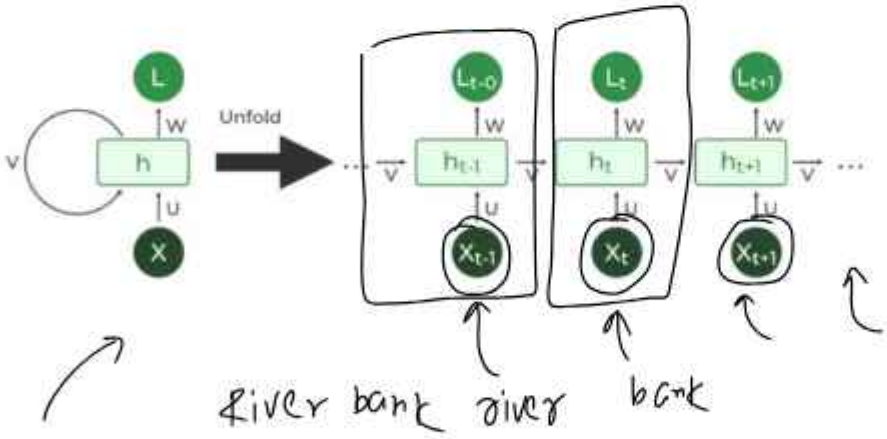
23 May 2024 14:33

Self-Attention



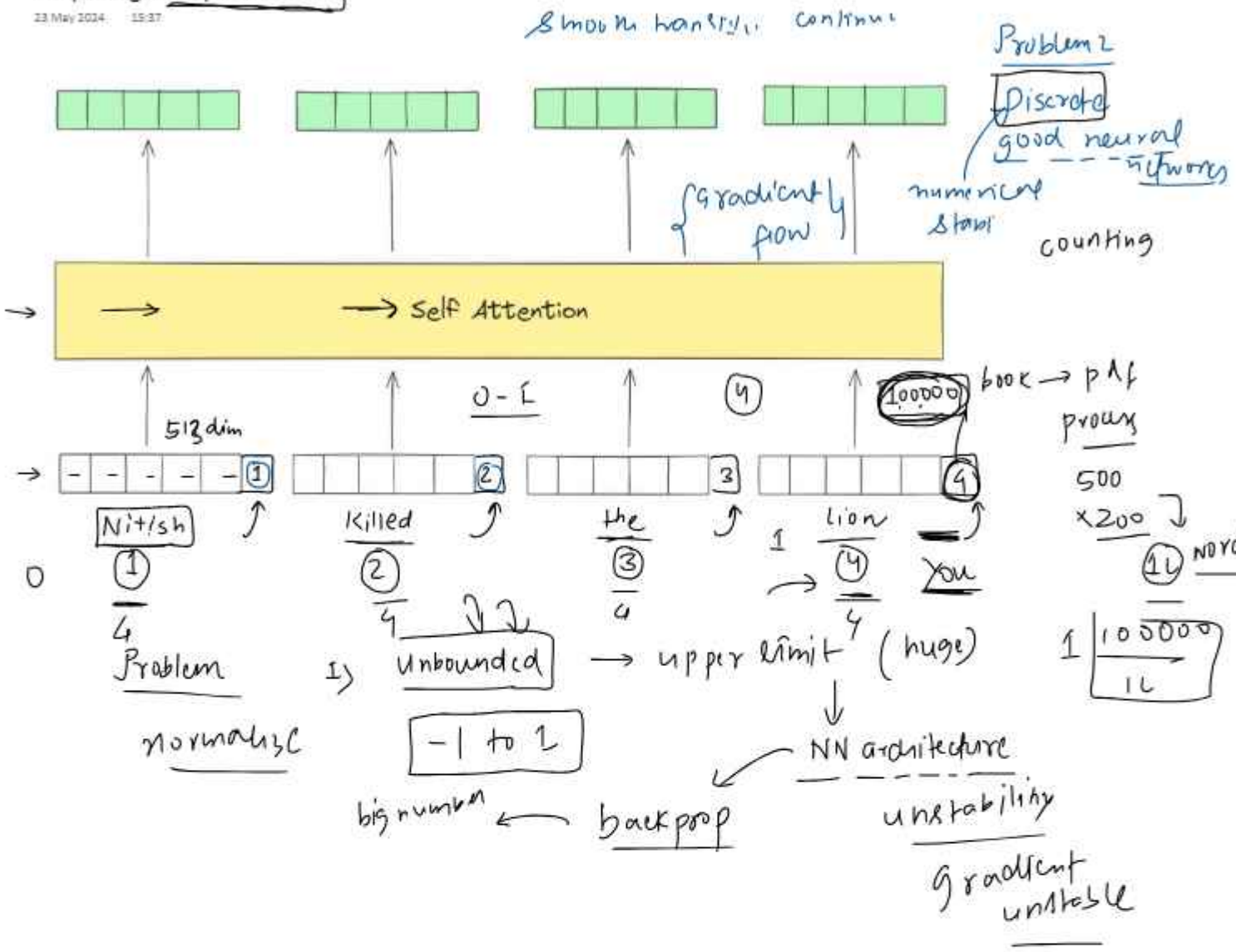
RNN → sequential

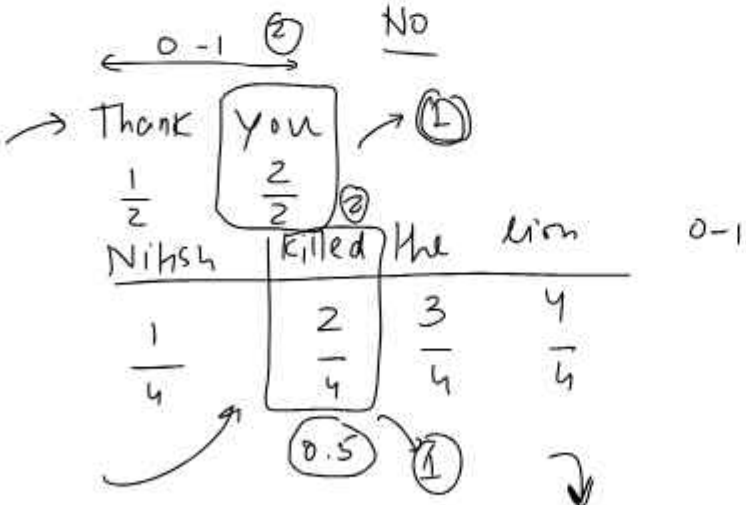
Nitish killed Lion



Proposing a simple solution

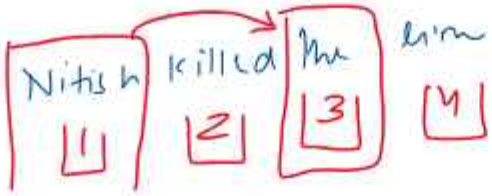
23 May 2024 15:37





Problem 3 → can't relative positionings

3-1 = 2



1 2 3 4

absolute positim | reative positim

discrete → periodic → relative positioning

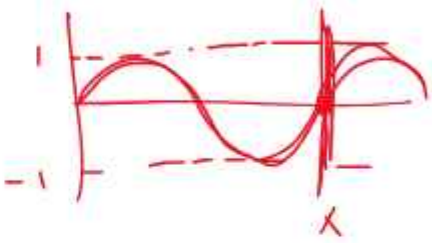
Problem

→ unbounded (bounded) ✓✓

→ discrete (continuous) → functim →

→ relative / periodic ✓

trigonometric → sine



positional encodn → sine → better solutim

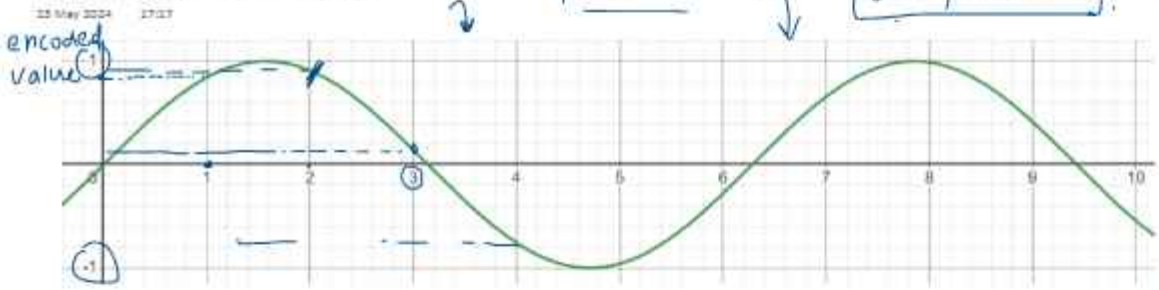
The sine function as a solution

23 May 2024 17:27

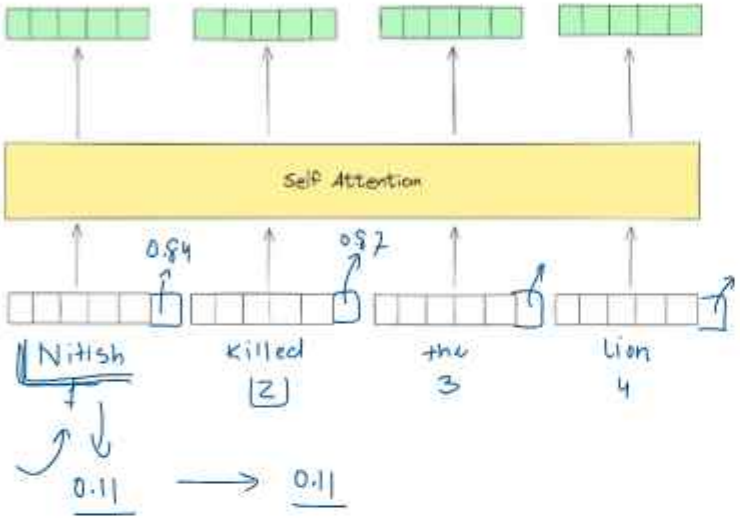
periodic

BIS problem

$y = \sin(pos)$



- 1) unbounded
- 2) discrete
- 3) relational positioning



encoded value

$y = \sin(position)$

$y = \sin(1) = 0.84$

$y = \sin(2) = 0.87$

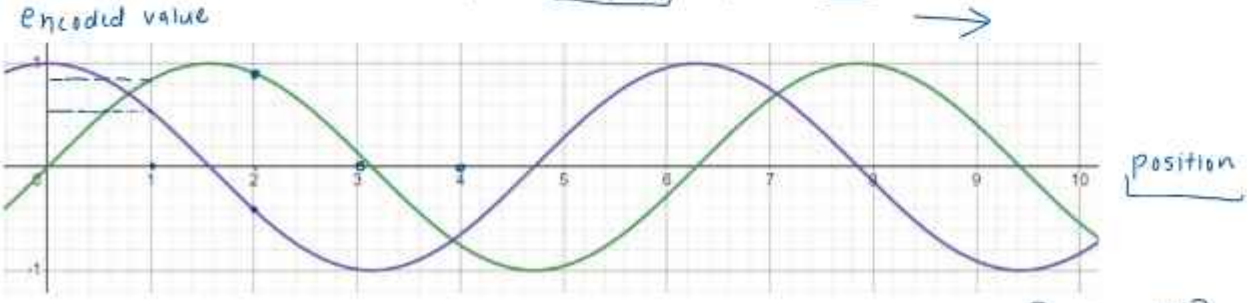
$y = \sin(3) = 0.10$

$y = \sin(4) = -0.90$

$y = \sin(1) = 0.84$

$y = \cos(1) = 0.5$

$y = \sin(pos)$ $y = \cos(pos)$

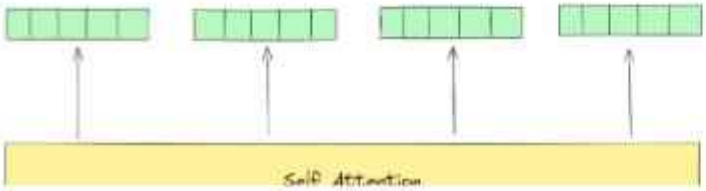


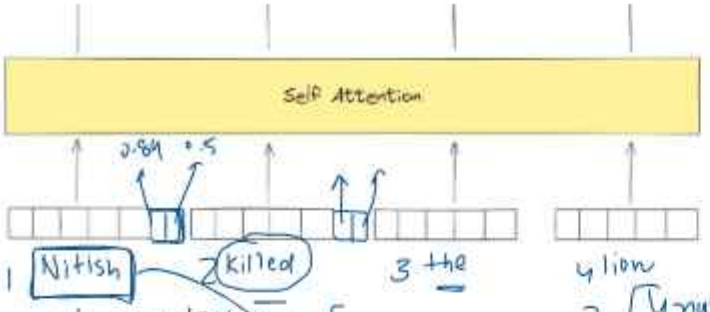
Nitish → [0.84, 0.5]

$y = \sin(2)$ $y = \cos(2)$

[0.87, -0.4]

another → vector



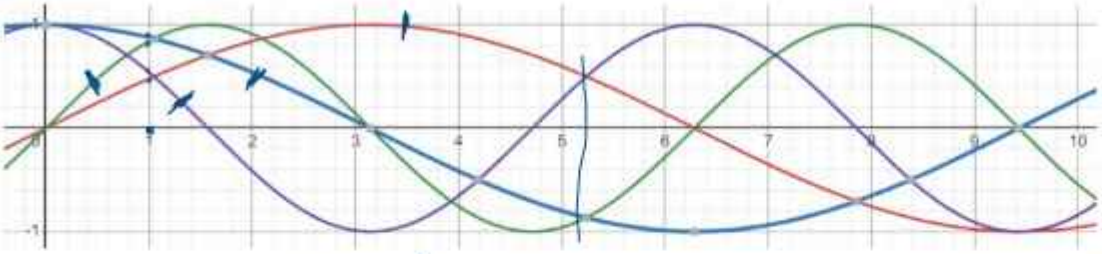


[0.87, -0.4] ~

scalar → vector

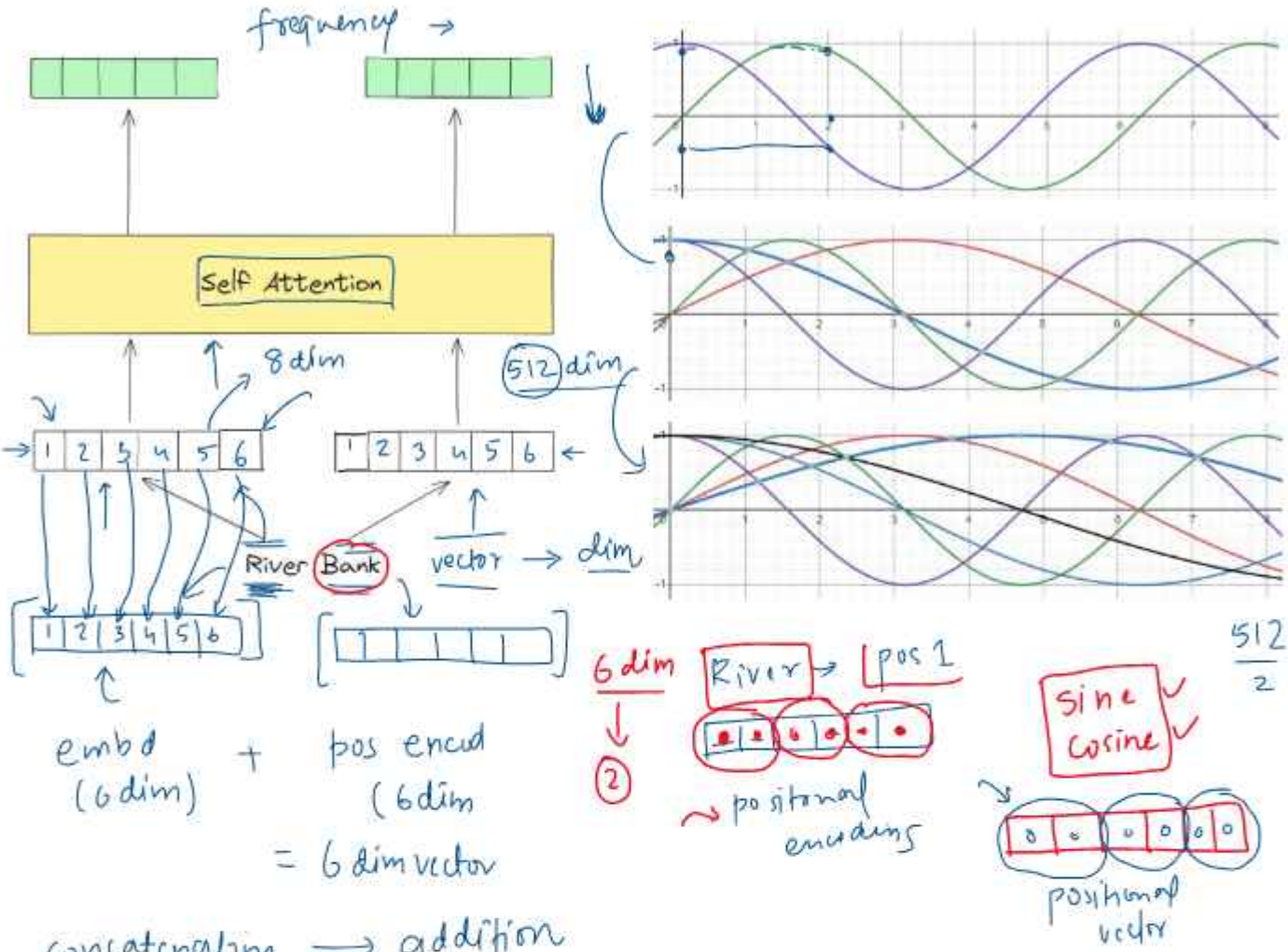
$y = \sin(pos)$ $y = \cos(pos)$

[4 number] $y = \sin(pos/2)$ $y = \cos(pos/2)$



pojt 6number vector $y = \sin(pos/3)$ $y = \cos(pos/3)$

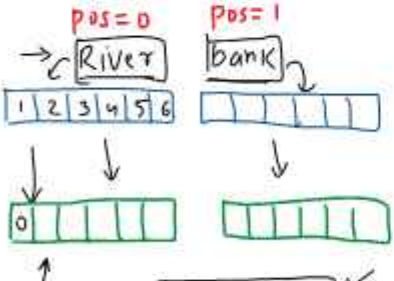
Positional Encoding



concatenation → addition

↳ 6 dim 6 dim

↳ 12 dim → parameters increase → training double up



pos → position

pos=0 pos=1

d_{model} → dim of embedding

$d_{model} = 6$

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{model}})$$

i = 0 1 2

i = 0 — $d_{model}/2$ + 1 → 0 - 6/2 3 → 0 - 3

for i = 0 → 0 - 2

for i = 0 → pos = 0

$PE(0,0) = \sin(0/10000^{0}) = 0$

$PE(0,1) = \cos(0/10000^{0}) = 1$

2i+1

$PE(1,0) = \sin(1/10000^{0}) = 0.84$

$PE(1,1) = \cos(1/10000^{0}) = 0.54$

for i = 1

for i = 1

1/1 = 0.04

for i=1 ✓

$PE(0,2) = \sin\left(0 / 10000^{1/3}\right) = 0$

$PE(0,3) = \cos\left(0 / 10000^{1/3}\right) = 1$

for i=1

$PE(1,2) = \sin\left(1 / 10000^{1/3}\right) = 0.04$

$PE(1,3) = \cos\left(1 / 100000^{1/3}\right) = 0.99$

for i=2 ✓

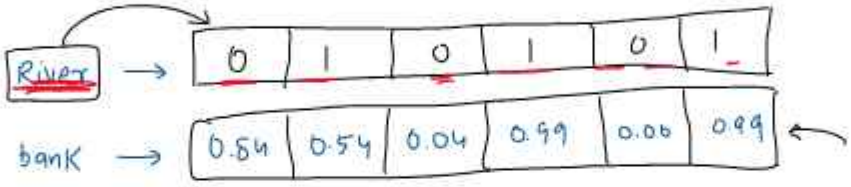
$PE(0,4) = \sin\left(0 / 10000^{2/3}\right) = 0$

$PE(0,5) = \cos\left(0 / 10000^{2/3}\right) = 1$

for i=2

$PE(1,4) = \sin\left(1 / 10000^{2/3}\right) = 0.00$

$PE(1,5) = \cos\left(1 / 10000^{2/3}\right) = 0.99$

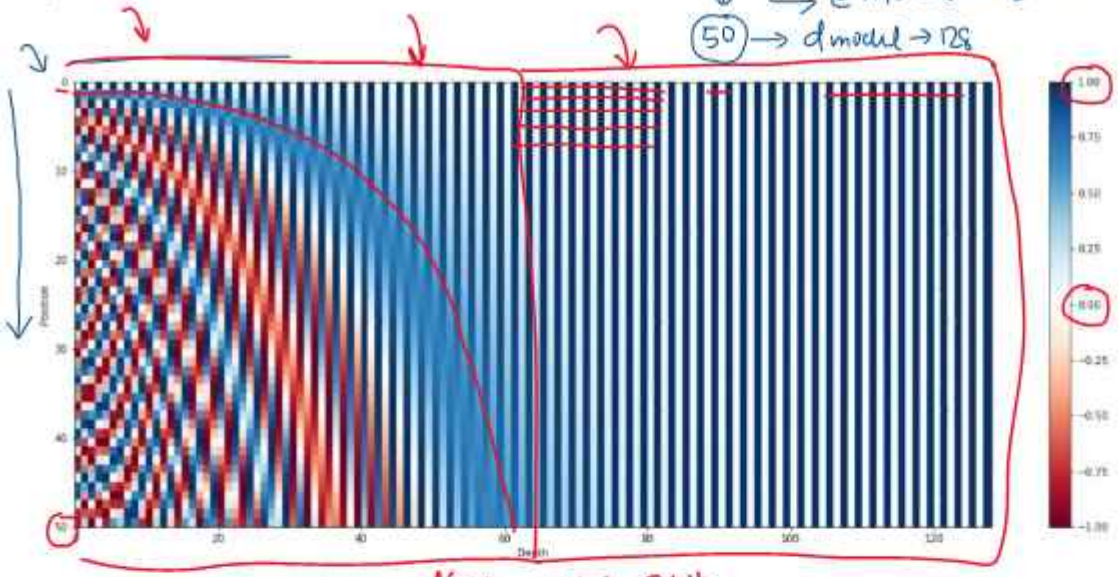


Interesting Observations

24 May 2024 01:06

sentence → 50 words → embedding → 128 dmodel = 128

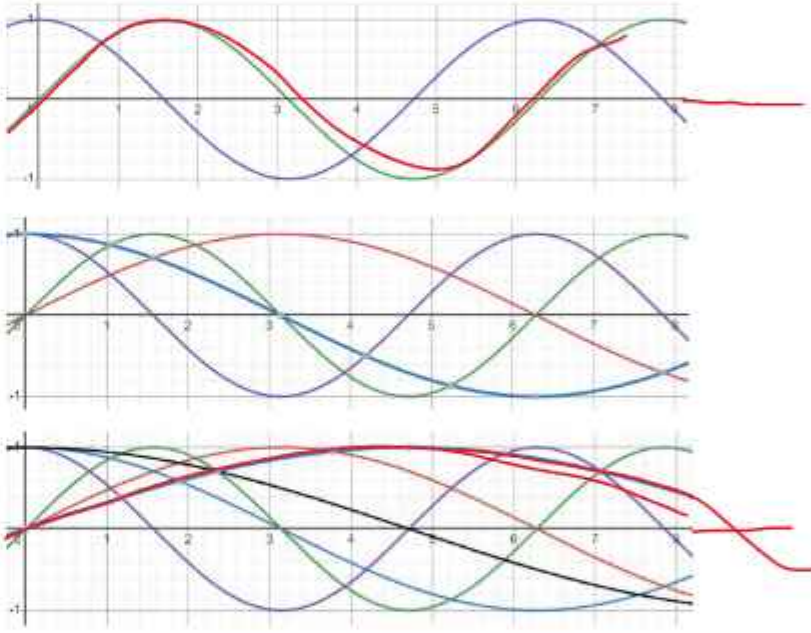
50 → dmodel → 128

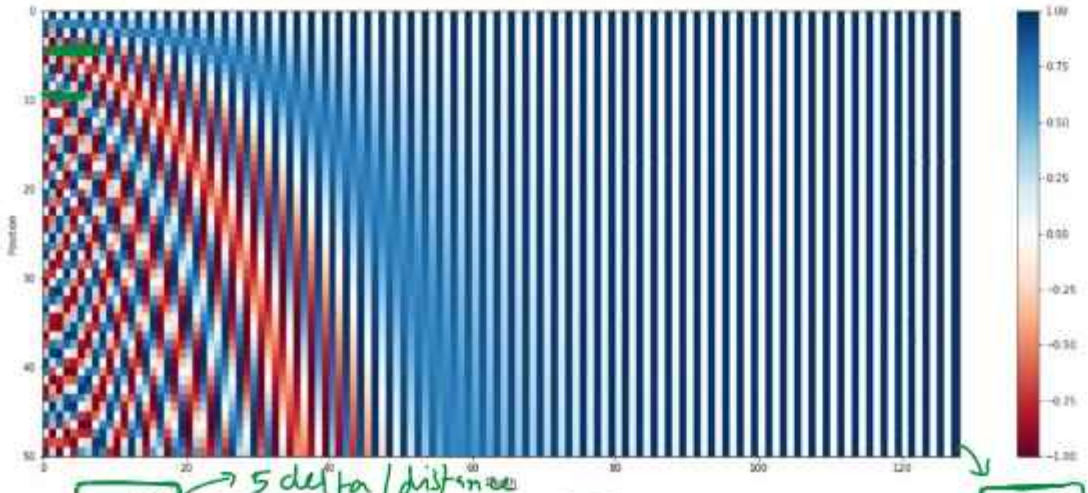


100

binary encoding 4bit 8bit

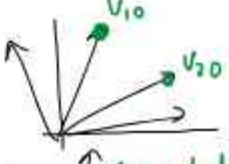
0:	0	0	0	0	8:	1	0	0	0
1:	0	0	0	1	9:	1	0	0	1
2:	0	0	1	0	10:	1	0	1	0
3:	0	0	1	1	11:	1	0	1	1
4:	0	1	0	0	12:	1	1	0	0
5:	0	1	0	1	13:	1	1	0	1
6:	0	1	1	0	14:	1	1	1	0
7:	0	1	1	1	15:	1	1	1	1



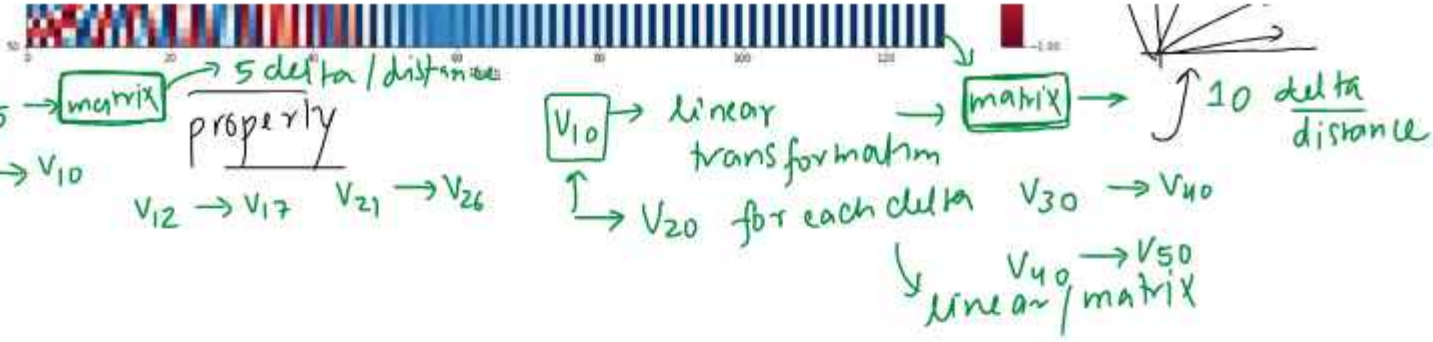


...mix → 5 delta / distance ... → ...nality

50 words
↓
50 vectors
↓
128 dim

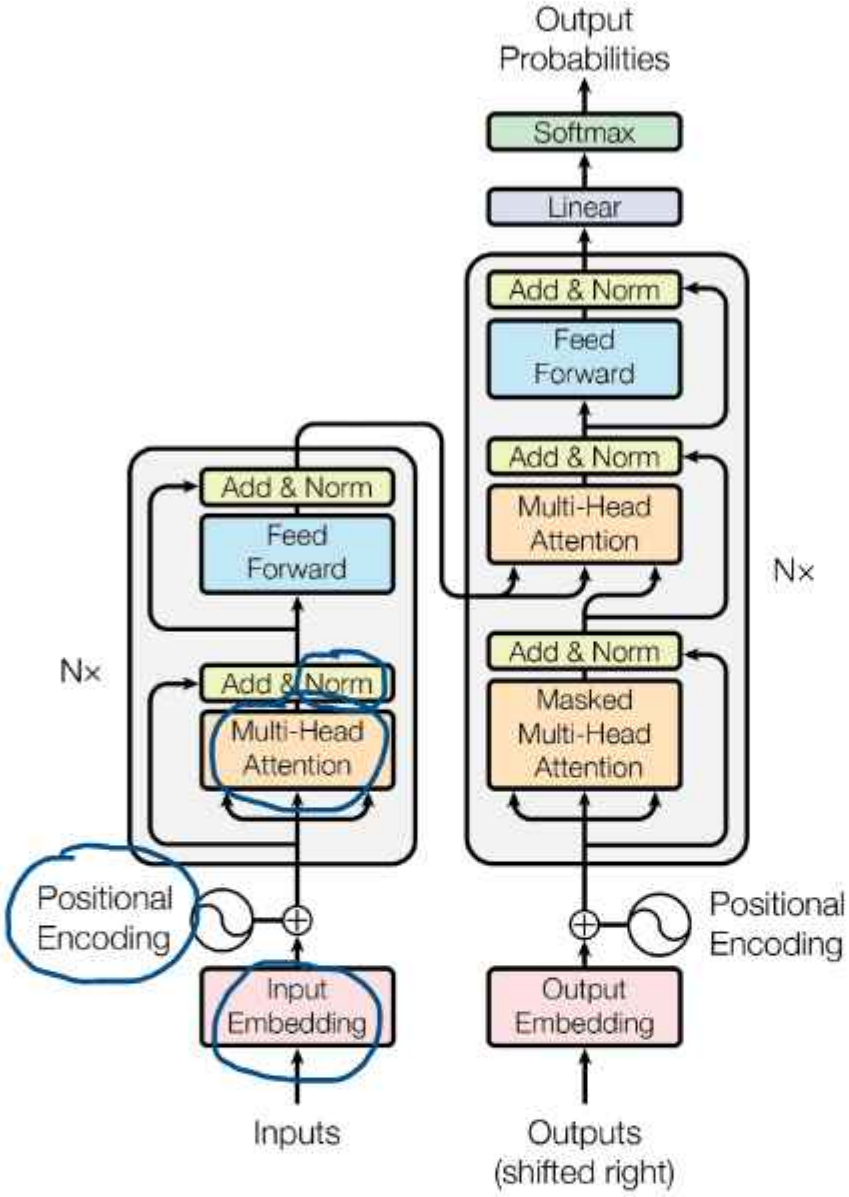


↑ 1 n delta


V_5 → matrix → 5 delta / distance
property
↳ V_{10}
$V_{12} \rightarrow V_{17}$ $V_{21} \rightarrow V_{26}$
V_{10} → linear transformation → matrix → ∫ 10 delta distance
↑ → V_{20} for each delta $V_{30} \rightarrow V_{40}$
↳ linear / matrix $V_{40} \rightarrow V_{50}$

Agenda

07 June 2024 02:03

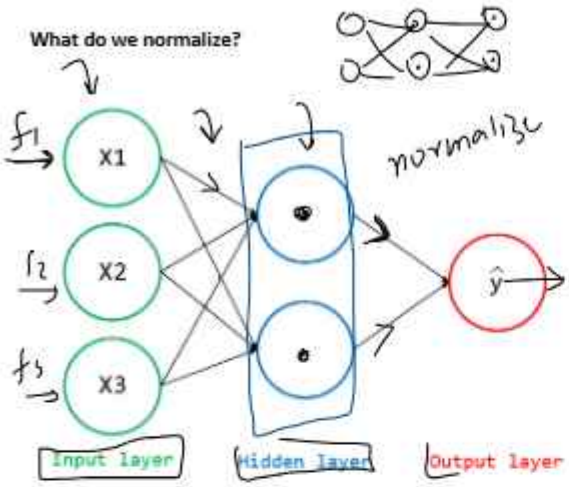


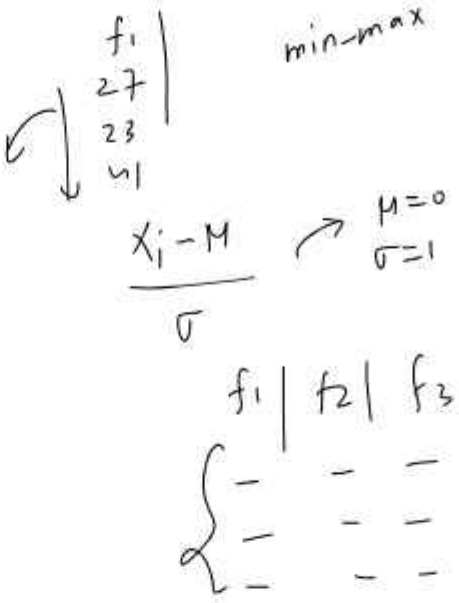
What is Normalization

05 June 2024 10:33

Normalization in deep learning refers to the process of transforming data or model outputs to have specific statistical properties, typically a mean of zero and a variance of one.

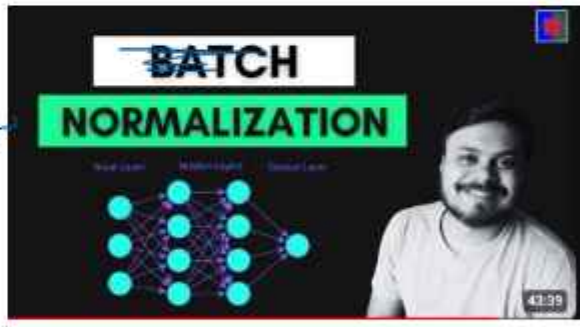
What do we normalize?

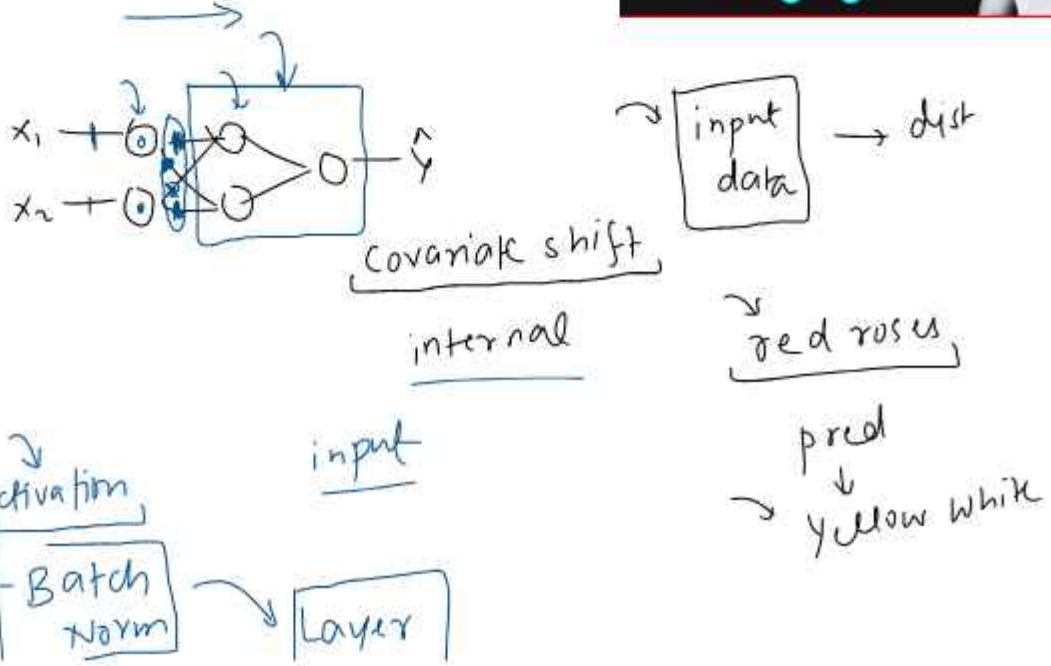


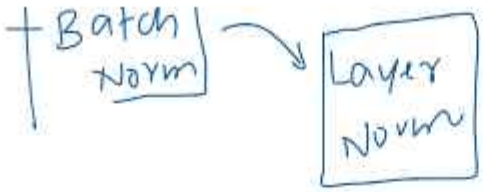


Benefits of Normalization in Deep Learning

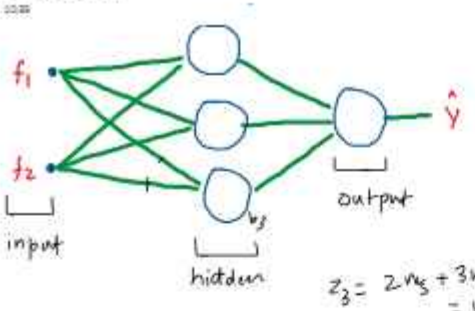
- Improved Training Stability:
  - Normalization helps to stabilize and accelerate the training process by reducing the likelihood of extreme values that can cause gradients to explode or vanish.
- Faster Convergence:
  - By normalizing inputs or activations, models can converge more quickly because the gradients have more consistent magnitudes. This allows for more stable updates during backpropagation.
- Mitigating Internal Covariate Shift:
  - Internal covariate shift refers to the change in the distribution of layer inputs during training. Normalization techniques, like batch normalization, help to reduce this shift, making the training process more robust.
- Regularization Effect:
  - Some normalization techniques, like batch normalization, introduce a slight regularizing effect by adding noise to the mini-batches during training. This can help to reduce overfitting.






Batch
Norm
Layer
Norm

Batch Norm(Revision)



f_1	f_2	z_1	z_2	z_3
2	3	7	5	4
1	1	2	3	4
5	4	1	2	3
6	1	7	5	6
7	1	3	3	4

$z_3 = 2w_5 + 3w_6 + b_3$
$= 4$

$z_1 = 2w_1 + 3w_2 + b_1 = 7$

$z_2 = 2w_3 + 3w_4 + b_2 = 5$

$\frac{7-\mu_1}{\sigma_1} = 0.56 \underset{(1)}{\gamma_1} + \underset{(0)}{\beta_1} = 0.86$

$\frac{2-\mu_1}{\sigma_1} = 0.71\gamma_1 + \beta_1 = 0.71$

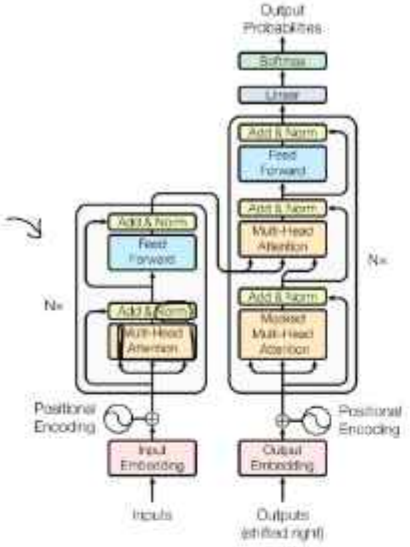
$\frac{5-\mu_2}{\sigma_2} = -0.21\gamma_2 + \beta_2 = -0.21$

$\frac{4-\mu_3}{\sigma_3} = 0.12\gamma_3 + \beta_3 = 0.12$

Why don't we use Batch Norm in Transformers?

09 June 2024 10:00

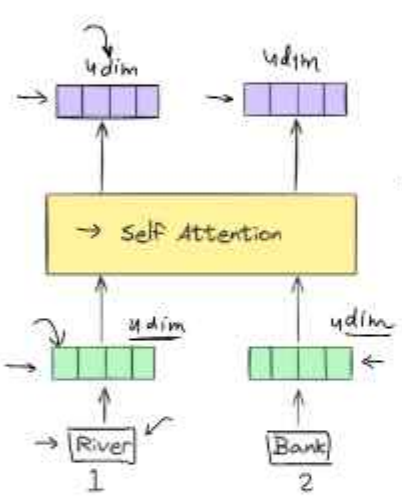
↳ self attention



→ Batch Norm

↑

Self - attention



Batch

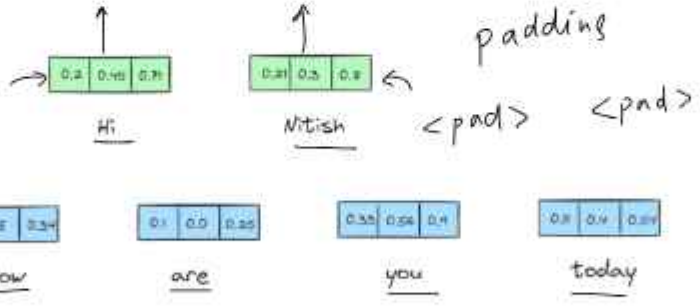
dim = 3

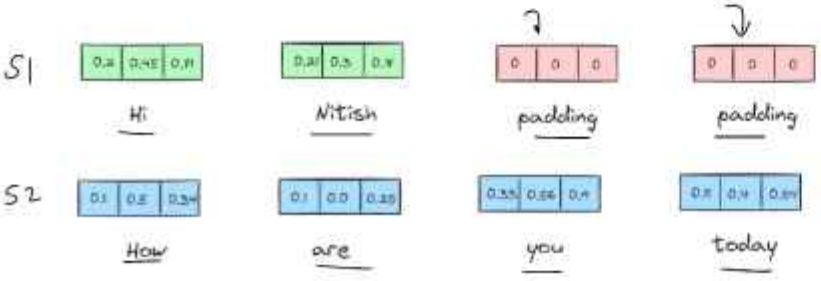
	Review	Sentiment
s1	Hi Nitish	1
s2	How are you today	0
s3	I am good	0
s4	You?	1

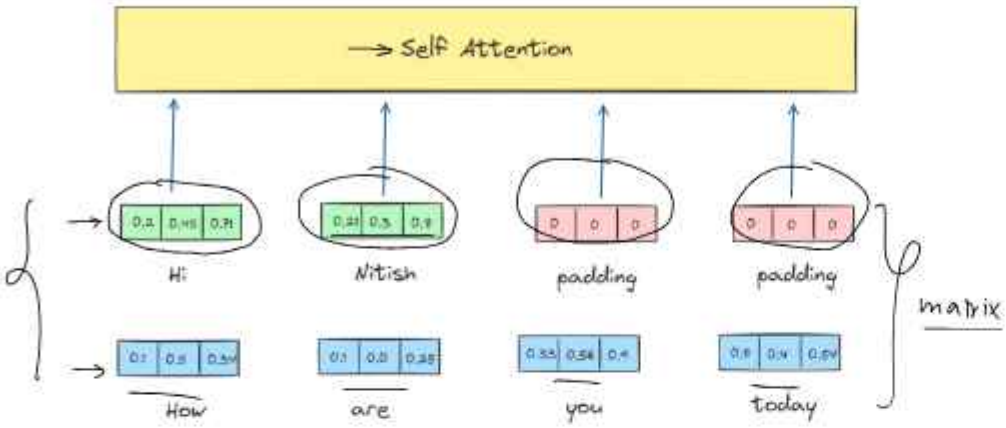
Embedding dimension - 3

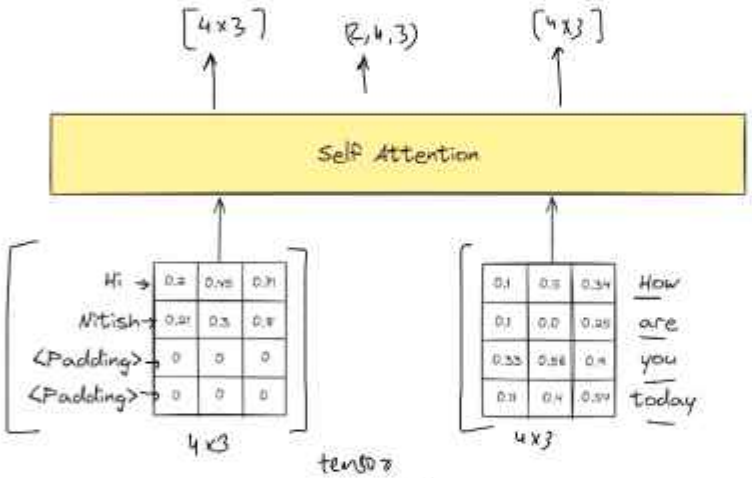
Batch Size - 2

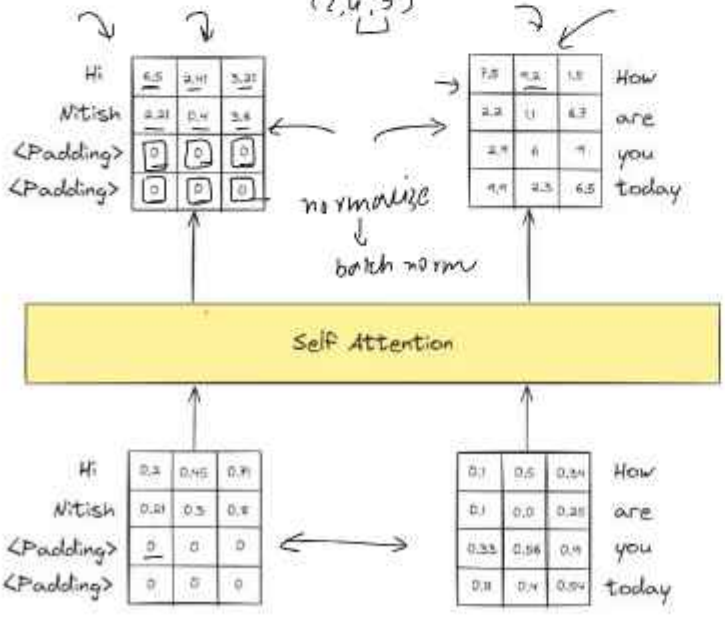
padding

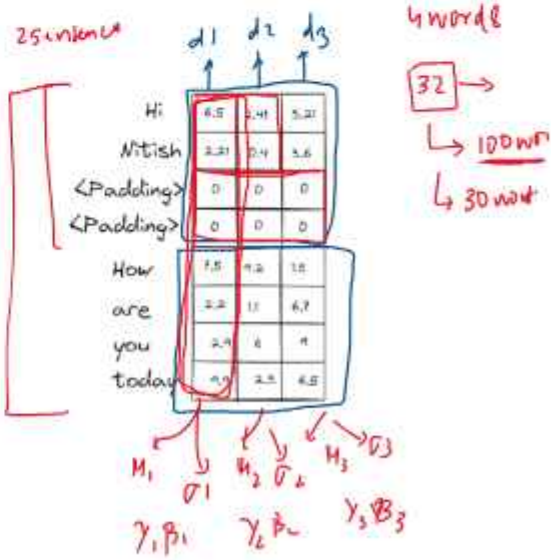





Self Attention
Hi
Nitish
padding
padding
matrix
How
are
you
today


[4x3]
(2,4,3)
[4x3]
Self Attention
Hi
Nitish
<Padding>
<Padding>
4x3
How
are
you
today
4x3

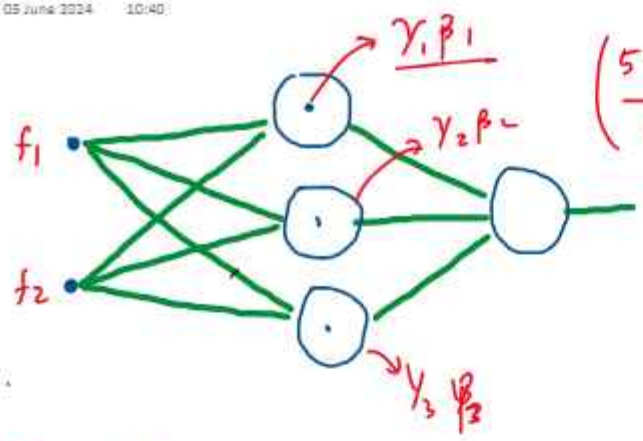

tensor
(2,4,3)
Hi
Nitish
<Padding>
<Padding>
How
are
you
today
normalize
batch norm
Self Attention
Hi
Nitish
<Padding>
<Padding>
How
are
you
today


25 instance
d1
d2
d3
4 words
32
100 words
30 words
Hi
Nitish
<Padding>
<Padding>
How
are
you
today
μ1
μ2
μ3
σ1
σ2
σ3
γ1 β1
γ2 β2
γ3 β3

Layer Norm

05 June 2024 10:40

across batch across feature



$\left(\frac{5-\mu_4}{\sigma_4}\right)\gamma_2+\beta_2$

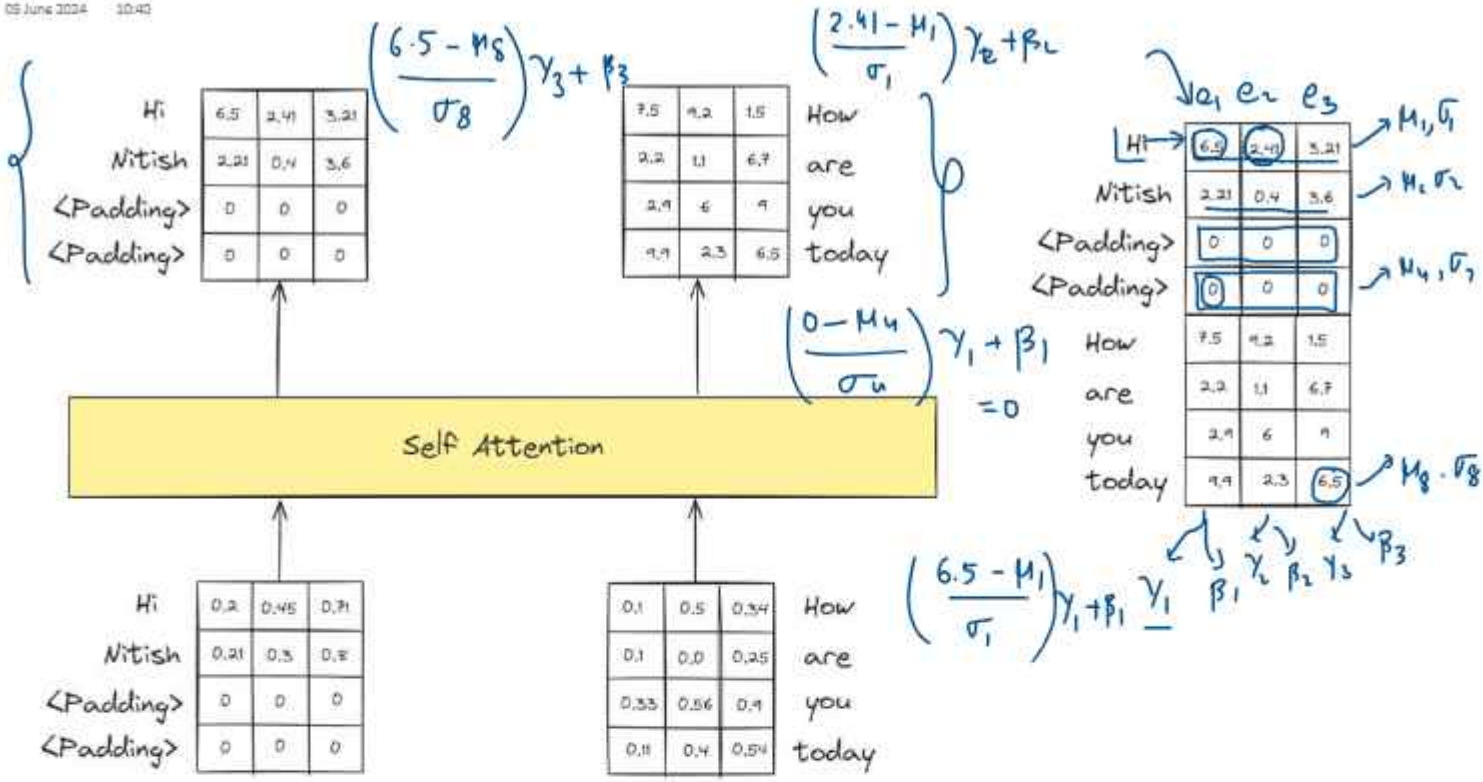
f_1	f_2	z_1	z_2	z_3	
2	3	7	5	4	μ_1, σ_1
1	1	2	3	4	μ_2, σ_2
5	4	1	2	3	
6	1	7	5	6	
7	1	3	3	4	μ_5, σ_5

$\left(\frac{2-\mu_2}{\sigma_2}\right)\gamma_1+\beta_1$ $\left(\frac{4-\mu_1}{\sigma_1}\right)\gamma_3+\beta_3$

$\left(\frac{5-\mu_1}{\sigma_1}\right)\gamma_2+\beta_2$ $\frac{7-\mu_1}{\sigma_1} = 0.3\gamma_1+\beta_1$

Layer Norm in Transformers

09 June 2024 10:40



Recap

09 July 2024 08:47



Introduction to Transformers | Transformers Part 1



What is Self Attention | Transformers Part 2 | CampusX



Self Attention in Transformers | Deep Learning | Simple Explanatio...



Scaled Dot Product Attention | Why do we scale Self Attention?



Self Attention Geometric Intuition | How to Visualize Self Attention |...



Why is Self Attention called "Self"? | Self Attention Vs Luong Attention i...



What is Multi-head Attention in Transformers | Multi-head Attentio...

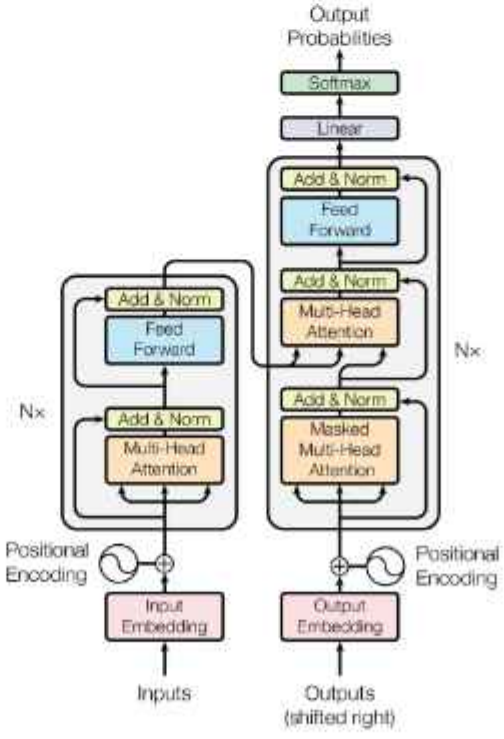


Positional Encoding in Transformers | Deep Learning | CampusX

7hrs

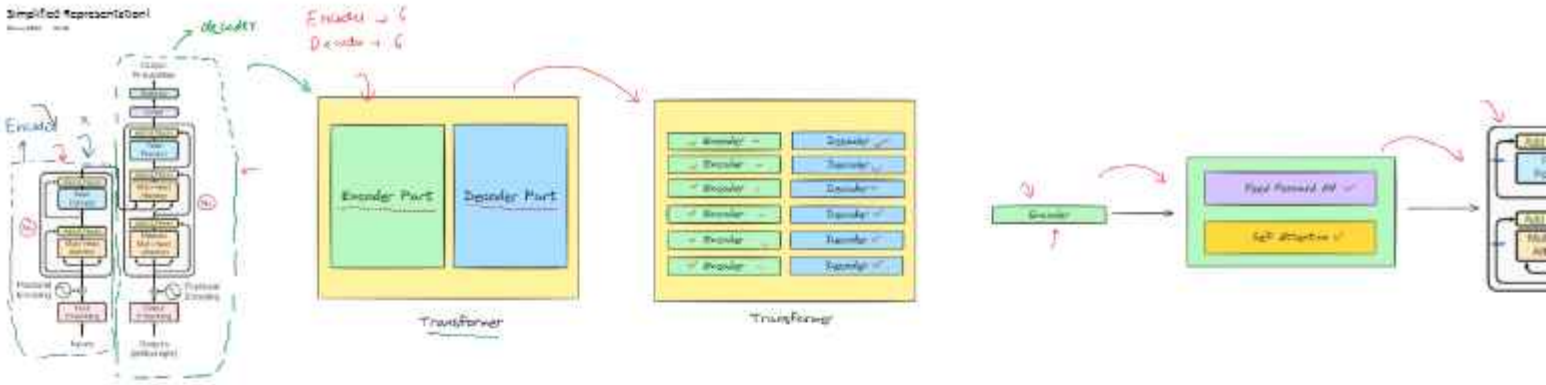


Layer Normalization in Transformers | Layer Norm Vs...



complex

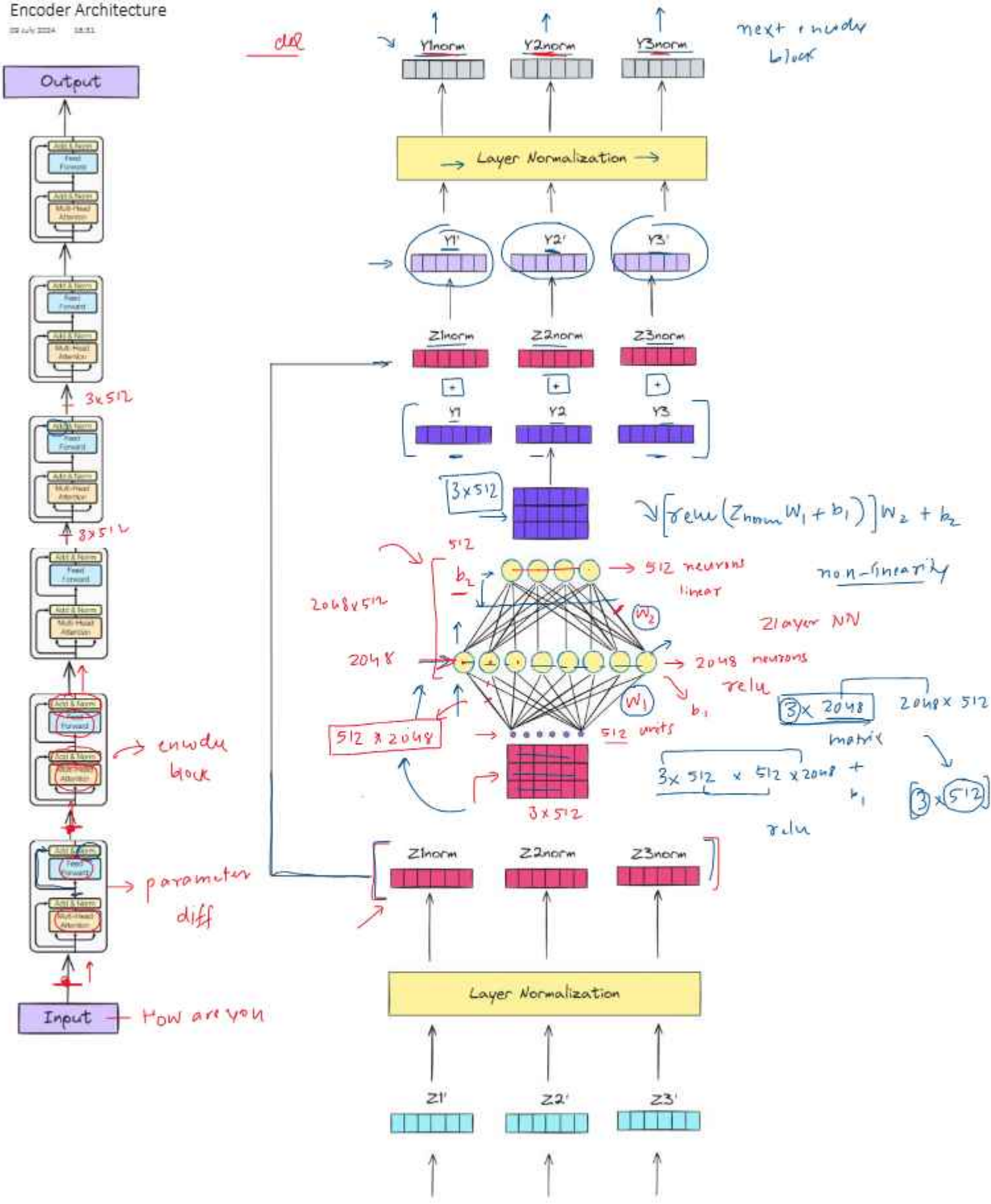
[encoder
Decode]

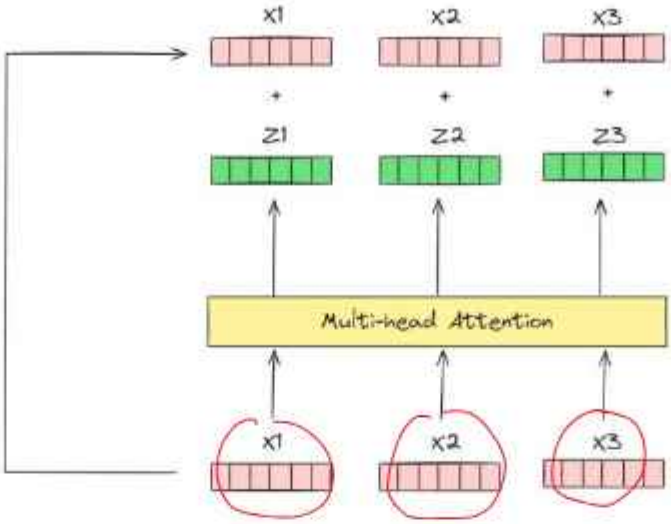

Simplified Representation!
decoder
Encoder → 6
Decoder → 6
Encoder
Encoder Part
Decoder Part
Transformer
Transformer
Encoder
Feed Forward NN
Self Attention

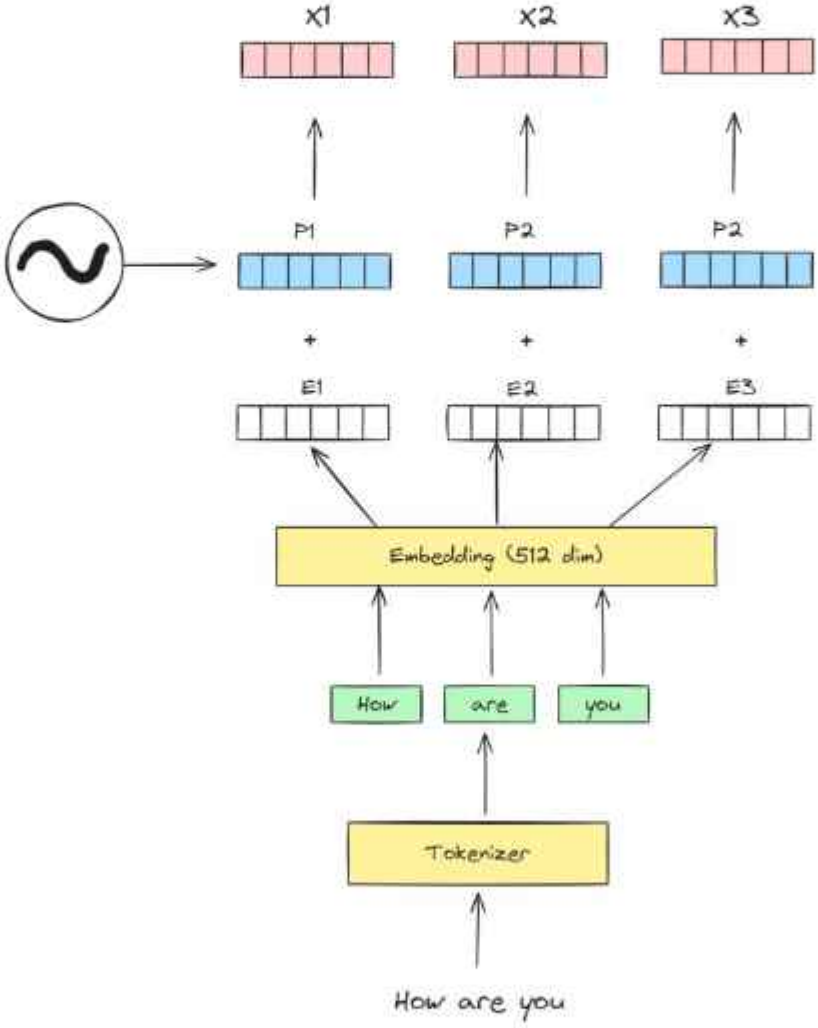




Encoder Architecture




X1
X2
X3
+
+
+
Z1
Z2
Z3
Multi-head Attention
X1
X2
X3


X1
X2
X3
P1
P2
P2
+
+
+
E1
E2
E3
Embedding (512 dim)
How
are
you
Tokenizer
How are you

Some questions

09 July 2024 20:48

1. Why use residual connections? → Kaggle → comment
2. Why use a FFNN?
3. Why use 6 encoder blocks?

language → complex

1) stable training — deep NN

ResNet

vanishing Grad

alternate

gradient flow

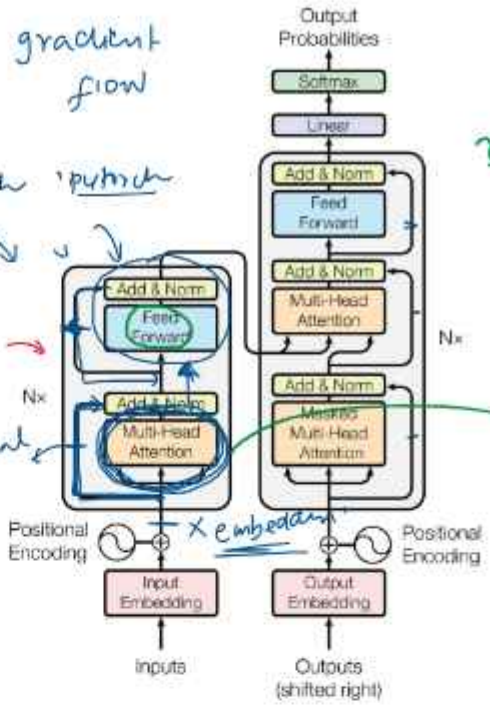
scratch python

relu

non-linearity

linear

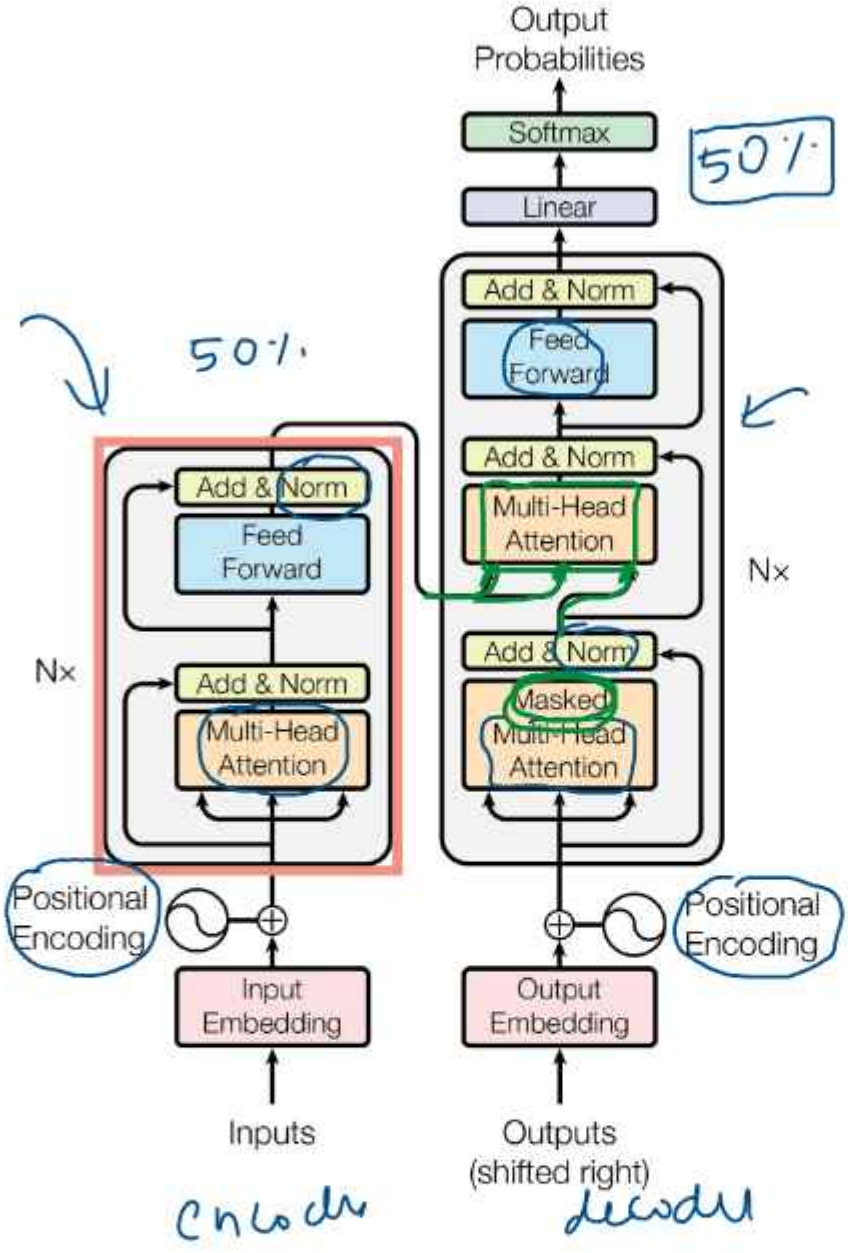
residual



2)

Recap

23 July 2024 17:09

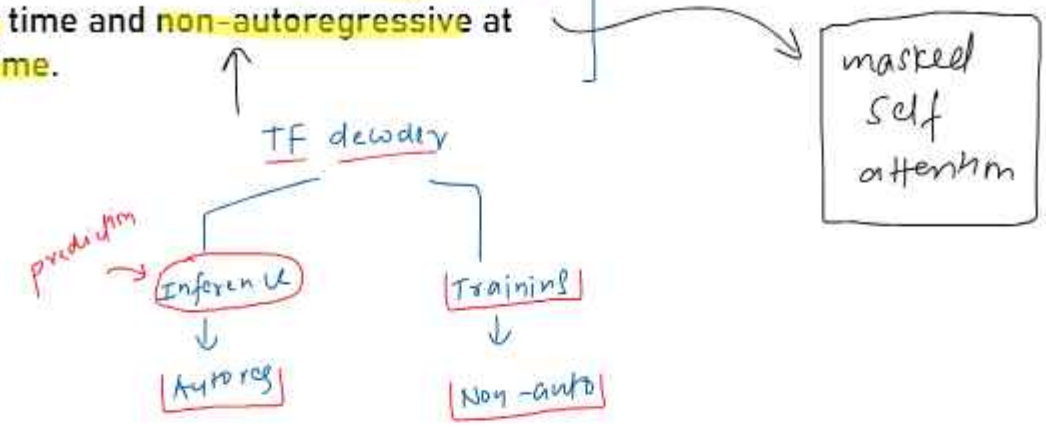


Autoregressive models

23 July 2024 17:59

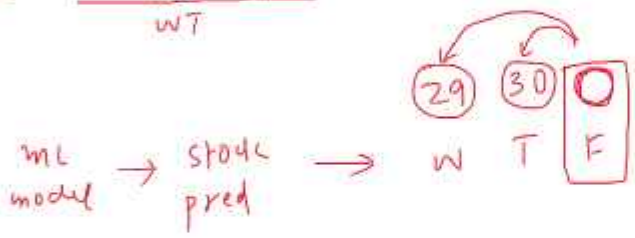
Economics → Time series

The Transformer decoder is autoregressive at inference time and non-autoregressive at training time.



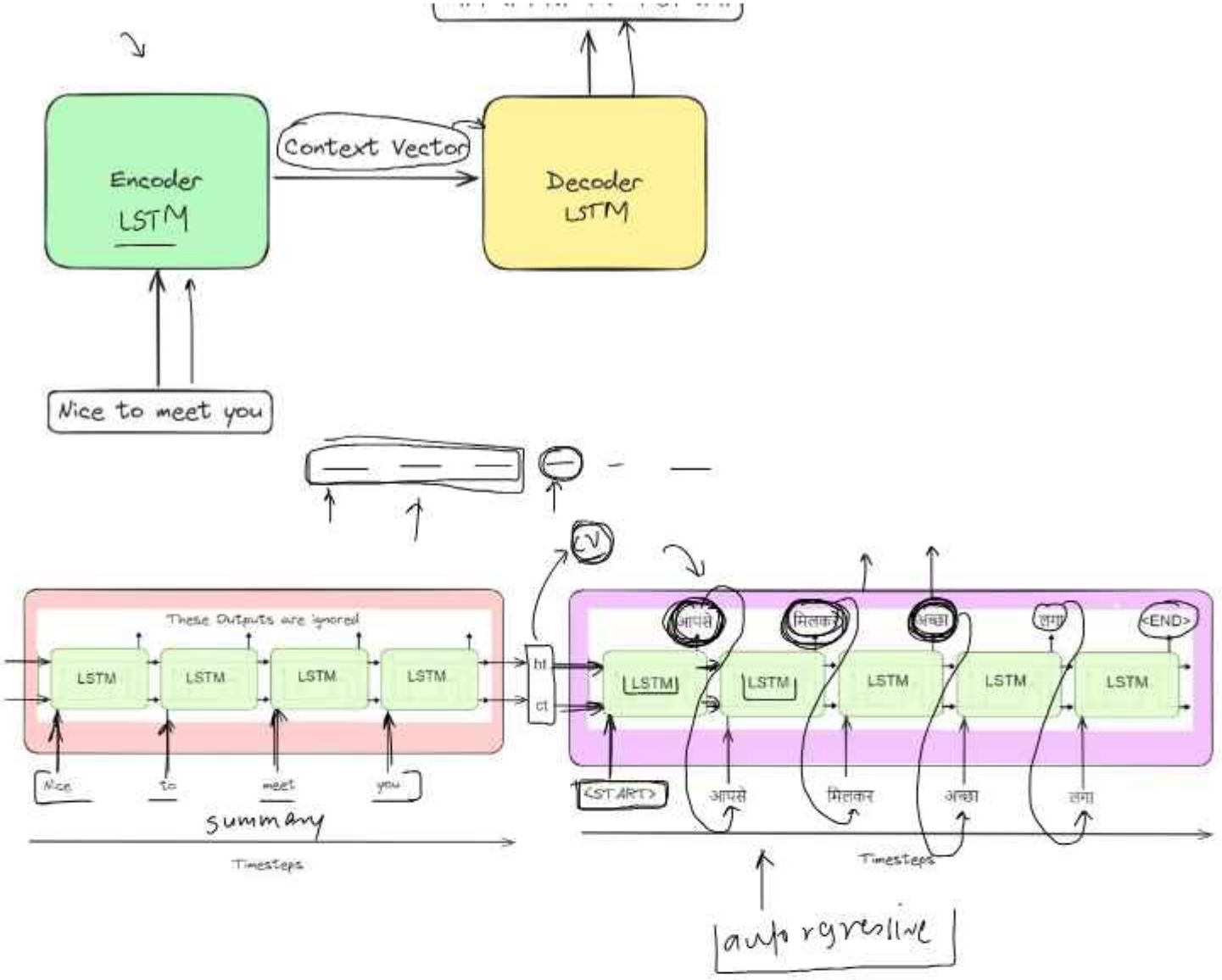
In the context of deep learning, autoregressive models are a class of models that generate data points in a sequence by conditioning each new point on the previously generated points. →

F WT



↳ Encoder - Decoder → Ilya




Encoder
LSTM
Context Vector
Decoder
LSTM
Nice to meet you
These Outputs are ignored
LSTM
LSTM
LSTM
LSTM
ht
ct
Nice
to
meet
you
summary
Timesteps
<START>
आपसे
मिलकर
अच्छा
लगा
<END>
Timesteps
auto regressive

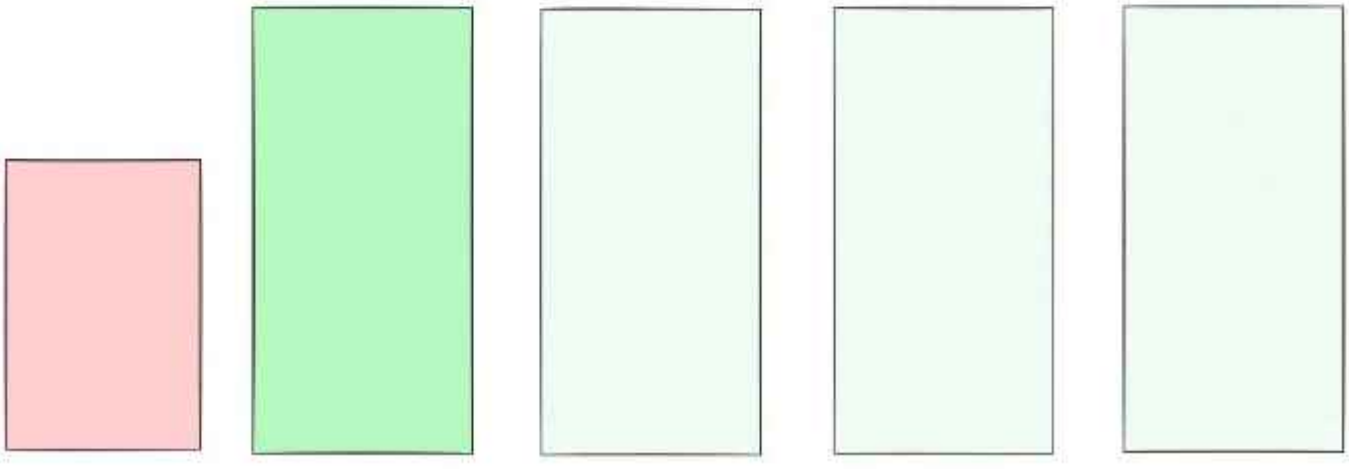
Transformer as an Autoregressive Model

23 July 2024 23:16

The Transformer decoder is autoregressive at inference time and non-autoregressive at training time.

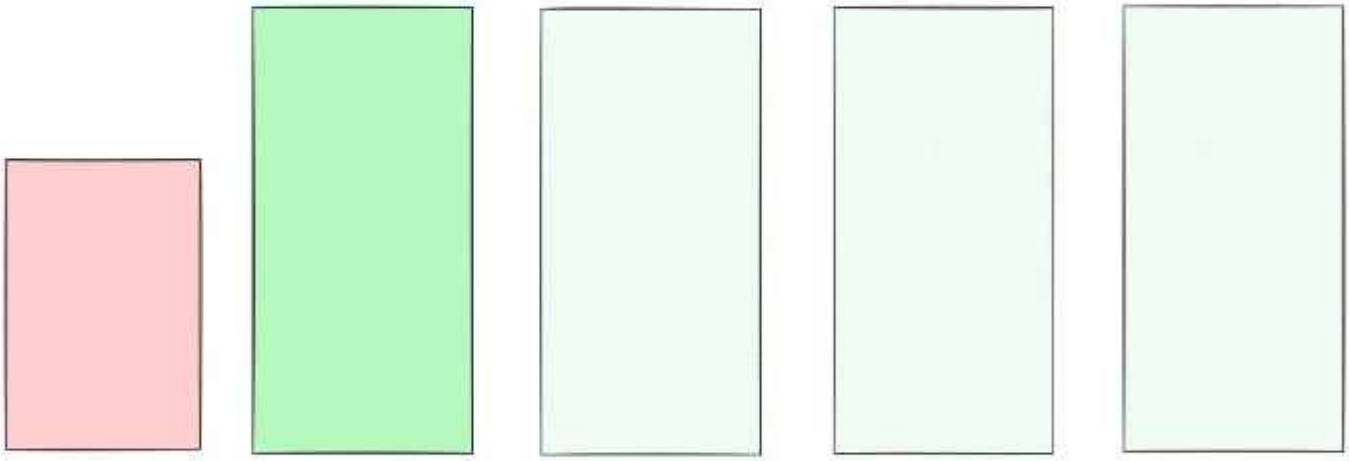
Inference

Query Sentence -> I am fine



Training

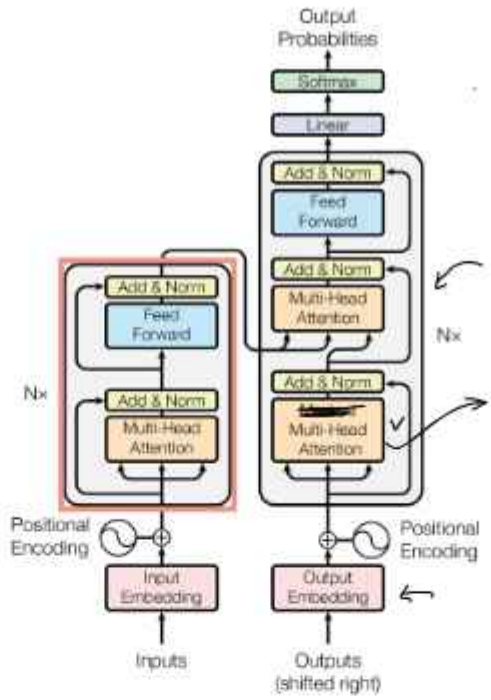
S.No	English Sentence	Hindi Sentence
1	How are you?	आप कैसे हैं
2	Congratulations	बधाई हो
3	Thank you	धन्यवाद



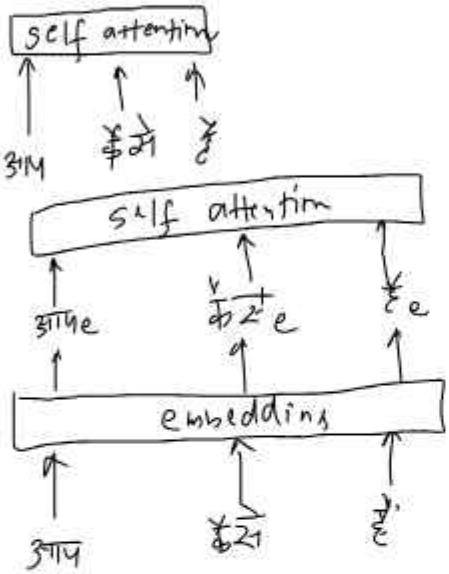
The problem in parallelizing

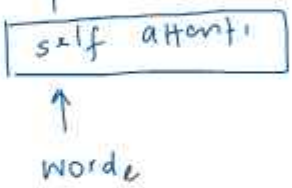
25 July 2024 22:55

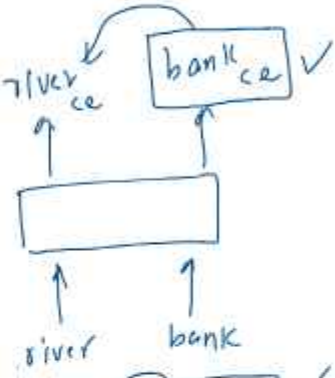
S.No	English Sentence	Hindi Sentence
1	How are you?	आप कैसे हैं
2	Congratulations	बधाई हो
3	Thank you	धन्यवाद

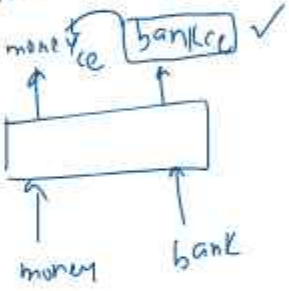


self attention







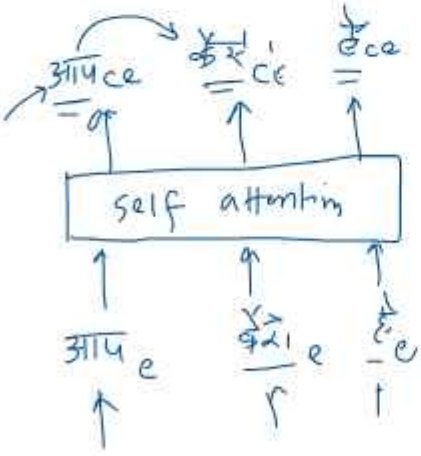


river bank

money bank

Data leakage

आप	कैसे	हैं



$\text{आप}_{ce} = 0.8\,\text{आप}_e + 0.1\,\text{कैसे}_e + 0.1\,\text{हैं}_e$

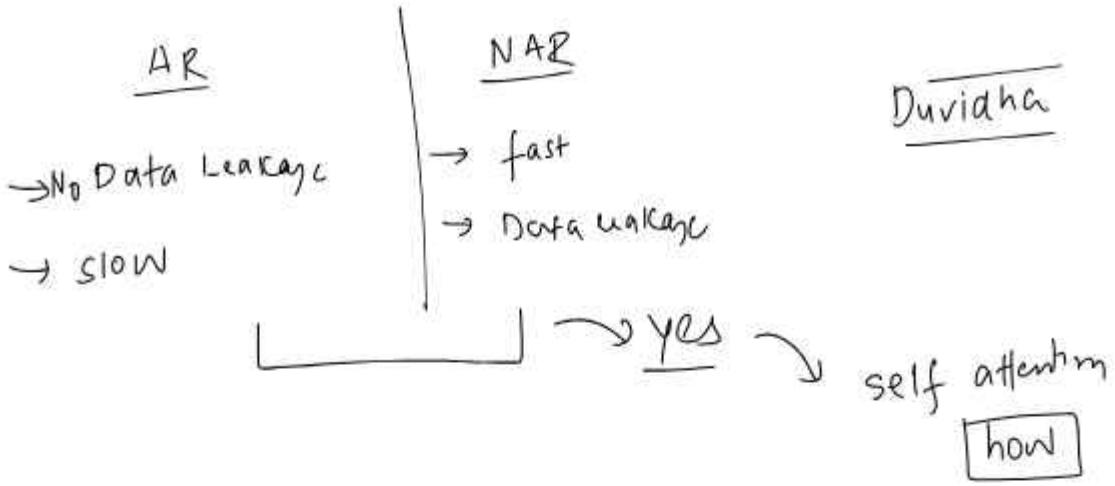
$\text{कैसे}_{ce} = 0.15\,\text{आप}_e + 0.75\,\text{कैसे}_e + 0.1\,\text{हैं}_e$

$\text{हैं}_{ce} = 0.1\,\text{आप}_e + 0.2\,\text{कैसे}_e + 0.7\,\text{हैं}_e$

current token value

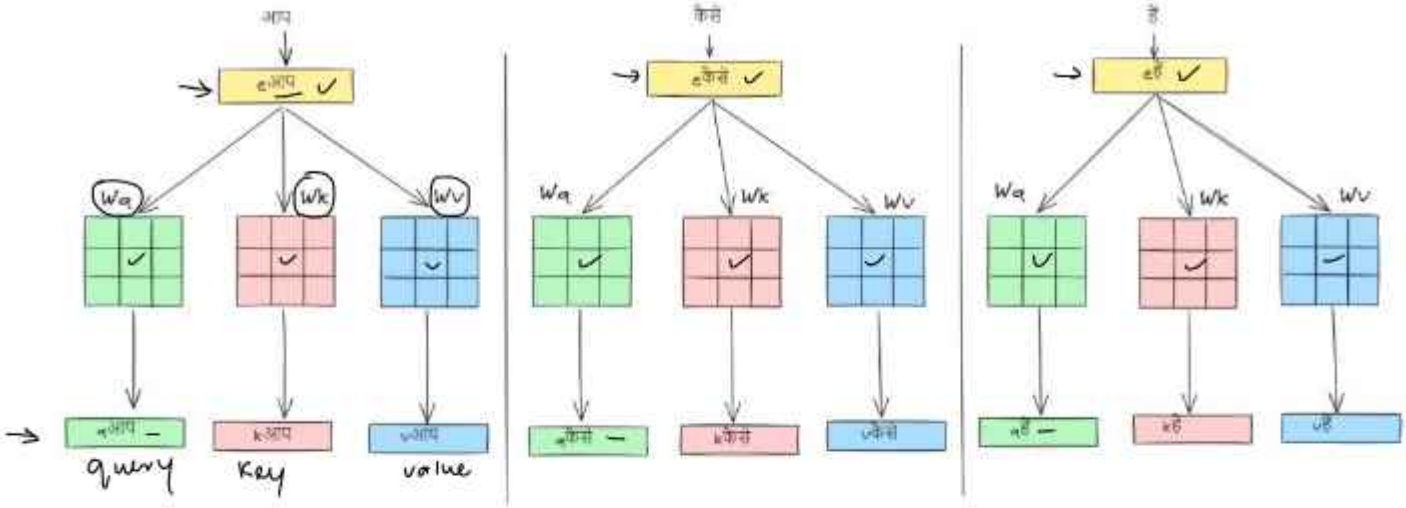
↓

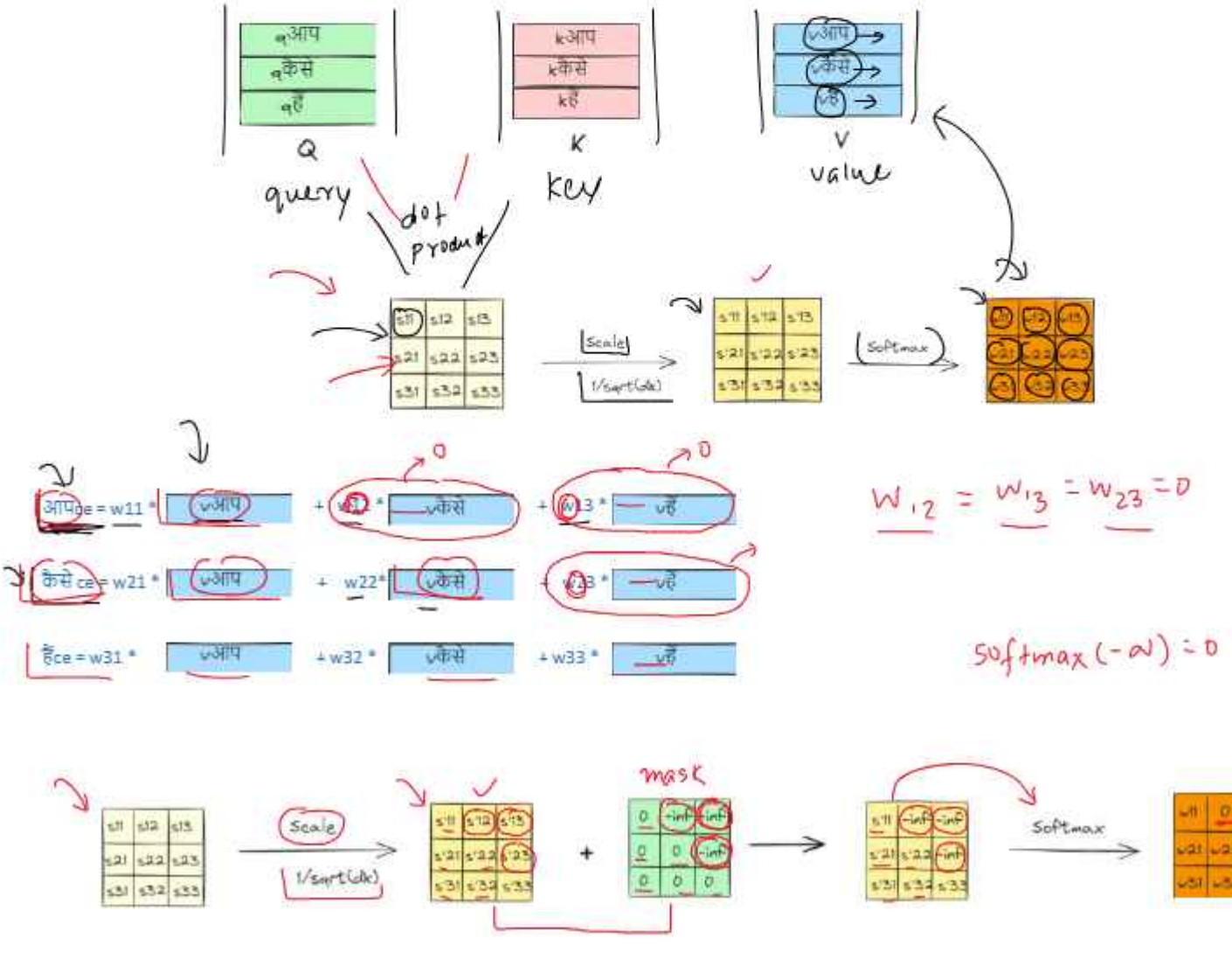
future token value


AR
NAR
Duvidha
→ No Data Leakage
→ slow
→ fast
→ Data Leakage
yes
self attention
how

Finding the answer

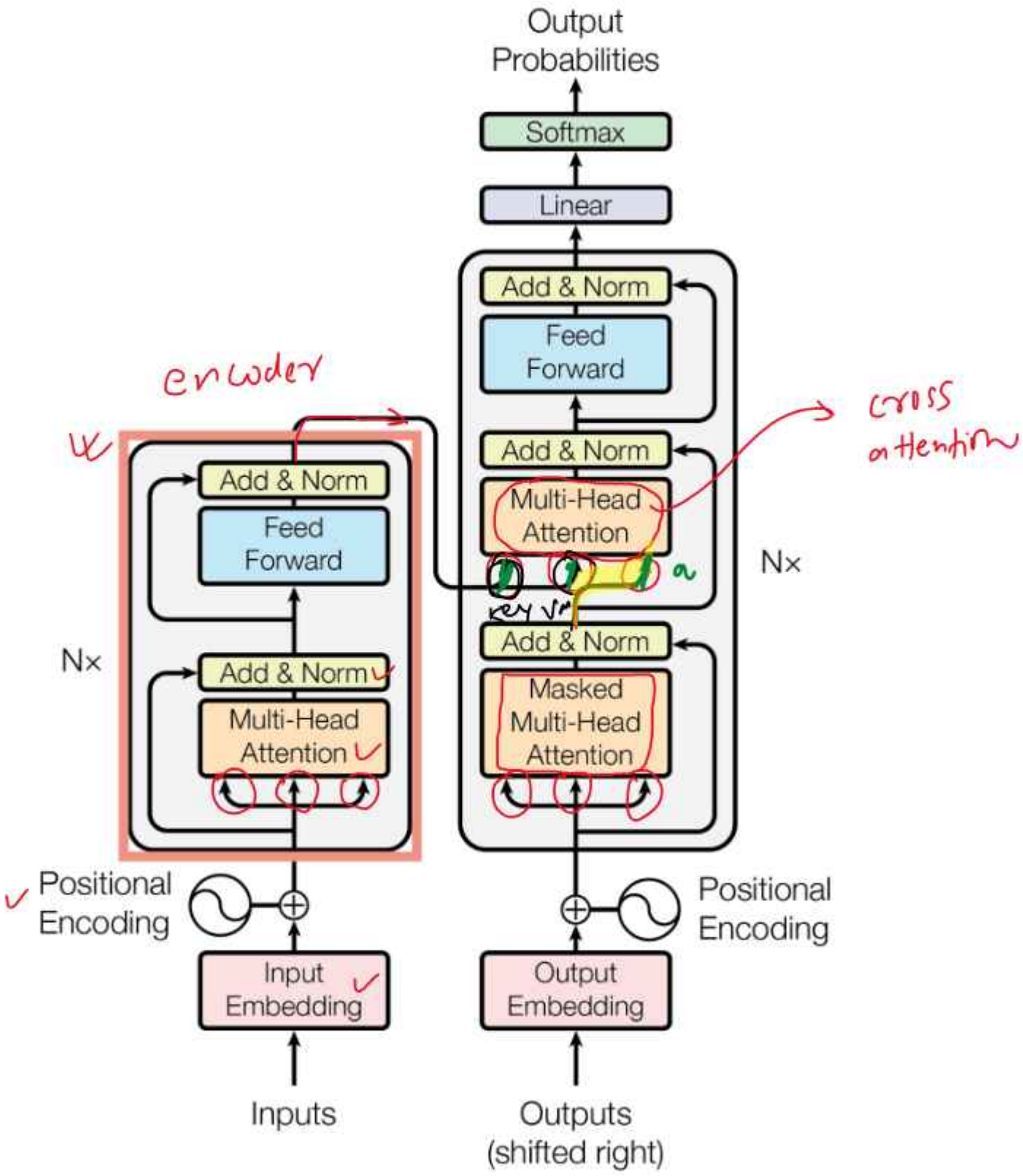
26 July 2024 00:21





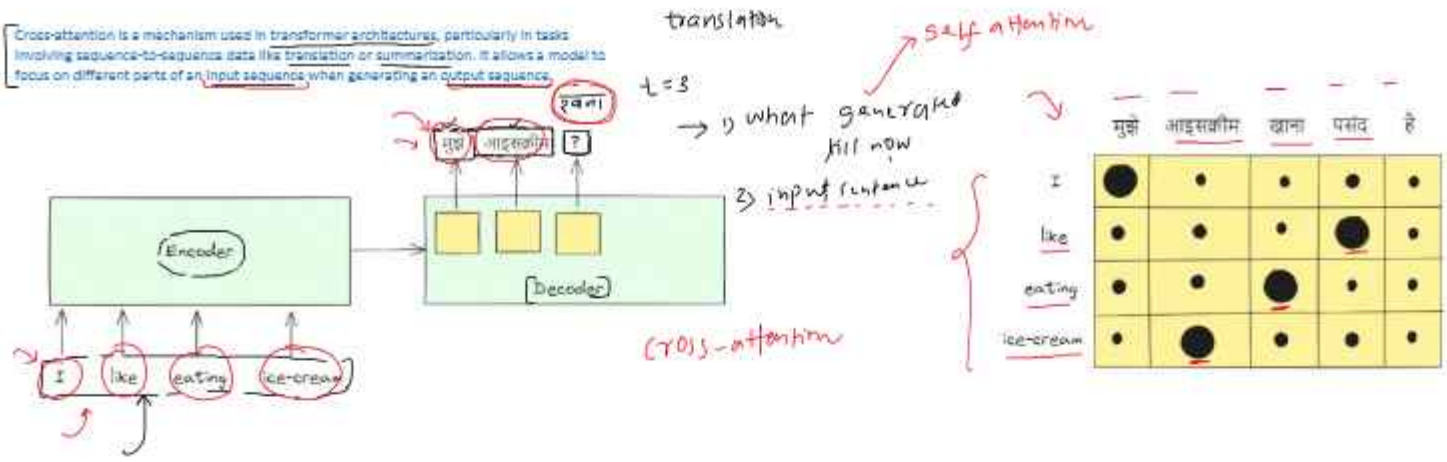
Plan of Action

12 August 2024 17:53



What is Cross Attention

Cross-attention is a mechanism used in transformer architectures, particularly in tasks involving sequence-to-sequence data like translation or summarization. It allows a model to focus on different parts of an input sequence when generating an output sequence.



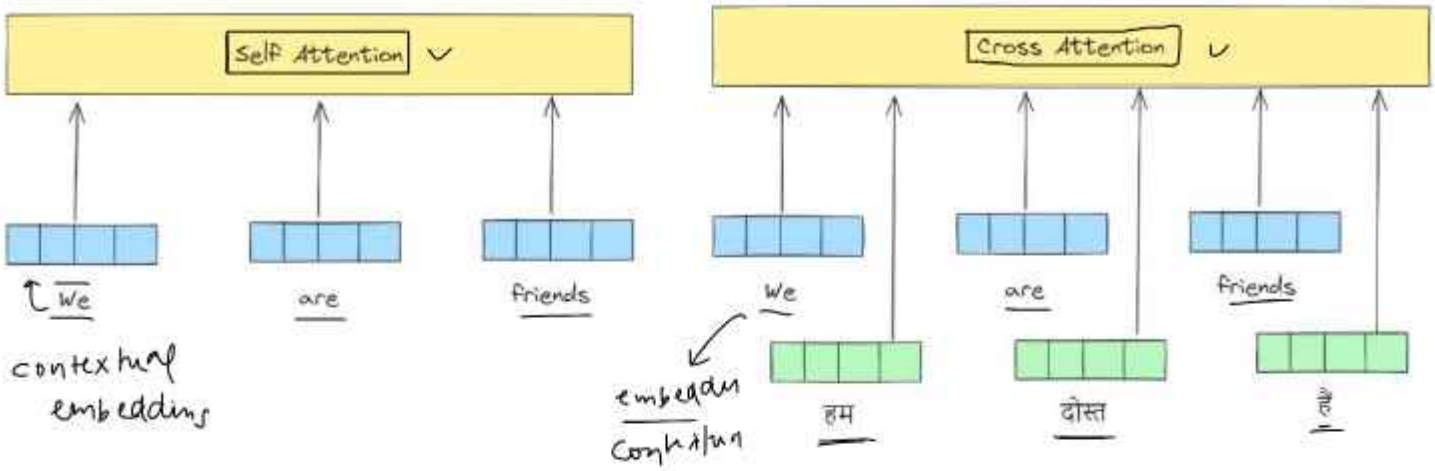
Cross Attention is conceptually very similar to Self-Attention

Self-Attention Vs Cross Attention

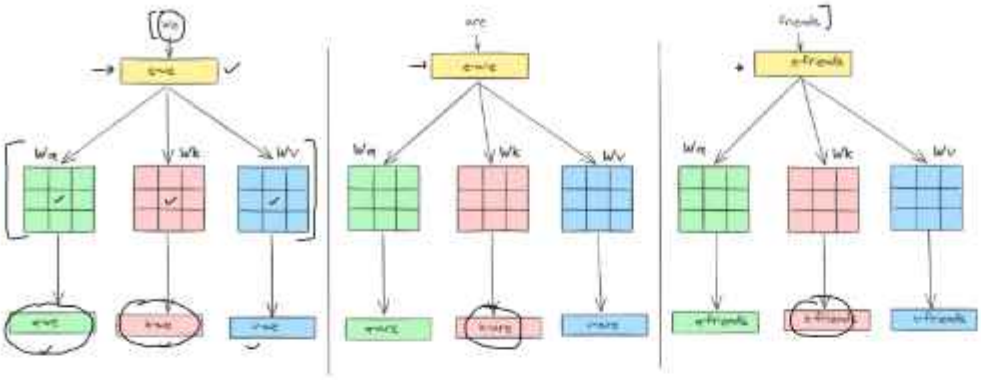
1. The input
2. The processing
3. The output

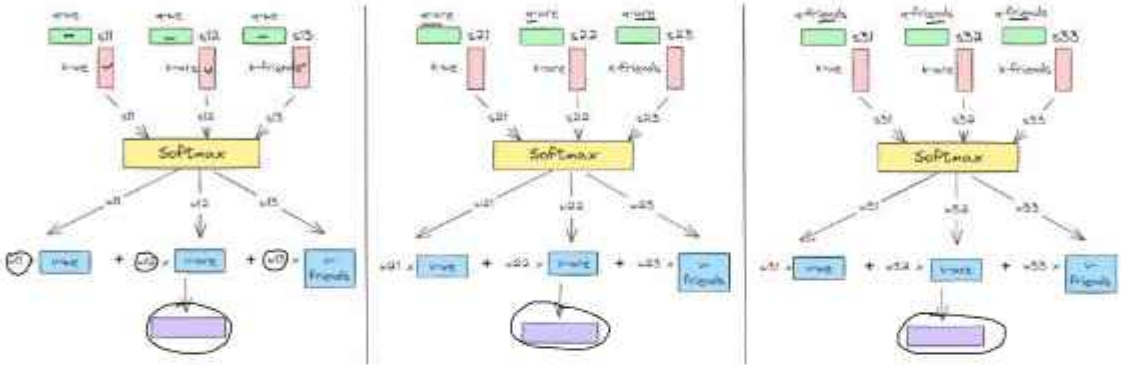
Self-Attention Vs Cross Attention (Input)

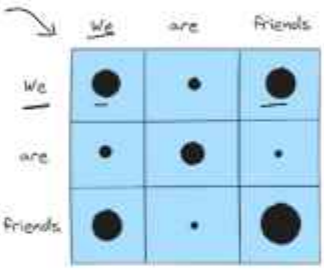
13 August 2024 08:22



Self-Attention Vs Cross Attention (Processing)

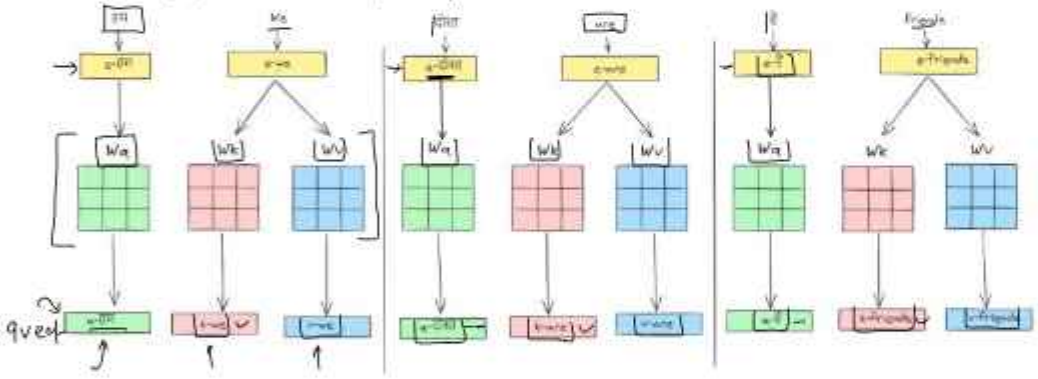


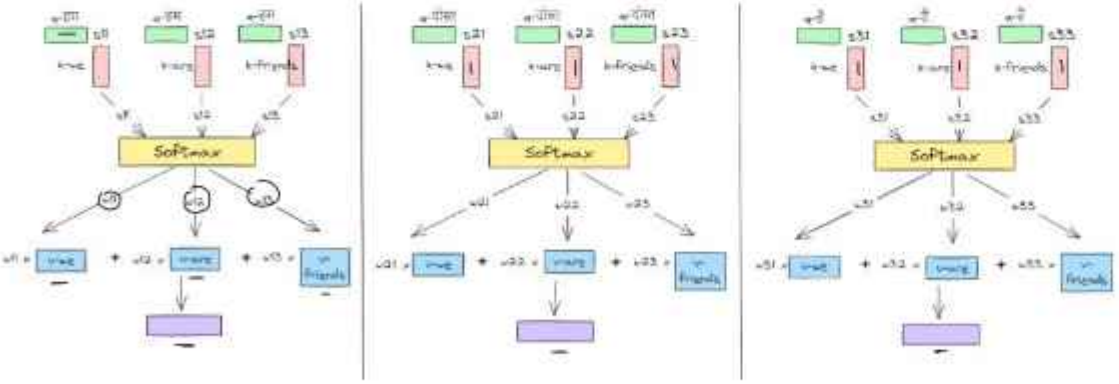


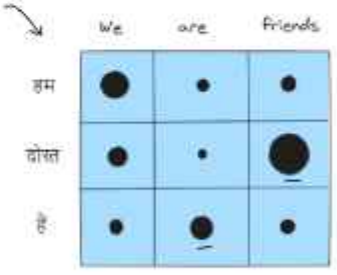


query → output seq
key and value → input seq

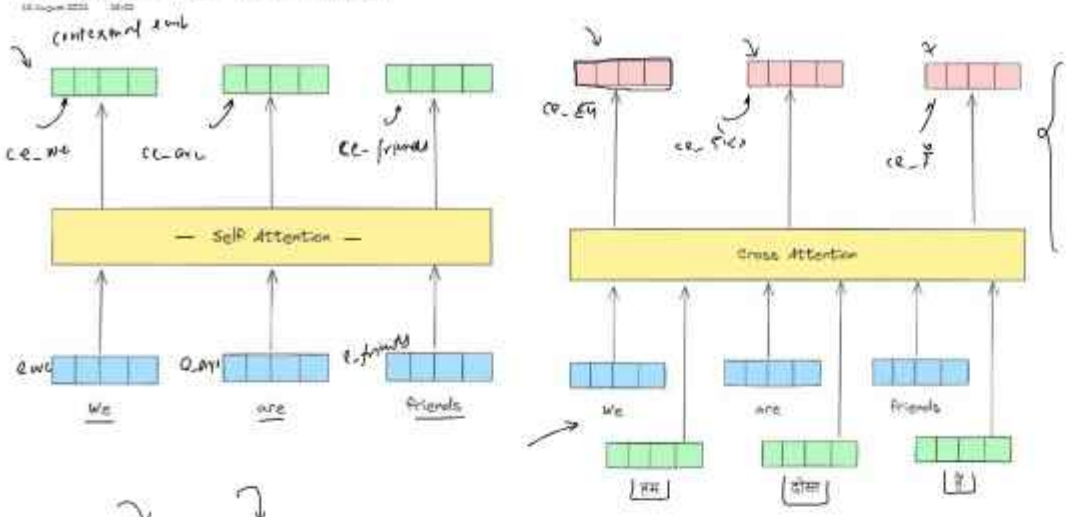
Chg_mind







Self-Attention Vs Cross Attention [Output]



$$ce_we = 0.8 \times e_we + 0.1 \times e_are + 0.1 \times e_friends$$

$$ce_are = 0.15 \times e_we + 0.75 \times e_are + 0.1 \times e_friends$$

$$ce_friends = 0.2 \times e_we + 0.1 \times e_are + 0.7 \times e_friends$$

$$ce_\text{हम} = 0.5 \times e_we + 0.3 \times e_are + 0.2 \times e_friends$$

$$ce_\text{दोस्त} = 0.2 \times e_we + 0.2 \times e_are + 0.6 \times e_friends$$

$$ce_\text{हैं} = 0.3 \times e_we + 0.4 \times e_are + 0.3 \times e_friends$$

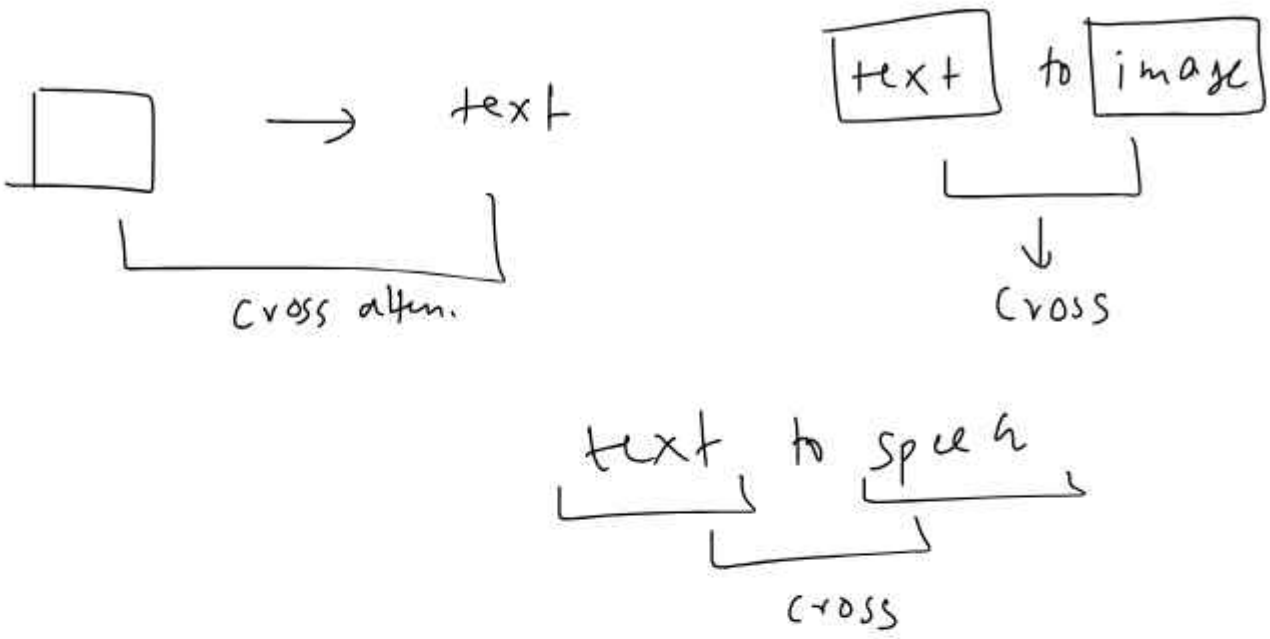
Cross Attention Vs Bahdanau/Luong Attention

12 August 2024 18:06

Use-cases

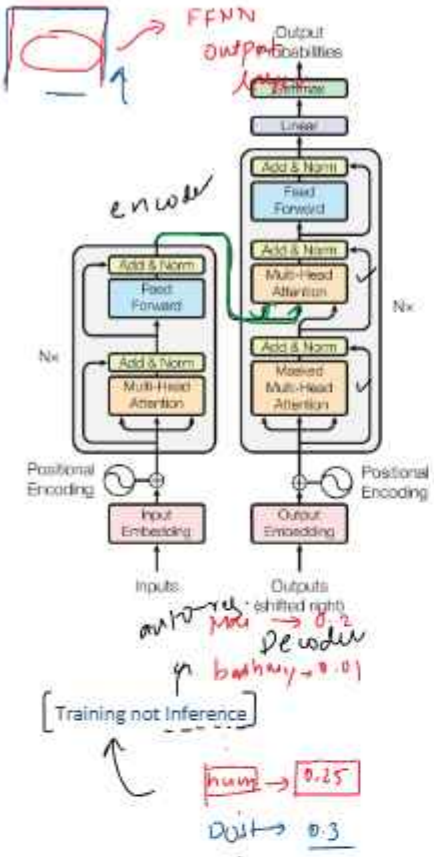
12 August 2024 18:07

image captin



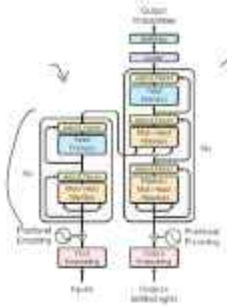
Plan of Attack

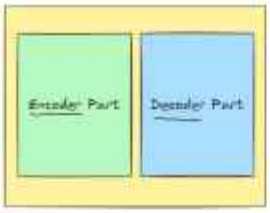
22 August 2024 15:58





Simplified view

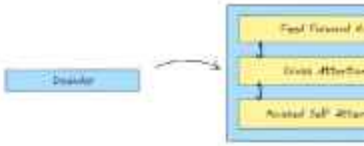


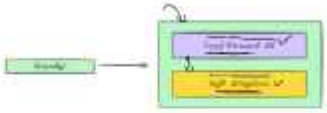


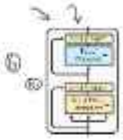
Transformer



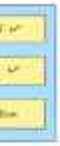
Transformer

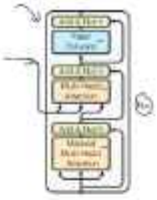






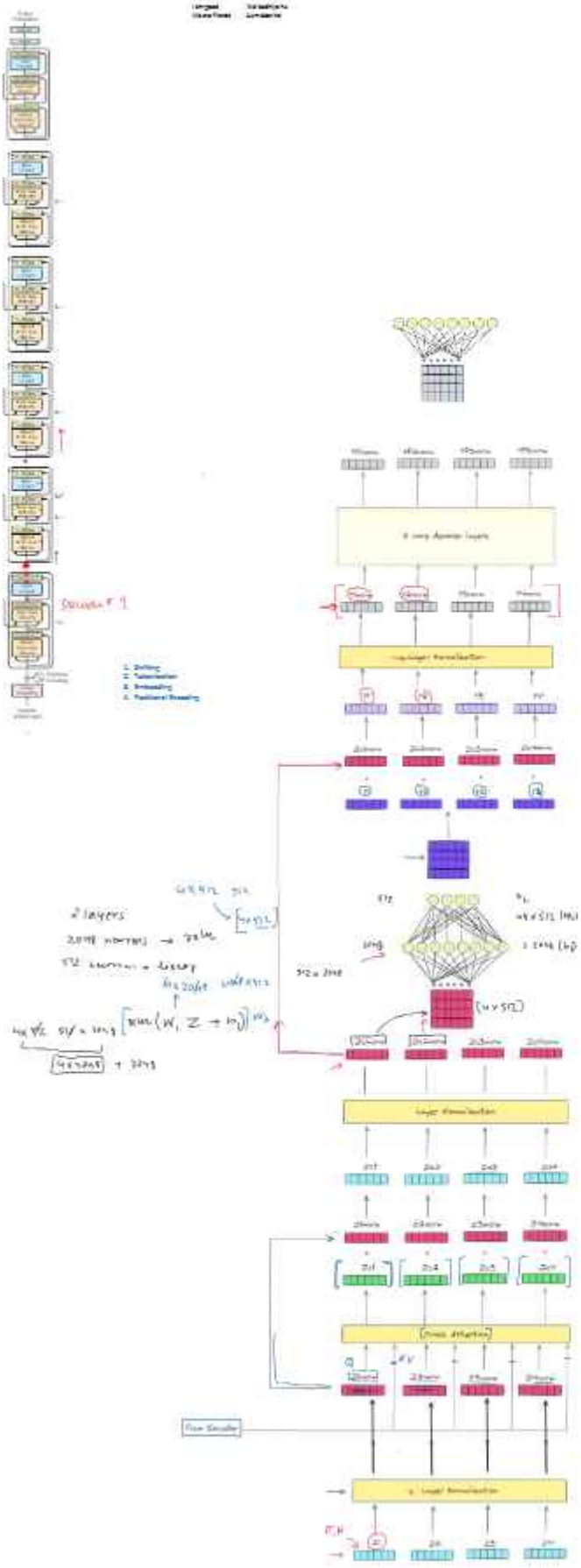


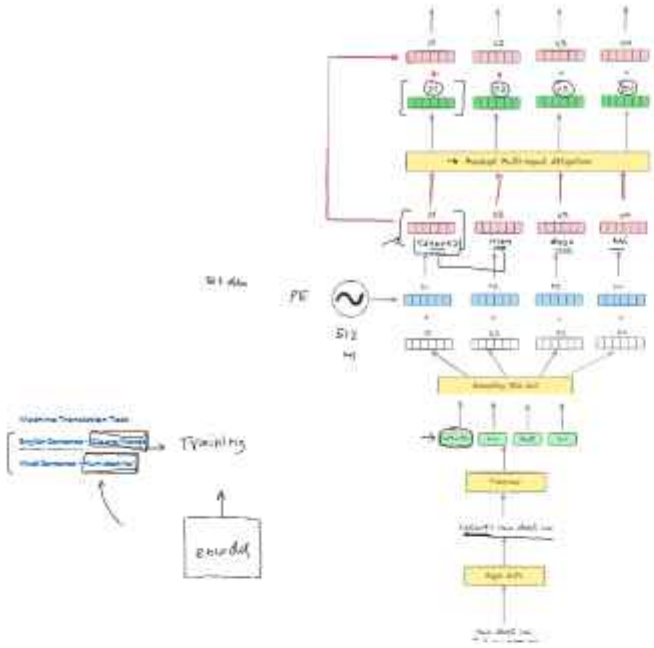






Decoder Architecture

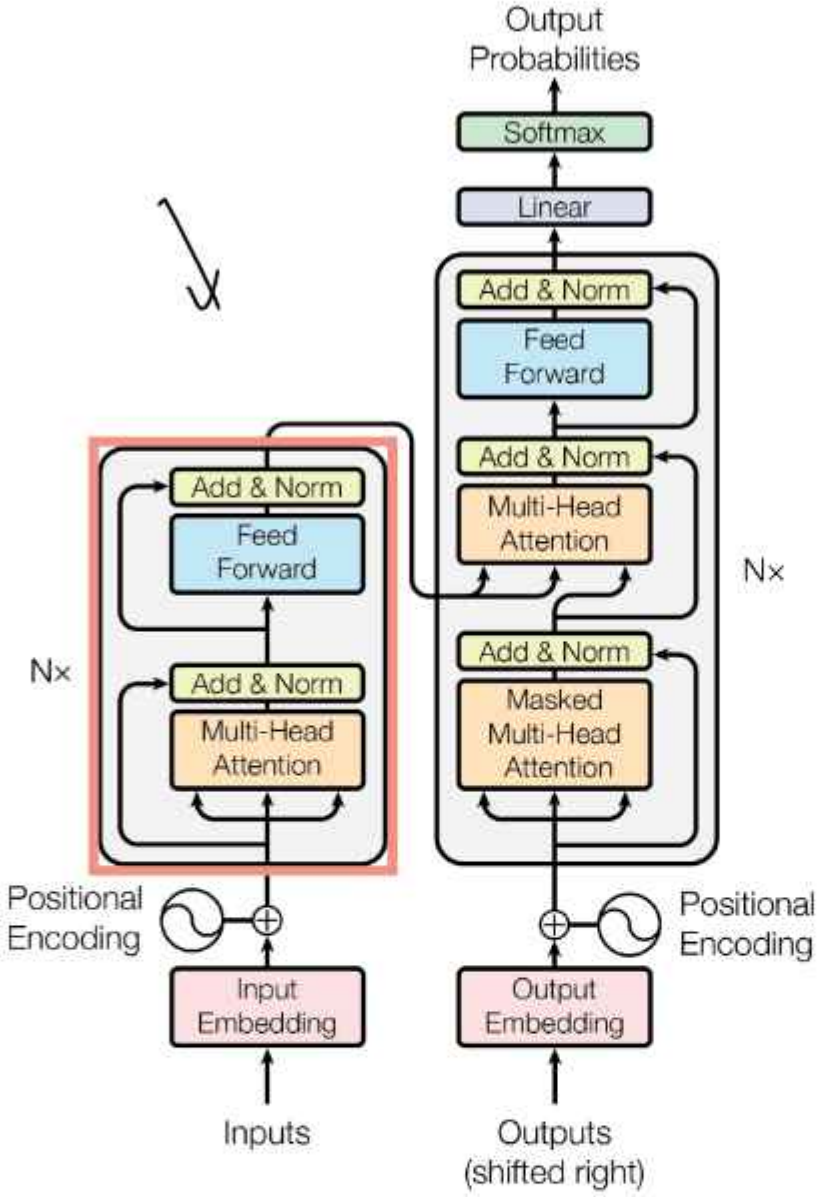



PE
Training
embed

Plan of Attack

03 September 2024 00:37

training



The Setup

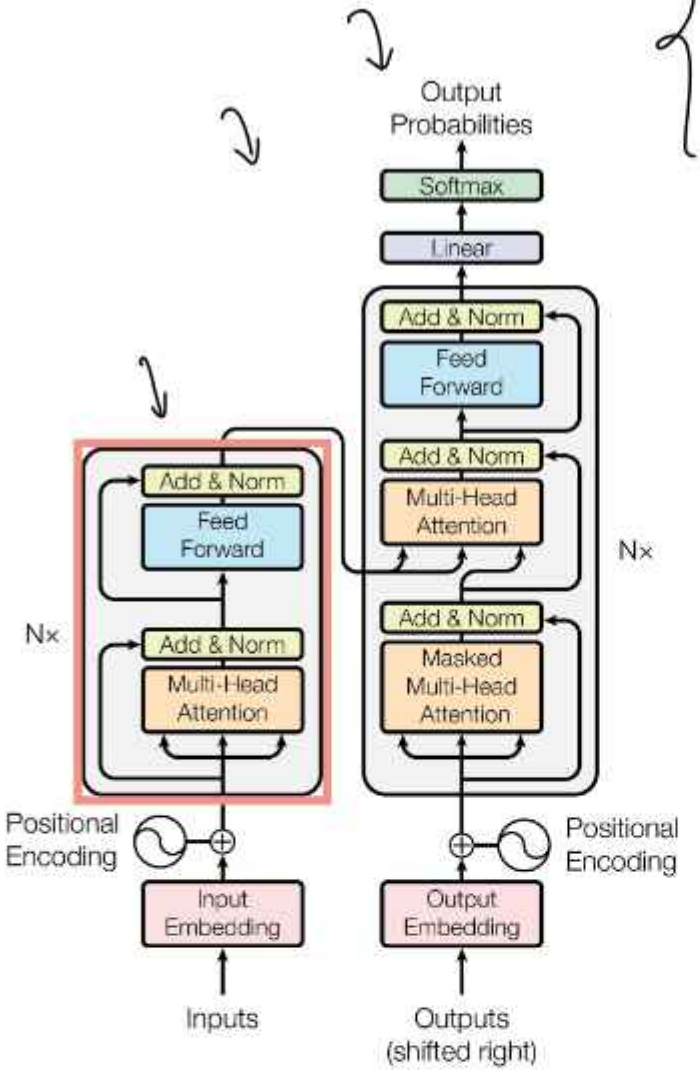
03 September 2024 00:39

Clarh

S no	English Sentence	Hindi Sentence
1	How are you	Aap kaise ho
2	Thank you	Dhanyawad
3	You are welcome	Apka swagat hai

Query Sentence

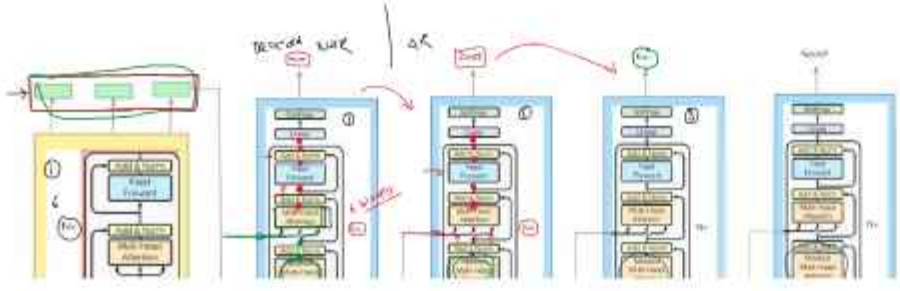
We are friends → ?

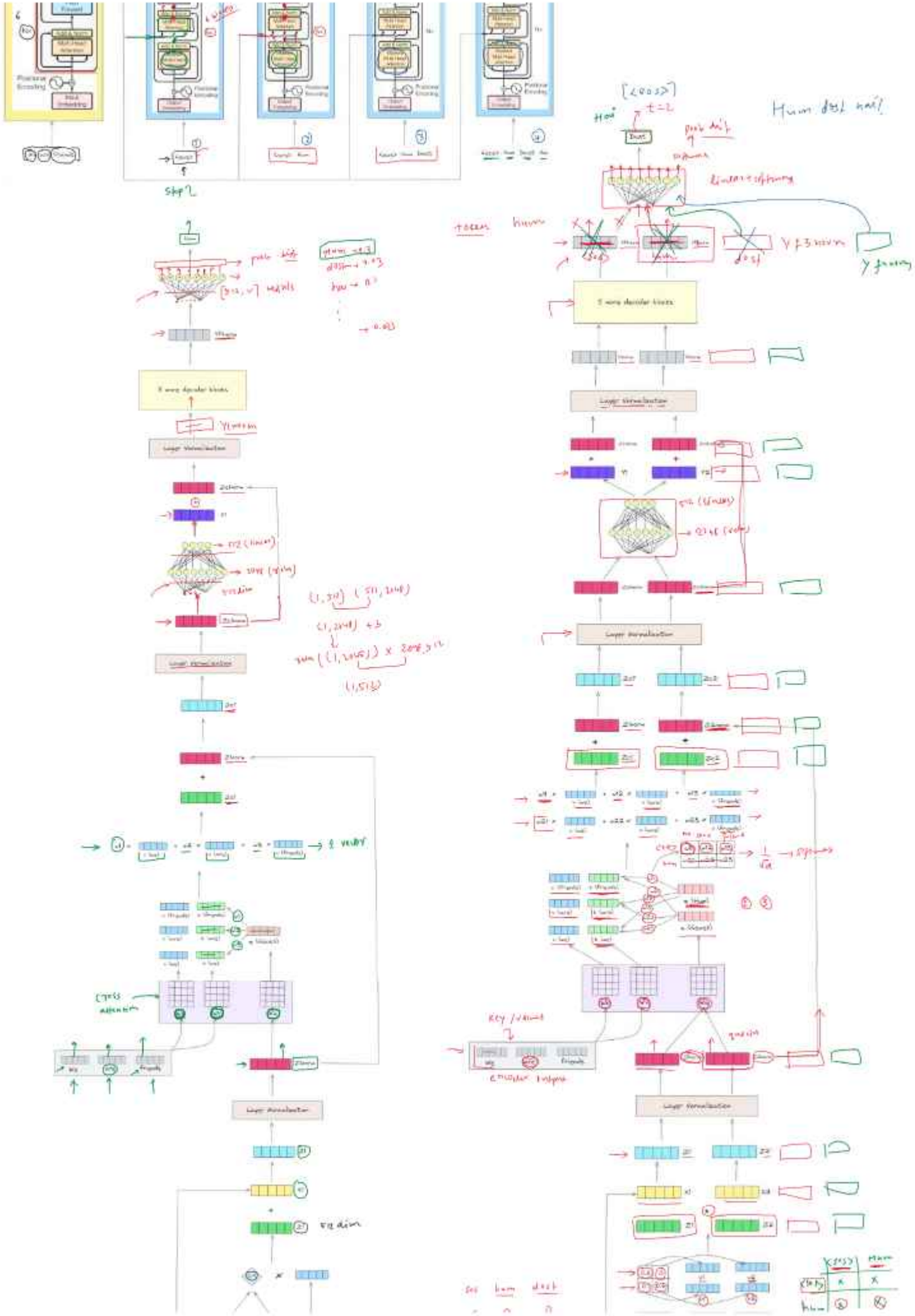


Transformer during Inference

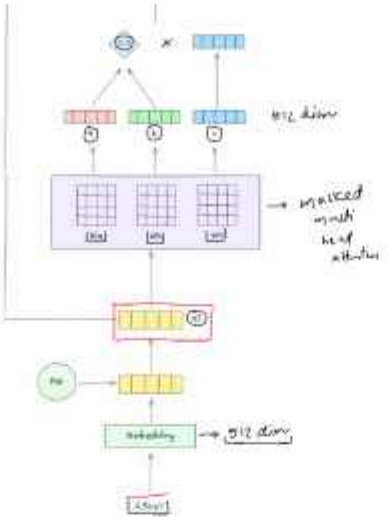
Training | Inference
Encoder same same

R









	Self	Enc-Dec	Feed Forward
Enc	X	O	O
Enc-Dec	X	X	O
Dec	X	X	X

